



ABAP RESTful Application Programming Model

Generated on: 2026-07-06 18:28:17 GMT+0000

ABAP Cloud | ABAP Cross-Product

Public

Original content: https://help.sap.com/docs/ABAP_Cloud/f055b8bf582d4f34b91da667bc1fcce6?locale=en-US&state=PRODUCTION&version=sap_cross_product_abap

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Getting Started with RAP

Getting Started with RAP provides you with the fundamental basics of development with the ABAP RESTful Application Programming Model.

For demonstration and learning purposes, we provide the **ABAP Flight Reference Scenario** that simulates an application used by a travel agency for booking flights. The first thing to do is therefore to import the ABAP Flight Reference Scenario in your ADT to get sample data: [Downloading the ABAP Flight Reference Scenario](#). You can find more details about the data model of the reference scenario in [ABAP Flight Reference Scenario](#).

The getting started guide helps you to create a complete application based on the existing data model from the ABAP Flight Reference Scenario with the most basic features: [Developing an OData Service for Simple List Reporting](#). Alternatively, you can use the Generate ABAP Repository Objects Wizard to create all RAP-service-related development objects on the basis of a database table to get started: [Generating a RAP Business Service with the Generate ABAP Repository Objects Wizards](#).

Prerequisites

Development Environment

- You have installed ABAP Development Tools (ADT) in Eclipse.

SAP recommends using the latest version of the client installation. The ADT in Eclipse download is available on the update site <https://tools.hana.ondemand.com/>.

- You have created an ABAP project to establish a connection between your ABAP back-end system and the Eclipse-based IDE.

Application Server ABAP

You are working on Application Server ABAP 7.55 SP00 or higher.

SAP Gateway

SAP Gateway is properly configured in your ABAP system for activating and testing the resulting OData service.

For more information, see [SAP Gateway Foundation](#).

Authorizations

To reproduce the steps described in this guide, the user on Application Server ABAP 7.55 SP00 with the following roles assigned is required:

- [SAP_BC_DWB_ABAPDEVELOPER](#) (ABAP Developer)
- [SAP_BC_DWB_WBDISPLAY](#) (Display User)

i Note

For behavior definitions and service bindings, you must add authorizations manually. See SAP Note [2827843](#) .

For SAP Gateway service development and consumption, roles of the following templates are needed:

- /IWFND/RT_DEVELOPER (SAP Gateway service development)
- /IWFND/RT_ADMIN (for publishing services)
- /IWFND/RT_GW_USER (for consuming services)

For more information, see [User, Developer and Administrator Roles](#)

Knowledge

Basic knowledge of

- ABAP Core Data Services (CDS)
- ABAP Objects.
- [ABAP Development](#)
- [ABAP Data Models](#)

Naming Conventions for Development Objects

Naming conventions facilitate the development. An addition to the name of development objects conveys standardized meaning and generates consistency in your development.

General Rules

→ Remember

The general guideline for development objects is the following: [/<namespace>/]
[<prefix>]_<object_name>_[<suffix>].

- Use your own namespace that is reserved for your organization.

i Note

Consider that the namespace /DMO/ is reserved for demo purposes. Do not use this namespace in your productive development.

- A prefix is used for cases when there are generically different types of one development object. Then, this prefix states the semantic difference that cannot be conveyed through the object type.

For example, a service binding can expose an OData service for UI purposes and as a Web API. That is why, for service bindings we introduce the prefixes UI_ and API_ to differentiate the semantics of service bindings.

- An object name <object_name> refers to the name of the repository object as it is defined in your IDE. It can be a semantic name based on business semantics that indicates what the repository object is about.
- A suffix is used for additional differentiation between different types of development objects. It helps to recognize more subtle or secondary differences in development objects.

For example, a UI service can be bound against the OData protocol OData, version 2 and OData, version 4. This difference can also be manifested by suffixing the name with _02 or _04.

i Note

In the ABAP Flight Reference Scenario we use another suffixed character (`_R`, `_M`, `_D`, `_U`, `_C`). This character identifies the development object to belong to one specific development guide (read-only, managed, draft, unmanaged, service consumption).

The following list provides an overview of the prefixing and suffixing guidelines on naming specific development objects.

ABAP Dictionary Objects

Database Tables

Use a prefix for database tables in scenarios, in which multiple technical representations of the same semantic data is necessary, for example in draft scenarios.

Use the prefix

- `A_` for the persistent database table, the table that contains the active data.
- `D_` for the draft database table.

Example: `/DMO/A_TRAVEL_D`

CDS Objects

CDS Entity

Use the prefix

- `I_` for an interface view.
- `R_` for the base CDS view entity.
- `C_` for a projection view. The character `C` represents the consumption layer. If there are multiple projections of one CDS entity, the object name should semantically represent the projection role.
- `E_` for an extension include view.

Use the suffix `TP` to indicate that a CDS view is relevant for transactional processing.

Example: `/DMO/R_AGENCYTP`, `/DMO/C_Travel_Processor_M`

Behavior Definition

A behavior definition has always the same name as the root entity of the business object.

Example: `/DMO/R_AGENCYTP`, `/DMO/C_Travel_M`

Metadata Extension

A metadata extension has the same name as the CDS entity it relates to. If you use more than one metadata extension for one CDS entity, you can add a numbered suffix.

Example: `/DMO/C_TRAVEL_U`, `/DMO/C_BOOKING_U_M2`

Business Services

Service Definition

Since a service definition- as a part of a business service - does not have different types or different specifications, there is (in general) no need for a prefix or suffix to differentiate meaning.

Example: **/DMO/TRAVEL_U**

However, in use cases where no reuse of the same service definition is planned for UI and API services, the prefix may follow the rules of the service binding.

Example: **/DMO/UI_TRAVEL_U**

Service Binding

Use the prefix

- **UI_** if the service is exposed as a UI service.
- **API_** if the service is exposed as Web API.

Use the suffix

- **_02** if the service is bound to OData protocol version 2.
- **_04** if the service is bound to OData protocol version 4.

Example: **/DMO/UI_TRAVEL_U_02**

Source Code Objects

Behavior Pool

Use the prefix

- **BP_** for an ABAP class that implements the behavior of a business object.

Example: **/DMO/BP_TRAVEL_U**

Handler and Saver Classes

Use the prefix

- **LHC_** for a local handler class.
- **LSC_** for a local saver class.

Depending on the modularization of your behavior implementation, you can provide the semantics of the coding in the name of the classes.

Example: **LHC_TRAVEL_CREATE**

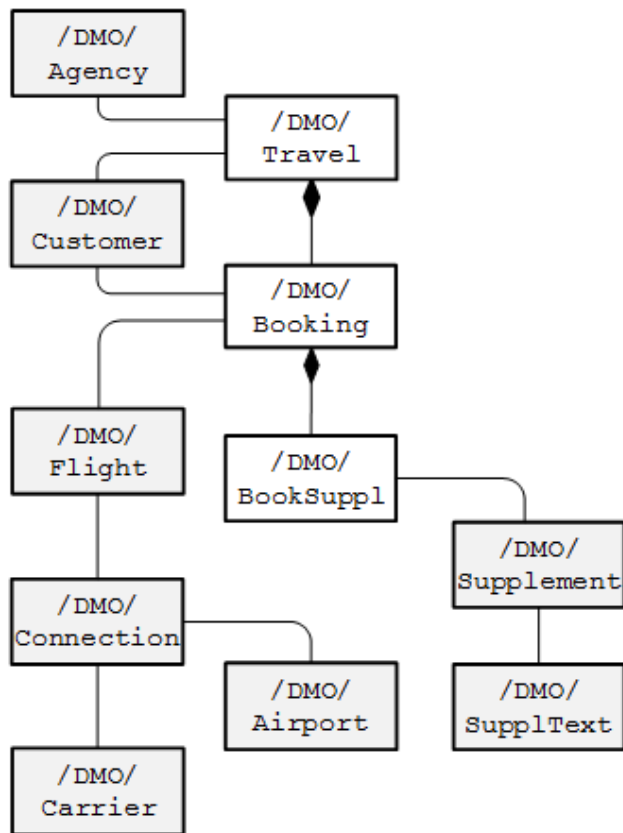
Example: **LHC_BOOKING_CUD**

ABAP Flight Reference Scenario

The ABAP Flight Reference Scenario helps you to get started with development in the context of the ABAP RESTful Application Programming Model. It contains demo content that you can play around with and use to build your own sample applications.

The reference scenario is based on multiple database tables which you can use to build your application. These tables are also used in the development guides in the [Developing RAP Business Objects](#) section.

Hover over each element for a description of the related database table. Click the element to view the database table fields.



Please note that image maps are not interactive in PDF outputs.

Downloading the ABAP Flight Reference Scenario from GitHub

You can download the complete ABAP Flight Reference Scenario for the ABAP RESTful Application Programming Model from GitHub. <https://github.com/SAP-samples/abap-platform-refscen-flight> ➡

The steps to import all relevant development objects are described in the [README.md](#) files of the respective branches..

→ Remember

The namespace `/DMO/` is reserved for the demo content. Apart from the downloaded ABAP Flight Scenario, do not use the namespace `/DMO/` and do not create any development objects in the downloaded packages. You can access the development objects in `/DMO/` from your own namespace.

Downloading the ABAP Flight Reference Scenario

The ABAP Flight Scenario contains demo content that you can import into your development environment.

The ABAP Flight Reference Scenario helps you to get started with development in the context of the ABAP RESTful Application Programming Model. It contains demo content that you can play around with and use to build your own sample applications.

Sample Data

First of all, the reference scenario contains data. You can use database tables that are filled with travel data including master data items, such as customer, flights, airports, or booking supplements. The structure of the complete data model allows you to build simple but also more complex services. In this way, it is easy to follow the steps in the development guides while building your own application based on the same database tables as in the given examples.

For an overview of the available database tables, see [ABAP Flight Reference Scenario](#). They are available in the package /DMO/FLIGHT_LEGACY. This package also includes a data generator with which you can fill the database tables.

Sample Services

The development guides for the ABAP RESTful Application Programming Model are based on the sample data from the ABAP Flight Reference Scenario. That means that you can compare the documentation with the productive code that was used to build the documentation scenario. In addition, the ABAP Flight Reference Scenario also includes a demo package with the development objects that are created during the course of the development guides. That means, the whole demo scenario can be downloaded and tested. You obtain full demo services with code built by following conventions and best practices and you can use and reuse the delivered objects for your development.

The following demo scenarios are available for you:

- [Developing Read-Only List Reporting Apps](#) in the package /DMO/FLIGHT_READONLY
- [Developing Read-Only RAP Analytical Tables](#) in the package /DMO/FLIGHT_ANA
- [Developing Unmanaged Transactional Apps](#) in the package /DMO/FLIGHT_UNMANAGED
- [Developing Managed Transactional Apps](#) in the package /DMO/FLIGHT_MANAGED
- [Developing Transactional Apps with Exclusive Draft Capabilities](#) in the package /DMO/FLIGHT_DRAFT
- [Developing Transactional Apps with Collaborative Draft Capabilities](#) in the package /DMO/FLIGHT_COLLDraft
- [Developing Cross-BO Scenarios with Draft Scope](#) in the package /DMO/FLIGHT_XB0
- [Developing Transactional Apps with Multi-Inline-Edit Capabilities](#) in the package /DMO/FLIGHT_REUSE_SUPPLEMENT
- [Developing Treeviews with Read-Only Capabilities](#) in the package /DMO/FLIGHT_HIERARCHY_R0
- [Developing Editable Treeviews with Draft Capabilities](#) in the package /DMO/FLIGHT_HIERARCHY_DRAFT
- [Adding RAP Recommendations](#) in the package /DMO/FLIGHT_REC
- Developing RAP Extensions with:
 - [Data Model Extensibility](#) in the package /DMO/FLIGHT_REUSE_AGENCY_SLOGN
 - [Behavior Extensibility](#) in the package /DMO/FLIGHT_REUSE_AGENCY_CNTRY
 - [Node Extensibility](#) in the package /DMO/FLIGHT_REUSE_AGENCY_REV

Legacy Coding

The reference scenario also includes legacy coding. This legacy coding is based on function modules and exemplifies legacy applications that you can include in your new ABAP code. Above all, the legacy coding is relevant for the development guide, that explains how to build a new service on the basis of an existing application. It illustrates how you build an application with the unmanaged implementation type. The legacy coding that is used in this scenario is available in the package /DMO/FLIGHT_LEGACY.

Downloading the ABAP Flight Reference Scenario from GitHub

You can download the complete ABAP Flight Reference Scenario for the ABAP RESTful Application Programming Model from GitHub. <https://github.com/SAP-samples/abap-platform-refscen-flight> ➔

The steps to import all relevant development objects are described in the [README.md](#) files of the respective branches..

→ Remember

The namespace `/DM0/` is reserved for the demo content. Apart from the downloaded ABAP Flight Scenario, do not use the namespace `/DM0/` and do not create any development objects in the downloaded packages. You can access the development objects in `/DM0/` from your own namespace.

Developing an OData Service for Simple List Reporting

The following guide describes the basic development tasks to create a simple list reporting app based on a query.

Introduction

An OData service makes it possible to create and consume queryable and interoperable RESTful APIs. An SAP Fiori Elements application consumes OData services like this, but it also possible for other Web clients to make use of an OData service that is created with the ABAP RESTful Application Programming Model.

This programming model provides a framework that facilitates your application development. All included technologies, such as Core Data (CDS) or business services, are usable and accessible with ABAP Development Tools (ADT), providing easy access to the necessary infrastructure.

The following guide starts from a data model assuming that database tables already exist. It uses the ABAP Flight Reference Scenario (in short Flight Scenario), which provides example data comprising travel information with flight data. For a detailed description of the database tables that are used in this scenario, refer to [ABAP Flight Reference Scenario](#)

You are guided step-by-step through the new application model in three consecutive building blocks:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

SERVICE
CONSUMPTION



BUSINESS
SERVICES
PROVISIONING



DATA MODELING

Please note that image maps are not interactive in PDF outputs.

You start by implementing a CDS view as a new data model layer using a data source that is already provided. You also use basic CDS annotations to manifest semantics for the data model. The next step is to create an OData service by defining and binding a service based on the corresponding CDS view. As soon as the OData service is published in the local system repository, it is ready to be consumed using an OData client, such as a SAP Fiori app. Finally, you learn how to use UI annotations as a UI technology independent semantic description of the user interface layout.

The result of this Getting Started guide is a consumable OData Service, which can be easily used to set up a Fiori Elements travel booking app, from which you can derive information about flight connections. Navigation properties are added to this application to receive more information about bookings, customers, and agencies in the other scenarios in the [Developing RAP Business Objects](#) section. These other development guides also cover extended read-only and transactional features, whereas the Getting Started guide only deals with the most basic read-only features for setting up an OData Service. The scenarios in the Develop section assume that you understood the steps that are described in the following guide.

i Note

Via ABAPGit you can import the service including the related development objects into your development environment for comparison and reuse. You find the service in the package /DM0/FLIGHT_READONLY. The suffix for development objects in this development guide is _R. Be aware that the development objects might contain more than explained in the Getting Started guide. This is because the Getting Started scenario is enhanced in the first development guide [Developing Read-Only List Reporting Apps](#) which uses this same demo objects.

For information about downloading the ABAP Flight Reference Scenario, see [Downloading the ABAP Flight Reference Scenario](#).

The following sections serve as an introductory guide for the development of an OData service based on the ABAP RESTful Application Programming Model. It forms the basic building block for more elaborate scenarios with extended read-only features or transactional processing.

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).

Objectives

By the end of this Getting Started section, you will be able to:

- Create a data definition and define a CDS view
- Implement an ABAP CDS view based on an existing database table
- Define an OData service and expose a CDS view for this service
- Bind the OData service against a protocol and publish it locally
- Use semantics annotations in CDS
- Understand some basic UI annotations in CDS

Defining the Data Model with CDS

The data model for an OData service must be defined in CDS.

This introductory programming guide uses example data from the Flight Reference Scenario. The Getting Started scenario uses the database table `/dmo/connection`. It provides information about airline and connection numbers, flight times, and data related to planes.

In the CDS layer we use and manipulate data that is persisted in the database. To make data available in the ABAP application server, CDS views use SQL queries to project persisted data to the ABAP layer. This is necessary to create an OData service to make the data ready to be consumed.

To define a data model based on the ABAP CDS view concept, you first need to create a data definition as the relevant ABAP Repository object using a wizard in [ABAP Development Tools](#).

Task 1: [Creating a CDS View Entity for the Connection Entity](#)

In the second step, you implement an elementary CDS view from scratch by defining a simple query for flights based on a single data source from the ABAP Flight Reference Scenario.

Task 2: [Implementing the CDS View as a Data Model](#)

In the final task of this section, you have the option of using the test environment to verify the output (a results set) of the CDS view you have just implemented.

Task 3: [Verifying the Results Set in the Data Preview Tool](#)

Creating a CDS View Entity for the Connection Entity

Use the data definition wizard to create the relevant development object for a CDS view.

Context

For our simple read-only scenario, we want to define data that is exposed by an OData service to make it available for an OData client. For this purpose, you create a development object to define an ABAP CDS entity (for example, a CDS view). The data definition provides you with the appropriate development object for the CDS view, which is included in ABAP development tools and directly accesses the standard ABAP functions.

To create a data definition for the CDS view, use the wizard for data definition.

Results

In the selected package, the ABAP back-end system creates an inactive version of a data definition and stores it in the ABAP Repository. As a result, the data definition editor is opened. The generated source code already provides you with the necessary view annotations and adds a placeholder for the name of the data source for query definition. The name for the actual CDS view is predefined on the basis of the name for the data definition, but can be changed in the data definition editor.

```

1 @AccessControl.authorizationCheck: #NOT_REQUIRED
2 @EndUserText.label: 'Connection View - CDS Data Model'
3
4 @UI.headerInfo: { typeName: 'Connection',
5                  typeNamePlural: 'Connections' }
6
7 @Search.searchable: true
8
9 define view entity /DMO/I_Connection_R as select from /dmo/connection as Connection
10

```

The generated template code in the data definition editor

Next Steps

Now that you have created a data definition, you can implement the CDS view as a data model for your OData service.

Implementing the CDS View as a Data Model

Use a predefined database table as the data source for a CDS view.

Prerequisites

- You have created the data definition artifact in ABAP Development Tools.
- The database table `/dmo/connection` is available for you.

Context

In this step, you implement an interface view as a new data model using a predefined data source.

Procedure

1. If you have not yet already done so, open the new data definition in the editor.
2. Specify the name of the CDS view: `/DMO/I_Connection_R`. The data definition editor already provides a suggestion for the name using the name that you specified for the data definition in the creation wizard. However, these names do not have to be the same. You can overwrite it in the `define` statement.

3. In the `SELECT` statement, enter the predefined database table `/dmo/connection` as a data source and define an optional alias name for the data source.

An alias is useful especially when you use multiple data sources or whenever the name of the data source is not descriptive or too long.

```
... select from /dmo/connection as Connection{
```

4. Add the fields of `/dmo/connection` to the `SELECT` list and assign alias names to each item field as follows:

```

{
  Connection.carrier_id      as AirlineID,
  Connection.connection_id  as ConnectionID,
  Connection.airport_from_id as DepartureAirport,
  Connection.airport_to_id  as DestinationAirport,
  Connection.departure_time as DepartureTime,
  Connection.arrival_time   as ArrivalTime,
  Connection.distance       as Distance,

```

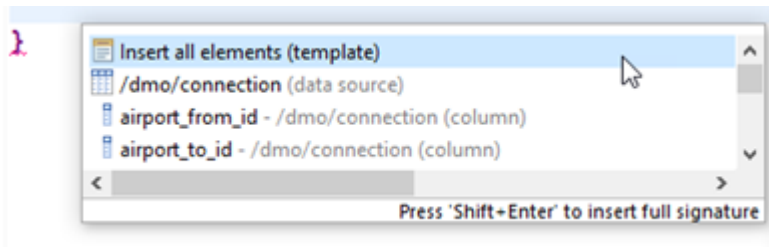
```

    Connection.distance_unit    as DistanceUnit
}

```

→ Tip

Whenever you insert table fields or view elements in the SELECT list, you can make use of the content assist function in the data definition editor (**CTRL** + **SPACE**).



Inserting fields using semantic auto-completion

5. To document the key semantics of the new data model, define the `AirlineID` and `ConnectionID` elements as **KEY** elements in the current CDS view:

```

key connection.carrier_id    as AirlineID,
key connection.connection_id as ConnectionID,

```

6. Change authorization check to "NOT_REQUIRED".

7. Click the activation button  or use the shortcut **Ctrl** + **F3** to activate the data definition.

To check the syntax before activation, click  or use the shortcut **Ctrl** + **F2** .

Results

The resulting source code for the CDS view is the following:

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Read-Only E2E: Data Model Connection'
define view entity /DMO/I_Connection_R
as select from /dmo/connection as Connection
{
  key Connection.carrier_id    as AirlineID,
  key Connection.connection_id as ConnectionID,
  Connection.airport_from_id  as DepartureAirport,
  Connection.airport_to_id    as DestinationAirport,
  Connection.departure_time    as DepartureTime,
  Connection.arrival_time     as ArrivalTime,
  Connection.distance         as Distance,
  Connection.distance_unit    as DistanceUnit
}

```

The source code above is used to define a quite simple CDS view named `/DMO/I_Connection_R`. This view is implemented using a query that performs a **SELECT** statement, where the database table `/dmo/connection` is used as the data source. The select list includes a set of fields that are relevant for the scenario. The **KEY** elements in the selection list are used to define the key field semantics of the CDS view.

When the data definition source is activated, the entity of the CDS view `/DMO/I_Connection_R` is created in ABAP Dictionary.

Next Steps

Mark the element `Distance` as semantically related to the element `DistanceUnit`.

Relating Semantically Dependent Elements

Use the `@Semantics` annotation to relate the quantity element to its unit of measure element.

Context

The CDS view `/DMO/I_Connection_R` that you created contains elements that are heavily dependent on each other **semantically**, namely `Distance` and `DistanceUnit`. In CDS, you can use semantic annotations to standardize semantics that have an impact on the consumer side for these elements. In general, elements that need to be marked as having semantic content to guarantee that they are handled correctly are elements that contain the following:

- Amounts of money

These elements need a reference to the currency related to this element.

- Amounts of measures

These elements need a reference to the unit of measure related to this element.

If you create annotations that define a link to the unit for the amounts, the amounts and their units are always handled as being dependent on each other in the OData service. On UIs in particular, amounts are displayed with the correct decimals with regard to their unit.

In the CDS view `/DMO/I_Connection_R`, you therefore need to proceed as described in the following to always display the distance together with the distance unit.

Procedure

1. Open the CDS view `/DMO/I_Connection_R`.
2. Define the relationship between amount and unit of measure with the annotation `@Semantics.quantity.unitOfMeasure: '<ElementRef>'` on `Distance` and reference the element `DistanceUnit`.

```
@Semantics.quantity.unitOfMeasure: 'DistanceUnit'
Connection.distance          as Distance,

Connection.distance_unit    as DistanceUnit
```

3. Activate the CDS view.

Results

If you expose the CDS view to an OData service, the elements are always handled as being semantically related to each other. This means that they are given the OData annotation `sap:unit` and `sap:semantics` in the OData metadata document. On UIs in particular, the elements are always displayed as being attached to each other.

Related Information

[CDS Annotations](#)

Verifying the Results Set in the Data Preview Tool

Use the data preview tool to check the elements in the CDS view.

Prerequisites

The data definition has correct syntax and has been activated.

Context

You have created a data definition and implemented a CDS view with data from the database table [/dmo/connection](#). Now you have the option of launching the test environment (in the data preview tool), which enables you to verify that the persisted data from the database is now displayed in the CDS view.

Procedure

In the data definition editor, position the cursor somewhere in the CDS source code. Open the context menu and choose **Open With** **Data Preview** or use the shortcut **F8**.

Results

The CDS view does not require any parameters, which means the data preview displays the results set of the data selection query directly.

Car...	Con...	Depart...	Desti...	Departu...	ArrialT...	Dis...	Dis...
SQ	0001	SFO	SIN	011500	115000	13.523	km
SQ	0002	SIN	SFO	063000	091500	13.523	km
SQ	0011	NRT	SIN	145500	205000	5.363	km
SQ	0012	SIN	NRT	095300	175400	5.363	km
UA	0058	SFO	FRA	134500	095500	9.608	km
UA	0059	FRA	SFO	135500	163000	9.608	km
UA	1537	EWR	MIA	215600	004700	1.752	km
AA	0322	MIA	EWR	201700	231900	1.752	km
AA	0017	MIA	HAV	071900	080300	520	km
AA	2678	HAV	MIA	061500	103000	520	km
AA	0015	JFK	SFO	071300	100400	4.156	km

Results sets in the data preview tool

Note

You can sort the entries by element by clicking the column header.

Creating an OData Service

Business service artifacts enable the publishing of an OData service using ABAP Development Tools.

In the previous step, you defined a data model based on the persisted data source [/dmo/connection](#) in the data definition [/DMO/I_Connection_R](#). You can now use this data model and expose it for an OData service. The OData service makes it possible for UI technologies to query data and consume it. The following steps are necessary to include the CDS view in an OData service.

To define a service, you first need to create a service definition as the relevant ABAP Repository object using a wizard.

Task 1: [Creating a Service Definition for the Read-Only Service](#)

The next step is to define the scope of the OData service by exposing the relevant CDS views (including their metadata and their behavior).

Task 2: [Exposing a CDS View for an OData Service](#)

To define the type and category of the OData service, you need to create a service binding as the relevant ABAP Repository object. There is also a wizard available for this.

Task 3: [Creating a Service Binding](#)

In the next step, you use the form-based editor of the service binding to publish the service locally.

Task 4: [Publishing the OData Service Locally](#)

You have the option of checking the resulting OData service by viewing its metadata. The service binding offers a simple solution for this.

Task 5: [Verifying the OData Metadata](#)

You can also take a look at how the UI of a Fiori Elements of the OData service looks like with the preview tool of the service binding.

Task 6: [Previewing the Resulting UI Service](#)

Creating a Service Definition for the Read-Only Service

Use the service definition wizard to create the relevant development object that defines the scope of the OData service

Context

The service definition is a projection of the models and related behavior that you want to expose. In a service definition, you define the OData service to determine which CDS entities are part of the service. This service is then exposed either as a UI service or a Web API by a service binding artifact. A service definition can be integrated in various protocols without any reimplementation.

To create a Service Definition use the wizard to create the relevant development object that defines the scope of the OData service.

Results

The ABAP back-end system creates an inactive version of a service definition and stores it in the ABAP Repository.

In the [Project Explorer](#), the new service definition is added to the [Business Services](#) folder of the corresponding package node. As a result, the service definition editor is opened.

Next Steps

Now that you have created a service definition, you can choose one or more CDS entities to be exposed in the service.

Exposing a CDS View for an OData Service

Prerequisites

You have created the service definition artifact in ABAP Development Tools.

Context

In the service definition editor, you determine the CDS entities that you want to expose in an OData service.

Procedure

1. If you have not yet already done so, open the new service definition in the editor.

The name of the service is already specified in accordance with the name you gave in the service definition wizard. It cannot be changed to a different name.

2. Specify the name of each CDS entity that you want to expose for the service. For the getting started read-only scenario, there is only one CDS view to be exposed: `/DMO/I_Connection_R`

3. Optionally, you can assign an alias for the CDS view.

An alias is useful, especially when you use multiple CDS views or whenever the name of the CDS view is not descriptive or too long.

4. Click the activation button  or use the shortcut `Ctrl + F3` to activate the service definition.

To check the syntax before activation, click  or use the shortcut `Ctrl + F2`.

Results

The resulting source code for the service definition is as follows:

```
@EndUserText.label: 'Read-Only E2E: SD for Managing Flights'
define service /DMO/FLIGHT_R {
    expose /DMO/I_Connection_R as Connection;
}
```

The source code above is used to define a service definition named `/DMO/FLIGHT_R`. It exposes the CDS view `/DMO/I_Connection_R` to be included in the service.

Next Steps

Now that the service exists, you can determine the binding type and category for the service using a service binding.

Creating a Service Binding

Use the service binding wizard to create the relevant development object to bind the service to a protocol and, if necessary, to an OData client.

Prerequisites

You have defined a service and exposed CDS entities that are included in the service.

Context

A service binding implements the protocol that is used for the OData service. It uses a service definition that projects the data models and their related behaviors to the service.

To create a Service Binding, use the wizard to create the relevant development object to bind the service to a protocol and, if necessary, to an OData client.

Use the [Binding Type](#) ODATA V2 - UI and the [Service Definition](#) /DMO/FLIGHT_R.

Results

The ABAP back end creates a service binding and stores it in the ABAP Repository.

In the [Project Explorer](#), the new service binding is added to the [Business Services](#) folder of the corresponding package node. As a result, the service binding form editor is opened and you can verify the information you have entered.

Next Steps

Activate the service binding to make it ready for consumption.

Related Information

[Creating Service Binding](#)

Publishing the OData Service Locally

To make the service ready for consumption, use the activation button in the service binding form editor.

Prerequisites

You have created the service binding and specified the binding type and category.

Context

To make the service available and consumable by an OData client you have to activate the service.

Procedure

1. If you have not already done so, open the new service binding in the form editor.

The binding type and category are already defined and cannot be changed once the service binding is created. You can verify the type and category in the general information section in the form editor. As soon as you have specified the binding for the service, it is ready for publishing. The service is then available for consumption.

2. Choose the [Publish](#) button in the form editor. For more information about the editor, see [Using Service Binding Editor for OData V2 Service](#).

Results

The OData service [/DMO/UI_FLIGHT_R_V2](#) is published locally, which means that it is activated in SAP Gateway. The service is bound to the protocol OData V2 for the category UI. This means it can now be consumed by a SAPUI5 application.

The binding type and service information is displayed in the service binding form editor

On the left side of the form editor, the service list with the version and the service definition is filled. The right side of the form editor shows the service details. It provides a URL to view the metadata of the service and lists the entity sets that are exposed for the service. The service contains the entities that you have exposed in the service definition. The service binding editor shows the names that you assigned as alias.

Related Information

[Using Service Binding Editor for OData V2 Service](#)

Verifying the OData Metadata

Use the URI in the service binding form editor to check the metadata document of the published OData service.

Prerequisites

You have published an OData service using a service binding.

Context

In the previous steps we defined an OData service and published it. It is now ready to be consumed by an HTTP protocol. To verify the data that the OData service exposes, the service offers a metadata document in which all relevant service elements are listed.

Procedure

1. If you have not already done so, open the service binding for the relevant service.
2. To open the service document of the OData service, choose the link to the service URL ([/sap/opu/odata/sap/DMO/UI_FLIGHT_R_V2](#)) that is provided in the form editor for the relevant line in the service details section.

A browser opens that displays the service document.

3. Add `/$metadata` to the URI in the address bar to view the metadata of the OData service.

`.../sap/opu/odata/DMO/UI_FLIGHT_R_V2/$metadata`

The metadata document displays the relevant information that the OData service provides for an OData client in a CSDL (Common Schema Definition Language).

i Note

As labels are language dependent, they are only displayed if the language of the browser and the maintained data elements are in the same language, or if a fallback language matches the browser configurations.

```

<?xml version="1.0" encoding="UTF-8"?>
- <edmx:Edmx xmlns:sap="http://www.sap.com/Protocols/SAPData" xmlns:m="http://schemas.microsoft.com/ado/2007/08/dataservices/metadata"
  xmlns:edmx="http://schemas.microsoft.com/ado/2007/06/edmx" Version="1.0">
+ <edmx:Reference xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Uri="https://HOST/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies
  (TechnicalName='%2FIWBEP%2FVOC_COMMON',Version='0001',SAP__Origin='')/$value">
+ <edmx:Reference xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Uri="https://HOST/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies
  (TechnicalName='%2FIWBEP%2FVOC_CAPABILITIES',Version='0001',SAP__Origin='')/$value">
+ <edmx:Reference xmlns:edmx="http://docs.oasis-open.org/odata/ns/edmx" Uri="https://HOST/sap/opu/odata/IWFND/CATALOGSERVICE;v=2/Vocabularies
  (TechnicalName='%2FIWBEP%2FVOC_COMMUNICATION',Version='0001',SAP__Origin='')/$value">
- <edmx:DataService m:DataServiceVersion="2.0">
  - <Schema xml:lang="en" Namespace="cds_xdmoxflight_r_sd" xmlns="http://schemas.microsoft.com/ado/2008/09/edm" sap:schema-version="1">
    - <EntityType sap:content-version="1" sap:label="Read-Only E2E: Data Model Connection" (Name)="ConnectionsType">
      - <Key>
        <PropertyRef Name="CarrierID"/>
        <PropertyRef Name="ConnectionID"/>
      </Key>
      <Property sap:label="Airline ID" (Name)="CarrierID" sap:quickinfo="Flight Reference Scenario: Carrier ID" sap:display-format="UpperCase" MaxLength="3"
        Nullable="false" Type="Edm.String"/>
      <Property sap:label="Flight Number" Name="ConnectionID" sap:quickinfo="Flight Reference Scenario: Connection ID" sap:display-format="NonNegative"
        MaxLength="4" Nullable="false" Type="Edm.String"/>
      <Property sap:label="Departure Airport" Name="DepartureAirport" sap:quickinfo="Flight Reference Scenario: From Airport" sap:display-format="UpperCase"
        MaxLength="3" Type="Edm.String"/>
      <Property sap:label="Destination Airport" Name="DestinationAirport" sap:quickinfo="Flight Reference Scenario: To Airport" sap:display-format="UpperCase"
        MaxLength="3" Type="Edm.String"/>
      <Property sap:label="Departure" Name="DepartureTime" sap:quickinfo="Flight Reference Scenario: Departure Time" Type="Edm.Time" Precision="0"/>
      <Property sap:label="Arrival" Name="ArrivalTime" sap:quickinfo="Flight Reference Scenario: Arrival Time" Type="Edm.Time" Precision="0"/>
      <Property sap:label="Flight Distance" Name="Distance" sap:quickinfo="Flight Reference Scenario: Flight Distance" Type="Edm.Int32"
        sap:unit="DistanceUnit"/>
      <Property sap:label="Internal UoM" Name="DistanceUnit" sap:quickinfo="Unit of Measurement" MaxLength="3" Type="Edm.String" sap:semantics="unit-of-
        measure"/>
    </EntityType>
+ <EntityContainer Name="cds_xdmoxflight_r_sd_Entities" sap:supported-formats="atom json xlsx" m:IsDefaultEntityContainer="true">
  <atom:link xmlns:atom="http://www.w3.org/2005/Atom" href="https://HOST/sap/opu/odata/DMO/FLIGHT_R_SB_UI_V2/$metadata" rel="self"/>
  <atom:link xmlns:atom="http://www.w3.org/2005/Atom" href="https://HOST/sap/opu/odata/DMO/FLIGHT_R_SB_UI_V2/$metadata" rel="latest-
    version"/>
  </EntityContainer>
  </Schema>
</edmx:DataService>
</edmx:Edmx>

```

OData metadata

i Note

Depending on your browser and the xml format you choose, the layout of the metadata might differ.

For the described scenario, the following OData annotations are relevant:

- o **EntityType**: Introduces a CDS entity that is exposed for the service.

sap:label: Provides a semantic description for the entity type. It retrieves the description that was entered in the wizard for the data definition as no other label is defined.

Name: Specifies the name of the OData entity. It uses the name of the CDS entity and attaches **Type**. If an alias is used in the service definition, it uses the alias.

- **Key**: Introduces the OData properties that are specified as keys for the OData entities. If the service is based on CDS entities, it uses the keys of the CDS entities.

Property: Introduces an OData property that is exposed in the service. If the service is based on a CDS entity, it uses the elements of the CDS view as properties.

sap:label: Provides a more informative description than just the name of the property. It retrieves the field label text of the data element if the CDS element is not labeled differently.

Name: Specifies the name of the OData property. The service uses the name of the CDS elements. It retrieves the alias if there is one.

sap:quickinfo: Provides a semantic description for the property. It retrieves the description of the data element that is used in the database table /dmo/connection if no other description is defined.

sap:unit: Specifies that the respective OData property describes an amount whose unit is provided with the referenced property. In this case, as we have defined it in CDS with a semantics annotation, the property DistanceUnit provides the unit for the Distance.

sap:semantics DistanceUnit only contains currency codes. This information is taken from the data element that is used for the database table /dmo/connection, which is stored in ABAP Dictionary.

i Note

The information that is taken from the data elements can be checked in the data definition. Click a CDS element in the data definition and press **F2**. A pop-up opens and you can navigate to all the underlying elements, displaying the

semantic information for the respective OData property.

Next Steps

To check the output of a **SAP Fiori UI** you can preview the app with the previewing functionality of the service binding.

Previewing the Resulting UI Service

Use the preview function in the service binding to check how the UI of a Fiori application looks like.

Prerequisites

You have published an OData service using a service binding.

Context

The published OData service is ready to be consumed by an HTTP protocol. You can set up a Fiori application based on this service. The service binding artifact offers a tool which you can use to preview a simple list reporting Fiori application.

Procedure

1. If you have not yet already done so, open the service binding for the relevant service.
2. To open the Fiori Elements app preview in the service information section, select the relevant entity set (**Connection**) and choose the **Preview** button as described in [Using Service Binding Editor for OData V2 Service](#).
3. Choose **Open Fiori Elements App Preview**.

Your internet browser opens a new tab.

4. You now have access to the system and the Fiori Elements app UI is displayed. The columns of the elements that you have in the CDS views appear. The app does not show any data yet.
5. To display data in the list report, first select the items that you want to display by clicking the configuration button  and choosing the elements from the column section.

You need to select at least one element, otherwise you get an error message when retrieving the data.

6. Choose **Go** to display the data of the items you selected.

Results

The Fiori Elements App preview opens in your browser. You see the connection data that you implemented in the CDS view. The following image displays the list report when selected all available fields.



Airline ID	Flight Nu...	Departure Airport	Destination Airport	Departure	Arrival	Flight Distance	Internal UoM	
AA	15	JFK	SFO	7:13:00 AM	10:04:00 AM	4,156	KM	KM >
AA	17	MIA	HAV	7:19:00 AM	8:03:00 AM	520	KM	KM >
AA	18	SFO	JFK	6:40:00 AM	3:06:00 PM	4,156	KM	KM >
AA	322	MIA	EWR	8:17:00 PM	11:19:00 PM	1,752	KM	KM >
AA	2678	HAV	MIA	6:15:00 AM	10:30:00 AM	520	KM	KM >
AZ	788	VCE	NRT	1:25:00 PM	10:13:00 AM	9,595	KM	KM >

Designing the User Interface for a Fiori Elements App

UI annotations can be used in CDS to configure the look of the user interface of a Fiori App.

To define annotations that concern the UI, we use CDS annotations. CDS offers the option of defining a universal setup for the presentation and order of items in the CDS layer. This is independent of the UI technology or application device, which benefits the reuse of one OData service for multiple applications. The application developer does not have to configure the setting for every application, but can reuse the settings that were defined in the back end.

i Note

You can use metadata extensions to separate the metadata specified by @UI or other UI related annotations from the actual data definition in the CDS view.

You are introduced to necessary and useful UI annotations that define the presentation of your data in a UI service.

Task: [Defining UI Annotations](#) .

The task addresses different components of the user interface separately.

In the section [List Items](#) CDS offers the option to define a universal setup for the presentation and order of the business data. You will learn how to order and label the columns of your list report.

The second section [List Report Header](#) deals with the items in the list report header.

The section [Object Page](#) describes the configuration of an object page and its items.

→ Tip

You can always check the influence of the UI annotations on the UI with the preview option in the service binding form editor.

Defining UI Annotations

The presentation and order of the CDS elements in a SAP Fiori Elements user interface is configured in CDS with annotations.

Context

You have created an OData service and published it locally. The UI can now be set up with UI annotations in the CDS layer to define a UI layout independent from the application or the user device. You can always check the influence of UI annotations by using the preview function in the service binding artifact.

List Items

Context

Using the following annotations, you specify which of the elements appear in the list report when starting the app. In addition to their order, you can also rename them if you want to display them with a name other than the name specified in the CDS entity. The columns that are shown in the UI are then predefined and you can retrieve data by choosing [GO](#) without determining the columns to be displayed.

Procedure

1. Open the CDS view for which you want to determine the list report. In our case: [/DMO/I_Connection_R](#).
2. For the headline of the list, use the annotation `@UI.headerInfo.typeNamePlural: 'name'`.

This annotation is an entity annotation because it concerns the whole entity rather than a specific element.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Read-Only E2E: Data Model Connection'
```

```
@UI.headerInfo.typeNamePlural: 'Connections'
```

```
define view entity /DMO/I_Connection_R
...
```

3. Specify a position for each element that you want to show in the list report with the annotation `@UI.lineItem: [ { position:decfloat } ]`.

i Note

The value's number does not represent an absolute measure and works as a relative value to the positions of the other elements instead. Hence, the elements are arranged in ascending order with regard to the annotation value.

```
...
define view entity /DMO/I_Connection_R
  as select from /dmo/connection as Connection
{
  @UI.lineItem: [ { position: 10 } ]
  key   Connection.carrier_id      as AirlineID,
  @UI.lineItem: [ { position: 20 } ]
  key   Connection.connection_id  as ConnectionID,
  @UI.lineItem: [ { position: 30 } ]
        Connection.airport_from_id as DepartureAirport,
  @UI.lineItem: [ { position: 40 } ]
        Connection.airport_to_id  as DestinationAirport,
  @UI.lineItem: [ { position: 50 } ]
        Connection.departure_time as DepartureTime,
  @UI.lineItem: [ { position: 60 } ]
        Connection.arrival_time   as ArrivalTime,
  @Semantics.quantity.unitOfMeasure: 'DistanceUnit'
        Connection.distance       as Distance,      /** secondary information, not to be disp
        Connection.distance_unit   as DistanceUnit   /** secondary information, not to be d
}
```

4. You can display the elements with a name other than the name specified in CDS by labeling them with the annotation `@UI.lineItem.label: label`. In particular, you can label element with names containing spaces. The label is

displayed in the column header of the list report.

```
...
define view entity /DMO/I_Connection_R
  as select from /dmo/connection as Connection

{
  @UI.lineItem: [ { position: 10, label: 'Airline'} ]
  key   Connection.carrier_id      as AirlineID,
  @UI.lineItem: [ { position: 20, label:'Connection Number' } ]
  key   Connection.connection_id  as ConnectionID,
  @UI.lineItem: [ { position: 30 , label: 'Departure Airport Code'} ]
  Connection.airport_from_id as DepartureAirport,
  @UI.lineItem: [ { position: 40 , label: 'Destination Airport Code'} ]
  Connection.airport_to_id  as DestinationAirport,
  @UI.lineItem: [ { position: 50 , label: 'Departure Time'} ]
  Connection.departure_time as DepartureTime,
  @UI.lineItem: [ { position: 60 , label: 'Arrival Time' } ]
  Connection.arrival_time  as ArrivalTime,
  @Semantics.quantity.unitOfMeasure: 'DistanceUnit'
  Connection.distance      as Distance,    /** secondary information, not to be displ
  Connection.distance_unit as DistanceUnit /** secondary information, not to be d
}
```

Results

The source code specifies which of the elements of the CDS view /DMO/I_Connection_R are displayed in the list report and in which order. In addition, the list report is given the title **Connections**. When starting the app, you do not have to select columns in the settings since they are already displayed. Press the **GO** button to retrieve data.

Standard*

Go

Connections (20)					
Airline	Connection Number	Departure Airport Code	Destination Airport Code	Departure Time	Arrival Time
AA	15	JFK	SFO	7:13:00 AM	10:04:00 AM
AA	17	MIA	HAV	7:19:00 AM	8:03:00 AM
AA	18	SFO	JFK	6:40:00 AM	3:06:00 PM

List report after UI configuration in the data definition

List Report Header

Context

The following annotations specify the items that are shown in the list report header.

You can define a header for the list report or you can implement selection fields on top of the list report to filter for a specific item. One selection field always refers to one element, but you can have more than one selection field in a single list report header.

Procedure

To include selection fields for the key elements in the header, use the annotation @UI.selectionField.position:decfloat on the respective elements.

i Note

The value's number does not represent an absolute measure and works as a relative value to the positions of the other selection fields instead. Hence, the selection fields are arranged in ascending order with regard to the annotation value.

```
...
@UI.lineItem: [ { position: 30 , label: 'Departure Airport Code'} ]
@UI.selectionField: [ { position: 10 } ]
    Connection.airport_from_id as DepartureAirport,
@UI.lineItem: [ { position: 40 , label: 'Destination Airport Code'} ]
@UI.selectionField: [ { position: 20 } ]
    Connection.airport_to_id as DestinationAirport,
...
```

Results

The selection field annotation is used on the key elements of the CDS view to create a selection field in the header on the list report. Using these selection fields, you can filter for specific list items.

Standard * 



Departure Airport:

Destination Airport:

Go

Adapt Filters

Connections (20)



Airline	Connection Number	Departure Airport Code	Destination Airport Code	Departure Time	Arrival Time	
AA	15	JFK	SFO	7:13:00 AM	10:04:00 AM	>
AA	17	MIA	HAV	7:19:00 AM	8:03:00 AM	>
AA	18	SFO	JFK	6:40:00 AM	3:06:00 PM	>

UI with selection fields filtered for connections to a specific destination airport

Object Page

Context

Whereas the list report gives a general overview of the list items, the object page shows more detailed information about a single list item. You navigate to the object page by clicking the item in the list report.

Procedure

1. [/DMO/I_Connection_R](#) using the annotation `@UI.headerInfo.typeName: 'name'`.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Read-Only E2E: Data Model Flight'

@UI.headerInfo.typeName: 'Connection'

define view /DMO/I_Connection_R
...
```

2. Create a standard facet for the object page with the annotation `@UI.facet.purpose: #STANDARD`. This annotation must be in the element section.

A facet is a type of section in the object page. It can contain diagrams or other information in a discrete part of the user interface.

```

define view entity /DMO/I_Connection_R
  as select from /dmo/connection as Connection

{
@UI.facet: [ { purpose: #STANDARD } ]
...

```

3. Specify the type of the facet. In our case, the object page displays the detailed information of one list item. Use the annotation `@UI.facet.type: #IDENTIFICATION_REFERENCE`.

```

define view entity /DMO/I_Connection_R
  as select from /dmo/connection as Connection{
    @UI.facet: [ {
      purpose: #STANDARD,
      type: #IDENTIFICATION_REFERENCE } ]
...

```

4. Specify a name for the object page facet header. Use the annotation `@UI.facet.label: 'name'`.

```

define view entity /DMO/I_Connection_R
  as select from /dmo/connection as Connection
{
  @UI.facet: [ {
    purpose: #STANDARD,
    type: #IDENTIFICATION_REFERENCE,
    label: 'Connection' } ]
...

```

5. To define the position of the facet, use the annotation `@UI.facet.position: decfloat`.

```

define view entity /DMO/I_Connection_R
  as select from /dmo/connection as Connection{
    @UI.facet: [ {
      purpose: #STANDARD,
      type: #IDENTIFICATION_REFERENCE,
      label: 'Connection',
      position: 10 } ]
...

```

An object page of a type identification reference is created. You can now define the elements that are displayed in the object page.

6. Specify the position and the label for each element that you want to show in the object page. Use the annotations `@UI.identification.position: 'decfloat'` and `@UI.identification.label: 'name'` on each element.

```

{ ...
  @UI: { identification:[ { position: 10, label: 'Airline' } ] }
key   Connection.carrier_id      as AirlineID,
  @UI: { identification:[ { position: 20, label: 'Connection Number' } ] }
key   Connection.connection_id   as ConnectionID,
  @UI: { identification:[ { position: 30, label: 'Departure Airport Code' } ] }
  @UI.selectionField: [ { position: 10 } ]
    Connection.airport_from_id as DepartureAirport,
  @UI: { identification:[ { position: 40, label: 'Destination Airport Code' } ] }
  @UI.selectionField: [ { position: 20 } ]
    Connection.airport_to_id   as DestinationAirport,
  @UI: { identification:[ { position: 50, label: 'Departure Time' } ] }
    Connection.departure_time as DepartureTime,
  @UI: { identification:[ { position: 60, label: 'Arrival Time' } ] }
    Connection.arrival_time   as ArrivalTime,
  @Semantics.quantity.unitOfMeasure: 'DistanceUnit'
  @UI: { identification:[ { position: 70, label: 'Distance' } ] }
    Connection.distance       as Distance,           /** secondary information, not to be di
    Connection.distance_unit   as DistanceUnit       /** secondary information, not to be dis
}

```

The following image displays the object page after clicking the connection item [JL 407](#).



Connection



Connection

Airline:

JL

Connection Number:

407

Departure Airport Code:

NRT

Destination Airport Code:

FRA

Departure Time:

1:23:00 PM

Arrival Time:

3:56:00 PM

Distance:

9,379 KM

Object page with identification reference

7. Activate the CDS view.

Results

The resulting source code, including all annotations that are relevant for the UI in the data definition, is as follows:

i *Expand the following code sample to view the source code*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Read-Only E2E: Data Model Connection'
```

```
@UI.headerInfo: { typeName: 'Connection',
                  typeNamePlural: 'Connections' }
```

```
define view entity /DMO/I_Connection_R
as select from /dmo/connection as Connection
```

```

{
    @UI.facet: [
        { id: 'Connection',
          purpose: #STANDARD, }
        type:      #IDENTIFICATION_REFERENCE,
        label:      'Connection' } ]

    @UI.lineItem: [ { position: 10, label: 'Airline'} ]
    @UI: { identification:[ { position: 10, label: 'Airline' } ] }
key   Connection.carrier_id      as AirlineID,

    @UI.lineItem: [ { position: 20, label:'Connection Number' } ]
    @UI: { identification:[ { position: 20, label: 'Connection Number' } ] }
key   Connection.connection_id   as ConnectionID,

    @UI: { identification:[ { position: 30, label: 'Departure Airport Code'} ]
    @UI.lineItem: [ { position: 40 , label: 'Destination Airport Code'} ]
    @UI.selectionField: [ { position: 10 } ]
    Connection.airport_from_id   as DepartureAirport,

    @UI.lineItem: [ { position: 40 , label: 'Destination Airport Code'} ]
    @UI: { identification:[ { position: 40, label: 'Destination Airport Code'} ] }
    @UI.selectionField: [ { position: 20 } ] }
    Connection.airport_to_id     as DestinationAirport,

    @UI.lineItem: [ { position: 50 , label: 'Departure Time'} ]
    @UI: { identification:[ { position: 50, label: 'Departure Time' } ] }
    Connection.departure_time    as DepartureTime,

    @UI.lineItem: [ { position: 60 , label: 'Arrival Time' } ]
    @UI: { identification:[ { position: 60, label: 'Arrival Time' } ] }
    Connection.arrival_time      as ArrivalTime,

    @Semantics.quantity.unitOfMeasure: 'DistanceUnit'
    @UI: { identification:[ { position: 70, label: 'Distance' } ] }
    Connection.distance          as Distance,          /** secondary information, not to be d

    Connection.distance_unit     as DistanceUnit       /** information is given with element D
}

```

Related Information

[CDS Annotations](#)

Generating a RAP Business Service with the Generate ABAP Repository Objects Wizards

Introduction

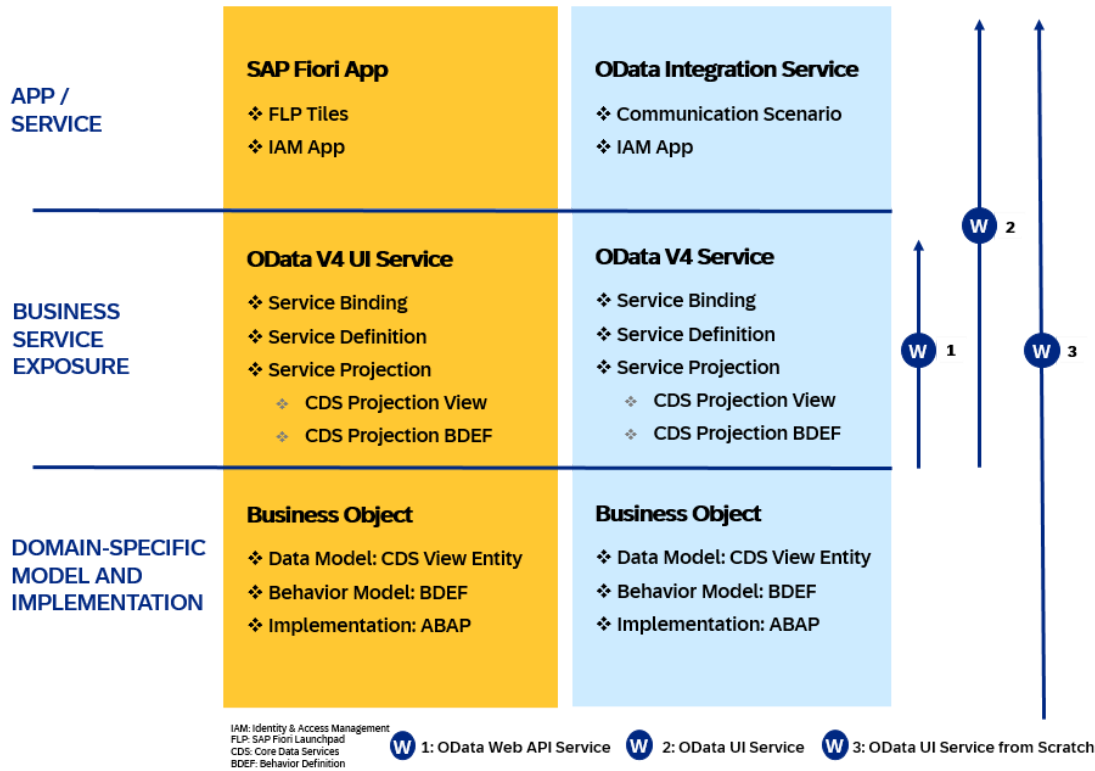
ABAP development tools for Eclipse offers you different wizards that support you in generating repository objects required for use cases like creating a transactional app, or creating integration services. Depending on the wizard you choose, you start the

generation process with an already existing repository object, or completely from scratch. You can also decide whether your business object (BO) should be generated as a UI service or a Web API service.

Wizard Map

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

App Overview and Creation Wizard Support



Please note that image maps are not interactive in PDF outputs.

Wizard Overview

OData Web API Service

This wizard generates a Web API service on the basis of a database table. The result is a Web API service that can be consumed by any OData client. However, this is not a UI service and therefore UI-specific information is not provided. For more information, see [OData Web API Service](#).

OData UI Service

This wizard generates a UI service on the basis of an already existing repository object such as a database table, a behavior definition or a CDS view entity. The result is a full-blown RAP UI service. For more information, see [OData UI Service](#).

OData UI Service from Scratch

i Note

This feature is currently only available in the SAP BTP ABAP environment.

This wizard generates a UI service completely from scratch. The result is a full-blown RAP UI service. For more information, see [OData UI Service from Scratch](#).

[Creating Integration Services](#)

With integration services, you can create connections to other systems and exchange data using different protocols. This enables seamless communication and data exchange between services and apps, and allows you to build end-to-end business processes across system boundaries. An integration use case is the [OData Web API Service](#) wizard, a service that can be exposed and consumed.

[Creating Transactional Apps](#)

Transactional services provide a robust and scalable solution for managing business processes and data. You can build business objects and expose them as services in order to consume them in UIs. Transactional apps run standard operations such as create, update or delete on the data set. The [OData UI Service](#) wizard and the [OData UI Service from Scratch](#) wizard allow you to create transactional apps and to consume them in UIs.

Creating Integration Services

With integration services, you can create connections to other systems and exchange data using different protocols. This enables seamless communication and data exchange between services and apps, and allows you to build end-to-end business processes across system boundaries. An integration use case is the [OData Web API Service](#) wizard, a service that can be exposed and consumed.

For more information about integration services, see [Develop Integration Services](#) and [Integration Use Cases - Initial Considerations](#).

[OData Web API Service](#)

You can create all RAP service-related repository objects on the basis of an already existing database table by using the [OData Web API Service](#) wizard. The end result is a RAP Web API service.

OData Web API Service

You can create all RAP service-related repository objects on the basis of an already existing database table by using the [OData Web API Service](#) wizard. The end result is a RAP Web API service.

Introduction

The [OData Web API Service](#) wizard generates all the repository objects that are relevant for a business object and for a RAP service on the basis of a database table. The result is a RAP Web API service. An OData service that is exposed as a Web API is used for all other scenarios apart from UIs. Web APIs can be consumed by any OData client and don't provide any UI-specific information in the service metadata. UI-specific repository objects are therefore not generated by the wizard, however, the following are:

- SAP object node type
- SAP object type
- Service binding
- Service definition
- Access control
- RAP projection behavior definition
- RAP behavior definition
- Data definition for CDS projection view entity
- Data definition for CDS view entity

- ABAP class for ABAP behavior pool

Prerequisites

To build a RAP BO with the [OData Web API Service](#) wizard, you need to prepare a database table that has a client field.

Use

1. In the project explorer, open the context menu on your database table and choose [Generate ABAP Repository Objects](#).

i Note

You can also start from your project or your package, but then you have to define a referenced database table when you select the [OData Web API Service](#) wizard.

2. Select [ABAP RESTful Application Programming Model: OData Web API Service](#) as use case and choose [Next](#).
3. If not already filled in automatically, enter your package name in the field [Package](#) and choose [Next](#).
4. Navigate through the wizard and provide names for the objects that are generated.

i Note

In the [General](#) tab you can provide a project name as well as a specific prefix and suffix for your object names.

If the names of the objects match your requirements, choose [Next](#).

5. The wizard shows you a list of the repository objects that are going to be generated and shows you a preview for most of the artifacts. If the list meets your requirements, choose [Next](#).
6. Select a transport request and choose [Finish](#).

[Example: OData Web API Service Generation Based on a Database Table](#)

This topic describes how you can create a Web API service with the [OData Web API Service](#) wizard. It is exemplified by a RAP BO based on the database table `/dmo/customer`.

Example: OData Web API Service Generation Based on a Database Table

This topic describes how you can create a Web API service with the [OData Web API Service](#) wizard. It is exemplified by a RAP BO based on the database table `/dmo/customer`.

Prepare Database Table `/dmo/customer`

To match the prerequisites mentioned in [OData Web API Service](#), add administrative fields to the database table. These fields are used by the managed BO provider to fill the correct values during runtime.

The field `local_last_changed_at` is used as ETag field for OData ETag comparison. The field `last_changed_at` is used as total ETag, which is necessary for draft BOs.

i Expand the following code sample to view the source code of database table `/dmo/customer`.

☰ Sample Code

```

@EndUserText.label : 'Flight Reference Scenario: Managing Customers'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/customer {
  key client          : abap.clnt not null;
  key customer_id     : /dmo/customer_id not null;
  first_name          : /dmo/first_name;
  last_name           : /dmo/last_name;
  title               : /dmo/title;
  street              : /dmo/street;
  postal_code         : /dmo/postal_code;
  city                : /dmo/city;
  country_code        : land1;
  phone_number        : /dmo/phone_number;
  email_address       : /dmo/email_address;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_created_by    : abp_creation_user;
  local_created_at    : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_last_changed_by : abp_locinst_lastchange_user;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
  last_changed_at     : abp_lastchange_tstmpl;
}

```

Use the OData Web API Service Wizard

1. In the project explorer, open the context menu on the database table `/dmo/customer` and choose **Generate ABAP Repository Objects**.
2. Choose **ABAP RESTful Application Programming Model: OData Web API Service** as use case.
3. Navigate through the wizard and provide names for the objects that are generated. For more information, see [OData Web API Service](#).

The following names are used for this example:

ABAP Artifacts Generation/Modification List

List of repository objects that are going to be generated.

Repository Object Name	ABAP Object Type	Operation	Notes
Service Artifacts			
Business Service Layer			
/DMO/API_CUSTOMER_O4_G	Service Definition	CREATE	
/DMO/API_CUSTOMER_O4_G	Service Binding	CREATE	Preview is not available, however the Service Binding '/DMO/API_CUSTOMER_O4_G' will be created.
SAP Object Type			
/DMO/CUSTOMER_G	SAP Object Node Type	CREATE	
/DMO/CUSTOMER_G	SAP Object Type	CREATE	
Business Object Projection Layer			
/DMO/C_CUSTOMER_G	Data Definition	CREATE	
/DMO/C_CUSTOMER_G	Behavior Definition	CREATE	
Business Object Layer			
/DMO/R_CUSTOMER_G	Data Definition	CREATE	
/DMO/R_CUSTOMER_G	Behavior Definition	CREATE	

Preview:

```

@EndUserText.label : 'Service Definition for /DMO/C_CUSTOMER_G'
@ObjectModel.leadingEntity.name : '/DMO/C_CUSTOMER_G'
define service /DMO/API_CUSTOMER_O4_G provider contracts odata_v4_webapi {
  expose /DMO/C_CUSTOMER_G ;
}

```

< Back Next > Finish Cancel

4. Choose **Next**, select a transport request and then choose **Finish** to start the generation of all repository objects.

You can easily add more BO entities to the composition structure or add other features to the generated objects. These features can be related to the data model, for example value helps or search capabilities, or related to the behavior. In the behavior definition, you can add business logic by defining and implementing actions, validations, or determinations. For more information about the result, see [Generation Result of the OData Web API Service Wizard](#).

i Note

When adding new CDS view entities to the compositional structure of the data model, make sure that you also extend the behavior definition for the new BO entities. Otherwise your business service will show errors.

[Generation Result of the OData Web API Service Wizard](#)

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Generation Result of the OData Web API Service Wizard

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Result

As a result, the wizard generates all repository objects that are displayed in the preview. All objects are activated automatically. The RAP business object is not draft-enabled and also not exposed as a UI service. You therefore don't have a database table for draft tables and no metadata extensions as generated repository artifacts.

Data Model

The generation wizard creates the business object on the basis of a database table. The data model in the CDS view entities uses all fields that are defined in the database table. It adds CDS annotations that can be retrieved from the database modeling, for example `@Semantics` annotations for administrative fields, which are needed for a managed BO. The wizard also creates alias names for the elements in the CDS view entity that represents the BO entity. The alias names are created in camel case.

The CDS projection view entity contains all relevant fields for the transactional data model and the dedicated admin field for ETag handling.

Behavior

The generation wizard creates a managed RAP BO without draft capabilities. The behavior definition defines strict mode. It contains the most basic behavior characteristics, for example the lock master and the authorization master for global authorization control. In addition, the ETag master is defined.

The generated RAP BO defines the standard operations. In addition, the administrative fields are set to read only and the field mapping is defined.

The RAP BO uses managed internal early numbering. For more information, see [Internal Early Numbering](#).

Business Service

The service definition exposes the generated CDS projection view and the service binding exposes an OData V4 Web API service.

Creating Transactional Apps

Transactional services provide a robust and scalable solution for managing business processes and data. You can build business objects and expose them as services in order to consume them in UIs. Transactional apps run standard operations such as create, update or delete on the data set. The [OData UI Service](#) wizard and the [OData UI Service from Scratch](#) wizard allow you to create transactional apps and to consume them in UIs.

For more information about transactional apps, see [Transactional Services](#) and [Transactional Use Cases - Initial Considerations](#).

[OData UI Service](#)

You can create all RAP service-related repository objects on the basis of an already existing repository object by using the [OData UI Service](#) wizard.

[OData UI Service from Scratch](#)

You can create all RAP service-related repository objects from scratch by using the [OData UI Service from Scratch](#) wizard. You don't need an already existing repository object, because you can start the wizard from an empty package. The wizard allows you to enter your BO requirements manually or by using the Joule chat.

OData UI Service

You can create all RAP service-related repository objects on the basis of an already existing repository object by using the [OData UI Service](#) wizard.

Introduction

Creating a RAP business object from scratch can be quite cumbersome as it involves many different repository objects that need to be created manually. The [OData UI Service](#) wizard has a solution for this in straight-forward use cases. On the basis of an already existing repository object such as a database table, a behavior definition or a CDS view entity, it creates all the repository objects that are relevant for a business object and for a RAP service. You simply have to define names for these objects and navigate through the wizard. The end result is a full-blown RAP UI service.

The following repository objects are created by the wizard:

- SAP object node type
- SAP object type
- Service binding
- Service definition
- Access control
- RAP projection behavior definition
- RAP behavior definition
- Data definition for CDS projection view entity
- Data definition for CDS view entity
- Metadata extension
- Database table for draft table
- ABAP class for ABAP behavior pool

i Note

The generated repository objects depend on your starting point.

Prerequisites

Prerequisite for a Database Table as Starting Point

To build a RAP BO with the **OData UI Service** wizard, you can use an already existing database table as starting point that

- has a client field
- defines administrative fields that use the RAP reuse elements.

For more information, see [RAP Reuse Data Elements](#).

Prerequisite for a Behavior Definition as Starting Point

To build a RAP BO with the **OData UI Service** wizard, you can use an already existing behavior definition as starting point.

Prerequisite for a CDS View as Starting Point

To build a RAP BO with the **OData UI Service** wizard, you can use an already existing CDS view as starting point.

Use

The **OData UI Service** wizard can be used for a quick start with RAP. It facilitates the creation of repository objects and ensures a consistent and functional business service with a few clicks and just one prepared repository object. The feature scope of the generation wizard is limited to creating a business service that exposes a draft-enabled managed BO that contains no more than one single BO entity.

i Note

It is not possible to create just a subset of the above-mentioned repository objects. It always generates the complete BO and the business service. The BO is always transaction- and draft-enabled and of implementation type managed.

1. In the project explorer, open the context menu on your repository object and choose **Generate ABAP Repository Objects**.

i Note

You can also start from your project or your package, but then you have to define a referenced object when you select the **OData UI Service** generator.

2. Select **ABAP RESTful Application Programming Model: OData UI Service** as use case and choose **Next**.
3. If not already filled in automatically, enter your package name in the field **Package** and choose **Next**.
4. Navigate through the wizard and provide names for the objects that are generated.

i Note

In the **General** tab you can provide a project name as well as a specific prefix and suffix for your object names.

If the names of the objects match your requirements, choose **Next**.

5. The wizard shows you a list of the repository objects that are going to be generated and shows you a preview for most of the artifacts. If the list meets your requirements, choose **Next**.
6. Select a transport request and choose **Finish**.

Generating a Transactional App from the Business Object Interface

The **OData UI Service** supports creating transactional apps based on the ABAP RESTful Application programming model by using released business object interfaces or custom business object interfaces to create a UI service. For more information, see [Generating a Transactional App from the Business Object Interface](#).

[Example: OData UI Service Generation Based on a Database Table](#)

This topic describes how you can create a UI service with the **OData UI Service** wizard. It is exemplified by a RAP BO based on the database table `/dmo/customer`.

Example: OData UI Service Generation Based on a Database Table

This topic describes how you can create a UI service with the **OData UI Service** wizard. It is exemplified by a RAP BO based on the database table `/dmo/customer`.

Prepare Database Table `/dmo/customer`

To match the prerequisites mentioned in [OData UI Service](#), add administrative fields to the database table. These fields are used by the managed BO provider to fill the correct values during runtime.

The field `local_last_changed_at` is used as ETag field for OData ETag comparison. The field `last_changed_at` is used as total ETag, which is necessary for draft BOs.

i *Expand the following code sample to view the source code of database table `/dmo/customer`.*

Sample Code

```
@EndUserText.label : 'Flight Reference Scenario: Managing Customers'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/customer {
  key client          : abap.clnt not null;
  key customer_id     : /dmo/customer_id not null;
  first_name          : /dmo/first_name;
  last_name           : /dmo/last_name;
  title               : /dmo/title;
  street              : /dmo/street;
  postal_code         : /dmo/postal_code;
  city                : /dmo/city;
  country_code        : land1;
  phone_number        : /dmo/phone_number;
  email_address       : /dmo/email_address;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_created_by    : abp_creation_user;
  local_created_at    : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_last_changed_by : abp_locinst_lastchange_user;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
  last_changed_at      : abp_lastchange_tstmpl;
```

}

Use the OData UI Service Wizard

1. In the project explorer, open the context menu on the database table `/dmo/customer` and choose **Generate ABAP Repository Objects**.
2. Choose **ABAP RESTful Application Programming Model: OData UI Service** as use case.
3. Navigate through the wizard and provide names for the objects that are generated. For more information, see [OData UI Service](#).

The following names are used for this example:

ABAP Artifacts Generation/Modification List
List of repository objects that are going to be generated.

Repository Object Name	ABAP Object Type	Operation	Notes
Service Artifacts			
Business Service Layer			
/DMO/UI_CUSTOMER_O4_G	Service Definition	CREATE	
/DMO/UI_CUSTOMER_O4_G	Service Binding	CREATE	Preview is not available, however the Service Binding '/DMO/UI_CUSTOMER_O4_G' will be created.
SAP Object Type			
/DMO/CUSTOMER_G	SAP Object Node Type	CREATE	
/DMO/CUSTOMER_G	SAP Object Type	CREATE	
Business Object Projection Layer			
/DMO/C_CUSTOMERS_G	Data Definition	CREATE	
/DMO/C_CUSTOMERS_G	Metadata Extension	CREATE	
/DMO/C_CUSTOMERS_G	Behavior Definition	CREATE	
Business Object Layer			
/DMO/R_CUSTOMERS_G	Data Definition	CREATE	
/DMO/R_CUSTOMERS_G	Behavior Definition	CREATE	
Data Base Model Layer			
/DMO/CUSTOMER_G	Database Table	CREATE	

Preview:

```
@EndUserText.label : 'Service Definition for /DMO/C_CUSTOMERS_G'
@ObjectModel.leadingEntity.name : '/DMO/C_CUSTOMERS_G'
define service /DMO/UI_CUSTOMER_O4_G provider contracts odata_v4_ui {
  expose /DMO/C_CUSTOMERS_G ;
}
```

< Back Next > Finish Cancel

Preview ABAP Artifacts Generation/Modification List

4. Choose **Next**, select a transport request and then choose **Finish** to start the generation of all repository objects.

You can easily add more BO entities to the composition structure or add other features to the generated objects. These features can be related to the data model, for example value helps or search capabilities, or related to the behavior. In the behavior definition, you can add business logic by defining and implementing actions, validations, or determinations. For more information about the result, see [Generation Result of the OData UI Service Wizard](#).

i Note

When adding new CDS view entities to the compositional structure of the data model, make sure that you also extend the behavior definition for the new BO entities. Otherwise your business service will show errors.

[Generation Result of the OData UI Service Wizard](#)

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Generation Result of the OData UI Service Wizard

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Result

As a result, the [OData UI Service](#) wizard generates all repository objects required for your BO. All objects are activated automatically. The draft-enabled RAP business object is exposed as a UI service. By opening the service binding editor, you can publish the service locally and open the [Fiori Elements App Preview](#) to see the result on the UI.

You can use all repository objects as usual and modify them. You can add data model options, for example value help or search capabilities to CDS view entities. You can also extend the behavior with actions and functions or other business logic with validations and determinations. In general, the generated business object is editable and useable in the same way as a manually developed business object.

Data Model

The generation wizard creates the business object on the basis of a database table. The data model in the CDS view entities uses all fields that are defined in the database table. It adds CDS annotations that can be retrieved from the database modeling, for example `@Semantics` annotations for administrative fields, which are needed for a managed BO. The wizard also creates alias names for the elements in the CDS view entity that represents the BO entity. The alias names are created in camel case.

The CDS projection view entity contains all relevant fields for the transactional data model and the dedicated admin field for ETag handling.

Metadata Extension

For UI scenarios, the generation wizard creates a metadata extension object with the basic UI-related annotations for a UI service. The UI service that is generated creates a basic list report application that exposes all transaction-enabled fields on the UI. The fields are annotated with the relevant `@UI.LineItem` and `@UI.Identification` annotation.

Behavior

The generation wizard creates a managed RAP BO with draft capabilities. The behavior definition defines strict mode. It contains the most basic behavior characteristics, for example the lock master and the authorization master for global authorization control. In addition, the ETag master is defined.

The generated RAP BO defines the standard operations and the draft actions. In addition, the administrative fields are set to read only and the field mapping is defined.

The RAP BO uses managed internal early numbering. For more information, see [Internal Early Numbering](#).

Business Service

The service definition exposes the generated CDS projection view and the service binding exposes an OData V4 UI service.

OData UI Service from Scratch

You can create all RAP service-related repository objects from scratch by using the [OData UI Service from Scratch](#) wizard. You don't need an already existing repository object, because you can start the wizard from an empty package. The wizard allows you to enter your BO requirements manually or by using the Joule chat.

Introduction

The **OData UI Service from Scratch** wizard allows you to create a RAP BO without the requirement of an already existing artifact and without having to manually create all the necessary repository objects yourself. This is where it differs from the other wizards, which are based on an already existing repository object such as a database table, a CDS view entity or a behavior definition. You can start the **OData UI Service from Scratch** wizard from an empty package and the end result is a full-blown RAP UI service.

The following repository objects are created by the wizard:

- SAP object node type
- SAP object type
- Service binding
- Service definition
- Access control
- RAP projection behavior definition
- RAP behavior definition
- Data definition for CDS projection view entity
- Data definition for CDS view entity
- Metadata extension
- Database table for draft table
- Database table for persistent table
- ABAP class for ABAP behavior pool

Current Limitations of the OData UI Service from Scratch Wizard

The **OData UI Service from Scratch** wizard can help you create a managed business object with internal early numbering. The wizard additionally offers the service types transactional with draft, transactional without draft and read-only. However, the wizard has the following limitations:

- Only one BO is created.
- Only UUID numbering is supported.

i Note

- UUIDs will always be generated implicitly as key fields for you.
- The UUID key fields will not be displayed in the configuration part of the generator.
- You can't define specific fields as UUID **key fields** by yourself.
- No unmanaged BO is created.
- No Web APIs or analytical services are supported.
- Actions, determinations or validations aren't supported.
- No interface layers are created. You only have the base layer and the projection layer.

- Associations and cross-BO associations aren't supported.

Prerequisites

This wizard allows you to start from an empty package without an already existing repository object in it. The only prerequisite for using this wizard is an already existing package.

[Using the OData UI Service from Scratch Wizard](#)

Learn how to use the [OData UI Service from Scratch](#) wizard in ABAP development tools for Eclipse in order to create a RAP business object by either using the wizard manually or with the aid of the Joule chat.

Using the OData UI Service from Scratch Wizard

Learn how to use the [OData UI Service from Scratch](#) wizard in ABAP development tools for Eclipse in order to create a RAP business object by either using the wizard manually or with the aid of the Joule chat.

Context

The [OData UI Service from Scratch](#) wizard can be started on the basis of an empty package with or without the aid of the Joule chat by working through the following steps.

Procedure

1. In the project explorer, open the context menu on your project or your package.
2. Choose [Generate ABAP Repository Objects](#).
3. Select [ABAP RESTful Application Programming Model: OData UI Service from Scratch](#) as use case and choose **Next**.
4. If not already filled in automatically, enter your package name in the field **Package** and choose **Next**.
5. In the [Configure Generator](#) window, navigate through the wizard.
 - a. Under the tab [Service Configuration](#), you can select a service type (read-only, with draft, or without draft), as well as a prefix and/or a suffix with a maximum length of three characters for the artifacts that are going to be generated for you. If you start the wizard from an already existing package with a specific namespace, the wizard automatically assigns the same namespace of your package to the generated artifacts.

→ Recommendation

We recommend that you use at least a suffix.
 - b. Additionally, you need to provide a project name. If you use the Joule chat for your BO creation, the project name will be generated for you.
6. In the [Configure Generator](#) window, you have two options to continue. You can either use the Joule chat to define the requirements for your BO, or you can add them manually in the wizard.

Option without Joule Chat	Option with Joule Chat
◦ In the tab Business Object Entities, choose Add to define the entities you would like to have for your BO. ◦ The first entity you enter is the root entity. For the following entities you can enter the cardinality. For all your entities, you can additionally use an already existing template, such as a database table or a DDIC structure.	◦ In the Joule chat window on the right-hand side of the wizard, enter a prompt with the requirements for your BO and press Enter. i Note

Option without Joule Chat	Option with Joule Chat
- o The tab Business Object Entity Fields automatically shows your entities as a drop-down list as soon as you entered them. Click on your entities and add your business object entity fields. If you have used a template, the fields of your entities are already filled in. - o Choose Next in order to get your generated artifacts. For more information, see Example: Option without Joule Chat.	Example Prompt: "Create a Travel BO with two entities." - o You have the option to use an already existing template, such as a database table or a DDIC structure to create an entity. You must specify in the prompt which template you want to use. - o Based on your prompt, Joule now proposes you a list of BO entities and BO entity fields. If this suggestion meets your requirements, choose Accept. The wizard now fills the configuration fields automatically. - o You can still adjust the fields manually, or you can use the Joule chat again for further adjustments. - o Choose Next in order to get your generated artifacts. For more information, see Example: Option with Joule Chat.

i Note

- o UUIDs will always be generated implicitly as key fields for you.
- o The UUID key fields will not be displayed in the configuration part of the generator.
- o You can't define specific fields as UUID **key fields** by yourself.

7. The wizard shows you a list of the repository objects that are going to be generated and shows you a preview for most of the artifacts. If the list meets your requirements, choose **Next**.

8. Select a transport request and choose **Finish**.

[Example: Option without Joule Chat](#)

This topic describes how you can create an OData UI service without an already existing repository object by using the **OData UI Service from Scratch** wizard without the Joule chat.

[Example: Option with Joule Chat](#)

This topic describes how you can create an OData UI service without an already existing repository object by using the **OData UI Service from Scratch** wizard with the aid of the Joule chat.

[Generation Result of the OData UI Service from Scratch Wizard](#)

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Example: Option without Joule Chat

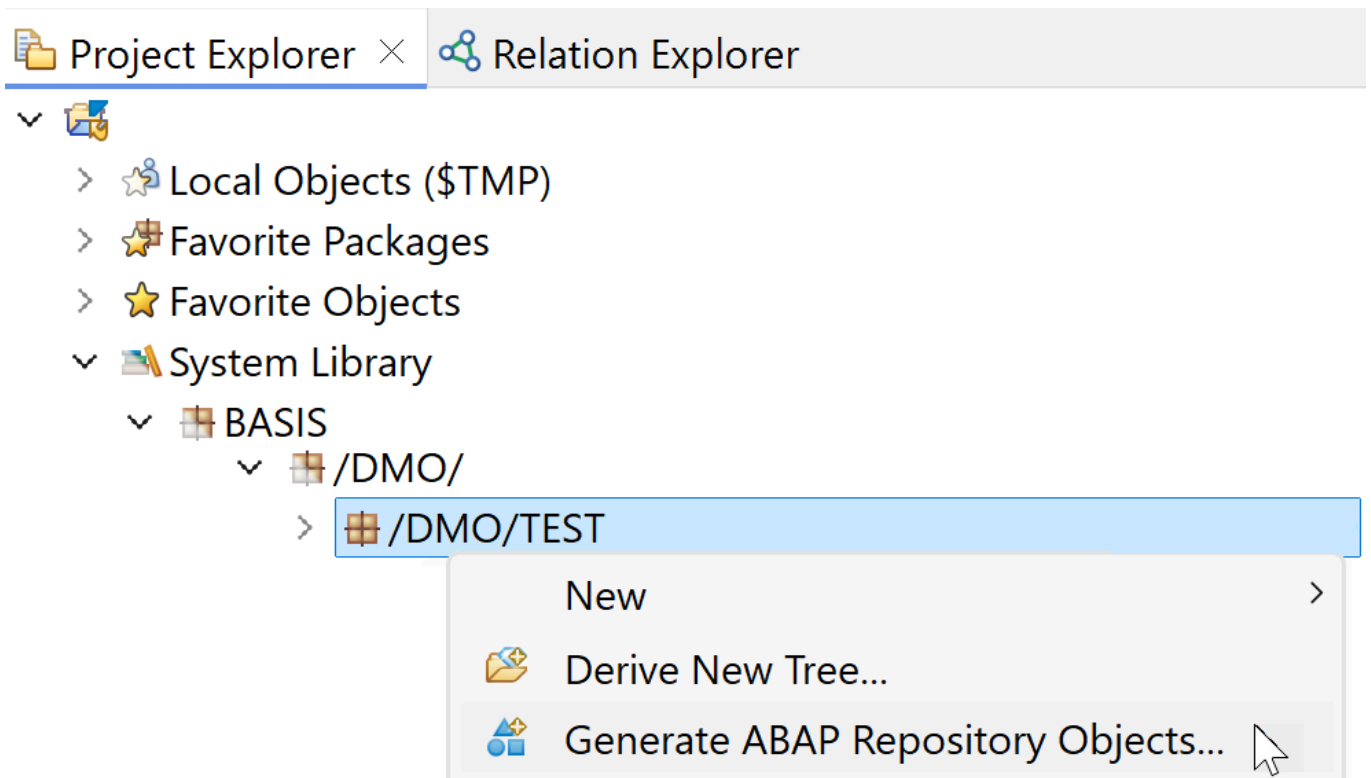
This topic describes how you can create an OData UI service without an already existing repository object by using the **OData UI Service from Scratch** wizard without the Joule chat.

Prepare a Package

For starting the wizard, the only prerequisite is a previously created package. You can then start the wizard based on your package. If you start the wizard from your project, you need to enter your package name in the field **Package** as described in [Using the OData UI Service from Scratch Wizard](#).

Use the OData UI Service from Scratch Wizard

1. In the project explorer, open the context menu on your package and choose **Generate ABAP Repository Objects**. In this example, a package with the namespace /DMO/ was used:



i Note

You can also open the context menu on your project, but then you have to enter your package name in the field **Package**.

2. Choose **ABAP RESTful Application Programming Model: OData UI Service from Scratch** as use case.
3. The **Configure Generator** window opens:

The screenshot shows the 'Configure Generator' window for 'Generate ABAP Repository Objects'. The window has a title bar 'Generate ABAP Repository Objects' and a subtitle 'Configure Generator'. Below the subtitle is a text field 'Specify values for the generator, 'OData UI Service from Scratch''. The main area is divided into two tabs: 'Configuration of Service Artifacts' and 'Service Configuration'. The 'Configuration of Service Artifacts' tab is active, showing a list of artifacts: 'Service Configuration', 'Business Object Entities', and 'Business Object Entity Fields'. The 'Service Configuration' tab is also visible, showing fields for 'Service Type' (set to 'Transactional with Draft'), 'Objects Naming' (with 'Artifact Prefix' and 'Artifact Suffix' fields), and 'SAP Object Type / Service Definition / Service Binding Naming' (with 'Project Name' field set to 'Travel BO'). At the bottom, there are buttons for '< Back', 'Next >', 'Finish', and 'Cancel'.

Navigate through the wizard. Decide on a service type (e.g. transactional with draft as in this example), enter a prefix and/or suffix (e.g. _G) and a project name.

4. You can then add business object entities and their respective business object entity fields:
 - o In the tab **Business Object Entities**, click **Add** to enter your business object entities (e.g. Travel and Booking).

- The tab **Business Object Entity Fields** automatically shows your entities as a drop-down list as soon as you entered them. Click on your entities and add your business object entity fields.

For more information concerning the configuration of the generator, see [Using the OData UI Service from Scratch Wizard](#).

5. Choose **Next** as soon as you finished the configuration.
6. Select a transport request and then choose **Finish** to start the generation of all repository objects.

Generated Repository Objects

The **OData UI Service from Scratch** wizard generates the following repository objects:

Generate ABAP Repository Objects

ABAP Artifacts Generation/Modification List

List of repository objects that are going to be generated.

type filter text

Repository Object Name	ABAP Object Type	Operation	Notes
Service Artifacts			
Business Service Layer			
/DMO/UI_TRAVEL_O4_G	Service Definition	CREATE	
/DMO/UI_TRAVEL_O4_G	Service Binding	CREATE	Preview is not available, however the Service Binding '/DMO/UI_TRAVEL_O4_G' will be created.
SAP Object Type			
/DMO/TRAVEL_G	SAP Object Node Type	CREATE	
/DMO/TRAVEL_G	SAP Object Type	CREATE	
Business Object Projection Layer			
/DMO/C_BOOKING_G	Data Definition	CREATE	
/DMO/C_TRAVEL_G	Data Definition	CREATE	
/DMO/C_BOOKING_G	Metadata Extension	CREATE	
/DMO/C_TRAVEL_G	Metadata Extension	CREATE	
/DMO/C_TRAVEL_G	Behavior Definition	CREATE	
/DMO/BP_C_TRAVEL_G	Class	CREATE	
Business Object Layer			
/DMO/R_BOOKING_G	Data Definition	CREATE	
/DMO/R_TRAVEL_G	Data Definition	CREATE	
/DMO/R_TRAVEL_G	Behavior Definition	CREATE	
/DMO/BP_R_TRAVEL_G	Class	CREATE	
Data Base Model Layer			
/DMO/BOOKING_D_G	Database Table	CREATE	
/DMO/BOOKING_G	Database Table	CREATE	
/DMO/TRAVEL_D_G	Database Table	CREATE	
/DMO/TRAVEL_G	Database Table	CREATE	

Preview:

@EndUserText.label : 'Service Definition for /DMO/C_TRAVEL_G'
@ObjectModel.leadingEntity.name : '/DMO/C_TRAVEL_G'
define service /DMO/UI_TRAVEL_O4_G provider contracts odata_v4_ui {
 expose /DMO/C_TRAVEL_G as Travel;
 expose /DMO/C_BOOKING_G as Booking;
}

< Back

Next >

Finish

Cancel

Since the wizard started from a package with the namespace `/DMO/`, all repository objects start with exactly this namespace.

You can easily add more BO entities to the composition structure or add other features to the generated objects. These features can be related to the data model, for example value helps or search capabilities, or related to the behavior. In the behavior definition, you can add business logic by defining and implementing actions, validations, or determinations. For more information, see [Generation Result of the OData UI Service from Scratch Wizard](#).

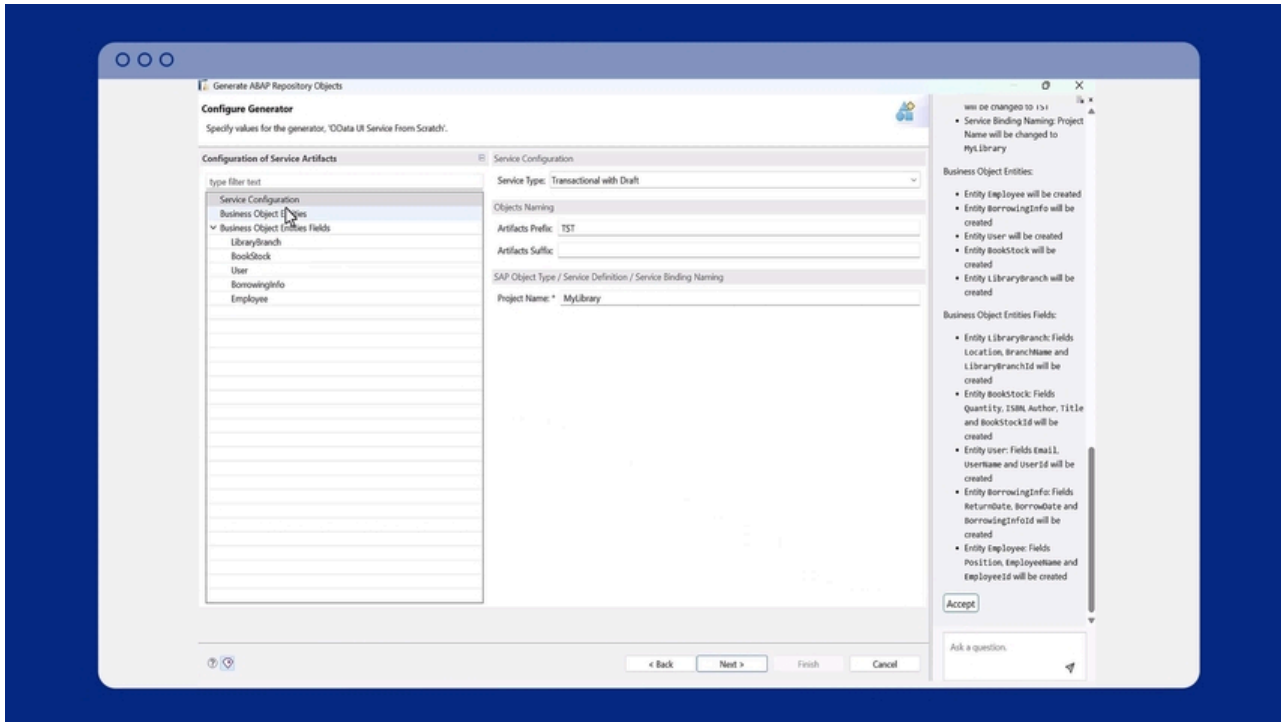
i Note

When adding new CDS view entities to the compositional structure of the data model, make sure that you also extend the behavior definition for the new BO entities. Otherwise your business service will show errors.

Example: Option with Joule Chat

This topic describes how you can create an OData UI service without an already existing repository object by using the **OData UI Service from Scratch** wizard with the aid of the Joule chat.

Demo Video of the OData UI Service from Scratch Wizard



[Open this video in a new window](#)

Below you have a step-by-step description of the **OData UI Service from Scratch** wizard using another example.

Prepare a Package

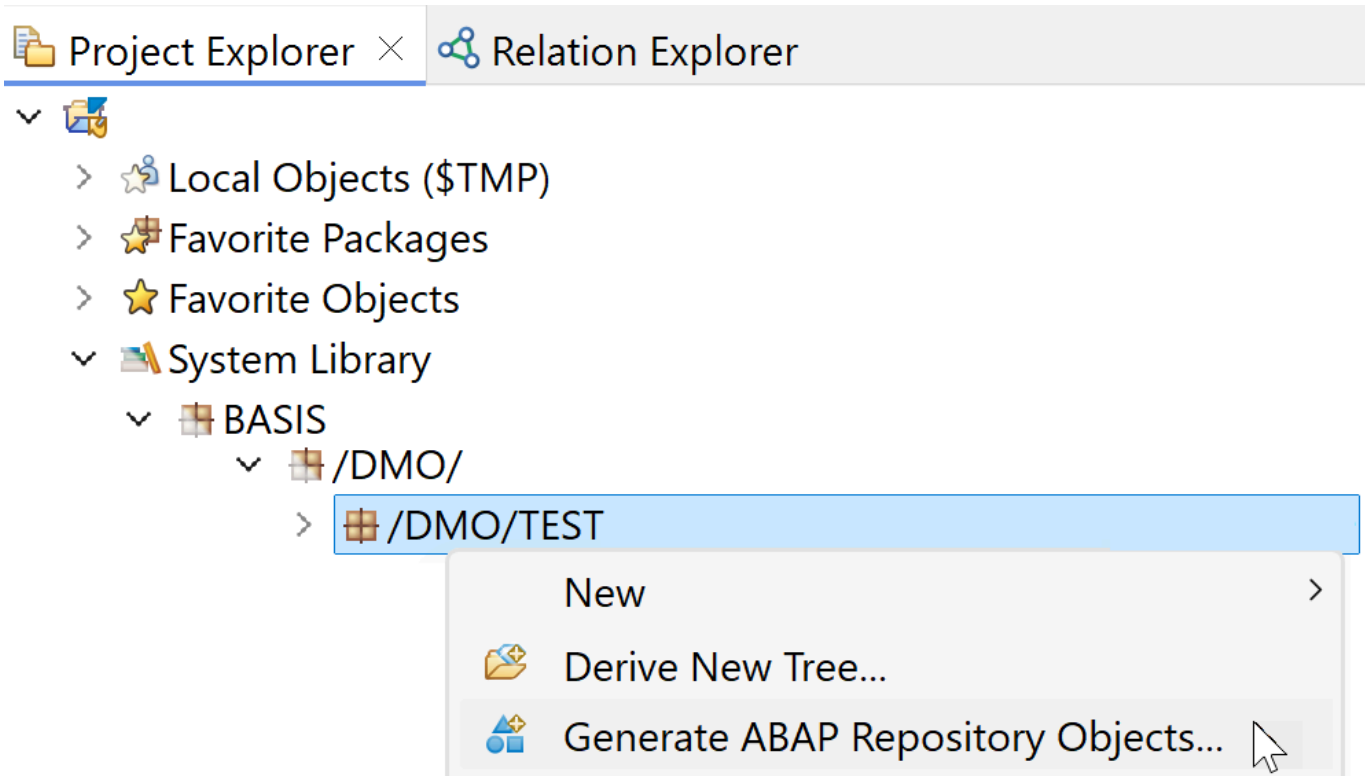
There are no prerequisites for starting the wizard other than having already created a package. You can then start the wizard based on your package. If you start the wizard from your project, you need to enter your package name in the field **Package** as described in [Using the OData UI Service from Scratch Wizard](#).

i Note

To leverage Joule's capabilities on-stack, you need to purchase an additional license: For SAP BTP ABAP environment and SAP S/4HANA Cloud Public Edition, see SAP Note [3571857](#).

Use the OData UI Service from Scratch Wizard

1. In the project explorer, open the context menu on your package and choose **Generate ABAP Repository Objects**. In this example, a package with the namespace **/DMO/** was used:



i Note

You can also open the context menu on your project, but then you have to enter your package name in the field **Package**.

2. Choose **ABAP RESTful Application Programming Model: OData UI Service from Scratch** as use case.
3. The **Configure Generator** window opens. Additionally, the Joule chat appears and allows you to enter a prompt based on the requirements for your business object. For this example, the following prompt is going to be used: "Create a Travel BO with two entities". Additionally, the suffix **_G** was added.
4. After the prompt was sent, Joule proposes you a list of BO entities and BO entity fields. If this suggestion meets your requirements, choose **Accept**.
5. The wizard fills the configuration fields automatically. You can still navigate through the wizard and edit the business object entity names and the business object entity field names provided by Joule. Decide on a service type. For more information concerning the configuration of the generator, see [Using the OData UI Service from Scratch Wizard](#).
6. Check if everything meets your requirements. Then choose **Next**.
7. Select a transport request and then choose **Finish** to start the generation of all repository objects.

Generated Repository Objects

The **OData UI Service from Scratch** wizard generates the following repository objects:

Generate ABAP Repository Objects

ABAP Artifacts Generation/Modification List

List of repository objects that are going to be generated.

type filter text

Repository Object Name	ABAP Object Type	Operation	Notes
Service Artifacts			
Business Service Layer			
/DMO/UI_TRAVEL_O4_G	Service Definition	CREATE	
/DMO/UI_TRAVEL_O4_G	Service Binding	CREATE	Preview is not available, however the Service Binding '/DMO/UI_TRAVEL_O4_G' will be created.
SAP Object Type			
/DMO/TRAVEL_G	SAP Object Node Type	CREATE	
/DMO/TRAVEL_G	SAP Object Type	CREATE	
Business Object Projection Layer			
/DMO/C_BOOKING_G	Data Definition	CREATE	
/DMO/C_TRAVEL_G	Data Definition	CREATE	
/DMO/C_BOOKING_G	Metadata Extension	CREATE	
/DMO/C_TRAVEL_G	Metadata Extension	CREATE	
/DMO/C_TRAVEL_G	Behavior Definition	CREATE	
/DMO/BP_C_TRAVEL_G	Class	CREATE	
Business Object Layer			
/DMO/R_BOOKING_G	Data Definition	CREATE	
/DMO/R_TRAVEL_G	Data Definition	CREATE	
/DMO/R_TRAVEL_G	Behavior Definition	CREATE	
/DMO/BP_R_TRAVEL_G	Class	CREATE	
Data Base Model Layer			
/DMO/BOOKING_D_G	Database Table	CREATE	
/DMO/BOOKING_G	Database Table	CREATE	
/DMO/TRAVEL_D_G	Database Table	CREATE	
/DMO/TRAVEL_G	Database Table	CREATE	

Preview:

```
@EndUserText.label : 'Service Definition for /DMO/C_TRAVEL_G'
@ObjectModel.leadingEntity.name : '/DMO/C_TRAVEL_G'
define service /DMO/UI_TRAVEL_O4_G provider contracts odata_v4_ui {
  expose /DMO/C_TRAVEL_G as Travel;
  expose /DMO/C_BOOKING_G as Booking;
}
```

?
< Back
Next >
Finish
Cancel

Since the wizard started from a package with the namespace `/DMO/`, all repository objects start with exactly this namespace.

You can easily add more BO entities to the composition structure or add other features to the generated objects. These features can be related to the data model, for example value helps or search capabilities, or related to the behavior. In the behavior definition, you can add business logic by defining and implementing actions, validations, or determinations.

Note

When adding new CDS view entities to the compositional structure of the data model, make sure that you also extend the behavior definition for the new BO entities. Otherwise your business service will show errors.

Related Information

[Generation Result of the OData UI Service from Scratch Wizard](#)

Generation Result of the OData UI Service from Scratch Wizard

This topic gives you a more detailed overview of the generation result and how you can proceed as soon as all your repository objects were generated.

Result

As a result, the **OData UI Service from Scratch** wizard generates all the repository objects required for your BO. All objects are activated automatically. Based on your service type selection in the wizard, the RAP business object is either read-only, draft-

enabled or not draft-enabled. It is also exposed as a UI service. By opening the service binding editor, you can publish the service locally and open the [Fiori Elements App Preview](#) to see the result on the UI. For more information, see [Previewing the Resulting UI Service](#).

You can use all repository objects as usual and modify them. You can add data model options, for example value help or search capabilities to CDS view entities. The business object is also enabled for development extensibility. You can extend the behavior with actions and functions or other business logic with validations and determinations. For more information, see [Validations](#), [Determinations](#) or [Actions](#). In general, the generated business object is editable and useable in the same way as a manually developed business object.

Data Model

The generation wizard creates the business object from scratch. The data model in the CDS view entities uses all fields that are defined in the database table. It adds CDS annotations that can be retrieved from the database modeling, for example `@Semantics` annotations for administrative fields, which are needed for a managed BO. The wizard also creates alias names for the elements in the CDS view entity that represents the BO entity. The alias names are created in camel case.

Additionally, the wizard generates the syntax for compositions for a BO with multiple entities. The cardinalities to the child entities in the compositions are based on what you have entered in the wizard before. For more information about compositions, see [Providing CDS Data Model with Business Object Structure](#).

The CDS projection view entity contains all relevant fields for the transactional data model and the dedicated admin field for ETag handling.

Metadata Extension

For UI scenarios, the generation wizard creates a metadata extension object with the basic UI-related annotations for a UI service. The UI service that is generated creates a basic list report application that exposes all transaction-enabled fields on the UI. The fields are annotated with the relevant `@UI.LineItem` and `@UI.Identification` annotation.

If you want to change the UI, you can do so by using UI annotations. You can use the [SAP Fiori Feature Showcase App for RAP](#)  to get an overview and examples for potential UI features.

Behavior

The generation wizard creates a managed RAP BO. The behavior definition defines strict mode. It contains the most basic behavior characteristics, for example the lock master and the authorization master for global authorization control. In addition, the ETag master is defined.

The generated RAP BO defines the standard operations and the draft actions (if it is draft-enabled). In addition, the administrative fields are set to read-only and the field mapping is defined.

The RAP BO uses managed internal early numbering. For more information, see [Internal Early Numbering](#).

Business Service

The service definition exposes the generated CDS projection view and the service binding exposes an OData V4 UI service.

If you want to deploy your app to a SAP Fiori launchpad and make it accessible for business users, see [App Deployment](#).

Developing RAP Business Objects

The documentation in this section provides a detailed step-by-step description on how to use and exploit the ABAP RESTful Application Programming Model in end-to-end scenarios.

The end-to-end scenarios in this section focus on specific development tasks. It depends on the aimed outcome of your development which scenario meets your requirements best.

The following table provides you with an overview of different RAP end-to-end scenarios and RAP features, with links to the relevant documentation.

End-to-End Scenario	Key Features of this Scenario	Explanation
Developing Read-Only List Reporting Apps	• Association modeling • Text associations • Value helps • Search capabilities • UI service	In this documentation, you find detailed information on how to develop various list-reporting features, such as creating associations, field labels and descriptions, text elements, value helps and search capabilities while creating a complete list-reporting app.
Developing Read-Only RAP Analytical Tables	• Aggregations • Grouping • UI service	In this documentation, you learn how to develop a RAP analytical table in a read-only business object. The RAP analytical table allows multiple possibilities for aggregating your data.
Developing Managed Transactional Apps	• Early Unmanaged Numbering • Actions • Static and Dynamic Feature Control • Validation • Determination • Additional save • Unmanaged save • UI service	This documentation leads you step-by-step through the implementation tasks to develop a transactional app from scratch.
Developing Unmanaged Transactional Apps	• Action • Static feature control • Locking • Etag • UI service	This documentation leads you step-by-step through the implementation tasks to develop a transactional app based on existing legacy logic.
Developing Transactional Apps with Exclusive Draft Capabilities	• Exclusive draft capabilities • Privileged mode • Authorization	This documentation is a walk-through tutorial that guides you through the main implementation steps to develop an application with exclusive draft support. Draft-enabled applications provide the business user with the highest flexibility when working with the app.

End-to-End Scenario	Key Features of this Scenario	Explanation
	• Actions with dynamic feature control • Actions with parameter and default function • Determinations • Validations • Determine actions • Side effects • Draft projection and non-draft projection • UI service	
Developing Transactional Apps with Collaborative Draft Capabilities	• Collaborative draft capabilities • Authorization • UI service	This documentation leads you step-by-step through the implementation tasks to develop a transactional app with collaborative draft capabilities. With the collaborative draft capability, a business user who has created a draft instance can invite other business users to the draft instance with the aid of a user list.
Developing Cross-BO Scenarios with Draft Scope	• Target-resolved association via reverse associations • Draft capabilities • Link/unlink action • Inverse function • Draft scope • Validation • Side effect • UI service	With draft scope in cross-BO scenarios, you can edit multiple related business objects as a single logical unit of work (LUW), ensuring consistency across related draft instances. In this documentation, you will learn how to add a draft scope to two separate, draft enabled BOs.
Developing Apps with Hierarchical Data Structures	• Hierarchical structures as treeviews • Draft capabilities • UI service	This documentation explains how to use CDS hierarchies in the context of the ABAP RESTful Application Programming Model (RAP), and expose the hierarchical structures as treeviews on the SAP Fiori UI.
Developing Transactional Apps with Multi-Inline-Edit Capabilities	• Transactional application with multi-inline-capabilities • Draft capabilities • UI service	This documentation guides you through the implementation steps to create an application with multi-inline-edit capabilities.
Developing a UI Service with Access to a Remote Service	• Remote service as data source	Based on a remote OData service, you create a new service that enhances the

End-to-End Scenario	Key Features of this Scenario	Explanation
	UI service	remote service with additional information.
Develop Web APIs	Web API service	This documentation explains how you expose an OData service as an API to be consumed by an undefined client.
Adding RAP Recommendations	- RAP recommendations - Generative AI - Value helps - Side effects - UI service	This documentation demonstrates how to implement RAP recommendations in a RAP business object. RAP recommendations provide context-aware input assistance by proposing field values for fields with value help, based on the user's input.

Related Information

[Develop Applications](#)

Development Constraints

The current version of the ABAP RESTful Application Programming Model still has some constraints for the following:

Development area	Subarea	Affected feature/capability	Restriction
Data Modeling & Persistence	Filtering & Sorting	Virtual Elements	Virtual elements can't be used to filter.
Data Modeling & Persistence	Filtering & Sorting	RAP Analytical Table	For RAP analytical tables, it is not possible to annotate calculation, filter or sort exits.
Data Modeling & Persistence	Data Types	CDS Enumerate Types	The metadata exposure of CDS enumerated types in RAP-based OData services isn't supported and can't be used as part of a RAP BO.
Service Exposure & Protocols	OData V2/ OData V4	CDS Enumerate Types	The metadata exposure of CDS enumerated types in RAP-based OData services isn't supported and can't be used as part of a RAP BO.
Data Modeling & Persistence	Client handling	Mixed Client Dependencies within RAP BOs	BOs with mixed client-dependencies in their composition hierarchy aren't consistently supported.
Data Modeling & Persistence	Search	Search on string fields in HANA Cloud systems	Search isn't possible on string fields (NCLOB) in HANA Cloud systems.
Behavior, Draft & Runtime	Draft	Collaborative Draft Handling	Collaborative draft only works in cross-BO scenarios if both BOs are collaborative draft enabled.

Development area	Subarea	Affected feature/capability	Restriction
Business Object	Cross-BO Scenarios	Collaborative Draft Handling	Collaborative draft only works in cross-BO scenarios if both BOs are collaborative draft enabled.
Business Object	Cross-BO Scenarios	Existing Active Instances in Draft Scope	Existing active instances cannot be assigned to a draft scope.
Behavior, Draft & Runtime	Draft	Reuse of Draft Tables	Reusing the same draft table within different BOs isn't supported by the RAP runtime and may lead to runtime errors.
Behavior, Draft & Runtime	Draft	Draft Tables	Annotations and associations are not allowed in draft persistency table entities. This means that only a simplified DCL model restricted to the root entity may be used for drafts. Note that DCLs for the draft persistency may be needed for security reasons for the collaborative draft.
Business Object	RAP BO	Reuse of Draft Tables	Reusing the same draft table within different BOs isn't supported by the RAP runtime and may lead to runtime errors.
Behavior, Draft & Runtime	Draft	Durable Lock	The durable lock handler doesn't support the following operations chains: • Changing an active instance followed by an edit • Activating an active instance followed by an edit • Discarding a draft instance (whose lock is still active) followed by an edit • Moving (delete and create) a new draft instance from a business object instance to another instance of the same business object • Recreate ◦ Deleting an active instance followed by a draft create with same key ◦ Creating a new draft instance, then discard, then create (draft or active) with same key • Discarding a draft instance (whose lock is not active anymore) followed by an edit
Behavior, Draft & Runtime	Draft Associations	Draft-enabled association	The query runtime doesn't support arbitrary associations as draft enabled. Draft-enabled associations are only supported if one of the following conditions is met: • Pure Foreign Key: The association is a pure foreign key association i.e. it has a

Development area	Subarea	Affected feature/capability	Restriction
			target cardinality of at most one and the on-condition maps source foreign key elements to target primary key elements and without additional conditions. • Specialization: The association is a specialization of a draft-enabled foreign key association. • Reverse: The association is a reverse association of a draft-enabled foreign key association, i.e. it mirrors the on-condition (additional conditions are allowed). • Traverse: The on-condition of the association declares equality of two foreign keys, where corresponding pure foreign key associations exist which have a common target entity and are draft enabled.
Behavior, Draft & Runtime	Behavior	Before-Modify and Augmentation Exits	If the augmentation adds operations on "non-original" instances (which are not part of the original request), the runtime automatically attempts to lock them. To ensure a proper response, the augmentation exit to must use the "relating by" syntax to relate non-original instances to original ones. This allows the runtime to map lock failures of non-original instances back to the corresponding related instances. If the non-original instances require authorization (not covered by the original instances), it is the responsibility of the augmentation exit to check them (for example, using GET PERMISSIONS).
Behavior, Draft & Runtime	Behavior	Using Determinations and Validations	When using determinations and validations, the following constraints apply: • The trigger operation <code>update</code> on save is only supported in combination with <code>create</code>. • The trigger operation <code>delete</code> is of limited use, unless the instances of entities are identified by semantic keys. • A determination or validation with the trigger operation <code>delete</code> can't be assigned to a determine action of a descendant entity.
Behavior, Draft & Runtime	Actions, function & parameters	Function Cardinality	For functions, only up to one result ([0..1] cardinality) is supported.
Behavior, Draft & Runtime	Actions, function & parameters	Static Actions	Dynamic feature control can't be used with static actions.

Development area	Subarea	Affected feature/capability	Restriction
Behavior, Draft & Runtime	Dynamic Feature Control	Static Actions	Dynamic feature control can't be used with static actions.
Behavior, Draft & Runtime	Static Feature Control	Static Fields	The RAP feature control attributes <code>readonly:create</code> and <code>mandatory:update</code> are currently only supported in Web APIs. They are not supported in SAP Fiori UI applications.
Behavior, Draft & Runtime	Actions, function & parameters	Large Objects	It's not possible to define an <code>Edm.Stream</code> as action parameter directly. A workaround is to use a complex type as parameter for an action with all the <code>Edm.Stream</code> properties.
Behavior, Draft & Runtime	Draft Scope	Draft-enabled Associations	It's not possible to define a draft-enabled target-resolved association in an extension of a released SAP BO (including the required reverse association in the target BO) to establish a transactionally-enabled cross-BO relationship to a released SAP BO or a custom BO. This modeling results in an error on service layer.
Behavior, Draft & Runtime	Draft Scope	Existing Active Instances	Existing active instances cannot be assigned to a draft scope.
Behavior, Draft & Runtime	Draft Scope	Performance issues for interconnected instances	If too many interconnected instances are included within the same draft scope, system performance may degrade significantly due to longer processing times, and higher database load. Therefore, the draft scope should be kept as focused and limited as possible to a small number of business object instances.
Extensibility and Release Contracts	Extensibility	Draft-enabled Associations	It's not possible to define a draft-enabled target-resolved association in an extension of a released SAP BO (including the required reverse association in the target BO) to establish a transactionally-enabled cross-BO relationship to a released SAP BO or a custom BO. This modeling results in an error on service layer.
Business Object	RAP BO	Draft-enabled Associations	It's not possible to define a draft-enabled target-resolved association in an extension of a released SAP BO (including the required reverse association in the target BO) to establish a transactionally-enabled cross-BO relationship to a released SAP BO or a custom BO. This modeling results in an error on service layer.
Business Object	Unmanaged Business Objects with Draft	Operations	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances: On draft instances, the following operations are not supported: Create-by-associations via intra-BO associations that are create-enabled but no compositions

Development area	Subarea	Affected feature/capability	Restriction
			• Direct creates on child instances
Behavior, Draft & Runtime	Draft	Operations	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances: On draft instances, the following operations are not supported: • Create-by-associations via intra-BO associations that are create-enabled but no compositions • Direct creates on child instances
Business Object	Unmanaged Business Objects with Draft	Draft Admin fields	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances: • The aggregated admin data fields aren't updated automatically when a draft instance is saved.
Behavior, Draft & Runtime	Draft	Draft Admin fields	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances: • The aggregated admin data fields aren't updated automatically when a draft instance is saved.
Business Object	Unmanaged Business Objects with Draft	Associations	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances. On draft instances, the following operations are not supported: • Associations that use NOT in the binding condition can't be draft-enabled. • There's no support for transactional draft-enabled associations leading from draft to active instances.
Behavior, Draft & Runtime	Draft	Associations	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances. On draft instances, the following operations are not supported: • Associations that use NOT in the binding condition can't be draft-enabled. • There's no support for transactional draft-enabled associations leading from draft to active instances.

Development area	Subarea	Affected feature/capability	Restriction
Business Object	Unmanaged Business Objects with Draft	Client handling	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances: On draft instances, client-independent CDS views aren't supported.
Behavior, Draft & Runtime	Draft	Client handling	Some features can be defined in the behavior definition of an unmanaged BO and used on active instances, but not on draft instances: On draft instances, client-independent CDS views aren't supported.
Business Object	Unmanaged Business Objects with Draft	Strict(2):	It is best practice to use <code>strict(2)</code> for all RAP BOs, except for: Business Configuration (BC) BOs - BOs that consume a BC API or one of the following: - CL_EXCHANGE_RATES - CL_NUMBERRANGE_INTERVALS - CL_UOM_DIM_MAINTENANCE - CL_UOM_MAINTENANCE XCO I18N APIs XCO Transport APIs
Behavior, Draft & Runtime	Draft	Draft Scope	Associations with a to-one-cardinality can't be handled by the SAP Fiori elements UI, while the backend fully supports it.
UI	Backend driven UI features	Event-Driven Side Effects	If both draft and non-draft entities are exposed in the same service binding of a BO, it isn't possible to run an event-driven side effect.
UI	Backend driven UI features	Event-Driven Side Effects	Newly created items on a list report are not reloaded when an event-driven side effect is executed that targets the complete list. Only already existing items on a list report can be updated once an event-driven side effect is executed.
UI	Backend driven UI features	Editable Treeview with Draft	For unmanaged entities with late numbering, the declaration with <code>cascading delete</code> is not supported.
UI	Backend driven UI features	Editable Treeview with Draft	Even though the cascading delete behavior is optional, the UI expects that the server cascades the deletion for subtrees and it doesn't behave consistently without the cascade as it doesn't send requests for multi-selected subtree nodes.

Development area	Subarea	Affected feature/capability	Restriction
UI	Backend driven UI features	Editable Treeview with Draft	The following hierarchy features are not supported: • Hierarchy with multiple parents • Temporal hierarchy • Heterogenous hierarchies (e.g., hierarchies with different node types that lead to different object pages in the UI)
UI	Backend driven UI features	Editable Treeview with Draft	The hierarchy-to-parent association must be a pure foreign key relation. This means, this association must precisely bind the hierarchy-parent primary key. For example, binding a non-primary semantic key is not supported.
UI	Backend driven UI features	Editable Treeview with Draft	Draft query view is not supported.
UI	Backend driven UI features	RAP Recommendations	If recommended values have assigned descriptions (e.g. country codes), the UI also displays the description, only if the recommendation target field is the only key field of the value help. If the value help uses additional binding (see Value Help with Additional Binding), there may be cases where the bound field is also relevant for RAP recommendations. In such cases, the UI doesn't display a description for the bound field as part of the RAP recommendation.
UI	Backend driven UI features	RAP Recommendations	RAP recommendations are not supported by RAP extensibility. Recommended values can't be assigned to extension fields, and extension fields can't be used as triggers for calculating recommended values.
Extensibility and Release Contracts	Extensibility	RAP Recommendations	RAP recommendations are not supported by RAP extensibility. Recommended values can't be assigned to extension fields, and extension fields can't be used as triggers for calculating recommended values.
UI	Backend driven UI features	Value help	In case a value help is added to an action parameter (e.g. via annotation <code>@ValueHelpDefinition</code> in corresponding abstract entity for the action parameters) the value help isn't added automatically to the service and must be added to the service definition explicitly.
Service Exposure & Protocols	OData V2	Optional action/function parameters in OData V2	In OData V2, the Gateway DPC interface doesn't indicate whether optional parameters were provided. As a result: • Requests with missing mandatory action or function parameters are not rejected.

Development area	Subarea	Affected feature/capability	Restriction
			Non-provided parameters are passed to the behavior implementation with initial values. To expose action extensions in the UI, at least one field must exist in the CDS view extension of the projection layer that UI annotations for the action extension can reference.
Behavior, Draft & Runtime	Actions, function & parameters	Optional action/function parameters in OData V2	In OData V2, the Gateway DPC interface doesn't indicate whether optional parameters were provided. As a result: - Requests with missing mandatory action or function parameters are not rejected. - Non-provided parameters are passed to the behavior implementation with initial values. To expose action extensions in the UI, at least one field must exist in the CDS view extension of the projection layer that UI annotations for the action extension can reference.
Service Exposure & Protocols	OData V2/ OData V4	Invalidating an OData Service	When changing (deep) action parameters or non-entity action results with an OData service (V2 and V4), the result is not invalidated correctly. These changes are not considered when calculating the last-changed-timestamp. As a workaround, activate the BDEF or service definition.
Behavior, Draft & Runtime	Actions, function & parameters	Invalidating an OData Service	When changing (deep) action parameters or non-entity action results with an OData service (V2 and V4), the result is not invalidated correctly. These changes are not considered when calculating the last-changed-timestamp. As a workaround, activate the BDEF or service definition.
Service Exposure & Protocols	OData V4	Large Objects	The RAP Runtime Engine rejects OData V4 requests having an \$expand clause where a \$select clause is stated for the expanded entity that contains the property explicitly. If the expanded entity contains a large object property, then such requests are rejected even if no \$select clause is specified for the expanded entity.
Behavior, Draft & Runtime	Runtime	Large Objects	The RAP Runtime Engine rejects OData V4 requests having an \$expand clause where a \$select clause is stated for the expanded entity that contains the property explicitly. If the expanded entity contains a large object property, then such requests are rejected even if no \$select clause is specified for the expanded entity.

Development area	Subarea	Affected feature/capability	Restriction
Service Exposure & Protocols	OData V4	OData V4 Referential Constraints	To meet OData regulations, referential constraints are not generated for key fields in draft enabled BOs used for OData V4 services. As a result, Quick Views aren't available for these fields.
Business Object	Unmanaged Business Objects with Draft	OData V4 Referential Constraints	To meet OData regulations, referential constraints are not generated for key fields in draft enabled BOs used for OData V4 services. As a result, Quick Views aren't available for these fields.
Service Exposure & Protocols	OData Client Proxy	OData Client Proxy Requests	OData Request: Delta Link Query Option can't be used with the strict mode in RAP. This kind of request doesn't fulfill the rules for transactional consistency.
Service Exposure & Protocols	OData Client Proxy	Using a Consumption Model with two Namespaces	The OData Client Proxy doesn't support two namespaces.
Service Exposure & Protocols	OData Client Proxy	OData Client Proxy	Only the Local Client Proxy can be used.
BO Consumption	OData Client Proxy	OData Client Proxy	Only the Local Client Proxy can be used.
Service Exposure & Protocols	OData V2	OData V2 remote consumption can't handle NULL values and omit handling	For remote OData V2 services that are consumed, customers using the OData Client won't be able to obtain information if the response from the server contains NULL values or if a property has been omitted from the payload. Depending on the service, this information may be relevant to the client.
Service Exposure & Protocols	Exposure	Deep Parameters in Non-Standard Operations	The following types are not supported for OData exposure: • scalar types • deep table parameters • deep parameters as input or output parameters of functions. If such parameters are used, the service binding reports an error. Operations with these parameters can still be consumed via EML.
BO Consumption	EML	Deep Parameters in Non-Standard Operations	To meet OData regulations, referential constraints are not generated for key fields in draft enabled BOs used for OData V4 services. As a result, Quick Views aren't available for these fields.
Behavior, Draft & Runtime	Actions, function & parameters	Deep Parameters in Non-Standard Operations	To meet OData regulations, referential constraints are not generated for key fields in draft enabled BOs used for OData V4 services. As a result, Quick Views aren't available for these fields.

Development area	Subarea	Affected feature/capability	Restriction
Service Exposure & Protocols	OData V2	Deep Parameters in Non-Standard Operations	OData V2 only allows non-standard operations with flat parameters. The service binding shows an error message if a non-standard operation with a deep parameter is included in the service.
Behavior, Draft & Runtime	Actions, function & parameters	Deep Parameters in Non-Standard Operations	OData V2 only allows non-standard operations with flat parameters. The service binding shows an error message if a non-standard operation with a deep parameter is included in the service.
Service Exposure & Protocols	Exposure	Deep Parameters in Non-Standard Operations	OData V2 only allows non-standard operations with flat parameters. The service binding shows an error message if a non-standard operation with a deep parameter is included in the service.
Service Exposure & Protocols	Exposure	Hierarchy Exposure in Services	Hierarchies can't be exposed directly in a service definition. If required, the hierarchy must be exposed via a projection view.
Service Exposure & Protocols	Exposure	CDS Source Cardinality	For the OData exposure in RAP, the defined CDS source cardinality is ignored and always treated as "1".
Service Exposure & Protocols	OData V4	Support for a Heterogeneous Request in Complex Collection During CREATE (OData V4)	This feature conflicts with application-specific value control revealed via a dedicated annotation. If at least one property in the service has such an explicit value control, the feature is disabled for the service entirely.
Service Exposure & Protocols	Projections	Different Projections	If multiple projections expose the same RAP BO, it must not be modified via different projections in the same LUW.
Business Object	RAP BO	Different Projections	If multiple projections expose the same RAP BO, it must not be modified via different projections in the same LUW.
Service Exposure & Protocols	Path expression	Unmanaged Queries in Sort or Filter Requests	An unmanaged query can't process path expressions using to-one associations with a foreign target in filter or sort requests.
BO Consumption	EML	CREATE and EDIT in Same ChangeSet	It isn't allowed to send CREATE and EDIT in the same ChangeSet (EML: Transaction). The managed draft provider should reject such requests, but it doesn't.
Extensibility and Release Contracts	Extensibility	Virtual Elements	Virtual elements can't be added to extensions on the projection layer.
Extensibility and Release Contracts	Extensibility	RAP Analytical Table	CDS entities annotated for RAP analytical tables consumption aren't enabled for extensibility.
Extensibility and Release Contracts	Extensibility	Behavior Extensibility	Extensions for determinations, validations, additional save and cleanup are only supported for the implementation type managed.

Development area	Subarea	Affected feature/capability	Restriction
Extensibility and Release Contracts	Extensibility	Action extensions	To expose action extensions on the UI, any field in the CDS view extend of the projection layer is required, which the UI annotations for the action extensions can refer to.
UI	Action extensions	Behavior Extensibility	Extensions for determinations, validations, additional save and cleanup are only supported for the implementation type managed.
Extensibility and Release Contracts	Extensibility	Augmentation	It's not possible to use augmentation in an extensibility scenario.
Extensibility and Release Contracts	Extensibility	MAP_MESSAGES	MAP_MESSAGES can't be implemented in extensions.
Extensibility and Release Contracts	Release Contract	Extend release contract	A RAP BO interface or CDS projection can't be draft-enable after they have been released for the Extend (C0) release contract.
Extensibility and Release Contracts	Release Contract	BDEF C1 Release	C1 release of BDEFs is not supported for: - Managed and unmanaged BDEFs (only projections, interfaces and abstract) - BDEFs with language version 5 (customer objects), but allowed for interfaces and abstract - BDEFs where entities contain virtual or path elements - BDEFs with draft scope
Business Object	RAP BO	BDEF C1 Release	C1 release of BDEFs is not supported for: - Managed and unmanaged BDEFs (only projections, interfaces and abstract) - BDEFs with language version 5 (customer objects), but allowed for interfaces and abstract - BDEFs where entities contain virtual or path elements - BDEFs with draft scope
Testability	Testability	RAP Testability	Testability isn't supported for the draft related draft-admin data, durable locks, and messages in draft-enabled BOs.
Testability	Testability	RAP Testability	Testability isn't supported for the draft related draft-admin data, durable locks, and messages in draft-enabled BOs.

Development area	Subarea	Affected feature/capability	Restriction
Testability	Testability	Event Test Double Framework	Events which are raised in the update task aren't considered by the Event Test Double Framework.

Develop Applications

Here you can find end-to-end scenarios for developing RAP applications.

[Developing Read-Only List Reporting Apps](#)

Based on existing persistent data sources, you create and implement a query for an OData service to get a running app with useful read-only features.

[Developing Read-Only RAP Analytical Tables](#)

Learn how to develop a RAP analytical table in a read-only business object.

[Developing Managed Transactional Apps](#)

In this chapter, you will be guided through all steps necessary to develop a travel administration app based on the business object's managed runtime infrastructure.

[Developing Unmanaged Transactional Apps](#)

This section explains the main development tasks required for enabling transactional processing in a business objects provider that integrates existing business logic.

[Developing Transactional Apps with Exclusive Draft Capabilities](#)

This chapter is a walk-through tutorial that guides you through the main implementation steps to develop an application with exclusive draft support. Draft-enabled applications provide the end user with the highest flexibility when working with the app.

[Developing Transactional Apps with Collaborative Draft Capabilities](#)

With the collaborative draft capability, you can invite users to edit the same draft instance simultaneously.

[Developing Cross-BO Scenarios with Draft Scope](#)

[Developing Apps with Hierarchical Data Structures](#)

The following topics explain how to use CDS hierarchies in the context of the ABAP RESTful Application Programming Model (RAP), and expose the hierarchical structures as treeviews on the SAP Fiori UI.

[Developing Transactional Apps with Multi-Inline-Edit Capabilities](#)

The following section guides you through the implementation steps to create an application with multi-inline-edit capabilities.

[Developing a UI Service with Access to a Remote Service](#)

Based on a remote OData service, you create a new service that enhances the remote service with additional information.

Developing Read-Only List Reporting Apps

Based on existing persistent data sources, you create and implement a query for an OData service to get a running app with useful read-only features.

Introduction

In this chapter, you learn how to develop an OData service including multiple read-only features. This OData service can be consumed by a Fiori Elements application or by any other OData client.

Starting from the elementary list reporting scenario that was introduced in the [Getting Started with RAP](#) section, you may want to add some further features to the existing elementary OData service. First of all, the user wants to be able to navigate to a second information layer for a flight connection to retrieve more detailed information about the flight, such as flight dates or plane types.

Secondly, if the list report contains a large number of rows, it becomes difficult for users to find the information they need. To make it easier to find this information, you can implement search capabilities or label the app elements differently than their presets in the data model layer in the back end. You might also want to enable value helps for selection field dialogs. All these features are implemented using specific CDS annotations, which you, as the developer, add to the source code of the respective CDS view.

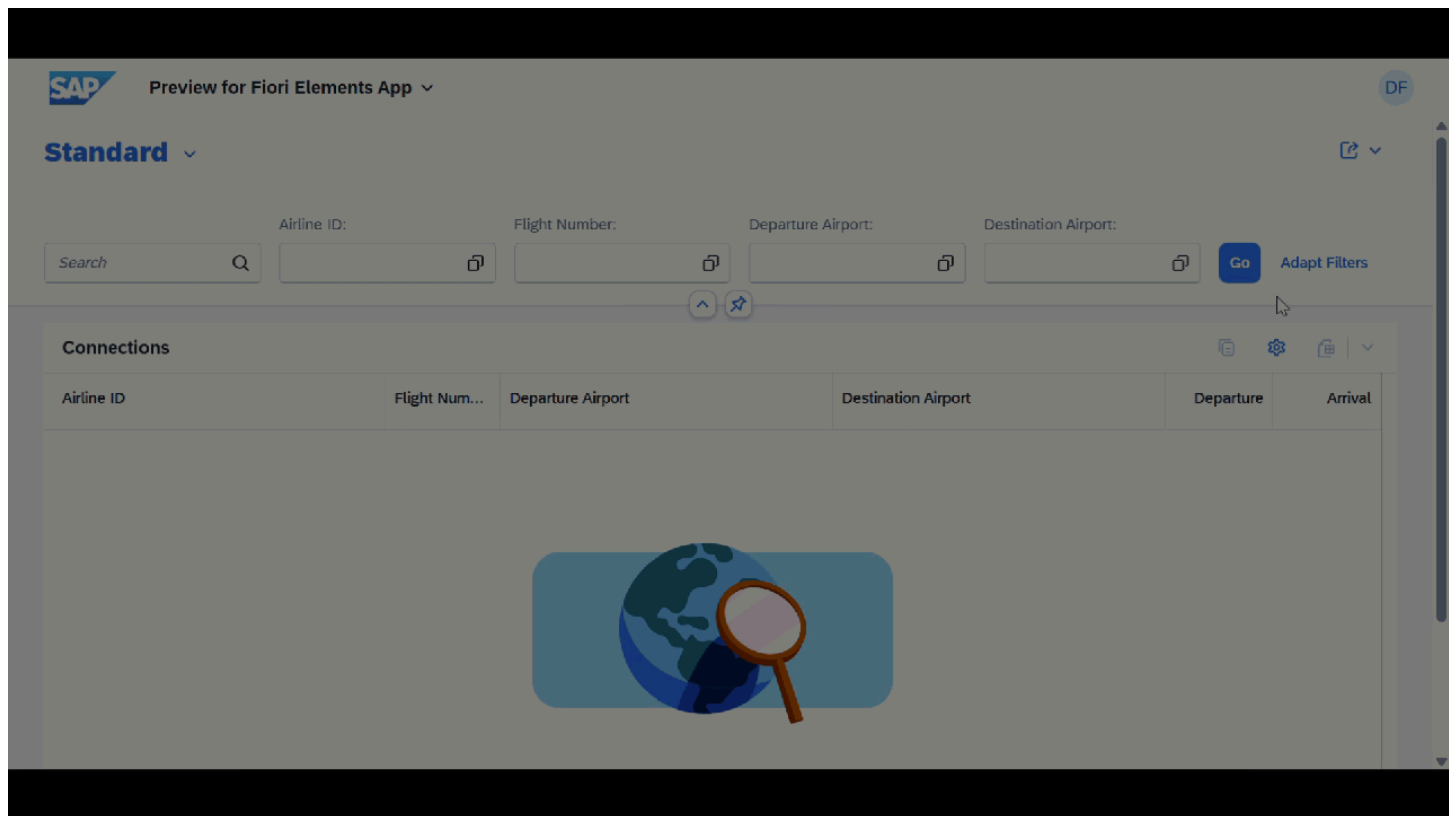
You're guided step by step through the application model and expand the OData service that you created in the [Getting Started](#) section with the following features and query capabilities.

Features

- Define the data model for the OData service
- Add value helps to the data model
- Use text associations to display texts for elements like ids
- Add search capabilities to the data model

App

Here you can see a preview of the app you're going to develop:



Defining the Data Model for the Read-Only Scenario

Starting Point

We build the list reporting scenario based on the service that was created in the [Getting Started with RAP](#) scenario. The CDS view `/DMO/I_Connection_R` is reused in the following scenario and the basis for the data model.

In the first step of this development guide, you expand the data model with additional CDS views to create a read only app for flights. The resulting running app is able to find flights based on connections and the airports involved. From a technical point of view, this means that the CDS data model is expanded with three other CDS views, one of which is connected using associations to enable navigation from one CDS view to another.

For background information about CDS view entities and associations, see [Associations](#).

1. [Defining CDS Views](#)

To provide a data model for a read-only OData service, you need to define the CDS views first.

2. [Adding Metadata Extensions](#)

Metadata extensions, short MDEs, are CDS objects that allow users to add annotations to a CDS entity outside its data definition.

3. [Implementing Associations to Enable Navigation](#)

Associations structure the relationships between CDS views. To enable navigation in the UI, associations must be implemented in CDS.

Defining CDS Views

To provide a data model for a read-only OData service, you need to define the CDS views first.

Context

Expand the data model for the flight scenario. We assume that the CDS view `/DMO/I_Connection_R` already exists. This CDS view defines the starting point of the app. The app is developed with two layers of CDS views:

- **Basic interface views** are placed directly on top of the database tables and are the only views that access the database tables directly. Basic interface views have the following purposes:
 - The table fields in the database layer have names that are hard to understand. In the basic interface view layer, they're renamed in terms of business semantics, resulting in more comprehensible field names.
 - Basic interface views enrich the data model derived from the underlying database tables with additional metadata, such as annotations and relations between different views.
- **Consumption views:** They're based on reuse views and access the database tables only indirectly through the basic interface views. These views are designed for a particular purpose with specific requirements, such as use by a particular transactional or analytical app.

For a detailed description of the following steps, look at [Defining the Data Model with CDS](#).

Procedure

1. For each new CDS view, do the following:
 - a. Create a data definition using the creation wizard in your development package.
 - b. Implement the CDS view and define the name of the CDS view as stated in the following table.

Data Definition	Data Source	Description
CDS View Name		
/DMO/I_Flight_R	/dmo/flight (DB table)	Provides information about the available flights including flight dates and plane information.

Data Definition CDS View Name	Data Source	Description
		You can navigate to this view once you've chosen a flight connection.
/DM0/C_Flight_R	/DM0/I_Flight_R	Provides the consumption view for the flight data.
/DM0/C_Connection_R	/DM0/I_Connection_R	Provides the consumption view for the connection data.

2. For each new CDS view, do the following:

- a. Insert all elements provided from the data source.
- b. Add a meaningful alias to each CDS element.

3. Add the relevant `@Semantics` annotation where necessary.

The price element in the CDS view `/DM0/I_FLIGHT_R` must be annotated with `@Semantics.amount.currencyCode: 'CurrencyCode'`. This annotation establishes the link between the amount and the currency element.

More information: [Relating Semantically Dependent Elements](#).

4. Activate the CDS views.

You've created the CDS views that read the data from the database tables.

[CDS Views for Connection and Flight](#)

Task overview: [Defining the Data Model for the Read-Only Scenario](#)

Next task: [Adding Metadata Extensions](#)

CDS Views for Connection and Flight

The following list shows the source code of the CDS views for the read-only scenario.

i Note

This is a simplified scenario that doesn't define DCLs for the CDS views. In general it's recommended to implement a suitable identity and access concept for your use case.

CDS View for Connection

Connection CDS view `/DM0/I_Connection_R`

This CDS view is the entry point of the flight reference app. It provides the elements for the connection search that the user can use to find suitable flights for a journey.

This CDS view `/DM0/I_Connection_R` was already modeled in the introductory guide in the getting started section.

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Connection View - CDS Data Model'

define view entity /DM0/I_Connection_R
as select from /dmo/connection as Connection
```

```

{
  key   Connection.carrier_id      as AirlineID,
  key   Connection.connection_id   as ConnectionID,

  Connection.airport_from_id as DepartureAirport,

  Connection.airport_to_id   as DestinationAirport,

  Connection.departure_time  as DepartureTime,
  Connection.arrival_time    as ArrivalTime,

  @Semantics.quantity.unitOfMeasure: 'DistanceUnit'
  Connection.distance        as Distance,
  Connection.distance_unit   as DistanceUnit
}

```

Connection CDS view /DMO/C_Connection_R

```

@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Connection Consumption View'

```

```

define view entity /DMO/C_Connection_R
  as select from /DMO/I_Connection_R

```

```

{
  key AirlineID,
  key ConnectionID,
  DepartureAirport,
  DestinationAirport,
  DepartureTime,
  ArrivalTime,
  Distance,
  DistanceUnit
}

```

CDS Views for Flight

Flight CDS view /DMO/I_Flight_R

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Flight View - CDS Data Model'

```

```

define view entity /DMO/I_Flight_R
  as select from /dmo/flight as Flight

```

```

{
  key Flight.carrier_id      as AirlineID,
  key Flight.connection_id   as ConnectionID,
  key Flight.flight_date     as FlightDate,

  @Semantics.amount.currencyCode: 'CurrencyCode'
  Flight.price               as Price,
  Flight.currency_code       as CurrencyCode,
  Flight.plane_type_id       as PlaneType,
  Flight.seats_max            as MaximumSeats,
  Flight.seats_occupied       as OccupiedSeats
}

```

Flight CDS view /DMO/C_Flight_R

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Flight Consumption View'
```

```
define view entity /DMO/C_Flight_R
as select from /DMO/I_Flight_R
{
  key AirlineID,
  key ConnectionID,
  key FlightDate,

  @Semantics.amount.currencyCode: 'CurrencyCode'
  Price,
  CurrencyCode,
  PlaneType,
  MaximumSeats,
  OccupiedSeats,
  OccupiedSeats as OccupiedSeatsForChart
}
```

Adding Metadata Extensions

Metadata extensions, short MDEs, are CDS objects that allow users to add annotations to a CDS entity outside its data definition.

Context

To integrate the UI metadata closely with the data model, the UI-specific metadata is added directly to the data model using UI annotations for RAP-based apps.

SAP Fiori elements consume the metadata accordingly, and create standardized UIs out of the box, based on the newest floor plans and designs for apps. This enables you to implement full-stack UI services without having to implement any additional front-end code, unless required by your use case. If needed, you can implement custom UI additions with tools like SAP Business Application Studio and SAP Fiori Tools .

UI information is always specified on the consumption layer of the CDS view stack.

To display the data from the data model on the UI, you have to add metadata extensions for the consume and the corresponding UI annotations.

Procedure

1. Add `@Metadata.allowExtensions: true` to the header of `/DMO/C_Flight_R` and `/DMO/C_Flight_R`.

This annotation allows you to create a metadata extension for the related CDS views

2. Create the metadata extensions `/DMO/C_Flight_R` and `/DMO/C_Connection_R` as described in [Creating CDS Metadata Extensions](#).

3. Add the following UI annotations to `/DMO/C_Connection_R`:

The connection information is displayed as list page in SAP Fiori. The `UI.header` annotations define the top-most area of a page or application. It provides users with key information, filter options, or search options. The header often includes elements like the page title, or application logo. For the connection, the annotation `UI.headerInfo` defines the displayed names and the `presentationVariant` annotation defines the layout- and display-relevant information between a navigation source and a navigation target in SAP Fiori apps, in this case the detail information for each flight.

`UI.facets` define the details for object page layout (navigation target). The `UI.lineItem` annotation defines the details of the displayed fields in the list report, like the order in which they're displayed. For more details, see [Using Facets to change the Object Page Layout](#) and [Fields on the UI](#).

Code Syntax

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Connection',
                    typeNamePlural: 'Connections',
                    title: { type: #STANDARD, value: 'ConnectionTitle' }},
      presentationVariant: [{ visualizations: [{type: #AS_LINEITEM}] }]}

annotate entity /DMO/C_Connection_R with
{
  @UI.facet: [
    {
      id: 'Connection',
      purpose: #STANDARD,
      type: #COLLECTION,
      label: 'Connection',
      position: 10
    },
    {
      id: 'General',
      targetQualifier: 'General_FG',
      parentId: 'Connection',
      purpose: #STANDARD,
      type: #FIELDGROUP_REFERENCE,
      label: 'General',
      position: 10
    },
    {
      id: 'Departure',
      targetQualifier: 'Departure_FG',
      parentId: 'Connection',
      purpose: #STANDARD,
      type: #FIELDGROUP_REFERENCE,
      label: 'Departure',
      position: 20
    },
    {
      id: 'Destination',
      targetQualifier: 'Destination_FG',
      parentId: 'Connection',
      purpose: #STANDARD,
      type: #FIELDGROUP_REFERENCE,
      label: 'Destination',
      position: 30
    },
    {
      id: 'Flight',
      type: #LINEITEM_REFERENCE,
      label: 'Flight',
      position: 20,
      targetElement: '_Flight'
    }
  ]
}

@UI: { lineItem: [{ position: 10 }],
      selectionField: [{ position: 10 }],
      fieldGroup:[{ position: 10, qualifier: 'General_FG' }]}
@EndUserText.quickInfo: 'Airline that operates the flight'
AirlineID;

@UI: { lineItem: [{ position: 20 }],
      selectionField: [{ position: 20 }],
      fieldGroup:[{ position: 20, qualifier: 'General_FG' }]}
ConnectionID;

@UI.fieldGroup: [{ position: 30, qualifier: 'General_FG' }]
Distance;

@UI.fieldGroup: [{ position: 40, qualifier: 'General_FG', label: 'Distance Unit' }]
DistanceUnit;

@UI: { lineItem: [{ position: 30 }],
      selectionField: [{ position: 30 }],

```

```

        fieldGroup:[{ position: 10, qualifier: 'Departure_FG' }]}}
@EndUserText.label: 'Departure Airport'
DepartureAirport;

@UI.lineItem: [{ position: 50 }]
@UI.fieldGroup: [{ position: 20, qualifier: 'Departure_FG' }]
DepartureTime;

@UI: { lineItem: [{ position: 40 }],
        selectionField: [{ position: 40 }],
        fieldGroup: [{ position: 10, qualifier: 'Destination_FG' }]}}
@EndUserText.label: 'Destination Airport'
DestinationAirport;

@UI.lineItem: [{ position: 60 }]
@UI.fieldGroup: [{ position: 20, qualifier: 'Destination_FG' }]
ArrivalTime;
}

```

4. Add the following UI annotation to /DMO/C_FLIGHT_R:

In addition to UI.header and UI.facet, this metadata extension also defines a chart that displays the occupied seats as radial chart. For more information, see [Data Points](#)

Code Syntax

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Flight',
                    typeNamePlural: 'Flights',
                    title: { type: #STANDARD, value: 'FlightDate' }},
        presentationVariant: [{ visualizations: [{type: #AS_LINEITEM}]}]}

@UI.chart: [
{
    qualifier: 'radialChart',
    title: 'Occupation Rate',
    description: 'Occupied seats in relation to maximum number of seats',
    chartType: #DONUT,
    measures: ['OccupiedSeatsForChart'],
    measureAttributes: [
        {
            measure: 'OccupiedSeatsForChart',
            role: #AXIS_1,
            asDataPoint: true
        }
    ]
}
]
}

]

annotate entity /DMO/C_Flight_R with
{
    @UI.facet: [
        {
            id: 'Flight',
            purpose: #STANDARD,
            type: #COLLECTION,
            label: 'Flight',
            position: 10
        },
        {
            id: 'General',
            targetQualifier: 'General_FG',
            parentId: 'Flight',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'General',
            position: 10
        },
        {

```

```

        id: 'Seats',
        targetQualifier: 'Seats_FG',
        parentId: 'Flight',
        purpose: #STANDARD,
        type: #FIELDGROUP_REFERENCE,
        label: 'Seats',
        position: 20
    },
    {
        purpose: #HEADER,
        type: #COLLECTION,
        id: 'CollectionFacet'
    },
    {
        parentId: 'CollectionFacet',
        type: #CHART_REFERENCE,
        targetQualifier: 'radialChart'
    }
]

@UI.fieldGroup: [{ position: 10, qualifier: 'General_FG' }]
AirlineID;

@UI.fieldGroup: [{ position: 20, qualifier: 'General_FG' }]
ConnectionID;

@UI.lineItem: [{ position: 10 }]
@UI.fieldGroup: [{ position: 30, qualifier: 'General_FG' }]
FlightDate;

@UI.lineItem: [{ position: 20 }]
@UI.fieldGroup: [{ position: 40, qualifier: 'General_FG' }]
Price;

@UI.lineItem: [{ position: 30 }]
@UI.fieldGroup: [{ position: 50, qualifier: 'General_FG' }]
PlaneType;

@UI.lineItem: [{ position: 40 }]
@UI.fieldGroup: [{ position: 10, qualifier: 'Seats_FG' }]
MaximumSeats;

@UI.lineItem: [{ position: 50 }]
@UI.fieldGroup: [{ position: 20, qualifier: 'Seats_FG' }]
OccupiedSeats;

@UI: {
    dataPoint: {
        qualifier: 'OccupiedSeatsForChart',
        targetValueElement: 'MaximumSeats'
    },
    lineItem: [
        {
            type: #AS_CHART,
            label: 'Occupation Rate',
            valueQualifier: 'radialChart',
            position: 60
        }
    ]
}
OccupiedSeatsForChart;
}

```

Task overview: [Defining the Data Model for the Read-Only Scenario](#)

Previous task: [Defining CDS Views](#)

Next task: [Implementing Associations to Enable Navigation](#)

Related Information

This is custom documentation. For more information, please visit [SAP Help Portal](#).

Implementing Associations to Enable Navigation

Associations structure the relationships between CDS views. To enable navigation in the UI, associations must be implemented in CDS.

Context

You created the flight CDS view that provides detailed information about the available flights for the connection. To be able to navigate to this view in the UI, you have to implement an association from `/DMO/I_Connection_R` to `/DMO/I_Flight_R`.

An association defines a structural and unidirectional relationship between two CDS views. Associations are used to navigate from a source CDS view to a related target CDS view.

Associations are defined in the source CDS view. The connections between the two CDS views are established by a join condition applied to the respective elements of the source and target view. Define a cardinality for the target CDS view to define how many records of the target CDS view are associated with one record of the source.

Procedure

1. Open the source CDS view `/DMO/I_Connection_R`.
2. Define the association after the select statement.
 - a. Introduce the association with the keyword *association*.
 - b. Define `[min .. max]` for the cardinality of the target view.

i Note

The cardinality defines the minimum and maximum number of associated entries of the target view.

In our example, the cardinality is one-to-many, since one entry in `/DMO/I_Connection_R` is associated to many entries of `/DMO/I_Flight_R`. This complies with business logic since one connection can be operated by flights on different dates. The alias after the `as` keyword defines the name with which the association is addressed in the CDS view. The `on` condition defines the mapping condition for the join operation. In this case, the `on` condition is defined for the `AirlineID` and `ConnectionID` key fields.

```
...
define view entity /DMO/I_Connection_R as select from /dmo/connection as Connection
  association [1..*] to /DMO/I_Flight_R as _Flight
    on $projection.AirlineID = _Fl:
    and $projection.ConnectionID = _Fl:
```

- c. Specify the mapping condition for the CDS views.

In our example, the associated CDS views are mapped to two elements. Therefore, both of them must be stated in the condition expression.

```
define view entity /DMO/I_Connection_R as select from /dmo/connection
  association [1..*] to /DMO/I_Flight_R as _Flight
    on $projection.AirlineID = _Fl:
    and $projection.ConnectionID = _Fl:
```

3. Add the association also to `/DMO/C_Connection_R` to define the relationship to `/DMO/C_Flight_R`. The cardinality remains unchanged.

In our example, the associated CDS views are mapped to two elements. Therefore, both of them must be stated in the condition expression. The fields are exposed in a facet, so the association target is the stable consumption view.

```
define view entity /DMO/C_Connection_R
as select from /DMO/I_Connection_R
```

```
association [1..*] to /DMO/C_Flight_R as _Flight on $projection.AirlineID = _Flight.Air
and $projection.ConnectionID = _Flight.Con
```

4. For the UI: The `UI.facet` annotation in the metadata extension defines the layout of the object page. The associations on interface and consumption level enable the navigation from the connection overview list page to flight details object page.

A detailed description of UI annotations can be found in [Defining UI Annotations](#).

```
@UI.facet: [
{ id:      'Connection',
purpose:   #STANDARD,
type:      #IDENTIFICATION_REFERENCE,
label:     'Connection',
position:  10 },
{ id:      'Flight',
purpose:   #STANDARD,
type:      #LINEITEM_REFERENCE,
label:     'Flight',
position:  20,
targetElement: '_Flight' }
]
```

5. Activate the CDS view.

Results

You've established a connection between the connection CDS view and the flight CDS view. With the right configuration of UI annotations, the user can now navigate to the flight information by selecting one connection in the list report app. The flight information is displayed as line items in the object page as can be seen in the following figures.

On the UI, user can click on one connection entry to navigate to the information for flights.

Standard*

Departure Airport:

Destination Airport:

Go

^

☆

Connections (20)

Airline ID	Flight Number	Departure Airport	Destination Airport	Departure	Arrival	
AA	15	JFK	SFO	7:13:00 AM	10:04:00 AM	>
AA	17	MIA	HAV	7:19:00 AM	8:03:00 AM	>
AA	18	SFO	JFK	6:40:00 AM	3:06:00 PM	>
AA	322	MIA	EWR	8:17:00 PM	11:19:00 PM	>
AA	2678	HAV	MIA	6:15:00 AM	10:30:00 AM	>
AZ	788	VCE	NRT	1:25:00 PM	10:13:00 AM	>
AZ	789	NRT	VCE	2:26:00 PM	9:31:00 PM	>

Selection of the connection **AZ 789 from NRT to VCE**

The object page then displays the information about the connection and the related flights for this connection in the second facet.

AA-15

Connection Flight

Connection

General

Airline ID:
American Airlines Inc. (AA)

Flight Distance:
4,156

Flight Number:
15

Distance Unit:
KM

Departure

Departure Airport:
John F. Kennedy International Airport (JFK)

Departure:
7:13:00 AM

Destination

Destination Airport:
San Francisco International Airport (SFO)

Arrival:
10:04:00 AM

Flight (2)

Flight Date	Flight Price	Plane Type	Maximum Capacity	Occupied Seats	Occupation Rate
Feb 8, 2025	3,117.00 USD	767-200	260	213	81.9%
Dec 5, 2025	1,911.00 USD	767-200	260	137	52.7%

Object page displaying all flights a connection

Task overview: [Defining the Data Model for the Read-Only Scenario](#)

Previous task: [Adding Metadata Extensions](#)

Improving the User Experience

Add different features to the OData read-only service to improve the user experience on the UI.

You've defined the data model for the read only service and the basic layout for the UI. However, there are still various features you can use to improve the user experience on the UI:

- Create consistent UI labels and hover texts
 - Create value helps to make the selection easier
 - Create text associations to improve the understandability of UI texts
 - Provide search capabilities to support out-of-the-box search for the displayed data
1. [Changing UI Field Labels and Descriptions](#)
Field labels and description help to customize the UI of the app.
 2. [Displaying Text for Unreadable Elements](#)
Use text associations to add readable texts to short forms or identifiers.
 3. [Providing Value Help for Selection Fields](#)
Use value helps to make it easier to find the correct value for selection fields on the UI.
 4. [Adding Search Capabilities](#)
Include a search input field to execute a text and fuzzy search on multiple elements.

Changing UI Field Labels and Descriptions

Field labels and description help to customize the UI of the app.

Meaningful descriptions of elements that appear on the user interface are a key concept for working with an app and improving the user experience. It's essential that all items on the UI are readable and understandable for the user. Although every data element in [ABAP Dictionary](#) is labeled with descriptions for presentation on the UI, sometimes you want to modify the description for a specific use case and give it a name other than the name predefined on the persistent database layer. In the CDS layer, database description labels can be redefined and given more information by using the annotations `@EndUserText.label`: '`<text>`' and `@EndUserText.quickInfo`: '`<text>`'.

The labels that are assigned in the CDS layer overwrite the database field descriptions and are exposed to the OData service. These labels are also propagated to the UI and displayed on the user interface if no UI labeling annotations exist that overwrite the CDS `EndUserText` annotations.

i Note

UI labeling annotations, such as `@UI.lineItem: [ { label: '' } ]`, overwrite any `@EndUserText` labeling annotations on the UI. However, they aren't manifested in the OData service metadata.

A tooltip can provide additional or more thorough information for the element. The tooltip is displayed on the UI as mouse over text. If no `@EndUserText` annotations are used, the text for the mouse over function is retrieved from the long description of the data element stored in [ABAP Dictionary](#).

In general, every text that is used in `EndUserText` annotations is translated into every relevant language by the SAP translation process, along with the labels that are given to the data elements.

For more information on how to integrate information for the user in your data model, refer to [Adding Field Labels and Descriptions](#) and [Overwriting Default Field Labels](#).

Integrating the Mouse-Over Function

If you want additional and longer information about an element, you can use the annotation `@EndUserText.quickInfo`: `<text>` to display a text when hovering over the element.

i Note

If you don't define a tooltip in CDS, the mouse over text displays the short description of the data element in [ABAP Dictionary](#).

To change the text of the mouse over, use the tooltip.

❖ Example

```
@UI: {
  lineItem: [ { position: 10, label: 'Airline' } ],
  identification:[ { position: 10, label: 'Airline' } ] }
  @EndUserText.quickInfo: 'Airline that operates the flight.'
key Connection.carrier_id      as AirlineID,
```

Connections (20)

Airline ID

Airline that operates the flight

Parent topic: [Improving the User Experience](#)

Next: [Displaying Text for Unreadable Elements](#)

Related Information

[Adding Field Labels and Descriptions](#)

Displaying Text for Unreadable Elements

Use text associations to add readable texts to short forms or identifiers.

Data that is stored in databases is kept as short as possible and consequently, many words are shortened or abbreviated with a character code. While this is convenient, it becomes a problem on the UI labels aren't understandable from the user perspective. Take the airline codes from the connections as an example: The names aren't human-readable and don't offer any information about the actual label.

By default, the airline is abbreviated with the two-letter airline code.

Airline ID	Flight Number	Departure Airport	Destination Airport	Departure	Arrival	
AA	15	JFK	SFO	7:13:00 AM	10:04:00 AM	>
AZ	789	NRT	VCE	2:26:00 PM	9:31:00 PM	>

The full name of the airlines is contained in the CDS view `/DMO/I_Carrier` based on the database `/DMO/carrier`. Using a text association, you can establish a link from the connection CDS view to the carrier CDS view and adopt the names of the airlines as an addition to the airline codes in the UI.

You need to use annotations from the `@semantics` and `@objectModel` domains to mark the names as readable text and to assign the text addition to the airline code elements.

The same is true for the departure and destination airport. Instead of the abbreviations, the full name is more useful for an user. To display the departure and destination airport name, the text view `/DMO/I_Airport` is used with different join conditions. The departure airport is defined with `on $projection.DepartureAirport = _AirportFrom.AirportID` and the destination airport is defined with `on $projection.DestinationAirport = _AirportTo.AirportID`.

For more detailed information about text elements, refer to [Working with Text Elements](#).

Parent topic: [Improving the User Experience](#)

Previous: [Changing UI Field Labels and Descriptions](#)

Next: [Providing Value Help for Selection Fields](#)

Getting Airline Names through a Text Association

Prerequisites

A text provider view already exists. In our case, the CDS view `/DMO/I_Carrier` contains the relevant text element `Name`.

Context

The readable name of the airlines can be displayed on the UI together with the two letter code. The text should be displayed on the list report page and on the object page to keep the user experience consistent. To achieve this you need to implement text associations for both views, `/DMO/I_Connection_R` and `/DMO/I_Flight_R`. Text associations are always defined on interface view level and are implicitly inherited by the consumption or projection views.

Procedure

1. Open the CDS view `/DMO/I_Carrier`. Use the annotation `@Semantics.text: true` on the element `Name` to identify the annotated element as a text and activate the view.

```
...
define view entity /DMO/I_Carrier
as select from /dmo/carrier as Airline

{
  key Airline.carrier_id      as AirlineID,

    @Semantics.text: true
    Airline.name              as Name,

    Airline.currency_code as CurrencyCode
}
```

i Note

In general, you can annotate more than one view field as a text field. However, only the first annotated field is respected in the text consumer view for OData exposure.

The `/DMO/I_Airport` is annotated in the same way with `@Semantics.text: true`:

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Airport View - CDS Data Model'

@Search.searchable: true

define view entity /DMO/I_Airport
as select from /dmo/airport as Airport

...
{
  ...

    @Semantics.text: true
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.8
    Airport.name          as Name,

    ...
}
```

2. Open the CDS view `/DMO/I_Connection_R` where you want to use the `Name`. Implement an association to the CDS view `/DMO/I_Carrier` with the join condition on `AirlineID`.
3. Use the annotation `@ObjectModel.text.association: '<_AssocToTextProvider>'` on the element `AirlineID` and reference the association `_Airline` as a text association. Then activate the CDS view.

```
...
define view entity /DMO/I_Connection_R
as select from /dmo/connection as Connection
```

```

...
association [1]    to /DMO/I_Carrier as _Airline    on $projection.AirlineID = _Airline.
{
    @ObjectModel.text.association: '_Airline'
    key    Connection.carrier_id      as AirlineID,
    key    Connection.connection_id   as ConnectionID,

    /* Associations */
    _Flight,
    _Airline,
}

```

4. Implement the association also for /DMO/I_Airport to get the departure and arrival airport names. Annotate the Connection.airport_from_id as DepartureAirport field and the Connection.airport_to_id as DestinationAirport fields with the @ObjectModel.text.association.

Source Code

```

...
define view entity /DMO/I_Connection_R
as select from /dmo/connection as Connection

...

association [1]    to /DMO/I_Carrier as _Airline    on $projection.AirlineID = _Airline
association [1]    to /DMO/I_Airport as _AirportFrom on $projection.DepartureAirport =
association [1]    to /DMO/I_Airport as _AirportTo  on $projection.DestinationAirport

{
    @ObjectModel.text.association: '_Airline'
    key    Connection.carrier_id      as AirlineID,
    key    Connection.connection_id   as ConnectionID,

    @ObjectModel.text.association: '_AirportFrom'
    Connection.airport_from_id as DepartureAirport,

    @ObjectModel.text.association: '_AirportTo'
    Connection.airport_to_id   as DestinationAirport,

    Connection.departure_time as DepartureTime,
    Connection.arrival_time  as ArrivalTime,

    ...

    /* Associations */
    _Flight,
    _Airline,
    _AirportFrom,
    _AirportTo
}

```

5. Open the CDS view /DMO/I_Flight_R for which you also want to display the text. Use the annotation @ObjectModel.text.association: '<_AssocToTextProvider>' on the element AirlineID and reference the association _Airline as a text association. Then activate the CDS view.

```

...
define view entity /DMO/I_Flight_R
as select from /dmo/flight as Flight

association [1]    to /DMO/I_Carrier_R as _Airline on $projection.AirlineID = _Airline.Air
{
    ...
    @ObjectModel.text.association: '_Airline'
    key Flight.carrier_id      as AirlineID,
    ...
    /*Association*/
    _Airline
}

```

Results

You've established a text association from the two CDS views that contain the two-letter airline code and for the departure and arrival airport names. The text is from the text provider view is displayed in the UI together with the two-letter code and the airport abbreviations. Now, the airlines are clearly identifiable by their full names.

Airline ID	Flight Num...	Departure Airport	Destination Airport	Departure	
Deutsche Lufthansa AG (LH)	403	Newark Liberty International Airport (EWR)	Frankfurt Airport (FRA)	6:09 PM	>
Singapore Airlines Limited (SQ)	1	San Francisco International Airport (SFO)	Singapore Changi Airport (SIN)	1:15 AM	>
Singapore Airlines Limited (SQ)	2	Singapore Changi Airport (SIN)	San Francisco International Airport (SFO)	6:30 AM	>
Singapore Airlines Limited (SQ)	11	Narita International Airport (NRT)	Singapore Changi Airport (SIN)	2:55 PM	>
Singapore Airlines Limited (SQ)	12	Singapore Changi Airport (SIN)	Narita International Airport (NRT)	9:53 AM	>
United Airlines, Inc. (UA)	58	San Francisco International Airport (SFO)	Frankfurt Airport (FRA)	1:45 PM	>
United Airlines, Inc. (UA)	59	Frankfurt Airport (FRA)	San Francisco International Airport (SFO)	1:55 PM	>
United Airlines, Inc. (UA)	1537	Newark Liberty International Airport (EWR)	Miami International Airport (MIA)	9:56 PM	>

Related Information

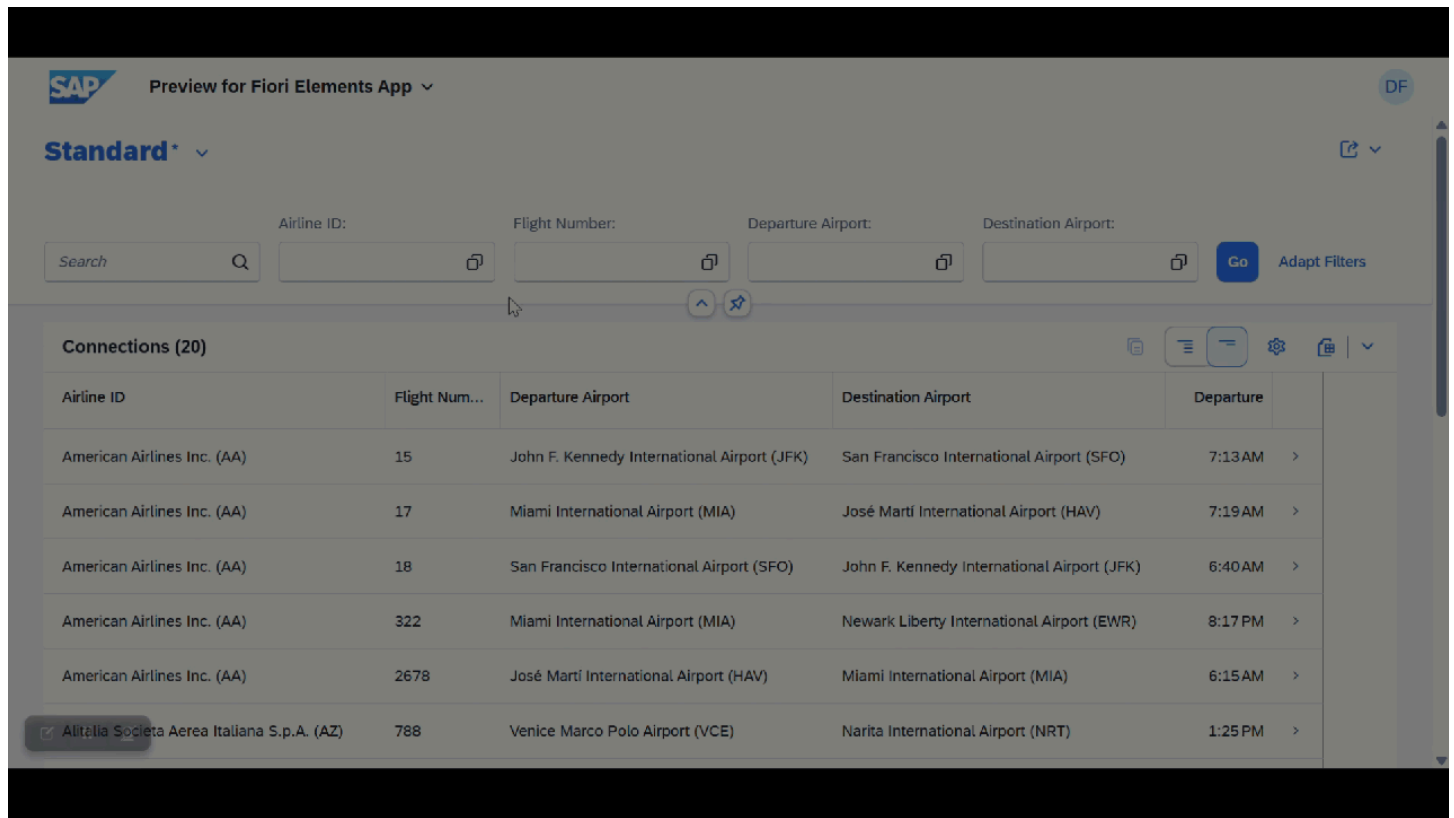
[Working with Text Elements](#)

Providing Value Help for Selection Fields

Use value helps to make it easier to find the correct value for selection fields on the UI.

You've created some selection fields for the UI to enable the user to find a suitable flight in the flight app. The selection fields for the header are defined with `UI.selectionField` like, for example, the `AirlineID`.

The difficulty is to know the correct three letter airport code to determine the correct departure or destination airport for a flight. You can enable a value help option to assist the user in finding the correct values. The user then gets direct access to a value help dialog in which they can enter values, such as city names or country/region codes, to find the suitable airport code.



Parent topic: [Improving the User Experience](#)

Previous: [Displaying Text for Unreadable Elements](#)

Next task: [Adding Search Capabilities](#)

Adding Annotations for Value Help

Prerequisites

A value help provider view already exists. In our case, the CDS view `/DM0/I_Airport` contains the relevant fields `Name`, `City` and `CountryCode` to help find the airport code.

Context

To provide a value help for the selection fields of the elements `Departure Airport Code` and `Destination Airport Code` in the CDS view `/DM0/C_Connection_R`, use the annotation `@Consumption.valueHelpDefinition`.

Procedure

1. Open the CDS view `/DM0/C_Connection_R`.
2. Annotate the elements `Departure Airport Code` and `Destination Airport Code`. You want to equip these elements with a value help dialog with the annotation `@Consumption.valueHelpDefinition: [{ entity: { name: '<target_view>', element: '<target_element>' } }]`. In our case, the target view is `/DM0/I_Airport_StdVH` with the target element `AirportID`, which works as the binding condition for both elements in the source view. The addition `useForValidation: true` marks value help that are used for validation of user input. The user input is automatically checked against the values contained in the value help.

```
{...
  @Consumption.valueHelpDefinition: [{ entity: {name: '/DM0/I_Airport_StdVH', element: 'A
  ...
  DepartureAirport,
```

```

        _AirportFrom.Name                                as DepartureAirportName,

        ...
        @Consumption.valueHelpDefinition: [{ entity: {name: '/DMO/I_Airport_StdVH', element: 'A
        DestinationAirport,
        _AirportTo.Name                                as DestinationAirportNam
    ...}

```

3. Activate the CDS view.

Results

You've implemented a value help for the selection fields that refer to the airport code elements. You can now use the value help dialog with the option to use all the elements of the value help provider view `/DMO/I_Airport_StdVH` to find the correct three letter airport code. Additionally, the selection fields are now equipped with a value completion option. This means that, once you start typing in the selection field, the possible values are displayed.

Related Information

[CDS Annotations](#)

[Adding Value Help](#)

Adding Search Capabilities

Include a search input field to execute a text and fuzzy search on multiple elements.

Context

In the previous chapter, you implemented a value help for the selection fields, which are based on airport code fields. This means you can easily search for connections from and to certain airports. However, it isn't possible to filter for connections by specific airlines or flights that are operated on a specific day. Maybe the user even wants to search for values of different elements, say that someone wants to find out which connections are available to or from Frankfurt on February 18, 2019. That's where search capabilities are required.

You use annotations to enable search capabilities for a specific CDS view and also to define the elements within the CDS that you want to be included in the search scope. You can also define fuzziness thresholds for the search, so that entries are also found if the values are entered in the search field with incorrect spelling. This makes the search easier and more effective.

The following section explains the process used to implement search capabilities:

- Connections operated by one or more certain airlines
- Connections on one or more certain days
- Connections with one or more plane types
- Or a combination of any of these.

Whereas the first two searches operate only on the CDS view `/DMO/C_Connection_R` (because the search target elements `Airline`, `Departure Airport Code` and `Destination Airport Code` are part of this view), the searches for days and plane types require the search to be operated on the associated view, since the elements `Flight Date` and `Plane Type` are part of `/DMO/C_Flight_R`. In addition, we also want to be able to search for the full airline name. That's why the text provider view `/DMO/I_Carrier` must also be search enabled.

Procedure

1. Enable the relevant elements for searches in `/DMO/C_Connection_R`

- a. Open the CDS view `/DM0/C_Connection_R`.
- b. Use the annotation `@Search.searchable: true` on the entity level to enable the CDS view for searches and to expose a standard search field on the UI.

i Note

If you use this annotation in a CDS view, you have to assign at least one default search element.

```
...
@UI.headerInfo: { typeName: 'Connection',
                  typeNamePlural: 'Connections' }
@Search.searchable: true      /*** exposes a standard search field on the UI

define view entity /DM0/C_Connection_R
as select from /DM0/I_Connection_R
...
```

- c. Choose the elements that you want to search for and annotate them with `@Search.defaultSearchElement: true`.
- d. Define a fuzziness threshold for the searchable elements with `@Search.fuzzinessThreshold: <fuzziness_value>`.

i Note

You can set a fuzziness threshold of 0–1. SAP recommends that you use 0.7 to begin with. This means that every item with a 70% match and greater is found. You can then customize the threshold.

The search elements in the connection view are `Airline`, `Departure Airport Code`, and `Destination Airport Code`. As the first one only consists of a two letter code, you don't need to define a fuzziness threshold for this element.

```
{
...
    @Search.defaultSearchElement: true
    key   ...
    Connection.carrier_id      as AirlineID,
...
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.7
    Connection.airport_from_id as DepartureAirport,
...
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.7
    Connection.airport_to_id   as DestinationAirport,
... }
```

- e. Annotate the association `_Flight` with `@Search.defaultSearchElement: true` to enable the search for elements in the associated view.

```
{...
    @Search.defaultSearchElement: true
    _Flight,
... }
```

- f. Activate the CDS view.

Now, the user can search for connections by specific airlines or to or from a specific airport. The following figure illustrates the connections operated by SQ (Singapore Airlines Limited).

2. For our use case, we not only want to search for elements in the main view of the app, but also for the fields of the associated view. Enable the relevant elements for search in `/DM0/C_Flight_R`. As previously described, open the CDS view `/DM0/C_Flight_R`, define it as `search enabled`, and then choose the elements to search on with a proper fuzziness threshold.

```
...
@Search.searchable: true
```

```

define view entity /DM0/C_Flight_R
as select from /DM0/I_Flight_R
{...
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.7
    ...
    key Flight.flight_date as FlightDate,
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.7
    Flight.plane_type_id as PlaneType,
    ... }

```

The association is now search enabled. This means it's possible to search for connections with a specific Airline and at a specific airport at the same time.

3. Enable the relevant elements for searches in `/DM0/I_Carrier` to be able to search for the full airline names and not only for the two letter code. Open the CDS view `/DM0/I_Carrier`, mark it as search enabled, and then choose the elements to search on with a proper fuzziness threshold.

```

...
@Search.searchable: true
define view entity /DM0/I_Carrier_R
as select from /dmo/carrier as Airline
{...
    @Search.defaultSearchElement: true
    @Search.fuzzinessThreshold: 0.7
    ...
    Airline.name as Name,
    ... }

```

The text association now enables the user to search for the full names of airlines.

Results

The search is now enabled for airlines and airport.

The screenshot displays the SAP Fiori Elements App interface. At the top, there's a header with the SAP logo and 'Preview for Fiori Elements App'. Below this, a search bar is visible with the text 'Standard'. The search bar includes input fields for 'Airline ID', 'Flight Number', 'Departure Airport', and 'Destination Airport', each with a magnifying glass icon. A 'Go' button and 'Adapt Filters' link are also present. Below the search bar, a table titled 'Connections (20)' is shown. The table has columns for 'Airline ID', 'Flight Num...', 'Departure Airport', 'Destination Airport', and 'Departure'. The table lists several flight connections, including American Airlines Inc. (AA) and Alitalia Società Aerea Italiana S.p.A. (AZ).

Airline ID	Flight Num...	Departure Airport	Destination Airport	Departure
American Airlines Inc. (AA)	15	John F. Kennedy International Airport (JFK)	San Francisco International Airport (SFO)	7:13 AM
American Airlines Inc. (AA)	17	Miami International Airport (MIA)	José Martí International Airport (HAV)	7:19 AM
American Airlines Inc. (AA)	18	San Francisco International Airport (SFO)	John F. Kennedy International Airport (JFK)	6:40 AM
American Airlines Inc. (AA)	322	Miami International Airport (MIA)	Newark Liberty International Airport (EWR)	8:17 PM
American Airlines Inc. (AA)	2678	José Martí International Airport (HAV)	Miami International Airport (MIA)	6:15 AM
Alitalia Società Aerea Italiana S.p.A. (AZ)	788	Venice Marco Polo Airport (VCE)	Narita International Airport (NRT)	1:25 PM

Task overview: [Improving the User Experience](#)

Previous: [Providing Value Help for Selection Fields](#)

Related Information

[Enabling Text and Fuzzy Searches in SAP Fiori Apps](#)

Creating a Service Definition and a Service Binding

Create a service definition and a service binding to display the data in the preview.

Context

The service definition is a projection of the data model that you want to expose. In a service definition, you define which CDS view entities are part of the service. This service is then exposed either as a UI service or a Web API by a service binding. A service definition can be integrated in various protocols without any reimplementation. In this case, the OData service is exposed as **UI Service - OData V4**.

To create a Service Definition, use the wizard to create the relevant development object that defines the scope of the OData service.

Procedure

1. Create a service definition /DMO/FLIGHT_R as described in [Creating a Service Definition](#).
2. Add the created CDS view entities to the service definition.

The service definition also already contains the CDS view required for [Displaying Text for Unreadable Elements](#) and [Providing Value Help for Selection Fields](#).

❖ Example

```
@EndUserText.label: 'Service Definition for Viewing Flights'
@ObjectModel.leadingEntity.name: '/DMO/C_Connection_R'
define service /DMO/FLIGHT_R {
    expose /DMO/C_Connection_R as Connection;
    expose /DMO/C_Flight_R as Flight;
    expose /DMO/I_Carrier_StdVH as Airline;
    expose /DMO/I_Airport_StdVH as Airport;
    expose I_CountryVH as Country;
}
```

3. Activate the service definition.
4. Create a service binding based on /DMO/FLIGHT_R as described in [Creating Service Binding](#)/DMO/UI_FLIGHT_R_V4 with type OData V4 UI.
5. Activate the service definition.
6. Use the preview to display the data. For details about the preview, see [Fiori Element App Preview \(FEAP\)](#).

Developing Read-Only RAP Analytical Tables

Learn how to develop a RAP analytical table in a read-only business object.

Introduction

The RAP analytical table is a type of data visualization on an SAP Fiori UI that contains a set of structured data and provides multiple possibilities for aggregating the data. It allows analytical evaluations in RAP applications. The modeling is based on a projection view annotated with `@OData.applySupportedForAggregation: #FULL`. For aggregating the data, the annotation `@Aggregation.default` is used. For more information, see:

- [@OData.applySupportedForAggregation \(ABAP Keyword Documentation\)](#)
- [@Aggregation.default \(ABAP Keyword Documentation\)](#)

For more information about RAP Analytical Tables, see [RAP Analytical Tables](#).

! Restriction

For RAP analytical tables, it is not possible to annotate calculation, filter or sort exits.

CDS entities annotated for RAP analytical tables consumption aren't enabled for extensibility.

Prerequisites

You need a read-only business object exposed via an OData V4 UI Service.

You can generate the necessary artifacts for a read-only business object (BO) from scratch with the [OData UI Service from Scratch](#) wizard. For more information, see [OData UI Service from Scratch](#).

Development Process in Overview

The following development steps are part of the reference scenario. This example is based on the database table `/DMO/A_TRVL_ANA`.

You can download the complete reference scenario from GitHub: [Downloading the ABAP Flight Reference Scenario](#).

[Interface View](#)

Learn how to add the required annotations to your interface view to implement a RAP analytical table.

[Projection View](#)

Learn how to add the required annotations to your projection view to implement a RAP analytical table.

[Displaying the RAP Analytical Table on the SAP Fiori UI](#)

Learn how to add the required annotations to your metadata extension and expose your RAP analytical table.

Interface View

Learn how to add the required annotations to your interface view to implement a RAP analytical table.

Context

As an example, the data definition `/DMO/I_TRAVEL_ANA` is used, which serves as the interface view for the database table `/DMO/A_TRVL_ANA`. In this view, add the respective elements that you want to aggregate later.

i Note

If you use the [OData UI Service from Scratch](#) wizard and you use a database table as a template, be aware that the wizard only generates the respective entity fields. It doesn't duplicate the whole data of the selected template.

Procedure

1. In your interface view, add the element `booking_fee` with alias `MinBookingFee` and the element `booking_fee` with alias `MaxBookingFee`. For both elements, use the annotation `@Semantics.amount.currencyCode: 'CurrencyCode'` as shown below:

```
"...
define root view entity /DMO/I_TRAVEL_ANA as select from /DMO/A_TRVL_ANA
{
    "...
    @Semantics.amount.currencyCode: 'CurrencyCode'
    booking_fee                as MinBookingFee,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    booking_fee                as MaxBookingFee,
    "...
}
```

i Note

If you have other currency elements, such as `BookingFee` and `TotalPrice`, make sure to add the annotation `@Semantics.amount.currencyCode: 'CurrencyCode'` also there.

2. Add the element `differentCurrencies` with `cast: cast ( 1 as abap.int4 ) as differentCurrencies`.
3. **i Expand the following code sample to view the source code of the interface view `/DMO/I_TRAVEL_ANA`.**

≡ Sample Code

```
@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Interface View Travel Analytical Table'
@Metadata.ignorePropagatedAnnotations: true
define root view entity /DMO/I_TRAVEL_ANA
  as select from /DMO/A_TRVL_ANA
{
  key travel_uuid          as TravelUuid,
  travel_id               as TravelId,
  agency_id              as AgencyId,
  customer_id            as CustomerId,
  begin_date             as BeginDate,
  end_date              as EndDate,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  booking_fee            as BookingFee,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  booking_fee            as MinBookingFee,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  booking_fee            as MaxBookingFee,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  total_price            as TotalPrice,
  cast ( 1 as abap.int4 ) as differentCurrencies,
  currency_code          as CurrencyCode,
  description            as Description,
  overall_status         as OverallStatus
}
```

Projection View

Learn how to add the required annotations to your projection view to implement a RAP analytical table.

Context

The data definition /DMO/C_TRAVEL_ANA is the projection view for the interface view /DMO/I_TRAVEL_ANA. In this projection view, add the header annotation @OData.applySupportedForAggregation: #FULL. You also add the respective aggregation annotations for your aggregation elements by adding the annotation @Aggregation.default: #<AGGR_FUNCTION>. The system calculates the minimum, maximum, and average booking fees and counts the number of distinct currencies.

Procedure

1. In the header of your projection view, add the annotation @OData.applySupportedForAggregation: #FULL. This annotation enables the analytical table features for OData V4.

```
@Metadata.allowExtensions: true
@AccessControl.authorizationCheck:#NOT_REQUIRED
@EndUserText.label: 'Projection View Travel Analytical Table'
@OData.applySupportedForAggregation: #FULL

define root view entity /DMO/C_Travel_ANA
  provider contract transactional_query
  as projection on /DMO/I_TRAVEL_ANA
{
  "...

```

2. Add the aggregation annotation @Aggregation.default: #AVG to the element BookingFee:

```
"...
define root view entity /DMO/C_Travel_ANA
  provider contract transactional_query
  as projection on /DMO/I_TRAVEL_ANA
{
  "...
  @Aggregation.default: #AVG
  @EndUserText.label: 'Booking Fee (#AVG)'
  @Semantics.amount.currencyCode: 'CurrencyCode'
  BookingFee,
  "...
}
```

3. Similar to the previous step, add the aggregation annotation @Aggregation.default: #MIN to the element MinBookingFee, the aggregation annotation @Aggregation.default: #MAX to the element MaxBookingFee and the aggregation annotation @Aggregation.default: #SUM to the element TotalPrice.
4. Add the aggregation annotation @Aggregation.default: #COUNT_DISTINCT to the element differentCurrencies. In this context, the annotation @Aggregation.referenceElement: ['CurrencyCode'] is also mandatory. The combination of both annotations counts the number of distinct values of another element that is referenced:

```
"...
define root view entity /DMO/C_Travel_ANA
  provider contract transactional_query
  as projection on /DMO/I_TRAVEL_ANA
{
  "...
  @Aggregation.default: #COUNT_DISTINCT
  @EndUserText.label: 'Different Currencies (#COUNT_DISTINCT)'
  @Aggregation.referenceElement: ['CurrencyCode']
  differentCurrencies,
  "...
}
```

5. **Expand the following code sample to view the source code of the projection view /DMO/C_TRAVEL_ANA.**

Sample Code

```
@Metadata.allowExtensions: true
@AccessControl.authorizationCheck:#NOT_REQUIRED
@EndUserText.label: 'Projection View Travel Analytical Table'
@OData.applySupportedForAggregation: #FULL
```



```

define root view entity /DMO/C_Travel_ANA
  provider contract transactional_query
  as projection on /DMO/I_TRAVEL_ANA
{
  key TravelUuid,
    TravelId,
    AgencyId,
    CustomerId,
    BeginDate,
    EndDate,
    @Aggregation.default: #AVG
    @EndUserText.label: 'Booking Fee (#AVG)'
    @Semantics.amount.currencyCode: 'CurrencyCode'
    BookingFee,
    @Aggregation.default: #MIN
    @EndUserText.label: 'Min Booking Fee (#MIN)'
    @Semantics.amount.currencyCode: 'CurrencyCode'
    MinBookingFee,
    @Aggregation.default: #MAX
    @EndUserText.label: 'Max Booking Fee (#MAX)'
    @Semantics.amount.currencyCode: 'CurrencyCode'
    MaxBookingFee,
    @Aggregation.default: #SUM
    @EndUserText.label: 'Total Price (#SUM)'
    TotalPrice,
    @Aggregation.default: #COUNT_DISTINCT
    @EndUserText.label: 'Different Currencies (#COUNT_DISTINCT)'
    @Aggregation.referenceElement: ['CurrencyCode']
    differentCurrencies,
    CurrencyCode,
    Description,
    OverallStatus
}

```

Displaying the RAP Analytical Table on the SAP Fiori UI

Learn how to add the required annotations to your metadata extension and expose your RAP analytical table.

Context

The metadata extension /DMO/C_TRAVEL_ANA contains the UI metadata relevant for the projection view /DMO/C_TRAVEL_ANA. In this artifact, you need to add the respective annotations for enabling the grouping function and for your aggregation elements. In a final step, you expose your business service as an OData V4 service.

Procedure

1. In the header of your metadata extension, add the annotation `@UI.presentationVariant: [{...}]` to group a specific element automatically. In this example, the element `OverallStatus` is grouped:

```

@Metadata.layer: #CORE
@UI.headerInfo.title.type: #STANDARD
@UI.headerInfo.title.value: 'TravelId'
@UI.headerInfo.description.type: #STANDARD
@UI.headerInfo.description.value: 'TravelId'
@UI.presentationVariant: [{ text: 'Default' ,
                           sortOrder:[{ by: 'OverallStatus', direction: #ASC }],
                           groupBy: [ 'OverallStatus' ],
                           visualizations: [ { type: #AS_LINEITEM },
                           { type: #AS_LINEITEM } ] } ]
annotate entity /DMO/C_Travel_ANA with
{
  "...

```

2. Adjust your metadata extension by adding all your elements with aliases and their annotations. For example, for the element `MinBookingFee`, you get the following code:

```
"...
annotate entity /DMO/C_Travel_ANA with
{
  @UI.identification: [ {
    position: 50 ,
    label: 'MinBookingFee (#MIN)'
  } ]
  @UI.lineItem: [ {
    position: 50 ,
    label: 'MinBookingFee (#MIN)'
  } ]
  @UI.selectionField: [ {
    position: 50
  } ]
  MinBookingFee;
"...
```

3. **i** Expand the following code sample to view the source code of the metadata extension `/DMO/C_TRAVEL_ANA`:

Sample Code

```
@Metadata.layer: #CORE
@UI.headerInfo.title.type: #STANDARD
@UI.headerInfo.title.value: 'TravelId'
@UI.headerInfo.description.type: #STANDARD
@UI.headerInfo.description.value: 'TravelId'
@UI.presentationVariant: [{ text: 'Default' ,
                             sortOrder:[{ by: 'OverallStatus', direction: #ASC }],
                             groupBy: [ 'OverallStatus' ],
                             visualizations: [ { type: #AS_LINEITEM },
                             { type: #AS_LINEITEM }]}]
annotate entity /DMO/C_Travel_ANA with
{
  @UI.hidden: true
  @EndUserText.label: 'Travel UUID'
  @UI.identification: [ {
    position: 10 ,
    label: 'UUID'
  } ]
  @UI.lineItem: [ {
    position: 10 ,
    label: 'UUID'
  } ]
  @UI.selectionField: [ {
    position: 10
  } ]
  TravelUuid;

  @EndUserText.label: 'TravelID'
  @UI.facet: [ {
    label: 'General Information',
    id: 'GeneralInfo',
    purpose: #STANDARD,
    position: 20 ,
    type: #IDENTIFICATION_REFERENCE
  } ]
  @UI.identification: [ {
    position: 20 ,
    label: 'TravelID'
  } ]
  @UI.lineItem: [ {
    position: 20 ,
    label: 'TravelID'
  } ]
  @UI.selectionField: [ {
    position: 20
  } ]
  TravelId;

  @EndUserText.label: 'Total Price (#SUM)'
```

```

@UI.identification: [ {
  position: 30 ,
  label: 'Total Price (#SUM)'
} ]
@UI.lineItem: [ {
  position: 30 ,
  label: 'Total Price (#SUM)'
} ]
@UI.selectionField: [ {
  position: 30
} ]
TotalPrice;

@endUserText.label: 'Booking Fee (#AVG)'
@UI.identification: [ {
  position: 40 ,
  label: 'Booking Fee (#AVG)'
} ]
@UI.lineItem: [ {
  position: 40 ,
  label: 'Booking Fee (#AVG)'
} ]
@UI.selectionField: [ {
  position: 40
} ]
BookingFee;

@UI.identification: [ {
  position: 50 ,
  label: 'MinBookingFee (#MIN)'
} ]
@UI.lineItem: [ {
  position: 50 ,
  label: 'MinBookingFee (#MIN)'
} ]
@UI.selectionField: [ {
  position: 50
} ]
MinBookingFee;

@UI.identification: [ {
  position: 60 ,
  label: 'MaxBookingFee (#MAX)'
} ]
@UI.lineItem: [ {
  position: 60 ,
  label: 'MaxBookingFee (#MAX)'
} ]
@UI.selectionField: [ {
  position: 60
} ]
MaxBookingFee;

@UI.identification: [ {
  position: 70 ,
  label: 'DifferentCurrencies (#COUNT_DISTINCT)'
} ]
@UI.lineItem: [ {
  position: 70 ,
  label: 'DifferentCurrencies (#COUNT_DISTINCT)'
} ]
@UI.selectionField: [ {
  position: 70
} ]
differentCurrencies;

@endUserText.label: 'BeginDate'
@UI.identification: [ {
  position: 80 ,
  label: 'BeginDate'
} ]
@UI.lineItem: [ {
  position: 80 ,
  label: 'BeginDate'
} ]

```

```

    } ]
    @UI.selectionField: [ {
      position: 80
    } ]
    BeginDate;

    @EndUserText.label: 'EndDate'
    @UI.identification: [ {
      position: 90 ,
      label: 'EndDate'
    } ]
    @UI.lineItem: [ {
      position: 90 ,
      label: 'EndDate'
    } ]
    @UI.selectionField: [ {
      position: 90
    } ]
    EndDate;

    @EndUserText.label: 'OverallStatus'
    @UI.identification: [ {
      position: 100 ,
      label: 'OverallStatus'
    } ]
    @UI.lineItem: [ {
      position: 100 ,
      label: 'OverallStatus'
    } ]
    @UI.selectionField: [ {
      position: 100
    } ]
    OverallStatus;

    @EndUserText.label: 'Description'
    @UI.identification: [ {
      position: 110 ,
      label: 'Description'
    } ]
    @UI.lineItem: [ {
      position: 110 ,
      label: 'Description'
    } ]
    @UI.selectionField: [ {
      position: 110
    } ]
    Description;
  }
}

```

4. Adapt your service definition in order to expose your service as OData V4. Your service definition should look like this:

```

@EndUserText: {
  label: 'Service Definition for Travel'
}
@ObjectModel: {
  leadingEntity: {
    name: '/DMO/C_TRAVEL_ANA'
  }
}
define service /DMO/UI_TRAVEL_04_ANA
  provider contracts odata_v4_ui {
    expose /DMO/C_Travel_ANA as Travel;
  }

```

5. Expose your business object as an OData V4 service.

For more information, see [Business Service](#).

Results

When exposed to the SAP Fiori UI, your data is displayed as an analytical table:

TravelID:
 Total Price (#SUM):
 Booking Fee (#AVG):
 BeginDate:
 EndDate:
 OverallStatus:
 Description:
[Go](#)
[Adapt Filters](#)

Overall Status: A														
1732	4,762.00	EUR	30.00	EUR	30.00	EUR	30.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
1749	3,263.00	EUR	20.00	EUR	20.00	EUR	20.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
1779	3,910.00	EUR	20.00	EUR	20.00	EUR	20.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
1791	6,544.00	EUR	30.00	EUR	30.00	EUR	30.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
1816	2,686.50	EUR	10.00	EUR	10.00	EUR	10.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
1818	2,685.50	EUR	10.00	EUR	10.00	EUR	10.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
1832	6,139.50	EUR	20.00	EUR	20.00	EUR	20.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Business Trip	>
97	21,691.00	EUR	80.00	EUR	80.00	EUR	80.00	EUR	1	Jun 24, 2025	Jun 27, 2025	A	Business Trip	>
102	32,488.30	EUR	120.00	EUR	120.00	EUR	120.00	EUR	1	Jun 24, 2025	Jun 27, 2025	A	Business Trip	>
106	21,634.80	EUR	80.00	EUR	80.00	EUR	80.00	EUR	1	Jun 24, 2025	Jun 27, 2025	A	Business Trip	>
114	22,358.50	EUR	90.00	EUR	90.00	EUR	90.00	EUR	1	Jun 24, 2025	Jun 26, 2025	A	Sightseeing in Frankfurt/Main	>
117	7,404.50	EUR	30.00	EUR	30.00	EUR	30.00	EUR	1	Jun 24, 2025	Jun 26, 2025	A	Sightseeing in Frankfurt/Main	>
125	22,458.80	EUR	90.00	EUR	90.00	EUR	90.00	EUR	1	Jun 24, 2025	Jun 26, 2025	A	Sightseeing in Frankfurt/Main	>
132	8,016.00	EUR	30.00	EUR	30.00	EUR	30.00	EUR	1	Jun 24, 2025	Jun 26, 2025	A	Sightseeing in Frankfurt/Main	>
133	15,793.00	EUR	60.00	EUR	60.00	EUR	60.00	EUR	1	Jun 24, 2025	Jun 26, 2025	A	Sightseeing in Frankfurt/Main	>
1730	3,185.50	EUR	20.00	EUR	20.00	EUR	20.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Sightseeing in Newark, New Jersey	>
1738	4,827.50	EUR	30.00	EUR	30.00	EUR	30.00	EUR	1	Jun 24, 2025	Jun 24, 2025	A	Sightseeing in Newark, New Jersey	>

Developing Managed Transactional Apps

In this chapter, you will be guided through all steps necessary to develop a travel administration app based on the business object's managed runtime infrastructure.

Introduction

Let's assume you want to develop completely new transactional apps for SAP Fiori UI or for an arbitrary Web API consumer. Let's further assume that you don't have any transactional buffer or business logic implementation or authorization functionality available to you.

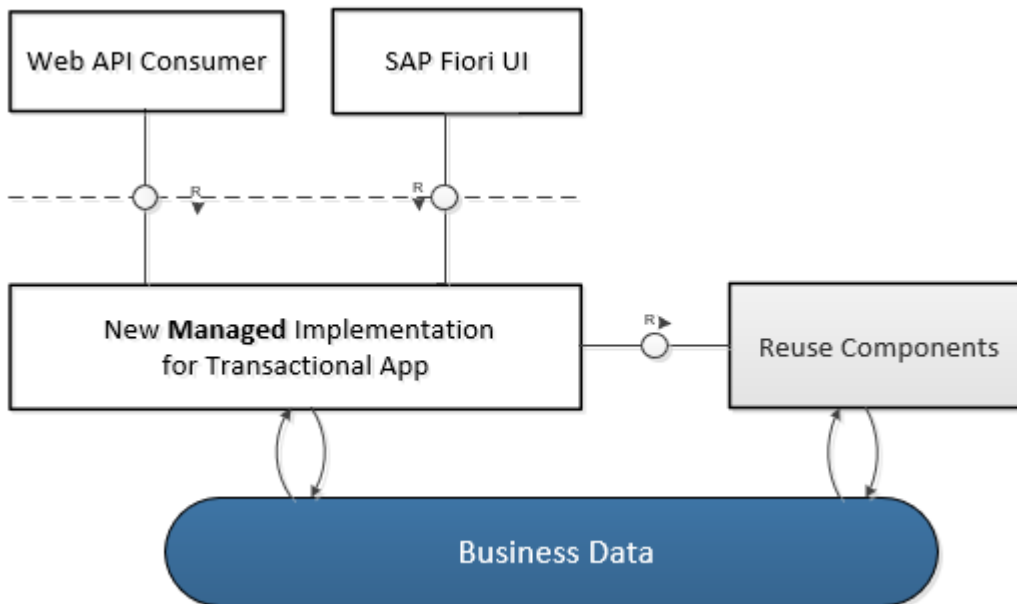
In that situation you can benefit from the **managed** implementation type of the ABAP RESTful Application Programming Model.

The managed implementation type addresses use cases where all essential parts of an application must be developed from scratch. However, these new applications can highly benefit from out-of-the-box support for transactional processing. Standard operations (create, update, delete) must only be specified in the behavior definition to obtain a ready-to-run business object. In addition the provisioning and handling of the transactional buffer is automatically done for you. The technical implementation aspects are taken over by the managed RAP BO provider. The interaction phase and the save sequence are implemented generically. The application developer can then focus on business logic that is implemented using actions, validations and determinations and user interaction. The corresponding transactional engine manages the entire life cycle of your business object and covers all aspects of your business application development.

Architecture Overview

The architecture for a managed scenario can be depicted as in the simplified figure below.

The new implementation of business objects and their functionality is a key component in the architecture. In addition to the generic implementation of transactional handling, these objects also provide the application-specific business logic. Even with new developments, you will certainly always use already available reuse functionality whenever possible. For example, our demo application uses value help views and text views for text information retrieval, currency conversion procedures, and message constants as reuse components.



Architecture Overview - Managed Implementation

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from [GitHub](#): [Downloading the ABAP Flight Reference Scenario](#).

→ Remember

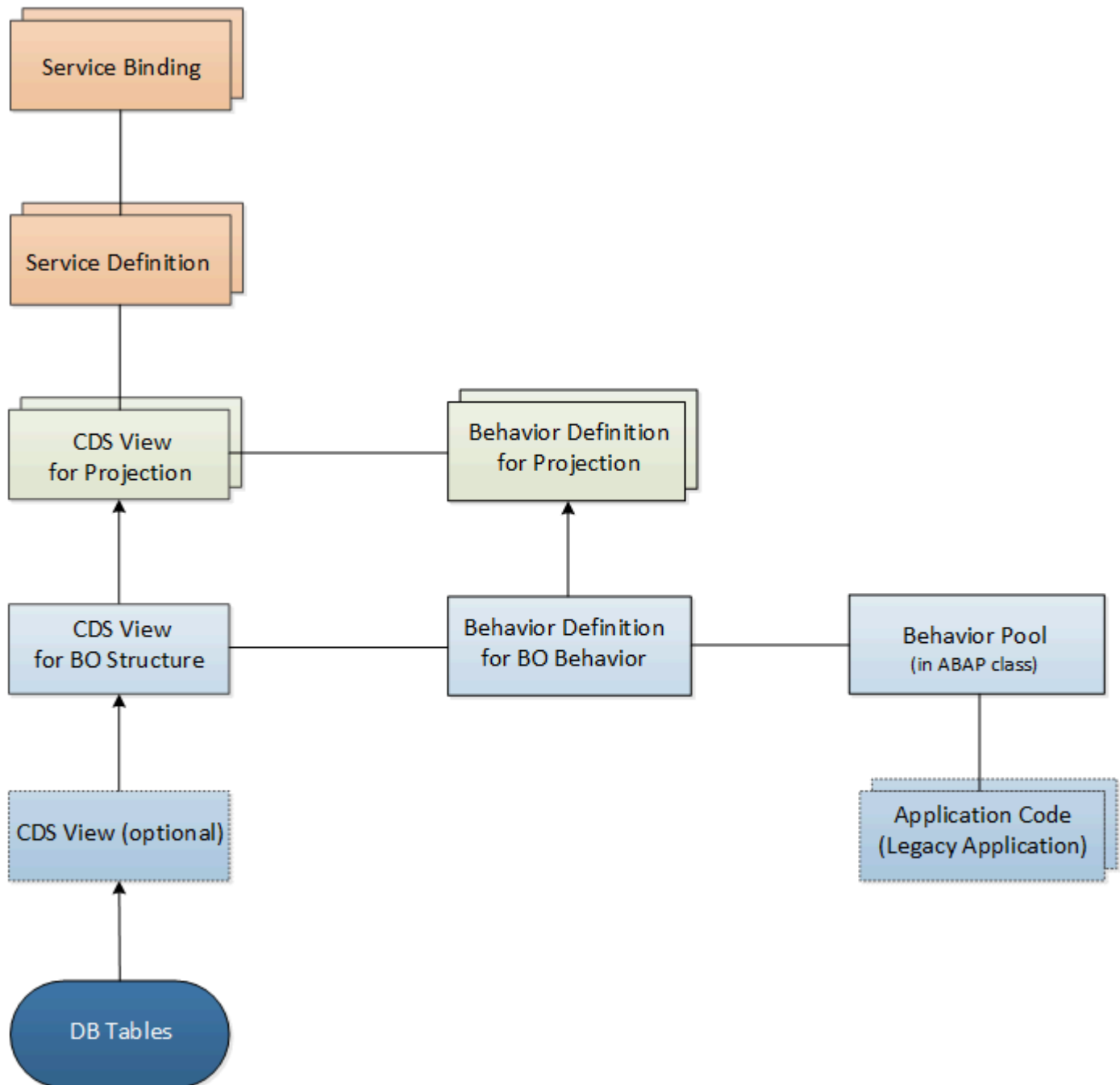
The namespace `/DMO/` is reserved for the demo content. Apart from the downloaded ABAP Flight Scenario, do not use the namespace `/DMO/` and do not create any development objects in the downloaded packages.

You can access the development objects in `/DMO/` from your own namespace.

However, if you want to recreate all development objects of this demo content, make sure that you use different names from this documentation.

Involved Development Objects

The figure below provides an overview of the main development objects involved when creating new transactional apps based on the managed implementation type.



Development Objects Involved When Implementing Managed Transactional Apps

Development Process in Overview

The development of new managed applications mainly requires developers to perform the following fundamental activities:

1. Developing Ready-to-Run Business Objects

In this section, you will create, from scratch, all the components of your application required to run ready-to-run business objects. Using the business object framework for managed implementation type, you will save time during the development cycle because you don't have to implement all the technical details yourself - details such as low-level transaction handling, buffer management, or business logic orchestration. This framework provides a set of generic services and functionalities to speed up, standardize, and modularize your development.

More on this: [Developing a Ready-to-Run Business Object](#)

2. Developing Business Logic

More on this: [Developing Business Logic](#)

3. Providing Projection Layer for Service Consumption

To enable a flexible consumption of the resulting business service, a projection is introduced as a separate layer within the application development. This layer is required for service projections for different consumption scenarios such as Web APIs and UI-related services according to Fiori UI role-based design.

This essentially includes three steps: providing a data model with CDS views for projections, modeling behavior definitions for projections as well as defining business services that expose projections for consumption.

More on this: [Developing a Projection Layer for Flexible Service Consumption](#)

4. Testing the OData UI Service

Using ABAP Development Tools, you have the option of publishing the service to the service repository of your local system. As soon as the service is published, it is ready for consumption through an OData client, such as an SAP Fiori app. The service binding editor also offers a preview tool that you can use for testing the resulting app within your ABAP development environment.

More on this: [Testing the Business Object](#)

Reference Business Scenario

In this demo scenario, we will implement a simple travel provider app that can be used to manage flight bookings. A single travel should be booked by the travel provider for an existing customer through a travel agency and include one or multiple flight bookings. In addition, one or more supplements should be able to be booked for each flight.

Requirements

Based on the same data model, two different views of the travel app are to be realized, each corresponding to two different user roles:

- The **processor** acquires all the data relevant to flight bookings: he or she creates individual travel instances, creates and manages individual flights, and adds supplements to flight bookings. The accumulated travel costs should be calculated automatically when the underlying bookings are updated. When editing individual travel data, validation must be made for data consistency and, in the case of an error, be issued as an appropriate message to the user.

Standard

Agency ID:

Customer ID:

Travels (4,183)

☐	Travel ID	Agency ID	Customer ID	Starting Date	End Date	Total Price	Overall Status	
☒	1	Maxitrip (70041)	Ryan (594)	Jun 4, 2022	Apr 2, 2023	1,782.24 USD	Rejected	>
☐	2	Hot Socks Travel (70007)	Prinz (608)	Jun 4, 2022	Jun 4, 2022	900.00 USD	Open	>
☐	3	Hendrik's (70046)	Detemple (93)	Jun 4, 2022	Apr 2, 2023	4,127.79 USD	Open	>

List of Travels - UI for Processor Role

Travel

Booking

Travel ID:

2

Agency ID: *

Hot Socks Travel (70007)

Customer ID: *

Prinz (608)

Starting Date: *

Jun 4, 2022

End Date: *

Jun 4, 2022

Booking Fee: *

20.00

USD

Total Price:

900.00

USD

Status: *

Open

Description:

Business Trip for Christine, Pierre

Creating a Travel - UI for Processor Role

- The role of the **approver** is limited to the verification of the recorded travel data entered by the processor. The approver has the option to accept or reject individual travels.

Search

Go Adapt Filters

Travels (4,185)							
Travel ID		Overall Status	Agency ID	Customer ID	Booking Fee		Total Price
☐	1	Rejected	Maxitrip (70041)	Ryan (594)	40.00	USD	1,782.24 USD
☒	2	Open	Hot Socks Travel (70007)	Prinz (608)	20.00	USD	900.00 USD
☐	3	Open	Hendrik's (70046)	Detemple (93)	80.00	USD	4,127.79 USD

List of Travels - UI for Approver Role

Data Model

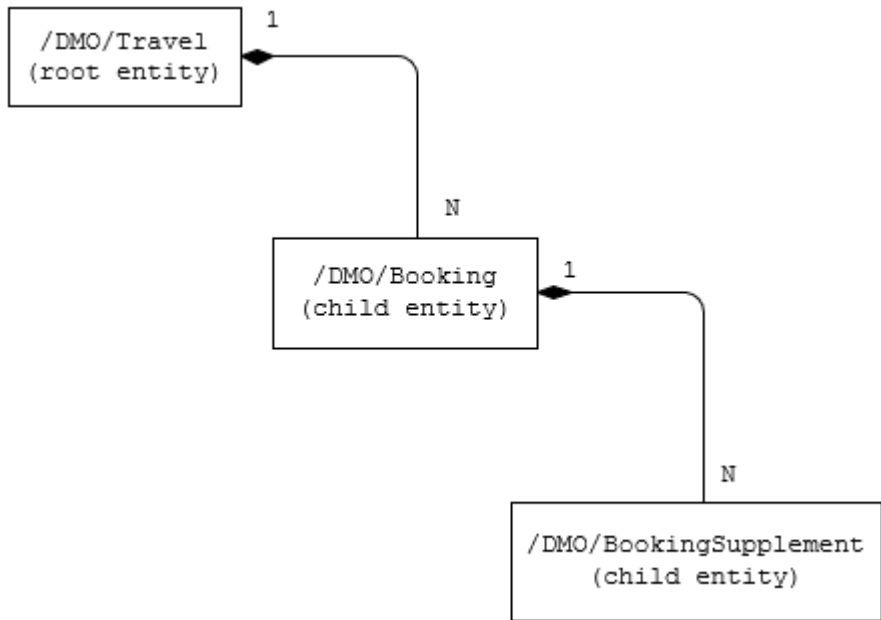
Entities for Transactional Data

As depicted in the figure below, the 3-tier entity hierarchy for managing transactional data in our scenario consists of the following editable entities, with a 1: N cardinality on each level:

- Travel
- Booking
- BookingSupplement

That is, each travel instance has 0..N bookings and each booking has 0..N booking supplements.

The figure below shows the composition relationship between the travel, the (flight) booking and the supplement entities, where the travel entity represents the root of the data model.



Editable Entities of the Data Model

Additional Entities for Master Data (Reuse Components)

To access business data from other entities in our demo content, we are going to reuse master data from the package /DMO/FLIGHT_REUSE that is part of the downloaded demo content /DMO/FLIGHT. Some of these entities will be used primarily as text provider views for retrieving text information and as value help provider views for individual input fields on the UI.

- ▼ /DMO/FLIGHT_REUSE (51) *Flight Reference Scenario: Reused Entities*
 - > /DMO/FLIGHT_REUSE_SUPPLEMENT (27) *Flight Reference Scenario: Supplement*
 - > Authorizations (2)
- ▼ Core Data Services (12)
 - ▼ Data Definitions (12)
 - > /DMO/I_AGENCY *Travel View - CDS Data Model*
 - > /DMO/I_AIRPORT *Airport View - CDS Data Model*
 - > /DMO/I_BOOKING_STATUS_VH_TEXT *Overall Status Value Help*
 - > /DMO/I_BOOKING_STATUS_VH *Overall Status Value Help*
 - > /DMO/I_CARRIER *Carrier View - CDS Data Model*
 - > /DMO/I_CONNECTION *Connection View - CDS Data Model*
 - > /DMO/I_CUSTOMER *Customer View - CDS Data Model*
 - > /DMO/I_FLIGHT *Flight View - CDS Data Model*
 - > /DMO/I_OVERALL_STATUS_VH_TEXT *Overall Status Value Help*
 - > /DMO/I_OVERALL_STATUS_VH *Overall Status Value Help*

Non-Editable Entities Reused in the Data Model

Behavior

In the following table, we briefly outline the scope of the business logic to be implemented in the demo scenario. Bear in mind that the managed implementation type comes into play with a few new concepts such as validations and determinations, which will be introduced later on over the course of this E2E guide.

Processor Role:

Processor

Entity	Behavior
Travel	Operations: • Create, update, delete • Create bookings by association • Action: copy travels Validations: • Validate editable input fields Feature Control: • Static and dynamic field control • Dynamic operation control • Dynamic action control
Booking	Operations: • Update • Create booking supplements by association Validations: • Validate editable input fields Determinations: • Calculate total price for bookings Feature Control: • Static and dynamic field control • Dynamic operation control
Booking Supplement	Operations: • Update Determinations: • Calculate total price for supplements Feature Control: • Static and dynamic field control

Approver Role:

Approver

Entity	Behavior
Travel	Operations: • Update (on limited set of travel fields) • Action: accept travel

Entity	Behavior
	- Action: reject travel Validations: - Validate editable fields Feature Control: - Static and dynamic field control - Dynamic action control
Booking	Read operations only

Restrictions

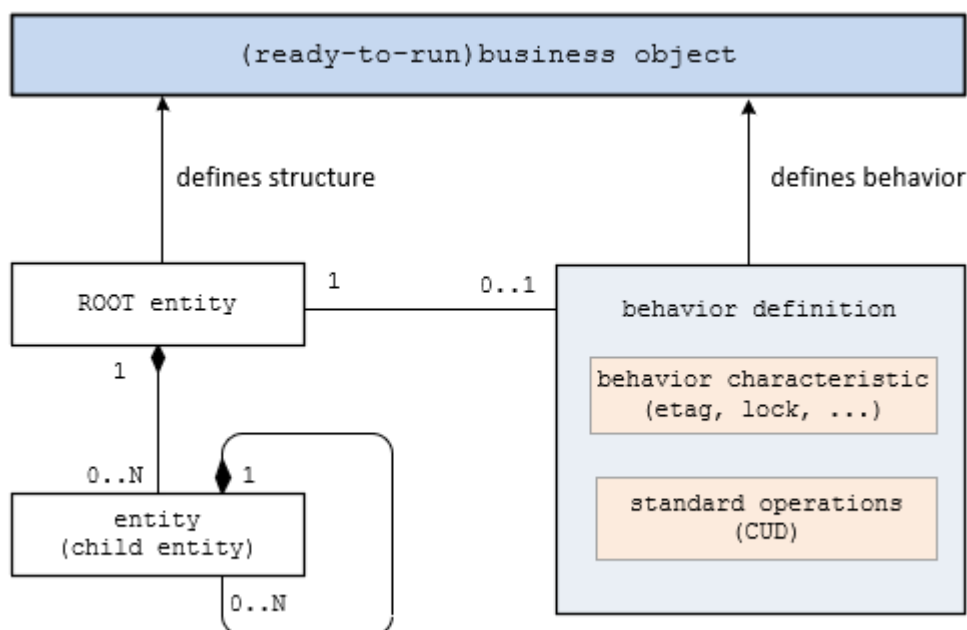
! Restriction

The current version of the ABAP RESTful programming model does not support late numbering for managed implementation type. Therefore, in our demo scenario, the numbering is not implemented in the save sequence using the `adjust_numbering( )` method when creating instances for travels, bookings, and booking supplements.

Developing a Ready-to-Run Business Object

Each business object is characterized by a structure, a behavior and a runtime implementation.

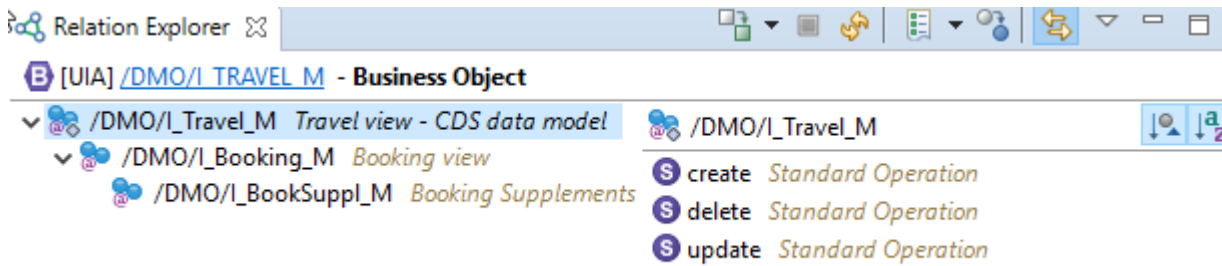
As shown in the figure below, the structure of a business object is defined by the sequence of compositions and to-parent associations between CDS entities with a root entity on top of the composition tree. Each level of a business object's composition tree can offer a set of operations that are specified in the behavior definition that refers to the entities of the CDS data model. In the case of managed implementation type, the standard operations (create, update, delete) must only be specified in the behavior definition to obtain a ready-to-run business object. The business object runtime infrastructure already implements the interaction phase and the save sequence generically and provides an out-of-the-box support for transactional processing.



Data Model and Behavior of a Ready-to-run Business Object

Preview: Resulting Business Object in Relation Explorer

The figure below displays the resulting “Travel” business object in **Relation Explorer** view. If you choose the **Business Object** context, the **Relation Explorer** provides the composition tree of the business object and displays all operations of the selected entity.



"Travel" Business Object in Relation Explorer

Activities Relevant to Developers

1. [Providing Business Object Structure](#)
 - a. [Providing Persistent Tables](#)
 - b. [Creating CDS Data Definitions](#)
 - c. [Defining Data Model and Business Object Structure](#)
2. [Defining Elementary Behavior for Ready-to-Run Business Object](#)
3. [Testing the Business Object](#)

Related Information

[Exploring Business Objects](#)

Providing Business Object Structure

From a structural point of view, a business object consists of a tree of entities that are linked by compositions. Every entity in this composition tree is an element that is modeled with a CDS entity.

For our demo travel booking scenario, we will implement a 3-level hierarchy composition tree.

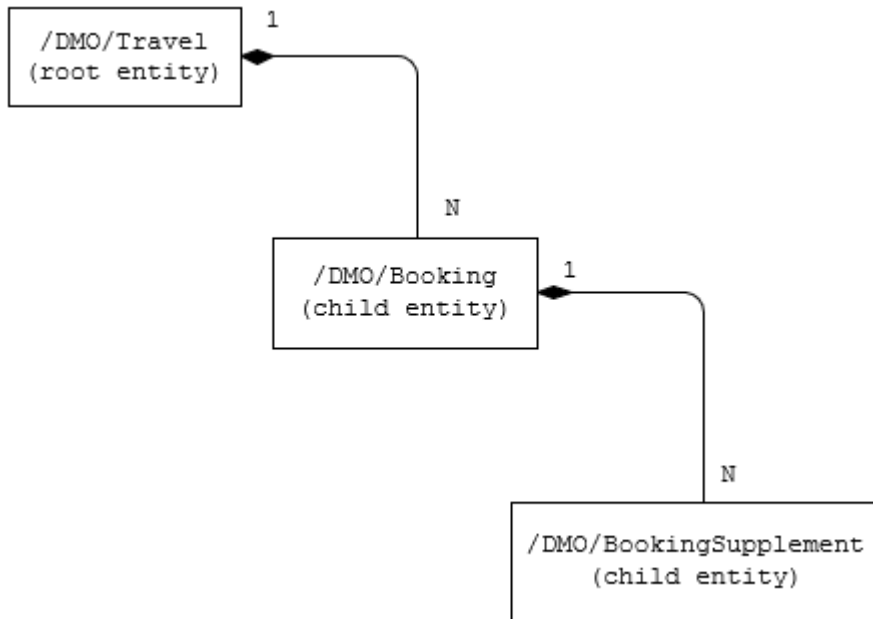
Entities for Transactional Data

As depicted in the figure below, the 3-tier entity hierarchy for managing transactional data in our scenario consists of the following editable entities, with a 1: N cardinality on each level:

- Travel
- Booking
- BookingSupplement

That is, each travel instance has 0..N bookings and each booking has 0..N booking supplements.

The figure below shows the composition relationship between the travel, the (flight) booking and the supplement entities, where the travel entity represents the root of the data model.



Editable Entities of the Data Model

Additional Entities for Master Data (Reuse Components)

To access business data from other entities in our demo content, we are going to reuse master data from the package /DMO/FLIGHT_REUSE that is part of the downloaded demo content /DMO/FLIGHT. Some of these entities will be used primarily as text provider views for retrieving text information and as value help provider views for individual input fields on the UI.

- ▼ /DMO/FLIGHT_REUSE (51) *Flight Reference Scenario: Reused Entities*
 - > /DMO/FLIGHT_REUSE_SUPPLEMENT (27) *Flight Reference Scenario: Supplement*
 - > Authorizations (2)
 - ▼ Core Data Services (12)
 - ▼ Data Definitions (12)
 - > /DMO/I_AGENCY *Travel View - CDS Data Model*
 - > /DMO/I_AIRPORT *Airport View - CDS Data Model*
 - > /DMO/I_BOOKING_STATUS_VH_TEXT *Overall Status Value Help*
 - > /DMO/I_BOOKING_STATUS_VH *Overall Status Value Help*
 - > /DMO/I_CARRIER *Carrier View - CDS Data Model*
 - > /DMO/I_CONNECTION *Connection View - CDS Data Model*
 - > /DMO/I_CUSTOMER *Customer View - CDS Data Model*
 - > /DMO/I_FLIGHT *Flight View - CDS Data Model*
 - > /DMO/I_OVERALL_STATUS_VH_TEXT *Overall Status Value Help*
 - > /DMO/I_OVERALL_STATUS_VH *Overall Status Value Help*

Non-Editable Entities Reused in the Data Model

i Note

Additional entities for currencies (I_Currency) and countries (I_Country) are generally available in your system and are included in our data model using associations.

Related Information

Providing Persistent Tables

Travel data of our demo application is distributed across multiple database tables: a table for the travel header data, a table for flight bookings, and booking supplements.

To build up our demo scenario from scratch, we will first create a suitable set of database tables using ADT ABAP Dictionary tools and then use these tables as a data source in the new CDS-based data model.

The standard use case is to work with client-dependent tables. That means, the database table has a key field in which the client is maintained. Like this, you can control the access to the database entries based on the client. However, there are use cases in which the instances of the database tables are not client-specific. These tables do not contain a client field. The RAP managed BO runtime also supports scenarios with client-independent database tables. For more information, see [Using Client-Independent Database Tables](#).

All RAP applications need administrative fields to log data access and modification. The RAP framework offers reuse data elements for these fields, see [RAP Reuse Data Elements](#).

Procedure: Creating Tables

To create and work with database tables in ABAP Development Tools (ADT), do the following:

1. Open the source-based ABAP Dictionary editor in ADT and create the corresponding database tables listed below.
2. Edit the database tables with the fields and metadata as depicted in the listings below.
3. Activate the tables

Table /DMO/TRAVEL_M: Persistent Table for Managing Travel Data

This table defines general travel data, such as the key element travel ID, agency ID or customer ID, the date for travel's begin and end, as well as the overall status of the travel bookings, and the total price of an individual travel. In addition, the fields for standard administration data, such as the respective user or the time of creation, are added to the table.

i *Expand the following code sample to view the source code of database table /dmo/travel_m.*

Sample Code

```
@EndUserText.label : 'Flight Reference Scenario: Managing Travels'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/travel_m {
  key client      : abap.clnt not null;
  key travel_id   : /dmo/travel_id not null;
  agency_id      : /dmo/agency_id;
  customer_id    : /dmo/customer_id;
  begin_date     : /dmo/begin_date;
  end_date       : /dmo/end_date;
  @Semantics.amount.currencyCode : '/dmo/travel_m.currency_code'
  booking_fee    : /dmo/booking_fee;
  @Semantics.amount.currencyCode : '/dmo/travel_m.currency_code'
  total_price    : /dmo/total_price;
  currency_code  : /dmo/currency_code;
```

```

description      : /dmo/description;
overall_status   : /dmo/overall_status;
@AbapCatalog.anonymizedWhenDelivered : true
created_by       : abp_creation_user;
created_at       : abp_creation_tstmpl;
@AbapCatalog.anonymizedWhenDelivered : true
last_changed_by  : abp_locinst_lastchange_user;
last_changed_at  : abp_locinst_lastchange_tstmpl;
}

```

Table /DMO/BOOKING_M: Persistent Table for Managing Bookings

This table will be used for managing flight booking data, such the flight connection, the carrier, or the price and flight date, and finally, the status of flight bookings.

i Expand the following code sample to view the source code of database table `/dmo/booking_m`.

Sample Code

```

@EndUserText.label : 'Flight Reference Scenario: Booking'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/booking_m {
  key client      : abap.clnt not null;
  @AbapCatalog.foreignKey.label : 'Travel'
  @AbapCatalog.foreignKey.screenCheck : false
  key travel_id   : /dmo/travel_id not null
    with foreign key [0..*,1] /dmo/travel_m
    where travel_id = /dmo/booking_m.travel_id;
  key booking_id  : /dmo/booking_id not null;
  booking_date    : /dmo/booking_date;
  customer_id     : /dmo/customer_id;
  carrier_id      : /dmo/carrier_id;
  connection_id   : /dmo/connection_id;
  flight_date     : /dmo/flight_date;
  @Semantics.amount.currencyCode : '/dmo/booking_m.currency_code'
  flight_price    : /dmo/flight_price;
  currency_code   : /dmo/currency_code;
  booking_status  : /dmo/booking_status;
  last_changed_at : abp_locinst_lastchange_tstmpl;
}

```

Table /DMO/BOOKSUPPL_M: Persistent Table for Managing Booking Supplement Data

This table is used to add additional products to a travel booking. For example, the customer can book together with a flight, a drink or a meal.

i Expand the following code sample to view the source code of database table `/dmo/booksuppl_m`.

Sample Code

```

@EndUserText.label : 'Flight Reference Scenario: Booking Supplement'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/booksuppl_m {
  key client      : abap.clnt not null;

```



```
@AbapCatalog.foreignKey.label : 'Travel'
@AbapCatalog.foreignKey.screenCheck : false
key travel_id : /dmo/travel_id not null
  with foreign key [0..*,1] /dmo/travel_m
    where travel_id = /dmo/booksuppl_m.travel_id;
@AbapCatalog.foreignKey.label : 'Booking'
@AbapCatalog.foreignKey.screenCheck : false
key booking_id : /dmo/booking_id not null
  with foreign key [0..*,1] /dmo/booking_m
    where travel_id = /dmo/booksuppl_m.travel_id
      and booking_id = /dmo/booksuppl_m.booking_id;
key booking_supplement_id : /dmo/booking_supplement_id not null;
supplement_id : /dmo/supplement_id;
@Semantics.amount.currencyCode : '/dmo/booksuppl_m.currency_code'
price : /dmo/supplement_price;
currency_code : /dmo/currency_code;
last_changed_at : abp_locinst_lastchange_tstmpl;

}
```

Related Information

[Working with Database Tables](#)

Creating CDS Data Definitions

In this step you create CDS views as the basis for the data model of our demo scenario. For each data definition a corresponding development object and the related CDS views are created.

Data Definitions and CDS Views to Create

i Note

Naming CDS views: Since CDS views are (public) interface views, they are prefixed with **I_** in accordance with the VDM (virtual data model) naming convention. In addition, we add the suffix **_M** to the view name in case it is specific for our managed implementation type scenario. For detailed information, see: [Naming Conventions for Development Objects](#)

Data Definitions Required for all Editable Entities of the Data Model

Data Definition CDS View Name	Data Source	Description
/DMO/I_TRAVEL_M /DMO/I_Trave ^l _M (root entity)	/DMO/TRAVEL_M (DB table)	A Travel entity defines general travel data, such as the agency ID or customer ID, overall status of the travel bookings, and the total price of a travel.
/DMO/I_BOOKING_M /DMO/I_Booking_M (child entity)	/DMO/BOOKING_M (DB table)	The booking entity is used for managing flight booking data, such as the customer, the flight connection, or the price and flight date.
/DMO/I_BOOKSUPPL_M /DMO/I_BookSuppl_M (child entity)	/DMO/BOOKSUPPL_M (DB table)	This entity is used to add additional products to a travel booking.

Procedure: Creating a Data Definition

To launch the wizard tool for creating a data definition, do the following:

1. Launch [ABAP Development Tools](#).
2. In your ABAP project (or ABAP cloud project), select the relevant package node in [Project Explorer](#).
3. Open the context menu and choose [New](#) > [Other ABAP Repository Object](#) > [Core Data Services](#) > [Data Definition](#) >

Defining Data Model and Business Object Structure

Travel Root View /DMO/I_Travel_M

For the root CDS view /DMO/I_Travel_M, the database table /dmo/travel_m serves as the data source. This CDS view defines the root entity of the managed business object and represents the root of compositional hierarchy for the travel business object to be created.

To define a composition relationship from the root to a child entity, the keyword **COMPOSITION** is used. In our example, we specify the /DMO/I_Booking_M as child entity in the composition_Booking. As a result, the booking node is defined as a direct child entity to the business object's root. The cardinality [0..*] specifies that any number of booking instances can be assigned to each travel instance.

To be able to access master data from other entities, a set of associations is defined in the CDS source code. These associations refer to CDS entities that are part of our demo application scenario. Some of these views are used primarily as text views for retrieving text information and as value help provider views for specific UI fields.

Finally, the fields for standard administration data are added to the select list, where the persistent field `last_changed_at` plays a special role as a field ETag.

To ensure uniform data processing on the consumer side, all administrative as well as quantity fields are provided with appropriate `@Semantics` annotations. These annotations are necessary to support managed ETag handling, that is the automatic update of the relevant ETag field on every operation.

i Expand the following code sample to view the source code of the CDS Root View /DMO/I_Travel_M.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Travel view - CDS data model'

define root view entity /DMO/I_Travel_M
  as select from /dmo/travel_m as Travel -- the travel table is the data source for this view

  composition [0..*] of /DMO/I_Booking_M as _Booking

  association [0..1] to /DMO/I_Agency      as _Agency  on $projection.agency_id = _Agency.AgencyI
  association [0..1] to /DMO/I_Customer  as _Customer on $projection.customer_id = _Customer.Cus
  association [0..1] to I_Currency       as _Currency on $projection.currency_code = _Currency.C
  association [1..1] to /DMO/I_Overall_Status_VH as _OverallStatus on $projection.overall_status

{
  key travel_id,
    agency_id,
    customer_id,
    begin_date,
    end_date,
    @Semantics.amount.currencyCode: 'currency_code'
    booking_fee,
    @Semantics.amount.currencyCode: 'currency_code'
    total_price,
    currency_code,
    overall_status,
```

```

description,
@Semantics.user.createdBy: true
created_by,
@Semantics.systemDateTime.createdAt: true
created_at,
@Semantics.user.localInstanceLastChangedBy: true
last_changed_by,
//local ETag field --> OData ETag
@Semantics.systemDateTime.localInstanceLastChangedAt: true
last_changed_at,

/* Associations */
_Booking,
_Agency,
_Customer,
_Currency,
_OverallStatus
}

```

Booking View /DMO/I_Booking_M

Listing 2 (below) provides you with a data model implementation of the booking entity. All fields that are declared in the database table are used in the CDS view. The administrative field `last_changed_at` is used for optimistic concurrency control on all nodes. That means, every business object entity stores its own eTag.

In the data definition of the root entity `/DMO/I_Travel_M`, we specified the booking entity `/DMO/I_Booking_M` as a composition child entity. Reversely, this relationship requires an association to their compositional parent entity – from the child entity. This relationship is expressed by the keyword `ASSOCIATION TO PARENT`.

To provide a data model for 3-tier entity hierarchy, we also define a composition relationship from booking to a booking supplement entity. In our example, we specify `/DMO/I_BookSuppl_M` as child entity in the composition `_BookSupplement`. The cardinality `[0 .. *]` expresses that any number of booking supplement instances can be assigned to each booking instance.

To be able to access data from other entities, a set of additional associations is defined in the CDS source code.

The `SELECT` list includes all elements of a booking entity that are relevant for consumption in a user interface.

i Expand the following code sample to view the source code of CDS View `/DMO/I_Booking_M`.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
//@Metadata.ignorePropagatedAnnotations:true
@EndUserText.label: 'Booking view'

define view entity /DMO/I_Booking_M
as select from /dmo/booking_m as Booking

    association          to parent /DMO/I_Travel_M      as _Travel          on $projection.travel_id = _
    composition [0..*] of /DMO/I_BookSuppl_M           as _BookSupplement

    association [1..1] to /DMO/I_Customer              as _Customer        on $projection.customer_id =
    association [1..1] to /DMO/I_Carrier                as _Carrier          on $projection.carrier_id =
    association [1..1] to /DMO/I_Connection             as _Connection      on $projection.carrier_id
                                                and $projection.connection_id
    association [1..1] to /DMO/I_Booking_Status_VH as _BookingStatus on $projection.booking_statu

{
    key travel_id,
    key booking_id,

    booking_date,
    customer_id,

```

```

    carrier_id,
    connection_id,
    flight_date,
    @Semantics.amount.currencyCode: 'currency_code'
    flight_price,
    currency_code,
    booking_status,

    //local ETag field --> OData ETag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    last_changed_at,

    /* Associations */
    _Travel,
    _BookSupplement,
    _Customer,
    _Carrier,
    _Connection,
    _BookingStatus
}

```

Booking Supplement View /DMO/I_BookSuppl_M

The 3-tier composition relationship requires an association for the booking supplement child entity /DMO/I_BookSuppl_M entity to their compositional parent entity.

Like the booking entity, the booking supplement entity also uses the field `last_changed_at` to store eTag values.

To access master data from other entities, additional associations are defined in the CDS source code.

i *Expand the following code sample to view the source code of CDS View /DMO/I_BookSuppl_M.*

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Booking Supplement View - CDS data model'

define view entity /DMO/I_BookSuppl_M
as select from /dmo/booksuppl_m as BookingSupplement

    association          to parent /DMO/I_Booking_M as _Booking          on $projection.travel_id =
    and $projection.booking_id =

    association [1..1] to /DMO/I_Travel_M          as _Travel          on $projection.travel_id =
    association [1..1] to /DMO/I_Supplement          as _Product          on $projection.supplement_id =
    association [1..*] to /DMO/I_SupplementText as _SupplementText on $projection.supplement_id =

{
    key travel_id,
    key booking_id,
    key booking_supplement_id,
    supplement_id,
    @Semantics.amount.currencyCode: 'currency_code'
    price,
    currency_code,

    //local ETag field --> OData ETag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    last_changed_at,

    /* Associations */
    _Travel,
    _Booking,
    _Product,
    _SupplementText
}

```

Defining Elementary Behavior for Ready-to-Run Business Object

In this step, we focus on modeling an elementary behavior in which only the standard operations `create()`, `update()`, and `delete()` are defined for each entity. These operations, along with some basic properties (behavior characteristics), are already sufficient to obtain a ready-to-run business object.

Creating a Behavior Definition /DMO/I_TRAVEL_M

Create the behavior definition by using the creation wizard in ADT. By using the context menu on the root view entity (`/DMO/I_Travel_M`), you can directly create a new behavior definition for the complete BO hierarchy. In the wizard, choose the implementation type `Managed`.

Modeling the Behavior for the Ready-to-Run Travel BO

As depicted in the codeblock below, the source code of the behavior definition consists of a header information and three definitions for entity behavior: one for the root travel entity and two for the child entities booking and booking supplements – corresponding to the composition tree of the business object.

For a detailed syntax description of a behavior definition header, see [CDS BDL - CDS behavior definition header \(ABAP Keyword Documentation\)](#).

For a detailed syntax description of an entity behavior definition, see [CDS BDL - entity behavior definition \(ABAP Keyword Documentation\)](#).

The header specifies managed implementation type of our business object's provider since we are going to implement all essential parts of an application from scratch.

Whereas for the unmanaged implementation type, the application developer must implement essential components of the REST contract manually, for the managed scenario, on the other hand, all required standard operations (create, update, delete) must only be specified in the behavior definition to obtain a ready-to-run business object. New applications can highly benefit from out-of-the-box support for transactional processing, since the technical implementation aspects are taken over by the business object runtime infrastructure. In this case, the business object framework implements generically the interaction phase and the save sequence. The application developer can then focus on business logic that is implemented using actions, validations, determinations and user interaction.

The strict mode ensures RAP best practices and up-to-date syntax for the behavior definition and behavior implementation. For more information, see [Strict Mode](#).

Our TRAVEL business object refers to the underlying CDS data model, which is represented by root entity `/DMO/I_Travel_M`. All data changes related to this entity that result from transactional behavior are stored in the database table `/DMO/TRAVEL_M`. This is the reason why this table is referenced as persistent table. The transactional handling of the business object's root entity travel is mainly determined by the standard operations `create`, `update`, and `delete`. The fact that in our scenario new instances of the booking child entity should also be created for a specific travel instance is considered by the addition of the `_Booking` association. The keyword `{ create; }` declares that this association is create-enabled what exactly means that instances of the associated bookings can only be created by a travel instance.

i Note

The `create_by_association` expects that the parent key fields of the child entity that is to be created are `read-only` as the parent key fields are filled automatically during runtime.

The sub node of the TRAVEL business object structure refers to the corresponding data model for bookings that is represented by the child entity `/DMO/I_Booking_M`. The transactional handling of booking child entity is determined by the standard operation

update and delete. In addition, the create-enabled association `_BookSupplement` is defined for creation of supplements as part of associated booking instances. Similarly, we model the behavior of the booking supplement entity `/DM0/I_BookSuppl_M`. Since we reach the end of the composition tree with this entity, there is no need to define a create-enabled association.

For each entity, the following behavior definitions are relevant:

When providing modifying operations, we also must take care for locking support for all relevant entities of the composition tree. For this purpose, the root entity `travel` is defined as `lock master` and the child entities as `lock dependent`. In the latter case, the binding information is specified via the association leading from child entity instance to its lock master instance. For more information, see [Pessimistic Concurrency Control \(Locking\)](#).

In the managed scenario, all entities receive a separate `eTag master` field to store `eTag` values. Like this, every entity can be accessed and modified independently of others. For more information, see [Optimistic Concurrency Control](#).

The `authorization master` defines how and when a RAP BO can be accessed. The strict mode requires that authorization is defined for the business object. With the addition `none`, no authorization checks are executed. This is a demo application, which is why the addition `none` is used. This approach is not recommended in real-world scenarios and violates best practices. For more information, see [Authorization Control](#).

For the fields that are used in the business object, a mapping prescription describes which fields of the database tables are to be used as fields in the business object. Since in this scenarios, the field names are the same, the corresponding operator can be used. For more information, see [Using Type and Control Mapping](#).

i Expand the following code sample to view the source code of the Behavior Definition `/DM0/I_Travel_M`.

Sample Code

```
managed;
strict(2);

define behavior for /DM0/I_Travel_M alias travel
persistent table /DM0/TRAVEL_M
etag master last_changed_at
lock master
authorization master ( none )

{
  mapping for /DM0/TRAVEL_M corresponding;

  create;
  update;
  delete;

  association _Booking { create; }

}

define behavior for /DM0/I_Booking_M alias booking
persistent table /DM0/BOOKING_M
etag master last_changed_at
lock dependent by _Travel
authorization dependent by _Travel

{
  field ( readonly ) travel_id;

  mapping for /DM0/BOOKING_M corresponding;

  update;

  association _BookSupplement { create; }
  association _Travel { }
```

```

    }

    define behavior for /DM0/I_BookSuppl_M alias booksuppl
    persistent table /DM0/BOOKSUPPL_M
    etag master last_changed_at
    lock dependent by _Travel
    authorization dependent by _Travel

    {
        field ( readonly ) travel_id, booking_id;

        mapping for /DM0/BOOKSUPPL_M corresponding;

        update;

        association _Travel { }
        association _Booking { }
    }

```

Testing the Business Object

At this point, you can now test the newly created travel business object for its basic functionality by creating some new instances for the 3 entities (travel, booking, booking supplement) that correspond to the business object's structure, modifying the existing data sets or deleting existing instances.

These options are available to you:

- Testing **with Fiori UI** by using the integrated Fiori UI preview function
- Testing **without Fiori UI**:
 - By executing automated ABAP Unit tests, starting from a generated test class
 - By consuming the newly created business object's entities with EML.

Procedures

To use the integrated Fiori UI preview, perform the following steps:

1. Add required UI annotations to the relevant CDS views.

Add, at least, the `@UI.identification` annotation on each element to be editable when creating or editing entity instances on the Fiori UI object pages.

2. Create the service definition.
3. Specify which CDS entities are exposed as a UI service.
4. Create a service binding for UI consumption.
5. Activate the UI service endpoint in the local service repository.
6. Run the resulting app by using the integrated UI preview function.

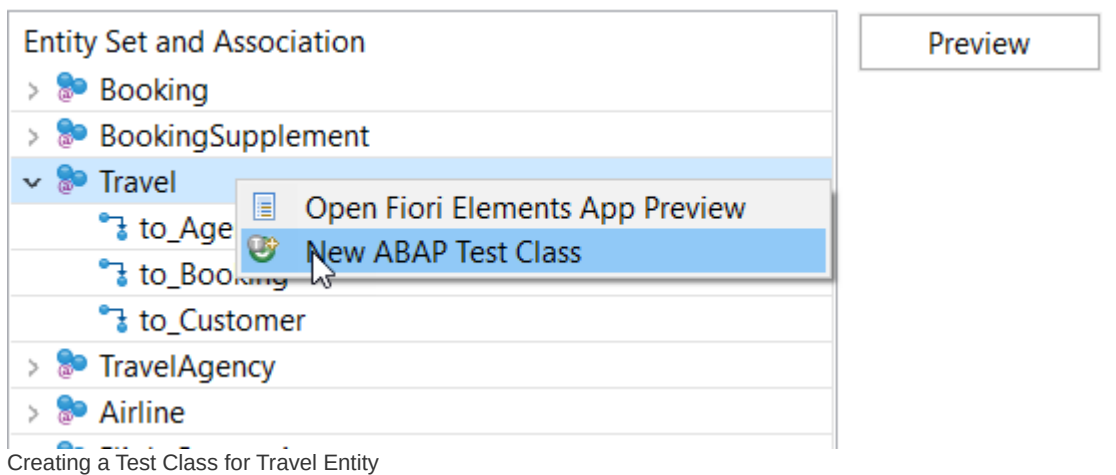
→ Tip

Since UI annotations are not yet defined in the CDS views provided, you can benefit from the generic templates by choosing the **Settings** (⚙️) button on the (initial) screen that appears. In the **Settings** dialog, click on the **Select all** box for **Columns** and confirm with **OK**. Then choose the **Go** button.

More on this: [Creating an OData Service](#) and [Designing the User Interface for a Fiori Elements App](#)

To generate a test class for an individual entity of the OData service, perform the following steps:

1. Create the service definition.
2. Specify which CDS entities are to be exposed as a service.
3. Create a service binding for service consumption.
4. Activate the service endpoint in the local service repository.
5. To launch the test class creation wizard, select the relevant entity in the service binding editor and choose **New ABAP Test Class** from context menu.



For the selected entity, the wizard creates a test class with a source code template for ABAP unit tests. After completing the test code, you can perform CUD operations on relevant entity.

i Note

You can either create a separate test class for each entity or copy and paste the generated code, then change the name of the entity accordingly for writing ABAP unit tests for other entity.

To test the business object functionality by implementing EML consumer class, you can, for example, proceed as follows:

1. Create an EML consumer class.
You can use, for example, the source code template from listing below.
2. Implement **MODIFY** calls for creating, updating, or deleting instances of business object's entities.

LISTING 1: Template for EML consumer class

```
CLASS MY_EML_CONSUMER_CLASS DEFINITION
  PUBLIC
  FINAL
  CREATE PUBLIC .
  PUBLIC SECTION.
    INTERFACES if_oo_adt_classrun.
  PROTECTED SECTION.
  PRIVATE SECTION.
ENDCLASS.

CLASS /dmo/cl_eml_travel_update IMPLEMENTATION.
```



```
METHOD if_oo_adt_classrun~main.
```

```
" To implement the MODIFY call, add EML code here!
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

→ Tip

To check the results of the MODIFY call implemented in the consumer class, run the main method of the consumer class by pressing **F9** key in the class editor and then search for the created, or updated (and deleted) travel, booking and booking supplement instances in the data preview tool (**F8**).

More on this: [Entity Manipulation Language \(EML\)](#) and [Examples for Accessing Business Objects with EML](#)

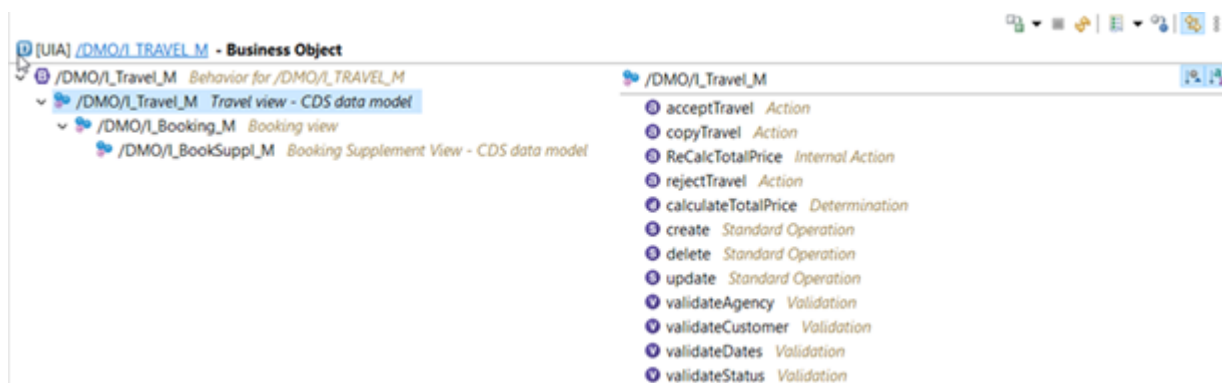
Developing Business Logic

The managed scenario addresses use cases where all essential parts of an application are to be developed from scratch. New applications like this can highly benefit from out-of-the-box support for transactional processing. The corresponding BO runtime manages the entire life cycle of your business objects and covers all aspects of your business application development.

In a managed scenario, the business object framework implements generically the interaction phase and the save sequence. You, as an application developer can then **focus on business logic** that is implemented by adding actions, validations, determinations, and user interaction.

Preview: Resulting Business Object in Relation Explorer

The figure below displays the resulting "Travel" business object in the **Relation Explorer** view. If you choose the **Business Object** context, the **Relation Explorer** provides the composition tree of the business object and displays all operations (including actions), determinations and validations defined by the selected entity.



"Travel" Business Object in Relation Explorer

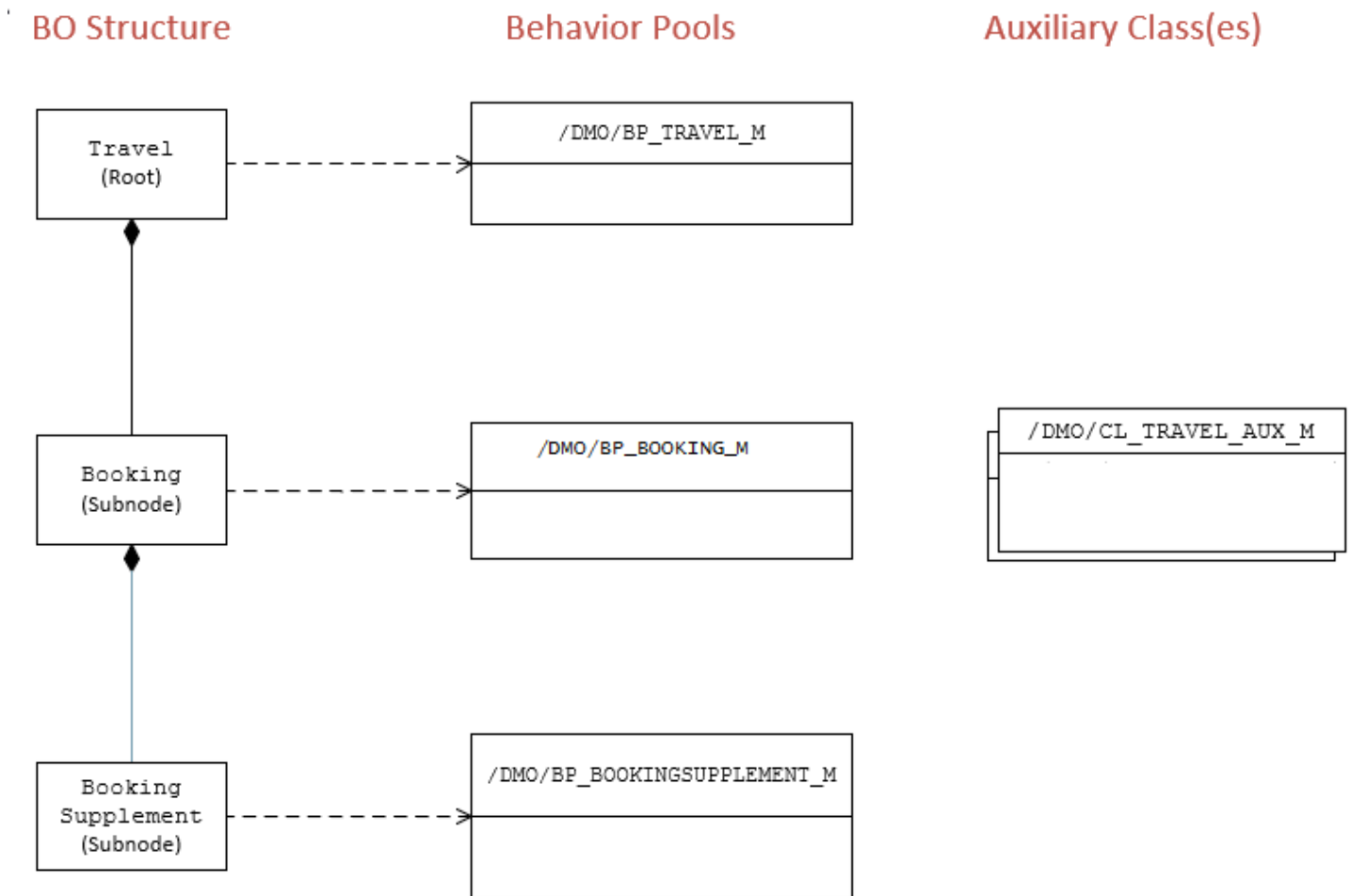
Creating ABAP Classes for Behavior Implementation

Until this step, we got by without any line of ABAP code. This was also not necessary, as in case of managed implementation type the technical implementation aspects are taken over by the business object runtime infrastructure itself. In this case, the business object framework implements generically the interaction phase and the save sequence.

However, to provide our application with specific business logic, we will on the one hand extend the behavior definition with actions, feature control, validations and determinations and on the other hand implement it in ABAP code.

In this step, you create the ABAP classes required for extending behavior artifacts of the corresponding behavior definition that you created earlier.

In doing so, we apply the contribution pattern and split the behavior implementation into different behavior pools, one for the travel root entity and the others for the booking and booking supplement child entities (as shown in figure below). In addition, we will make use of a separate auxiliary class for implementing helper methods (such as for methods reused in different handlers) that can be reused in each behavior implementation class.



Splitting Implementation into Different Behavior Pools and Auxiliary Classes

Creating the Behavior Pool /DMO/BP_TRAVEL_M

The behavior definition is directly linked to the implementation class. Therefore, it offers a quick fix to create a behavior pool with a template for methods to implement the corresponding behavior in the behavior definition.

Create the behavior pool for the travel entity with the following steps:

1. In the behavior definition header, after the implementation type, add the keywords `implementation in class /dmo/bp_travel_m unique;`. The specified implementation class will then be used for global, entity independent behavior.
2. As the behavior pool with the name `/dmo/bp_travel_m` does not exist yet, the behavior definition offers a quick fix to create it.

Follow the quick fix to start the wizard for creating an ABAP class.
3. Add the keywords `implementation in class /DMO/BP_TRAVEL_M unique` for the `travel` node. The specified implementation class will then be used for entity specific behavior.

≡, Sample Code

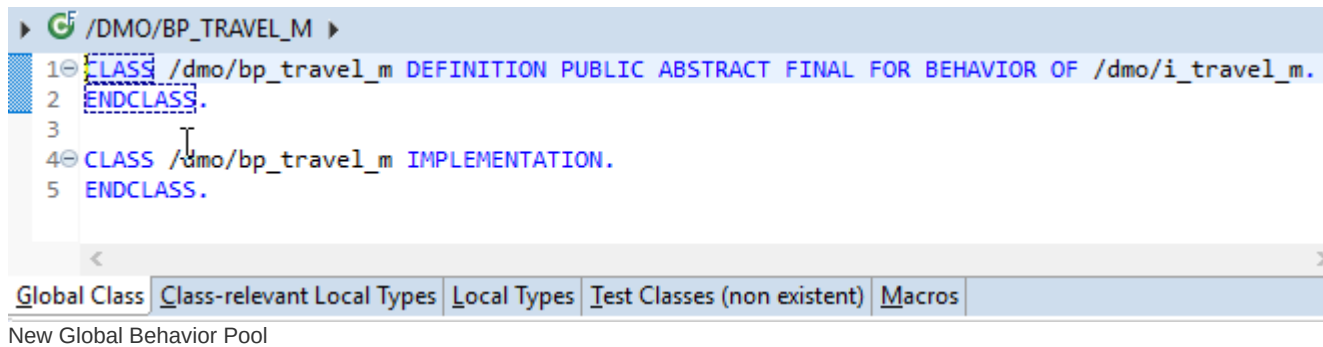
```

managed implementation in class /DMO/BP_TRAVEL_M unique;
strict(2);

define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
persistent table /DMO/TRAVEL_M
with additional save
etag master last_changed_at
lock master
early numbering
authorization master(global)

```

Compared to the standard ABAP class, the generated behavior pool (in our case /DMO/BP_TRAVEL_M) provides you with an extension FOR BEHAVIOR OF.



The real substance of a behavior pool is located in **Local Types** (there is currently no implementation yet). Here you can implement special local classes, namely handler classes for additional operations (such as actions), validations and determinations that are triggered at specific points in time within the interaction phase.

Note that these classes can be instantiated or invoked only by the ABAP runtime environment (virtual machine).

Creating the Behavior Pool /DMO/BP_BOOKING_M

Create another behavior pool class /DMO/BP_BOOKING_M for the booking entity, by doing the same steps as described for the travel entity.

```

CLASS /dmo/bp_booking_m DEFINITION PUBLIC ABSTRACT FINAL FOR BEHAVIOR OF /dmo/i_travel_m.
ENDCLASS.

```

```

CLASS /dmo/bp_booking_m IMPLEMENTATION.

```

```

ENDCLASS.

```

Creating the Behavior Pool /DMO/BP_BOOKINGSUPPLEMENT_M

Create another behavior pool class /DMO/BP_BOOKINGSUPPLEMENT_M for the booking supplement entity as described for the travel entity.

```

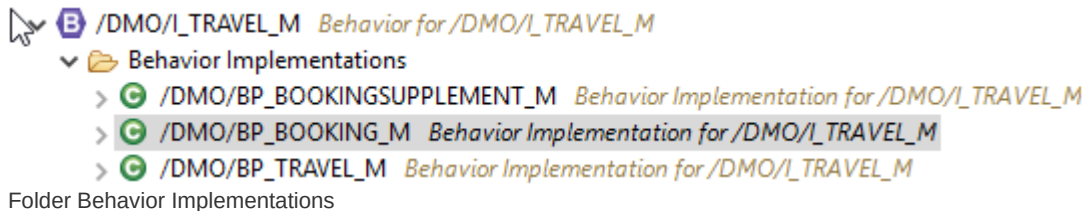
CLASS /dmo/bp_bookingsupplement_m DEFINITION PUBLIC ABSTRACT FINAL FOR BEHAVIOR OF /dmo/i_travel_m.
ENDCLASS.

```

```
CLASS /dmo/bp_bookingsupplement_m IMPLEMENTATION.
ENDCLASS.
```

Results

The newly created behavior pools are located in the **Behavior Implementations** folder of the corresponding behavior definition node **/DMO/I_TRAVEL_M**.



Developing Early Unmanaged Numbering

Early unmanaged numbering is used to automatically assign a key to a BO entity instance during the CREATE or CREATE - BY - ASSOCIATION. The key assignment is implemented in the FOR NUMBERING method in the behavior pool.

About Early Unmanaged Numbering

In this demo, early unmanaged numbering is implemented for the Travel, Booking and BookingSupplement business object entities. The early unmanaged numbering method is triggered during the CREATE or CREATE - BY - ASSOCIATION of a BO entity instance and a key is assigned based on the implementation in the FOR NUMBERING method. Every assigned key must be unique, otherwise the new instance is rejected by the database which results in a dump. It is recommended to use a numbering logic which guarantees that every drawn key is unique and also takes care of locking in case of parallel execution of a CREATE operation.

In this example, the key assignment on root entity level is implemented with a **Number Range Object**, which guarantees unique keys and a working locking mechanism during the number distribution so that a number isn't assigned twice. BO subnodes always have a combined key consisting of the keys of the parent entities and the key of the subnode itself. That's why it's sufficient to guarantee a unique key solely for one part of the composite key to guarantee that the whole key is unique. In this demo, a number range object on root entity level guarantees that all composite keys for all subnodes are unique. Consequently, the FOR NUMBERING implementation for a booking or a bookingSupplement instance only derives the maximum key already assigned and increases the key count for the current instance.

For more general information about early unmanaged numbering, see [Early Numbering](#) and for more details about the FOR NUMBERING method, see [<method> FOR NUMBERING](#).

Defining Early Unmanaged Numbering

Procedure: Adding Early Unmanaged Numbering to the Behavior Definition

Early unmanaged numbering is always defined on entity level and applied to the CREATE operation of the respective BO entity. Early unmanaged numbering is applied to all keys of a BO entity. That means that all keys must be mapped accordingly in the FOR NUMBERING method for all instances as soon as early unmanaged numbering is defined for a BO entity. The implementation is then called during every CREATE or CREATE - BY - ASSOCIATION operation to assign keys automatically during a BO instantiation. Since no external input is required, all key fields are defined as **readOnly** in the behavior definition.

Add early unmanaged numbering to the behavior definition of /DMO/I_TRAVEL_M corresponding to the following sample:

i Expand the following code sample to view the source code

☰ Sample Code

```
managed implementation in class /DMO/BP_TRAVEL_M unique;
strict(2);

define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
persistent table /DMO/TRAVEL_M
...
early numbering
...
{
    field ( readonly ) travel_id;
    ...
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
persistent table /DMO/BOOKING_M
etag master last_changed_at
lock dependent by _Travel
early numbering
...
{
    field ( readonly ) travel_id, booking_id;
    ...
}

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
persistent table /DMO/BOOKSUPPL_M
with unmanaged save
etag master last_changed_at
lock dependent by _Travel
early numbering
...
{
    field ( readonly ) travel_id, booking_id, booking_supplement_id;
}
```

Implementing Early Unmanaged Numbering

In this topic, we explain how you can implement early unmanaged numbering for your BO.

Travel

The early unmanaged numbering is implemented in the `earlynumbering_create` method within the handler class `lhc_travel` (local types) of the behavior pool `/DMO/BP_TRAVEL_M`.

Definition

In the behavior definition, the early unmanaged numbering for the travel entity is defined as follows:

```
managed implementation in class /DMO/BP_TRAVEL_M unique;
strict(2);

define behavior for /DMO/I_Travel_M alias travel
```

```

implementation in class /DMO/BP_TRAVEL_M unique
persistent table /DMO/TRAVEL_M
with additional save
etag master last_changed_at
lock master
early numbering
...
{
    field ( readonly ) travel_id;
    ...
}

```

Implementation

The early unmanaged numbering implementation on root node level must always be unique in order to guarantee unique keys for all child nodes. In this example, a `Number Range Object` is used to draw the keys for the travel entity. This guarantees, that every number is assigned only once even when the method is called from several services simultaneously. The number range can be defined freely - in this implementation, the count is always increased by 1.

(1) Because the `FOR NUMBERING` method is called twice in a managed scenario with draft capabilities (once during the `CREATE` and once during the save sequence), the implementation includes a mapping for travels that already have an ID, so that the ID assigned during the `CREATE` isn't overwritten during the save sequence. This is only a preparation step for later draft enablement and not required for an active-only managed BO.

Then travels which already have an ID are deleted from the list of travels. In the subsequent implementation, IDs are only assigned to travels that didn't have an ID before.

(2) The actual key assignment is implemented using the number range object `/DMO/TRV_M`. The method `number_get` has the following importing parameters:

- **nr_range_nr**: ID of the intervals that is used to assign the keys.
- **object**: Name of the number range object.
- **quantity**: Quantity of required number (number of travels without an ID in `entities_wo_travelid`)

The method returns the ID that is to be assigned, a return code that indicates if there was a problem with the numbers drawn and the actual quantity of IDs that were returned. You can implement the error handling according to your requirements - in this example, any problem with the number range object leads to an entry in the `failed` table to abort the `CREATE` and an error message to provide more details about the error occurred.

(3) The keys are then assigned and mapped in `mapped - travel`. If several numbers are requested, the `number` always returns the highest number and the starting point of the assignment is calculated by subtracting the requested number of numbers. The number count is then increased by 1 for each instance.

i Expand the following code sample to view the source code Of early unmanaged numbering for travel.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler

PRIVATE SECTION.
    METHODS earlynumbering_create FOR NUMBERING
    IMPORTING entities FOR CREATE travel.

ENDCLASS.

```

CLASS lhc_travel IMPLEMENTATION.

METHOD earlynumbering_create.

DATA:

```
entity          TYPE STRUCTURE FOR CREATE /DMO/I_Travel_M,
travel_id_max TYPE /dmo/travel_id.
```

```
" Ensure Travel ID is not set yet (idempotent)- must be checked when B0 is draft-enabled
LOOP AT entities INTO entity WHERE travel_id IS NOT INITIAL.
  APPEND CORRESPONDING #( entity ) TO mapped-travel.
ENDLOOP.
```

```
DATA(entities_wo_travelid) = entities.
DELETE entities_wo_travelid WHERE travel_id IS NOT INITIAL.
```

" Get Numbers

TRY.

```
cl_numberrange_runtime=>number_get(
  EXPORTING
    nr_range_nr      = '01'
    object           = '/DMO/TRV_M'
    quantity         = CONV #( lines( entities_wo_travelid ) )
  IMPORTING
    number           = DATA(number_range_key)
    returncode       = DATA(number_range_return_code)
    returned_quantity = DATA(number_range_returned_quantity)
).
```

CATCH cx_number_ranges INTO DATA(lx_number_ranges).

LOOP AT entities_wo_travelid INTO entity.

```
  APPEND VALUE #( %cid = entity-%cid
                  %key = entity-%key
                  %msg = lx_number_ranges
                  ) TO reported-travel.
```

```
  APPEND VALUE #( %cid = entity-%cid
                  %key = entity-%key
                  ) TO failed-travel.
```

ENDLOOP.

EXIT.

ENDTRY.

CASE number_range_return_code.

WHEN '1'.

" 1 - the returned number is in a critical range (specified under "percentage warning" i

LOOP AT entities_wo_travelid INTO entity.

```
  APPEND VALUE #( %cid = entity-%cid
                  %key = entity-%key
                  %msg = NEW /dmo/cm_flight_messages(
                          textid = /dmo/cm_flight_messages=>number_range_depleted
                          severity = if_abap_behv_message=>severity-warning )
                  ) TO reported-travel.
```

ENDLOOP.

WHEN '2' OR '3'.

" 2 - the last number of the interval was returned

" 3 - if fewer numbers are available than requested, the return code is 3

LOOP AT entities_wo_travelid INTO entity.

```
  APPEND VALUE #( %cid = entity-%cid
                  %key = entity-%key
                  %msg = NEW /dmo/cm_flight_messages(
                          textid = /dmo/cm_flight_messages=>not_sufficient_numbers
                          severity = if_abap_behv_message=>severity-warning )
                  ) TO reported-travel.
```

```
  APPEND VALUE #( %cid = entity-%cid
                  %key = entity-%key
                  %fail-cause = if_abap_behv=>cause-conflict
                  ) TO failed-travel.
```

ENDLOOP.

EXIT.

ENDCASE.

" At this point ALL entities get a number!

ASSERT number_range_returned_quantity = lines(entities_wo_travelid).

```

travel_id_max = number_range_key - number_range_returned_quantity.

" Set Travel ID
LOOP AT entities_wo_travelid INTO entity.
  travel_id_max += 1.
  entity-travel_id = travel_id_max .

  APPEND VALUE #( %cid = entity-%cid
                  %key = entity-%key
                  ) TO mapped-travel.

ENDLOOP.

ENDMETHOD.

ENDCLASS.

```

Booking

The early unmanaged numbering is implemented in the `earlynumbering_cba_booking` method within the handler class `lhc_travel` (local types) of the behavior pool `/DMO/BP_TRAVEL_M`.

Definition

In the behavior definition, the early unmanaged numbering for the booking entity is defined as follows:

```

managed;

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
persistent table /DMO/BOOKING_M
etag master last_changed_at
lock dependent by _Travel
early numbering
...
{
  field ( readonly ) travel_id, booking_id;
  ...
}

```

Implementation

The number range object for the travel entity guarantees unique keys for the root entity. For booking entities, the numbering starts from 0 for every travel entity.

(1) With a read-by-association all booking-ids are retrieved.

(2) The highest booking number for the current travel is retrieved.

(3) Bookings that already have an ID are assigned to `mapped-booking` and no new number is assigned. This can happen when internal operations execute the create-by-association that calls the early numbering exit then. In this scenario, this is the case with the `copyTravel` action that copies every `booking_id` of the source booking instances that are copied with their parent travel instance.

(4) The `max_booking_id` is increased by 10 for each new booking.

i Expand the following code sample to view the source code Of early unmanaged numbering for booking.

≡, Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler

  PRIVATE SECTION.
    METHODS earlynumbering_cba_booking FOR NUMBERING
      IMPORTING entities FOR CREATE travel\_booking.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.

  METHOD earlynumbering_cba_booking.
    DATA: max_booking_id TYPE /dmo/booking_id.

    READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
      ENTITY travel BY \_booking
      FROM CORRESPONDING #( entities )
      LINK DATA(bookings).

    " Loop over all unique TravelIDs
    LOOP AT entities ASSIGNING FIELD-SYMBOL(<travel>) GROUP BY <travel>-travel_id.

      " Get highest booking_id from existing bookings belonging to travel
      max_booking_id = REDUCE #( INIT max = CONV /dmo/booking_id( '0' )
                                FOR booking IN bookings USING KEY entity WHERE ( source-travel
                                NEXT max = COND /dmo/booking_id( WHEN booking-target-booking_id
                                                                THEN booking-target-booking_
                                                                ELSE max )
                                ).

      " Get highest assigned booking_id from incoming entities, eg from internal operations
      max_booking_id = REDUCE #( INIT max = max_booking_id
                                FOR entity IN entities USING KEY entity WHERE ( travel_id = <
                                FOR target IN entity-%target
                                NEXT max = COND /dmo/booking_id( WHEN target-booking_id > max
                                                                THEN target-booking_id
                                                                ELSE max )
                                ).

      " Assign new booking-ids if not already assigned
      LOOP AT <travel>-%target ASSIGNING FIELD-SYMBOL(<booking_wo_numbers>).
        APPEND CORRESPONDING #( <booking_wo_numbers> ) TO mapped-booking ASSIGNING FIELD-SYMBOL(
          IF <booking_wo_numbers>-booking_id IS INITIAL.
            max_booking_id += 10 .
            <mapped_booking>-booking_id = max_booking_id .
          ENDIF.
        ENDLOOP.

      ENDLOOP.
    ENDMETHOD.

  ENDCLASS.

```

Booking-Supplement

The early unmanaged numbering is implemented in the `earlynumbering_cba_booksupplem` method within the handler class `lhc_travel` (local types) of the behavior pool `/DMO/BP_BOOKING_M`.

Definition

In the behavior definition, the early unmanaged numbering for the `bookingsupplement` entity is defined as follows:

```
managed;
```

```
define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
```

```

persistent table /DMO/BOOKSUPPL_M
with unmanaged save
etag master last_changed_at
lock dependent by _Travel
early numbering
...
{
  field ( readonly ) travel_id, booking_id, booking_supplement_id;
}

```

Implementation

The number range object for the travel entity guarantees unique keys for the root entity. For booking supplements, the numbering starts from 0 for every bookingSupplement instance.

(1) With a read-by-association all supplement_ids are retrieved.

(2) The highest supplement number for the current booking is retrieved together with the corresponding travel_id.

(3) Bookings that already have an ID are assigned to mapped - booksuppl and no new number is assigned. This can happen when internal operations execute the create-by-association that calls the early numbering exit then. In this scenario, this is the case with the copyTravel action that copies every booking_supplement_id of the source booking supplement instances that are copied with their parent booking and travel instances.

(4) The max_booking_suppl_id is increased by 1 for each new booking.

i Expand the following code sample to view the source code of early unmanaged numbering for bookingSupplement.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

  ...
  METHODS earlynumbering_cba_booksupplem FOR NUMBERING
    IMPORTING entities FOR CREATE booking\_booksupplement.
  ...
ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.

  METHOD earlynumbering_cba_booksupplem.
    DATA: max_booking_suppl_id TYPE /dmo/booking_supplement_id .

    READ ENTITIES OF /dmo/i_travel_m IN LOCAL MODE
      ENTITY booking BY \_booksupplement
      FROM CORRESPONDING #( entities )
      LINK DATA(booking_supplements).

    " Loop over all unique tky (TravelID + BookingID)
    LOOP AT entities ASSIGNING FIELD-SYMBOL(<booking>) GROUP BY <booking>-%tky.

      " Get highest bookingsupplement_id from bookings belonging to booking
      max_booking_suppl_id = REDUCE #( INIT max = CONV /dmo/booking_supplement_id( '0' )
                                      FOR booksuppl IN booking_supplements USING KEY entity
                                      WHERE ( source-
                                              AND source-
                                              NEXT max = COND /dmo/booking_supplement_id( WHEN booksu
                                              THEN booksuppl-target-
                                              ELSE max )
                                      ).

      " Get highest assigned bookingsupplement_id from incoming entities

```

```

max_booking_suppl_id = REDUCE #( INIT max = max_booking_suppl_id
                                FOR  entity IN entities USING KEY entity
                                    WHERE (      travel_id = <booking
                                                AND booking_id = <booking
FOR  target IN entity-%target
NEXT max = COND /dmo/booking_supplement_id( WHEN  target
                                           THEN target
                                           ELSE max )
                                ).

```

```

" Assign new booking_supplement-ids
LOOP AT <booking>-%target ASSIGNING FIELD-SYMBOL(<booksuppl_wo_numbers>).
  APPEND CORRESPONDING #( <booksuppl_wo_numbers> ) TO mapped-booksuppl ASSIGNING FIELD-S
  IF <booksuppl_wo_numbers>-booking_supplement_id IS INITIAL.
    max_booking_suppl_id += 1 .
    <mapped_booksuppl>-booking_supplement_id = max_booking_suppl_id .
  ENDIF.
ENDLOOP.

```

```

ENDLOOP.
ENDMETHOD.

```

Developing Actions

An action is assigned to an individual entity of a business object and is used to implement a modifying non-standard operation as part of the business logic.

This demo scenario implements four actions for the travel entity.

The action `acceptTravel` sets the overall status of the travel entity to accepted (A). The action `rejectTravel` sets the overall status of the travel entity to rejected (X). These actions are instance actions as there is one specific instance of the travel entity for which the status is changed. They return exactly one instance, namely the instance on which the action is executed. The result is therefore defined as `result [1] $self`.

Standard*  

Not Filtered 

Travels (40)		Standard* 		Reject Travel	Accept Travel	
Travel ID	Overall Status	Agency ID	Customer ID	Booking Fee	Total Price	
 10005	 O	Happy Hopping (70003)	Buchholm (4)	20.00 USD	2,532.00 USD	
 10007	 O	Sunshine Travel (70001)	Neubasler (71)	33.00 EUR	6,865.42 EUR	
 10009	 O	Hot Socks Travel (70007)	Buchholm (7)	17.00 EUR	1,266.00 EUR	

Accept and Travel Action on the Fiori UI

The action `copyTravel` copies a travel instance as well as its associated booking and booking supplement instances and creates new instances based on the copied data. A new travel ID is assigned to the new travel instance by early unmanaged numbering. This action is an instance factory action as the end user has to select one instance which serves as the basis for the instance to be created. It returns exactly one instance of the same entity type the action is assigned for.

Standard ▾



Search

Agency ID:

Customer ID:

Adapt Filters

Go

Travels (4,167)

Standard

Copy Travel

Create

Delete

Show Details

Travel ID	Agency ID	Customer ID	Starting Date	Overall Status	
1	Maxitrip (70041)	Ryan (594)	Oct 24, 2020	X	>
2	Hot Socks Travel (70007)	Prinz (608)	Oct 24, 2020	O	>
3	Hendrik's (70046)	Detemple (93)	Oct 24, 2020	O	>

Copy Travel Action

The action `ReCalcTotalPrice` is used to calculate the total price of one travel BO instance, that is one travel entity instance with all its booking instances and the corresponding booking supplement instances. The relevant amounts of the booking fee, the flight price and the booking supplement price are added up in the field `total_price` of the travel instance. The action is not exposed to the UI, as it is only used internally and called by determinations, whenever one of the relevant fields is changed. The action does not have a result parameter so the UI does not navigate to the travel instance whenever the determination is triggered on one of the child instances.

For more information about actions in general, see [Actions](#).

Activities Relevant to Developers

1. [Defining Actions as Part of the Behavior Definition](#)
2. [Implementing Actions](#)
3. [Enabling Actions for UI Consumption](#)

Defining Actions as Part of the Behavior Definition

Actions are defined in the behavior definition.

Corresponding to the codeblock below, add the following actions to the behavior definition `/DMO/I_TRAVEL_M`.

All three actions `acceptTravel`, `rejectTravel`, and `copyTravel` have a similar syntax: they have no input parameters and the output parameter is the same entity for which the action is executed.

The action `ReCalcTotalPrice` is an internal action with no result parameter.

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
{
  ...
  // instance actions
  action acceptTravel result [1] $self;
  action rejectTravel result [1] $self;

  // instance factory action for copying travel instances
```

```

factory action copyTravel [1];

// internal action that is called by determinations
internal action ReCalcTotalPrice;
...
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
...
{...
// No actions defined for bookings
}

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
...
{...
// No actions defined for booking supplements
}

```

Implementing Actions

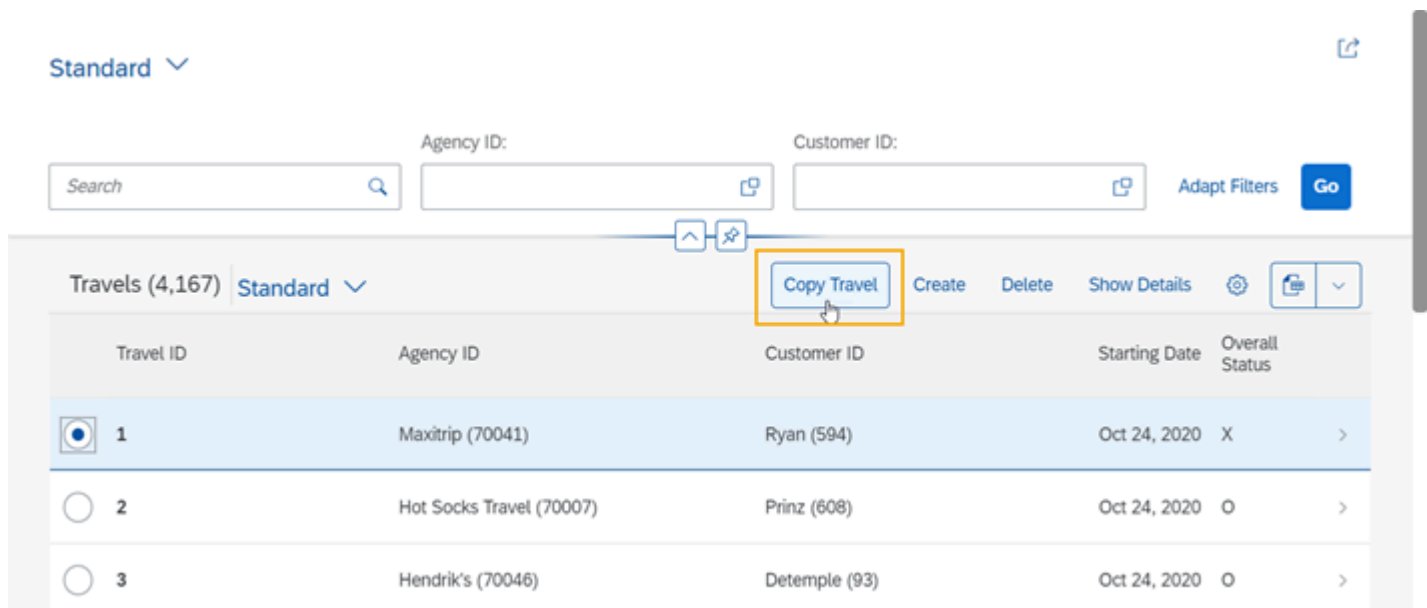
In this topic we demonstrate how you can implement the two actions `copyTravel` and `acceptTravel`.

Action `copyTravel`

This action copies a travel instance as well as its associated booking and booking supplement instances and creates new instances based on the copied data. A new travel ID is assigned to the new travel instance by early unmanaged numbering.

UI Preview

When we run the final app, the UI screen provides the button **Copy Travel** for the action as shown in the figure below.



Accessing the Action on Fiori UI

As soon as the user selects a travel and chooses this button, a copy of the travel instance and its associated booking and supplement instances are created and persisted in the corresponding database table.

Travels (4,182)

Standard

Copy Travel

Create

Delete

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Total Price	Overall Status	
35641593	Maxitrip (70041)	Ryan (594)	Feb 11, 2021	Mar 13, 2021	1,288.36 USD	O	>
35641592	Maxitrip (70041)	Ryan (594)	Feb 10, 2021	Mar 12, 2021	1,288.36 USD	O	>
35641591	Maxitrip (70041)	Ryan (594)	Feb 10, 2021	Mar 12, 2021	1,288.36 USD	O	>

Created Travel Instance

Definition

In the behavior definition, the action `copyTravel` is defined as follows:

```
define behavior for /DMO/I_Travel_M alias travel
{
  ...
  factory action copyTravel [1];
  ...
}
```

Implementing `copyTravel` Action in the Handler Class

The `copyTravel` action is implemented in the `copyTravel` method within the handler class `lhc_travel` which is part of the behavior pool `/DMO/BP_TRAVEL_M`.

The local handler class `lhc_travel` inherits from class `cl_abap_behavior_handler` and is automatically instantiated by the framework.

The signature of the `copyTravel` method includes the importing parameter `keys` for referring to the `travel` (root) instances, which contains the data to be copied. To identify the root entity, the alias `travel` is used - according to the alias that is specified in the behavior definition. The action is then addressed with `FOR ACTION travel~copyTravel`.

As given in the listing below, the basic structure of the `copyTravel` method includes:

- An assertion to ensure that the code is only called for action executions with filled import parameter `%cid`, which is mandatory for factory actions. If the consumer doesn't fill `%cid` the subsequent create-by-association will fail for all source travel instances that have child instances. Since an initial `%cid` is a programming error, an assertion is made.
- EML READ statements that read all fields of the selected travel instance and, if successful, all fields of its associated booking and booking supplement instances. The selected travel instance is identified by the importing parameter `keys`.
- LOOP statements in which structures and tables for the subsequent create and create-by-association operations are populated on the basis of the `%cid` to identify the instance, which is to be created. For the travel instance, the importing parameter `%cid` is used. The other content identifiers for the booking and booking supplements must be concatenated manually. In this implementation, the imported `%cid` is used and the corresponding id of the source instance is concatenated. The content identifier of the parent instances are then used as values for `%cid_ref` for the respective child entities. Some fields for the target instances are handled separately to meet the requirements for new travels, for example the travel dates that are checked by the validation `validateDates`.
- The copy action can fail when keys for the instance that is copied from are not found. These fail keys are appended to the methods output fail parameter.
- An EML MODIFY statement that creates the travel, booking and booking supplement instances based on the internal tables populated above. The new travel ID is assigned to the new travel instance by early unmanaged numbering. The content IDs of the instances and the corresponding keys provided by the create operations are stored in the `mapped_create`

structure of the MODIFY statement and are mapped to the determined key values. The modify operation with in local mode is not supposed to fail. Hence, in this case, there is no need for a failed data container.

- For the method changing parameter mapped - travel, the previously determined structures of the corresponding operations are assigned. The RAP contract for instance factory actions only allows one instance to be returned in mapped. Therefore, only the mapping condition for the new travel instance is returned by the method, but no mapping for the child entities.

i Expand the following code sample to view the source code of Action CopyTravel.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS copyTravel FOR MODIFY IMPORTING keys FOR ACTION travel~copyTravel RESULT result.
    ...

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  ...

  METHOD copyTravel.

    DATA:
      travels          TYPE TABLE FOR CREATE /DMO/I_Travel_M\\travel,
      bookings_cba     TYPE TABLE FOR CREATE /DMO/I_Travel_M\\travel\\booking,
      booksuppl_cba    TYPE TABLE FOR CREATE /DMO/I_Travel_M\\booking\\booksupplement.

    READ TABLE keys WITH KEY %cid = '' INTO DATA(key_with_initial_cid).
    ASSERT key_with_initial_cid IS INITIAL.

    READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY travel
    ALL FIELDS WITH CORRESPONDING #( keys )
    RESULT DATA(travel_read_result).

    READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY travel BY {color}_booking
    ALL FIELDS WITH CORRESPONDING #( travel_read_result )
    RESULT DATA(book_read_result).

    READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY booking BY {color}_booksupplement
    ALL FIELDS WITH CORRESPONDING #( book_read_result )
    RESULT DATA(booksuppl_read_result).

    LOOP AT keys INTO DATA(key).
      READ TABLE travel_read_result ASSIGNING FIELD-SYMBOL(<travel>) WITH KEY id COMPONENTS %tky
      IF sy-subrc EQ 0.
        "Fill travel container for creating new travel instance
        APPEND VALUE #( %cid = key-%cid
          %data = CORRESPONDING #( <travel> EXCEPT travel_id ) )
        TO travels ASSIGNING FIELD-SYMBOL(<new_travel>).

        "Fill %cid_ref of travel as instance identifier for cba booking
        APPEND VALUE #( %cid_ref = key-%cid )
        TO bookings_cba ASSIGNING FIELD-SYMBOL(<bookings_cba>).

        <new_travel>-begin_date = cl_abap_context_info=>get_system_date( ).
        <new_travel>-end_date = cl_abap_context_info=>get_system_date( ) + 30.
        <new_travel>-overall_status = '0'. "Set to open to allow an editable instance

      LOOP AT book_read_result ASSIGNING FIELD-SYMBOL(<booking>) USING KEY entity WHERE travel.
        "Fill booking container for creating booking with cba
        APPEND VALUE #( %cid = key-%cid && <booking>-booking_id
          %data = CORRESPONDING #( book_read_result[ KEY entity %tky = <booking>-%tky ] EXCEPT t
          TO <bookings_cba>-%target ASSIGNING FIELD-SYMBOL(<new_booking>).

```

```
"Fill %cid_ref of booking as instance identifier for cba booksuppl
APPEND VALUE #( %cid_ref = key-%cid && <booking>-booking_id )
TO booksuppl_cba ASSIGNING FIELD-SYMBOL(<booksuppl_cba>).

<new_booking>-booking_status = 'N'.

LOOP AT booksuppl_read_result ASSIGNING FIELD-SYMBOL(<booksuppl>) USING KEY entity WHE
AND booking_id EQ <booking>-booking_id.
  "Fill booksuppl container for creating supplement with cba
  APPEND VALUE #( %cid = key-%cid && <booking>-booking_id && <booksuppl>-booking_suppl
    %data = CORRESPONDING #( <booksuppl> EXCEPT travel_id booking_id ) )
  TO <booksuppl_cba>-%target.
ENDLOOP.
ENDLOOP.

ELSE.
  APPEND CORRESPONDING #( key MAPPING %fail = DEFAULT VALUE #( cause = if_abap_behv=>cause
  ENDIF.
ENDLOOP.

"create new B0 instance
MODIFY ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
ENTITY travel
  CREATE FIELDS ( agency_id customer_id begin_date end_date booking_fee total_price curren
    WITH travels
  CREATE BY \_Booking FIELDS ( booking_id booking_date customer_id carrier_id connection_i
    WITH bookings_cba
ENTITY booking
  CREATE BY \_BookSupplement FIELDS ( booking_supplement_id supplement_id price currency_c
    WITH booksuppl_cba
  MAPPED DATA(mapped_create).

mapped-travel    = mapped_create-travel .

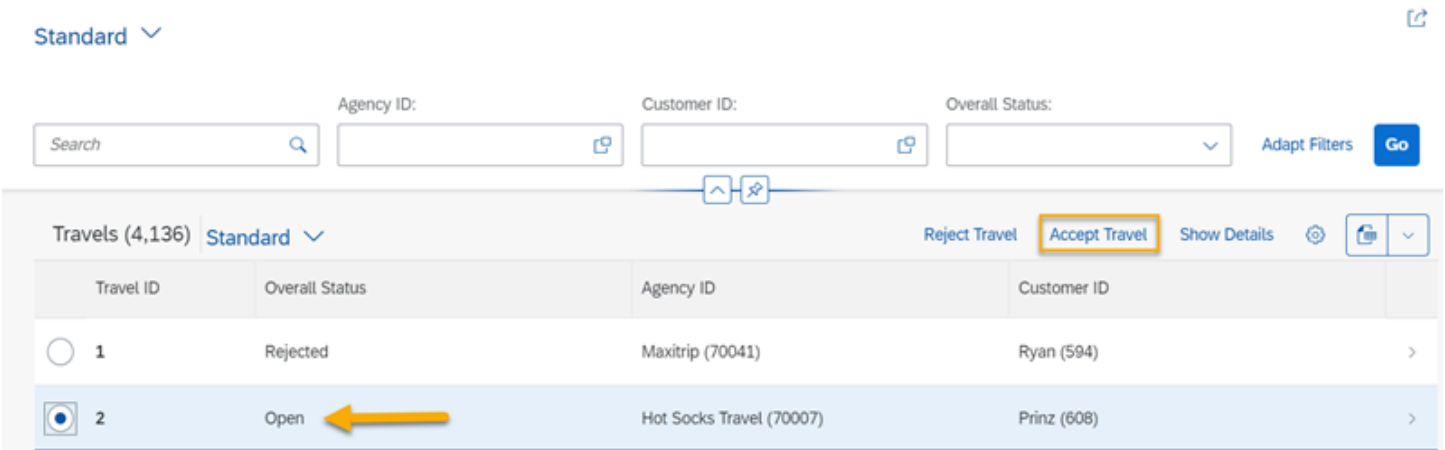
ENDMETHOD.
```

Action acceptTravel

This action provides the end user with the option of accepting individual travels without switching to EDIT mode.

UI Preview

If you run the app, the resulting UI screen provides you with the label **Accept Travel** for the new action - as shown in the figure below. The action is equipped with feature control. Only if the instance's status is not already **Accepted** can the action be executed. For implementation details, see [Adding Static and Dynamic Feature Control](#).



Action on Fiori UI

Standard Not Filtered

Travels (4,136) Standard Reject Travel Accept Travel Show Details

Travel ID	Overall Status	Agency ID	Customer ID
9	Accepted	Sunny, Sunny, Sunny (70032)	Benz (115)

Changed Status as a Result of Action Execution

Definition

Remember, in the behavior definition, the action `acceptTravel` is defined as follows:

```
define behavior for /DMO/I_Travel_M alias travel
{
    ...

    action acceptTravel result [1] $self;
    ...
}
```

Implementing `acceptTravel` Action in the Handler Class

The `acceptTravel` action is implemented in the `set_status_accepted` method within the handler class `lhc_travel` which belongs to **local types** component of the behavior pool `/DMO/BP_TRAVEL_M`.

To update `overall_status` data of a selected travel instance, the operation `UPDATE` is specified in the `EML MODIFY` statement. Note that the modifying operation takes place in `LOCAL MODE`. As a result, the update operation is excluded from the authorization checks (that may be implemented later on). An EML modify request in local mode for a managed BO provider is not supposed to fail. Therefore, the `failed` parameter does not need to be retrieved.

A subsequent EML `READ` statement provides the instance data for the result of the action.

i Expand the following code sample to view the source code of Action `AcceptTravel`.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
    METHODS set_status_accepted FOR MODIFY IMPORTING keys FOR ACTION travel~acceptTravel RESULT re
    ...
ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    ...

    METHOD set_status_accepted.
        " Modify in local mode: BO-related updates that are not relevant for authorization checks
        MODIFY ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY travel
                UPDATE FIELDS ( overall_status )
                    WITH VALUE #( FOR key IN keys ( %tky          = key-%tky
                                                            overall_status = 'A' ) ). " Accepted

        " Read changed data for action result
        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY travel
```

```

ALL FIELDS WITH
CORRESPONDING #( keys )
RESULT DATA(travels).

```

```

result = VALUE #( FOR travel IN travels ( %tky      = travel-%tky
                                           %param    = travel ) ).

```

```
ENDMETHOD.
```

```
ENDCLASS.
```

→ Tip

The implementation of the `rejectTravel` action is done in analogy to the `acceptTravel` action in the handler class `lhc_travel`.

Action ReCalcTotalPrice

The action is called by the determinations whenever a relevant amount field is changed to calculate the total travel price based on the amounts in the relevant instances.

As this action is an internal action that can only be called from inside the business object, it is not exposed to the UI.

Definition

The action is defined in the behavior definition as follows:

```

define behavior for /DMO/I_Travel_M alias travel
...
{
...
internal action ReCalcTotalPrice;
...
}

```

Implementing ReCalcTotalPrice

The `ReCalcTotalPrice` action is implemented in the method with the same name in the handler class `lhc_travel`.

The calculation of the total price is done for each travel instance separately. EML READ statements read all the requested travel instances with their booking fees and currency codes, the associated bookings with their flight prices and currency codes, and their associated booking supplements with the price and currency codes. A loop on the result table `travels` ensures that the amounts are summed for the corresponding travel instance. In the loop, the prices of all associated bookings and their associated booking supplements are added to the travel instance's booking fee if it has the same currency code. If the currency code differs the public method for currency conversion is called to get all flight prices in the currency of the travel's booking fee.

In the end, the calculated total price for the instance is modified via an EML modify statement that update the travel instance.

→ Remember

An EML modify statement in local mode in a managed BO is not supposed to fail. Authorization, feature and field control is not applied to EML calls in local mode. Therefore, there is no need to retrieve the `failed` or `reported` parameter.

i *Expand the following code sample to view the source code of Action `ReCalcTotalPrice`.*

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.

    METHODS recalctotalprice FOR MODIFY
        IMPORTING keys FOR ACTION travel~recalctotalprice.
    ...

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    ...

    METHOD recalctotalprice.
        TYPES: BEGIN OF ty_amount_per_currencycode,
            amount      TYPE /dmo/total_price,
            currency_code TYPE /dmo/currency_code,
        END OF ty_amount_per_currencycode.

        DATA: amounts_per_currencycode TYPE STANDARD TABLE OF ty_amount_per_currencycode.

        " Read all relevant travel instances.
        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY travel
                FIELDS ( booking_fee currency_code )
            WITH CORRESPONDING #( keys )
            RESULT DATA(travels).

        DELETE travels WHERE currency_code IS INITIAL.

        " Read all associated bookings and add them to the total price.
        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY travel BY \_booking
                FIELDS ( flight_price currency_code )
            WITH CORRESPONDING #( travels )
            RESULT DATA(bookings).

        " Read all associated booking supplements and add them to the total price.
        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY booking BY \_booksupplement
                FIELDS ( price currency_code )
            WITH CORRESPONDING #( bookings )
            RESULT DATA(bookingsupplements).

        LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
            " Set the start for the calculation by adding the booking fee.
            amounts_per_currencycode = VALUE #( ( amount      = <travel>-booking_fee
                                                  currency_code = <travel>-currency_code ) ).

            LOOP AT bookings INTO DATA(booking) USING KEY id WHERE      travel_id = <travel>-travel_id
                AND      currency_code IS NOT INITIAL.
                COLLECT VALUE ty_amount_per_currencycode( amount      = booking-flight_price
                                                            currency_code = booking-currency_code
                                                            ) INTO amounts_per_currencycode.
            ENDLOOP.

            LOOP AT bookingsupplements INTO DATA(bookingsupplement) USING KEY id WHERE      travel_id = <
                AND      currency_code
                COLLECT VALUE ty_amount_per_currencycode( amount      = bookingsupplement-price
                                                            currency_code = bookingsupplement-currency_cod
                                                            ) INTO amounts_per_currencycode.
            ENDLOOP.

            DELETE amounts_per_currencycode WHERE currency_code IS INITIAL.

            CLEAR <travel>-total_price.
            LOOP AT amounts_per_currencycode INTO DATA(amount_per_currencycode).
                " If needed do a Currency Conversion
                IF amount_per_currencycode-currency_code = <travel>-currency_code.
                    <travel>-total_price += amount_per_currencycode-amount.

```

```

ELSE.
  /dmo/cl_flight_amdp=>convert_currency(
    EXPORTING
      iv_amount                = amount_per_currencycode-amount
      iv_currency_code_source  = amount_per_currencycode-currency_code
      iv_currency_code_target  = <travel>-currency_code
      iv_exchange_rate_date    = cl_abap_context_info=>get_system_date( )
    IMPORTING
      ev_amount                = DATA(total_booking_price_per_curr)
  ).
  <travel>-total_price += total_booking_price_per_curr.
ENDIF.
ENDLOOP.
ENDLOOP.

" write back the modified total_price of travels
MODIFY ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
ENTITY travel
  UPDATE FIELDS ( total_price )
  WITH CORRESPONDING #( travels ).

ENDMETHOD.

ENDCLASS.

```

Enabling Actions for UI Consumption

On the UI, actions are triggered by choosing an action button. This action button must be configured in the backend in the related CDS view. Depending on where you want to use an action button on the UI (list report, or object page), use the annotation `@UI.LineItem` or `@UI.identification` to display an action button.

i Note

The UI-annotations for actions must be used as element annotation. However, it does not have an impact on which element the action is annotated. The action button always appears at the same position on the UI.

You can use the following UI annotation to expose actions to the consumer:

Syntax

```

define view <CDSEntity> as select from <DATA_SOURCE> as ..
{
  @UI.LineItem: [ ...
    { type: #FOR_ACTION, dataAction: 'action_1', label: 'label 1', position: 10 },
    { type: #FOR_ACTION, dataAction: 'action_2', label: 'label 2', position: 20 },
    ... ]
  @UI.identification: [ ...
    { type: #FOR_ACTION, dataAction: 'action_1', label: 'label 1', position: 10 },
    { type: #FOR_ACTION, dataAction: 'action_2', label: 'label 2', position: 20 },
    ... ]
}

```

i Note

The `dataAction` element references the name of the action as it is defined in the behavior definition. In case of using external names and having at least two determine actions defined, for `dataAction`, use the external name for the action to be in consistency with `$Metadata`.

Adding Static and Dynamic Feature Control

As an application developer you may want to determine, which entities of your business object should be create-, delete- and update-enabled, so that they can be modified during consumption using EML or OData services. In addition, you may also want to control which (UI) fields of an entity are read-only or which actions in which usage scenarios are enabled or disabled for execution by the end users.

In ABAP RESTful Application Programming Model feature control is precisely the way to accomplish such tasks. It allows you to control the visibility and changeability of fields, operations or entire entities.

The availability of feature control values is modeled in a behavior definition. Unlike static feature control, instance feature control requires not only a definition but also an implementation in a handler class of the behavior pool. Therefore, we also talk about **dynamic feature control** in case of instance feature control.

Feature control can be related to an entire entity or to individual elements of an entity, such as individual fields or operations. For more information, see [Feature Control](#).

Activities Relevant to Developers

1. [Modeling Static and Dynamic Feature Control](#)
2. [Implementing Dynamic Feature Control](#)

Modeling Static and Dynamic Feature Control

Both, static and dynamic feature control is defined for different levels (entity, field, or action level) in the behavior definition.

For a description on field characteristics, see [CDS BDL - field characteristics, projection BDEF \(ABAP Keyword Documentation\)](#).

For a description on dynamic feature control, see [CDS BDL - feature control \(ABAP Keyword Documentation\)](#).

Adding Feature Control to Behavior Definition /DMO/I_Travel_M

Corresponding to the coding below, add the static and dynamic feature control to each entity in the behavior definition.

i Expand the following code sample to view the source code of defining the behavior for /DMO/I_Travel_M.

Sample Code

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
etag master last_changed_at
...
{
  // administrative fields: read only
  field ( readonly ) last_changed_at, last_changed_by, created_at, created_by;

  // mandatory fields that are required to create a travel
  field ( mandatory ) agency_id, customer_id, begin_date, end_date, overall_status, booking_fee,

  // Semantic Key field, which is readonly for the consumer, value is assigned in early numberin
  field ( readonly ) travel_id;

  // dynamic action control
  action ( features: instance ) acceptTravel result [1] $self;
  action ( features: instance ) rejectTravel result [1] $self;
```

```

// dynamic operation control
association _Booking { create (features:instance); }
...
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
etag master last_changed_at
...
{
// administrative fields: read only
field ( readonly ) last_changed_at;

// static field control
field ( mandatory ) carrier_id, connection_id, flight_date, booking_status;
field ( readonly ) travel_id, booking_id;

// Fields that are mandatory for create but should be read-only afterwards
field ( mandatory : create, readonly : update) booking_date, customer_id;

// dynamic operation control
association _BookSupplement { create (features:instance); }
...
}

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
...
{
// administrative fields: read only
field ( readonly ) last_changed_at;

// static field control
field ( mandatory ) price,supplement_id;
field ( readonly ) travel_id, booking_id, booking_supplement_id;
...
}

```

For the travel entity, we define all admin fields as read-only, whereas the fields that are required for creating a travel instance are defined as mandatory.

The key field `travel_id` is also set to `readonly` as the value is determined by early numbering. The consumer of the BO must not be able to set the travel ID manually. The same is true for the other key fields of the child entities and some other booking-specifics fields. This kind of field control is static, which means it does not have to be implemented in the corresponding feature control implementation.

For ETag fields, as dedicated administrative fields, it is recommended that you specify the respective fields as read-only.

Examples of dynamic action control are the two methods `acceptTravel` and `rejectTravel`. Depending on the status value (`overall_status`), these actions can become enabled or disabled.

Creating bookings by association is dynamically controlled. You can only create new bookings if the `overall_status` of the corresponding travel instance is not rejected.

Implementing Dynamic Feature Control

The implementation of dynamic feature control is based on the ABAP language in a local handler class (`lhc_handler`) as part of the behavior pool. For more information about instance dynamic feature control, see [Instance Feature Control](#).

Feature Control for Travel Entity

In the following step, we will apply the feature control specifically to our demo scenario.

UI Preview

The figure below shows the realization of the static and dynamic field control using the example of the travel object page that has been switched to edit mode. All mandatory fields are marked with a red star.

Travel ID: 2

Starting Date: * Jan 7, 2023

Total Price: 900.00 USD

Agency ID: * Hot Socks Travel (70007)

End Date: * Jan 7, 2023

Status: * Open

Customer ID: * Prinz (608)

Booking Fee: * 20.00 USD

Description: Business Trip for Christine, Pierre

Static and Dynamic Feature Control on Field Level

The following figure shows the effect of dynamic control on the two action buttons **Accept Travel** and **Reject Travel**: Since the selected travel instance has the status **Accepted**, the action button **Accept Travel** is disabled.

Standard * ▾

No filters active

Travels (4,137)

Reject Travel Accept Travel

Travel ID	Overall Status	Agency ID	Customer ID
☒ 9	Accepted	Sunny, Sunny, Sunny (70032)	Benz (115)
☐ 10	Open	Ali's Bazar (70016)	Leisert (697)

Dynamic Feature Control for Actions

Definition

In the behavior definition, the feature control for travel entity is defined as follows:

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
...
{
  // administrative fields: read only
  field ( readonly ) last_changed_at, last_changed_by, created_at, created_by;

  // mandatory fields that are required to create a travel
  field ( mandatory ) agency_id, customer_id, begin_date, end_date, overall_status, booking_fee, ci

  // Semantic Key field, which is readonly for the consumer, value is assigned in early numbering
  field ( readonly ) travel_id;

  // instance action and dynamic action control
  action ( features: instance ) acceptTravel result [1] $self;
  action ( features: instance ) rejectTravel result [1] $self;

  association _Booking { create (features:instance); }
...
}
```

Implementing Dynamic Feature Control for Travel Entity

The method implementation begins with reading the `travel_id` and the `overall_status` field that is designated for dynamic field control. The result of this read operation is stored in `travels`.

Since the feature control implementation is called before any other modification request, this EML read request might fail if the imported keys parameter contains invalid key values. Therefore the changing parameter `failed` is retrieved and its content directly written to the method's `failed` parameter.

Depending on the value of `overall_status` field, the actions `rejectTravel` and `acceptTravel` are enabled or disabled, and the create by association is possible or not.

i *Expand the following code sample to view the source code of Feature Control in /DMO/I_Travel_M.*

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.
    ...

    METHODS get_features FOR INSTANCE FEATURES
        IMPORTING keys REQUEST requested_features FOR travel RESULT result.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    ...

    METHOD get_features.

        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY travel
                FIELDS ( travel_id overall_status )
                WITH CORRESPONDING #( keys )
            RESULT DATA(travels)
            FAILED failed.

        result = VALUE #( FOR travel IN travels
            ( %tky                                     = travel-%tky
              %features-%action-rejecttravel = COND #( WHEN travel-overall_status = '
                                                         THEN if_abap_behv=>fc-o-disabl
              %features-%action-accepttravel = COND #( WHEN travel-overall_status = '
                                                         THEN if_abap_behv=>fc-o-disabl
              %assoc-_booking                = COND #( WHEN travel-overall_status = '
                                                         THEN if_abap_behv=>fc-o-disabl
            ) ).

    ENDMETHOD.

ENDCLASS.

```

Dynamic Feature Control for Booking Entity

UI Preview

The following figure shows the effect of static and dynamic field control after switching the Fiori UI object page for bookings in edit mode.

Booking Booking Supplement

Booking Number: 2	Airline ID: * American Airlines Inc. (AA)	Flight Price: 438.00 USD
Booking Date: Jan 7, 2023	Flight Number: * 322	Status: * New
Customer ID: Ryan (594)	Flight Date: * Jan 9, 2023	

Static and Dynamic Feature Control at Field Level

Definition

```
define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
...
{
  // administrative fields: read only
  field ( readonly ) last_changed_at;

  // static field control
  field ( mandatory ) carrier_id, connection_id, flight_date, booking_status;
  field ( readonly ) travel_id, booking_id;

  // Fields that are mandatory for create but should be read-only afterwards
  field ( mandatory : create, readonly : update) booking_date, customer_id;

  // dynamic feature control create booking supplement by association
  association _BookSupplement { create (features:instance); }
  ...
}
```

Implementing Dynamic Feature Control for Booking Entity

The `get_features` method implements dynamic field control for the create booking supplements by association. Like the feature control for the travel entity, the affected booking instances must be read to retrieve the values of the fields that are relevant for the feature control. Then the feature control values must be assigned.

i *Expand the following code sample to view the source code of Feature Control in the Booking Entity.*

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
...

  IMPORTING keys FOR booking~validatestatus.
  METHODS get_features FOR INSTANCE FEATURES

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
...

  READ ENTITIES OF /dmo/i_travel_m IN LOCAL MODE
  ENTITY booking
    FIELDS ( booking_id booking_status )
    WITH CORRESPONDING #( keys )
  RESULT DATA(bookings)
  FAILED failed.

  result = VALUE #( FOR booking IN bookings
    ( %tky = booking-%tky
      %assoc-_booksupplement = COND #( WHEN booking-booking_status = 'B'
        THEN if_abap_behv=>fc-o-disabled ELSE
```

Developing Validations

For the managed scenario, add validations to check the values provided by the client.

Context and Procedure

Validations are used to check whether provided values by a client are consistent. They give direct feedback (messages) before the BO instance is saved to the database. For conceptual information about validations, see [Validations](#).

For this demo scenario, define and implement validations for the entities **travel** and **booking**.

- Entity **travel**
 - The validation `validateCustomer` checks if the customer ID that is entered by the consumer is valid.
 - The validation `validateAgency` checks if the agency ID that is entered by the consumer is valid.
 - The validation `validateDates` checks if the `begin_date` is in the future and if the value of the `end_date` is after the `begin_date`.
 - The validation `validateStatus` checks if the value of the `overall_status` field is valid.
 - The validation `validateCurrencyCode` checks if the value the `currency_code` field is valid.
 - The validation `validateBookingFee` checks if the `booking_fee` field is valid.
- Entity **booking**
 - The validation `validateStatus` checks if the `booking_status` field is valid.
 - The validation `validateCurrencyCode` checks if the `currency_code` field is valid.
 - The validation `validateCustomer` checks if the `customer_id` field is valid.
 - The validation `validateConnection` checks if the fields `carrier_id`, `connection_id` and `flight_date` have been provided and if they refer to an existing flight.
 - The validation `validateFlightPrice` checks if the `flight_price` field is valid.
- Entity **booksuppl**
 - The validation `validateSupplement` checks if a supplement ID has been provided by the consumer and if the provided supplement ID is valid.
 - The validation `validateCurrencyCode` checks if the `currency_code` field is valid.
 - The validation `validatePrice` checks if the `price` field is valid.

Defining Validations

Validations are defined in the behavior definition.

As depicted in the following sample code, add the following validations to each entity in the behavior definition.

i *Expand the following code sample to view the source code*

Sample Code

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
...
{
    ...
    validation validateCustomer      on save { create; field customer_id; }
    validation validateAgency       on save { create; field agency_id; }
    validation validateDates         on save { create; field begin_date, end_date; }
    validation validateStatus        on save { create; field overall_status; }
    validation validateCurrencyCode  on save { create; field currency_code; }
    validation validateBookingFee    on save { create; field booking_fee; }
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
...

{
    ...
    validation validateStatus        on save { create; field booking_status; }
    validation validateCustomer      on save { create; field customer_id; }
    validation validateCurrencyCode  on save { create; field currency_code; }
    validation validateConnection    on save { create; field carrier_id, connection_id, flight_date }
    validation validateFlightPrice   on save { create; field flight_price; }
}

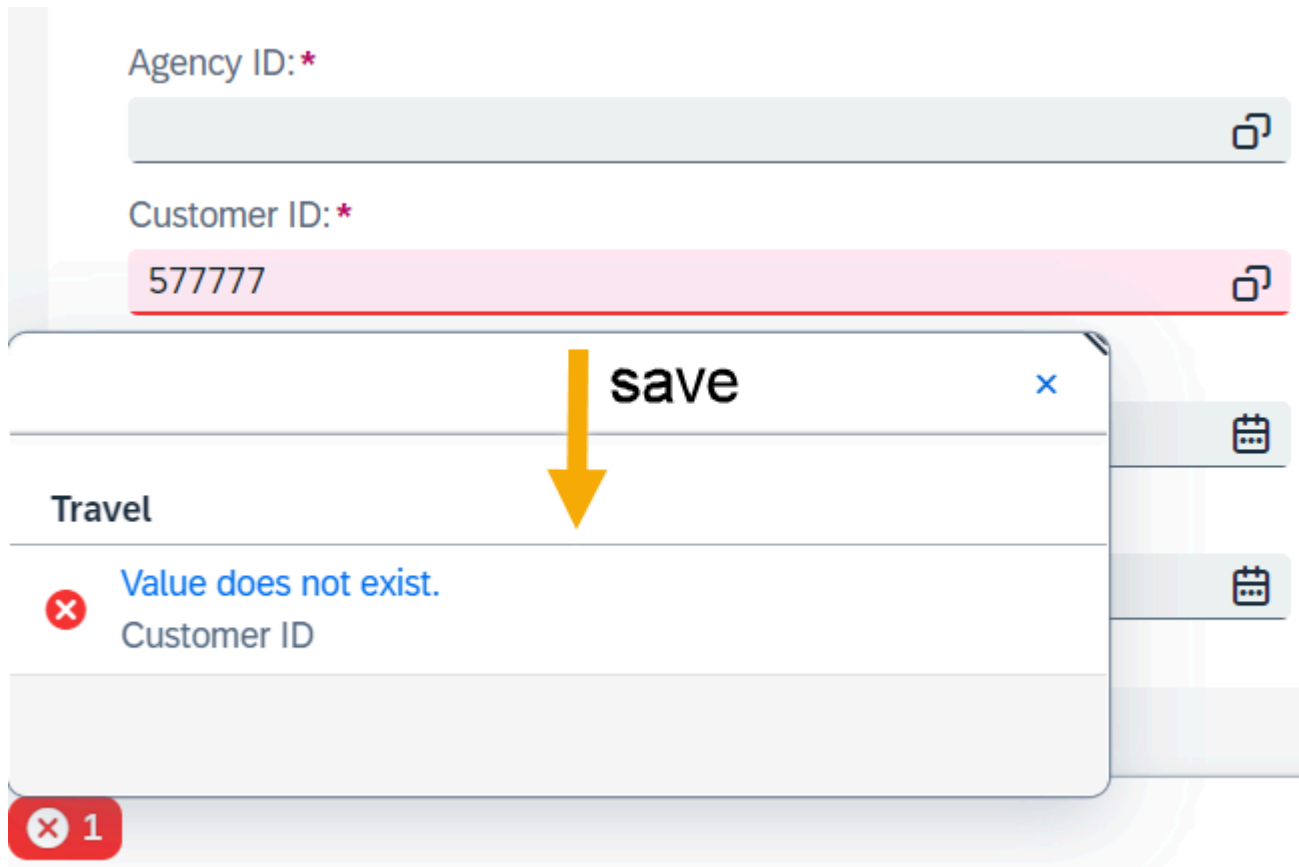
define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
...
{
    ...
    validation validateCurrencyCode  on save { create; field currency_code; }
    validation validateSupplement     on save { create; field supplement_id; }
    validation validatePrice          on save { create; field price; }
}
```

Implementing Validations

Validations on Travel Entity

UI Preview

If the user enters an invalid Customer ID (an ID that is not available in the customer database table `/DMO/Customer`) the validation is initiated at the save time. As a result saving the instance data is rejected and a corresponding message is returned to the user.



Customer ID field validation on Fiori UI

Definition

In the behavior definition, the validations on the travel entity are defined as follows:

```
define behavior for /DMO/I_Travel_M alias travel
...
{
...
validation validateCustomer on save { create; field customer_id; }
validation validateAgency    on save { create; field agency_id; }
validation validateDates     on save { create; field begin_date, end_date; }
validation validateStatus    on save { create; field overall_status; }
validation validateCurrencyCode on save { create; field currency_code; }
validation validateBookingFee  on save { create; field booking_fee; }
}
```

Implementation of validateCustomer

It should come as no surprise that the signatures of all methods for implementing validations are very similar.

In the following codeblock, you see details about the implementation of the `validate_customer` method: First, the EML read operation `READ ENTITY` provides read access to the selected travel instance by using the key.

`FIELDS ( )` addition is used. As a result of the read operation the entered (changed) value of the To access data of the relevant entity fields, the `customer_id` field for the selected travel instance are written into the table `travels` To access data of the relevant. Only this value is relevant for the validation. relevant for the validation.

In the following lines of code, we prepare an optimization for the following database select. By using the sorted table `customers`, we ensure that only data records with non-initial customer IDs are considered for database access.

By accessing the contents of the database table `/dmo/customer`, we can check whether the entered customer ID exists on the database at all.

If the validation detects inconsistencies (customer ID is not valid or is initial), we must provide the key of all inconsistent instances as failed key and return error messages to the consumer. For all failed instances, also corresponding messages are created, which is done with the NEW statement. For access to suitable message texts, a message class /dmo/cm_flight_messages from the /DMO/FLIGHT package is used. For more information, about raising messages in the RAP context, see [Messages](#).

i Expand the following code sample to view the source code of Validation `validateCustomer`.

Sample Code

```

CLASS ltc_managed DEFINITION DEFERRED FOR TESTING.

    PRIVATE SECTION.

        METHODS validate_customer FOR VALIDATE ON SAVE
            IMPORTING keys FOR travel~validatecustomer.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.

    METHOD validate_customer.
        " Read relevant travel instance data
        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
        ENTITY travel
        FIELDS ( customer_id )
        WITH CORRESPONDING #( keys )
        RESULT DATA(travels).

        DATA customers TYPE SORTED TABLE OF /dmo/customer WITH UNIQUE KEY customer_id.

        " Optimization of DB select: extract distinct non-initial customer IDs
        customers = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING customer_id = customer_id
        DELETE customers WHERE customer_id IS INITIAL.
        IF customers IS NOT INITIAL.

            " Check if customer ID exists
            SELECT FROM /dmo/customer FIELDS customer_id
            FOR ALL ENTRIES IN @customers
            WHERE customer_id = @customers-customer_id
            INTO TABLE @DATA(customers_db).
        ENDIF.
        " Raise msg for non existing and initial customer id
        LOOP AT travels INTO DATA(travel).
            IF travel-customer_id IS INITIAL
                OR NOT line_exists( customers_db[ customer_id = travel-customer_id ] ).

                APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.
                APPEND VALUE #( %tky = travel-%tky
                                %msg      = NEW /dmo/cm_flight_messages(
                                    customer_id = travel-customer_id
                                    textid      = /dmo/cm_flight_messages=>customer_unkown
                                    severity    = if_abap_behv_message=>severity-error )
                                %element-customer_id = if_abap_behv=>mk-on
                                ) TO reported-travel.

            ENDIF.
        ENDLOOP.

    ENDMETHOD.

ENDCLASS.

```

Implementation of `validateAgency`

The validation to check if the agency ID is valid has exactly the same structure as `validate_customer`. The code can be downloaded from ABAP Flight Reference Scenario in /DMO/BP_TRAVEL_M.

Implementation of `validateDates`

In the following codeblock, you see details about the implementation of the `validate_dates` method:

The EML read operation `READ ENTITIES` provides read access to data from trigger fields `begin_date` and `end_date`. As a result of the read operation the entered (changed) values of the `begin_date` and `end_date` fields for the selected travel instance are written into the table row `travels`.

The validation detects inconsistencies if the date value of `end_date` is before the date value of `begin_date` or if the date value of `begin_date` is in the past. Each validation can produce failed keys and messages. Any failed keys are stored in the table `FAILED` whereas the `REPORTED` table includes all instance-specific messages.

i Expand the following code sample to view the source code of Validation `validateDates`.

Sample Code

```

CLASS ltc_managed DEFINITION DEFERRED FOR TESTING.

    PRIVATE SECTION.

        METHODS validate_dates FOR VALIDATE ON SAVE
            IMPORTING keys FOR travel~validatedates.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    METHOD validate_dates.

        READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
            ENTITY travel
                FIELDS ( begin_date end_date )
                WITH CORRESPONDING #( keys )
            RESULT DATA(travels).

        LOOP AT travels INTO DATA(travel).

            IF travel-end_date < travel-begin_date. "end_date before begin_date

                APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

                APPEND VALUE #( %tky = travel-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                        textid      = /dmo/cm_flight_messages=>begin_date_bef_end_date
                        severity    = if_abap_behv_message=>severity-error
                        begin_date  = travel-begin_date
                        end_date    = travel-end_date
                        travel_id   = travel-travel_id )
                    %element-begin_date = if_abap_behv=>mk-on
                    %element-end_date  = if_abap_behv=>mk-on
                ) TO reported-travel.

            ELSEIF travel-begin_date < cl_abap_context_info=>get_system_date( ). "begin_date must be

                APPEND VALUE #( %tky      = travel-%tky ) TO failed-travel.

                APPEND VALUE #( %tky = travel-%tky

```

```

        %msg = NEW /dmo/cm_flight_messages(
            textid    = /dmo/cm_flight_messages=>begin_date_on_or_bef_sys
            severity = if_abap_behv_message=>severity-error )
        %element-begin_date = if_abap_behv=>mk-on
        %element-end_date   = if_abap_behv=>mk-on
    ) TO reported-travel.

ENDIF.

ENDLOOP.

ENDMETHOD.

ENDCLASS.

```

Implementation of **validateStatus**

In the following codeblock, you see details about the implementation of the `validateStatus` method:

Checking the validity of `overall_status` values is performed within a case loop. The valid values 0, X, and A are specified directly in the source code.

*of Validation **validateStatus**.*

Sample Code

```

CLASS ltc_managed DEFINITION DEFERRED FOR TESTING.

PRIVATE SECTION.

METHODS validate_customer FOR VALIDATE ON SAVE
IMPORTING keys FOR travel~validatecustomer.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
METHOD validate_travel_status.

READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
ENTITY travel
  FIELDS ( overall_status )
  WITH CORRESPONDING #( keys )
RESULT DATA(travels).

LOOP AT travels INTO DATA(travel).
CASE travel-overall_status.
  WHEN '0'. " Open
  WHEN 'X'. " Cancelled
  WHEN 'A'. " Accepted

  WHEN OTHERS.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

    APPEND VALUE #( %tky = travel-%tky
      %msg = NEW /dmo/cm_flight_messages(
        textid = /dmo/cm_flight_messages=>status_invalid
        severity = if_abap_behv_message=>severity-error
        status = travel-overall_status )
        %element-overall_status = if_abap_behv=>mk-on
      ) TO reported-travel.

ENDCASE.

ENDLOOP.

ENDMETHOD.

```

ENDCLASS.

Implementation of validateCurrencyCode

In the following codeblock, you see details about the implementation of the `validate_currencycode` method:

First, the EML read operation `READ ENTITY` provides read access to the selected travel instance by using the key.

To access data of the relevant entity fields, the `FIELDS ( )` addition is used. As a result of the read operation the entered (changed) value of the `currency_code` field for the selected travel instance is written into the table `travels`. Only this value is relevant for the validation.

In the following lines of code, we prepare an optimization for the following database select. By using the sorted table `currencies`, we ensure that only data records with non-initial currency codes are considered for database access.

By accessing the contents of the standard CDS view `I_currency`, we can check whether the entered currency code exists on the database at all.

If the validation detects inconsistencies (currency code is not valid or is initial), we must provide the key of all inconsistent instances as failed key and return error messages to the consumer. For all failed instances, also corresponding messages are created, which is done with the `NEW` statement. For access to suitable message texts, a message class `/dmo/cm_flight_messages` from the `/DMO/FLIGHT` package is used. For more information, about raising messages in the RAP context, see [Messages](#).

i Expand the following code sample to view the source code of Validation `validateCurrencyCode`.

Sample Code

```

CLASS ltc_managed DEFINITION DEFERRED FOR TESTING.

  PRIVATE SECTION.

    METHODS validate_currencycode FOR VALIDATE ON SAVE
      IMPORTING keys FOR travel~validatecurrencycode.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  METHOD validate_currencycode.
    READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
      ENTITY travel
        FIELDS ( currency_code )
        WITH CORRESPONDING #( keys )
      RESULT DATA(travels).

    DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

    currencies = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING currency = currency_cod
    DELETE currencies WHERE currency IS INITIAL.

    IF currencies IS NOT INITIAL.
      SELECT FROM I_Currency FIELDS currency
        FOR ALL ENTRIES IN @currencies
        WHERE currency = @currencies-currency
        INTO TABLE @DATA(currency_db).
    ENDIF.

    LOOP AT travels INTO DATA(travel).
      IF travel-currency_code IS INITIAL.
        " Raise message for empty Currency
        APPEND VALUE #( %tky                                = travel-%tky ) TO failed-travel.
        APPEND VALUE #( %tky                                = travel-%tky
                        %msg                                = NEW /dmo/cm_flight_messages(

```



```

                                textid      = /dmo/cm_flight_messages=>cur
                                severity    = if_abap_behv_message=>severi
                                %element-currency_code = if_abap_behv=>mk-on
                                ) TO reported-travel.
ELSEIF NOT line_exists( currency_db[ currency = travel-currency_code ] ).
  " Raise message for not existing Currency
  APPEND VALUE #( %tky          = travel-%tky ) TO failed-travel.
  APPEND VALUE #( %tky          = travel-%tky
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=
                                severity    = if_abap_behv_message=>se
                                currency_code = travel-currency_code )
                                %element-currency_code = if_abap_behv=>mk-on
                                ) TO reported-travel.

ENDIF.
ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

Implementation of validateBookingFee

In the following codeblock, you see details about the implementation of the `validateBookingFee` method:

First, the EML read operation `READ ENTITY` provides read access to the selected travel instances by using the keys.

To access data of the relevant entity fields, the `FIELDS ( )` addition is used. As a result of the read operation the entered (changed) value of the `booking_fee` field for the selected travel instance is written into the table `travels`.

Loop over the table `travels`. For each travel instance whose booking fee is less than 0, append a suitable message to the reported response. For more information, about raising messages in the RAP context, see [Messages](#).

i Expand the following code sample to view the source code of Validation `validate_BookingFee`.

Sample Code

```

METHOD validate_BookingFee.

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY travel
      FIELDS ( booking_fee )
      WITH CORRESPONDING #( keys )
      RESULT DATA(travels).

  LOOP AT travels INTO DATA(travel) WHERE booking_fee < 0.
    " Raise message for booking fee < 0
    APPEND VALUE #( %tky          = travel-%tky ) TO failed-travel.
    APPEND VALUE #( %tky          = travel-%tky
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>book
                                severity    = if_abap_behv_message=>severit
                                %element-booking_fee = if_abap_behv=>mk-on
                                ) TO reported-travel.

  ENDLLOOP.

ENDMETHOD.

```

Validation on Booking Entity

UI Preview

In this case, we want to validate the status values of booking instances. If a user enters the wrong value **K**, according to the figure below, the instance with its data is not saved and an error message is displayed instead.

1 /

1

Booking Booking Supplement

Booking Number: 1

Booking Date: Dec 21, 2022

Customer ID: Ryan (594)

Airline ID: * United Airlines, Inc. (UA)

Flight Number: * 1537

Flight Date: * Jan 7, 2023

Flight Price: 438.00 USD

Status: * K

Product Price: 4.50 EUR

Search

Save Cancel

Booking

Status K is not valid.

Status

Booking Status Validation on Fiori UI

Definition

```
define behavior for /DMO/I_Booking_M alias booking
...
{
...
validation validateStatus      on save { create; field booking_status; }
validation validateCustomer    on save { create; field customer_id; }
validation validateConnection  on save { create; field carrier_id, connection_id, flight_date; }
validation validateCurrencyCode on save { create; field currency_code; }
validation validateFlightPrice on save { create; field flight_price; }
}
```

Implementation of `validateStatus` for the Booking entity

The `validate_booking_status` method is implemented analogously to the one of the previous case when checking travel status.

i Expand the following code sample to view the source code of Validation `validateStatus`.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.

METHODS validateStatus FOR VALIDATE ON SAVE
importing keys FOR booking~validateStatus.
...

ENDCLASS.
```

```

CLASS lhc_travel IMPLEMENTATION.

METHOD validateStatus.

READ ENTITIES OF /dmo/i_travel_m IN LOCAL MODE
  ENTITY booking
    FIELDS ( booking_status )
    WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

LOOP AT bookings INTO DATA(booking).
  CASE booking-booking_status.
    WHEN 'N'. " New
    WHEN 'X'. " Canceled
    WHEN 'B'. " Booked

    WHEN OTHERS.
      APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.

      APPEND VALUE #( %tky = booking-%tky
        %msg = NEW /dmo/cm_flight_messages(
          textid = /dmo/cm_flight_messages=>status_invalid
          status = booking-booking_status
          severity = if_abap_behv_message=>severity-error )
        %element-booking_status = if_abap_behv=>mk-on
        %path = VALUE #( travel-travel_id = booking-travel_id )
      ) TO reported-booking.

  ENDCASE.

ENDLOOP.

ENDMETHOD.

ENDCLASS.

```

Implementation of **validateCurrencyCode** for the Booking entity

The `validate_currencycode` method is implemented analogously to the one on the travel entity..

i *Expand the following code sample to view the source code of Validation **validateCurrencyCode**.*

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

    METHODS validate_currencycode FOR VALIDATE ON SAVE
      IMPORTING keys FOR booking~validatecurrencycode.
    ...

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.

METHOD validate_currencycode.
  READ ENTITIES OF /dmo/i_travel_m IN LOCAL MODE
    ENTITY booking
      FIELDS ( currency_code )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  DATA: currencies TYPE SORTED TABLE OF i_currency WITH UNIQUE KEY currency.

  currencies = CORRESPONDING #( bookings DISCARDING DUPLICATES MAPPING currency = currency_cod
  DELETE currencies WHERE currency IS INITIAL.

  IF currencies IS NOT INITIAL.
    SELECT FROM i_currency FIELDS currency
      FOR ALL ENTRIES IN @currencies

```

```

WHERE currency = @currencies-currency
INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT bookings INTO DATA(booking).
  IF booking-currency_code IS INITIAL.
    " Raise message for empty Currency
    APPEND VALUE #( %tky                = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky                = booking-%tky
                    %msg                = NEW /dmo/cm_flight_messages(
                                          textid    = /dmo/cm_flight_messages=>cur
                                          severity  = if_abap_behv_message=>severi
                                          %element-currency_code = if_abap_behv=>mk-on
                                          %path      = VALUE #( travel-travel_id    = booking-travel_
                                          ) TO reported-booking.
    ELSEIF NOT line_exists( currency_db[ currency = booking-currency_code ] ).
    " Raise message for not existing Currency
    APPEND VALUE #( %tky                = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky                = booking-%tky
                    %msg                = NEW /dmo/cm_flight_messages(
                                          textid    = /dmo/cm_flight_messages=>cur
                                          severity  = if_abap_behv_message=>severi
                                          currency_code = booking-currency_code )
                                          %path      = VALUE #( travel-travel_id    = booking-travel_
                                          %element-currency_code = if_abap_behv=>mk-on
                                          ) TO reported-booking.
  ENDIF.
ENDLOOP.
ENDMETHOD.

ENDCLASS.

```

Implementation of **validateCustomer** for the Booking entity

The **validateCustomer** method is implemented analogously to the corresponding method for the travel entity. In addition, the corresponding travel instances are provided for each booking instance in the reported response.

i Expand the following code sample to view the source code of Validation **validateCustomer**.

Sample Code

```

METHOD validate_customer.

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY Booking
      FIELDS ( customer_id )
      WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FROM CORRESPONDING #( bookings )
  LINK DATA(travel_booking_links).

  DATA customers TYPE SORTED TABLE OF /dmo/customer WITH UNIQUE KEY customer_id.

  " Optimization of DB select: extract distinct non-initial customer IDs
  customers = CORRESPONDING #( bookings DISCARDING DUPLICATES MAPPING customer_id = customer_i
  DELETE customers WHERE customer_id IS INITIAL.

  IF customers IS NOT INITIAL.
    " Check if customer ID exists
    SELECT FROM /dmo/customer FIELDS customer_id
      FOR ALL ENTRIES IN @customers
      WHERE customer_id = @customers-customer_id
    INTO TABLE @DATA(valid_customers).
  ENDIF.

```

```

" Raise message for non existing and initial customer id
LOOP AT bookings INTO DATA(booking).
  IF booking-customer_id IS INITIAL.

    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                     %msg = NEW /dmo/cm_flight_messages(
                               textid = /dmo/cm_flight_messages
                               severity = if_abap_behv_message=
                               %path = VALUE #( travel-%tky = travel_booking_links[ KEY
                               %element-customer_id = if_abap_behv=>mk-on
                               ) TO reported-booking.

  ELSEIF booking-customer_id IS NOT INITIAL AND NOT line_exists( valid_customers[ customer_i

    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                     %msg = NEW /dmo/cm_flight_messages(
                               textid = /dmo/cm_flight_messages
                               customer_id = booking-customer_i
                               severity = if_abap_behv_message=
                               %path = VALUE #( travel-%tky = travel_booking_links[ KEY
                               %element-customer_id = if_abap_behv=>mk-on
                               ) TO reported-booking.

  ENDIF.
ENDLOOP.

ENDMETHOD.

```

Implementation of **validateConnection** for the Booking entity

The **validateConnection** method first reads the relevant fields as well as the key of the corresponding travel instance for each booking instance using the association **_Travel**.

The method then checks if the fields **carrier_id**, **connection_id** and **flight_date** are initial. If so, the method returns a failed key and reports a corresponding message.

If none of the fields are initial, the method checks if their values match a flight in the table **/dmo/flight**. If not, the method returns a failed key and reports a corresponding message.

i Expand the following code sample to view the source code of Validation **validate_connection.**

Sample Code

```

METHOD validate_connection.

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY Booking
    FIELDS ( booking_id carrier_id connection_id flight_date )
    WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY Booking BY \_Travel
  FROM CORRESPONDING #( bookings )
  LINK DATA(travel_booking_links).

  LOOP AT bookings ASSIGNING FIELD-SYMBOL(<booking>).
    " Raise message for non existing airline ID
    IF <booking>-carrier_id IS INITIAL.

      APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.
      APPEND VALUE #( %tky = <booking>-%tky
                     %msg = NEW /dmo/cm_flight_messages(
                               textid = /dmo/cm_flight_messages
                               severity = if_abap_behv_message=
                               %path = VALUE #( travel-%tky = travel_booking_links[ KEY i

```

```

        %element-carrier_id = if_abap_behv=>mk-on
    ) T0 reported-booking.
ENDIF.
" Raise message for non existing connection ID
IF <booking>-connection_id IS INITIAL.

    APPEND VALUE #( %tky = <booking>-%tky ) T0 failed-booking.
    APPEND VALUE #( %tky
                    %msg
                    %path
                    %element-connection_id
                    ) T0 reported-booking.
ENDIF.
" Raise message for non existing flight date
IF <booking>-flight_date IS INITIAL.

    APPEND VALUE #( %tky = <booking>-%tky ) T0 failed-booking.
    APPEND VALUE #( %tky
                    %msg
                    %path
                    %element-flight_date
                    ) T0 reported-booking.
ENDIF.
" check if flight connection exists
IF <booking>-carrier_id IS NOT INITIAL AND
   <booking>-connection_id IS NOT INITIAL AND
   <booking>-flight_date IS NOT INITIAL.

    SELECT SINGLE Carrier_ID, Connection_ID, Flight_Date FROM /dmo/flight WHERE carrier_
        AND connection_id = @<booking>-c
        AND flight_date = @<booking>-f
        INTO @DATA(flight).

    IF sy-subrc <> 0.

        APPEND VALUE #( %tky = <booking>-%tky ) T0 failed-booking.
        APPEND VALUE #( %tky
                        %msg
                        %path
                        %element-flight_date
                        %element-carrier_id
                        %element-connection_id
                        ) T0 reported-booking.

    ENDIF.

ENDIF.

ENDLOOP.

ENDMETHOD.

```

Implementation of validateFlightPrice

In the following codeblock, you see details about the implementation of the `validate_flight_price` method:

Read the relevant fields as well as the key of the corresponding travel instance for each booking instance using the association `_Travel`.

Loop over the table `bookings`. For each booking instance whose flight price is less than 0, append a suitable message to the reported response. Use the link table `travel_booking_links` to provide the path to the corresponding travel instance for each

booking instance in the reported response. For more information, about raising messages in the RAP context, see [Messages](#).

i Expand the following code sample to view the source code of Validation *validate_flight_price*.

Sample Code

```
METHOD validate_flight_price.

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY booking
      FIELDS ( flight_price )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY Booking BY \_Travel
    FROM CORRESPONDING #( bookings )
    LINK DATA(travel_booking_links).

  LOOP AT bookings INTO DATA(booking) WHERE flight_price < 0.
    " Raise message for flight price < 0
    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                        textid = /dmo/cm_flight_messages=>flig
                        severity = if_abap_behv_message=>severit
                        %element-flight_price = if_abap_behv=>mk-on
                        %path = VALUE #( travel-%tky = travel_booking_links[ KEY i
                    ) TO reported-booking.
  ENDLLOOP.

ENDMETHOD.
```

Validation on supplement entity

Implementation of *validate_supplement*

Read the supplement IDs for the imported keys and write the result to the internal table `bookingsupplements`. Write failed read keys to the reported response of the validation.

Read the corresponding booking and travel instance keys for each supplement instance using the associations `_Booking` and `_Travel`.

Write the distinct supplement IDs from the read result table `bookingsupplements` to an internal table that is typed with the database table `/dmo/supplement`. If the resulting internal table is not initial, write all supplement IDs from this internal table that match supplement IDs from the database table `/dmo/supplement` to another internal table `valid_supplements`.

Loop over the read result table `bookingsupplements`. For each entry with an initial supplement ID, append the instance key to the supplement table in the failed response and append a suitable message to the reported response. Proceed analogously if a supplement ID is not contained in the table `valid_supplements`. In both cases, provide the booking and travel instance keys for each supplement instance in the `%path` component of the reported response.

i Expand the following code sample to view the source code of Validation *validate_supplement*.

Sample Code

```
METHOD validate_supplement.

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
    ENTITY booksuppl
      FIELDS ( supplement_id )
      WITH CORRESPONDING #( keys )
```

```

RESULT DATA(bookingsupplements)
FAILED DATA(read_failed).

failed = CORRESPONDING #( DEEP read_failed ).

READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY booksuppl BY \_Booking
  FROM CORRESPONDING #( bookingsupplements )
  LINK DATA(booksuppl_booking_links).

READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY booksuppl BY \_Travel
  FROM CORRESPONDING #( bookingsupplements )
  LINK DATA(booksuppl_travel_links).

DATA supplements TYPE SORTED TABLE OF /dmo/supplement WITH UNIQUE KEY supplement_id.

supplements = CORRESPONDING #( bookingsupplements DISCARDING DUPLICATES MAPPING supplement_id
DELETE supplements WHERE supplement_id IS INITIAL.

IF supplements IS NOT INITIAL.
  " Check if supplement ID exists
  SELECT FROM /dmo/supplement FIELDS supplement_id
        FOR ALL ENTRIES IN @supplements
        WHERE supplement_id = @supplements-supplement_id
  INTO TABLE @DATA(valid_supplements).
ENDIF.

LOOP AT bookingsupplements ASSIGNING FIELD-SYMBOL(<bookingsupplement>).

  IF <bookingsupplement>-supplement_id IS INITIAL.

    APPEND VALUE #( %tky = <bookingsupplement>-%tky ) TO failed-booksuppl.
    APPEND VALUE #( %tky = <bookingsupplement>-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_messages
                                severity = if_abap_behv_message=
                                %path = VALUE #( booking-%tky = booksuppl_booking_links
                                                    travel-%tky = booksuppl_travel_links[
                                                    %element-supplement_id = if_abap_behv=>mk-on
                                                    ) TO reported-booksuppl.

  ELSEIF <bookingsupplement>-supplement_id IS NOT INITIAL AND NOT line_exists( valid_supplem

    APPEND VALUE #( %tky = <bookingsupplement>-%tky ) TO failed-booksuppl.
    APPEND VALUE #( %tky = <bookingsupplement>-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_messages
                                severity = if_abap_behv_message=
                                %path = VALUE #( booking-%tky = booksuppl_booking_links
                                                    travel-%tky = booksuppl_travel_links[
                                                    %element-supplement_id = if_abap_behv=>mk-on
                                                    ) TO reported-booksuppl.

  ENDIF.

ENDLOOP.

ENDMETHOD.

```

Implementation of `validateCurrencyCode` for the supplement entity

The `validateCurrencyCode` method is implemented analogously to the corresponding methods for the travel and the booking entity.

i *Expand the following code sample to view the source code of Validation `validateCurrencyCode`.*

≡, Sample Code

```

METHOD validate_currencycode.
  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY booksuppl
  FIELDS ( currency_code )
  WITH CORRESPONDING #( keys )
  RESULT DATA(booking_supplements).

  DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

  currencies = CORRESPONDING #( booking_supplements DISCARDING DUPLICATES MAPPING curr
  DELETE currencies WHERE currency IS INITIAL.

  IF currencies IS NOT INITIAL.
    SELECT FROM I_Currency FIELDS currency
    FOR ALL ENTRIES IN @currencies
    WHERE currency = @currencies-currency
    INTO TABLE @DATA(currency_db).
  ENDIF.

  LOOP AT booking_supplements INTO DATA(booking_supplement).
    IF booking_supplement-currency_code IS INITIAL.
      " Raise message for empty Currency
      APPEND VALUE #( %tky = booking_supplement-%tky ) TO failed-booksup
      APPEND VALUE #( %tky = booking_supplement-%tky
      %msg = NEW /dmo/cm_flight_messages(
      textid = /dmo/cm_flight_messages=>currency_required
      severity = if_abap_behv_message=>severity-error )
      %element-currency_code = if_abap_behv=>mk-on
      %path = VALUE #( travel-travel_id = booking_supplement-travel_id
      booking-booking_id = booking_supplement-booking_id )
      ) TO reported-booksuppl.
    ELSEIF NOT line_exists( currency_db[ currency = booking_supplement-currency_code ] )
      " Raise message for not existing Currency
      APPEND VALUE #( %tky = booking_supplement-%tky ) TO failed-booksup
      APPEND VALUE #( %tky = booking_supplement-%tky
      %msg = NEW /dmo/cm_flight_messages(
      textid = /dmo/cm_flight_messages=>currency_not_existing
      severity = if_abap_behv_message=>severity-error
      currency_code = booking_supplement-currency_code )
      %element-currency_code = if_abap_behv=>mk-on
      ) TO reported-booksuppl.
    ENDIF.
  ENDLOOP.
ENDMETHOD.

```

Implementation of **validate_price** for the supplement entity

Read the relevant fields as well as the keys of the corresponding booking and travel instances for each supplement instance using the associations **_Booking** and **_Travel**.

Loop over the read supplement instances. If the price of a supplement instance is less than 0, write its key to the failed response and append a suitable message to the reported response. Provide the corresponding booking and travel instance keys for each supplement instance in the **%path** component of the reported response.

i *Expand the following code sample to view the source code of Validation **validate_price**.*

≡, Sample Code

```

METHOD validate_price.

  READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY booksuppl

```

```

        FIELDS ( price )
        WITH CORRESPONDING #( keys )
        RESULT DATA(booking_supplements).

READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY booksuppl BY \_Booking
  FROM CORRESPONDING #( booking_supplements )
  LINK DATA(booksuppl_booking_links).

READ ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
  ENTITY booksuppl BY \_Travel
  FROM CORRESPONDING #( booking_supplements )
  LINK DATA(booksuppl_travel_links).

LOOP AT booking_supplements INTO DATA(book_suppl) WHERE price < 0.
  " Raise message for supplement price < 0
  APPEND VALUE #( %tky                = book_suppl-%tky ) TO failed-booksuppl.
  APPEND VALUE #( %tky                = book_suppl-%tky
                  %msg                = NEW /dmo/cm_flight_messages(
                                      textid      = /dmo/cm_flight_messages=>suppl
                                      severity    = if_abap_behv_message=>severity
                                      %path       = VALUE #( booking-%tky = booksuppl_booking_links[
                                                         travel-%tky = booksuppl_travel_links[
                                                         %element-price = if_abap_behv=>mk-on
                                                         ) TO reported-booksuppl.
ENDLOOP.
ENDMETHOD.

```

Developing Determinations

For the managed scenario, add determinations to calculate values implicitly.

Context and Procedure

With determinations, you can generalize the calculation of values in the business logic of a managed BO. For conceptual information about determinations, see [Determinations](#).

For this demo scenario, define and implement determinations for the entities **travel**, **booking** and **booking supplement**.

- Entity **travel**

The determination `calculateTotalPrice` calculates the total price for a travel, including the `booking_fee` (travel entity), `flight_price` (booking entity) and `price` (booking supplement entity). Technically speaking, this determination is a `determination on modify` with the trigger fields `booking_fee` and `currency_code`.

- Entity **booking**

The determination `calculateTotalPrice` invokes the same calculation as the determination on the travel entity. Technically speaking, this determination is a `determination on modify` with the trigger fields `flight_price` and `currency_code`.

- Entity **booking supplement**

The determination `calculateTotalPrice` invokes the same calculation as the determination on the travel entity. Technically speaking, this determination is a `determination on modify` with the trigger fields `price` and `currency_code`.

The best practice for determinations that trigger the same calculation is to outsource the calculation to an internal action. Like this, every determination on its entity respectively calls the action that is defined and implementation on the root entity.

Defining Determinations

Determinations are defined in the behavior definition.

As depicted in the following sample coding, add the following determinations to relevant entities in the behavior definition.

i *Expand the following code sample to view the source code of defining behavior for /DMO/I_Travel_M.*

Sample Code

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
...
{ ...
    internal action ReCalcTotalPrice;

    determination calculateTotalPrice on modify {create; field booking_fee, currency_code; }
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
...
{ ...
    // determination for calculation of total flight price
    determination calculateTotalPrice on modify { create; field flight_price, currency_code; }
}

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
...
{ ...
    // determination for calculation of total suppl. price
    determination calculateTotalPrice on modify {create; field price, currency_code; }
}
```

The determination `calculateTotalPrice`, which is defined for all three entities handles the calculation of total price of one travel. It includes the booking fee, the flight prices and the booking supplement prices. The determination will be triggered by `on modify` as determination time when creating new instances or updating the prices that are included in the calculation or when changing the currency. In other words: the trigger conditions for all determinations are `create` and the respective trigger fields on each entity.

Implementing Determinations

Determination on Travel Entity

Definition

In the behavior definition, the determination on the travel entity is defined as follows:

```
define behavior for /DMO/I_Booking_M alias booking
...
```

```
{
  ...
  determination calculateTotalPrice on modify {create; field flight_price, currency_code; }
}
```

calculateTotalPrice

Since the pricing calculation is required for all three determinations and we will access them from different handler classes, we outsource the more generic code to the internal action Implementation of ReCalcTotalPrice, which we already developed in one of the previous steps, see [Developing Actions](#). The implementation for the determination simply consists of an EML call to execute the action.

i Expand the following code sample to view the source code of determination calculateTotalPrice.

 Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    ...
    METHODS calculatetotalprice FOR DETERMINE ON MODIFY
      IMPORTING keys FOR travel~calculatetotalprice.
  ENDClass.

CLASS lhc_travel IMPLEMENTATION.

  METHOD calculatetotalprice.

    MODIFY ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
      ENTITY travel
        EXECUTE recalctotalprice
          FROM CORRESPONDING #( keys ).

  ENDMETHOD.

  ...
ENDCLASS.
```

Determination on Booking Entity

UI Preview

The figure below refers to the starting point of viewing with a newly created travel instance with the initial amount (**Total Price**) and the travel currency **0.00 USD**.

Travels (1)

Copy Travel Create Delete   

 Travel ID	Agency ID	Customer ID	Starting Date	End Date	Total Price	Overall Status	
☒ 4144	Sunshine Travel (70001)	Buchholm (3)	Apr 6, 2023	Apr 13, 2023	0.00 USD	Open	>

The Newly Created Travel with a Total Price 0.00 USD.

If a user adds a flight booking to the travel, then also the travel's **Total Price** is updated.

4144

Edit

Delete



Travel Booking

Travel ID:
4144Starting Date:
Apr 6, 2023Total Price:
422.00 USDAgency ID:
Sunshine Travel (70001)End Date:
Apr 13, 2023Status:
OpenCustomer ID:
Buchholm (3)Booking Fee:
0.00 USDDescription:
-**Booking**

Bookings (1) Standard* ▾

Search



Create



Booking Number	Booking Date	Customer ID	Airline ID	Flight Number	Flight Date	Flight Price
10	Apr 6, 2023	Gutenberg (35)	United Airlines, Inc. (UA)	1537	Apr 7, 2023	422.00 USD
Status: New						

Added Bookings Triggers an Update on Travel Data

If the user switches the booking's object page to edit mode and then changes the **Flight Price**, then the **Total Price** is also updated at root level.

Booking Number	Booking Date	Customer ID	Airline ID	Flight Number	Flight Date	Flight Price
10	Apr 6, 2023	Gutenberg (35)	UA	1537	Apr 7, 2023	411.00 USD

Status: New ▾




Travel ID	Agency ID	Customer ID	Starting Date	End Date	Total Price	Overall Status
4144	Sunshine Travel (70001)	Buchholm (3)	Apr 6, 2023	Apr 13, 2023	411.00 USD	Open

Updated Total Price at Root Level

Definition

In the behavior definition, the determination on the booking entity is defined as follows:

```

define behavior for /DMO/I_Booking_M alias booking
{
    ...
    determination calculateTotalPrice on modify {create; field flight_price, currency_code; }
}

```

Implementation of calculateTotalPrice on the Booking Entity

Since the pricing calculation is required for all three determinations and we will access them from different handler classes, we outsource the more generic code to the internal action `ReCalcTotalPrice`, which we already developed in one of the previous steps, see [Developing Actions](#). In the implementation of the determination, we first discard duplicate travel IDs from the imported keys and then execute the action for the remaining travel IDs.

i Expand the following code sample to view the source code of determination *calculateTotalPrice*.

≡ **Sample Code**

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    ...
    METHODS calculateTotalPrice FOR DETERMINE ON MODIFY
      IMPORTING keys FOR booking~calculateTotalPrice.
  ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.

  METHOD calculatetotalprice.
    DATA: travel_ids TYPE STANDARD TABLE OF /dmo/i_travel_m WITH UNIQUE HASHED KEY key COMPONENT
      travel_ids = CORRESPONDING #( keys DISCARDING DUPLICATES MAPPING travel_id = travel_id ).

    MODIFY ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
      ENTITY Travel
      EXECUTE ReCalcTotalPrice
      FROM CORRESPONDING #( travel_ids ).

  ENDMETHOD.
  ...
ENDCLASS.
```

Determination on Booking Supplement Entity

UI Preview

If the user adds a supplement to a given flight booking, then the travel amount is re-calculated.

10

Edit 

Booking Booking Supplement

Booking Number:
10

Booking Date:
Apr 6, 2023

Customer ID:
Gutenberg (35)

Airline ID:
United Airlines, Inc. (UA)

Flight Number:
1537

Flight Date:
Apr 7, 2023

Flight Price:
411.00 USD

Status:
New

Booking Supplement

Booking Supplements (1) | Standard *      

Book. Supp. Number	Product ID	Product Price
1	Hamburg Salad with Eggs (ML-0030)	12.00 USD >

Added Supplement Changes the Travel Amount

The updated travel amount is displayed in as new value of Total Price.

 Travel ID	Agency ID	Customer ID	Starting Date	End Date	Total Price	Overall Status	
☐ 4144	Sunshine Travel (70001)	Buchholm (3)	Apr 6, 2023	Apr 13, 2023	423.00 USD	Open	>

Updated Total Price of the Travel Instance

Definition

In the behavior definition, the determination on the booking supplement entity is defined as follows:

```
define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
...
{
    ...
    determination calculateTotalPrice on modify {create; field price, currency_code; }
}
```

Implementation of calculateTotalPrice on the Booking Supplement Entity

Since the pricing calculation is required for all three determinations and we will access them from different handler classes, we outsource the more generic code to the internal action ReCalcTotalPrice, which we already developed in one of the previous steps, see [Developing Actions](#). In the implementation of the determination, we first discard duplicate travel IDs from the imported keys and then execute the action for the remaining travel IDs.

i Expand the following code sample to view the source code of determination calculateTotalPrice.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

    METHODS calculateTotalPrice FOR DETERMINE ON MODIFY
      IMPORTING keys FOR booksuppl~calculateTotalPrice.

    ...

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.

  METHOD calculateTotalPrice.
    DATA: travel_ids TYPE STANDARD TABLE OF /dmo/i_travel_m WITH UNIQUE HASHED KEY key COMPONENT
      travel_id.

    travel_ids = CORRESPONDING #( keys DISCARDING DUPLICATES MAPPING travel_id = travel_id ).

    MODIFY ENTITIES OF /DMO/I_Travel_M IN LOCAL MODE
      ENTITY Travel
        EXECUTE ReCalcTotalPrice
          FROM CORRESPONDING #( travel_ids ).
    ENDMETHOD.

ENDCLASS.
```

Integrating Additional Save in Managed Business Objects

This section explains how you can integrate Additional Save within the transactional life cycle of managed business objects.

Context

This is custom documentation. For more information, please visit [SAP Help Portal](#).

For context information about the Additional Save and a description of possible use cases, see [Additional Save](#) in the [Learning About RAP](#) section of this guide.

Activities Relevant to Developers

- [Defining Additional Save in the Behavior Definition](#)
- [Implementing Additional Save](#)

Defining Additional Save in the Behavior Definition

In this topic, you will learn the syntax for defining the additional save for managed business objects.

In the following behavior definition, the additional save is defined for the `travel` (root) entity and the `booksuppl` child entity, whereas for the `booking` child entity, the (default) managed save is defined.

For a detailed syntax description, see [CDS BDL - saving options \(ABAP - Keyword Documentation\)](#).

Behavior Definition with Additional Save

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
persistent table /DMO/TRAVEL_M
with additional save
...

{
  ...
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
persistent table /DMO/BOOKING_M
...
{
  ...
}

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
persistent table /DMO/BOOKSUPPL_M
with additional save
...
{
  ...
}
```

Implementing Additional Save

The additional save of the relevant business object's entity is implemented in the behavior pool (ABAP_CLASS) that is specified in the behavior definition by the keyword `implementation in class ABAP_CLASS [unique]`.

The implementation takes place in a local saver class as a part of the behavior pool. As depicted in the listing below, each such local class inherits from the base saver class `CL_ABAP_BEHAVIOR_SAVER`. This superclass provides the predefined method `save_modified` that needs to be redefined in the local saver class.

General Implementation Steps

The following listing provides a template with the main steps for implementing additional save within the `save_modified` method.

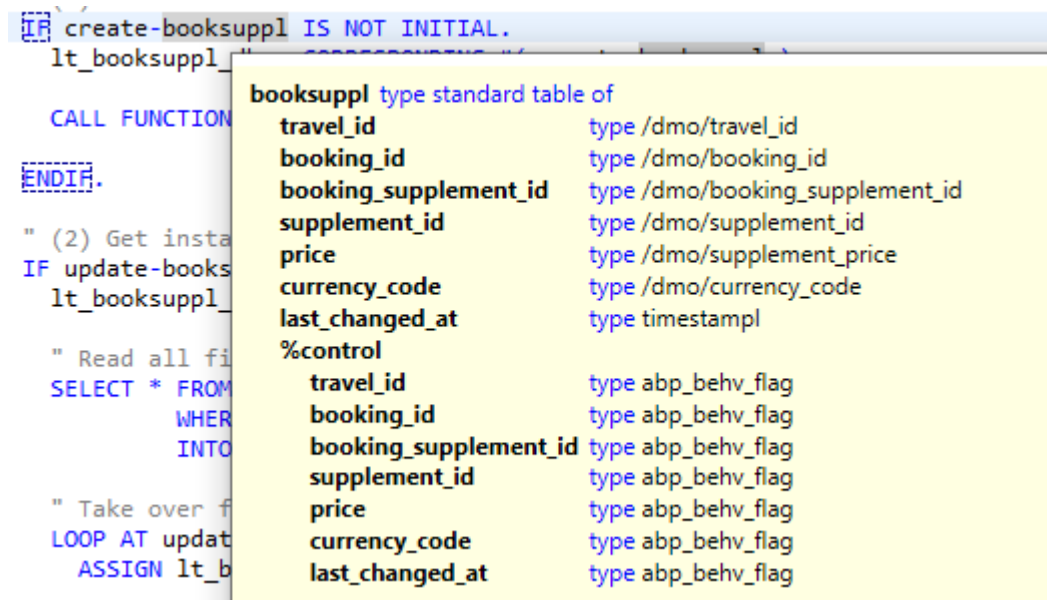
The essential elements of this method are the predefined, implicit parameters:

- CREATE-EntityName
- UPDATE-EntityName
- DELETE-EntityName

These parameters contain not only the table type of the entity to be modified, but also the `%control` structure that identifies the fields that have been changed in the current LUW. Only the fields that are flagged with `true` in the `%control` structure contain the correct values for saving in the additional save.

i Note

If you use function modules in the additional save implementation that import all fields of an instance and write all of them to the database, execute a preliminary read and read all fields of the instance, before you use the instance in your function module.



Accessing Element Information for the Implicit Parameter Type (F2)

Template for Implementing Additional Save

i Expand the following code sample to view the source code

of method **Save_Modified**.

Sample Code

```
CLASS lcl_saver DEFINITION INHERITING FROM cl_abap_behavior_saver.

  PROTECTED SECTION.
    METHODS save_modified REDEFINITION.

ENDCLASS.

CLASS lcl_saver IMPLEMENTATION.

  METHOD save_modified.

    IF CREATE-EntityName IS NOT INITIAL.
```

```

" Provide table of instance data of all instances that have been created during current tr
" Use %CONTROL to get information on what entity fields have been set when creating the in
ENDIF.

IF UPDATE-EntityName IS NOT INITIAL.
" Provide table of instance data of all instances that have been updated during current tr
" Use %CONTROL to get information on what entity fields have been updated
ENDIF.

IF DELETE-EntityName IS NOT INITIAL.
" Provide table with keys of all instances that have been deleted during current transacti
" NOTE: There is no information on fields when deleting instances
ENDIF.

ENDMETHOD.

...

ENDCLASS.

```

Additional Save Implementation in the Travel BO

The following example shows you in detail how you can implement additional save based on our [travel demo scenario](#).

In this example, all essential changes to the root instance of the travel business object are recorded in a log table. This is defined in such a way that it can hold different travel instance data and contains the following fields:

Log Table

Log Table Field	Description
change_id	Identifier for an individual change
travel_id	Key field of the travel entity
changing_operation	Standard operation for travel instances: create, update and delete
changed_field_name	Name of the field that has been created or changed
changed_value	Value of a created or changed entity field
created_at	Date and time of instance data creation, update or deletion

The following listing shows you the definition of the corresponding table /DMO/LOG_TRAVEL in the table editor.

Listing: Source code with table definition

```

@EndUserText.label : 'flight reference scenario: log changes to travel entity'
@AbapCatalog.enhancementCategory : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #LIMITED

define table /dmo/log_travel {
  key client      : abap.clnt not null;
  key change_id   : abap.raw(16);
  travel_id       : /dmo/travel_id not null;

```

7/6/26, 6:28 PM

```
changing_operation : abap.char(10);
changed_field_name : abap.char(32);
changed_value      : abap.char(32);
created_at         : timestampl;
```

```
}
```

UI Preview

Travel ID [1,...,99999999]:
1

*Agency ID:
70001

*Customer ID:
1

*Starting Date:
Oct 7, 2019

*End Date:
Oct 14, 2019

*Booking Fee:
11.00 EUR

Total Price:
19,869.20 EUR

*Status [O(Open)|A(Accepted)|X(Canceled)]:
O

Description:
Changed description

changed fields

Save Cancel

Consumer modifies business data on Fiori UI

[Y11] /DMO/LOG_TRAVEL

Raw Data

Filter pattern

2 rows retrieved - 15 ms

SQL Console

11 Number of Entries

Select Columns

AB	CHA...	TRA...	CHANGING_OP...	CHANGED_FI...	CHANGED_VAL...	12	CREATED_AT
100	0258820C7...	00000001	UPDATE	customer_id	000001		20191011143741....
100	0258820C7...	00000001	UPDATE	description	Changed descript		20191011143741....

After successful save, the relevant entries are recorded in the log table

Definition

In the behavior definition, the additional save for the root entity may be defined as follows:

```
define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
persistent table /DMO/TRAVEL_M
with additional save
...
{
  ...
}
```

Implementation

The source code of our example implementation is divided into three sections:

Each of these sections is initiated with an IF statement, each of which checks whether

- the travel instances have been created (IF create-travel IS NOT INITIAL),
- their elements have been modified (IF update-travel IS NOT INITIAL), or
- deleted (IF delete-travel IS NOT INITIAL) by a consumer.

The relevant instance data is written to the internal table `travel_log` and passed to the log table for persistent storage on the database (INSERT /dmo/log_travel...).

When creating new travel instances and updating instance data, the `%control` structure is used to get information on what fields have been provided or updated by the consumer. The `%control` structure contains a flag `if_abap_behv=>mk-on` for each field, which indicates whether the field was provided (or changed) by the consumer or not.

When creating instances (1), the new values for relevant fields of the travel entity are written into the internal table `travel_log_create`. In our demo code, we select the two fields `booking_fee` and `overall_status` as an example. Their values are transferred as separate rows by means of `APPEND` into `travel_log_create` and finally written into the log table `/dmo/log_travel` with `INSERT`.

Similarly, in the update case (2), we also select two fields, namely `customer_id` and `description`, as relevant fields for recording. So whenever the value of one of these fields is changed for an existing travel instance by a consumer, a new table row (with the corresponding change ID) is appended to the internal table `travel_log_create`.

The last section (3) deals with the deletion of travel instances. However, in this case we are only interested in the information of which instances have been deleted. Therefore, there is no information of fields available when deleting instances.

i Expand the following code sample to view the source code of method `Save_Modified`.

Sample Code

```

CLASS lcl_save DEFINITION INHERITING FROM cl_abap_behavior_saver.

    PROTECTED SECTION.
        METHODS save_modified REDEFINITION.

ENDCLASS.

CLASS lcl_save IMPLEMENTATION.

    METHOD save_modified.

CLASS lcl_save DEFINITION INHERITING FROM cl_abap_behavior_saver.

    PROTECTED SECTION.
        METHODS save_modified REDEFINITION.

ENDCLASS.

CLASS lcl_save IMPLEMENTATION.

    METHOD save_modified.

        DATA travel_log          TYPE STANDARD TABLE OF /dmo/log_travel.
        DATA travel_log_create TYPE STANDARD TABLE OF /dmo/log_travel.
        DATA travel_log_update TYPE STANDARD TABLE OF /dmo/log_travel.

        " (1) Get instance data of all instances that have been created
        IF create-travel IS NOT INITIAL.
            " Creates internal table with instance data
            travel_log = CORRESPONDING #( create-travel ).

            LOOP AT travel_log ASSIGNING FIELD-SYMBOL(<travel_log>).

```

```

<travel_log>-changing_operation = 'CREATE'.

" Generate time stamp
GET TIME STAMP FIELD <travel_log>-created_at.

" Read travel instance data into ls_travel that includes %control structure
READ TABLE create-travel WITH TABLE KEY entity COMPONENTS travel_id = <travel_log>-trave
IF sy-subrc = 0.

" If new value of the booking_fee field created
IF travel-%control-booking_fee = cl_abap_behv=>flag_changed.
  " Generate uuid as value of the change_id field
  TRY.
    <travel_log>-change_id = cl_system_uuid=>create_uuid_x16_static( ) .
    CATCH cx_uuid_error.
      "handle exception
  ENDTRY.
  <travel_log>-changed_field_name = 'booking_fee'.
  <travel_log>-changed_value = travel-booking_fee.
  APPEND <travel_log> TO travel_log_create.
ENDIF.

" If new value of the overall_status field created
IF travel-%control-overall_status = cl_abap_behv=>flag_changed.
  " Generate uuid as value of the change_id field
  TRY.
    <travel_log>-change_id = cl_system_uuid=>create_uuid_x16_static( ) .
    CATCH cx_uuid_error.
      "handle exception
  ENDTRY.
  <travel_log>-changed_field_name = 'overall_status'.
  <travel_log>-changed_value = travel-overall_status.
  APPEND <travel_log> TO travel_log_create.
ENDIF.

" IF ls_travel-%control-...

ENDIF.

ENDLOOP.

" Inserts rows specified in lt_travel_log_c into the DB table /dmo/log_travel
INSERT /dmo/log_travel FROM TABLE @travel_log_create.

ENDIF.

" (2) Get instance data of all instances that have been updated during the transaction
IF update-travel IS NOT INITIAL.
  travel_log = CORRESPONDING #( update-travel ).

  LOOP AT update-travel ASSIGNING FIELD-SYMBOL(<travel_log_update>).

    ASSIGN travel_log[ travel_id = <travel_log_update>-travel_id ] TO FIELD-SYMBOL(<travel_l
    <travel_log_db>-changing_operation = 'UPDATE'.

    " Generate time stamp
    GET TIME STAMP FIELD <travel_log_db>-created_at.

    IF <travel_log_update>-%control-customer_id = if_abap_behv=>mk-on.
      <travel_log_db>-changed_value = <travel_log_update>-customer_id.
      " Generate uuid as value of the change_id field
      TRY.
        <travel_log_db>-change_id = cl_system_uuid=>create_uuid_x16_static( ) .
        CATCH cx_uuid_error.
          "handle exception
      ENDTRY.

      <travel_log_db>-changed_field_name = 'customer_id'.

      APPEND <travel_log_db> TO travel_log_update.
    
```

```
ENDIF.
```

```
IF <travel_log_update>->control-description = if_abap_behv=>mk-on.  
  <travel_log_db>->changed_value = <travel_log_update>->description.
```

```
  " Generate uuid as value of the change_id field
```

```
  TRY.
```

```
    <travel_log_db>->change_id = cl_system_uuid=>create_uuid_x16_static( ) .
```

```
    CATCH cx_uuid_error.
```

```
      "handle exception
```

```
  ENDTRY.
```

```
  <travel_log_db>->changed_field_name = 'description'.
```

```
  APPEND <travel_log_db> TO travel_log_update.
```

```
ENDIF.
```

```
"IF <fs_travel_log_u>->control-...
```

```
ENDLOOP.
```

```
" Inserts rows specified in lt_travel_log_u into the DB table /dmo/log_travel  
INSERT /dmo/log_travel FROM TABLE @travel_log_update.
```

```
ENDIF.
```

```
" (3) Get keys of all travel instances that have been deleted during the transaction  
IF delete-travel IS NOT INITIAL.
```

```
  travel_log = CORRESPONDING #( delete-travel ).
```

```
  LOOP AT travel_log ASSIGNING FIELD-SYMBOL(<travel_log_delete>).
```

```
    <travel_log_delete>->changing_operation = 'DELETE'.
```

```
    " Generate time stamp
```

```
    GET TIME STAMP FIELD <travel_log_delete>->created_at.
```

```
    " Generate uuid as value of the change_id field
```

```
    TRY.
```

```
      <travel_log_delete>->change_id = cl_system_uuid=>create_uuid_x16_static( ) .
```

```
      CATCH cx_uuid_error.
```

```
        "handle exception
```

```
    ENDTRY.
```

```
ENDLOOP.
```

```
" Inserts rows specified in lt_travel_log into the DB table /dmo/log_travel  
INSERT /dmo/log_travel FROM TABLE @travel_log.
```

```
ENDIF.
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Integrating Unmanaged Save in Managed Business Objects

This section explains how you can integrate unmanaged save within the transactional life cycle of managed business objects.

Context

For context information about the Unmanaged Save and a description of possible use cases, see [Unmanaged Save](#) in the [Learning About RAP](#) section of this guide.

Activities Relevant to Developers

In order to integrate unmanaged save into the save sequence as a part of the managed runtime, you must first add the corresponding syntax to the behavior definition and then implement the saver handler method as a part of the behavior pool.

- [Defining Unmanaged Save in the Behavior Definition](#)
- [Implementing the Save Handler Method](#)

Defining Unmanaged Save in the Behavior Definition

In this topic, you will learn the syntax for defining unmanaged save for managed business objects.

travel (root) entity, whereas for the child entity booking the (default) managed save and for the child entity booksuppl an additional save is defined.

For a detailed syntax description, see [CDS BDL - saving options \(ABAP - Keyword Documentation\)](#).

Behavior Definition with Unmanaged Save

```
managed;

define behavior for /DMO/I_Travel_M alias travel
implementation in class /DMO/BP_TRAVEL_M unique
with additional save
...

{
    ...
}

define behavior for /DMO/I_Booking_M alias booking
implementation in class /DMO/BP_BOOKING_M unique
persistent table /DMO/BOOKING_M
...
{
    ...
}

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
persistent table /DMO/BOOKSUPPL_M
with unmanaged save
...
{
    ...
}
```

Implementing the Save Handler Method

The unmanaged save of the relevant business object's entity is implemented in the behavior pool (ABAP_CLASS) that is specified in the behavior definition by the keyword `implementation in class ABAP_CLASS [unique]`.

The implementation takes place in a local saver class as a part of the behavior pool. As depicted in the listing below, each such local class inherits from the base saver class `CL_ABAP_BEHAVIOR_SAVER`. This superclass provides the predefined method `save_modified` that needs to be redefined in the local saver class `lhc_saver`.

→ Remember

Convention: The local saver class that implements the `save_modified` method is either a separate global class or a part of the root implementation (behavior pool for the root entity).

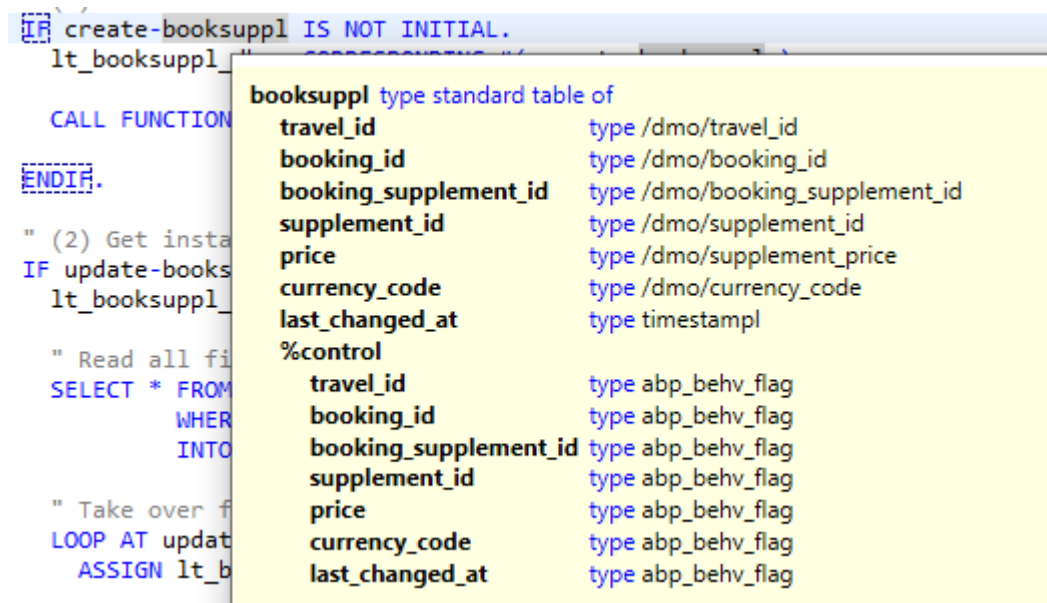
General Implementation Steps

The following listing provides a template with the main steps for implementing unmanaged save within the `save_modified` method.

The essential elements of this method are the predefined, implicit parameters:

- CREATE-EntityName
- UPDATE-EntityName
- DELETE-EntityName

These parameters contain not only the table type of the entity to be modified, but also the [%control](#) structure that can be used for identifying which elements have been changed during the current transaction. .



Accessing Element Information for the Implicit Parameter Type (`F2`)

The actual implementation steps of the `save_modified` method are shown in the template below:

Template for Implementing Unmanaged Save

```
CLASS lcl_saver DEFINITION INHERITING FROM cl_abap_behavior_saver.
```

```
  PROTECTED SECTION.
```

```
  METHODS save_modified REDEFINITION.
```

```
ENDCLASS.
```

```
CLASS lcl_saver IMPLEMENTATION.
```

```
  METHOD save_modified.
```

```
    IF CREATE-EntityName IS NOT INITIAL.
```

```
      " Provide table of instance data of all instances that have been created during current trans
```

```
      " Use %CONTROL to get information on what entity fields have been set or updated during the c
    ENDIF.
```



```

IF UPDATE-EntityName IS NOT INITIAL.

    " Provide table of instance data of all instances that have been updated during current trans

    " Use %CONTROL to get information on what entity fields have been updated
ENDIF.

IF DELETE-EntityName IS NOT INITIAL.

    " Provide table with keys of all instances that have been deleted during current transaction

    " NOTE: There is no information on fields when deleting instances
ENDIF.

ENDMETHOD.

...

ENDCLASS.

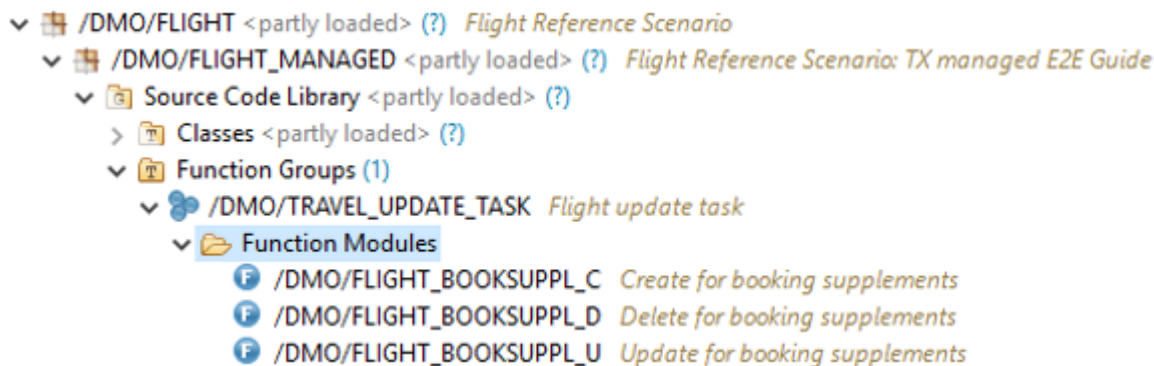
```

Implementing Unmanaged Save in the Travel BO

The following example shows you in detail how you can implement unmanaged save specifically based on our [travel demo scenario](#). In particular, the previous implementation must be extended in such a way that the available function modules are used to save changes to business data of the booking supplement child entity.

Function Modules

In our example, the corresponding function modules for creating, changing, and deleting instances of the booking supplement entity are already available in the corresponding function group of the demo package /DMO/FLIGHT_MANAGED.



Available Function Modules

Function module /DMO/FLIGHT_BOOKSUPPL_C

```

FUNCTION /dmo/flight_booksuppl_c
IMPORTING
    VALUE(values) TYPE /dmo/tt_booksuppl_m.

    INSERT /dmo/booksuppl_m FROM TABLE @values.

ENDFUNCTION.

```

i Note

To use this source code, the table type /dmo/tt_booksuppl_m is also required for the values importing parameter.

 **Table Type: /DMO/TT_BOOKSUPPL_M** 



Row Type	Initialization and Access
Category: * Dictionary Type	Initial Number of Rows: 0
Type Name: * /DMO/BOOKSUPPL_M Browse...	Access: Standard Table

Key Overview	Primary Key Details
Define primary and secondary keys	Specify key attributes
  <Primary Key>  <Secondary Keys>	Key Definition: * Standard Key
	Key Category: * Non-Unique
	Alias:

Definition of Table Type /DMO/TT_BOOKSUPPL_M in the Dictionary Tool

The following listing provides you with the function module's source code for persistent storage of individual elements of existing booking supplement instances.

Function module /DMO/FLIGHT_BOOKSUPPL_U

```
FUNCTION /dmo/flight_booksuppl_u
  IMPORTING
    VALUE(values) TYPE /dmo/tt_booksuppl_m.

  UPDATE /dmo/booksuppl_m FROM TABLE @values.

ENDFUNCTION.
```

The following listing provides you with the source code of the function module for deleting booking supplement instances.

Function module /DMO/FLIGHT_BOOKSUPPL_D

```
FUNCTION /dmo/flight_booksuppl_d
  IMPORTING
    VALUE(values) TYPE /dmo/tt_booksuppl_m.

  DELETE /dmo/booksuppl_m FROM TABLE @values.

ENDFUNCTION.
```

Definition

In the behavior definition, the unmanaged save for the child entity `booksuppl` may be defined as follows:

```
define behavior for /DMO/I_Travel_M alias travel
  implementation in class /DMO/BP_TRAVEL_M unique
```

```

...

{
    ...
}
...

define behavior for /DMO/I_BookSuppl_M alias booksuppl
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_M unique
with unmanaged save

...

{
    ...
}

```

Implementation

As shown in the listing below, the source code of our example implementation is divided into three sections:

Each of these sections is initiated with an IF statement, each of which checks whether the booking supplement instances have been created, their elements have been modified, or deleted by a consumer. The relevant instance data is written to the internal table `booksuppls_db` and passed to the respective function module for persistent storage on the database.

In case of updating instance data (IF `update-booksuppl IS NOT INITIAL`), the `%control` structure is used to get information on what fields have been updated by the consumer. The `%control` structure contains a flag `if_abap_behv=>mk-on` for each field, which indicates whether the field was provided (changed) by the consumer or not.

To eliminate the option that the unchanged fields are overwritten with default values, we must ensure that they are kept according to the database data. This is the reason why we read the current data set from the database using the statement `SELECT * FROM /dmo/booksuppl_m FOR ALL ENTRIES IN @booksuppls_db...`

i Expand the following code sample to view the source code of method *Save_Modified*.

Sample Code

```

CLASS lcl_save DEFINITION INHERITING FROM cl_abap_behavior_saver.

    PROTECTED SECTION.
        METHODS save_modified REDEFINITION.

ENDCLASS.

CLASS lcl_save IMPLEMENTATION.

    METHOD save_modified.

        DATA booksuppls_db TYPE STANDARD TABLE OF /dmo/booksuppl_m.

        " (1) Get instance data of all instances that have been created
        IF create-booksuppl IS NOT INITIAL.
            booksuppls_db = CORRESPONDING #( create-booksuppl ).

            CALL FUNCTION '/DMO/FLIGHT_BOOKSUPPL_C' EXPORTING values = booksuppls_db .

        ENDIF.

        " (2) Get instance data of all instances that have been updated during the transaction
        booksuppls_db = CORRESPONDING #( update-booksuppl ).
        IF booksuppls_db IS NOT INITIAL.

            " Read all field values from database

```

```

SELECT * FROM /dmo/booksuppl_m FOR ALL ENTRIES IN @booksuppls_db
      WHERE booking_supplement_id = @booksuppls_db-booking_supplement_id
      INTO TABLE @booksuppls_db .

" Take over field values that have been changed during the transaction
LOOP AT update-booksuppl ASSIGNING FIELD-SYMBOL(<unmanaged_booksuppl>).
  ASSIGN booksuppls_db[ travel_id = <unmanaged_booksuppl>-travel_id
        booking_id = <unmanaged_booksuppl>-booking_id
        booking_supplement_id = <unmanaged_booksuppl>-booking_supplement_id
      ] TO FIELD-SYMBOL(<booksuppl_db>).

  IF <unmanaged_booksuppl>-%control-supplement_id = if_abap_behv=>mk-on.
    <booksuppl_db>-supplement_id = <unmanaged_booksuppl>-supplement_id.
  ENDIF.

  IF <unmanaged_booksuppl>-%control-price = if_abap_behv=>mk-on.
    <booksuppl_db>-price = <unmanaged_booksuppl>-price.
  ENDIF.

  IF <unmanaged_booksuppl>-%control-currency_code = if_abap_behv=>mk-on.
    <booksuppl_db>-currency_code = <unmanaged_booksuppl>-currency_code.
  ENDIF.

ENDLOOP.

" Update the complete instance data
CALL FUNCTION '/DMO/FLIGHT_BOOKSUPPL_U' EXPORTING values = booksuppls_db .

ENDIF.

" (3) Get keys of all travel instances that have been deleted during the transaction
IF delete-booksuppl IS NOT INITIAL.
  booksuppls_db = CORRESPONDING #( delete-booksuppl ).

  CALL FUNCTION '/DMO/FLIGHT_BOOKSUPPL_D' EXPORTING values = booksuppls_db .

ENDIF.

ENDMETHOD.

ENDCLASS.

```

Developing a Projection Layer for Flexible Service Consumption

For a more flexible service consumption, every transactional business object is projected onto a service specific context. In other words, only those elements of the data model and those behavior characteristics and operations that are needed in the relevant business service context are exposed for the service. By means of projections, you can expose one BO in different business contexts by using different BO subsets. The general business logic is defined in the BO whereas the BO projection adopts a subset of the business logic.

A layering with projections enables robust application programming. You can change or enhance the BO without changing the exposed service as the scope of the service is defined in the projection layer. Enhancing the business object with additional structure or behavior does not have any effect on the resulting service.

Projection Layers in the Travel Business Scenario

The business object that you developed with the help of the previous sections is ready to run. It uses the managed runtime for CRUD (create, read, update, delete) operations. In addition, the relevant business logic for managing travels with action, determinations and validations was implemented. The BO CDS entities expose every element that might be relevant for any business service scenario. The behavior is defined and implemented for any kind of business service.

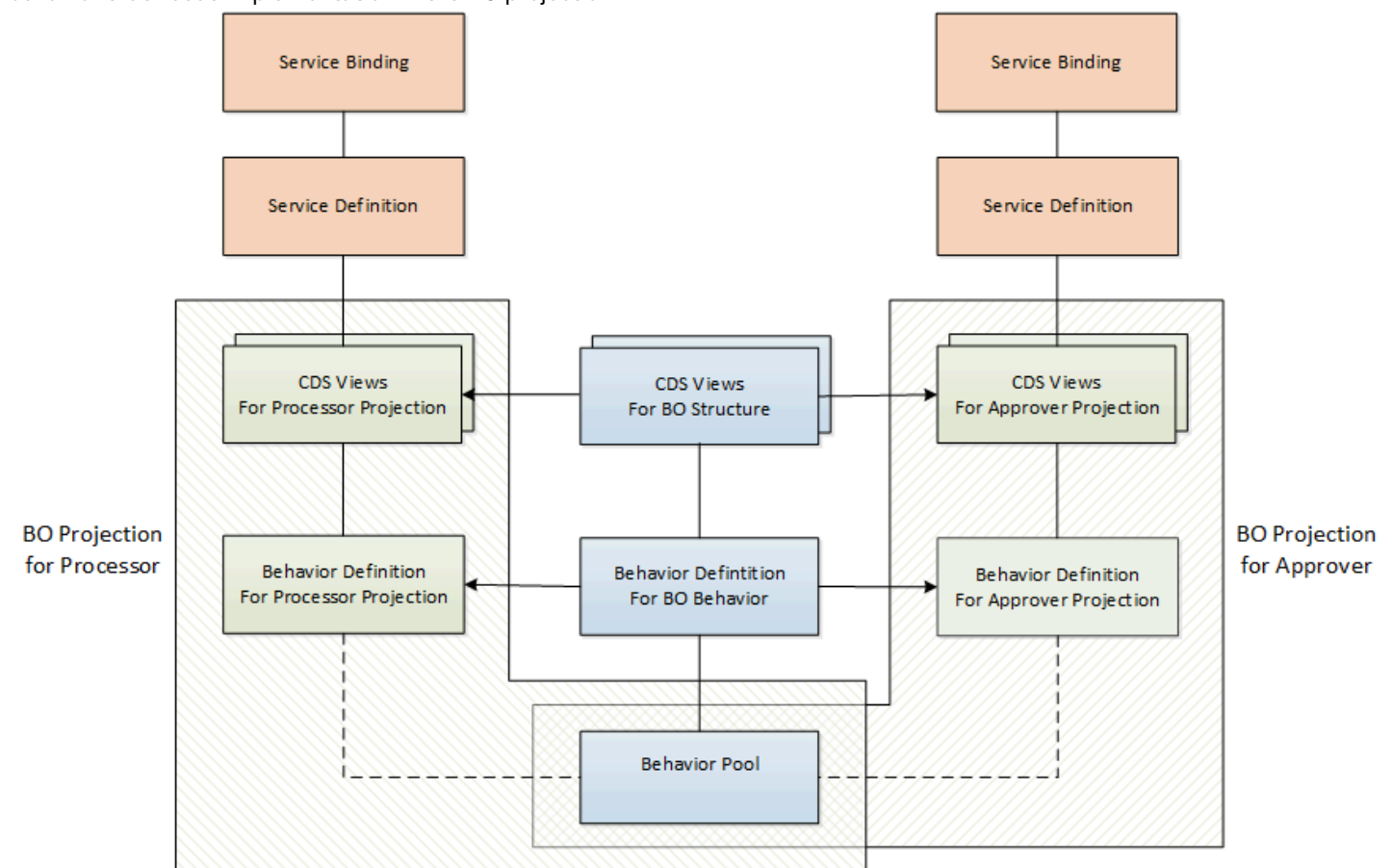
This demo scenario uses two projections with a different service scope. The resulting apps represent two role-based approaches for managing travels:

One business object projection uses the BO characteristics and the operations that are relevant for processing travel data. This resulting UI service serves the role of a data processor. The responsible person can enter the information about travels, bookings and booking supplements into the role-based app for processing travel data. This person needs functionality to create, update and delete entries on all three tiers of the business object. In addition, the action `copyTravel` is designed to facilitate the creation of new travel entries. The instance-bound action reads the values of the selected entry and its subentries and creates new entries based on these values.

The other business object projection is the basis for a UI service that contains the functionality that is relevant for an approver. Imagine a person, maybe a manager of a travel agency, that approves the data that was entered by the processor. That means, this person sees the travel information and the corresponding booking data. Based on this data, the approver can either accept or reject the travel. For a minimal scope of fields in the travel entity the approver is enabled to edit the values, for example the `BookingFee` or the `Description`. The information about bookings is set to read-only for the approver. The approver is not allowed to change the fields of the booking entity.

See [Business Scenario](#) for a detailed list of features for the respective roles.

The design time artifacts that you need for these projection scenarios are illustrated in the following figure. The CDS views, as well as the behavior definition must be projected for both roles. To expose the BO projections for a UI service, you need to create a service definition and binding for both BO projections. The behavior implementation is not projected. Every behavior characteristic or operation that is used in the BO projection must be implemented in the underlying BO implementation. You cannot define new behavior that needs implementation in the BO projection.



Business Projections for the Processor and the Approver

Activities Relevant to Developers

1. [Providing a Data Model for Projections](#)
 - a. [Projection Views for the Processor BO Projection](#)

b. [Projection Views for the Approver BO Projection](#)

2. [Providing Behavior for Projections](#)

a. [Behavior for the Processor BO Projection](#)

b. [Behavior for the Approver BO Projection](#)

3. [Defining Business Services Based on Projections](#)

Related Information

[Business Object Projection](#)

Providing a Data Model for Projections

The data model for the BO projection is defined in CDS projection views. Projection views are data definition artifacts with a syntax that differs slightly from CDS views. With projection views, you define the consumption-specific data model.

For a detailed explanation of the syntax, see [CDS Projection View](#).

Data Model in the Travel Scenario

For our travel scenario, the data models for the two projections have to be defined. For the processor BO, all three entities of the underlying BO are projected; the approver BO only uses the travel entity and the booking entity. All elements are aliased as an automatic mapping is provided for the elements in the projection views. For both projections, we use all elements from the underlying CDS views and the associations that are defined in the projected entity. For the processor BO, the only language-dependent text element in the booking supplement entity must be localized to get the description in the relevant language.

For both BO projections, the compositions have to be redirected.

UI Specifics for the Travel Scenario

Since the projection layer is the first service-specific layer, all UI specification must be defined in the CDS projection views. In the travel scenario, the following UI specifics are relevant on the projection layer:

- UI annotations defining position, labels, and facets of UI elements
- Search Enablement
- Text elements (language dependent and independent)
- Value Helps

These features have to be defined via annotations in the projection views.

The following sections provide a detailed description on how to project the existing BO to define a data model for one business object that is tailored to expose a UI service for a data processor and one that is tailored for a data approver.

- [Projection Views for the Processor BO Projection](#)
- [Projection Views for the Approver BO Projection](#)

Related Information

[CDS Projection View](#)

Projection Views for the Processor BO Projection

To define a data model for the BO projection that defines the scope for the processor application, the following tasks need to be done:

- [Creating the Projection CDS Views for the Processor](#)
- [Defining the Data Model for the Processor Projection Views](#)

Creating the Projection CDS Views for the Processor

A data processor needs to be able to create, update, and delete entries for the travel entity, the booking entity, and the booking supplement entity. That means, all three nodes of the composition structure must be projected.

For the following CDS views, create the corresponding projection views by choosing the projection view template in the creation wizard for data definitions.

CDS views for BO structure	/DMO/I_TRAVEL_M	/DMO/I_BOOKING_M	/DMO/I_BOOKSUPPL_M
CDS views for BO projection	/DMO/C_TRAVEL_PROCESSOR_M	/DMO/C_BOOKING_PROCESSOR_M	/DMO/C_BOOKSUPPL_PROCESSOR_M

i Note

The names are assigned according to the naming conventions for projection views: [Naming Conventions for Development Objects](#).

For information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Defining the Data Model for the Processor Projection Views

The following topics provide you with a detailed description on how to define the data model for the CDS projection views that are used in the BO projection for the processor.

- [Travel Projection View /DMO/C_TRAVEL_PROCESSOR_M](#)
- [Booking Projection View /DMO/C_BOOKING_PROCESSOR_M](#)
- [Booking Supplement Projection View /DMO/C_BOOKSUPPL_PROCESSOR_M](#)

Travel Projection View /DMO/C_TRAVEL_PROCESSOR_M

For the service specific projection, the elements as well as all the UI specifics need to be defined.

The data model defines which elements are exposed for the UI service. In addition, in data definitions for projection views you define all UI specifications.

The following UI is achieved by implementing the corresponding features in the CDS Travel projection view for the processor.

Preview: UI Application for Processor

Agency ID:

Customer ID:

Overall Status:

Search



Go

Adapt Filters



Travels (4,137)

Copy Travel

Create

Delete



Travel ID	Agency ID	Customer ID	Starting Date	End Date	Total Price		Overall Status	
☒ 1	Maxitrip (70041)	Ryan (594)	Jan 7, 2023	Nov 5, 2023	1,889.00	USD	Open	>
☐ 2	Hot Socks Travel (70007)	Prinz (608)	Jan 7, 2023	Jan 7, 2023	900.00	USD	Open	>
☐ 3	Hendrik's (70046)	Detemple (93)	Jan 7, 2023	Nov 5, 2023	4,164.00	USD	Open	>

Travel List Report

1

Edit

Delete



Travel

Booking

Travel ID:

1

Starting Date:

Jan 7, 2023

Total Price:

1,889.00 USD

Agency ID:

Maxitrip (70041)

End Date:

Nov 5, 2023

Status:

Open

Customer ID:

Ryan (594)

Booking Fee:

40.00 USD

Description:

Vacation

Travel Object Page

i Expand the following code sample to view the source code of the Travel Projection View /DMO/C_Travel_Processor_M.

Sample Code

```
@EndUserText.label: 'Travel projection view'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@UI: {
  headerInfo: { typeName: 'Travel', typeNamePlural: 'Travels', title: { type: #STANDARD, value:

@Search.searchable: true

define root view entity /DMO/C_Travel_Processor_M
  provider contract transactional_query
  as projection on /DMO/I_Travel_M
{
  @UI.facet: [ { id:
                  'Travel',
                purpose:
                  #STANDARD,
                type:
                  #IDENTIFICATION_REFERENCE,
                label:
                  'Travel',
                position:
                  10 },
    { id:
      'Booking',
      purpose:
        #STANDARD,
      type:
        #LINEITEM_REFERENCE,
      label:
        'Booking',
      position:
        20,
      targetElement:
        '_Booking'}]

  @UI: {
    listItem: [ { position: 10, importance: #HIGH },
                 { type: #FOR_ACTION, dataAction: 'copyTravel', label: 'Copy Travel' }
    identification: [ { position: 10, label: 'Travel ID' } ] ] }
```



```

@Search.defaultSearchElement: true
key travel_id          as TravelID,

@UI: {
  lineItem:            [ { position: 20, importance: #HIGH } ],
  identification:      [ { position: 20 } ],
  selectionField:      [ { position: 20 } ] }
@Consumption.valueHelpDefinition: [{ entity : {name: '/DM0/I_Agency', element: 'AgencyID'
@ObjectModel.text.element: ['AgencyName']
@Search.defaultSearchElement: true
agency_id              as AgencyID,
_Agency.Name           as AgencyName,

@UI: {
  lineItem:            [ { position: 30, importance: #HIGH } ],
  identification:      [ { position: 30 } ],
  selectionField:      [ { position: 30 } ] }
@Consumption.valueHelpDefinition: [{ entity : {name: '/DM0/I_Customer', element: 'Customer
@ObjectModel.text.element: ['CustomerName']
@Search.defaultSearchElement: true
customer_id            as CustomerID,
_Customer.LastName     as CustomerName,

@UI: {
  lineItem:            [ { position: 40, importance: #MEDIUM } ],
  identification:      [ { position: 40 } ] }
begin_date             as BeginDate,

@UI: {
  lineItem:            [ { position: 41, importance: #MEDIUM } ],
  identification:      [ { position: 41 } ] }
end_date               as EndDate,

@UI: {
  identification:      [ { position: 42 } ] }
@Semantics.amount.currencyCode: 'CurrencyCode'
booking_fee            as BookingFee,

@UI: {
  lineItem:            [ { position: 43, importance: #MEDIUM } ],
  identification:      [ { position: 43, label: 'Total Price' } ] }
@Semantics.amount.currencyCode: 'CurrencyCode'
total_price            as TotalPrice,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
currency_code          as CurrencyCode,

@UI: {
  lineItem:            [ { position: 50, importance: #HIGH } ],
  identification:      [ { position: 45, label: 'Status' } ],
  selectionField:      [{ position: 40 }],
  textArrangement:    #TEXT_ONLY
  }
@Consumption.valueHelpDefinition: [{ entity: { name: '/DM0/I_Overall_Status_VH', element:
@ObjectModel.text.element: ['OverallStatusText']
overall_status         as OverallStatus,

@UI.hidden: true
_OverallStatus._Text.Text as OverallStatusText : localized,

@UI: {
  identification:[ { position: 46 } ] }
description            as Description,

@UI.hidden: true
last_changed_at        as LastChangedAt,

/* Associations */
_Booking : redirected to composition child /DM0/C_Booking_Processor_M,
_Agency,
_Customer,
_OverallStatus
}

```

Explanation

For the data model of the travel projection view in our scenario, you can adopt all elements of the projected view, except for `created_by`, `created_at` and `last_changed_by`. Those elements are not needed for our service use cases. The element `last_changed_at`, however, is needed to store the eTag, but the other administrative elements are not needed in the scenario. The other elements for travel information are used to process travel data.

→ Remember

The eTag is needed for optimistic concurrency check. In the travel BO, all nodes use their own master eTag.

All elements of the projection can be given an alias with an automatic mapping done by the service framework.

The travel projection view uses a subset of the associations that are defined in the projected view. `_Agency`, `_Customer` and `_OverallStatus` are needed for text provisioning. These associations can simply be adopted in the projection view. On the other hand, the composition to the child entity booking must be redirected as the target entity changes in the projection layer. The association `_Currency` is not necessary in the projection view. It is only defined in the underlying BO data model to represent a complete data model structure.

i Note

Before you can activate the travel projection root view for the processor, you need to create the booking projection view with the redirection of the composition to parent in the booking projection child view. Compositions must always be consistent from parent to child and vice-versa.

UI Specifics

The UI header information is given in an entity annotation to label the list report page.

The travel processor projection view is the root node of the BO projection. When opening the travel processing app, the travel entries are displayed as list items with a navigation to their object page and the corresponding bookings. From this object page, it is possible to navigate to the booking supplements. In the back end, navigating is implemented by compositions. For the UI to enable navigation, UI facets need to be defined in the travel projection view for the identification reference of the travel entity on the object page and for the line item reference of the booking entity.

In addition, the elements for list items and the identification reference for the object page need to be annotated in the projection view with the respective UI annotations to define position, importance and possible labels.

For more information about UI navigation and positioning of elements, see [Designing the User Interface for a Fiori Elements App](#) or [CDS Annotations](#).

The annotations that are used in the projected entity are propagated to the projection view. You do not need to reannotate elements with the same annotations as in the projected entity. However, if an annotation draws reference to a specific element and the name of that specific element is changed with an alias in the projection view, the propagated annotation keeps the reference that was given in the projected entity. A semantic relationship between two elements can then be lost. In such a case, you have to reuse the same annotation and use the alias name in the element reference of the annotation.

In our example scenario, this is the case for the semantic relationship between `CurrencyCode` and `TotalPrice` or `BookingFee`. In the projection view, you do not need to annotate `CurrencyCode` with `@Semantics.currencyCode: true` as this annotation is inherited from the projected entity. The annotation `@Semantics.amount.currencyCode: 'currency_code'` is inherited as well, but the name of the field has changed in the projection view. So you need to reannotate the element with the new alias name: `@Semantics.amount.currencyCode: 'CurrencyCode'`.

→ Tip

Check the **Active AnnotationsView** to find out which annotations are active for the current CDS view and what the values of the active annotations are.

To be able to search for a specific data set, the travel projection view must be search enabled and at least one element must be assigned as default search element. In addition, you can define selection fields to provide a filter bar for certain elements. For more information about search enabling, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#). For more information about selection fields, see [List Report Header](#)

To access corresponding texts or descriptions, the relationship between the elements `AgencyID`, `CustomerID`, `OverallStatus` and their text elements in the associated text provider views must be established. The text elements of the text provider view must be integrated in the projection view. The text provider view must be associated to the projection view and the text element in the text provider view must be annotated with `@Semantics.text: true`. The annotation `@ObjectModel.text.element` establishes the link between the annotated element and its descriptive text element whose values are displayed in the respective logon language. To indicate, that only the text element is to be displayed, the annotation `textArrangement: #TEXT_ONLY` is used for the element `OverallStatus`. The denormalized field `OverallStatusText` is hidden using the annotation `@UI.hidden: true` to prevent the display of redundant information. For more information about text provisioning, see [Working with Text Elements](#).

Especially for a data processing role, value helps are particularly important to find the necessary values for the elements `AgencyID`, `CustomerID`, `CurrencyCode` and `OverallStatus`. Value helps are defined with the annotation `@Consumption.valueHelpDefinition`. The value help provider view does not have to be associated to get the value help as the entity and the element are referenced in the annotation. For more information, see [Adding Value Help](#).

In the projection view, you also have to define the position of the execution button of actions, that you have defined in the behavior definition. On the list report page, the position of the button for the action `copyTravel` is defined. For more information about the action, see [Developing Actions](#).

Booking Projection View /DMO/C_BOOKING_PROCESSOR_M

For the service-specific projection, the elements as well as all the UI specifics must be defined.

The data model defines which elements are exposed for the UI service. In addition, in data definitions you have to define all UI specifications.

The following UI is achieved by implementing the corresponding features in the CDS Booking projection view for the processor.

Preview: UI Application for Processor

Bookings (8) | Standard* ▾

Search

Q

Create

⚙

📄

▾

Booking Number	Booking Date	Customer ID	Airline ID	Flight Number	Flight Date	Flight Price		Status	
1	Dec 18, 2022	Detemple (93)	United Airlines, Inc. (UA)	1537	Jan 7, 2023	438.00	USD	New	>
2	Dec 18, 2022	Müller (706)	United Airlines, Inc. (UA)	1537	Jan 7, 2023	438.00	USD	New	>
3	Dec 21, 2022	Detemple (93)	American Airlines Inc. (AA)	322	Jan 9, 2023	438.00	USD	New	>

Booking List Report Page

Booking

Booking Supplement

Booking Number:	Airline ID:	Flight Price:
1	United Airlines, Inc. (UA)	438.00 USD
Booking Date:	Flight Number:	Status:
Dec 18, 2022	1537	New
Customer ID:	Flight Date:	
Detemple (93)	Jan 7, 2023	

Booking Object Page

i Expand the following code sample to view the source code of Booking Projection View /DMO/C_Booking_Processor_M.

Sample Code

```
@EndUserText.label: 'Booking projection view'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@UI: {
  headerInfo: { typeName: 'Booking',
                 typeNamePlural: 'Bookings',
                 title: { type: #STANDARD, value: 'BookingID' } } }

@Search.searchable: true

define view entity /DMO/C_Booking_Processor_M
  as projection on /DMO/I_Booking_M
{
  @UI.facet: [ { id: 'Booking',
                  purpose: #STANDARD,
                  type: #IDENTIFICATION_REFERENCE,
                  label: 'Booking',
                  position: 10 },
               { id: 'BookingSupplement',
                  purpose: #STANDARD,
                  type: #LINEITEM_REFERENCE,
                  label: 'Booking Supplement',
                  position: 20,
                  targetElement: '_BookSupplement' } ]

  @Search.defaultSearchElement: true
  key travel_id as TravelID,

  @UI: { lineItem: [ { position: 20, importance: #HIGH } ],
        identification: [ { position: 20 } ] }
  @Search.defaultSearchElement: true
  key booking_id as BookingID,

  @UI: { lineItem: [ { position: 30, importance: #HIGH } ],
        identification: [ { position: 30 } ] }
  booking_date as BookingDate,

  @UI: { lineItem: [ { position: 40, importance: #HIGH } ],
        identification: [ { position: 40 } ] }
  @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Customer', element: 'CustomerID'}
  @ObjectModel.text.element: ['CustomerName']}
  @Search.defaultSearchElement: true
  customer_id as CustomerID,
  _Customer.LastName as CustomerName,

  @UI: { lineItem: [ { position: 50, importance: #HIGH } ],
```

```

        identification: [ { position: 50 } ] }
@Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Carrier', element: 'AirlineID'
@ObjectModel.text.element: ['CarrierName']}
carrier_id          as CarrierID,
_carrier.Name       as CarrierName,

@UI: { lineItem:      [ { position: 60, importance: #HIGH } ],
      identification: [ { position: 60 } ] }
@Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Flight', element: 'ConnectionI
      additionalBinding: [ { localElement: 'FlightDate',
                           { localElement: 'CarrierID',
                           { localElement: 'FlightPrice',
                           { localElement: 'CurrencyCode',

connection_id       as ConnectionID,

@UI: { lineItem:      [ { position: 70, importance: #HIGH } ],
      identification: [ { position: 70 } ] }
@Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Flight', element: 'FlightDate'
      additionalBinding: [ { localElement: 'ConnectionID',
                           { localElement: 'CarrierID',
                           { localElement: 'FlightPrice',
                           { localElement: 'CurrencyCode',

flight_date         as FlightDate,

@UI: { lineItem:      [ { position: 80, importance: #HIGH } ],
      identification: [ { position: 80 } ] }
@Semantics.amount.currencyCode: 'CurrencyCode'
flight_price        as FlightPrice,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
currency_code       as CurrencyCode,

@UI: { lineItem:      [ { position: 90, importance: #HIGH, label: 'Status' } ],
      identification: [ { position: 90, label: 'Status' } ],
      textArrangement: #TEXT_ONLY }
@Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Booking_Status_VH', element:
@ObjectModel.text.element: ['BookingStatusText']}
booking_status      as BookingStatus,

@UI.hidden: true
_bookingStatus._Text.Text as BookingStatusText : localized,

@UI.hidden: true
last_changed_at     as LastChangedAt,

/* Associations */
_travel             : redirected to parent /DMO/C_Travel_Processor_M,
_bookSupplement     : redirected to composition child /DMO/C_BookSuppl_Processor_M,
_customer,
_carrier,
_bookingStatus
}

```

Explanation

For the data model of the booking projection view, you can adopt all elements of the projected view.

All elements of the projection can be given an alias with an automatic mapping done by the service framework.

The booking projection view uses a subset of the associations that are defined in the projected view. The associations `_Customer`, `_Carrier` and `_BookingStatus` are needed for text provisioning. These associations can simply be adopted in the projection view. On the other hand, the compositions to the parent entity `_Travel` and to the child entity `_BookSupplement` must be redirected as the target entities change in the projection layer. The association `_Connection` is not necessary in the projection view. It is defined in the underlying BO data model to complete the BO data model structure.

i Note

Before you can activate the booking projection view for the processor, you need to create the booking supplement projection view with the redirection to the composition to parent from the booking supplement projection child view. Compositions must always be consistent from parent to child and vice-versa.

UI Specifics

Like in the travel projection view, the UI header information for the booking projection view is given in an entity annotation.

For the UI to enable navigation from the Booking to the BookingSupplement entity, you need to define UI facets. The booking entity must be defined as identification reference and the BookingSupplement as line item reference.

In addition, the elements for list items and identification reference for the second navigation need to be annotated in the booking projection view with the respective UI annotations to define position, importance, and possible labels.

For more information about UI navigation and positioning of elements, see [Designing the User Interface for a Fiori Elements App](#) or [CDS Annotations](#).

As in the travel projection view, the annotation `@Semantics.amount.currencyCode: 'CurrencyCode'` needs to be repeated in the projection view, since the annotation value changes due to aliasing.

To be able to search for a specific data set, the booking projection view must be search enabled and at least one element must be assigned as default search element. For more information about search enabling, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To access the corresponding texts or descriptions, the relationship between the elements `CarrierID`, `BookingStatus` and their text element in the associated text provider view must be established. Therefore, you need the association to the text provider view. The text element of the text provider view must be integrated in the projection view. The text provider view must be associated to the projection view and the text element in the text provider view must be annotated with `@Semantics.text: true`. The annotation `@ObjectModel.text.element` establishes the link between the annotated element and its descriptive text element whose values are displayed in the respective logon language. To indicate, that only the text element is to be displayed, the annotation `textArrangement: #TEXT_ONLY` is used for the element `BookingStatus`. The denormalized field `BookingStatusText` is hidden using the annotation `@UI.hidden: true` to prevent the display of redundant information. For more information about text provisioning, see [Working with Text Elements](#).

Especially for a data processing role, value helps are important to find the necessary values for the elements `CustomerID`, `CarrierID`, `ConnectionID` and to find adequate values for `FlightDate`, `CurrencyCode` and `BookingStatus`. Value helps are defined with the annotation `@Consumption.valueHelpDefinition`. The value help for `ConnectionID` and `FlightDate` use additional bindings, so that only those values appear that match the entries in the given local elements. The value help provider view does not have to be associated to get the value help as the entity and the element are referenced in the annotation. However, it needs to be included in the service definition. For more information, see [Adding Value Help](#).

The administrative field `last_changed_at` is only used for concurrent processing and does not have to be displayed on the UI. The annotation `@UI.hidden = true` is used for that purpose.

Booking Supplement Projection View /DMO/C_BOOKSUPPL_PROCESSOR_M

For the service-specific projection, the elements as well as all the UI specifics must be defined.

The data model defines which elements are exposed for the UI service. In addition, in data definitions you have to define all UI specifications.

The following UI is achieved by implementing the corresponding features in the CDS Booking Supplement projection view for the processor.

Preview: UI Application for Processor

Booking Supplements (5) Standard* Search Create Settings Filter More				
Book. Supp. Number	Product ID	Product Price		
1	Chocolate Cake (ML-0035)	2.00	EUR	>
2	Trout Au Bleu (ML-0015)	21.00	EUR	>
3	Paris Salad (ML-0031)	9.00	EUR	>
4	Apple Juice (BV-0005)	3.50	EUR	>
5	Turkey Steak (ML-0019)	12.00	EUR	>

Booking Supplement List Report Page

i Expand the following code sample to view the source code of Booking Supplement Projection View /DMO/C_BookSuppl_Processor_M.

Sample Code

```
@EndUserText.label: 'Booking supplement projection view'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@UI: { headerInfo: { typeName: 'Booking Supplement',
                        typeNamePlural: 'Booking Supplements',
                        title: { type: #STANDARD,
                                label: 'Booking Supplement',
                                value: 'BookingSupplementID' } } }

@Search.searchable: true

define view entity /DMO/C_BookSuppl_Processor_M as projection on /DMO/I_BookSuppl_M
{
    @UI.facet: [ { id: 'BookingSupplement',
                    purpose: #STANDARD,
                    type: #IDENTIFICATION_REFERENCE,
                    label: 'Booking Supplement',
                    position: 10 } ]

    @Search.defaultSearchElement: true
    key travel_id as TravelID,

    @Search.defaultSearchElement: true
    key booking_id as BookingID,

    @UI: { lineItem: [ { position: 10, importance: #HIGH } ],
            identification: [ { position: 10 } ] }
    key booking_supplement_id as BookingSupplementID,

    @UI: { lineItem: [ { position: 20, importance: #HIGH } ],
            identification: [ { position: 20 } ] }
    @Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_SUPPLEMENT', element: 'Supplemen
                                            additionalBinding: [ { localElement: 'Price', element:
                                                                { localElement: 'CurrencyCode', el

    @ObjectModel.text.element: ['SupplementDescription']
    supplement_id as SupplementID,
    _SupplementText.Description as SupplementDescription: localized,
```

```

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
      identification: [ { position: 30 } ] }
@Semantics.amount.currencyCode: 'CurrencyCode'
price                           as Price,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
currency_code                   as CurrencyCode,

@UI.hidden: true
last_changed_at                as LastChangedAt,

/* Associations */
_Travel : redirected to /DM0/C_Travel_Processor_M,
_Booking : redirected to parent /DM0/C_Booking_Processor_M,
_SupplementText
}

```

For the data model of the booking supplement projection view, you can adopt all elements of the projected view.

All elements of the projection can be given an alias with an automatic mapping done by the service framework.

The booking supplement projection view uses a subset of the associations that are defined in the projected view. The association `_SupplementText` is needed for text provisioning. This association can simply be adopted in the projection view. On the other hand, the composition to the parent entity `_Booking` and to the root entity `_Travel` must be redirected as the target entity changes in the projection layer. The association `_Supplement` is not necessary in the projection view of the service context. It is defined in the underlying BO data model to complete the BO data model structure.

i Note

Now, that all compositions are redirected, you can activate the three projection views for the processor.

UI Specifics

Like in the travel and the booking projection view, the UI header information for the booking supplement projection view is given in an entity annotation.

! Restriction

The Fiori Elements Preview does not support the navigation to more than one child entity. Hence, when accessing the preview via entity set `Travel` and association to `_Booking`, it is not possible to navigate to the object page of the `BookingSupplement` entity. That means, some of the UI annotations are not relevant if you only use the preview to test your application. For example, the UI annotations referring to `identification` cannot be shown in the preview, when testing via the root node `Travel`.

However, if you want to develop a real application or test your service with the Web IDE, you can configure the application to enable navigation to any number of child entities. That is why, the UI annotation concerning `identification` are included in the following description.

You can imitate the behavior of the Web IDE for the second-level navigation by accessing the Fiori Elements Preview via entity set `Booking` and Association to `_BookSupplement`.

To show the entries of the booking supplement entity on its object page, the UI facet for `#IDENTIFICATION_REFERENCE` must be defined.

In addition, the elements for list items (to appear on the object page of the booking entity) and identification must be annotated in the Booking Supplement projection view with the respective UI annotation to define position, importance, and possible labels.

For more information about UI navigation and positioning of elements, see [Designing the User Interface for a Fiori Elements App](#) or [CDS Annotations](#).

As in the travel projection view, the annotation `@Semantics.amount.currencyCode: 'CurrencyCode'` needs to be repeated in the projection view, since the annotation value changes due to aliasing.

To be able to search for a specific data set, the booking supplement projection view must be search enabled and at least one element must be assigned as default search element. For more information about search enabling, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To access the corresponding texts or descriptions for ID elements, the relationship between the element `SupplementID` and its text element in the associated text provider view must be established. Therefore, you need the association to the text provider view. The text element of the text provider view must be integrated in the projection view. The text provider view must be associated to the projection view and the text element in the text provider view must be annotated with `@Semantics.text: true`. For more information about text provisioning, see [Working with Text Elements](#).

Especially for a data processing role, value helps are important to find the necessary values for the ID element `SupplementID` as well as for `CurrencyCode`. Value helps are defined with the annotation `@Consumption.valueHelpDefinition`. The value help for `SupplementID` uses additional binding, so that only those values appear that match the entry in the field `CurrencyCode` field. The value help provider view does not have to be associated to get the value help as the entity and the element are referenced in the annotation. For more information, see [Adding Value Help](#).

The administrative field `last_changed_at` is only used for concurrent processing and does not have to be displayed on the UI. The annotations `@UI.hidden = true` is used for that purpose.

Projection Views for the Approver BO Projection

To define a data model for the BO projection that defines the scope for the approver application, the following tasks need to be done:

- [Creating the Projection CDS Views for the Approver](#)
- [Defining the Data Model for the Approver Projection Views](#)

Creating the Projection CDS Views for the Approver

The scope of the UI service for the approver is more limited than for the processor. The approver can only modify the travel entity with accepting or rejecting the travel entries. The values in the corresponding booking entries are the basis for this decision-making. Only these two entities are relevant for the approver app. That means, only these two entities must be projected for the approver BO projection.

For the following CDS views, create the corresponding projection views by choosing the projection view template in the creation wizard for data definitions.

CDS views for BO structure	/DMO/I_TRAVEL_M	/DMO/I_BOOKING_M
CDS views for BO projection	/DMO/C_TRAVEL_APPROVER_M	/DMO/C_BOOKING_APPROVER_M

i Note

The names are assigned according to the naming conventions for projection views: [Naming Conventions for Development Objects](#).

```
define [root] view entity <projection_view>
  [provider contract transactional_query]
  as projection on <projected_view>
```

For more information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Defining the Data Model for the Approver Projection Views

The following topics provide you with a detailed description on how to define the data model for the CDS projection views that are used in the BO projection for the approver.

- [Travel Projection View /DMO/C_TRAVEL_APPROVER_M](#)
- [Booking Projection View /DMO/C_BOOKING_APPROVER_M](#)

Travel Projection View /DMO/C_TRAVEL_APPROVER_M

For the service-specific projection, the elements as well as all the UI specifics for the approver BO projection must be defined.

The data model defines which elements are exposed for the UI service. In addition, in data definitions you have to define all UI specifications.

The following UI is achieved by implementing the corresponding features in the CDS Travel projection view for the approver.

Preview: UI Application for Approver

Standard

Agency ID:

Customer ID:

Overall Status:

Search

Go

Adapt Filters

Travels (4,136)

Reject Travel

Accept Travel

Travel ID	Overall Status	Agency ID	Customer ID	Booking Fee	Total Price
1	Open	Maxitrip (70041)	Ryan (594)	40.00 USD	1,889.00 USD
2	Open	Hot Socks Travel (70007)	Prinz (608)	20.00 USD	900.00 USD

Travel List Report Page

2

Edit

Accept Travel

Reject Travel

Travel

Booking

Travel ID:

2

Overall Status:

Open

Agency ID:

Hot Socks Travel (70007)

Customer ID:

Prinz (608)

Starting Date:

Jan 8, 2022

End Date:

Jan 8, 2022

Booking Fee:

20.00 USD

Total Price:

900.00 USD

Description:

Business Trip for Christine, Pierre

Travel Object Page

i Expand the following code sample to view the source code of Travel Projection View /DMO/C_Travel_Approver_m.

≡ Sample Code

```
@EndUserText.label: 'Travel projection view'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@UI: {
  headerInfo: { typeName: 'Travel', typeNamePlural: 'Travels', title: { type: #STANDARD, value:

@Search.searchable: true

define root view entity /DMO/C_Travel_Approver_M
  provider contract transactional_query
  as projection on /DMO/I_Travel_M
{
  @UI.facet: [ { id:
                purpose:
                type:
                label:
                position:
                { id:
                purpose:
                type:
                label:
                position:
                targetElement:

  @UI: {
    listItem:      [ { position: 10, importance: #HIGH } ],
    identification: [ { position: 10 } ] }
  @Search.defaultSearchElement: true
key travel_id      as TravelID,

  @UI: {
    listItem:      [ { position: 20, importance: #HIGH } ],
    identification: [ { position: 20 } ],
    selectionField: [ { position: 20 } ] }
  @Consumption.valueHelpDefinition: [ { entity : {name: '/DMO/I_Agency', element: 'AgencyID'

  @ObjectModel.text.element: ['AgencyName']
  @Search.defaultSearchElement: true
agency_id      as AgencyID,
_Agency.Name   as AgencyName,

  @UI: {
    listItem:      [ { position: 30, importance: #HIGH } ],
    identification: [ { position: 30 } ],
    selectionField: [ { position: 30 } ] }
  @Consumption.valueHelpDefinition: [ { entity : {name: '/DMO/I_Customer', element: 'Custome

  @ObjectModel.text.element: ['CustomerName']
  @Search.defaultSearchElement: true
customer_id    as CustomerID,
_Customer.LastName as CustomerName,

  @UI: {
    identification:[ { position: 40 } ] }
begin_date     as BeginDate,

  @UI: {
    identification:[ { position: 41 } ] }
end_date       as EndDate,

  @UI: {
    listItem:      [ { position: 42, importance: #MEDIUM } ],
    identification: [ { position: 42 } ] }
  @Semantics.amount.currencyCode: 'CurrencyCode'
booking_fee    as BookingFee,

  @UI: {
    listItem:      [ { position: 43, importance: #MEDIUM } ],
    identification: [ { position: 43, label: 'Total Price' } ] }
```

```

@Semantics.amount.currencyCode: 'CurrencyCode'
total_price          as TotalPrice,

@Consumption.valueHelpDefinition: [ {entity: {name: 'I_Currency', element: 'Currency' } }
currency_code        as CurrencyCode,

@UI: {
  listItem:          [ { position: 15, importance: #HIGH },
                        { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trav
                        { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trav
  identification: [ { position: 15 },
                    { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trav
                    { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trav
  textArrangement: #TEXT_ONLY,
  selectionField: [ { position: 40 } ] }
@endUserText.label: 'Overall Status'
@Consumption.valueHelpDefinition: [{ entity: { name: '/DM0/I_Overall_Status_VH', element:
@ObjectModel.text.element: ['OverallStatusText']
overall_status        as OverallStatus,

@UI.hidden: true
_OverallStatus._Text.Text as OverallStatusText : localized,

@UI: {
  listItem: [ { position: 45, importance: #MEDIUM } ],
  identification:[ { position: 45 } ] }
description          as Description,

/* Administrative fields */
@UI.hidden: true
last_changed_at      as LastChangedAt,

/* Associations */
_booking : redirected to composition child /DM0/C_Booking_Approver_M,
_agency,
_customer,
_overallStatus
}

```

Explanation

Except for the actions, which are different in the processor and the approver projection, the CDS projection views for the processor and the approver BO are identical. Refer to [explanation](#) for a thorough description on the travel projection view.

Actions

The position and the label for the action button must be defined in the CDS projection views. In the case of an approver, the available actions concerning the travel entity set are **Accept Travel** and **Reject Travel**. The implementation of these actions is done in the behavior pool, see [Developing Actions](#). It is simply the UI appearance that needs to be configured in the projection view. The action buttons for the respective actions are designed to appear on the travel list report page and on the travel object page. That is why the annotations are used in the list item and identification UI annotation. When executing the action on the list report page, a travel instance must be selected to assign an instance for the instance-bound action. On the object page, the instance for which the action shall be executed is clear. For more information about the annotations to define the action buttons, see [Enabling Actions for UI Consumption](#).

Booking Projection View /DM0/C_BOOKING_APPROVER_M

The data model defines which elements are exposed for the UI service. In addition, in data definitions you have to define all UI specifications.

Preview: UI Application Approver

Booking**Bookings (2)** Standard* ▾

Search



Booking Number	Booking Date	Customer ID	Airline ID	Flight Number	Flight Date	Flight Price		Status	
1	Jan 6, 2022	Detemple (99)	United Airlines, Inc. (UA)	1537	Jan 8, 2022	438.00	USD	New	>
2	Jan 6, 2022	Sisko (660)	United Airlines, Inc. (UA)	1537	Jan 8, 2022	438.00	USD	New	>

Booking List Report Page

2 /

1**Booking**

Booking Number:

1

Airline ID:

United Airlines, Inc. (UA)

Flight Price:

438.00 USD

Booking Date:

Jan 6, 2022

Flight Number:

1537

Status:

New

Customer ID:

Detemple (99)

Flight Date:

Jan 8, 2022

Booking Object Page

i Expand the following code sample to view the source code of Booking Projection View /DMO/C_Booking_Approver_M.**Sample Code**

```

@EndUserText.label: 'Booking projection view'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@UI: {
  headerInfo: { typeName: 'Booking',
                 typeNamePlural: 'Bookings',
                 title: { type: #STANDARD, value: 'BookingID' } }
}

@Search.searchable: true

define view entity /DMO/C_Booking_Approver_M
  as projection on /DMO/I_Booking_M
  {
    @UI.facet: [ { id: 'Booking',
                   purpose: #STANDARD,
                   type: #IDENTIFICATION_REFERENCE,
                   label: 'Booking',
                   position: 10 } ]

    @Search.defaultSearchElement: true
    key travel_id as TravelID,

    @UI: { lineItem: [ { position: 20, importance: #HIGH } ],
          identification: [ { position: 20 } ] }
    @Search.defaultSearchElement: true
    key booking_id as BookingID,

    @UI: { lineItem: [ { position: 30, importance: #HIGH } ],
          identification: [ { position: 30 } ] }
    booking_date as BookingDate,

    @UI: { lineItem: [ { position: 40, importance: #HIGH } ],
          identification: [ { position: 40 } ],
          selectionField: [ { position: 10 } ] }
    @ObjectModel.text.element: ['CustomerName']
    @Search.defaultSearchElement: true
    customer_id as CustomerID,
    _Customer.LastName as CustomerName,
  }

```

```

@UI: { lineItem:      [ { position: 50, importance: #HIGH } ],
      identification: [ { position: 50 } ] }
@ObjectModel.text.element: ['CarrierName']
carrier_id          as CarrierID,
_Carrier.Name       as CarrierName,

@UI: { lineItem:      [ { position: 60, importance: #HIGH } ],
      identification: [ { position: 60 } ] }
connection_id       as ConnectionID,

@UI: { lineItem:      [ { position: 70, importance: #HIGH } ],
      identification: [ { position: 70 } ] }
flight_date         as FlightDate,

@UI: { lineItem:      [ { position: 80, importance: #HIGH } ],
      identification: [ { position: 80 } ] }
@Semantics.amount.currencyCode: 'CurrencyCode'
flight_price        as FlightPrice,

currency_code       as CurrencyCode,

@UI: { lineItem:      [ { position: 90, importance: #HIGH, label: 'Status' } ],
      identification: [ { position: 90, label: 'Status' } ],
      textArrangement: #TEXT_ONLY }
@Consumption.valueHelpDefinition: [{ entity: { name: '/DMO/I_Booking_Status_VH', element:
@ObjectModel.text.element: ['BookingStatusText']
booking_status      as BookingStatus,

@UI.hidden: true
_BookingStatus._Text.Text as BookingStatusText : localized,

/* Administrative fields */
@UI.hidden: true
last_changed_at     as LastChangedAt,

/* Associations */
_Travel : redirected to parent /DMO/C_Travel_Approver_M,
_Customer,
_Carrier,
_BookingStatus
}

```

Explanation

The CDS projection views for the processor and the approver BO are almost identical. Refer to [Booking Projection View /DMO/C BOOKING PROCESSOR M](#) for a thorough description on the booking projection view.

Providing Behavior for Projections

The behavior for the BO projection is defined in a behavior definition of type `projection`. The type is defined in the behavior definition header. The projection behavior definition provides the behavior for the projection CDS view. All characteristics and operations that you want to include in the BO projection must be listed explicitly. The keyword for this is `use`.

For a detailed description on the syntax of projection behavior definitions, see [CDS BDL - CDS behavior definition header, projection BDEF \(ABAP - Keyword Documentation\)](#) and [CDS BDL - entity behavior definition, projection BDEF \(ABAP - Keyword Documentation\)](#).

Defining the BO Projection Behavior in the Travel Scenario

As described in [Reference Business Scenario](#), the BO projections for the processor and the approver differ with regard to their behavior. The following sections provide a detailed description on how to project the existing BO to define a behavior for one

business object that is tailored to expose a UI service for a data processor and one that is tailored for a data approver.

- [Behavior for the Processor BO Projection](#)
- [Behavior for the Approver BO Projection](#)

Related Information

[Projection Behavior Definition](#)

Behavior for the Processor BO Projection

The behavior for the BO projection that defines the scope for the processor application is defined in a behavior definition with type projection.

Creating a Behavior Definition for the Processor BO Projection

The easiest way to create a projection behavior definition is to use the context menu in the project explorer by selecting the relevant projection root view `/DMO/C_TRAVEL_PROCESSOR_M` and choosing **New Behavior Definition**. The behavior definition always uses the same name as the corresponding root view.

As the behavior definition is created on the basis of the root projection view, the template with type `projection` is generated.

Defining the Behavior for the Processor BO Projection

When creating the behavior definition based on the projection view, the template automatically creates the type `projection` and lists all available characteristics and operations of the underlying behavior definition. That means, if nothing is done explicitly the BO projection has exactly the same behavior as the underlying BO.

For the processor projection, only the following elements are used:

```
projection;
strict(2);

define behavior for /DMO/C_Travel_Processor_M alias TravelProcessor
use etag

{
    field ( readonly ) TotalPrice;

    use create;
    use update;
    use delete;

    use action copyTravel;

    use association _BOOKING { create; }
}
```

```

define behavior for /DMO/C_Booking_Processor_M alias BookingProcessor
use etag
{
    use update;
    // use delete; // workaround for missing determination on delete

    use association _BOOKSUPPLEMENT { create; }
    use association _Travel { }
}

define behavior for /DMO/C_BookSuppl_Processor_M alias BookSupplProcessor
use etag
{
    use update;
    // use delete; // workaround for missing determination on delete
    use association _Travel { }
    use association _Booking { }
}

```

Explanation

Only the characteristics and operations that are relevant for the processor are used in the projection behavior definition. This is only a subset of the behavior that was defined in the underlying BO. See [Developing Business Logic](#) to compare the projection BO to the underlying one.

The ETag handling that was defined in the underlying BO is used for all three entities. Especially for the processor role, which is enabled to modify, it is necessary to have a concurrency check. By using a master ETag on all entities, concurrent processing is enabled for the travel BO.

The static field control that was defined for the underlying BO cannot be modified.

All standard operations are used for the processor on all the root entity. The child entities can only be created via a `create_by_association`. The delete is not enabled for the view `Booking` and `BookingSupplement` as the determination to calculate the total flight price is not triggered on `delete`.

For the travel entity, the action to copy travels is enabled for the processor.

Behavior for the Approver BO Projection

The behavior for the BO projection that defines the scope for the approver application id defined in a behavior definition with type `projection`.

Creating a Behavior Definition for the Approver BO Projection

The easiest way to create a projection behavior definition is to use the context menu in the project explorer by selecting the relevant projection root view `/DMO/C_TRAVEL_APPROVER_M` and choosing **New Behavior Definition**. The behavior definition always uses the same name as the corresponding root view.

As the behavior definition is created on the basis of the root projection view, the template with type `projection` is generated.

Defining the Behavior for the Approver BO Projection

When creating the behavior definition based on the projection, the template automatically creates the type `projection` and lists all available characteristics and operations of the underlying behavior definition. That means, if nothing is done explicitly the BO projection has exactly the same behavior as the underlying BO.

For the approver projection, only the following elements are used:

```
projection;
strict(2);

define behavior for /DMO/C_Travel_Approver_M alias Approver
use etag
{
    field ( readonly ) BeginDate, EndDate, TotalPrice, CustomerID;

    use update;

    use action acceptTravel;
    use action rejectTravel;
}
```

Explanation

Only the characteristics and operations that are relevant for the approver are used in the projection behavior definition. This is only a subset of the behavior that was defined in the underlying BO. See [Developing Business Logic](#) to compare the projection BO to the underlying one.

The `eTag` that was defined in the underlying BO is used in the approver projection as well. The concurrency check is relevant for the approver. It must be ensured that the data has not been changed between checking the data and executing the action `accept` or `reject` travel.

The static field control that was defined for the underlying BO cannot be modified. However, new controls are added to correspond the approver role. The fields that are mandatory for the processor are set to read-only for the approver. In the approver application, one can only change the fields `OverallStatus`, `AgencyID`, `BookingFee` and `Description`.

The `update` operation is enabled for the approver as a modification on the travel entries must be available.

The actions `accept` and `reject` are enabled to change the status of the travel entry.

There is no behavior defined for the booking entity. All fields are read-only in this case.

The booking supplement entity is not part of the approver BO projection, so there is no behavior for this entity either.

Defining Business Services Based on Projections

With the help of the previous sections, you developed a business object and its projection for two complementary business roles. The next step is to build a business service for both projections in order to consume the business object. The business service defines the scope of the service and binds it to a specific OData protocol. For more information, see [Business Service](#).

This scenario is designed to build a UI service for both business object projections. Follow the development steps to build an application for both BO projections. For a detailed step-by-step description, see [Creating an OData Service](#).

1. Create a service definition for the processor service and one for the approver service.
2. Expose the relevant CDS views for each service.

i Note

Only the projection CDS views of the business object projection must be exposed for the service. The delegation to the underlying BO is automatically done.

a. Service Definition for Processor Service:

```
@EndUserText.label: 'Service definition for managing travels'
@ObjectModel.leadingEntity.name: '/DMO/C_Travel_Processor_M'
define service /DMO/UI_TRAVEL_PROCESSOR_M {
    expose /DMO/C_Travel_Processor_M as Travel;
    expose /DMO/C_Booking_Processor_M as Booking;
    expose /DMO/C_BookSuppl_Processor_M as BookingSupplement;
    expose /DMO/I_Supplement_StdVH as Supplement;
    expose /DMO/I_SupplementCategory_VH as SupplementCategory;
    expose /DMO/I_Customer_StdVH as Passenger;
    expose /DMO/I_Agency_StdVH as TravelAgency;
    expose /DMO/I_Carrier_StdVH as Airline;
    expose /DMO/I_Connection_StdVH as FlightConnection;
    expose /DMO/I_Flight_StdVH as Flight;
    expose /DMO/I_Airport_StdVH as Airport;
    expose /DMO/I_Overall_Status_VH as OverallStatus;
    expose /DMO/I_Booking_Status_VH as BookingStatus;
    expose I_CurrencyStdVH as Currency;
    expose I_CountryVH as Country;
}
```

The complete composition hierarchy is exposed for the data processing service. In addition, the text and value help provider views are necessary components of the service scope to get the respective feature for the service.

b. Service Definition for Approver Service

```
@EndUserText.label: 'Service definition for managing travels'
@ObjectModel.leadingEntity.name: '/DMO/C_Travel_Approver_M'
define service /DMO/UI_TRAVEL_Approver_M {
    expose /DMO/C_Travel_Approver_M as Travel;
    expose /DMO/C_Booking_Approver_M as Booking;
    expose /DMO/I_Customer_StdVH as Passenger;
    expose /DMO/I_Agency_StdVH as TravelAgency;
    expose I_CurrencyStdVH as Currency;
    expose I_CountryVH as Country;
    expose /DMO/I_Overall_Status_VH as OverallStatus;
    expose /DMO/I_Booking_Status_VH as BookingStatus;
}
```

The approver service contains only two entities of the travel BO and a more limited number of text and value help provider views.

3. Create a service binding with binding type ODATA V2 - UI for both service definitions and activate the local service endpoints.

i Note

It's recommended to use OData V4 instead of OData V2 for future-proof design.

As soon as the service is activated, it is ready for consumption through an OData client such as an SAP Fiori app. You can use the preview function in the service binding to check how the UI of the Fiori application looks like.

Developing Unmanaged Transactional Apps

This section explains the main development tasks required for enabling transactional processing in a business objects provider that integrates existing business logic.

Based on an end-to-end example, you create and implement all requisite artifacts for providing OData services that combine CDS data model and business object semantics with transactional processing from legacy application logic.

Introduction

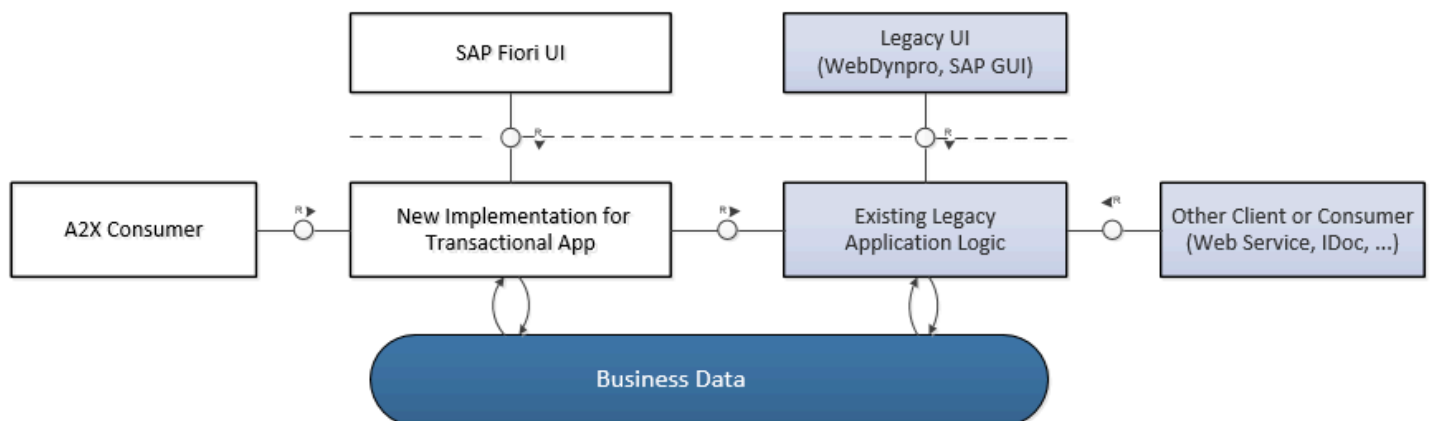
In unmanaged business objects, the application developer must implement essential components of the REST contract. There is no standard functionality provided by the RAP framework. It is only the frame of a business object, that an application developer needs to adhere to. The behavior must be specified in the behavior definition but implemented manually in ABAP classes in the behavior pool. All existing business logic can be integrated, including standard behavior like create, update and delete. In addition, the transactional buffer must be provided and handled by the application developer in the ABAP behavior pool.

i Note

In a managed implementation type, on the other hand, a behavior definition would on its own be sufficient to obtain a ready-to-run business object.

Architecture Overview

The underlying scenario reuses the existing business application logic and the existing persistence, which manages business data.



Architecture Overview – Integration of Existing Application Logic

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).

→ Remember

The namespace `/DMO/` is reserved for the demo content. Apart from the downloaded **ABAP Flight Scenario**, do not use the namespace `/DMO/` and do not create any development objects in the downloaded packages.

You can access the development objects in `/DMO/` from your own namespace.

However, if you want to recreate all development objects of this demo content, make sure that you use different names from this documentation.

Development Process in Overview

The development of new business services by integrating the transactional behavior of an existing (legacy) application mainly requires developers to perform the following fundamental activities:

1. Defining a CDS Data Model and the Business Object Structure

The formal structure of a business object consists of a tree of entities (**Travel**, **Booking**, **Passenger**, and so on) where the entities are linked using associations. Each entity of this tree structure is an element that is modeled with a CDS entity. Entities of this kind are CDS views that are generally defined on top of the underlying persistence layer, which in turn is based on the corresponding database tables or public interface CDS views.

The root entity is of particular importance: this is indicated in the source code of the CDS data definition by the keyword `ROOT`. The root entity is a representation of the business object and defines the top node in a business object's structure.

The structure of the business object is projected in CDS projection views where you can define service-specifics for the data model. For the UI-related annotations, metadata extensions are used to separate data model from domain-specific semantics

More on this: [Providing CDS Data Model with Business Object Structure](#)

2. Defining and Implementing the Transactional Behavior of Business Objects

Each node of a business object can offer the standard operations `create()`, `update()`, and `delete()` and specific operations with a dedicated input and output structure known as actions. All operations provided by a business object are defined in the behavior definition artifact that is created as an ABAP repository object.

The implementation of the transactional behavior is done in specific class pools, which refer to the behavior definition. The concrete implementation of the business object provider is based on the ABAP language (which has been expanded from the standard with a special syntax) and the corresponding **API for Implementing the Unmanaged BO Contract**. The implementation tasks are roughly divided into an **interaction phase** and a **save sequence**.

The behavior that is relevant for the specific UI service is then projected in projection behavior definition.

More on this: [Defining and Implementing Behavior of the Business Object](#)

3. Exposing the Relevant Application Artifacts for OData Service Enablement

For the service enablement, the relevant artifacts must be exposed to OData as a canonical OData service. This is implemented by data and behavior models, where the data model and the related behavior is projected in a service-specific way. This projection is separated into two different artifacts: the service definition and the service binding. The service definition is a projection of the data model and the related behavior to be exposed, whereas the service binding implements a specific protocol and the kind of service to be offered to a consumer.

More on this: [Defining Business Service for Fiori UI](#)

4. Testing the OData (UI) Service

In ABAP Development Tools, you have the option of publishing the service to the local system repository. As soon as the service is published, it is ready for consumption through an OData client, such as an SAP Fiori app. The service binding editor offers a preview tool that you can use for testing the resulting app within your ABAP development environment.

i Note

Via ABAPGit You can import the service including the related development objects into your development environment for comparison and reuse. You find the service in the package /DMO/FLIGHT_UNMANAGED. The suffix for development objects in this development guide is _U.

For information about downloading the ABAP Flight Reference Scenario, see [Downloading the ABAP Flight Reference Scenario](#).

Reference Business Scenario

The **Managing of Flight Travels** scenario used in this guide provides an example of an existing stateful business application whose business logic is reused in the new implementation for transactional apps. This application represents only a part of the full **ABAP Flight Reference Scenario** (in short: Flight Scenario) that is intended to be used for demonstration and learning purposes in the context of the ABAP RESTful programming model.

The application demo provided (which represents a legacy stateful application) allows a user to create and manipulate flight bookings. It involves different data sources and entities such as travel, travel agencies, customers (passengers), flights, and bookings. Some of these are editable (that is, they can be created or manipulated) and some are not.

Persistency and Data Model of an Existing Application

The following table gives an overview of the different travel entities involved in the current scenario, including a categorization into editable and non-editable entities.

Flight Reference Scenario Objects Involved in the Business Scenario

Entity	Description	Editable
Travel	A Travel entity defines general travel data, such as the agency ID or customer ID, status of the travel booking, and the price of travel. The travel data is stored in the database table /DMO/TRAVEL.	Yes
Agency	An Agency entity defines travel agency data, such as the address and contact data. The corresponding data is stored in the database table /DMO/AGENCY. The flight data model defines a 1 : n cardinality between Agency and Travel.	No
Booking	The booking data is stored in the database table /DMO/BOOKING. The flight data model defines a 1 : n cardinality between a Travel and the Booking entity.	Yes
Flight	The specific flight data for each connection is stored in the database table /DMO/FLIGHT. The flight data model defines a 1 : n cardinality between a Connection and the Flight	No

Entity	Description	Editable
	entity.	
Connection	The flight connections are stored in the database table /DMO/CONNECTION.	No
Carrier	The IDs and names of airlines are stored in the database table /DMO/CARRIER. Each airline has a number of flight connections. Therefore, the data model defines a 1:n cardinality between a Carrier and the Connection entity.	No
Customer	A Customer entity provides a detailed description of a flight customer (passenger) such as the name, the address, and contact data. The corresponding data is stored in the database table /DMO/CUSTOMER. The flight data model defines a 1:n cardinality between Customer and Travel.	No
Booking Supplement	This entity is used to add additional products to a travel booking. The booking supplement data is stored in the database table /DMO/BOOK_SUPPL. The flight data model defines an n:1 cardinality between a Booking Supplement entity and a Booking entity.	Yes
Supplement	A Supplement entity defines product data that the customer can book together with a flight, for example a drink or a meal. The supplement data is stored in the database table /DMO/SUPPLEMENT. The flight data model defines a 1:1 cardinality between a Supplement entity and the Booking Supplement entity.	No
Travel Status	This entity is used to provide the possible values for the travel status. The corresponding data is stored in the database table /dmo/trvl_stat. The flight data model defines a 1:1 cardinality between Travel and Travel Status. The localized texts for the travel statuses are provided by the Travel Status Text entity.	No
Travel Status Text	This entity is used to provide the localized texts for the travel statuses. The corresponding data is stored in the database table /dmo/trvl_stat_t. The flight data model defines a 1:N cardinality between Travel Status and Travel Status Text.	No

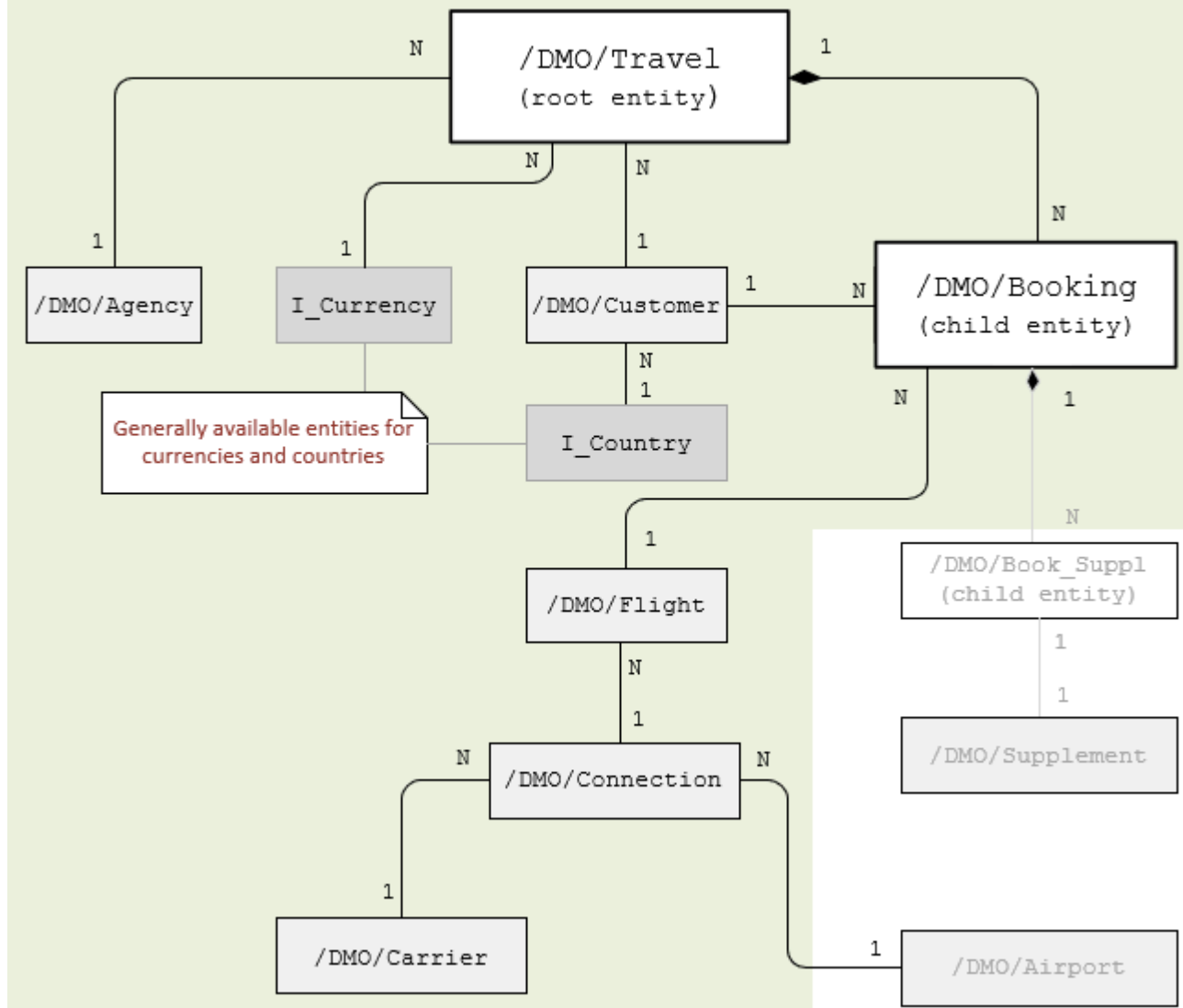
Compositions and Associations

The figure below shows the relationships between the travel, travel status, agency, customer, and booking entities, where the travel entity represents the root of the data model. Additional entities for currencies (I_Currency) and countries (I_Country) are generally available in your system and are included in our data model using associations.

i Note

For didactic reasons, we have kept the data model as simple as possible. We have hence reduced the number of entities in our end-to-end guide (compared with the predefined ABAP flight model) to a minimum set of entities.

Focus on reduced data model in this E2E guide

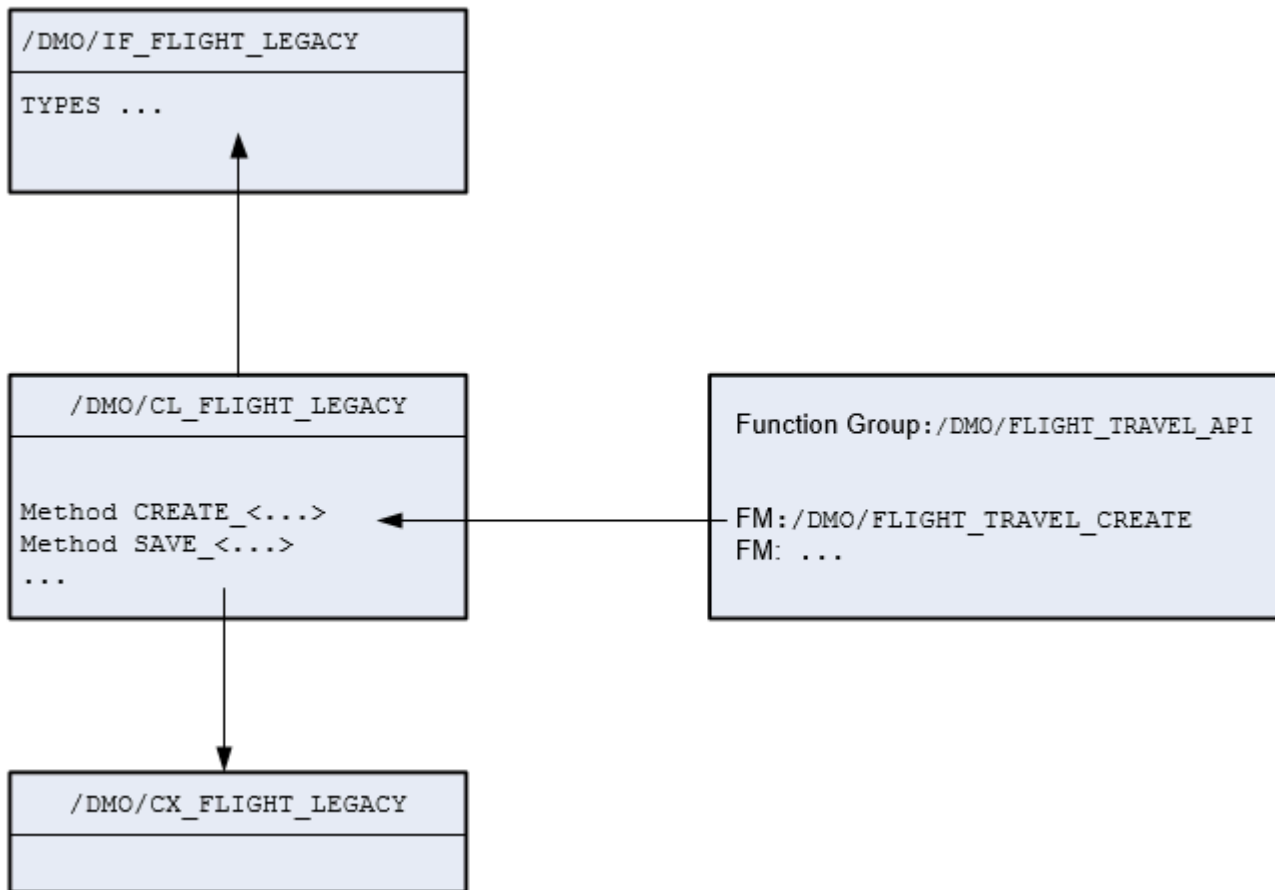


Travel Entities Involved in the Present Scenario and Their Relationships

Business Logic

The following figure summarizes the essential elements of the business logic:

- The function group `/DMO/FLIGHT_TRAVEL_API` is used to group all function modules that represent the application's legacy code.
- The class `/DMO/CL_FLIGHT_LEGACY` provides the actual implementation of the business logic in a more convenient (object-oriented) way.
- The interface `/DMO/IF_FLIGHT_LEGACY` defines global types and constants for reuse.
- Exception class `/DMO/CX_FLIGHT_LEGACY`.



Main Elements of the Legacy Business Logic

Providing CDS Data Model with Business Object Structure

From a structural point of view, a business object consists of a tree of entities that are linked by special associations known as compositions. A composition is a specialized association that defines a whole-part relationship. A composite part only exists together with its parent entity (whole).

i Note

For didactic reasons, we will demonstrate a one-level composition in our sample application, defining a root entity for the Travel BO and one child entity for bookings. This relationship also means that booking data can only be created to a given travel instance.

Every entity in this composition tree is an element that is modeled with a CDS entity. The root entity is of particular importance, since it defines the top node in a business object's structure and serves as a representation of the business object. This is considered in the source code of the CDS data definition with the keyword `ROOT` when defining the corresponding CDS entity.

For details about the syntax for defining a root entity, see [CDS DDL - DEFINE VIEW ENTITY \(ABAP Keyword Documentation\)](#)

Effect:

Using this syntax, you define the `root_entity` as a root of the compositional hierarchy for the business object to be created.

With the keyword `COMPOSITION`, a `child_entity` is defined as a direct child entity to the business object's root. The `child_entity` is a CDS entity, which is the target of the composition. `_comp_name` defines the name of the composition and must be added to the `element_list` (like associations). The cardinality to the child entity is expressed in the composition definition with square brackets `[min .. max]`.

For `min` and `max`, positive integers (including 0) and asterisks (*) can be specified:

- max cannot be 0.
- An asterisk * for max means any number of rows.
- min cannot be *.

The meaning of the other elements of the syntax is identical to that of `DEFINE VIEW ENTITY`.

Further information: [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

For details about the syntax for defining a child entity, see [CDS DDL - DEFINE VIEW ENTITY \(ABAP Keyword Documentation\)](#).

Effect:

Using this syntax, you define a CDS entity `child_entity` that serves as a sub node in the compositional hierarchy of the business object structure. The sub node is a node in a business object's structure that is directly connected to another node when moving away from the root.

CDS entities that do not represent the root node of the hierarchy must have an association to their compositional parent entity `parent_entity` or `root_entity`. This relationship is expressed by the keyword `ASSOCIATION TO PARENT...`

The meaning of the other elements in the association syntax is identical to that of `ASSOCIATION` in the CDS `SELECT` statement.

Next Steps

[Creating Data Definitions for CDS Views](#)

[Defining the Data Model in CDS Views](#)

Creating Data Definitions for CDS Views

In this step you create a CDS views as the basis for the data model of our demo scenario. To do this, you create the appropriate data definitions as transportable ABAP repository objects, as specified in the table below.

Data Definitions and CDS Views to Create

i Note

Naming CDS views: Since CDS views are (public) interface views, they are prefixed with `I_` in accordance with the VDM (virtual data model) naming convention. In addition, we add the suffix `_U` to the view name in case it is specific for our unmanaged implementation type scenario. For detailed information, see: [Naming Conventions for Development Objects](#)

→ Remember

The namespace `/DMO/` is reserved for the demo content. Therefore, do not use the namespace `/DMO/` when creating your own development objects and do not create any development objects in the downloaded packages.

Data Definitions Required for the Root Node (Travel):

Data Definition CDS View Name	Data Source	Description
/DMO/I_TRAVEL_U /DMO/I_Travel_U	/DMO/TRAVEL (DB table)	This CDS view defines the root entity. The root entity is a representation of the travel business object and defines the top node in a business object's structure. It is used for managing general travel data, such as the booking status of a travel or the total price of a travel.
/DMO/I_AGENCY /DMO/I_Agency	/DMO/AGENCY (DB table)	This CDS view represents the travel agency in the data model of our demo scenario.
/DMO/I_CUSTOMER /DMO/I_Customer	/DMO/CUSTOMER (DB table)	This CDS view defines the data model for managing flight travel customers (passengers).
/DMO/I_TRAVEL_STATUS_VH /DMO/I_Travel_Status_VH	/dmo/trvl_stat (DB table)	This CDS view defines the data model for managing travel statuses.
/DMO/I_TRAVEL_STATUS_VH_TEXT /DMO/I_Travel_Status_VH_Text	/dmo/trvl_stat_t (DB table)	This CDS view defines the data model for managing localized texts of travel statuses.

Data Definitions Required for the Sub Node (Booking):

Data Definition CDS View Name	Data Source	Description
/DMO/I_BOOKING_U /DMO/I_Booking_U	/DMO/BOOKING (DB table)	This CDS view defines the flight booking entity. The booking entity is a sub node representation of the travel business object structure. It is used for managing flight booking data, such as the customer, the flight connection, or the price and flight date.
/DMO/I_FLIGHT /DMO/I_Flight	/DMO/FLIGHT (DB table)	This CDS view represents the concrete flights in the travel data model. In our demo scenario, the CDS view is used for value help definition for specific elements in the booking view.
/DMO/I_CONNECTION /DMO/I_Connection	/DMO/CONNECTION (DB table)	This CDS view defines the data model for managing flight connections. In our demo scenario, the connection view is used to retrieve the text information for the associated elements in the booking view.
/DMO/I_CARRIER /DMO/I_Carrier	/DMO/CARRIER (DB table)	This CDS view defines the data model for managing the airline data (ID and the name). In our demo scenario, the carrier view is used to retrieve the text information for the associated elements in the booking view.

Procedure: Creating a Data Definition

To launch the wizard tool for creating a data definition, do the following:

1. Launch [ABAP Development Tools](#).
2. In your ABAP project (or ABAP cloud project), select the relevant package node in [Project Explorer](#).

Results

This procedure creates a data definition as a transportable development object in the selected package. For each data definition, the related CDS view is created.

Defining the Data Model in CDS Views

Travel Root View /DM0/I_Travel_U

The listing 1 (below) provides you with the implementation of the CDS data model for managing flights, where the database table `/dmo/travel` serves as the data source for the corresponding CDS view `/DM0/I_Travel_U` (note the camel case notation).

This CDS view defines the root entity of the data model and represents the root of the compositional hierarchy for the travel business object to be created.

From a structural point of view, a business object consists of a tree of nodes that are linked by special associations known as compositions. To define a composition relationship from the root to a child entity the keyword `COMPOSITION` is used. In our example, you specify the `/DM0/I_Booking_U` as child entity of the composition `_Booking`. As a result, the booking node is defined as a direct sub node to the business object's root. With cardinality `[0 .. *]` you express that any number of booking instances can be assigned to each travel instance.

To be able to access business data from semantically related entities, a set of associations is defined in the CDS source code. These associations refer to CDS views that are part of our demo application scenario. Some of these views are used primarily as text views for retrieving text information and as value help provider views for specific UI fields, see [Projecting the Data Model in CDS Projection Views](#).

Except for the administrative fields `createdby`, `lastchangedby`, and `createdat`, all fields of the data source table `/dmo/travel` have been added to the element list in the CDS view. The database table provides several administrative fields that are used for administrative data which usually includes the user who created or last changed an instance and the corresponding timestamps. In this example however, the element `LastChangedAt` plays a special part, as it is used for ETag checks to determine whether two representations of an entity are the same. If the representation of the entity ever changes, a new and different ETag value is assigned. ETags play a significant part in the lock lifetime when working with business objects.

For all elements, we can provide an alias to provide a readable name for the elements.

The price elements and the element `CurrencyCode` have a semantic relationship. This relationship is manifested in the business object view via the `@semantics` annotations. In a Fiori UI, the amount and the currency value can then be displayed together.

i *Expand the following code sample to view the source code of CDS Root View /DM0/I_Travel_U.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Travel view - CDS data model'

define root view entity /DM0/I_Travel_U
  as select from /dmo/travel as Travel -- the travel table is the data source for this view

  composition [0..*] of /DM0/I_Booking_U as _Booking

  association [0..1] to /DM0/I_Agency      as _Agency on $projection.AgencyID = _Agency.AgencyID
  association [0..1] to /DM0/I_Customer  as _Customer on $projection.CustomerID = _Customer.Cust
  association [0..1] to I_Currency        as _Currency on $projection.CurrencyCode = _Currency.Cu
```

```

association [1..1] to /DM0/I_Travel_Status_VH as _TravelStatus on $projection.Status = _Travel
{
  key Travel.travel_id      as TravelID,
    Travel.agency_id        as AgencyID,
    Travel.customer_id      as CustomerID,
    Travel.begin_date       as BeginDate,
    Travel.end_date         as EndDate,

    @Semantics.amount.currencyCode: 'CurrencyCode'
    Travel.booking_fee      as BookingFee,

    @Semantics.amount.currencyCode: 'CurrencyCode'
    Travel.total_price      as TotalPrice,

    Travel.currency_code    as CurrencyCode,
    Travel.description      as Memo,
    Travel.status           as Status,
    Travel.lastchangedat    as LastChangedAt,

    /* Associations */
    _Booking,
    _Agency,
    _Customer,
    _Currency,
    _TravelStatus
}

```

Booking View /DM0/I_Booking_U

Listing 2 (below) provides you with a data model implementation of the booking entity. In the data definition of the root entity /DM0/I_Travel_U, you specified the booking entity /DM0/I_Booking_U as a child entity. This composition relationship requires an association to their compositional parent entity for the booking child entity to be specified in the data model implementation. This relationship is expressed by the keyword **ASSOCIATION TO PARENT**. Using this syntax, you define the CDS entity /DM0/I_Booking_U as a direct sub node in the compositional hierarchy of the travel business object structure.

The **SELECT** list includes all elements of a booking entity that are relevant for building the BO structure. These elements are provided with an alias. Like in the Travel CDS view, the semantic relationship between Price and CurrencyCode is established with the **@Semantics** annotation.

To be able to access data from other entities, a set of additional associations (**_Customer**, **_Carrier**, and **_Connection**) is defined in the CDS source code. These views are primarily used as text views for retrieving text information and as value help provider views for specific booking fields on the UI, [Projecting the Data Model in CDS Projection Views](#).

i Expand the following code sample to view the source code of CDS view /DM0/I_Booking_U.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Booking view'

define view entity /DM0/I_Booking_U
  as select from /dmo/booking as Booking

  association          to parent /DM0/I_Travel_U      as _Travel      on $projection.TravelID = _Tr

```

```

association [1..1] to /DMO/I_Customer      as _Customer on $projection.CustomerID = _C
association [1..1] to /DMO/I_Carrier       as _Carrier  on $projection.AirlineID = _C
association [1..1] to /DMO/I_Connection   as _Connection on $projection.AirlineID = _C
                                           and $projection.ConnectionID =
{
  key Booking.travel_id      as TravelID,
  key Booking.booking_id     as BookingID,
  Booking.booking_date       as BookingDate,
  Booking.customer_id        as CustomerID,
  Booking.carrier_id         as AirlineID,
  Booking.connection_id      as ConnectionID,
  Booking.flight_date        as FlightDate,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  Booking.flight_price       as FlightPrice,
  Booking.currency_code      as CurrencyCode,
  /* Associations */
  _Travel,
  _Customer,
  _Carrier,
  _Connection
}

```

Travel Status View /DMO/I_Travel_Status_VH

Listing 3 (below) provides you with a data definition for handling travel status data. The CDS view /DMO/I_Travel_Status_VH serves as a value help view providing the possible travel status values.

The database table /dmo/trvl_stat is the data source for the CDS view. The travel status field in table /dmo/trvl_stat has been added to the element list in the CDS view.

To enable a value selection via a drop-down-menu, the annotation `@ObjectModel.resultSet.sizeCategory: #XS` is used.

The associated CDS view /DMO/I_Travel_Status_VH_Text serves as a provider view for localized texts. Hence, the CDS view /DMO/I_Travel_Status_VH contains an association as well as the annotation `@ObjectModel.text.association: '_Text'` pointing to the CDS view /DMO/I_Travel_Status_VH_Text.

To ensure that only the texts provided by the associated CDS view /DMO/I_Travel_Status_VH_Text are displayed in the value help, the annotation `@UI.textArrangement: #TEXT_ONLY` is used.

The annotation `@UI.lineItem: [{importance: #HIGH}]` ensures that the travel status data are displayed even when little display space is available.

The `@Objectmodel.usageType` annotations define the expected performance parameters and contained data class for the value help view. For more information, refer to [CDS Annotations](#).

i Expand the following code sample to view the source code of the travel status view /DMO/I_Travel_Status_VH.

Sample Code

```

@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED

```

```

@EndUserText.label: 'Travel Status Value Help'
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType:{
  serviceQuality: #A,
  sizeCategory: #S,
  dataClass: #MASTER
}
@ObjectModel.resultSet.sizeCategory: #XS
define view entity /DMO/I_Travel_Status_VH
  as select from /dmo/trvl_stat

  association [0..*] to /DMO/I_Travel_Status_VH_Text as _Text on $projection.TravelStatus = _Text
{
  @UI.textArrangement: #TEXT_ONLY
  @UI.lineItem: [{importance: #HIGH}]
  @ObjectModel.text.association: '_Text'
  key travel_status as TravelStatus,
    _Text
}

```

Travel Status Text View /DMO/I_Travel_Status_VH_Text

Listing 4 (below) provides you with a data definition for handling travel status text data. This CDS entity provides the texts for the travel statuses on the list report page and in the value help.

The database table /dmo/trvl_stat_t is the data source for the corresponding CDS view /DMO/I_Travel_Status_VH_Text. All fields have been added to the element list in the CDS view.

To enable a value selection via a drop-down-menu, the annotation `@ObjectModel.resultSet.sizeCategory: #XS` is used.

The annotation `@ObjectModel.text.element: ['Text']` establishes the link between the annotated element and its descriptive element Text. By using the annotation `@Semantics.text: true` for the element Text, at runtime, this field is read from the database and filtered by the logon language of the OData consumer automatically. The annotation `@Semantics.language: true` specifies the field used for this language filter.

The `@ObjectModel.usageType` annotations define the expected performance parameters and contained data class for the value help view. For more information, refer to [CDS Annotations](#).

i *Expand the following code sample to view the source code of travel status text view /DMO/I_Travel_Status_VH_Text.*

Sample Code

```

@AbapCatalog.viewEnhancementCategory: [#NONE]
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Travel Status Value Help'
@Metadata.ignorePropagatedAnnotations: true
@ObjectModel.usageType:{
  serviceQuality: #A,
  sizeCategory: #S,
  dataClass: #MASTER
}
@ObjectModel.resultSet.sizeCategory: #XS
define view entity /DMO/I_Travel_Status_VH_Text
  as select from /dmo/trvl_stat_t

  association [1..1] to /DMO/I_Travel_Status_VH as _TravelStatus on $projection.TravelStatus = _
{
  @ObjectModel.text.element: ['Text']
  key travel_status as TravelStatus,

  @Semantics.language: true
}

```

```

    key language          as Language,

    @Semantics.text: true
    text                  as Text,

    _TravelStatus
}

```

Travel Agency View /DMO/I_Agency

Listing 5 (below) provides you with a data definition for handling travel agency data. The database table /dmo/agency is the data source for the corresponding CDS view /DMO/I_Agency.

Since the travel agency's data can vary from one country to another, the data model refers to the I_Country view using the association _Country.

This CDS entity also serves as a text provider view. For this purpose, the annotation @Semantics.text: true is used to identify the Name element as a text element, which - in this case - points to a textual description of agency names. In the travel and the booking projection views, the associated text element is added as a field to the referencing entity. At runtime, this field is read from the database and filtered by the logon language of the OData consumer automatically.

In addition, the data model enables search capabilities on the Name element: The annotation @Search.searchable: true marks the CDS view as searchable, whereas @Search.defaultSearchElement: true specifies that the annotated Name element is to be considered in a full-text search. For detailed information, see: [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#)

i Expand the following code sample to view the source code of agency CDS view /DMO/I_Agency.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Agency view - CDS data model'
@Search.searchable: true

define view entity /DMO/I_Agency

  as select from /dmo/agency as Agency -- the agency table serves as the data source for this vi
  association [0..1] to I_Country as _Country on $projection.CountryCode = _Country.Country

{

  key Agency.agency_id          as AgencyID,
  @Semantics.text: true
  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.8
  Agency.name                   as Name,
  Agency.street                  as Street,
  Agency.postal_code             as PostalCode,
  Agency.city                    as City,
  Agency.country_code            as CountryCode,
  Agency.phone_number            as PhoneNumber,
  Agency.email_address           as EMailAddress,
  Agency.web_address             as WebAddress,

  /* Associations */
  _Country
}

```

Customer View /DMO/I_Customer

Listing 6 (below) is used as a data model implementation for managing passenger data. The database table `/dmo/customer` serves as the data source for the corresponding CDS view `/DMO/I_Customer`.

Except for the administrative fields (`createdby`, `createdat`, `lastchangedat` and `lastchangedby`), all fields of the table `/dmo/customer` have been added to the element list in the CDS view.

Since a passenger's data can vary from one country to another, the data model refers to the `I_Country` view using a corresponding association `_Country`.

The annotation `@Semantics.text: true` is added to the `LastName` element. This element serves as a text element, which - in this case - points to texts with customer names. This text annotation allows you to use this customer view as a text provider for the associated elements in the target views `/DMO/I_Travel_U` and `/DMO/I_Booking_U`.

i Expand the following code sample to view the source code of CDS view `/DMO/I_Customer`.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Customer view - CDS data model'
@Search.searchable: true

define view entity /DMO/I_Customer
  as select from /dmo/customer as Customer -- the customer table serves as the data source

  association [0..1] to I_Country as _Country on $projection.CountryCode = _Country.Country

{
  key Customer.customer_id      as CustomerID,
  Customer.first_name           as FirstName,
  @Semantics.text: true
  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.8
  Customer.last_name            as LastName,
  Customer.title                as Title,
  Customer.street               as Street,
  Customer.postal_code          as PostalCode,
  Customer.city                 as City,
  Customer.country_code         as CountryCode,
  Customer.phone_number         as PhoneNumber,
  Customer.email_address        as EMailAddress,

  /* Associations */
  _Country
}
```

Flight View `/DMO/I_Flight`

Listing 7 (below) provides you with a data definition for flights. The data of specific flights is stored in the database table `/dmo/flight`, which serves as the data source for the corresponding CDS view `/DMO/I_Flight`.

As demonstrated in listing 2 (above), you can implement the value help with additional binding. To define filter conditions for the value help based on the same value help provider, the flight view `/DMO/I_Flight` is used to filter the value help result list for the annotated elements.

i Expand the following code sample to view the source code of CDS view `/DMO/I_Flight`.

Sample Code


```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Flight view'

define view entity /DMO/I_Flight as select from /dmo/flight as Flight
{
    key Flight.carrier_id           as AirlineID,
    key Flight.connection_id       as ConnectionID,
    key Flight.flight_date         as FlightDate,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    Flight.price                   as Price,
    Flight.currency_code           as CurrencyCode,
    Flight.plane_type_id           as PlaneType,
    Flight.seats_max               as MaximumSeats,
    Flight.seats_occupied          as OccupiedSeats
}

```

Flight Connections View /DMO/I_Connection

Listing 8 (below) is used as a data definition for flight connections. The flight connections are stored in the database table `/dmo/connection`, which serves as the data source for the corresponding CDS view `/DMO/I_Connection`.

Except for the administrative fields, all fields in the table `/dmo/connection` have been added to the element list in the CDS view.

Using the annotation `@Semantics.quantity.unitOfMeasure`, the `DistanceUnit` element is referenced as an element containing a unit of measure for the value stored in the element `Distance`.

i Expand the following code sample to view the source code of CDS view `/DMO/I_Connection`.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Connection view'

define view entity /DMO/I_Connection
as select from /dmo/connection as Connection
{
    key Connection.carrier_id           as AirlineID,
    key Connection.connection_id       as ConnectionID,

    Connection.airport_from_id         as DepartureAirport,
    Connection.airport_to_id           as DestinationAirport,
    Connection.departure_time           as DepartureTime,
    Connection.arrival_time             as ArrivalTime,
    @Semantics.quantity.unitOfMeasure: 'DistanceUnit'
    Connection.distance                 as Distance,
    Connection.distance_unit            as DistanceUnit
}

```

Carrier View /DMO/I_Carrier

The following data definition for the carrier CDS entity provides you with IDs and names of airlines that are stored in the database table `/DMO/CARRIER`.

This CDS entity mainly serves as a text provider view. It provides text data through text associations as defined in the travel and the booking views (Listing 1 and Listing 2).

For this purpose, a text annotation is required at element level in order to annotate the text elements from the view's element list:
In this example, the Name element is identified as the text element.

i *Expand the following code sample to view the source code of CDS view /DM0/I_Carrier.*

 Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Carrier view'

define view entity /DM0/I_Carrier as select from /dmo/carrier as Airline
{
  key Airline.carrier_id          as AirlineID,

  @Semantics.text: true
  Airline.name                    as Name,

  Airline.currency_code          as CurrencyCode
}
```

Projecting the Data Model in CDS Projection Views

You use CDS projection views to expose the general data model for a specific business service.

Whereas the general business object defines all available entities and elements, the [Business Object Projection](#) defines those entities and elements that are used for a specific service, in our case a service that is exposed for UI consumption. That means, in the projection layer, we define the data model and its functionality that is needed for the use case of a UI service of the travel scenario. UI services include:

- value helps
- text elements
- search
- UI layout annotations

These UI enrichments are modeled via CDS annotations in the CDS projection view. It is best practice, to outsource the annotations related to UI layout in metadata extensions, as these annotations can easily overload the projection view. A CDS metadata extension is always assigned to a layer such as industry, partner or customer to easily extend the scope of the metadata extension.

Creating CDS Projection Views

For the Travel UI service, create projection views for those CDS entities that are part of the compositional hierarchy of the business object. These are:

	root layer	1st child layer
CDS views for BO structure	/DM0/I_TRAVEL_U	/DM0/I_BOOKING_U
CDS views for BO projection	/DM0/C_TRAVEL_U	/DM0/C_BOOKING_U

i **Note**

The names are assigned according to the naming conventions for projection views: [Naming Conventions for Development Objects](#).

For information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Data Modeling for the Business Object Projection

The unmanaged scenario provides just one projection use case that reuses all elements that are defined in the business object views. In the projection layer the data model is enriched with UI functionality that is implemented via CDS annotations or keywords in the projection view and manifested in the service metadata.

Root Projection View `/DMO/C_Travel_U`

The following codeblock provides you with the implementation of the CDS view projection, where the CDS view `/DMO/I_Travel_U` serves as the projection source for the corresponding CDS projection view `/DMO/C_Travel_U`. It projects every element from the underlying view.

The Travel projection view also exposes all associations that are defined in the underlying travel view `/DMO/I_Travel_U`. However, the composition to the child `/DMO/I_Booking_U` must be redirected to the newly created projection counterpart. The syntax for defining redirected compositions is described in [CDS Projection View](#).

To define a relationship between the elements `AgencyID`, `CustomerID`, `Status` and their corresponding texts or descriptions, the text elements must be denormalized in the CDS projection view. Therefore the elements of the associated text provider views (`_Agency.Name`, `_Customer.LastName` and `_TravelStatus._Text.Text`) are included in the `select` list of the projection view. Their corresponding ID elements are annotated with `@ObjectModel.text.element`. At runtime, the referenced element is read from the database and filtered by the logon language of the OData consumer automatically. For detailed information, see: [Working with Text Elements](#)

Value helps are defined in the source code of the CDS projection view `/DMO/C_Travel_U` by adding the annotation `@Consumption.valueHelpDefinition` to the elements `AgencyID`, `CustomerID`, `CurrencyCode` and `Status`. The value help annotation allows you to reference the value help provider view without implementing an association. You simply assign a CDS entity as the value help provider and specify an element for the mapping in the annotation. All fields of the value help provider are displayed on the UI. When the end user chooses one of the entries of the value help provider, the value of the referenced element is transferred to the corresponding input field on the UI. For detailed information, see: [Simple Value Help](#)

i Note

For the default implementation of value help, note that you can reuse any CDS entity that contains the required values of the element that corresponds to the input field on the UI. You do not need to explicitly define a CDS entity as the value help provider

In the projection view, you model the ability to search for specific values in the view. The entity annotation `@Search.searchable` is used in the travel projection to enable the general HANA search. This annotation also triggers the search bar in the Fiori elements UI. By using this annotation, you have to define elements that are primarily used as the search target for a free text search. These elements are annotated with `@Search.defaultSearchElement`. In addition, you can define a fuzziness threshold that defines the how exact the search values must be to be able to find element values. For more information on search, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

The listing below does not include annotations that define the UI layout. They are outsourced to metadata extensions, see [Adding UI Metadata to the Data Model](#).

i *Expand the following code sample to view the source code of CDS Root Projection View `/DMO/C_Travel_U`.*

☰ Sample Code

```

@EndUserText.label: 'Travel Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true

@Search.searchable: true
define root view entity /DM0/C_Travel_U
  provider contract transactional_query
  as projection on /DM0/I_Travel_U

{
  ///DM0/I_Travel_U

  key TravelID,

  @Consumption.valueHelpDefinition: [{ entity : {name: '/DM0/I_Agency_StdVH', element: 'Agen
  @ObjectModel.text.element: ['AgencyName']
  @Search.defaultSearchElement: true
  AgencyID,
  _Agency.Name          as AgencyName,

  @Consumption.valueHelpDefinition: [{entity: {name: '/DM0/I_Customer_StdVH', element: 'Cust
  @ObjectModel.text.element: ['CustomerName']
  @Search.defaultSearchElement: true
  CustomerID,
  _Customer.LastName as CustomerName,

  BeginDate,

  EndDate,

  BookingFee,

  TotalPrice,

  @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
  CurrencyCode,

  Memo,

  @Consumption.valueHelpDefinition: [{ entity: { name: '/DM0/I_Travel_Status_VH', element: ''
  @ObjectModel.text.element: ['StatusText']
  Status,

  _TravelStatus._Text.Text as StatusText : localized,

  LastChangedAt,
  /* Associations */
  ///DM0/I_Travel_U
  _Booking : redirected to composition child /DM0/C_Booking_U,
  _Agency,
  _Currency,
  _Customer,

```

}

Child Projection View /DMO/C_Booking_U

The following codeblock provides you with the implementation of the CDS view projection, where the CDS view /DMO/I_Booking_U serves as the projection source for the corresponding CDS projection view /DMO/C_Booking_U. It projects every element from the underlying view.

The Booking projection view also exposes all associations that are defined in the underlying booking view /DMO/I_Travel_U. However, the composition to the parent /DMO/I_Travel_U must be redirected to the newly created projection counterparts. The syntax for defining redirected compositions is described in [CDS Projection View](#).

To access the corresponding texts or descriptions, the relationship between the elements `AirlineID` and `ConnectionID` and the text elements in the associated text provider views /DMO/I_Carrier and /DMO/I_Connection are used in this example. Their corresponding ID elements are annotated with `@ObjectModel.text.element`. At runtime, the referenced element is read from the database and filtered by the logon language of the OData consumer automatically. For detailed information, see: [Working with Text Elements](#)

Value helps are defined in the source code of the booking entity /DMO/C_Booking_U by adding the annotation `@Consumption.valueHelpDefinition` to the relevant elements. You simply assign a CDS entity as the value help provider to the elements `CustomerID`, `AirlineID`, and `CurrencyCode`, and specify an element for the mapping in the annotation. This simple value help approach is convenient if you only want to display values from the value help provider view for an input field. In this case, the annotation defines the binding to the value help providing entity. You only have to specify the entity name and the element providing the possible values for the annotated element. For detailed information, see: [Simple Value Help](#)

Listing 2 also demonstrates how you can implement the value help with additional binding, which defines a filter condition. Different filter conditions for the value help on the same value help provider entity /DMO/I_Flight are defined for filtering the value help result list for the elements `ConnectionID` and `FlightDate`. For detailed information, see: [Value Help with Additional Binding](#)

The elements in the booking projection view are also searchable in a UI service. The annotations related to search are therefore also maintained in the booking view on entity level and on certain elements to mark them as primary search element. For more information on search, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

The listing below does not include annotations that define the UI layout. They are outsourced to metadata extensions, see [Adding UI Metadata to the Data Model](#).

i Expand the following code sample to view the source code of CDS Root Projection View /DMO/C_Booking_U.

Sample Code

```
@EndUserText.label: 'Booking Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true

@Search.searchable: true
define view entity /DMO/C_Booking_U
as projection on /DMO/I_Booking_U
```

```

{      ///DMO/I_Booking_U
      @Search.defaultSearchElement: true
      key TravelID,

      @Search.defaultSearchElement: true
      key BookingID,

      BookingDate,

      @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Customer_StdVH', element: 'Cust
      @Search.defaultSearchElement: true
      @ObjectModel.text.element: ['CustomerName']
      CustomerID,
      _Customer.LastName      as CustomerName,

      @ObjectModel.text.element: ['CarrierName']
      @Consumption.valueHelpDefinition: [
        { entity: {name: '/DMO/I_Flight_StdVH', element: 'AirlineID'},
          additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                                { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                                { localElement: 'FlightPrice', element: 'Price', usage:
                                { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                                useForValidation: true }
        ]
      AirlineID,
      _Carrier.Name      as CarrierName,

      @Consumption.valueHelpDefinition: [
        { entity: {name: '/DMO/I_Flight_StdVH', element: 'ConnectionID'},
          additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                                { localElement: 'AirlineID', element: 'AirlineID', usage:
                                { localElement: 'FlightPrice', element: 'Price', usage:
                                { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                                useForValidation: true }
        ]
      ConnectionID,

      @Consumption.valueHelpDefinition: [
        { entity: {name: '/DMO/I_Flight_StdVH', element: 'FlightDate'},
          additionalBinding: [ { localElement: 'AirlineID', element: 'AirlineID', usage:
                                { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                                { localElement: 'FlightPrice', element: 'Price', usage:
                                { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                                useForValidation: true }
        ]
      FlightDate,

      @Consumption.valueHelpDefinition: [
        { entity: {name: '/DMO/I_Flight_StdVH', element: 'Price'},
          additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                                { localElement: 'AirlineID', element: 'AirlineID', usage:
                                { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                                { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                                useForValidation: true }
        ]
      FlightPrice,

      @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
      CurrencyCode,

      //      LastChangedAt,
      /* Associations */
      ///DMO/I_Booking_U
      _Travel: redirected to parent /DMO/C_Travel_U,
      _BookSupplement: redirected to composition child /DMO/C_BookingSupplement_U,
      _Carrier,
      _Connection,
      _Customer
    }

```

The other CDS views that are relevant for text provisioning and value helps to complete the data model structure do not have to be projected as they are not part of the business object.

Related Information

[Business Object Projection](#)

Adding UI Metadata to the Data Model

Use @UI annotations to define the layout of the Fiori UI.

To enable the business service to be consumable by any UI client, UI relevant metadata are added to the backend service. These metadata are maintained with @UI annotations that are either added to the whole entity or, if it relates to a specific UI element, to the corresponding CDS element. Since these annotations can become excessive, it is recommended to maintain the UI annotations in metadata extensions.

Metadata extensions are used to define CDS annotations for a CDS view outside of the corresponding data definition. The use of metadata extensions allows the separation of concerns by separating the data model from domain-specific semantics, such as UI-related information for UI consumption. A CDS metadata extension is always assigned to a specific layer such as core, industry, partner or customer. These industries can extend the metadata for the data model.

Creating Metadata Extension for the Projection Views

For the Travel UI service, create metadata extensions for the CDS projection views that are part of the compositional hierarchy of the business object. These are:

	root layer	1st child layer
CDS views for BO structure	/DMO/I_TRAVEL_U	/DMO/I_BOOKING_U
CDS views for BO projection	/DMO/C_TRAVEL_U	/DMO/C_BOOKING_U
Metadata extension for UI annotations	/DMO/C_TRAVEL_U	/DMO/C_BOOKING_U

i Note

The names of metadata extensions are the same as their related CDS entities, according to the naming conventions for metadata extensions: [Naming Conventions for Development Objects](#).

To create a metadata extension object, open the context menu in the project explorer of the related CDS entity and choose **New Metadata Extension**. Follow the steps in the wizard to get a metadata extension template that annotates the related projection view.

Syntax:

```
@Metadata.layer: layer
annotate view CDSProjectionView
  with
{
  element_name;
}
```

Define a metadata layer for your metadata extension and insert the elements that you want to annotate. The layering of metadata extension allows a stacking for each development layer, beginning with #CORE. Each layer can then add its own layer specific metadata to the service.

To enable the usage of metadata extensions for the projection views, add the annotation `@Metadata.allowExtensions:true` on entity level in your projection view.

Annotating CDS Elements in Metadata Extensions

Any CDS element that you want to annotate in metadata extensions must be inserted in the element list.

Metadata Extension for CDS Projection View `/DMO/C_Travel_U`

The listing below (listing 1) provides you with the implementation of the metadata extension `/DMO/C_Travel_U` for the projection view with the same name. It annotates the projection view itself and its elements.

To specify the header texts for Fiori UIs, the annotation `@UI.headerInfo` is used at entity level. It specifies the list title as well as the title for the object page. The main building blocks of the UI are specified as UI facets. The annotation `@UI.facet` also enables the navigation to the child entity, which is represented as list in the page body of the object page.

The annotations on element level are used to define the position of the element in the list report and on the object page. In addition, they specify the importance of the element for the list report. If elements are marked with low importance, they are not shown on a device with a narrow screen. The selection fields are also defined on the elements that require a filter bar in the UI.

For actions, you can define an action button on the Fiori Elements UI. This scenario contains the action to set the travel status to booked. To expose the action on the list report page, the information for the action button is added to the `@UI.lineItem` annotation on an element.

Syntax to expose an action button on the UI:

```
@UI: { lineItem: { type: #FOR_ACTION, dataAction: 'action_name', label: 'Button Label' } }
element;
```

i Expand the following code sample to view the source code of metadata extension `/DMO/_C_Travel_U`.

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Travel',
                    typeNamePlural: 'Travels',
                    title: { type: #STANDARD,
                           value: 'TravelID' } } }

annotate view /DMO/C_Travel_U with

{
  @UI.facet: [ { id:          'Travel',
                 purpose:    #STANDARD,
                 type:       #IDENTIFICATION_REFERENCE,
                 label:      'Travel',
                 position:   10 },
               { id:          'Booking',
                 purpose:    #STANDARD,
                 type:       #LINEITEM_REFERENCE,
                 label:      'Booking',
                 position:   20,
                 targetElement: '_Booking'}]
```



```

@UI: { lineItem:      [ { position: 10,
                        importance: #HIGH } ],
      identification: [ { position: 10 } ],
      selectionField: [ { position: 10 } ] }
TravelID;

@UI: { lineItem:      [ { position: 20,
                        importance: #HIGH } ],
      identification: [ { position: 20 } ],
      selectionField: [ { position: 20 } ] }
AgencyID;

@UI: { lineItem:      [ { position: 30,
                        importance: #HIGH } ],
      identification: [ { position: 30 } ],
      selectionField: [ { position: 30 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 40,
                        importance: #MEDIUM } ],
      identification: [ { position: 40 } ] }
BeginDate;

@UI: { lineItem:      [ { position: 41,
                        importance: #MEDIUM } ],
      identification: [ { position: 41 } ] }
EndDate;

@UI: { identification: [ { position: 42 } ] }
BookingFee;

@UI: { identification: [ { position: 43 } ] }
TotalPrice;

@UI: { identification:[ { position: 45,
                        label: 'Comment' } ] }
Memo;

@UI: { lineItem:      [ { position: 50,
                        importance: #HIGH },
                        { type: #FOR_ACTION,
                          dataAction: 'set_status_booked',
                          label: 'Set to Booked' } ],
      identification: [ { position: 46, label: 'Travel Status' } ],
      selectionField: [{ position: 40 }],
      textArrangement: #TEXT_ONLY }
Status;

@UI.hidden: true
StatusText;
}

```

Metadata Extension for CDS Projection View /DMO/C_Booking_U

The listing below (listing 2) provides you with the implementation of the metadata extension /DMO/C_Booking_U for the projection view with the same name. It annotates the projection view itself and its elements.

To specify the header texts for Fiori UIs, the annotation @UI . headerInfo is used at entity level. It specifies the list title as well as the title for the object page. The main building blocks of the UI are specified as UI facets.

The annotations on element level are used to define the position of the element in the list report and on the object page. In addition, they specify the importance of the element for the list report. The selection fields are also defined on the elements that require a filter bar in the UI.

i *Expand the following code sample to view the source code of metadata extension /DMO/C_Booking_U.*

≡, Sample Code

```

@Metadata.layer: #CORE

@UI: {
  headerInfo: { typeName: 'Booking',
                typeNamePlural: 'Bookings',
                title: { type: #STANDARD,
                       value: 'BookingID' } } }

annotate view /DMO/C_Booking_U with
{
  @UI.facet: [ { id:          'Booking',
                 purpose:     #STANDARD,
                 type:        #IDENTIFICATION_REFERENCE,
                 label:       'Booking',
                 position:    10 } ]

  @UI: { lineItem:      [ { position: 20,
                          importance: #HIGH } ],
        identification: [ { position: 20 } ] }
  BookingID;

  @UI: { lineItem:      [ { position: 30,
                          importance: #HIGH } ],
        identification: [ { position: 30 } ] }
  BookingDate;

  @UI: { lineItem:      [ { position: 40,
                          importance: #HIGH } ],
        identification: [ { position: 40 } ] }
  CustomerID;

  @UI: { lineItem:      [ { position: 50,
                          importance: #HIGH } ],
        identification: [ { position: 50 } ] }

  AirlineID;

  @UI: { lineItem:      [ { position: 60,
                          importance: #HIGH } ],
        identification: [ { position: 60 } ] }
  ConnectionID;

  @UI: { lineItem:      [ { position: 70,
                          importance: #HIGH } ],
        identification: [ { position: 70 } ] }

  FlightDate;
  @UI: { lineItem:      [ { position: 80,
                          importance: #HIGH } ],
        identification: [ { position: 80 } ] }

  FlightPrice;
}

```

Related Information

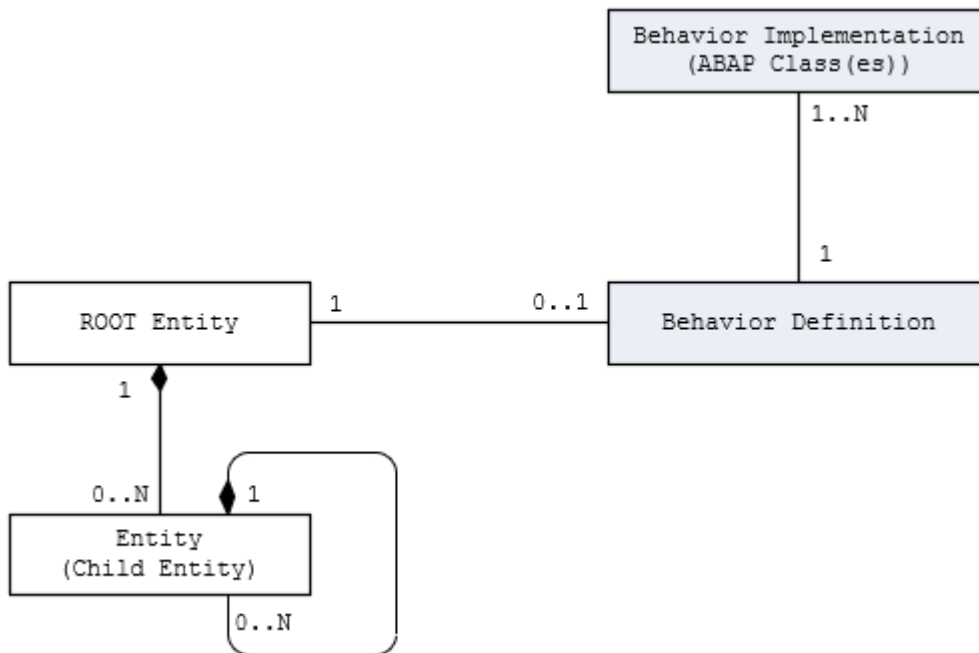
[CDS Metadata Extensions](#)

Defining and Implementing Behavior of the Business Object

Behavior of a Business Object

To specify the business object's behavior, the behavior definition as the corresponding development object is used. A business object behavior definition (behavior definition for short) is an ABAP Repository object that describes the behavior of a business object in the context of the ABAP RESTful Application Programming Model. A behavior definition is defined using the Behavior Definition Language (BDL).

A behavior definition always refers to a CDS data model. As shown in the figure below, a behavior definition relies directly on the CDS root entity. One behavior definition refers exactly to one root entity and one CDS root entity has at most one behavior definition (a 0..1 cardinality), which also handles all included child entities that are included in the composition tree. The implementation of a behavior definition can be done in a single ABAP class (behavior pool) or can be split between an arbitrary set of ABAP classes (behavior pools). The application developer can assign any number of behavior pools to a behavior definition (1..N cardinality).



Relationship Between the CDS Entities and the Business Object Behavior

Overview of Steps

1. Create the Behavior Definition Object
2. Model the Behavior for Managing Travels
3. Create the Behavior Pool
4. Implement the Transactional Behavior of the Travel Business Object

Related Information

[Business Object](#)
[Business Service](#)

Adding Behavior to the Business Object

Creating a Behavior Definition /DMO/I_TRAVEL_U

Create the behavior definition by using the creation wizard in ADT. When using the context menu on the root view entity (/DMO/I_Travel_M), you can directly create a new behavior definition for the complete BO hierarchy. In the wizard, choose the

implementation type **Unmanaged**.

→ Remember

By creating a behavior definition, the referenced root entity and its compositions (child entities) gain a transactional character. The behavior definition is hence the implementation of the BO concept within the context of the current programming model. All supported transactional operations of a concrete business object must be specified in the same behavior definition.

Defining the Transactional Behavior of the TRAVEL Business Object

It consists of a header information and two definitions for entity behavior: one for the root entity and one for the child entity – corresponding to the composition tree of the business object. Note that for each entity of the composition tree, the transactional behavior can be defined in the behavior definition at most once. All supported transactional operations of a concrete business object's node must be specified in the same behavior definition (that is introduced by the keyword **DEFINE BEHAVIOR FOR** ...).

For a detailed syntax description of a behavior definition header, see [CDS BDL - CDS behavior definition header \(ABAP Keyword Documentation\)](#).

For a detailed syntax description of an entity behavior definition, see [CDS BDL - entity behavior definition \(ABAP Keyword Documentation\)](#).

As expected, the header specifies the **unmanaged** implementation type of our business object's contract provider since we're going to integrate the legacy business logic in the new app. For this implementation type, you as application developer must implement the essential components of the business object's itself. In this case, you must specify all required operations (**create**, **update**, **delete**, or any application-specific actions) in the corresponding behavior definition and implement them manually in ABAP.

Our TRAVEL business object refers to the underlying CDS data model, which is represented by root entity `/DMO/I_Travel_U`. Behavior for the root entity can only be implemented in the specified behavior pool `/DMO/BP_TRAVEL_U`.

Static field control is defined in the behavior definition. In our scenario, the value for the key field `TravelID` is drawn by the function module for creating travel instances. Thus, the field must not be editable by the end user on the UI. Likewise, the field `TotalPrice` is set to read only. The total price is not calculated by the price of the associated bookings and the booking fee in the function module. In this scenario, the total price is editable by the end user.

Mandatory fields are `AgencyID`, `CustomerID`, `BeginDate` and `EndDate`. These fields contain mandatory information for a travel instance. For more information about field control, see [Instance Feature Control](#).

The transactional handling of the business object's root node is determined by the standard operations **create**, **update**, and **delete**, and an instance-related action `set_status_booked`. Using this action, the end user is able to set the status of selected travel instances to booked. The action in our example affects the output instances with the same entity type and one input instance is related to exactly one output instance. Therefore, the output parameter is defined with the predefined type `$self` and the cardinality `[1]`. The fact that in our scenario new instances of the booking sub node can only be created for a specific travel instance is considered by the addition of the `_Booking` association. The keyword `{create;}` declares that this association is create-enabled what exactly means that instances of the associated bookings can be created by a travel instance.

Dynamic operation control is defined for the **create by association** operation. That means, new bookings can only be created if the corresponding travel instance isn't set to booked or canceled. For more information about field control, see [Feature Control](#).

The names of the database table fields and the names of the CDS data model names differ. That is why, we specify the mapping contract for every field in the CDS data model. To map the control structure accordingly, the control structure `/dmo/s_booking_intx` is defined for the database table. For more information, see [Using Type and Control Mapping](#).

The `etag master` field ensures transactional access to data by multiple users while avoiding inconsistencies and unintentional changes of already modified data. Each `MODIFY` is logged with the `etag` field (in this case a timestamp) to ensure data consistency. It is recommended that you specify the respective `ETag` field as read-only in the behavior definition, using the field characteristic `readOnly`. For more information, see [Optimistic Concurrency Control](#).

The `lock master` prevents parallel modification to data on the database by more than one user and ensures data consistency. For more information, see [Pessimistic Concurrency Control \(Locking\)](#).

The `authorization master` defines how and when a RAP BO can be accessed. The strict mode requires that authorization is defined for the business object. With the addition `none`, no authorization checks are executed. This is a demo application, which is why the addition `none` is used. This approach is not recommended in real-world scenarios and violates best practices. For more information, see [Authorization Control](#).

`late numbering` defines that the primary ID value is only assigned just before the instance is saved on the database. For late numbering, the method `ADJUST NUMBERS` must be implemented. For more information, see [Unmanaged Internal Late Numbering](#).

The sub node of `TRAVEL` business object refers to the corresponding data model for bookings that is represented by the child entity for `/DMO/I_Booking_U`. Behavior for the child entity can only be implemented in the specified behavior pool `/DMO/BP_BOOKING_U`. The transactional handling of the booking sub node of `TRAVEL` business object is determined by the standard operations `update` and `delete`. The creation of booking instances is handled by the `create by association`, that means bookings can only be created as a subnode of its travel parent.

i *Expand the following code sample to view the source code of behavior definition `/DMO/I_Travel_U`.*

Sample Code

```
unmanaged implementation in class /DMO/BP_TRAVEL_U unique;
strict(2);

// behavior definition for the TRAVEL root entity
define behavior for /DMO/I_Travel_U alias Travel
implementation in class /DMO/BP_TRAVEL_U unique
etag master LastChangedAt
lock master
authorization master ( none )
late numbering

{
  field ( readOnly ) TravelID, LastChangedAt;
  field ( mandatory ) AgencyID, CustomerID, BeginDate, EndDate;

  create;
  update;
  delete;

  action ( features : instance ) set_status_booked result [1] $self;

  association _Booking { create (features: instance); }

  mapping for /dmo/travel control /dmo/s_travel_intx
  {
    AgencyID      = agency_id;
    BeginDate     = begin_date;
    BookingFee    = booking_fee;
    CurrencyCode  = currency_code;
    CustomerID    = customer_id;
    EndDate       = end_date;
    LastChangedAt = lastchangedat;
    Memo          = description;
    Status        = status;
    TotalPrice    = total_price;
    TravelID      = travel_id;
```

```

    }
}

// behavior definition for the BOOKING child entity
define behavior for /DMO/I_Booking_U alias Booking
implementation in class /DMO/BP_BOOKING_U unique
etag dependent by _Travel
lock dependent by _Travel
authorization dependent by _Travel
late numbering

{
  field ( readonly ) TravelID, BookingID;
  field ( mandatory ) BookingDate, CustomerID, AirlineID, ConnectionID, FlightDate;

  update;
  delete;

  association _BookSupplement { create; }
  association _Travel;

  mapping for /dmo/booking control /dmo/s_booking_intx
  {
    AirlineID      = carrier_id;
    BookingDate    = booking_date;
    BookingID      = booking_id;
    ConnectionID   = connection_id;
    CurrencyCode   = currency_code;
    CustomerID     = customer_id;
    FlightDate     = flight_date;
    FlightPrice    = flight_price;
    TravelID       = travel_id;
  }
}

```

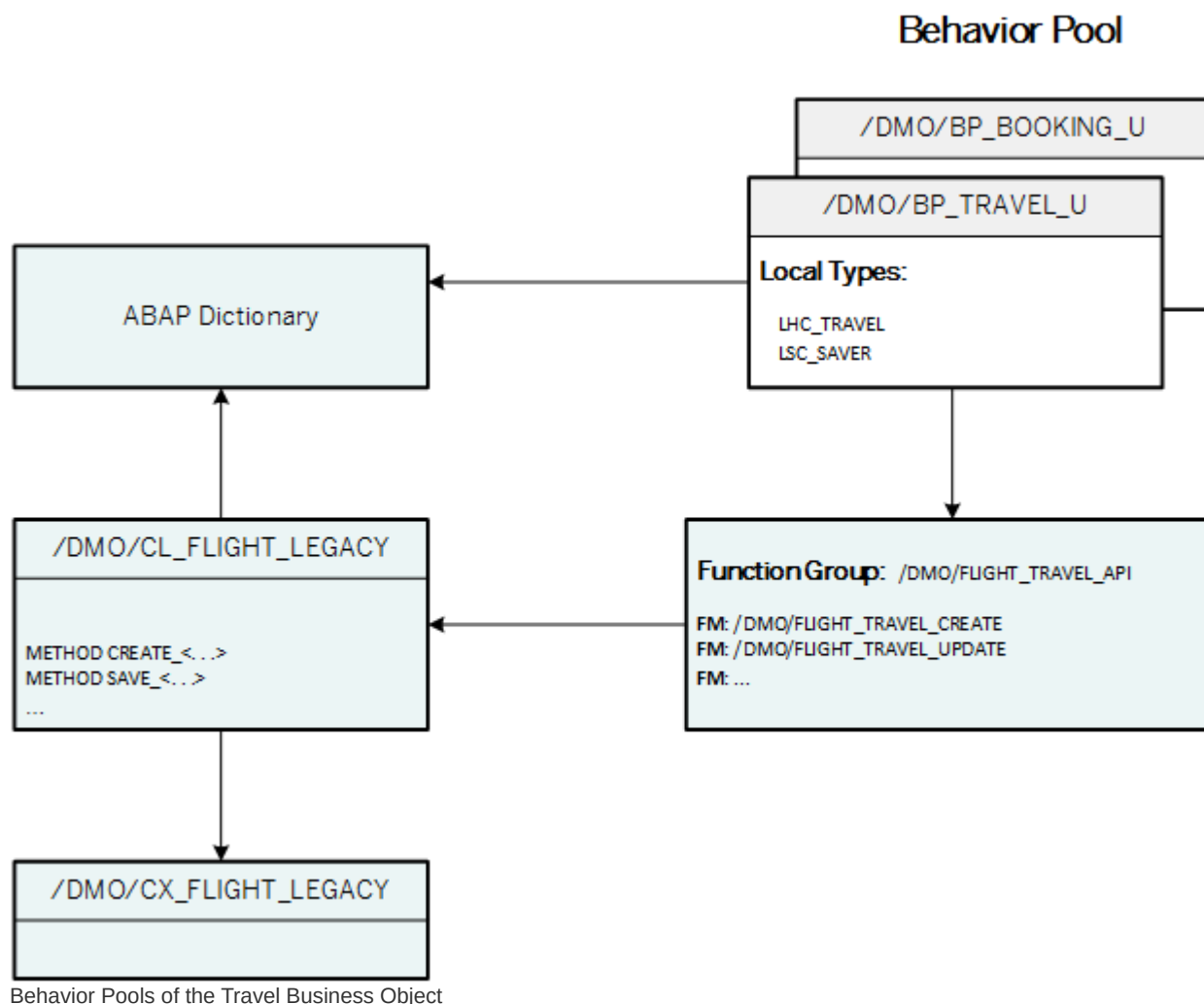
Related Information

[Business Object](#)

Implementing the Behavior of the Business Object

Behavior Pool

The transactional behavior of a business object in the context of the current programming model is implemented in one or more global ABAP classes. These special classes are dedicated only to implementing the business object's behavior and are called **behavior pools**. You can assign any number of behavior pools to a behavior definition (a 1: n relationship). Within a single global class, you can define multiple local classes that handle the business object's behavior. The global class is just a container and is basically empty while the actual behavior logic is implemented in local classes.



For details about the syntax for defining a behavior pool, see [CLASS, FOR BEHAVIOR OF \(ABAP Keyword Documentation\)](#).

Effect

The above syntax defines a special ABAP class (a behavior pool) for the behavior specified in the behavior definition `MyRootBehavior` (which in turn has the same name as the CDS root entity). This special property and relationship are persisted and transported using a corresponding system table. This specific information is assigned to the properties of the behavior pool and can no longer be changed. The behavior pool is dependent on the behavior definition, meaning the changes to the behavior definition cause it to be regenerated.

This global class is defined as abstract and final, which means there is no reason to instantiate or inherit this global behavior pool. In addition, such a ban prevents possible misuse.

A behavior pool can have static methods, namely `CLASS-DATA`, `CONSTANTS`, and `TYPES`. The application may place common or even public aspects of its implementation in these methods.

Distributing Behavior Pool Implementation

You can assign any number of behavior pools to a behavior definition (1: n relationship). This allows the application developers to distribute their implementations between multiple units, for example one global class (behavior pool) for each business object's node, and one or more separate auxiliary classes for implementing helper methods. The figure below illustrates this distribution pattern for our sample application.

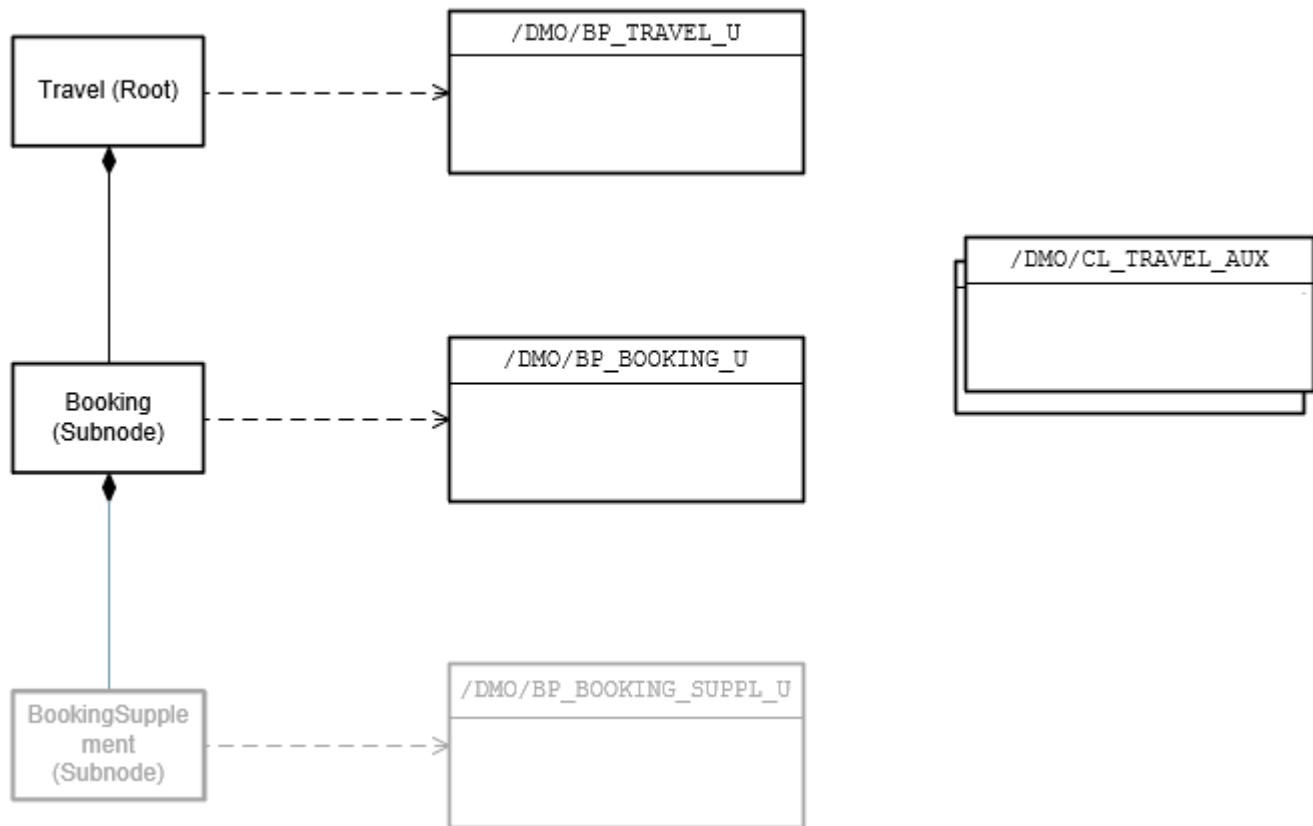
i Note

Best Practices: Splitting the implementation into different global classes allows developers to work in parallel (distributed work mode). If operations on each node have to be forwarded to different APIs (function module calls), then we recommend using a separate global class (behavior pool) for each node of the business object's compositional tree.

BO Structure

Behavior Pools

Auxiliary Classes



Splitting the implementation into different behavior pools

Related Information

[Business Object Provider API](#)

[Handler Classes](#)

[Saver Classes](#)

[Naming Conventions for Development Objects](#)

Creating the Behavior Pool for the Root Entity

In this step, you create a behavior pool that is the implementation artifact of the corresponding behavior definition that you created earlier.

In doing so, we apply the contribution pattern and split the behavior implementation into two different behavior pools, one for the travel root entity and the other for the booking child entity. In addition, we create a separate auxiliary class for implementing helper methods (such as for mapping and message handling) that can be reused in both behavior implementation classes.

i Note

Best Practices for Modularization and Performance:

The granularity of the existing application code influences the granularity of handler implementation. If the existing legacy application logic has different APIs (function modules) for create, update, delete, and other transactional operations, then we

recommend spreading the operations across different handler classes.

If, for example, the called API only implements one operation for **CREATE** and another API implements one for **UPDATE**, it is advisable to implement each operation in a different local handler class.

If, on the other hand, the called API of your application code is able to process **multiple changes in one call**, the handler should reflect this to achieve the best performance. In such a case, we combine all operations supported by this API in one common handler class.

Otherwise the application code is called multiple times with different input, which can result in bad performance.

If the application code supports a **deep create** (for example creation for root and child entity in one step), then this should be reflected in the design of handler classes to achieve best performance.

→ Remember

Beyond the performance aspects, it is beneficial to implement operations in different **FOR MODIFY** methods, since the orchestration is then passed to the BO runtime and the code of the behavior implementation is more **readable**.

i Note

Convention: The saver class that implements the save sequence for data persistence is either a separate global class or a part of the root implementation (behavior pool for the root entity).

Procedure 1: Create a Behavior Pool /DMO/BP_TRAVEL_U

To launch the wizard tool for creating a behavior implementation, do the following:

1. Add the following syntax to the behavior definition in the section of the entity behavior for the travel entity.

```
implementation in class /dmo/bp_travel_u unique
```

2. The behavior definition offers a quick fix to create a behavior pool corresponding to the behavior described in the behavior definition. Follow the quick fix to start the wizard for creating an ABAP class.

New Behavior Class

ABAP Class
Create an ABAP class

Project: * Browse...

Package: * Browse...

☐ Add to favorite packages

Name: *

Description: *

Original Language:

Behavior Definition: * Browse...

< Back **Next >** Finish Cancel

Creating a Behavior Pool – Wizard

Results: Global Behavior Pool

The generated class pool (in our case Procedure 2: Define a Skeleton of Local Classes Corresponding to the Behavior Model /DMO/BP_TRAVEL_U) provides you with an extension FOR BEHAVIOR OF.

```

1 CLASS /dmo/bp_travel_u DEFINITION
2   PUBLIC
3   ABSTRACT
4   FINAL
5   FOR BEHAVIOR OF /dmo/i_travel_u .
6
7   PUBLIC SECTION.
8   PROTECTED SECTION.
9   PRIVATE SECTION.
10  ENDClass.
11
12
13
14 CLASS /dmo/bp_travel_u IMPLEMENTATION.
15 ENDClass.

```

Global Class Class-relevant Local T... **Local Types** Test Classes (non exis... Macros

New Global Behavior Pool

The real substance of a behavior pool is located in **Local Types**. Here you can define two types of special local classes, namely handler classes for the operations within the **Procedure 2: Define a Skeleton of Local Classes Corresponding to the Behavior**

interaction phase and saver classes for the operations within the **save sequence**. These classes can be instantiated or invoked only by the ABAP runtime environment (virtual machine).

i Note

All local class source code within a single global class is stored within a single include, the CCIMP include.

Based on the declarations in the behavior definition /DMO/I_TRAVEL_U, the behavior implementation offers a skeleton of the local classes for the root entity:

i Expand the following code sample to view the source code of the template for local classes of /DMO/BP_Travel_U.

Sample Code

```
*****
*
* Handler class for managing travels
*
*****
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.
METHODS get_instance_features FOR INSTANCE FEATURES
IMPORTING keys REQUEST requested_features FOR travel RESULT result.

METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
IMPORTING REQUEST requested_authorizations FOR travel RESULT result.

METHODS create FOR MODIFY
IMPORTING entities FOR CREATE travel.

METHODS update FOR MODIFY
IMPORTING entities FOR UPDATE travel.

METHODS delete FOR MODIFY
IMPORTING keys FOR DELETE travel.

METHODS read FOR READ
IMPORTING keys FOR READ travel RESULT result.

METHODS lock FOR LOCK
IMPORTING keys FOR LOCK travel.

METHODS rba_Booking FOR READ
IMPORTING keys_rba FOR READ travel\_Booking FULL result_requested RESULT result LINK assoc

METHODS cba_Booking FOR MODIFY
IMPORTING entities_cba FOR CREATE travel\_Booking.

METHODS set_status_booked FOR MODIFY
IMPORTING keys FOR ACTION travel~set_status_booked RESULT result.

METHODS map_messages
IMPORTING
  cid TYPE string OPTIONAL
  travel_id TYPE /dmo/travel_id OPTIONAL
  messages TYPE /dmo/t_message
EXPORTING
  failed_added TYPE abap_bool
CHANGING
  failed TYPE tt_travel_failed
  reported TYPE tt_travel_reported.

METHODS map_messages_assoc_to_booking
IMPORTING
  cid TYPE string
  is_dependend TYPE abap_bool DEFAULT abap_false
  messages TYPE /dmo/t_message
```

```

EXPORTING
  failed_added TYPE abap_bool
CHANGING
  failed      TYPE tt_booking_failed
  reported    TYPE tt_booking_reported.
ENDCLASS.
CLASS lhc_travel IMPLEMENTATION.

```

```

METHOD get_instance_features.
ENDMETHOD.

```

```

METHOD get_global_authorizations.
ENDMETHOD.

```

```

METHOD create.
ENDMETHOD.

```

```

METHOD update.
ENDMETHOD.

```

```

METHOD delete.
ENDMETHOD.

```

```

METHOD read.
ENDMETHOD.

```

```

METHOD lock.
ENDMETHOD.

```

```

METHOD rba_Booking.
ENDMETHOD.

```

```

METHOD cba_Booking.
ENDMETHOD.

```

```

METHOD set_status_booked.
ENDMETHOD.

```

```

METHOD map_messages.
ENDMETHOD.

```

```

METHOD map_messages_assoc_to_booking
.
ENDMETHOD.
ENDCLASS.

```

```

*****
*

```

```

* Saver class implements the save sequence for data persistence
*

```

```

*****
CLASS lsc_saver DEFINITION INHERITING FROM cl_abap_behavior_saver.
PROTECTED SECTION.

```

```

  METHODS finalize          REDEFINITION.
  METHODS check_before_save REDEFINITION.
  METHODS save              REDEFINITION.
  METHODS cleanup           REDEFINITION.

```

```

ENDCLASS.

```

```

CLASS lsc_saver IMPLEMENTATION.
METHOD finalize REDEFINITION.
ENDMETHOD.

```

```

METHOD check_before_save REDEFINITION.
ENDMETHOD.

```

```

METHOD adjust_numbers REDEFINITION.
ENDMETHOD.

```

```

METHOD save REDEFINITION.
ENDMETHOD.

```

```
METHOD cleanup REDEFINITION.  
ENDMETHOD.
```

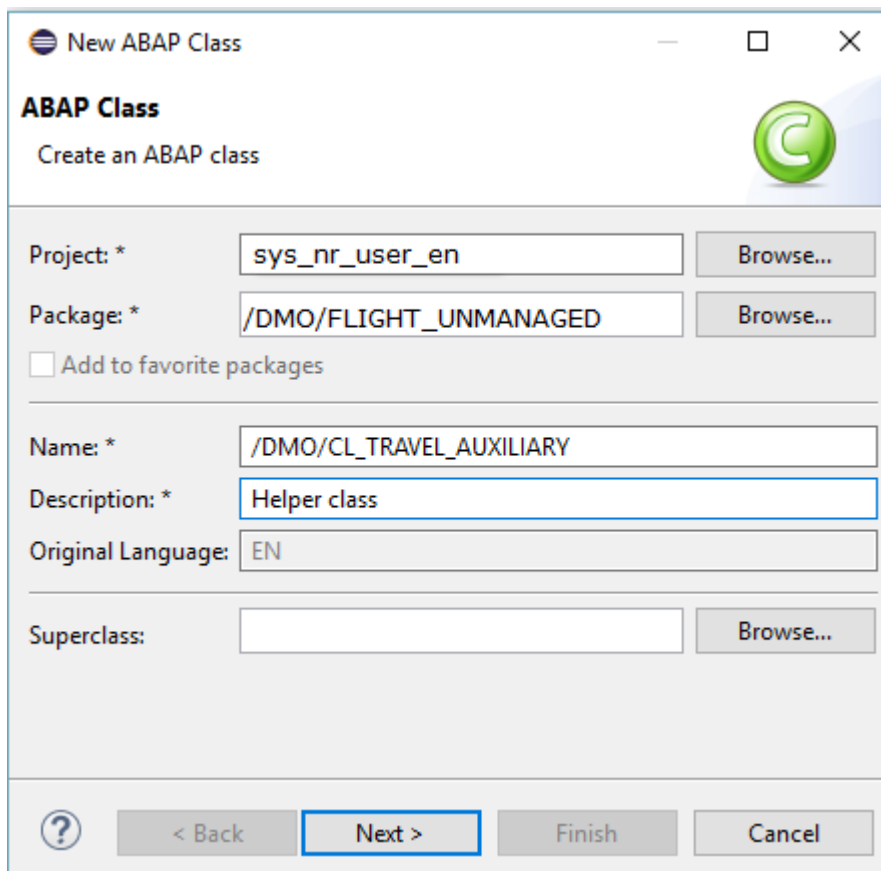
```
METHOD cleanup_finalize REDEFINITION.  
ENDMETHOD.
```

```
ENDCLASS.
```

Procedure 3: Create an Auxiliary Class /DMO/CL_TRAVEL_AUXILIARY

Message handling can be reused by different behavior pools and is therefore outsourced in a separate helper class.

1. In your ABAP project (or [ABAP Cloud Project](#)), select the [Source Code Library](#) > [Classes](#) node of the relevant package.
2. To launch the creation wizard, open the context menu and choose [New ABAP Class](#).



Creating ABAP Class – Wizard

Implementing the Interaction Phase and the Save Sequence

In this section, you can find information on how to implement the various steps of the interaction phase and the save sequence in to the runtime of an unmanaged business object.

Implementation Steps

1. [Implementing the CREATE Operation for Travel Instances](#)
2. [Implementing the UPDATE Operation for Travel Data](#)
3. [Implementing the READ Operation for Travel Data](#)

- 4. [Implementing the DELETE Operation for Travel Instances](#)
- 5. [Implementing the CREATE Operation for Associated Bookings](#)
- 6. [Implementing the READ Operation for Associated Bookings](#)
- 7. [Implementing the SET STATUS BOOKED Action](#)
- 8. [Instance Feature Control](#)
- 9. [Implementing the UPDATE, DELETE, and READ Operations for Booking Instances](#)

Related Information

[Handler Classes](#)

[Saver Classes](#)

Implementing the CREATE Operation for Travel Instances

In this topic, you will be guided through all implementation steps required for creation of new travel instances. We will motivate the steps for the implementation, starting from the UI.

Preview

If you run the UI service based on **SAP Fiori Elements Fiori Launchpad**, the resulting UI provides you with a list of existing travel items, including all fields exposed for UI consumption.

To create a travel item, the end user must click the **Create** button and fill all required fields in the related object page to specify the required information for a new travel instance.

Travel

Travel ID:

—

Agency ID: *

Hot Socks Travel (70007)

Customer ID: *

Detemple (99)

Starting Date: *

Nov 17, 2023

End Date: *

Nov 24, 2023

Booking Fee:

45.00

EUR

Total Price:

1,777.00

EUR

Comment:

Business trip to WDF

Travel Status:

New

Create

Cancel

Object Page for Editing Individual Values

As soon as the user clicks the **Save** button on the object page, the data is persisted in the corresponding database table and a travel instance with a new travel ID is created.

 Travel ID	Agency ID	Customer ID	Starting Date	End Date	Travel Status
☐ 4132	Hot Socks Travel (70007)	Kramer (394)	Mar 30, 2023	Mar 30, 2023	New
☒ 17144	Hot Socks Travel (70007)	Detemple (99)	Nov 17, 2023	Nov 24, 2023	New

Saved Data Displayed on Fiori UI

Implementation Steps

Defining the Handler Class for Creation of Travel Instances

Corresponding to the [template](#) for the root node behavior implementation, a local handler class `lhc_travel` is defined to implement each changing operation in one individual `FOR MODIFY` method. In this case, the `create FOR MODIFY` method should only be used to implement the create operation for root instances. Therefore, the signature of this method includes only one import parameter `it_travel_create` for referring to the travel (root) instances to be created. To identify the root entity, the alias `travel` is used - according to the alias that is specified in the behavior definition. Note that import parameter `entities` does not have fixed data type at the design time. At runtime, the data type is assigned by the compiler with the types derived from behavior definition.

i Note

The local handler class `lhc_travel` inherits from class `cl_abap_behavior_handler` and is automatically instantiated by the framework.

Further information: [<method> FOR MODIFY](#)

Implementing the <method> FOR MODIFY for Creation of New Travel Instances

As given in the listing below, the basic structure of the <method> `FOR MODIFY` implementation includes:

- A loop on all new travel instances to be created for the root node.
- Mapping the CDS view field names to the database table fields names by using the operator `MAPPING FROM ENTITY USING CONTROL`. This operator maps only the fields that are flagged in the control structure of the importing parameter.
- Call of the business logic function module `/DMO/FLIGHT_TRAVEL_CREATE` for creation of new travel instances.
- Call of `ADJUST_NUMBERS` to assign the `TRAVEL_ID` and `BOOKING_ID`
- Message handling for processing messages in case of failure. See [Message Handling](#).

Each create action call can produce failed keys (`<travel_create>-%cid`) and messages (`messages`). Any failed keys are stored in the table [FAILED](#) whereas the [REPORTED](#) table includes all messages. For more information about `FAILED` and `REPORTED`, see [ABAP EML -Response Parameters \(ABAP Keyword Documentation\)](#). In some use cases, a consumer works with data that is not yet persisted and might not have a primary key yet. The primary key can be created in the <method> `FOR MODIFY` call or later in the save sequence (late numbering). In the late numbering use case that is used in this scenario, a temporary primary key, the content ID (`%CID` and `%PID`) for an instance, is used for processing as long as no primary key was created by the RAP BO runtime. The content ID is consequently also used then as a foreign key.

In case of success (`messages IS INITIAL`), the two values with the content ID `%CID` and the new key `travelid` assigned in the `ADJUST_NUMBERS` method are written into the mapped - `travel` table.

i Expand the following code sample to view the source code of method `create`.

≡, Sample Code

```

CLASS lhc_travel IMPLEMENTATION.

METHOD create.
  DATA: messages    TYPE /dmo/t_message,
        travel_in    TYPE /dmo/travel,
        travel_out    TYPE /dmo/travel.

  LOOP AT entities ASSIGNING FIELD-SYMBOL(<travel_create>).

    travel_in = CORRESPONDING #( <travel_create> MAPPING FROM ENTITY USING CONTROL ).

    CALL FUNCTION '/DMO/FLIGHT_TRAVEL_CREATE'
      EXPORTING
        iv_numbering_mode = /dmo/if_flight_legacy=>numbering_mode-late
      IMPORTING
        es_travel    = travel_out
        et_messages  = messages.

    map_messages(
      EXPORTING
        cid          = <travel_create>-%cid
        messages     = messages
      IMPORTING
        failed_added = DATA(failed_added)
      CHANGING
        failed       = failed-travel
        reported     = reported-travel
    ).

    IF failed_added = abap_false.
      INSERT VALUE #(
        %cid          = <travel_create>-%cid
        travelid      = travel_out-travel_id )
        INTO TABLE mapped-travel.
    ENDIF.

  ENDLLOOP.
ENDMETHOD.

...

ENDCLASS.

```

Message Handling

i Expand the following code sample to view the source code of the method `get_cause_from_message`.

≡, Sample Code

```

CLASS /dmo/cl_travel_auxiliary DEFINITION
  INHERITING FROM cl_abap_behv
  PUBLIC
  FINAL
  CREATE PUBLIC .

  PUBLIC SECTION.

    CLASS-METHODS get_cause_from_message
      IMPORTING
        msgid          TYPE symmsgid
        msgno          TYPE symsgno
        is_dependent    TYPE abap_bool DEFAULT abap_false
      RETURNING
        VALUE(fail_cause) TYPE if_abap_behv=>t_fail_cause.

```


ENDCLASS.

CLASS /dmo/cl_travel_auxiliary IMPLEMENTATION.

METHOD get_cause_from_message.

fail_cause = if_abap_behv=>cause-unspecific.

IF msgid = '/DMO/CM_FLIGHT_LEGAC'.

CASE msgno.

WHEN '009' "Travel Key initial

OR '016' "Travel does not exist

OR '017' "Booking does not exist

OR '021' "BookingSupplement does not exist

IF is_dependend = abap_true.

fail_cause = if_abap_behv=>cause-unspecific.

ELSE.

fail_cause = if_abap_behv=>cause-not_found.

ENDIF.

WHEN '032' "Travel is locked by

fail_cause = if_abap_behv=>cause-locked.

WHEN '046' "You are not authorized

fail_cause = if_abap_behv=>cause-unauthorized.

ENDCASE.

ENDIF.

ENDMETHOD.

ENDCLASS.

Performing a Final Commit and Releasing Caches

ADJUST NUMBERS

When the ADJUST_NUMBERS method is called during the save after the point-of-no-return, the function module /DMO/FLIGHT_TRAVEL_ADJ_NUMBERS is executed and key values are generated. Then the preliminary key values are mapped from %tmp to the final key values. In this use case, inplace numbering is used, meaning that the temporary key values are assigned to %key during the RAP interaction phase. For more information %tmp, see [ABAP EML -%tmp \(ABAP Keyword Documentation\)](#). For more information about ADJUST_NUMBERS, see [ADJUST NUMBERS](#).

i Expand the following code sample to view the source code of method `adjust_numbers`.

Sample Code

CLASS lsc_saver IMPLEMENTATION.

...

METHOD adjust_numbers.

DATA: travel_mapping TYPE /dmo/if_flight_legacy=>tt_ln_travel_mapping,
 booking_mapping TYPE /dmo/if_flight_legacy=>tt_ln_booking_mapping,
 bookingsuppl_mapping TYPE /dmo/if_flight_legacy=>tt_ln_bookingsuppl_mapping.

CALL FUNCTION '/DMO/FLIGHT_TRAVEL_ADJ_NUMBERS'

IMPORTING

et_travel_mapping = travel_mapping

et_booking_mapping = booking_mapping

et_bookingsuppl_mapping = bookingsuppl_mapping.

mapped-travel = VALUE #(FOR travel IN travel_mapping (%tmp
 TravelID

mapped-booking = VALUE #(FOR booking IN booking_mapping (%tmp

TravelID
 BookingID

```
mapped-bookingsupplement = VALUE #( FOR bookingsuppl IN bookingsuppl_mapping ( %tmp
```

```
TravelID
BookingID
BookingSupple
```

```
ENDMETHOD.
```

SAVE Sequence

When the `save` method is called, the final commit is executed on the database and the data entered by the user is persisted to the new travel instance. As depicted in the listing below, the save method only executes a call to the function module `/DMO/FLIGHT_TRAVEL_SAVE` from the legacy business logic.

Further information: [SAVE](#)

To discard all changes after the last save, the `cleanup` method is used. This method delegates the call to the function module `/DMO/FLIGHT_TRAVEL_INITIALIZE` from legacy code.

Add the definition for the cleanup method to `lsc_saver` and call the function module in the implementation.

i *Expand the following code sample to view the source code of the **save sequence**.*

Sample Code

```
CLASS lsc_saver IMPLEMENTATION.
...
METHOD save.
    CALL FUNCTION '/DMO/FLIGHT_TRAVEL_SAVE'.
ENDMETHOD.
METHOD cleanup.
    CALL FUNCTION '/DMO/FLIGHT_TRAVEL_INITIALIZE'.
ENDMETHOD.

ENDCLASS.
```

Checking Results

At this point, you have the opportunity to check how does the resulting app work, and especially the new implementation of the `CREATE` operation. For this to happen, however, a suitable business service for UI consumption must first be defined and published.

For more information, see: [Defining Business Service for Fiori UI](#)

Related Information

[Handler Classes](#)

[Saver Classes](#)

[Derived Data Types](#)

[Implicit Response Parameters](#)

Implementing the UPDATE Operation for Travel Data

This topic guides you through the implementation steps required for data updates to an existing travel instance. In this case, however, in addition to the <method> FOR MODIFY, the <method> FOR READ must also be implemented. It provides read access to the application buffer, which is necessary for **ETag** comparison.

Preview

In our travel application scenario, the appropriate business data should be modifiable for all required items of the travel instance when, for example, the user clicks the **Edit** button on the Fiori UI.

The screenshot shows the SAP Fiori 'Travel' app interface. At the top, there is a back arrow and the title 'Travel'. Below the title, a large number '5' is displayed. To the right of the '5' is an 'Edit' button (highlighted with an orange box) and a 'Delete' button. Below these buttons are two tabs: 'Travel' (selected) and 'Booking'. The main content area shows the following data:

Travel ID:	5
Agency ID:	Hot Socks Travel (70007)
Customer ID:	Mechler (225)

Below the data table, it says 'Saved Travel Data'.

In change mode, the end user is able to change the relevant travel fields as shown in the figure below. As soon as the user chooses the **Save** button on the object page, the changed travel data is saved in the corresponding tables and a new version of the related travel instance is created.

The screenshot shows the 'Changed data' section of the SAP Fiori 'Travel' app. The section is highlighted with an orange box. The data shown is:

Booking Fee:	45.00 EUR
Total Price:	1,123.00 EUR
Comment:	Business Trip to WDF, John
Travel Status:	Planned

Below the data table, it says 'Your changes have been saved.'.

Below the 'Your changes have been saved.' message, it says 'Saved Travel Data'.

Implementation Steps

Defining the Method for Implementing Travel Data Update

Corresponding to the [template](#) for the root node behavior implementation, a local handler class `lhc_travel` is defined to implement each changing operation. In this case, the `update FOR MODIFY` method should only be used to implement the update operation for root instances. Therefore, the signature of this method includes only one import parameter `entities` for referring to the `travel` (root) instances to be updated.

Signature of the `<method> FOR MODIFY` (excerpt from template)

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS:
      ...
      METHODS update FOR MODIFY
        IMPORTING entities FOR UPDATE travel.
ENDCLASS.
```

Implementing the `<method> FOR MODIFY` for Travel Data Update

The basic structure of the `FOR MODIFY` method implementation is very similar to that of the handler class for creation of travel instances:

- A loop on all new travel instances to be updated for the root node.
- Mapping the CDS view field names to the database table fields names using the operator `MAPPING FROM ENTITY`. This operator maps every field that is declared in the mapping specification in the behavior definition.
- Mapping of the `%control`-structure of the importing parameter to the predefined flag structure `ls_travelx`. The control structure identifies which entity fields were changed by the client. It contains the key and the flag structure to the data fields and is used in BAPIs to flag each individual data field as changed.
- Call of the business logic function module `/DMO/FLIGHT_TRAVEL_UPDATE` to update travel instances.
- Message handling for processing messages in case of failure.

Each update call can produce failed keys and messages (`lt_messages`). Failed keys are addressed by the content ID (`<travel_update>-%cid_ref`) and the value `<travel_update>-travelid`). In case of failure, failed keys are saved in the `failed-travel` table, whereas the `reported-travel` table includes all instance-specific messages.

i Expand the following code sample to view the source code of the method `update`.

Sample Code

```
CLASS lhc_travel IMPLEMENTATION.
  ...
  METHOD update.
    DATA: messages TYPE /dmo/t_message,
          travel    TYPE /dmo/travel,
          travelx   TYPE /dmo/s_travel_inx. "refers to x structure (> BAPIs)

    LOOP AT entities ASSIGNING FIELD-SYMBOL(<travel_update>).

      travel = CORRESPONDING #( <travel_update> MAPPING FROM ENTITY ).

      travelx-travel_id = <travel_update>-TravelID.
      travelx-_intx     = CORRESPONDING #( <travel_update> MAPPING FROM ENTITY ).
```

```

CALL FUNCTION '/DMO/FLIGHT_TRAVEL_UPDATE'
  EXPORTING
    is_travel    = CORRESPONDING /dmo/s_travel_in( travel )
    is_travelx   = travelx
  IMPORTING
    et_messages = messages.

map_messages(
  EXPORTING
    cid      = <travel_update>-%cid_ref
    travel_id = <travel_update>-travelid
    messages = messages
  CHANGING
    failed    = failed-travel
    reported  = reported-travel
).

ENDLOOP.
ENDMETHOD.

...

ENDCLASS.

```

Updating data of travel instances

The screenshot shows the SAP IDE interface. A red arrow labeled 'F2' points to the entity name '<fs_travel_update>' in the code editor. A pop-up window displays the data dictionary for this entity, listing its attributes and their corresponding data types.

Attribute	Type
%cid_ref	type abp_behv_cid
travelid	type /dmo/travel_id
agencyid	type /dmo/agency_id
customerid	type /dmo/customer_id
begindate	type /dmo/begin_date
enddate	type /dmo/end_date
bookingfee	type /dmo/booking_fee
totalprice	type /dmo/total_price
currencycode	type /dmo/currency_code
memo	type /dmo/description
status	type /dmo/travel_status
lastchangedat	type timestamp
%control	
travelid	type abp_behv_flag
agencyid	type abp_behv_flag
customerid	type abp_behv_flag
begindate	type abp_behv_flag
enddate	type abp_behv_flag
bookingfee	type abp_behv_flag
totalprice	type abp_behv_flag
currencycode	type abp_behv_flag
memo	type abp_behv_flag
status	type abp_behv_flag
lastchangedat	type abp_behv_flag

F2 Access for <fs_travel_update>

Implementing the <method> FOR READ for ETag Handling

In the context of data updates to an existing travel data set, it is important to retrieve current data from the application buffer. As you remember, we specified an ETag for the root entity in the behavior definition:

```

define behavior for /DMO/I_Travel_U alias travel
etag master LastChangedAt
{
...
}

```

An ETag determines the changes to the requested data set to help prevent simultaneous updates of a data set from overwriting each other. This is precisely the reason why the ETag check requires data from the buffer. The `<method> FOR READ` is designed to return the data from the application buffer.

For more information on how to implement this method, see [Implementing the READ Operation for Travel Data](#).

Related Information

[Handler Classes](#)

[Derived Data Types](#)

[Implicit Response Parameters](#)

Implementing the READ Operation for Travel Data

This topic guides you through the implementation steps required to read travel instances from the transactional buffer.

Context

In contrast to the CREATE, UPDATE, or DELETE operation, the READ does not have its direct trigger on a Fiori UI.

→ Remember

The **GO** button on the UI does not call the `<method> FOR READ` (the transactional READ) in the behavior pool. Instead, the **GO** button executes a query via the RAP runtime engine and reads data directly from the database.

The READ operation provides read access to the application buffer. It is used to retrieve data for further processing during the interaction phase. This is necessary, for example, for ETag comparison when executing an UPDATE. The conditions for actions are checked to decide if they are enabled or disabled. In the travel scenario, the action `<method> for UPDATE` is called, the ETag value in the application buffer must be compared to the value that is displayed on the UI. Only if these values correspond is the UPDATE triggered. This check ensures that the data the end user sees on the UI has not been changed by other consumers.

The READ is also necessary for [dynamic action control](#) `set_to_booked` is disabled when the status is booked. Hence, the data from the transactional buffer must be read to display the action button correspondingly.

Both use cases are relevant for our travel scenario. Therefore it is necessary to implement a `<method> FOR READ`.

The READ is also necessary to complete the business object and make it accessible by EML. In this case, you can consume the business object directly from ABAP.

Implementation Steps

The READ operation cannot be declared in the behavior definition as it is always implicitly assumed that a READ is implemented.

i Note

If you use groups in your behavior definition, you must explicitly specify the READ operation in one of the groups. For more information, see [Using Groups in Large Development Projects](#).

Defining the signature of the <method> FOR READ

In the definition part of the handler class for travel, define the method `read_travel` for READ with an importing parameter and a result parameter.

Signature of the <method> for READ

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    METHODS:
      ...
      METHODS read FOR READ
        IMPORTING keys FOR READ travel RESULT result.
      ...
ENDCLASS.
```

For more information, see [<method> FOR READ](#).

Implementing the <method> FOR READ

To return the actual data from the application buffer, the travel ID that is imported is passed to the reading function module `/DMO/FLIGHT_TRAVEL_READ`. The function module returns the entity instance corresponding to the travel ID and messages if an error occurs. The result parameter `result` is then filled with the keys and the values of the fields that the consumer has requested. These fields are flagged in the control structure of the importing parameter `keys`.

With the changing parameter `failed`, you specify the fail cause if something goes wrong. If, for example, the used function module returns the message number 16, the travel ID was not found. In this case, you can return the fail cause `not_found`. For all other errors, return fail cause `unspecific`.

i *Expand the following code sample to view the source code of the method `read`.*

Sample Code

```
...
METHOD read.
  DATA: travel_out TYPE /dmo/travel,
        messages   TYPE /dmo/t_message.

  LOOP AT keys ASSIGNING FIELD-SYMBOL(<travel_to_read>) GROUP BY <travel_to_read>-%tky.

    CALL FUNCTION '/DMO/FLIGHT_TRAVEL_READ'
      EXPORTING
        iv_travel_id = <travel_to_read>-travelid
      IMPORTING
        es_travel    = travel_out
        et_messages  = messages.

    map_messages(
      EXPORTING
        travel_id      = <travel_to_read>-TravelID
        messages       = messages
      IMPORTING
        failed_added = DATA(failed_added)
      CHANGING
        failed       = failed-travel
        reported     = reported-travel
    ).
  }
```

```
IF failed_added = abap_false.  
  INSERT CORRESPONDING #( travel_out MAPPING TO ENTITY ) INTO TABLE result.  
ENDIF.  
ENDLOOP.  
ENDMETHOD.  
...  

```

Testing the READ

In contrast to the CREATE, UPDATE and DELETE operation, the READ cannot be easily tested using a Fiori UI, as there is no direct execution trigger for reading the data from the application buffer. Nevertheless, you can test your READ implementation by using EML.

Example

```
READ ENTITY /DM0/I_Travel_U FIELDS ( TravelID  
                                     AgencyID  
                                     CustomerID  
                                     BeginDate  
                                     EndDate  
                                     BookingFee  
                                     TotalPrice  
                                     CurrencyCode  
                                     Memo  
                                     Status  
                                     LastChangedAt )  
WITH VALUE #( ( %key-TravelID = lv_travel_id ) )  
RESULT DATA(lt_received_travel_data)  
FAILED DATA(ls_failed).
```

To retrieve the complete entity instance for the respective travel ID, you have to flag every element explicitly. For more information about EML, see [Entity Manipulation Language \(EML\)](#).

Implementing the DELETE Operation for Travel Instances

This topic guides you through the implementation steps required to delete an existing travel instance.

Preview

In our scenario, the appropriate travel instance should be deleted when, for example, the user clicks the **Delete** button on the Fiori UI.

Travels (1) Set to Booked Create Delete ⚙️ 🔍 ⌵

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Travel Status	
☒ 10	Ali's Bazar (70016)	Leisert (697)	Jun 4, 2022	Jun 6, 2022	New	>

Selecting a Travel Entry and Clicking the Delete Button

Travel ID	Agency ID	Customer ID	Starting Date	End Date	Travel Status
-----------	-----------	-------------	---------------	----------	---------------

No data found. Try adjusting the search or filter criteria.

Object was deleted.

Expected Behavior

Implementation Steps

Defining the Handler Class for Deletion of Travel Instances

Corresponding to the [template](#) for the root node behavior implementation, a local handler class is defined to implement each changing operation. In this case, the `delete FOR MODIFY` is used to implement the delete operation for root instances. As given in the listing below, the signature of the `<method> FOR MODIFY` includes only one import parameter `keys` for referring to the `travel (root)` instances to be deleted.

Signature of the `delete FOR MODIFY` (excerpt from template)

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    METHODS:
      ...
    METHODS delete FOR MODIFY
      IMPORTING keys FOR DELETE travel.
      ...
ENDCLASS.
```

Implementing the Deletion of Travel Instances

To delete travel instances, the function module `/DMO/FLIGHT_TRAVEL_DELETE` of the legacy business logic is called.

Each delete operation call can produce failed keys and messages (`lt_messages`). Failed keys are addressed by the content ID (`<travel_delete>-%cid_ref`) and the key value `<travel_delete>-travel_id`. In case of failure, failed keys are saved in the `failed-travel` table, whereas the `reported-travel` table includes all instance-specific messages.

i *Expand the following code sample to view the source code of the method `delete`.*

Sample Code

```
CLASS lhc_travel IMPLEMENTATION.

  ...

  METHOD delete.
    DATA: messages TYPE /dmo/t_message.

    LOOP AT keys ASSIGNING FIELD-SYMBOL(<travel_delete>).

      CALL FUNCTION '/DMO/FLIGHT_TRAVEL_DELETE'
        EXPORTING
          iv_travel_id = <travel_delete>-travelid
        IMPORTING
          et_messages = messages.

      map_messages(
        EXPORTING
          cid      = <travel_delete>-%cid_ref
          travel_id = <travel_delete>-travelid
          messages = messages
        CHANGING
          failed   = failed-travel
          reported = reported-travel
      ).

    ENDLLOOP.
  ENDMETHOD.

  ...
```

ENDCLASS.

Related Information

[Handler Classes](#)
[Saver Classes](#)
[Derived Data Types](#)
[Implicit Response Parameters](#)

Implementing the CREATE Operation for Associated Bookings

In this topic, you will be guided through all implementation steps required for creation of new bookings.

In our demo application, we assume that new bookings cannot be created separately but can only in conjunction with a given travel instance.

The fact that new instances of the bookings can only be created for a specific travel instance is considered in the behavior definition by the `_Booking` association:

```
define behavior for /DMO/I_Travel_U alias travel
...
{
    ...
    association _Booking { create; }
}
```

The keyword `{ create; }` declares that this association is create-enabled what exactly means that instances of the associated bookings are created by a travel instance.

Preview

To add a new booking to a selected travel instance, the end user must click the **Create** button and edit some fields to specify the required information for a new booking.

Booking

Booking Number:	Airline ID: *	Flight Date: *
–	American Airlines Inc. (AA)	Oct 26, 2023
Booking Date: *	Flight Number: *	Flight Price:
Mar 23, 2023	18	3,781.00 USD
Customer ID: *		
Detemple (99)		

Object Page for Editing Individual Booking Values

As soon as the user clicks the **Create** button on the Fiori object page, a booking data set with a new booking number is created.

Bookings (3) Standard* ▼		Search Q		Create	Delete			▼
☐ Booking Number	Booking Date	Customer ID	Airline ID	Flight Number	Flight Date	Flight Price		
☐ 1	Dec 28, 2022	Hoffen (161)	United Airlines, Inc. (UA)	1537	Dec 31, 2022	438.00	USD	>
☐ 2	Dec 28, 2022	Miguel (461)	United Airlines, Inc. (UA)	1537	Dec 31, 2022	438.00	USD	>
☐ 3	Mar 23, 2023	Detemple (99)	American Airlines Inc. (AA)	18	Oct 26, 2023	3,781.00	USD	>

New Booking Data Set

Implementation Steps

Defining Method for Creation of Associated Bookings

Corresponding to the [template](#) for the root entity, the local handler class `lhc_travel` is defined to implement each changing operation in one individual `<method> FOR MODIFY`. In this case, the `cba_booking FOR MODIFY` method should only be used to implement the create operation for booking instances by means of an association. The signature of the `cba_Booking FOR MODIFY` includes only one import parameter `entities_cba` to refer to the associated booking instances to be created.

To identify the associated bookings, the aliases for the root entity and the child entity are used - according to the aliases specified in the behavior definition. The association is expressed in the form:

```
... FOR CREATE root\_child_entity.
```

Signature of the `cba_booking FOR MODIFY` method (excerpt from template)

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS:
      ...
      METHODS cba_Booking FOR MODIFY
      IMPORTING entities_cba FOR CREATE travel\_Booking.
ENDCLASS.
```

Implementing a `FOR MODIFY` Method for Creation of New Booking Instances

As reproduced in the listing below, the implementation of the `FOR MODIFY` method is initiated by a `loop` across all selected travel instances for which associated bookings are to be created. Each selected travel (root) instance is represented by the travel ID as a primary key (`travelid`).

Even in a case like this, it can happen that a consumer works with data that is not yet persisted and might not have a primary key yet (for example, if the primary key is going to be created later in the save sequence (late numbering)). In such cases, a temporary primary key, the content ID (`%CID`) for the travel instance is used as long as no primary key was created by BO runtime. The content ID is then written to the mapped - `travel` table.

Before a new booking data set is created, we first need to retrieve all bookings (`booking_old`) that already exist for the selected travel instance. This is done by calling the `/DMO/FLIGHT_TRAVEL_READ` function module.

In case of failure, the message is handled by means of the `map_messages` method, as we know from the implementation of `FOR MODIFY` in the previous topics.

In case of success, the maximum booking number has to be determined. The last given booking number `last_booking_id` is increased by 1 for the new booking instance.

The creation of new bookings for a given travel instance takes place in a further loop across the booking instances to be created, which are addressed by the association <travel>-%target. This association includes the predefined component %target, which is used to address the target of composition.

To provide the incoming structure for bookings with data, the mapping between the element in CDS views and the original table fields is required. This mapping is implemented by the operator `MAPPING FROM ENTITY USING CONTROL`, which maps the CDS view fields to the database table fields based on the mapping specification in the behavior definition and the control structure.

Before the new booking data sets are created by calling the function module `/DMO/FLIGHT_TRAVEL_UPDATE`, the booking ID for the booking instance to be created is easily determined by the statement `booking-booking_id += 1`.

In case of success, the values with the content ID %CID and the key values `travelid`, and `bookingid` are written to the `mapped-booking` table.

The function call can produce failed keys (<bbooking_create>-%cid) and messages (messages). Any failed keys are stored in the table `failed-booking` whereas the `reported-booking` table includes all messages that are specific for the failed booking instance

i Expand the following code sample to view the source code of method `cba_Booking`.

☰ Sample Code

```
CLASS lhc_travel IMPLEMENTATION.
...
METHOD cba_Booking.
  DATA: messages      TYPE /dmo/t_message,
        booking_old   TYPE /dmo/t_booking,
        booking       TYPE /dmo/booking,
        last_booking_id TYPE /dmo/booking_id VALUE '0'.

  LOOP AT entities_cba ASSIGNING FIELD-SYMBOL(<travel>).

    DATA(travelid) = <travel>-travelid.

    CALL FUNCTION '/DMO/FLIGHT_TRAVEL_READ'
      EXPORTING
        iv_travel_id = travelid
      IMPORTING
        et_booking   = booking_old
        et_messages  = messages.

    map_messages(
      EXPORTING
        cid      = <travel>-%cid_ref
        travel_id = <travel>-TravelID
        messages  = messages
      IMPORTING
        failed_added = DATA(failed_added)
      CHANGING
        failed      = failed-travel
        reported     = reported-travel
    ).

    IF failed_added = abap_true.
      LOOP AT <travel>-%target ASSIGNING FIELD-SYMBOL(<booking>).
        map_messages_assoc_to_booking(
          EXPORTING
            cid      = <booking>-%cid
            travel_id = <travel>-TravelID
            booking_id = <booking>-BookingID
            is_dependend = abap_true
            messages  = messages
          CHANGING
```

```

        failed      = failed-booking
        reported    = reported-booking
    ).
ENDLOOP.

ELSE.

    " Set the last_booking_id to the highest value of booking_old booking_id or initial valu
    last_booking_id = VALUE #( booking_old[ lines( booking_old ) ]-booking_id OPTIONAL ).

    LOOP AT <travel>-%target ASSIGNING FIELD-SYMBOL(<booking_create>).

        booking = CORRESPONDING #( <booking_create> MAPPING FROM ENTITY USING CONTROL ) .

        last_booking_id += 1.
        booking-booking_id = last_booking_id.

        CALL FUNCTION '/DMO/FLIGHT_TRAVEL_UPDATE'
            EXPORTING
                is_travel      = VALUE /dmo/s_travel_in( travel_id = travelid )
                is_travelx     = VALUE /dmo/s_travel_inx( travel_id = travelid )
                it_booking     = VALUE /dmo/t_booking_in( ( CORRESPONDING #( booking ) ) )
                it_bookingx    = VALUE /dmo/t_booking_inx(
                    (
                        booking_id = booking-booking_id
                        action_code = /dmo/if_flight_legacy=>action_code-create
                    )
                )
            IMPORTING
                et_messages = messages.

        map_messages_assoc_to_booking(
            EXPORTING
                travel_id      = <booking_create>-TravelID
                booking_id     = <booking_create>-bookingID
                messages       = messages
            IMPORTING
                failed_added = failed_added
            CHANGING
                failed      = failed-booking
                reported    = reported-booking
        ).

        IF failed_added = abap_false.
            INSERT
                VALUE #(
                    %cid      = <booking_create>-%cid
                    travelid   = travelid
                    bookingid  = booking-booking_id
                ) INTO TABLE mapped-booking.
        ENDIF.

    ENDLOOP.
ENDIF.
ENDLOOP.
ENDMETHOD.

```

Related Information

[Handler Classes](#)

[Derived Data Types](#)

[Implicit Response Parameters](#)

Implementing Dynamic Operation Control for Create By Association

This topic guides you through the required implementation steps to dynamically enable and disable the create by association operation for bookings.

Context

Depending on the value of the Status field of the travel instance, the create by association operation is enabled or disabled on the corresponding travel instance.

The Fiori Elements app preview displays the create button for the associated bookings on the object page only if the operation is enabled.

Implementation Steps

Defining dynamic feature control in the behavior definition

In the behavior definition, the feature control for the create by association is defined as follows:

```
...
define behavior for /DMO/I_Travel_U alias travel
implementation in class /DMO/BP_TRAVEL_U unique
...

{
  ...

  association _Booking { create (features: instance); }
  ...
}
```

Implementing dynamic feature control in the behavior pool

The <method> FOR FEATURES must be declared. For more information, see [<method> FOR FEATURES](#).

The travel instance is read and, depending on the value of the Status field, the create by association is enabled or disabled.

Expand the following code sample to view the source code of the CLASS `lhc_travel` DEFINITION INHERITING FROM `cl_abap_behavior_handler`.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS:
      ...
      METHODS get_instance_features FOR INSTANCE FEATURES
        IMPORTING keys REQUEST requested_features FOR travel RESULT result.
ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  ...
  METHOD get_instance_features.
    READ ENTITIES OF /DMO/I_Travel_U IN LOCAL MODE
      ENTITY Travel
        FIELDS ( TravelID Status )
        WITH CORRESPONDING #( keys )
    RESULT DATA(travel_read_results)
    FAILED failed.
```

```

    result = VALUE #(
      FOR travel_read_result IN travel_read_results (
        %tky                                = travel_read_result-%tky
        %features-%action-set_status_booked = COND #( WHEN travel_read_result-Status = 'B'
                                                         THEN if_abap_behv=>fc-o-disabled ELSE if_a
        %assoc-_Booking                     = COND #( WHEN travel_read_result-Status = 'B' OR tr
                                                         THEN if_abap_behv=>fc-o-disabled ELSE if_a
      ) ).
    ENDMETHOD.
  ENDClass.

```

Related Information

[Feature Control](#)

Implementing the READ Operation for Associated Bookings

This topic guides you through the implementation steps required to read booking instances associated to a travel instance from the buffer.

Context

Just like the READ for travel data, the READ `by association` does not have an explicit trigger on the UI. Nevertheless, the BO must be fully consumable by EML and therefore it is necessary to implement a method that reads the bookings that are associated to a specific travel ID in our travel scenario.

The READ `by association` operation provides read access to the application buffer by reading child entity instances of a parent entity. It is used to retrieve data for further processing during the interaction phase.

Implementation Steps

The READ `by association` operation does not have to be declared in the behavior definition as it always implicitly assumed that READ `by association` is implemented. However, you can specify it explicitly:

```

define behavior for /DMO/I_Travel_U alias travel
{
  ...
  association _Booking;
}

```

i Note

☁ If you use groups in your behavior definition, you must explicitly specify the READ `by association` operation in one of the groups. If CREATE `by association` is also enabled for the BO, the READ and the CREATE `by association` must be assigned to the same group due to its syntactical relation. For more information, see [Using Groups in Large Development Projects](#).

1. Defining the signature of the <method> FOR READ `by association`

In the definition part of the handler class for travel, define the method `read_booking_ba` for `READ travel\_Booking` with the entity input parameters `IMPORTING` and `FULL`, and the output parameters `RESULT` and `LINK`.

The boolean parameter `FULL` indicates whether the consumer requests complete entity instances or only the keys which are returned with the parameter `LINK`.

The parameter `RESULT` returns the complete set of associated entity instances based on the control structure of the importing parameter.

The parameter `LINK` returns the key of the source entity and of the associated target entities.

Signature of the `<method> FOR READ by association`

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

    PRIVATE SECTION.
        METHODS:
            ...
        METHODS rba_Booking FOR READ
            IMPORTING keys_rba FOR READ travel\_Booking FULL result_requested RESULT result LINK associat
            ... .
ENDCLASS.
```

Implementing the `<method> FOR READ by association`

To return the actual data from the application buffer, the imported travel ID is passed to the reading function module `/DMO/FLIGHT_TRAVEL_READ`. The function module returns the entity instances of the associated bookings corresponding to the travel ID and possible messages if an error occurs. If no message is returned, for each associated booking, fill the output parameter `association_links` with the corresponding source key for the travel instance and the target key for the booking instance.

In the changing parameter `failed`, you specify the fail cause if something goes wrong. If, for example, the used function module returns the message number 16, the travel ID was not found. In this case, you can return the fail cause `not_found`. For all other error, return fail cause `unspecific`.

i *Expand the following code sample to view the source code of the method `rba_Booking`.*

Sample Code

```
METHOD rba_Booking.
    DATA: travel_out TYPE /dmo/travel,
           booking_out TYPE /dmo/t_booking,
           booking LIKE LINE OF result,
           messages TYPE /dmo/t_message.

    LOOP AT keys_rba ASSIGNING FIELD-SYMBOL(<travel_rba>) GROUP BY <travel_rba>-TravelID.

        CALL FUNCTION '/DMO/FLIGHT_TRAVEL_READ'
            EXPORTING
                iv_travel_id = <travel_rba>-travelid
            IMPORTING
                es_travel = travel_out
                et_booking = booking_out
                et_messages = messages.

        map_messages(
```



```

EXPORTING
  travel_id      = <travel_rba>-TravelID
  messages      = messages
IMPORTING
  failed_added = DATA(failed_added)
CHANGING
  failed        = failed-travel
  reported      = reported-travel
).

IF failed_added = abap_false.
  LOOP AT booking_out ASSIGNING FIELD-SYMBOL(<booking>).
    "fill link table with key fields

    INSERT
      VALUE #(
        source-%tky = <travel_rba>-%tky
        target-%tky = VALUE #(
          TravelID = <booking>-travel_id
          BookingID = <booking>-booking_id
        ) )
      INTO TABLE association_links.

    IF result_requested = abap_true.
      booking = CORRESPONDING #( <booking> MAPPING TO ENTITY ).
      INSERT booking INTO TABLE result.
    ENDIF.

  ENDLOOP.
ENDIF.

ENDLOOP.

SORT association_links BY target ASCENDING.
DELETE ADJACENT DUPLICATES FROM association_links COMPARING ALL FIELDS.

SORT result BY %tky ASCENDING.
DELETE ADJACENT DUPLICATES FROM result COMPARING ALL FIELDS.
ENDMETHOD.

```

Testing

Just like the READ operation, the READ by association cannot be easily tested using a Fiori UI, as there is no direct execution trigger for reading the associated booking from the application buffer. Nevertheless, you can test your READ by association implementation by using EML.

♣ Example

```

READ ENTITY /dmo/i_travel_u
BY \_Booking FIELDS ( AirlineID
                     BookingDate
                     BookingID
                     CustomerID
                     CurrencyCode
                     LastChangedAt )
WITH VALUE #( ( %key-TravelID = travelID ) )
RESULT DATA(lt_read_bookings_ba)
LINK DATA(lt_link_table)
FAILED DATA(ls_failed_rba).

```

If you request the RESULT parameter, the importing parameter FULL is set and you retrieve the values based on the control flags. The parameter LINK is independent on the control flags, it always retrieves the keys of the source and target entity.

Implementing the LOCK Operation

This topic guides you through the implementation steps required to lock instances on the database.

Context

Locking prevents simultaneous changes on the database.

The lock operation is executed before modify operations. It locks the entity instance during the time of the modify operation and disables other clients to modify the specific instance and all its locking related instances. As soon as the modify operation is finished, the lock is released.

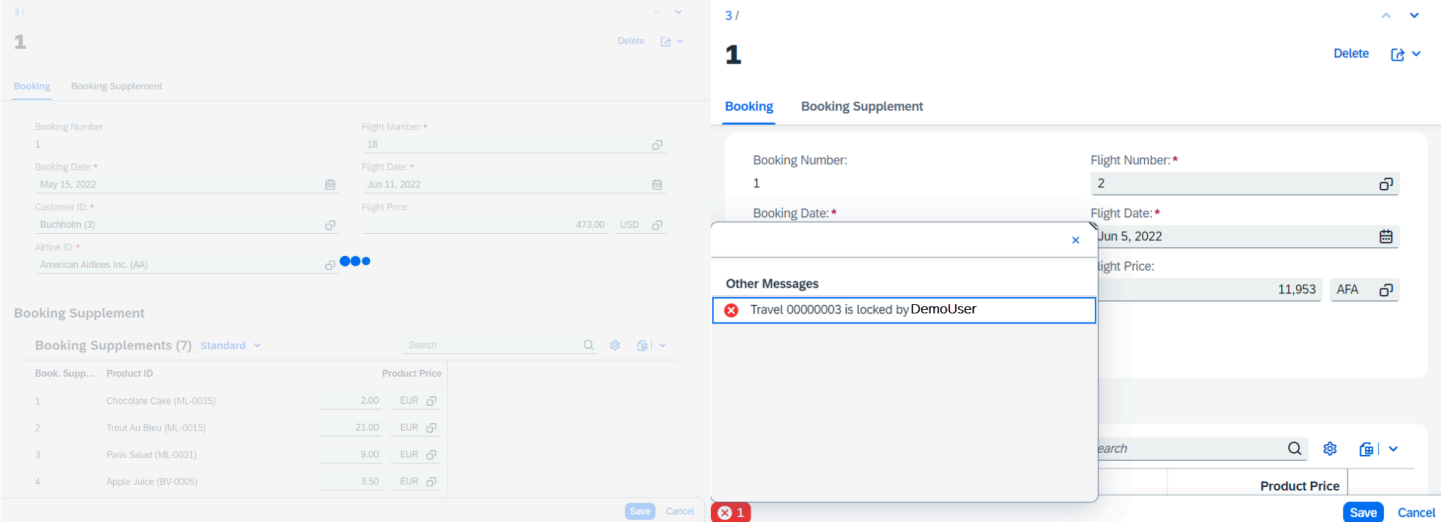
For more information, see [Pessimistic Concurrency Control \(Locking\)](#).

In our unmanaged scenario, the travel entity was defined as lock master and the booking entity as lock dependent. That means, the locking mechanism always locks the lock master instances and all its dependents. For example, if the lock operation is executed because a booking instance is updated, the parent travel instance and all its booking instances are also being locked.

To enable locking, the method `FOR LOCK` must be implemented. The legacy code of the unmanaged scenario includes a lock object, which must be called in the method `FOR LOCK` to lock the relevant entity instances.

Preview

In our scenario, it should not be possible to save changes on the database, while another entity instance of the same lock master entity instance is being changed at the same moment. In this case, the end user on the UI gets an error message:



Concurrent Changes of Entity Instances of the Same Lock Master Instance

Implementation Steps

The legacy code of the unmanaged scenario includes a lock object, which must be called in the method `FOR LOCK` to lock the relevant entity instances.

Defining the Handler Class for Locking Travel Instances

Corresponding to the [template](#) for the root node behavior implementation, a local handler class is defined to implement each operation. In this case, the method `lock FOR LOCK` is used to implement the lock operation for the travel business object. As you can see in the listing below, the signature of `<method> FOR LOCK` includes only one importing parameter `keys` for referring to the lock master's key.

Signature of the `lock FOR LOCK` (excerpt from template)

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.
METHODS:
...
METHODS lock FOR LOCK
IMPORTING keys FOR LOCK travel.
...
ENDCLASS.

```

Implementing Locking of Travel Instances

To lock a travel instance and its lock dependents, instantiate the lock object /DMO/ETRAVEL of the legacy business logic. This is done by calling the factory method `get_instance` of `cl_abap_lock_object_factory`. This instantiation must not fail, except for technical errors. You can raise a short dump if this happens.

Call the `enqueue` method of the lock object and pass the imported travel ID to execute locking. If the instance is already locked, you will get an exception. This exception can be handled by the `handle_travel_messages` of the auxiliary class to fill `failed` and `reported` table. If the lock fails for technical reasons, raise a short dump.

i *Expand the following code sample to view the source code of the method `lock`.*

Sample Code

```

CLASS lhc_travel IMPLEMENTATION.
...
METHOD lock.
TRY.
  "Instantiate lock object
  DATA(lock) = cl_abap_lock_object_factory=>get_instance( iv_name = '/DMO/ETRAVEL' ).
  CATCH cx_abap_lock_failure INTO DATA(exception).
    RAISE SHORTDUMP exception.
ENDTRY.

LOOP AT keys ASSIGNING FIELD-SYMBOL(<travel>).
TRY.
  "enqueue travel instance
  lock->enqueue(
    it_parameter = VALUE #( ( name = 'TRAVEL_ID' value = REF #( <travel>-travelid )
  ).
  "if foreign lock exists
  CATCH cx_abap_foreign_lock INTO DATA(foreign_lock).
    map_messages(
      EXPORTING
        travel_id = <travel>-TravelID
        messages = VALUE #( (
          msgid = '/DMO/CM_FLIGHT_LEGAC'
          msgty = 'E'
          msgno = '032'
          msgv1 = <travel>-travelid
          msgv2 = foreign_lock->user_name )
      )
      CHANGING
        failed = failed-travel
        reported = reported-travel
    ).

    CATCH cx_abap_lock_failure INTO exception.
      RAISE SHORTDUMP exception.
ENDTRY.
ENDLOOP.
ENDMETHOD.
...

```

ENDCLASS.

Testing

To test if the locking mechanism works properly, open the preview twice and set a breakpoint after the method enqueue.

In the first preview application, delete or update any entity instance. The ADT debugger stops at the breakpoint. At this point, the enqueue lock is already set on the chosen entity instance's lock master and its dependents. Try to do a modify operation with the other preview application on the same entity instance or any of its lock master's dependents. Do not stop at the breakpoint this time. You will see that the modify operation is rejected because the entity instance is locked by the client of the first preview application.

Related Information

[Handler Classes](#)

[Saver Classes](#)

[Derived Data Types](#)

[Implicit Response Parameters](#)

Implementing the SET_STATUS_BOOKED Action

This topic describes the implementation of an action related to the travel instances. Using this action, the end user should be able to change the status of travel processing.

Preview

Again, we use the option of running the resulting app based on [Fiori Elements](#) to check the action execution. When we run the app, the UI screen provides the button **Set to Booked** for the action as shown in the figure below.

Travels (1)					Set to Booked Create Delete	
Travel ID	Agency ID	Customer ID	Starting Date	Travel Status		
☒ 5	Hot Socks Travel (70007)	Mechler (225)	Jun 4, 2022	Planned	>	

Original Status is P (= In Process)

Travel ID	Agency ID	Customer ID	Starting Date	Travel Status		
☒ 5	Hot Socks Travel (70007)	Mechler (225)	Jun 4, 2022	Booked	>	

Changed Status After Action Execution (B = Booked)

Implementation Steps

Once an action is defined in the behavior definition, it must be implemented in the behavior pool of the business object.

Defining the Method for Action

As described in [Action Implementation](#) the method FOR ACTION must be declared in the private section of the behavior pool.

The beginning of the source code excerpt in the following listing shows the declaration of two table types, one for ACTION IMPORT (the importing parameter) and the other for ACTION RESULT (the exporting parameter).

The importing parameter is a freely selected name. The action name `set_status_booked` refers to the name of the action defined in the behavior definition and the entity `travel`, on which the action is assigned. The result parameter is also a freely selected name.

Defining the signature of `set_travel_status FOR MODIFY`

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

    METHODS:
      ...
    METHODS set_status_booked FOR MODIFY
      IMPORTING keys FOR ACTION travel~set_status_booked RESULT result.
      ...

ENDCLASS.
```

Implementing `set_travel_status FOR MODIFY`

To execute the actual action, the function module `/DMO/FLIGHT_TRAVEL_SET_BOOKING` of the legacy business logic is called.

The main implementation steps are the same as in the implementation of the CUD operations.

To fill the action result parameter accordingly to the action definition, a read must be executed.

i *Expand the following code sample to view the source code of the method `set_status_booked`.*

Sample Code

```
CLASS lhc_travel IMPLEMENTATION.
  ...

  METHOD set_status_booked.
    DATA: messages TYPE /dmo/t_message,
           travel_out TYPE /dmo/travel,
           travel_set_status_booked LIKE LINE OF result.

    CLEAR result.

    LOOP AT keys ASSIGNING FIELD-SYMBOL(<travel_set_status_booked>).

      DATA(travelid) = <travel_set_status_booked>-travelid.

      CALL FUNCTION '/DMO/FLIGHT_TRAVEL_SET_BOOKING'
        EXPORTING
          iv_travel_id = travelid
        IMPORTING
          et_messages = messages.

      map_messages(
        EXPORTING
          travel_id = <travel_set_status_booked>-TravelID
          messages = messages
        IMPORTING
          failed_added = DATA(failed_added)
        CHANGING
          failed = failed-travel
          reported = reported-travel
      ).

      IF failed_added = abap_false.
```

```
CALL FUNCTION '/DMO/FLIGHT_TRAVEL_READ'
EXPORTING
  iv_travel_id = travelid
IMPORTING
  es_travel    = travel_out.

travel_set_status_booked-travelid      = travelid.
travel_set_status_booked-%param        = CORRESPONDING #( travel_out MAPPING TO ENTITY
travel_set_status_booked-%param-travelid = travelid.
APPEND travel_set_status_booked TO result.
ENDIF.

ENDLOOP.
ENDMETHOD.
```

Implementing Dynamic Action Control

For dynamic control of actions acting on individual entity instances, the option (features: instance) must be added to the action set_status_booked in the behavior definition. The required implementation must be provided in the referenced class pool. In the implementation handler of the class pool you can specify the condition on which the action is enabled or disabled.

More on this: [FOR INSTANCE FEATURES, FEATURES \(ABAP Keyword Documentation\)](#)

Related Information

- [Handler Classes](#)
- [Saver Classes](#)
- [Derived Data Types](#)
- [Implicit Response Parameters](#)

Implementing Dynamic Action Control

This topic guides you through the required implementation steps to dynamically enable and disable actions.

Preview

The following figure shows the effect of dynamic control on the action button **Set to Booked**: Since the selected travel instance has a status of **B** (Booked), the action button is disabled.

Travels (4,179)

Set to Booked

Create

Delete

 Travel ID	Agency ID	Customer ID	Starting Date	End Date	Travel Status	
☒ 15	Aussie Travel (70028)	Miguel (466)	Jun 4, 2022	Mar 31, 2023	Booked	>
☐ 16	Not Only By Bike (70050)	Kirk (640)	Jun 4, 2022	Apr 2, 2023	Planned	>

Dynamic Feature Control for an Action

Implementation Steps

1. Defining dynamic feature control in the behavior definition

In the behavior definition, the feature control for the action set_status_booked is defined as follows:

```
define behavior for /DMO/I_Travel_U alias travel
implementation in class /DMO/BP_TRAVEL_U unique
...
```

```

{
  ...

  action ( features : instance ) set_status_booked result [1] $self;
  ...
}

```

2. Adding dynamic feature control for actions to the method **get_features** in the behavior pool

The feature control for actions must be added to the method `get_features` in the behavior pool. You can implement feature control for different actions or operations in the same method.

Depending on the value of `Status` field, the action `set_status_booked` is enabled or disabled.

i *Expand the following code sample to view the source code of the method `get_features`.*

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS:
      ...
      get_features      FOR FEATURES
                        IMPORTING keys REQUEST requested_features FOR travel
                        RESULT result.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  ...

  *****
  *
  * Implements the dynamic action handling for travel instances
  *
  *****

  METHOD get_features.

    READ ENTITIES OF /DMO/I_Travel_U IN LOCAL MODE
      ENTITY Travel
        FIELDS ( TravelID Status )
        WITH CORRESPONDING #( keys )
      RESULT DATA(lt_travel_result)
      FAILED failed.

    result = VALUE #(
      FOR ls_travel IN lt_travel_result (
        %key                                = ls_travel-%key
        %features-%action-set_status_booked = COND #( WHEN ls_travel-Status = 'B'
                                                         THEN if_abap_behv=>fc-o-disabled ELSE if_a
                                                         %assoc-_Booking
                                                         = COND #( WHEN ls_travel-Status = 'B' OR ls_travel-S
                                                         THEN if_abap_behv=>fc-o-disabled ELSE if_a

        ) ).

    ENDMETHOD.

  ENDClass.

```

Related Information

[Feature Control](#)

Creating the Behavior Pool for the Booking Child Entity

Procedure 1: Create a Behavior Pool /DMO/BP_BOOKING_U

To launch the wizard tool for creating a behavior implementation, do the following:

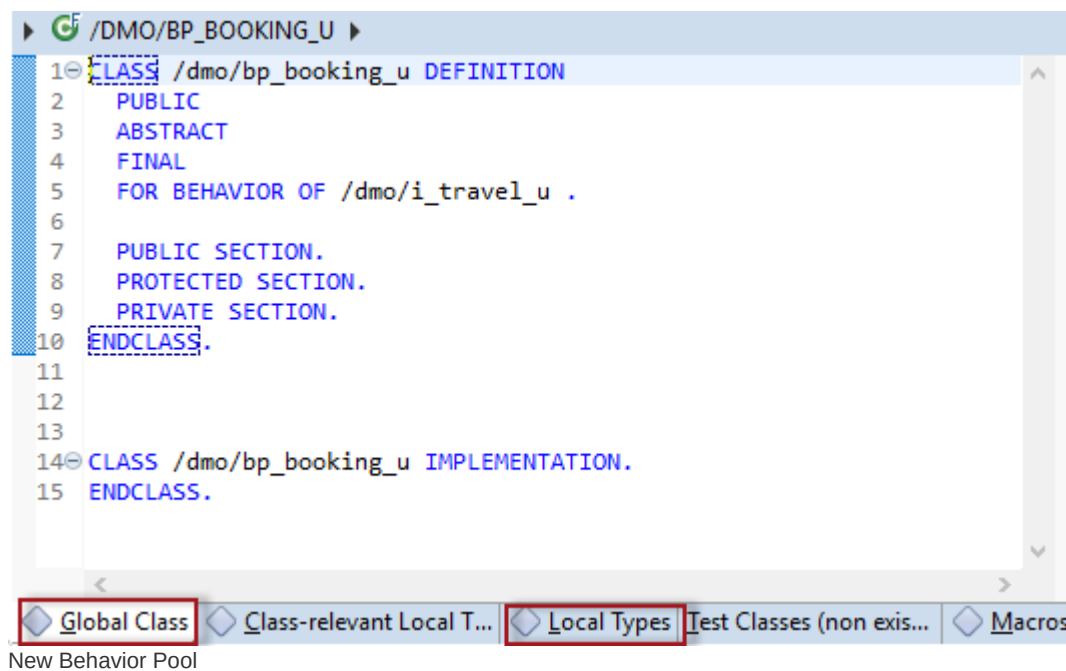
1. Add the following syntax to the behavior definition in the section of the entity behavior for the travel entity.

```
implementation in class /dmo/bp_booking_u unique
```

2. The behavior definition offers a quick to create a behavior pool corresponding to the behavior described in the behavior definition. Follow the quick fix to start the wizard for creating an ABAP class.

Results: Behavior Pool for the Booking Child Entity

The generated behavior pool (in our case /DMO/CL_BOOKING_U) provides you with an extension FOR BEHAVIOR OF.



Procedure 2: Define a Skeleton of Local Classes Corresponding to the Behavior Model

Based on the declarations in the behavior definition /DMO/I_TRAVEL_U, the behavior implementation offers a skeleton of the local classes for the booking entity:

i Expand the following code sample to view the source code of the template for local classes of /dmo/bp_booking_u.

Sample Code

```

CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.

  TYPES tt_booking_failed    TYPE TABLE FOR FAILED    /dmo/i_booking_u.
  TYPES tt_booking_reported TYPE TABLE FOR REPORTED /dmo/i_booking_u.

  TYPES tt_bookingsupplement_failed TYPE TABLE FOR FAILED /dmo/i_bookingsupplement_u.
  TYPES tt_bookingsupplement_reported TYPE TABLE FOR REPORTED /dmo/i_bookingsupplement_u.

  METHODS update FOR MODIFY
    IMPORTING entities FOR UPDATE booking.
  
```



```

METHODS delete FOR MODIFY
  IMPORTING keys FOR DELETE booking.

METHODS read FOR READ
  IMPORTING keys FOR READ booking RESULT result.

METHODS rba_Booksupplement FOR READ
  IMPORTING keys_rba FOR READ booking\_Booksupplement FULL result_requested RESULT result LI

METHODS rba_Travel FOR READ
  IMPORTING keys_rba FOR READ booking\_Travel FULL result_requested RESULT result LINK assoc

METHODS cba_Booksupplement FOR MODIFY
  IMPORTING entities_cba FOR CREATE booking\_Booksupplement.

METHODS map_messages
  IMPORTING
    cid          TYPE string OPTIONAL
    travel_id     TYPE /dmo/travel_id OPTIONAL
    booking_id    TYPE /dmo/booking_id OPTIONAL
    messages      TYPE /dmo/t_message
  EXPORTING
    failed_added TYPE abap_bool
  CHANGING
    failed        TYPE tt_booking_failed
    reported      TYPE tt_booking_reported.

METHODS map_messages_assoc_to_booksupp
  IMPORTING
    cid          TYPE string OPTIONAL
    travel_id     TYPE /dmo/travel_id OPTIONAL
    booking_id    TYPE /dmo/booking_id OPTIONAL
    booking_supplement_id TYPE /dmo/booking_supplement_id OPTIONAL
    is_dependend  TYPE abap_bool DEFAULT abap_false
    messages      TYPE /dmo/t_message
  EXPORTING
    failed_added TYPE abap_bool
  CHANGING
    failed        TYPE tt_bookingsupplement_failed
    reported      TYPE tt_bookingsupplement_reported.

ENDCLASS.

ENDCLASS.

CLASS lhc_booking IMPLEMENTATION.
...
ENDCLASS.

```

Implementing the UPDATE, DELETE, and READ Operations for Booking Instances

This topic guides you through all implementation steps required for data updates and deletion of booking data sets.

The behavior definition in our demo application scenario requires the standard operations `update`, and `delete` for the booking child entity. Note that the `read` operation is not explicitly declared in the behavior definition, but implicitly expected. The `create` is handled in the travel behavior by a `create_by_association`. In addition, since we use different CDS view field names than in the database table, we need a mapping specification from CDS names to database fields names.

```

define behavior for /DMO/I_Booking_U alias Booking
implementation in class /DMO/BP_BOOKING_U unique
etag dependent by _Travel
lock dependent by _Travel
authorization dependent by _Travel

```

```

{
  field ( readonly ) TravelID, BookingID;
  field ( mandatory ) BookingDate, CustomerID, AirlineID, ConnectionID, FlightDate;

  update;
  delete;

  association _BookSupplement { create; }
  association _Travel;

  mapping for /dmo/booking control /dmo/s_booking_intx
  {
    AirlineID      = carrier_id;
    BookingDate    = booking_date;
    BookingID      = booking_id;
    ConnectionID   = connection_id;
    CurrencyCode   = currency_code;
    CustomerID     = customer_id;
    FlightDate     = flight_date;
    FlightPrice    = flight_price;
    TravelID       = travel_id;
  }
}

```

The create operation is already implemented by using the association relation between the `travel` root entity and the booking child entity. **Further information:** [Implementing the CREATE Operation for Associated Bookings](#)

Preview (Update of Booking Data Sets)

In our application scenario, the appropriate booking data sets should be modifiable for all required fields of the booking instance when, for example, the user clicks the **Edit** button on the object page of the Fiori UI.

3



Delete



Travel

Booking

Travel ID:	Booking Fee:
3	80.00 USD
Agency ID:	Total Price:
Hendrik's (70046)	4,164.00 USD
Customer ID:	Comment:
Detemple (93)	Vacation
Starting Date:	Travel Status:
Jun 4, 2022	Booked
End Date:	
Apr 2, 2023	

Edit and Delete Buttons are Available for Each Booking Data Set

In change mode, the end user is able to change the relevant travel fields as shown in the figure below. As soon as the user chooses the **Save** button on the object page, the changed booking data is persisted on the database and a new version of the related booking instance is created.

Travel ID:	3	Booking Fee:	80.00	USD	
Agency ID: *	Hendrik's (70046)	Total Price:	4,164.00	USD	
Customer ID: *	Detemple (93)	Comment:	Vacation		
Starting Date: *	Jun 4, 2022	Travel Status:	Booked		

Save Cancel

Object Page for Editing Individual Booking Values

Implementation Steps

Defining and Implementing UPDATE for Booking Data

Corresponding to the [template](#) for behavior implementation of the booking child entity, one local handler class `lhc_booking` is defined to implement each changing operation in one individual `<method> FOR MODIFY`, one for updating booking data sets and another one for deleting bookings.

The `update_booking FOR MODIFY` method of the handler `lhc_booking` implements the update operation for bookings. The signature of this method includes only one import table parameter `entities` for referring to the booking instances to be updated.

To update data of bookings, the function module `/DMO/FLIGHT_TRAVEL_UPDATE` is called. In addition to the incoming parameters `is_travel` and `it_booking`, the corresponding flag structure `is_travelx` as well as the flag table type `it_bookingx` are used.

When updating booking data sets, we must first check which individual data was changed by the end user. This check is done by mapping the control structure of the importing parameter on a local structure that can be passed to the update function module.

Message handling for processing instance-specific messages in case of failure is implemented by the `handle_booking_messages` method. Failed keys are addressed by the booking content ID (`<booking>-%cid_ref`) and the values for the travel ID (`<booking>-travelid`) and the booking ID (`<booking>-bookingid`). In case of failure, failed keys are saved in the `failed-booking` table, whereas the `reported-booking` table contains all instance-specific messages.

i Expand the following code sample to view the source code of the method `update`.

Sample Code

```
*****
*
* Handler class for managing bookings
*
*****
CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.

METHODS:
  update FOR MODIFY
    IMPORTING entities FOR UPDATE booking,
```

...

ENDCLASS.

CLASS lhc_booking IMPLEMENTATION.

*

* Implements the UPDATE operation for a set of booking instances

*

METHOD update.

DATA messages TYPE /dmo/t_message.

DATA booking TYPE /dmo/booking.

DATA bookingx TYPE /dmo/s_booking_inx.

LOOP AT entities ASSIGNING FIELD-SYMBOL(<booking>).

booking = CORRESPONDING #(<booking> MAPPING FROM ENTITY).

bookingx-booking_id = <booking>-BookingID.

bookingx-_intx = CORRESPONDING #(<booking> MAPPING FROM ENTITY).

bookingx-action_code = /dmo/if_flight_legacy=>action_code-update.

CALL FUNCTION '/DMO/FLIGHT_TRAVEL_UPDATE'

EXPORTING

is_travel = VALUE /dmo/s_travel_in(travel_id = <booking>-travelid)

is_travelx = VALUE /dmo/s_travel_inx(travel_id = <booking>-travelid)

it_booking = VALUE /dmo/t_booking_in((CORRESPONDING #(booking)))

it_bookingx = VALUE /dmo/t_booking_inx((bookingx))

IMPORTING

et_messages = messages.

/dmo/cl_travel_auxiliary=>handle_booking_messages(

EXPORTING

cid = <booking>-%cid_ref

travel_id = <booking>-travelid

booking_id = <booking>-bookingid

messages = messages

CHANGING

failed = failed-booking

reported = reported-booking).

ENDLOOP.

ENDMETHOD.

Adding Data Type Declarations for the Booking Entity in the Auxiliary Class

To use the required import, export, or changing parameters in the signature of methods to be defined for handling booking operations (later on, in the booking behavior pool's **FOR MODIFY** methods), we must first add the appropriate type declarations to the **PUBLIC** section in the definition part of our auxiliary class.

i *Expand the following code sample to view the source code of the auxiliary class /dmo/cl_travel_auxiliary.*

Sample Code

CLASS /dmo/cl_travel_auxiliary DEFINITION

INHERITING FROM cl_abap_behv

PUBLIC

FINAL

CREATE PUBLIC .

PUBLIC SECTION.

* Type definition for import parameters -----

```

TYPES tt_booking_failed    TYPE TABLE FOR FAILED    /dmo/i_booking_u.
TYPES tt_booking_reported  TYPE TABLE FOR REPORTED  /dmo/i_booking_u.

```

```
...
```

```
ENDCLASS.
```

Implementing Message Handling

When handling changing operations for booking instances, fault events may occur. For the processing of instance-specific messages in such a case, the method `handle_booking_messages` is used.

To refer to an individual data set where an error (`msgty = 'E'`) or an abort (`msgty = 'A'`) occurred, the `FAILED` table is used, whereas the instance-specific messages are stored in the `REPORTED` table.

However, messages that originate from the legacy code must be mapped to the messages of the class-based BO framework. To be reused in different behavior pools, the corresponding message handler method is defined and implemented in the helper class `/dmo/cl_travel_auxiliary`.

i *Expand the following code sample to view the source code of the method `handle_booking_messages`.*

Sample Code

```

CLASS /dmo/cl_travel_auxiliary DEFINITION
  INHERITING FROM cl_abap_behv
  PUBLIC
  FINAL
  CREATE PUBLIC .

PUBLIC SECTION.
...

CLASS-METHODS handle_booking_messages
  IMPORTING
    cid          TYPE string OPTIONAL
    travel_id    TYPE /dmo/travel_id OPTIONAL
    booking_id   TYPE /dmo/booking_id OPTIONAL
    messages     TYPE /dmo/t_message
  CHANGING
    failed       TYPE tt_booking_failed
    reported     TYPE tt_booking_reported.

ENDCLASS.

CLASS /dmo/cl_travel_auxiliary IMPLEMENTATION.
...

METHOD handle_booking_messages.

  LOOP AT messages INTO DATA(message) WHERE msgty = 'E' OR msgty = 'A'.
    APPEND VALUE #( %cid          = cid
                    travelid      = travel_id
                    bookingid     = booking_id
                    %fail-cause = get_cause_from_message(
                                iv_msgid = message-msgid
                                iv_msgno = message-msgno
                                ) ) TO failed.

    APPEND VALUE #( %msg = get_message_object( )->new_message(
                                id          = message-msgid
                                number     = message-msgno
                                severity   = if_abap_behv_message=>severity-error
                                v1         = message-msgv1
                                v2         = message-msgv2

```

```

                                v3      = message-msgv3
                                v4      = message-msgv4 )
%key-TravelID = travel_id
%cid         = cid
TravelID     = travel_id
BookingID    = booking_id ) T0 reported.

```

```
ENDLOOP.
```

```
ENDMETHOD.
```

```
...
```

```
ENDCLASS.
```

Defining and Implementing the DELETE operation for Booking Instances

In this case, the FOR MODIFY method is used to implement the delete operation for booking data sets. Therefore, the signature of the <method> FOR MODIFY includes only one import parameter keys for referring to the booking instance to be deleted. To identify the child entity for bookings, the alias **booking** is used - according to the alias specified in the behavior definition.

The basic steps within the method implementation are similar to those we got to know in the previous method update. The function module /DMO/FLIGHT_TRAVEL_UPDATE is also called for the delete operation. Here, as in the previous case, the same incoming parameters are used, including the flag structure is_travelx as well as the flag table it_bookingx). The appropriate action code for deleting bookings is defined by the statement `action_code = /dmo/if_flight_legacy=>action_code-delete`.

i Expand the following code sample to view the source code of the method delete.

Sample Code

```

*****
*
* Handler class for managing bookings
*
*****
CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.

    PRIVATE SECTION.

        METHODS:
            ...
            delete FOR MODIFY
                IMPORTING keys FOR DELETE booking,
            ...

ENDCLASS.

CLASS lhc_booking IMPLEMENTATION.

*****
*
* Implements the DELETE operation for a set of booking instances
*
*****
METHOD delete.
    DATA messages TYPE /dmo/t_message.

    LOOP AT keys ASSIGNING FIELD-SYMBOL(<booking>).

        CALL FUNCTION '/DMO/FLIGHT_TRAVEL_UPDATE'
            EXPORTING
                is_travel    = VALUE /dmo/s_travel_in( travel_id = <booking>-travelid )
                is_travelx   = VALUE /dmo/s_travel_inx( travel_id = <booking>-travelid )
                it_booking   = VALUE /dmo/t_booking_in( ( booking_id = <booking>-bookingid ) )
                it_bookingx  = VALUE /dmo/t_booking_inx( ( booking_id = <booking>-bookingid

```

```

IMPORTING
  et_messages = messages.

map_messages(
  EXPORTING
    cid          = <booking>-%cid_ref
    travel_id    = <booking>-travelid
    booking_id   = <booking>-bookingid
    messages     = messages
  CHANGING
    failed       = failed-booking
    reported     = reported-booking ).
ENDLOOP.
ENDMETHOD.

```

Defining and Implementing the READ operation for Booking Instances

The READ operation is not specified in the behavior definition. To complete the BO, it is expected that the READ for all entities is available to consume the BO via EML.

i Note

If you use groups in your behavior definition, you must explicitly specify the READ operation in one of the groups. For more information, see [Using Groups in Large Development Projects](#).

In the handler class for booking entities add the declaration of the method `read_booking` FOR READ with importing parameter `keys` and the result parameter `result`.

For more information, see [<method> FOR READ](#).

The implementation steps for reading booking entities are similar to those for reading travel entities, see [Implementing the READ Operation for Travel Data](#). The same function module is used. It reads the booking instances based on the travel ID that is passed to the function module. For performance reasons, we only call the function module once for all bookings with the same travel ID. That is why, the loop at the importing parameter is grouped by the travel ID. As the function module retrieves all available bookings for the travel ID, the output table must be read for the bookings that match the importing parameter `keys`. The read results are then be passed to the result parameter `result` for those fields that are requested by the consumer. The key values are always passed to the result.

The error handling is separated for booking IDs that are not found for the respective travel IDs, for travel IDs that are not found by the function module, and for other fail causes that cannot be specified.

i Expand the following code sample to view the source code of the method `read`.

≡ Sample Code

```

*****
*
* Handler class for managing bookings
*
*****

CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    ...
    METHODS read FOR READ
      IMPORTING keys FOR READ booking RESULT result.
ENDCLASS.

*****

```

* Implements the READ operation for a set of booking instances

CLASS lhc_booking IMPLEMENTATION.

METHOD read.

DATA: travel_out TYPE /dmo/travel,
bookings_out TYPE /dmo/t_booking,
messages TYPE /dmo/t_message.

"Only one function call for each requested travelid

LOOP AT keys ASSIGNING FIELD-SYMBOL(<booking_by_travel>)
GROUP BY <booking_by_travel>-travelid .

CALL FUNCTION '/DMO/FLIGHT_TRAVEL_READ'

EXPORTING

iv_travel_id = <booking_by_travel>-travelid

IMPORTING

es_travel = travel_out

et_booking = bookings_out

et_messages = messages.

map_messages(

EXPORTING

travel_id = <booking_by_travel>-travelid

booking_id = <booking_by_travel>-bookingid

messages = messages

IMPORTING

failed_added = DATA(failed_added)

CHANGING

failed = failed-booking

reported = reported-booking).

IF failed_added = abap_false.

"For each travelID find the requested bookings

LOOP AT GROUP <booking_by_travel> ASSIGNING FIELD-SYMBOL(<booking>)
GROUP BY <booking>-%tky.

READ TABLE bookings_out INTO DATA(booking_out) WITH KEY travel_id = <booking>-%key-Tr
booking_id = <booking>-%key-Bo

"if read was successful

IF sy-subrc = 0.

INSERT CORRESPONDING #(booking_out MAPPING TO ENTITY) INTO TABLE result.

ELSE.

"BookingID not found

INSERT VALUE #(travelid = <booking>-TravelID

bookingid = <booking>-BookingID

%fail-cause = if_abap_behv=>cause-not_found)

INTO TABLE failed-booking.

ENDIF.

ENDLOOP.

ENDIF.

ENDLOOP.

ENDMETHOD.

Defining and Implementing the READ Operation for the Associated Travel Instance

The read-by-association from booking instances to their parent travel instance is used to read the ETag master of the booking instances. That is why, it is essential to support complete ETag handling for the travel BO. In addition, the read-by-association must be implemented to complete the transactional handling for the BO, so that it is consumable via EML.

In order to use the association transactionally, in this case, via a transactional read, the association must be specified in the behavior definition for the booking instance:

```
define behavior for /DMO/I_Booking_U alias booking
{
```



```

...
association _Travel;
...
}

```

In the handler class for booking entities, add the declaration of the method `rba_travel FOR READ booking\_travel` with importing parameter `keys_rba`, and `result_requested` for full consumption and declare, `RESULT result` and `LINK association_links` as changing parameters.

For more information, see [<method> FOR READ By Association](#).

In the implementation for reading travel instances by association from booking entities, loop at the incoming booking and group by the incoming `TravelID`. This grouping optimizes the performance as the function call is only executed once for each travel ID. If the function call to read the associated travel instance is successful, group at the group `<travel>` to fill the link table for each requested booking. If all fields of the travel instance are requested, fill the result parameter `result` with the values that are read by the function module. To ensure that only those fields are filled that were requested (indicated by the `%control` structure of the importing parameter), the corresponding operator with the addition `MAPPING TO ENTITY` is used.

To avoid duplicate result keys, delete duplicates from the link table and `result`.

If the function modules returns messages, fill the `failed` table with the corresponding entries. The error handling is separated for booking IDs that are not found for the respective travel IDs, for travel IDs that are not found by the function module, and for other fail causes that cannot be specified.

i Expand the following code sample to view the source code of the method `rba_travel`.

Sample Code

```

*****
*
* Handler class for managing bookings
*
*****

CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    ...
    METHODS rba_Travel FOR READ
      IMPORTING keys_rba FOR READ booking\_Travel FULL result_requested RESULT result LINK assoc
ENDCLASS.

CLASS lhc_booking IMPLEMENTATION.

  ...
  *****
  *
  * Read travel by association
  *
  *****
  METHOD rba_Travel.
    DATA: travel TYPE /dmo/travel,
          messages TYPE /dmo/t_message.

    "Only one function call for each requested travelid
    LOOP AT keys_rba ASSIGNING FIELD-SYMBOL(<booking_by_travel>)
      GROUP BY <booking_by_travel>-travelid .

      CALL FUNCTION '/DMO/FLIGHT_TRAVEL_READ'
        EXPORTING
          iv_travel_id = <booking_by_travel>-travelid
        IMPORTING

```

```

        es_travel      = travel
        et_messages    = messages.

map_messages(
  EXPORTING
    travel_id = <booking_by_travel>-travelid
    booking_id = <booking_by_travel>-bookingid
    messages  = messages
  IMPORTING
    failed_added = DATA(failed_added)
  CHANGING
    failed      = failed-booking
    reported    = reported-booking ).

IF failed_added = abap_false.
  LOOP AT keys_rba ASSIGNING FIELD-SYMBOL(<travel>) USING KEY entity WHERE TravelID = <boo
    INSERT VALUE #(
      source-%tky      = <travel>-%tky
      target-travelid = <travel>-TravelID
    ) INTO TABLE association_links.

    IF result_requested = abap_true.
      APPEND CORRESPONDING #( travel MAPPING TO ENTITY ) TO result.
    ENDIF.
  ENDLOOP.
ENDIF.

ENDLOOP.

SORT association_links BY source ASCENDING.
DELETE ADJACENT DUPLICATES FROM association_links COMPARING ALL FIELDS.

SORT result BY %tky ASCENDING.
DELETE ADJACENT DUPLICATES FROM result COMPARING ALL FIELDS.
ENDMETHOD.

```

Checking Results

At this point, you have the opportunity to check how the resulting app works for the CREATE and the UPDATE. For this to happen, however, a suitable business service for UI consumption must first be defined and published.

For more information, see: [Defining Business Service for Fiori UI](#)

The READ cannot be easily tested using a Fiori UI, as there is no direct execution trigger for reading the data from the application buffer. Nevertheless, you can test your READ implementation by using EML.

❁ Example

```

READ ENTITY /dmo/i_booking_U
FIELDS ( TravelID
         BookingID
         CustomerID
         BookingDate )
WITH VALUE #( ( %key = VALUE #( TravelID = travelid
                                BookingID = bookingid ) ) )
RESULT DATA(read_result_travels)
FAILED DATA(failed_rba).

```

To retrieve the complete entity instance for the respective travel ID, you have to set every element explicitly. For more information about EML, see [Examples for Accessing Business Objects with EML](#).

Projecting the Behavior

You project the behavior to define the behavior for the specific business service.

Whereas the general business object defines and implements the behavior of what can be done in general with the data provided by the data model, the BO projection defines only the behavior that is relevant for the specific service. In this travel scenario, the BO projection does not define a specific role of the end user, but projects the entire behavior for the UI service.

In the behavior projection, you only define the behavior that is relevant for the specific service. The implementation of the individual characteristics is only done in the general BO. The projection behavior definition delegates to the underlying layer for the behavior that is defined in the projection layer.

For more information, see [Business Object Projection](#).

Creating the Projection Behavior Definition for the Travel BO

To create a projection behavior definition, do the following:

1. Open the context menu of the root projection view `/DMO/C_Travel_U`.
2. Choose **New Behavior Definition**.
3. In the wizard to create a new behavior definition, choose the implementation type **Projection** to get the template for projection.

A new projection behavior definition is created that defines the behavior for the specific UI service. The behavior definition uses the same name as the root projection view.

For the UI service, the entire behavior that is defined in the underlying BO is exposed. This includes:

- the `etag` to prevent overwriting on simultaneous editing
- the standard operations `create`, `update`, `delete` for the root entity,
- the standard operations `update` and `delete` for the child entity,
- the operation `create by association` for the child entity,
- the action `set_status_booked` on the root entity.

The following listing displays the source code of the projection behavior definition.

i *Expand the following code sample to view the source code of the projection behavior definition `/DMO/C_Travel_U`.*

Sample Code

```
projection;
strict(2);

define behavior for /DMO/C_Travel_U alias travel

use etag

{
  use create;
  use update;
  use delete;
  use action set_status_booked;
  use association _BOOKING { create; }
}

define behavior for /DMO/C_Booking_U alias booking
use etag
```

```

{
  use update;
  use delete;
  use association _BOOKSUPPLEMENT { create; }
  use association _Travel;
}

define behavior for /DMO/C_BookingSupplement_U alias bookingsupplement
use etag
{
  use update;
  use delete;

  use association _Travel;
  use association _Booking;
}

```

For more information, see [Projection Behavior Definition](#).

Defining Business Service for Fiori UI

This section explains how you can model an OData service based on the data model and the related behavior model. A service like this consists of two artifacts, a service definition and a service binding.

The **service definition** is a projection of the data model and the related behavior to be exposed, whereas the **service binding** implements a specific protocol and the kind of service offered to a consumer.

Further information: [Business Service](#)

Steps Relevant to Developers

1. Create the service definition
2. Specify which CDS entities are exposed as a UI service
3. Create a service binding
4. Publish the UI service locally
5. [Optional] Run the resulting app

Exposing the Relevant CDS Views as a Service

To describe the consumer-specific perspective as a data model, you need to create a business service definition (service definition for short) as an ABAP Repository object. A service definition represents the service model that is derived from the underlying CDS-based data model.

Further information: [Service Definition](#)

Create and Define the Service Definition

Create a service definition and add all relevant data model artifacts to be exposed by the service.

i *Expand the following code sample to view the source code of the service definition /DMO/TRAVEL_U.*

≡, Sample Code

```
@EndUserText.label: 'Service Definition for Managing Travels'
@ObjectModel.leadingEntity.name: '/DMO/I_Travel_U'
define service /DMO/TRAVEL_U {
    expose /DMO/C_Travel_U as Travel;
    expose /DMO/C_Booking_U as Booking;
    expose /DMO/C_BookingSupplement_U as BookingSupplement;
    expose /DMO/I_Supplement as Supplement;
    expose /DMO/I_SupplementText as SupplementText;
    expose /DMO/I_Customer as Passenger;
    expose /DMO/I_Agency as TravelAgency;
    expose I_Currency as Currency;
    expose I_Country as Country;
    expose /DMO/I_Carrier as Airline;
    expose /DMO/I_Connection as FlightConnection;
    expose /DMO/I_Flight as Flight;
    expose /DMO/I_Airport as Airport;
    expose /DMO/I_Travel_Status_VH as TravelStatusVH;
}
```

The service definition contains all the related CDS entities which are to be exposed as part of the UI service - including their compositions and associations with the relevant entities. Note that the value help provider or text provider views must also be exposed for the OData service to make use of the value help and text associations. The main entity, the root entity of the business object is marked by the annotation `@ObjectModel.leadingEntity.name`. In the service binding this entity appears first in the entity list.

Further information: [Syntax for Defining a Service](#)

Creating a Service Binding

Using the business service binding (service binding for short), you can bind a service definition to a client-server communication protocol.

Further information: [Service Binding](#) (Concept Information)

i Note

It's recommended to use OData V4 instead of OData V2 for future-proof design.

Procedure: Creating a Service Binding

To launch the wizard tool for creating a service binding, do the following:

1. Launch [ABAP Development Tools](#).
2. In your ABAP project (or [ABAP Cloud Project](#)), select the relevant package node in [Project Explorer](#).
3. Open the context menu and choose [New Service Binding](#) to launch the creation wizard.

Procedure: Activating the New Service

After successful activation, the editor provides additional information about the entire entity set as well as about the navigation path of the respective entity.

General Information

This section describes general information about this service binding

Binding Type: ODATA V2 (UI - User Interface: Consumed in SAPUI5 Apps)

Service Versions

View information on the service versions associated with the Service Binding

[illegible]

Service Binding Editor

[View information on selected service version](#)

Local service endpoint: Active

Local Service Endpoint Information

Service URL: /sap/opu/odata/DMO/UI_TRAVEL_U_V2

Preview

Type filter text

Entity Set and Association

▼ TravelAgency

```
graph TD; A(( )) --> B[to_Country];
```

> BookingSupplement

> Booking

 Airline

 FlightConnection

> Passenger

 Flight

> Supplement

 SupplementText

▼ Travel

to_Agency

- to Booking

to Op

to Customer

[Open Fiori Elements App Preview](#)

[Optional] Procedure: Running the Resulting UI Service

As soon as the service is activated, it is ready for consumption through an OData client such as an SAP Fiori app.

In the course of the UI development in the SAP Web IDE, you have the option of testing the resulting app within the SAP Fiori launchpad environment.

→ **Tip**

Alternatively, you can use the preview function in the service binding to check how the UI of a Fiori application looks like.

Adding Another Layer to the Transactional Data Model

In this section, you get a brief overview on how you can proceed when going to add a further layer to the flight demo scenario. Specifically, a booking supplement entity is now to be added to the previous 2-tier layer of the travel business object.

Entities for Transactional Data

As depicted in the figure below, the 3-tier entity hierarchy for managing transactional data in our scenario consists of the following editable entities, with a 1: n cardinality on each level:

- Travel

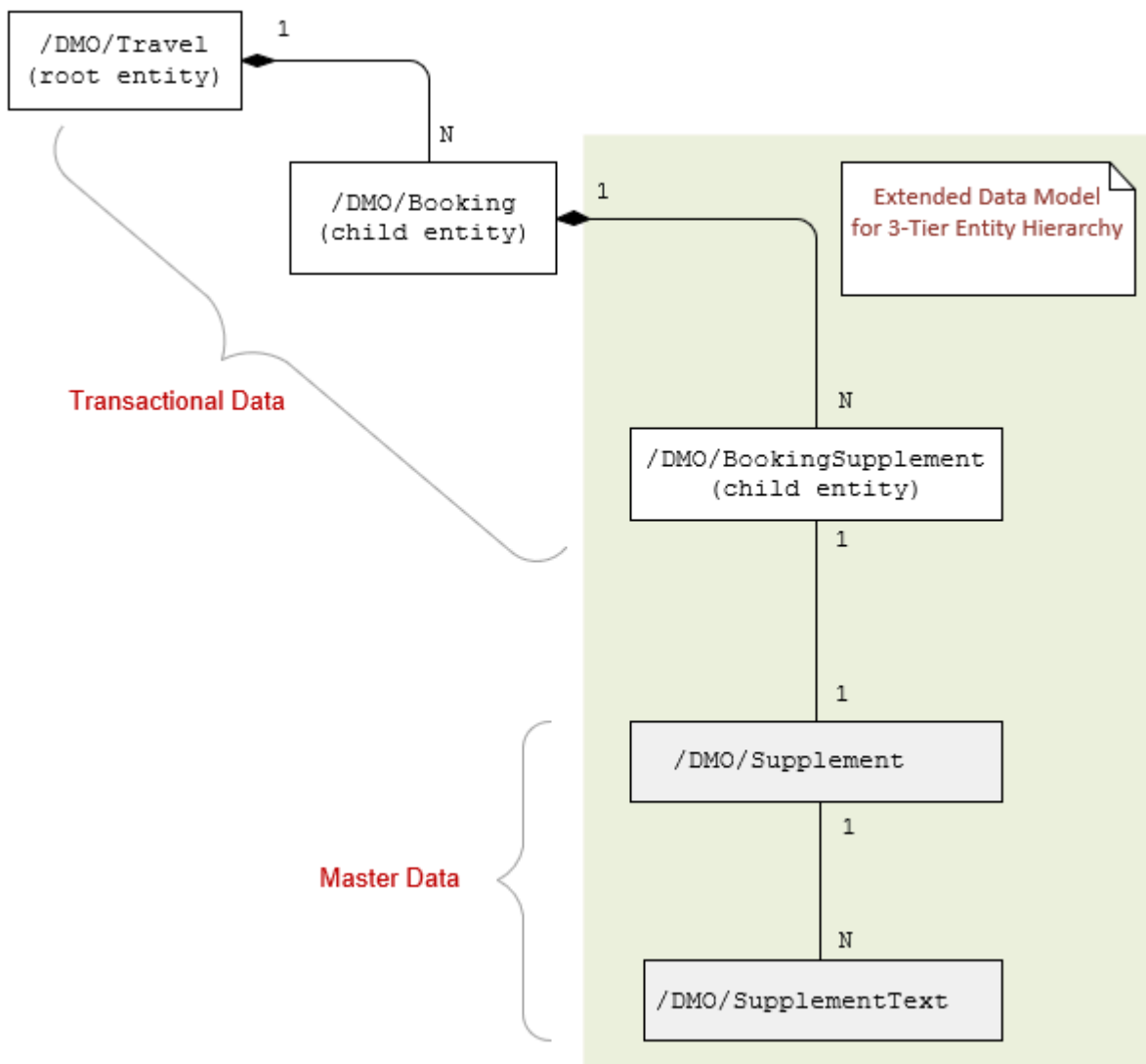
- Booking
- BookingSupplement

That is, each travel instance has 0..n bookings and each booking has 0..n booking supplements.

Additional Entities for Master Data

To populate the booking supplement instances with business data, you also need to extend the data model for master data by the following entities:

- Supplement
- SupplementText



Data Model for 3-tier Entity Hierarchy

i Note

With the knowledge so far, you can easily reproduce the concrete steps of this extension. Therefore, in this topic, we will only outline the implementation steps in a nutshell and refer to the full implementation as it is available in the demo ABAP package **/DMO/FLIGHT_UNMANAGED**.

1. Adding a New BO Node

Adding the CDS Entities to Extend the Data Model

Extended Data Model

Entity	Description	Editable
BookingSupplement	This entity is used to add additional products to a travel booking. The booking supplement data is stored in the database table /DMO/BOOK_SUPPL. The flight data model defines an n:1 cardinality between a Booking Supplement entity and a Booking entity.	Yes
Supplement	A Supplement entity defines product data that the customer can book together with a flight, for example a drink or a meal. The supplement data is stored in the database table /DMO/SUPPLEMENT. The flight data model defines a 1:1 cardinality between a Supplement entity and the Booking Supplement entity.	No
SupplementText	This entity mainly serves a text provider for the associated elements in the target entity BookingSupplement. By using a text association, you define the relationship between an element of the target entity and its corresponding texts or descriptions. More on this: Getting Text Through Text Associations	NO

The BookingSupplement entity is part of the compositional hierarchy of the travel business object. This composition relationship requires an association to their compositional parent entity. This relationship is expressed by the keyword **ASSOCIATION TO PARENT**. Using this syntax, you define the CDS entity BookingSupplement as a sub node in the compositional hierarchy of the travel business object structure. For etag and lock master handling, the entity contains the association **_Travel** to the root entity. To access master data from other entities, additional associations **_Product** and **_SupplementText** are defined in the CDS source code. The associated views are primarily used as a text view for retrieving text information and as value help provider for specific product fields.

i Expand the following code sample to view the source code of the CDS View /DMO/I_BookingSupplement_U.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Booking Supplement view - CDS data model'

define view entity /DMO/I_BookingSupplement_U
  as select from /dmo/book_suppl as BookingSupplement

  association          to parent /DMO/I_Booking_U as _Booking          on $projection.TravelID = _
                        and $projection.BookingID = _
  association [1..1] to /DMO/I_Travel_U          as _Travel          on $projection.TravelID = _

  association [1..1] to /DMO/I_Supplement          as _Product          on $projection.SupplementID
  association [1..*] to /DMO/I_SupplementText      as _SupplementText on $projection.SupplementID

{
  key BookingSupplement.travel_id          as TravelID,
  key BookingSupplement.booking_id         as BookingID,
  key BookingSupplement.booking_supplement_id as BookingSupplementID,
  BookingSupplement.supplement_id          as SupplementID,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  BookingSupplement.price                  as Price,
  BookingSupplement.currency_code          as CurrencyCode,
  //      _Booking.LastChangedAt            as LastChangedAt,
  /* Associations */
```



```

    _Booking,
    _Travel,
    _Product,
    _SupplementText
}

```

Since the `BookingSupplement` entity is part of the compositional hierarchy of the business object, it is projected in a projection view. In the projection view, the text relationships, the search and value helps are defined via CDS annotations. In addition, the associations to parent and root must be redirected respectively.

i Expand the following code sample to view the source code CDS Projection View `/DMO/C_BookingSupplement_U`.

Sample Code

```

@EndUserText.label: 'Booking Supplement Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true

@Search.searchable: true
define view entity /DMO/C_BookingSupplement_U
  as projection on /DMO/I_BookingSupplement_U

{
    ///DMO/I_BookingSupplement_U
    @Search.defaultSearchElement: true
    key TravelID,

    @Search.defaultSearchElement: true
    key BookingID,

    key BookingSupplementID,

    @Consumption.valueHelpDefinition: [ {entity: { name:    '/DMO/I_SUPPLEMENT',
                                                    element: 'SupplementID' }},
                                         {localElement: 'Price',
                                          {localElement: 'CurrencyCode',

@ObjectModel.text.element: ['SupplementText']
SupplementID,
_SupplementText.Description as SupplementText : localized,

Price,

@Consumption.valueHelpDefinition: [ { entity: { name:    'I_Currency',
                                                    element: 'Currency' } } ]

CurrencyCode,

//      LastChangedAt,

/* Associations */
///DMO/I_BookingSupplement_U
Booking : redirected to parent /DMO/C_Booking_U,
Travel : redirected to /DMO/C_Travel_U ,
Product,
_SupplementText

}

```

The `BookingSupplement` Entity also has a representation in the UI. It must therefore be equipped with UI annotations, which are outsourced to metadata extension as done with the other BO entities.

i Expand the following code sample to view the source code of the Metadata Extension `/DMO/C_BookingSupplement_U`.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Booking Supplement',

```

```

        typeNamePlural: 'Booking Supplements',
        title: { type: #STANDARD, label: 'Booking Supplement', value: 'BookingSuppl

annotate view /DMO/C_BookingSupplement_U with
{
    @UI.facet: [ { id:          'BookingSupplement',
                    purpose:    #STANDARD,
                    type:       #IDENTIFICATION_REFERENCE,
                    label:      'Booking Supplement',
                    position:    10 } ,
                { id:          'PriceHeader',
                    type:       #DATAPOINT_REFERENCE,
                    purpose:    #HEADER,
                    targetQualifier: 'Price',
                    label:      'Price',
                    position:    20 } ]

    @UI: { lineItem:      [ { position: 10, importance: #HIGH } ],
          identification: [ { position: 10 } ] }
    BookingSupplementID;

    @UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
          identification: [ { position: 20 } ] }
    SupplementID;

    @UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
          identification: [ { position: 30 } ],
          dataPoint:      { title: 'Price' }
          }
    Price;
}

```

i Note

To complete the compositional hierarchy of the business object, you need to add the composition to child in the **Booking** entity **/DMO/I_Booking_U**, as well as the redirected composition in the projection view **/DMO/C_Booking_U**.

In order to display and change booking supplement data on Fiori UI, you must add the corresponding UI facet in the booking metadata extension **/DMO/C_Booking_U**.

Adding a CDS View for Supplements

The source code below provides you with a data definition for additional products that a customer can book together with a flight booking.

i Expand the following code sample to view the source code of CDS View **/DMO/I_Supplement.**

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'supplement view - cds data model'

@Search.searchable: true

define root view entity /DMO/I_Supplement
as select from /dmo/supplement as supplement

    composition [0..*] of /DMO/I_SupplementText as _SupplementText
    association [0..1] to I_Currency as _Currency on $projection.Curre
    association [0..1] to /DMO/I_SupplementCategory_VH as _SupplementCategory on $projection.Suppl

{
    @Search.defaultSearchElement: true
    key supplement.supplement_id as SupplementID,

```

```

    supplement.supplement_category    as SupplementCategory,

    @Search.defaultSearchElement: true
    @Semantics.amount.currencyCode: 'currencycode'
    supplement.price                  as Price,

    supplement.currency_code          as CurrencyCode,

    @Semantics.user.createdBy: true
    supplement.local_created_by       as LocalCreatedBy,
    @Semantics.systemDateTime.createdAt: true
    supplement.local_created_at       as LocalCreatedAt,
    @Semantics.user.localInstanceLastChangedBy: true
    supplement.local_last_changed_by as LocalLastChangedBy,
    //local etag field --> odata etag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    supplement.local_last_changed_at as LocalLastChangedAt,

    //total etag field
    @Semantics.systemDateTime.lastChangedAt: true
    supplement.last_changed_at        as LastChangedAt,

    /* associations */
    @Search.defaultSearchElement: true
    _SupplementText,
    _SupplementCategory,
    _Currency
}

```

Adding a Text Provider View for Supplements

The source code below provides you with a data definition that serves as text provider. It provides text data through text associations as defined in the booking supplement view .

i *Expand the following code sample to view the source code of CDS view /DMO/I_Supplement.*

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Supplement View - CDS Data Model'

@Search.searchable: true

define view entity /DMO/I_Supplement
as select from /dmo/supplement as Supplement

association [0..*] to /DMO/I_SupplementText as _SupplText on $projection.SupplementID = _SupplID
association [0..1] to I_Currency as _Currency on $projection.CurrencyCode = _CurrencyCode

{
    @ObjectModel.text.association: '_SupplText'
    key Supplement.supplement_id as SupplementID,

    @Semantics.amount.currencyCode: 'CurrencyCode'
    Supplement.price as Price,

    Supplement.currency_code as CurrencyCode,

    /* Associations */
    @Search.defaultSearchElement: true
    _SupplText,
    _Currency
}

```

2. Extending the Behavior Definition for the BookingSupplement Entity

The fact that in our scenario new instances of the booking supplement entity can only be created for a specific travel and booking instance is considered by the addition of the `_BookSupplement` association. The keyword `{create;}` declares that this association is create-enabled what exactly means that instances of the associated booking supplements can be created by an individual booking instance.

The sub node of travel business object refers to the corresponding data model for booking supplements that is represented by the child entity for `/DMO/I_BookingSupplement_U`. The transactional behavior of the booking supplement sub node is determined by the standard operations `create`, `update`, and `delete`. In addition, since we use different CDS view field names than in the database table, we need a mapping specification from CDS names to database fields names.

i Expand the following code sample to view the source code of Extending the Behavior Definition `/DMO/I_TRAVEL_U`.

Sample Code

```
...
define behavior for /DMO/I_Travel_U alias travel
{...
}

define behavior for /DMO/I_Booking_U alias booking
{
...
  association _BookSupplement { create; }
}

define behavior for /DMO/I_BOOKINGSUPPLEMENT_U alias bookingsupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_U unique
etag dependent by _Travel
lock dependent by _Travel
{

  field (read only) TravelID, BookingID, BookingSupplementID;
  field (mandatory) SupplementID, Price;

  create;
  update;
  delete;

  association _Travel;

  mapping for /dmo/book_suppl control /dmo/s_booking_supplement_intx
  {
    BookingID          = booking_id;
    BookingSupplementID = booking_supplement_id;
    CurrencyCode       = currency_code;
    Price              = price;
    SupplementID       = supplement_id;
    TravelID           = travel_id;
  }
}
```

3. Creating and Implementing the Behavior Pool for BookingSupplement

Creating a Behavior Pool for BookingSupplement Entity

Behavior Pool for **BookingSupplement** Child Entity

```
CLASS /dmo/bp_bookingsupplement_u DEFINITION
PUBLIC
ABSTRACT
FINAL
FOR BEHAVIOR OF /dmo/i_travel_u.

PUBLIC SECTION.
```

```

PROTECTED SECTION.
PRIVATE SECTION.
ENDCLASS.

```

```

CLASS /dmo/bp_bookingsupplement_u IMPLEMENTATION.
ENDCLASS.

```

Implementing the Handler for UPDATE, DELETE, READ and READ BY ASSOCIATION

You will find the same basic structure when implementing the handler methods for update and delete:

- A loop on booking supplement instances to be updated or deleted.
- Call of the business logic function module.
- Error handling for processing messages in case of failure.

Updating data of booking supplements in `lhc_supplement`

```

CLASS lhc_supplement DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
  TYPES:
    tt_bookingsupplement_update TYPE TABLE FOR UPDATE      /dmo/i_bookingsupplement_u.
    update FOR MODIFY
    IMPORTING entities FOR UPDATE bookingsupplement,
    ...
ENDCLASS.

CLASS lhc_supplement IMPLEMENTATION.
  METHOD update.
    ...
  ENDMETHOD.
ENDCLASS.

```

Deleting booking supplements in `lhc_supplement`

```

CLASS lhc_supplement DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
  ...
  delete FOR MODIFY
  IMPORTING entities FOR DELETE bookingsupplement,
ENDCLASS.

CLASS lhc_supplement IMPLEMENTATION.
  ...
  METHOD delete.
    ...
  ENDMETHOD.
ENDCLASS.

```

Reading booking supplements in `lhc_supplement`

```
CLASS lhc_supplement DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```
PRIVATE SECTION.
```

```
...
```

```
METHODS:
```

```
read FOR READ
```

```
IMPORTING keys FOR READ bookingsupplement
```

```
RESULT result,
```

```
ENDCLASS.
```

```
CLASS lhc_supplement IMPLEMENTATION.
```

```
...
```

```
METHOD read.
```

```
...
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Reading travel by association in **lhc_supplement**

```
CLASS lhc_supplement DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```
PRIVATE SECTION.
```

```
...
```

```
METHODS:
```

```
rba_travel FOR READ
```

```
IMPORTING keys_rba FOR READ bookingsupplement\_travel FULL result_requested
```

```
RESULT result LINK association_links.
```

```
ENDCLASS.
```

```
CLASS lhc_supplement IMPLEMENTATION.
```

```
...
```

```
METHOD rba_travel.
```

```
...
```

```
ENDMETHOD.
```

```
ENDCLASS.
```

Implementing the CREATE Handler for Associated Booking Supplements

In our demo scenario, we assume that new booking supplements cannot be created separately but only in conjunction with a given booking that, in turn, belongs to an individual travel instance.

To identify the associated booking supplements, the aliases for the parent entity and the child entity are used - according to the aliases specified in the behavior definition. The association is expressed in the signature of the implementing method `cba_supplement` of the booking behavior pool `/DMO/BP_BOOKING_U` in the form: ... `FOR CREATE parent\_child_entity`.

Creating booking supplements in the **cba_supplement** method

```
CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.
```

```
PRIVATE SECTION.
```

```
METHODS:
  cba_booksupplement FOR MODIFY
    IMPORTING entities FOR CREATE booking\_booksupplement,
    ...
ENDCLASS.

CLASS lhc_booking IMPLEMENTATION.
  ...

  METHOD cba_booksupplement
    ...
  ENDMETHOD.
ENDCLASS.
```

Booking Supplement

Book. Supp. Number:

—

Product ID: *

Nice Salad (ML-0032)

Product Price: *

4.50

EUR

Create

Cancel

Preview – Object Page when Creating Booking Supplements

3 /

1

Booking Supplements (7) | Standard * ▾

Edit

Delete

^

▾

Create

Delete

▾

☐	Book. Supp. Number	Product ID	Product Price	
☐	1	Chocolate Cake (ML-0035)	2.00 EUR	>
☐	2	Trout Au Bleu (ML-0015)	21.00 EUR	>
☐	3	Paris Salad (ML-0031)	9.00 EUR	>
☐	4	Apple Juice (BV-0005)	3.50 EUR	>
☐	5	Turkey Steak (ML-0019)	12.00 EUR	>
☐	6	Nice Salad (ML-0032)	8.00 EUR	>
☒	7	Nice Salad (ML-0032)	4.50 EUR	>

Preview - Saved Data Displayed in the List of Items

Implementing the READ for Associated Booking Supplements

To complete the BO, you must implement a READ operation for associated bookings. The READ by association is implicitly defined in the behavior definition in the behavior for bookings by the definition of CREATE by association:

```
define behavior for /DMO/I_Booking_U alias booking
{
...
    association _BookSupplement { create; }
}
```

The declaration and the implementation of the method to read booking supplements by association is done in the handler class for bookings /DMO/BP_BOOKING_U in the same manner as in [Implementing the READ Operation for Associated Bookings](#):

- Group the imported booking keys by travel ID
- Call the function module /DMO/FLIGHT_TRAVEL_READ
- Filter for the requested booking keys
- Fill the link table with the respective source and target keys
- Fill the result parameter, if the full parameter is set
- Handle errors

Read booking supplements in the read_supplement_ba method

```
CLASS lhc_booking DEFINITION INHERITING FROM cl_abap_behavior_handler.
PRIVATE SECTION.
METHODS:
    rba_booksupplement FOR READ
        IMPORTING keys_rba FOR READ booking\_booksupplement FULL result_requested
        RESULT result LINK association_links.
...
ENDCLASS.

CLASS lhc_booking IMPLEMENTATION.
...
METHOD rba_booksupplement
...
ENDMETHOD.
ENDCLASS.
```

The READ by association can be tested by using EML.

4. Projecting the Behavior for the Booking Supplement Entity

As described in [Projecting the Behavior](#), the behavior of the travel BO must be projected in the projection behavior definition /DMO/C_Travel_U.

The behavior that is relevant for the booking supplement entity is

- the standard operation update and delete on the behavior node /DMO/C_BookingSupplement_U and

- the create-by-association operation for booking supplements on the behavior node `/DMO/C_Booking_U`.

i Expand the following code sample to view the source code of Projection Behavior Definition `/DMO/C_Travel_U`.

≡ Sample Code

```
...
define behavior for /DMO/C_Travel_U alias travel
use etag
{
  use create;
  use update;
  use delete;
  use action set_status_booked;
  use association _BOOKING { create; }
}

define behavior for /DMO/C_Booking_U alias booking
use etag
{
  use update;
  use delete;
  use association _BOOKSUPPLEMENT { create; }
}

define behavior for /DMO/C_BookingSupplement_U alias bookingsupplement
use etag
{
  use update;
  use delete;
}
```

5. Exposing New Entities in the Service Definition

Corresponding to the listing below, the following CDS entities are to be exposed with the UI service:

Listing 12: Added Entities in the Service Definition

```
@EndUserText.label: 'Service definition for managing travels'
define service /DMO/TRAVEL_U {
  ...
  expose /DMO/C_BookingSupplement_U as BookingSupplement;
  expose /DMO/I_Supplement as Supplement;
  expose /DMO/I_SupplementText as SupplementText;
  ...
}
```

Developing Transactional Apps with Exclusive Draft Capabilities

This chapter is a walk-through tutorial that guides you through the main implementation steps to develop an application with exclusive draft support. Draft-enabled applications provide the end user with the highest flexibility when working with the app.

From a technical perspective, the outcome of these chapters is a business service that is developed based on the implementation type **managed with draft** of the ABAP RESTful Application Programming Model using UUID keys. For more information about draft business objects, see [Draft](#). The business object (BO) is exposed as a UI service. In this example scenario, the Fiori Elements App Preview is used to illustrate the examples. Yet, the same business object can also be consumed as a Web API.

Introduction

The standard use case for creating a managed transactional app with draft capabilities is the following:

You want to create a completely new transactional app for a Fiori Elements UI following the greenfield approach. That means, there is no legacy business application logic involved. You create everything from scratch. Following from that, you can benefit from the lightweight BO standard implementation of the managed implementation type, in which the RAP managed runtime framework assumes an extensive part of the standard implementation with out-of-the-box support for create, update, and delete. In addition, you may need to provide the end user with capabilities to store changed data in the backend at any time and proceed at a later point in time, or to recover such data from a different device, even if the application client has terminated unexpectedly. This use case is supported by the draft concept of the ABAP RESTful Application Programming Model. Draft-enabled business objects persist the state of the transactional buffer after every transaction on a designated draft database table. This allows the end user to stop and continue work processes at any point in time, even with inconsistent data. For more detailed information about the draft concept, see [Draft](#).

The example scenario uses a data model with UUID-key layout. In managed scenarios, UUIDs are beneficial as they can be drawn automatically by the RAP managed framework upon creating new instances. However, the draft scenario can also be developed with semantic primary keys.

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from [GitHub: Downloading the ABAP Flight Reference Scenario](#).

Development Flow

The example development scenario is designed to support two approaches. On the one hand, it teaches you how to build a managed business object with exclusive draft from scratch. On the other hand, if you just want to add draft capabilities to your existing BO, you can leave out the first steps to create a managed business object and start directly with [Draft-Enabling the Managed Business Object](#).

The development of an application with draft capabilities based on a managed business object requires the following activities:

1. Providing the CDS Data Model based on UUID keys

The first chapter provides you with information on how to create suitable database tables and CDS view entities for the managed business object. The example scenario guides you to build a three-node business object, which illustrates how root and child entities are handled in the draft scenario.

See [Data Model and Business Object Structure](#).

2. Developing Transactional Behavior for the Managed Business Object

In this section, you will equip the data model with transactional behavior. The business object framework for managed implementation type provides you with out-of-the-box support for standard implementation tasks. Other business-specific logic must be implemented using actions, determinations, and validations.

See [Developing Transactional Behavior](#).

3. Exposing the Managed Business Object for a UI service

Consuming the managed business object via a UI requires defining the projection layer, where you define all service-specific features, like UI annotations for UI configuration, search capabilities or value helps. The projected BO and all its related entities must then be defined in the service definition and then bound to a protocol. This UI service remains stable even when you add draft capabilities later on. Whereas the underlying BO changes, the BO projection filters only the non-draft features. The steps will be explained in this section.

See [Exposing the Managed Business Object for a UI Business Service](#).

4. Adding Exclusive Draft Capabilities to the Managed Business Object

This section explains how you draft-enable the existing business object with exclusive draft. On the one hand, draft-enabling requires the revision of all existing business logic. Sometimes the BO provider needs to distinguish if draft instances or active instances are processed. On the other hand, there are some draft features that you need to implement to add draft qualities to the business service.

See [Draft-Enabling the Managed Business Object](#).

5. Exposing the Draft-Enabled Business Object for a UI service

To enable the consumption of the draft-enabled BO via a Fiori Elements UI, the projection layer must be defined. It reuses all the draft capabilities that are defined in the underlying layer. In addition, the service must be defined and bound to a protocol.

See [Exposing the Draft Business Object for a UI Business Service](#).

Related Information

[Exclusive Draft Handling](#)

Building a Managed Business Object with UUID Keys

The first part of the managed development scenario with draft describes setting up a managed business object from scratch.

The assumption in this scenario is that you do not have any data or legacy coding that you want to include in the new business object. Everything is created from scratch. Therefore the first step is to create database table to store the data that the business object consumes. We use a simplified approach to have one database table for each business object entity. In real-world scenario, this might rarely be the case, but this demo scenario mainly demonstrates how you handle the transactional behavior with draft for the business object and does not focus on database structures for business objects.

The key structure of the data model in this scenario is based on UUID keys. That means, every BO entity instance is uniquely identifiable by one single unique key.

Based on these database tables, the data model and structure for the business object is defined via CDS view entities. For more information about the basic structure of a business object, see [Data Modeling and Behavior](#).

In a second step, the second component of a business object, the behavior, is defined and implemented for the business object. For more information about the business object behavior, see [Business Object](#)

1. [Data Model and Business Object Structure](#)

2. [Developing Transactional Behavior](#)

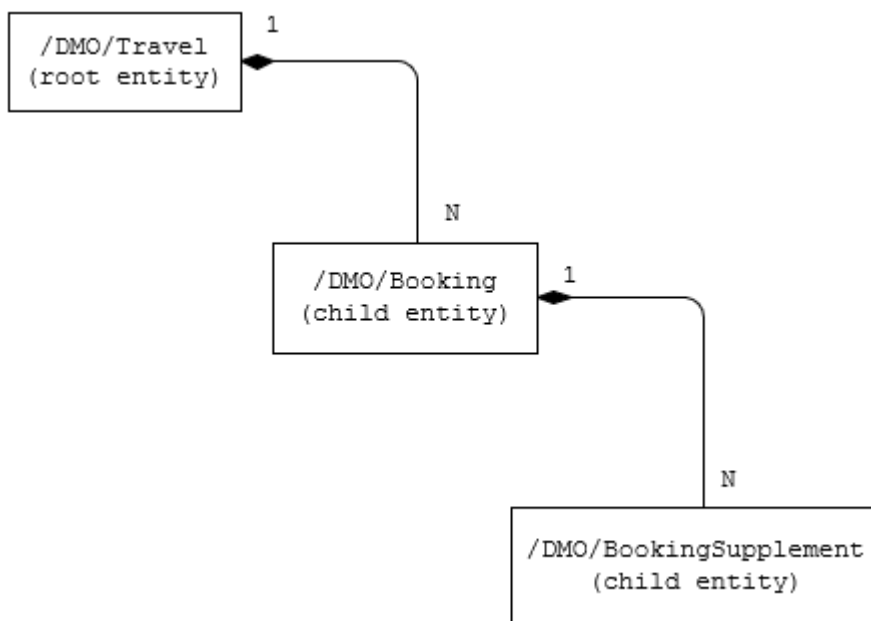
Data Model and Business Object Structure

The draft scenario uses the same data model as the other development scenarios. The final app processes travel data, primarily flight data, to create, update, and delete travel instances.

The travel business object consists of three entities that are structured in a hierarchy tree. Every entity in the BO-composition tree is modeled with a CDS view entity. The following entities are used for the transactional data model:

- **Travel:** The root entity comprises general travel data, like a trip's start and end dates, and the relevant customer and agency data.
- **Booking:** The booking entity depicts which bookings are relevant for a certain travel. In the current data model, only flights are available as bookings.
- **BookingSupplement:** The booking supplement entity presents additional bookable supplement for a certain booking. These are meals and beverages for the flights.

The BO-entities are related to each other in a composition structure. They maintain a dependency relationship, which means that a child entity cannot exist without its parent entity. In addition, a parent entity can have one to many child entities. The following diagram illustrates this relationship.



Editable Entities of the Business Object

The root entity is the head of the business object. You address a BO with the name of the root entity. It is also the root entity that receives special capabilities during runtime. In managed scenarios, only root entities can be lock masters. That means, if one of the entities that are part of the composition tree is requested for lock, it is the lock master instance and its child instances that are locked for the request. In addition, in the draft scenario, the lock master, in this case the root entity, receives an additional field for optimistic concurrency control in the transition from draft to active data. This field is called `total ETag`. For more information about the locking mechanism and the total ETag, see [Concurrency Control](#).

The composition tree includes child entities that are also part of the business object. They are connected via compositions, bidirectional connections by which the relationship to the root entity is defined.

Whereas the BO-entities are built up from scratch, as you create both, the database table and the related CDS entity, there are additional entities for master data that are reused from the demo content of the ABAP Flight Reference Scenario. These entities are not part of the transactional BO-structure. Most of these entities are used primarily as value help provider views for individual input fields on the UI.

- ▼  /DMO/FLIGHT_REUSE (51) *Flight Reference Scenario: Reused Entities*
 - >  /DMO/FLIGHT_REUSE_SUPPLEMENT (27) *Flight Reference Scenario: Supplement*
 - >  Authorizations (2)
- ▼  Core Data Services (12)
 - ▼  Data Definitions (12)
 - >  /DMO/I_AGENCY *Travel View - CDS Data Model*
 - >  /DMO/I_AIRPORT *Airport View - CDS Data Model*
 - >  /DMO/I_BOOKING_STATUS_VH_TEXT *Overall Status Value Help*
 - >  /DMO/I_BOOKING_STATUS_VH *Overall Status Value Help*
 - >  /DMO/I_CARRIER *Carrier View - CDS Data Model*
 - >  /DMO/I_CONNECTION *Connection View - CDS Data Model*
 - >  /DMO/I_CUSTOMER *Customer View - CDS Data Model*
 - >  /DMO/I_FLIGHT *Flight View - CDS Data Model*
 - >  /DMO/I_OVERALL_STATUS_VH_TEXT *Overall Status Value Help*
 - >  /DMO/I_OVERALL_STATUS_VH *Overall Status Value Help*

Non-Editable Entities Reused in the Data Model

Additional entities for currencies and countries are generally available in your system and are included in our data model using associations. These are I_Currency and I_Country.

Tasks for the Developer

1. [Creating Persistent Database Tables](#)
2. [Creating CDS View Entities](#)

Creating Persistent Database Tables

For the managed scenario with draft, create new database tables for the business object data model that match the transactional data structure of travel, booking and booking supplement.

Context and Procedure

In scenarios without existing legacy code, the first step is to create persistent database tables that store data.

Create these tables by using the creation wizard for creating database tables in ADT.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. For the persistent database tables, we use the prefix `A_` to indicate the active persistence. For detailed information, see [Naming Conventions for Development Objects](#).

When creating database tables for a managed business object with draft that use UUID key layout, the following considerations are relevant:

- The RAP managed runtime engine requires administrative data fields for internal processing of the BO entities. Especially, a field that denotes the timestamp when the instance was last changed is important for the ETag master handling on each entity. The RAP framework offers reuse data elements to define administrative fields on the database, see [RAP Reuse Data Elements](#). If the administrative fields on local instances are annotated in CDS with the annotation

@Semantics.systemDateTime.localInstanceLastChangedAt: true, they are automatically filled by the RAP managed runtime engine. For more information, see [Optimistic Concurrency Control](#).

- In a UUID scenario, every entity needs a field for the UUID key. In addition, child entities must have a field for their parent's UUID. Otherwise, you cannot define associations for the composition relationship. In addition, any child entity that has no parent-child relationship with the lock master entity must have a field with the lock master's UUID. Otherwise you cannot define associations to the lock master entity, which is important for lock and ETag handling.
- Apart from the UUID key, each database table also has a semantic ID field. These fields are no technical keys, but are used to semantically distinguish the entity instance in the business object.

Table /DMO/A_TRAVEL_D: Persistent Table for Managing Travel Data

This tables defines general travel data, data that describe the basic information of a trip, such as the travel ID, customer and agency ID, and dates. Since this scenario is based on a UUID key layout, the table also entails a field with a UUID data type. In addition, the fields for standard administration data, such as the respective user or the time of creation are added to the table. In managed business objects, these fields can be filled automatically by the RAP managed runtime framework if the corresponding @Semantics annotations are added to the fields in the CDS view entity. Administrative fields, in particular the field that determines when instance data is changed, are important for concurrency control.

i Expand the following code sample to view the source code of /DMO/A_TRAVEL_D.

Sample Code

```
@EndUserText.label : 'Active Travel Persistence for Draft Reference Scenario'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_TRAVEL_D{
  key client          : abap.clnt not null;
  key travel_uuid     : sysuuid_x16 not null;
  travel_id           : /dmo/travel_id not null;
  agency_id           : /dmo/agency_id;
  customer_id         : /dmo/customer_id;
  begin_date          : /dmo/begin_date;
  end_date            : /dmo/end_date;
  @Semantics.amount.currencyCode : '/dmo/a_travel_d.currency_code'
  booking_fee         : /dmo/booking_fee;
  @Semantics.amount.currencyCode : '/dmo/a_travel_d.currency_code'
  total_price         : /dmo/total_price;
  currency_code       : /dmo/currency_code;
  description         : /dmo/description;
  overall_status      : /dmo/overall_status;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_created_by    : abp_creation_user;
  local_created_at     : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_last_changed_by : abp_locinst_lastchange_user;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
  last_changed_at      : abp_lastchange_tstmpl;
}
```

Table /DMO/A_BOOKING_D: Persistent Table for Managing Booking Data

This table defines the booking data for a specific travel instance. It contains general booking data, such as the booking ID, booking date, and general flight data. To identify a booking instance uniquely, the table must contain a UUID field for the booking. In UUID scenarios, the parent UUID acts as a foreign-key field in the database for the child entity. For the ETag master handling, the booking database table receives a field to store the timestamp when the instance was updated.

i Expand the following code sample to view the source code of `/DMO/A_BOOKING_D`.

Sample Code

```
@EndUserText.label : 'Active Booking Persistence for Draft Reference Scenario'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_BOOKING_D{
  key client      : abap.clnt not null;
  key booking_uuid : sysuuid_x16 not null;
  parent_uuid     : sysuuid_x16;
  booking_id      : /dmo/booking_id not null;
  booking_date    : /dmo/booking_date;
  customer_id     : /dmo/customer_id;
  carrier_id      : /dmo/carrier_id;
  connection_id   : /dmo/connection_id;
  flight_date     : /dmo/flight_date;
  @Semantics.amount.currencyCode : '/dmo/a_booking_d.currency_code'
  flight_price    : /dmo/flight_price;
  currency_code   : /dmo/currency_code;
  booking_status  : /dmo/booking_status;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

Table `/DMO/A_BKSUPPL_D`: Persistent Table for Managing Booking Supplement Data

This table defines the supplement for a specific booking instance. It contains general booking supplement data, such as the booking supplement ID and price information. To identify a booking supplement instance uniquely, the table must contain a UUID field for the booking supplement. In UUID scenarios, the parent UUID and the lock master UUID (in our case the root UUID) act as a foreign-key field in the database. For the ETag master handling, the booking supplement database table receives a field to store the timestamp when the instance was updated.

i Expand the following code sample to view the source code of `/DMO/A_BKSUPPL_D`.

Sample Code

```
@EndUserText.label : 'Active BookingSupplement Persistence for Draft Ref. Scenario'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_BKSUPPL_D{
  key client      : abap.clnt not null;
  key booksuppl_uuid : sysuuid_x16 not null;
  root_uuid       : sysuuid_x16;
  parent_uuid     : sysuuid_x16;
  booking_supplement_id : /dmo/booking_supplement_id not null;
  supplement_id     : /dmo/supplement_id;
  @Semantics.amount.currencyCode : '/dmo/a_bksuppl_d.currency_code'
  price            : /dmo/supplement_price;
  currency_code    : /dmo/currency_code;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
}
```

Creating CDS View Entities

For the managed scenario with draft, create data definitions for CDS view entities to build the structure for the business object.

Context and Procedure

Previously, you have created persistent database table for the travel business object. The structure of the managed business object must now be defined by CDS data modeling.

Create data definitions for CDS view entities that represent the three transactional entities of the business object: **Travel**, **Booking**, and **Booking Supplement**.

i Note

CDS view entities are the successor DDIC-based CDS views. CDS view entities offer many advantages, such as optimization and simplification of syntax checks, and improved performance during view activations.

Select the template **Define Root View Entity** for the travel entity.

Select the template **Define View Entity with To-Parent Association** for the booking and the booking supplement entity.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. Since CDS view entities are interface views, they are prefixed with `R_` accordance with the VDM (virtual data model) naming convention. For detailed information, see [Naming Conventions for Development Objects](#).

When using CDS view entities for a managed business object with draft that use UUID key layout, the following considerations are relevant:

- For a business object, you must define a root entity that represents the head of the business object and provides the name with which the BO is addressed.
- The composition structure of the managed business object must be defined in the CDS view entities by compositions to the child entity and associations to the parent entity. In addition, an association to the lock and authorization master entity (in our scenario, the root entity) must be available from all entities in the business object.
- The RAP managed runtime framework fills the administrative fields automatically if they are annotated with the corresponding `@Semantics` annotation. These fields are only necessary on the root entity of the business object. In particular, the field that is annotated with `@Semantics.systemDateTime.localInstanceLastChangedAt: true` is updated whenever the instance is changed. In our scenario, this field is used for ETag comparison.

Data Definition `/DMO/R_Travel_D`: Travel Root View

The data source for the travel root view is the database table `/DMO/A_TRAVEL_D`. All fields from the database table are used for the data model.

To retrieve master data for value helps from other entities, define the association to the CDS entities `/DMO/I_Agency`, `/DMO/I_Customer`, `I_Currency`, `/DMO/I_Overall_Status_VH` and `I_Currency`. The former are part of the ABAP Flight Reference Scenario, which you can download. The CDS view `I_Currency` is generally available in your development system.

Annotate the price fields with the annotations `@Semantics.amount.currencyCode` to define which field is used as currency.

i Note

In contrast to DDIC-based CDS views, in CDS view entities, you do not need to specify the currency code field as a currency code if the field is already specified in the database table.

Use semantics annotations for the administrative fields. The ETag field `local_instance_last_changed_at` receives the annotation `@Semantics.systemDateTime.localInstanceLastChangedAt: true`.

As soon as the entity of the composition structure exists, define the compositional relationship to the child entity `/DMO/R_Booking_D` with the keyword `composition`.

i Expand the following code sample to view the source code of `/DMO/R_Travel_D`.

Sample Code

```
@EndUserText.label: 'Travel View Entity for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/R_Travel_D
as select from /DMO/A_TRAVEL_D

    composition [0..*] of /DMO/R_Booking_D as _Booking

    association [0..1] to /DMO/I_Agency as _Agency          on $projection.AgencyID = _Agency.AgencyID
    association [0..1] to /DMO/I_Customer as _Customer      on $projection.CustomerID = _Customer.CustomerID
    association [1..1] to /DMO/I_Overall_Status_VH as _OverallStatus on $projection.OverallStatus = _OverallStatus.OverallStatus
    association [0..1] to I_Currency as _Currency          on $projection.CurrencyCode = _Currency.CurrencyCode

{ ///DMO/A_TRAVEL_D
    key travel_uuid          as TravelUUID,

        travel_id           as TravelID,
        agency_id           as AgencyID,
        customer_id         as CustomerID,
        begin_date          as BeginDate,
        end_date            as EndDate,
        @Semantics.amount.currencyCode: 'CurrencyCode'
        booking_fee         as BookingFee,
        @Semantics.amount.currencyCode: 'CurrencyCode'
        total_price         as TotalPrice,
        currency_code       as CurrencyCode,
        description         as Description,
        overall_status      as OverallStatus,

        @Semantics.user.createdBy: true
        local_created_by    as LocalCreatedBy,
        @Semantics.systemDateTime.createdAt: true
        local_created_at    as LocalCreatedAt,
        @Semantics.user.localInstanceLastChangedBy: true
        local_last_changed_by as LocalLastChangedBy,
        //local ETag field --> OData ETag
        @Semantics.systemDateTime.localInstanceLastChangedAt: true
        local_last_changed_at as LocalLastChangedAt,

        //total ETag field
        @Semantics.systemDateTime.lastChangedAt: true
        last_changed_at     as LastChangedAt,

        //Associations
        _Booking,
        _Agency,
        _Customer,
        _OverallStatus,
        _Currency
}
```

Data Definition `/DMO/R_Booking_D`: Booking View

The data source for the booking view is the database table `/DMO/A_BOOKING_D`. All fields from the database table are used for the data model.

To retrieve master data for value helps from other entities, define the association to the CDS entities `/DMO/I_Customer`, `/DMO/I_Carrier`, `/DMO/I_Connection` and `/DMO/I_Booking_Status_VH`. These CDS views are part of the ABAP Flight Reference Scenario, which you can download.

Annotate the price fields with the annotations `@Semantics.amount.currencyCode` to define which field is used as currency.

i Note

In contrast to CDS views, in CDS view entities, you do not need to specify the currency code field as a currency code if the field is already specified in the database table.

Use semantics annotations for the administrative field. The local ETag field `local_instance_last_changed_at` receives the annotation `@Semantics.systemDateTime.localInstanceLastChangedAt: true`.

As soon as the entities of the composition structure exist, define the compositional relationship to the child entity `/DMO/R_BookingSupplement_D` with the keyword `composition`, and define the compositional relationship to the parent entity `/DMO/R_Travel_D` with the keyword `association to parent`.

i Expand the following code sample to view the source code of `/DMO/R_Booking_D`.

≡ Sample Code

```
@EndUserText.label: 'Booking View Entity for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED
```

```
define view entity /DMO/R_Booking_D as select from /DMO/A_BOOKING_D
```

```
    association to parent /DMO/R_Travel_D as _Travel on $projection.TravelUUID = _
    composition [0..*] of /DMO/R_BookingSupplement_D as _BookingSupplement
```

```
    association [1..1] to /DMO/I_Customer as _Customer on $projection.CustomerID
    association [1..1] to /DMO/I_Carrier as _Carrier on $projection.AirlineID
    association [1..1] to /DMO/I_Connection as _Connection on $projection.AirlineID
    and $projection.ConnectionID
    association [1..1] to /DMO/I_Booking_Status_VH as _BookingStatus on $projection.BookingSta
```

```
{ ///DMO/A_BOOKING_D
    key booking_uuid as BookingUUID,
    parent_uuid as TravelUUID,

    booking_id as BookingID,
    booking_date as BookingDate,
    customer_id as CustomerID,
    carrier_id as AirlineID,
    connection_id as ConnectionID,
    flight_date as FlightDate,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    flight_price as FlightPrice,
    currency_code as CurrencyCode,
    booking_status as BookingStatus,

    //local ETag field --> OData ETag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    local_last_changed_at as LocalLastChangedAt,

    //Associations
    _Travel,
    _BookingSupplement,

    _Customer,
    _Carrier,
```

```

        _Connection,
        _BookingStatus
    }

```

Data Definition /DMO/R_BookingSupplement_D: Booking Supplement View

The data source for the booking view is the database table /DMO/A_BKSUPPL_D. All fields from the database table are used for the data model.

To retrieve master data for value helps and texts from other entities, define the association to the CDS entities /DMO/I_Supplement and /DMO/I_SupplementText. These CDS views are part of the ABAP Flight Reference Scenario, which you can download.

Annotate the price fields with the annotations @Semantics.amount.currencyCode to define which field is used as currency.

i Note

In contrast to CDS views, in CDS view entities, you do not need to specify the currency code field as a currency code if the field is already specified in the database table.

Use semantics annotations for the administrative field. The local ETag field local_instance_last_changed_at receives the annotation @Semantics.systemDateTime.localInstanceLastChangedAt: true.

As soon as the entities of the composition structure exist, define the compositional relationship to the parent entity /DMO/R_Booking_D with the keyword association to parent, and define the compositional relationship to the root entity /DMO/R_Travel_D.

i Expand the following code sample to view the source code of /DMO/R_BookingSupplement_D.

≡ Sample Code

```

@EndUserText.label: 'BookSuppl View Entity fro Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define view entity /DMO/R_BookingSupplement_D
as select from /DMO/A_BKSUPPL_D

    association          to parent /DMO/R_Booking_D as _Booking          on $projection.BookingUUID
    association [1..1] to /DMO/R_Travel_D as _Travel          on $projection.TravelUUID = _Travel

    association [1..1] to /DMO/I_Supplement          as _Product          on $projection.SupplementID
    association [1..*] to /DMO/I_SupplementText      as _SupplementText on $projection.SupplementID

{ ///DMO/A_BKSUPPL_D
    key booksuppl_uuid          as BookSupplUUID,
      root_uuid                as TravelUUID,
      parent_uuid              as BookingUUID,

    booking_supplement_id as BookingSupplementID,
    supplement_id          as SupplementID,
    @Semantics.amount.currencyCode: 'CurrencyCode'
    price                  as BookSupplPrice,
    currency_code          as CurrencyCode,

    //local ETag field --> OData ETag
    @Semantics.systemDateTime.localInstanceLastChangedAt: true
    local_last_changed_at as LocalLastChangedAt,

    //Associations
    _Travel,
    _Booking,

```

```

    _Product,
    _SupplementText
}

```

Developing Transactional Behavior

This topic describes the behavior that is used for the business object in the managed scenario with draft.

The managed business object comprises the following behavior. The keywords in brackets refer to the technical features for this behavior.

1. Root Entity **Travel**:

a. You can create new instances of the travel entity.

On creating travel instances, the following behavior is implemented:

- The UUID key for new instances is automatically assigned (managed internal numbering).
- The travel UUID, the travel ID, the overall status, the total travel price, and administrative fields can't be provided by the consumer (read-only fields).
- The semantic travel ID is calculated (determination on save).
- The overall status for new travel instances is set to initial (determination on modify).
- The administrative fields are filled automatically (internal draft processing of the RAP runtime framework).
- The total travel price is calculated based on the booking fee of new travels (determination on modify, internal action).
- The customer ID, the agency ID, the begin and the end date must be provided on creating new travel instances (mandatory fields).
- All modify operations are checked for authorization against the user profile (global authorization).
- The travel instance can only be modified depending on the country code of the agency that is used for the respective travel. (instance authorization)
- The agency ID is checked for existence, validity, and authorization (validation on save).
- The customer ID is checked for existence and validity (validation on save).
- The travel dates are checked for consistency (validation on save).

b. You can update instances of the travel entity.

On updating travel instances, the following behavior is implemented:

- The travel ID, the overall status, the total travel price, and administrative fields cannot be modified BO-externally (read-only fields).
- You can only update the booking fee if the overall status is not set to accepted ('A') (dynamic field control).
- If the customer ID, the agency ID or the travel dates are changed, they are checked for existence and consistency (validations on save).
- You can set the overall status to accepted ('A') by an action, if the travel does not already have this status (dynamic action control).

- You can set the overall status to rejected ('X') by an action, if the travel does not already have this status (dynamic action control).
 - You can deduct a discount for the travel with a parameter action if the overall status is not already accepted ('A') (action with parameter).
- c. You can delete instances of the travel entity. Due to compositional relationship, all instances of child entities are deleted as well.

2. Child Entity **Booking**

- a. You can create new booking instances for travel instances.

On creating booking instances, the following behavior is implemented:

- The UUID key for new instances is automatically assigned (managed internal numbering).
- The travel UUID, the booking ID, the booking date, and administrative fields cannot be provided by the consumer (read-only fields).
- The semantic booking ID is calculated (determination on save).
- The booking date is set automatically (determination on save).
- The administrative fields are filled automatically (internal draft processing of the RAP runtime framework).
- The customer ID, the airline ID, the connection ID, and the flight date must be provided on creating new booking instances (mandatory fields).
- The customer ID is checked for existence and validity (validation on save).
- The airline ID is checked for existence and compatibility with the connection ID and the flight date (validation on save).
- The connection ID is checked for existence and compatibility with the airline ID and the flight date (validation on save).
- The flight date is checked for compatibility with the airline ID and the connection ID (validation on save).

- b. You can update instances of the booking entity.

On updating booking instances, the following behavior is implemented:

- The booking UUID, travel UUID, the booking ID, the booking date, and administrative fields cannot be modified BO-externally (read-only fields).
- If the customer ID, the airline ID, the connection ID, or the flight date are changed, they are checked for existence and consistency (validations on save).

- c. You can delete instances of the booking entity. Due to compositional relationship, all instances of the child entity are deleted as well.

3. Child entity **BookingSupplement**

- a. You can create new booking supplement instances for booking instances.

On creating booking supplement instances, the following behavior is implemented:

- The UUID key for new instances is automatically assigned (managed internal numbering).
- The travel UUID, the booking UUID, the booking supplement UUID, the booking supplement ID, and administrative fields cannot be provided by the consumer (read-only fields).
- The semantic booking supplement ID is calculated (determination on save).

- The administrative fields are filled automatically (internal draft processing of the RAP runtime framework).
- The supplement ID must be provided on creating new booking supplement instances (mandatory field).
- The supplement ID is checked for existence and validity (validation on save).

b. You can update instances of the booking supplement entity.

On updating booking supplement instances, the following behavior is implemented:

- The travel UUID, the booking UUID, the booking supplement UUID, the booking supplement ID, and administrative fields be modified BO-externally (read-only fields).
- If the supplement ID is changed, it is checked for existence and consistency (validation on save).

c. You can delete instances of the booking supplement entity.

Defining Standard Behavior

For the managed scenario with draft, create a behavior definition to define the behavior for the business object.

Context and Procedure

The behavior definition is the development object that defines the implementation type behavior for the business object. When using the implementation type **managed**, for some features, you only need to define behavior characteristics and standard operations in the behavior definition. The RAP managed runtime framework provides a generic solution for

- `strict mode`
- `create`
- `update`
- `delete`
- `create-by-association`
- `lock handling`
- `ETag handling`.

For more information, see [Defining Elementary Behavior for Ready-to-Run Business Object](#).

Create the behavior definition by using the creation wizard in ADT. By using the context menu on the root view (`/DMO/R_Travel_D`), you can directly create a behavior definition for the travel data model. In the wizard, choose the implementation **Managed**.

For the demo scenario with a managed business object with draft, the following considerations are relevant.

- The behavior of each entity is implemented in a separate behavior class pool. Hence, the implementation class must be created for each entity separately. For more information, see [Best Practices for Modularization and Performance](#).
- The `managed` implementation type requires the specification of `lock master` or `lock dependent` on each entity. For more information, see [Pessimistic Concurrency Control \(Locking\)](#).
- The `strict mode` ensures RAP best practices and up-to-date syntax for the behavior definition and behavior implementation. For conceptual background, see [Strict Mode](#).

- It is recommended to use authorization control to protect your business object against unauthorized access. Define `authorization master` or `authorization dependent` by `_Assoc` on each entity. The authorization behavior must then be implemented in the corresponding methods in the behavior pool. For conceptual background, see [Authorization Control](#). For detailed information about the implementation, see [Developing Authorization](#).
- Define the authorization context with reference to the authorization object `/DMO/TRVL`. Define a full authorization context by `define own authorization context by privileged mode`. In addition define privileged access.
- In managed business objects, it is best practice to define a local ETag master on each entity. With an ETag master on every entity, you ensure that the ETag check is done for every entity independently. For more information, see [Optimistic Concurrency Control](#).
- The RAP managed runtime framework is able to automatically draw primary key values in UUID scenarios. You use this functionality by defining early managed numbering in the behavior definition. Setting the primary key field to `read only` defines strict internal numbering. An external BO consumer is not allowed to provide the primary key values in this case. For more detailed information, see [Automatically Drawing Primary Key Values in Managed BOs](#).
- If different names are used on the database table and in the CDS data model, you need to define the mapping for these fields. This is done via the mapping operator in the behavior definition.

Behavior Definition `/DMO/R_Travel_D`

The template for the `managed` implementation type provides the behavior definition for every entity in the composition tree of the root entity. In addition, it specifies the data source of the respective entities as persistent table. The behavior definition proposes to include the behavior characteristics `lock`, `authorization` and `etag`. The standard operations `create`, `update` and `delete`, as well as the association `_Booking {create}` are included in the template for each entity. The `create` operation in child entities is not supported in RAP scenarios. Instances of child entities can only be created by `create-by-association` operation.

Define the root entity as `lock master`. The current version of the ABAP RESTful Application Programming Model only supports the specification of `lock master` on the root entity. Define the booking entity and the booking supplement entity as `lock dependent`. Lock dependents require an association to the lock master entity. In the CDS data model, you have specified the association to the travel entity `_Travel`. To use the association for the lock dependent, define the association as `transaction-enabled` in the behavior definition. For more information, see [Lock Dependent](#).

As the `TravelUUID` is used for the lock dependent relationship, it must be set to `readonly` in the `Booking` and `BookingSupplement` entity behavior.

Define the root entity as `authorization master`. The current version of the ABAP RESTful Application Programming Model only supports the specification of `authorization master` on the root entity. Define the booking entity and the booking supplement entity as `authorization dependent`. Just like lock dependents, authorization dependents require an association to the authorization master entity. For more information, see [Authorization Dependent](#).

Define an ETag master field on each entity. Use the field `LocalLastChangedAt` that is part of the entity's data model. Since this field is annotated with the annotation `@Semantics.systemDateTime.localInstanceLastChangedAt: true`, it is updated automatically by the RAP managed runtime framework. It is recommended that you specify the respective field as `read-only` in the behavior definition, using the field characteristic `readonly`.

Use early managed numbering for the primary key fields on every entity and set the fields to `read only`, so that the UUID values are drawn strictly internally.

Map the field names in the CDS data definition to the fields on the database table.

For each entity, define an alias and an implementation class. The behavior definition editor provides a quickfix to directly create the behavior class with the name that you specify in the behavior definition. The only behavior that needs implementation is

authorization control. The corresponding methods for global and instance authorization control are generated in the local types of the ABAP behavior pool.

i *Expand the following code sample to view the source code of /DMO/R_Travel_D.*

Sample Code

```
managed;
strict(2);

with privileged mode disabling NoCheckWhenPrivileged;
define authorization context NoCheckWhenPrivileged { '/DMO/TRVL'; }
define own authorization context by privileged mode;

define behavior for /DMO/R_Travel_D alias Travel
implementation in class /dmo/bp_i_travel_d unique
persistent table /DMO/A_TRAVEL_D
lock master
authorization master ( instance,global )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;
  association _Booking { create; }

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) LocalLastChangedAt;

  mapping for /DMO/A_TRAVEL_D
  { AgencyID = agency_id;
    BeginDate = begin_date;
    BookingFee = booking_fee;
    CurrencyCode = currency_code;
    CustomerID = customer_id;
    Description = description;
    EndDate = end_date;
    LocalCreatedAt = local_created_at;
    LocalCreatedBy = local_created_by;
    LocalLastChangedAt = local_last_changed_at;
    LocalLastChangedBy = local_last_changed_by;
    OverallStatus = overall_status;
    TotalPrice = total_price;
    TravelID = travel_id;
    TravelUUID = travel_uuid; }
}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /dmo/bp_i_booking_d unique
persistent table /DMO/A_BOOKING_D
lock dependent by _Travel
authorization dependent by _Travel
etag master LocalLastChangedAt
{
  update;
  delete;
  association _BookingSupplement { create; }

  association _Travel { }

  field ( readonly ) TravelUUID, LocalLastChangedAt;
  field ( numbering : managed, readonly ) BookingUUID;

  mapping for /DMO/A_BOOKING_D
  { AirlineID = carrier_id;
    BookingDate = booking_date;
    BookingID = booking_id;
    BookingStatus = booking_status;
    BookingUUID = booking_uuid;
    ConnectionID = connection_id;
    CurrencyCode = currency_code;
    CustomerID = customer_id;
```



```

    FlightDate = flight_date;
    FlightPrice = flight_price;
    LocalLastChangedAt = local_last_changed_at;
    TravelUUID = parent_uuid; }

}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /dmo/bp_i_bookingsupplement_d unique
persistent table /DMO/A_BKSUPPL_D
lock dependent by _Travel
authorization dependent by _Travel
etag master LocalLastChangedAt
{
    update;
    delete;

    association _Travel { }

    field ( readonly ) TravelUUID, LocalLastChangedAt;
    field ( numbering : managed, readonly ) BookSupplUUID;

    mapping for /DMO/A_BKSUPPL_D
    { BookSupplPrice = price;
      BookSupplUUID = booksuppl_uuid;
      BookingSupplementID = booking_supplement_id;
      BookingUUID = parent_uuid;
      CurrencyCode = currency_code;
      LocalLastChangedAt = local_last_changed_at;
      SupplementID = supplement_id;
      TravelUUID = root_uuid; }

}

```

Developing Actions

For the managed scenario with draft, add actions for nonstandard modify operations.

Context and Procedure

Actions are used to manifest business-logic-specific work-flows in one operation. You can implement simple status changes or a complete creation work-flow in one operation. For the UI, you can define action buttons that execute the action directly when the consumer chooses the button. For more detailed information, see [Actions](#).

For this scenario, define and implement the following actions:

- For the **travel entity**, the following actions are described:

- **acceptTravel**

The action sets the travel status to **Accepted (A)**.

Technically speaking, this action is an **instance** action with return parameter `$self`. The value of the field **OverallStatus** is changed by executing a modify request to update this field with the value A.

- **rejectTravel**

The action sets the travel status to **Rejected (X)**.

Technically speaking, this action is an **instance** action with return parameter `$self`. The value of the field **OverallStatus** is changed by executing a modify request to update this field with the value X.

- **deductDiscount**

The action calculates a new travel price by deducting a discount for the `BookingFee`. The consumer can freely choose the percentage that is deducted. The value must be passed together with the action request. On the Fiori UI, the UI provides a pop-up window for the consumer to fill in the requested value. The action parameter is modeled with an abstract entity.

Technically speaking, the action is an `instance action` with an importing action parameter and returns itself. The parameter is modeled with an abstract entity using a simple type for its parameter field. The value of the field `BookingFee` is changed by executing a modify request to update this field with the calculated value.

- `reCalcTotalPrice`

The action calculates the total price for one travel instance. It adds up the prices of all bookings, including their supplements, and the booking fee of the travel instance. If different currencies are used, the prices are converted to the currency of the travel instance.

Technically speaking, the action is an `internal instance action`. This action is invoked by determinations that are triggered when one of the involved fields is changed: `BookingFee` (travel entity), `FlightPrice` (booking entity), and `Price` (booking supplement entity).

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Implementing Actions for the Travel Entity](#)

For the managed scenario with draft, define and implement actions on the travel entity.

Implementing Actions for the Travel Entity

For the managed scenario with draft, define and implement actions on the travel entity.

Actions `acceptTravel` and `rejectTravel`

Both actions change the status of a travel instance. They are implemented exactly in the same way: Define the actions in the behavior definition and implement them in the behavior implementation class for `Travel`.

Definition

Define instance actions with result `$self`. The field `OverallStatus` is set to `read only`, so the consumer can only change the status via the actions `acceptTravel` and `rejectTravel`.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) OverallStatus;

  action acceptTravel result [1] $self;
  action rejectTravel result [1] $self;
  ... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

1. Define constants that are available for the travel status in the private section of the local types.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.
    CONSTANTS:
      BEGIN OF travel_status,
        open      TYPE c LENGTH 1 VALUE 'O', "Open
        accepted  TYPE c LENGTH 1 VALUE 'A', "Accepted
        rejected  TYPE c LENGTH 1 VALUE 'X', "Rejected
      END OF travel_status.

    ...
ENDCLASS.
```

2. Update the field `OverallStatus` with a modify request for all instances with the given keys. Provide the value `accpeted` for accepting the travel, and `rejected` for rejecting the travel.
3. The actions return `$self`. That means, they must return the updated instances in the result parameter. For that reason, read the updated instances from the buffer and fill an internal table.
4. Pass the internal table to the result parameter.

i *Expand the following code sample to view the source code of Action **acceptTravel**.*

Sample Code

```
METHOD acceptTravel.

  "Modify travel instance
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( FOR key IN keys ( %tky          = key-%tky
                                   OverallStatus = travel_status-accepted ) ).

  "Read changed data for action result
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    ALL FIELDS WITH
    CORRESPONDING #( keys )
  RESULT DATA(travels).

  result = VALUE #( FOR travel IN travels ( %tky   = travel-%tky
                                           %param = travel ) ).

ENDMETHOD.
```

i *Expand the following code sample to view the source code of Action **rejectTravel**.*

Sample Code

```
METHOD rejectTravel.

  "Modify travel instance
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( OverallStatus )
    WITH VALUE #( FOR key IN keys ( %tky          = key-%tky
                                   OverallStatus = travel_status-rejected ) ).

ENDMETHOD.
```

```

"Read changed data for action result
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    ALL FIELDS WITH
    CORRESPONDING #( keys )
  RESULT DATA(travels).

result = VALUE #( FOR travel IN travels ( %tky    = travel-%tky
                                           %param = travel ) ).

ENDMETHOD.

```

Action deductDiscount

Define the action in the behavior definition and implement it in the behavior implementation class for [Travel](#).

`deductDiscount` requires an importing parameter for entering the discount percentage. The value must be passed together with the action request. The action parameter is modeled with an abstract entity which uses a simple type for its parameter field.

Creating a Simple Type for the Parameter Field in the Abstract Entity

1. Create a CDS type without enumeration.
2. Define a simple type for the abstract entity field with the data type `abap.int1`.

Sample Code

```

@EndUserText.label: 'Discount Percentage'
define type /DMO/BT_DiscountPercentage: abap.int1

```

Creating an Abstract Entity for the Action Input Parameter

1. Create a data definition for a CDS abstract entity.
2. Define an element for the discount percentage which uses the simple type defined above.

Sample Code

```

@EndUserText.label: 'Abstract Entity for Deducting Discount'
define abstract entity /DMO/A_TRAVEL_DISCOUNT
{
  discount_percent : /DMO/BT_DiscountPercentage;
}

```

Define the action in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define an instance action with an action parameter `/DMO/A_TRAVEL_DISCOUNT` and the result `$self`.

For the `deductDiscount` action, a default values function is defined which provides default values for the input parameter once the action is triggered on the Fiori UI. For information on how to define and implement this default values function, see [Defaulting Input Parameters for Operations](#).

For conceptual information on default values functions, see [Operation Defaulting](#).

≡ Sample Code

```
define behavior for /DM0/R_Travel_D alias Travel
...
{ ...
  action deductDiscount parameter /DM0/A_TRAVEL_DISCOUNT result [1] $self { default function Ge
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

i Note

The input parameter of parameter actions is passed to the action method as the component `%param` in the importing parameter.

Calculate the new total price for the travel instance by deducting the provided discount percentages. The following steps guide you through the implementation.

1. Define an internal table and pass the entries of the importing parameter `keys`. Loop at the internal table for the keys with parameter values that do not match the preconditions for percentage value:
 - a. It must not be initial.
 - b. It must be less than 100.
 - c. It must be greater or equal than 0.
2. Append the transactional key to the failed table and append the corresponding message to the reported table if there are keys that do not match the preconditions. Delete the entries in the internal table that do not match the preconditions.
3. If there are still entries in the internal table that match the preconditions, read all travel instances with the imported keys into an internal table `travels` for the remaining entries.
4. Loop over the internal table and calculate the new reduced booking fee and append it to an internal table for update.
5. Execute a modify request to update the field `BookingFee` for the instances in the buffer.
6. To fill the action result, read the involved travel instances and fill the result correspondingly.

i Expand the following code sample to view the source code of Action `deductDiscount`.

≡ Sample Code

```
METHOD deductDiscount.
  DATA travels_for_update TYPE TABLE FOR UPDATE /DM0/R_Travel_D.
  DATA(keys_with_valid_discount) = keys.

  LOOP AT keys_with_valid_discount ASSIGNING FIELD-SYMBOL(<key_with_valid_discount>) WHERE %pa
      OR %param-discount_percent > 100
      OR %param-discount_percent <= 0.

      APPEND VALUE #( %tky                                = <key_with_valid_discount>-%tky ) TO failed-tr
      APPEND VALUE #( %tky                                = <key_with_valid_discount>-%tky
                      %msg                                = NEW /dmo/cm_flight_messages(
```

```

                                textid = /dmo/cm_flight_messages=>disco
                                severity = if_abap_behv_message=>severity
                                %element-TotalPrice          = if_abap_behv=>mk-on
                                %op-%action-deductDiscount = if_abap_behv=>mk-on
                                ) TO reported-travel.

DELETE keys_with_valid_discount.
ENDLOOP.

CHECK keys_with_valid_discount IS NOT INITIAL.

"get total price
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    FIELDS ( BookingFee )
    WITH CORRESPONDING #( keys_with_valid_discount )
  RESULT DATA(travels).

LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
  DATA percentage TYPE decfloat16.
  DATA(discount_percent) = keys_with_valid_discount[ KEY id %tky = <travel>-%tky ]-%param-d
  percentage = discount_percent / 100 .
  DATA(reduced_fee) = <travel>-BookingFee * ( 1 - percentage ) .

  APPEND VALUE #( %tky          = <travel>-%tky
                  BookingFee = reduced_fee
                  ) TO travels_for_update.
ENDLOOP.

"update total price with reduced price
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    UPDATE FIELDS ( BookingFee )
    WITH travels_for_update.

"Read changed data for action result
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Travel
    ALL FIELDS WITH
    CORRESPONDING #( travels )
  RESULT DATA(travels_with_discount).

result = VALUE #( FOR travel IN travels_with_discount ( %tky   = travel-%tky
                                                         %param = travel ) ).

ENDMETHOD.

```

Action reCalcTotalPrice

The action `reCalcTotalPrice` is called by determinations if changes on prices in the travel entity itself or in the child entities are executed. The total price on the travel entity is then recalculated.

Definition

Define the internal action in the behavior definition and implement it in the behavior implementation class for [Travel](#).

→ Remember

Internal actions can only be called from BO internal consumers.

In our demo scenario, the action is called from determinations on all three BO entities, whenever a price field or a currency code field is changed.

☰ Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  internal action reCalcTotalPrice;
  ... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

To determine the total price of a travel, all prices must be converted to the travel currency and then summed up. This is done for each BO entity separately. The following steps guide you through the implementation.

1. Define an internal standard table `amounts_per_currencycode` to process amounts and the related currency code.
2. Read all the instances with the imported keys into an internal table `travels`.
3. Delete instances with empty `CurrencyCode`. For such instances, the total price cannot be calculated.
4. Read all bookings for the returned travels into an internal table `bookings`. Return the link table to receive the associated bookings for each travel.
5. Read all booking supplements for the returned bookings into an internal table `bookingsupplements`. Return the link table to receive the associated bookingsupplements for each affected booking.
6. Loop at `travels` and start filling the amounts per currency with the corresponding values from the travel instance.
7. Loop at `booking_links` where the travel ID matches the respective travel and add the price to the travel price in the table `amounts_per_currencycode` if it has the same currency code. The ABAP statement `COLLECT` is useful in this case, as it sums the amounts with the same currency code and appends a new line if the currency codes differ.
8. Loop at `bookingsupplement_links` where the booking ID matches the respective booking and add the price to the travel price in the table `amounts_per_currencycode` if it has the same currency code.
9. Delete the entries in this table where the `currency_code` is not filled.
10. Loop at the table `amounts_per_currencycode` and convert the amounts with currency codes other than the travel currency code into the currency code of the travel instance.
11. Sum up the converted amounts for one travel.
12. Modify the travel entity with the new total price for all requested travel instances.

i *Expand the following code sample to view the source code of Action `reCalcTotalPrice`.*

Sample Code

```

METHOD reCalcTotalPrice.
  TYPES: BEGIN OF ty_amount_per_currencycode,
          amount      TYPE /dmo/total_price,
          currency_code TYPE /dmo/currency_code,
        END OF ty_amount_per_currencycode.

  DATA: amounts_per_currencycode TYPE STANDARD TABLE OF ty_amount_per_currencycode.

  " Read all relevant travel instances.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BookingFee CurrencyCode )
      WITH CORRESPONDING #( keys )

```

```
RESULT DATA(travels).
```

```
DELETE travels WHERE CurrencyCode IS INITIAL.
```

```
" Read all associated bookings and add them to the total price.
```

```
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
```

```
ENTITY Travel BY \_Booking
```

```
FIELDS ( FlightPrice CurrencyCode )
```

```
WITH CORRESPONDING #( travels )
```

```
LINK DATA(booking_links)
```

```
RESULT DATA(bookings).
```

```
" Read all associated booking supplements and add them to the total price.
```

```
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
```

```
ENTITY Booking BY \_BookingSupplement
```

```
FIELDS ( BookSupplPrice CurrencyCode )
```

```
WITH CORRESPONDING #( bookings )
```

```
LINK DATA(bookingsupplement_links)
```

```
RESULT DATA(bookingsupplements).
```

```
LOOP AT travels ASSIGNING FIELD-SYMBOL(<travel>).
```

```
" Set the start for the calculation by adding the booking fee.
```

```
amounts_per_currencycode = VALUE #( ( amount      = <travel>-bookingfee
                                     currency_code = <travel>-currencycode ) ).
```

```
LOOP AT booking_links INTO DATA(booking_link) USING KEY id WHERE source-%tky = <travel>-%t
```

```
" Short dump occurs if link table does not match read table, which must never happen
```

```
DATA(booking) = bookings[ KEY id %tky = booking_link-target-%tky ].
```

```
COLLECT VALUE ty_amount_per_currencycode( amount      = booking-flightprice
                                           currency_code = booking-currencycode ) INTO am
```

```
LOOP AT bookingsupplement_links INTO DATA(bookingsupplement_link) USING KEY id WHERE sou
```

```
DATA(bookingsupplement) = bookingsupplements[ KEY id %tky = bookingsupplement_link-ta
```

```
COLLECT VALUE ty_amount_per_currencycode( amount      = bookingsupplement-booksupplp
                                           currency_code = bookingsupplement-currencyco
```

```
ENDLOOP.
```

```
ENDLOOP.
```

```
DELETE amounts_per_currencycode WHERE currency_code IS INITIAL.
```

```
CLEAR <travel>-TotalPrice.
```

```
LOOP AT amounts_per_currencycode INTO DATA(amount_per_currencycode).
```

```
" If needed do a Currency Conversion
```

```
IF amount_per_currencycode-currency_code = <travel>-CurrencyCode.
```

```
<travel>-TotalPrice += amount_per_currencycode-amount.
```

```
ELSE.
```

```
  /DMO/CL_FLIGHT_AMDP=>convert_currency(
```

```
    EXPORTING
```

```
      iv_amount      = amount_per_currencycode-amount
```

```
      iv_currency_code_source = amount_per_currencycode-currency_code
```

```
      iv_currency_code_target = <travel>-CurrencyCode
```

```
      iv_exchange_rate_date   = cl_abap_context_info=>get_system_date( )
```

```
    IMPORTING
```

```
      ev_amount      = DATA(total_booking_price_per_curr)
```

```
  ).
```

```
<travel>-TotalPrice += total_booking_price_per_curr.
```

```
ENDIF.
```

```
ENDLOOP.
```

```
ENDLOOP.
```

```
" write back the modified total_price of travels
```

```
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
```

```
ENTITY travel
```

```
UPDATE FIELDS ( TotalPrice )
```

```
WITH CORRESPONDING #( travels ).
```

```
ENDMETHOD.
```

Developing Determinations

For the managed scenario with draft, add determinations to calculate values implicitly.

Context and Procedure

With determinations, you can generalize the calculation of values in the business logic of a managed BO. For more detailed information, see [Determinations](#).

i Note

You can assign determinations to determine actions that are called by RAP side effects. This enables immediate feedback after the user changes UI fields or field groups. For more detailed information, see [RAP Side Effects](#).

For this demo scenario, define and implement determinations for all three BO entities.

- For the **travel entity**, the following determinations are described:

- **setTravelNumber**

The semantic ID for the travel entity is drawn by the determination when a new instance is created. Since the semantic `TravelID` should not be provided by the client, the field must be set to `read only`.

i Note

In this demo scenario, we use a simplified approach to determine new readable IDs for the BO instances. To ensure that gap-free and non-duplicate IDs are assigned, use a number range object.

Technically speaking, this determination is a `determination on save` with the trigger operation `create`.

- **setStatusToOpen**

The travel is set to **Open** when a new instance is created.

Technically speaking, this determination is a `determination on modify` with the trigger operation `create`.

- **calculateTotalPrice**

The determination adds the prices of the travel (`BookingFee`), the booking (`FlightPrice`), and the booking supplement entity (`Price`). The sum of these values is the total price of the travel. The determination is triggered whenever one of the fields or the corresponding currency field is changed, and when a travel instance is created. Since the recalculation should be triggered whenever one of the mentioned fields is changed, the calculation of the total price is outsourced to an action. This action is triggered by a determination on each entity.

i Note

You can only define trigger fields for a determination from the same entity the determination is assigned to. A determination that is defined for the travel entity cannot have trigger fields from the booking entity.

Technically speaking, this determination is a `determination on modify` with the trigger operation `create` and with the field triggers `BookingFee` and `CurrencyCode`.

- For the **booking entity**, the following determinations are described:

- **setBookingNumber**

The semantic ID for the booking entity is drawn by the determination when a new instance is created. Since the semantic `BookingID` should not be provided by the client, the field must be set to `read only`.

Technically speaking, this is a `determination on save` with the trigger operation `create`.

- **setBookingDate**

The booking date is set when the booking instance is saved to the database. The booking date is only set internally by the determination and must not be changed after the instance is saved. Therefore, set the field **BookingDate** to **read only**. #

Technically speaking, this is a **determination on save** with the trigger operation **create**.

- **calculateTotalPrice**

See description above. Since the recalculation must also be triggered when the related fields of the booking entity are changed, the determination must be defined and implemented again.

Technically speaking, this is a **determination on modify** with the field triggers **FlightPrice** and **CurrencyCode**.

- For the **booking supplement entity**, the following determinations are described:

- **setBookSupplNumber**

The semantic ID for the booking supplement entity is drawn by the determination when a new instance is created. Since the semantic **BookingSupplementID** should not be provided by the client, the field must be set to **read only**.

Technically speaking, this is a **determination on save** with the trigger operation **create**.

- **calculateTotalPrice**

See description above. Since the recalculation must also be triggered when the related fields of the booking supplement entity are changed, the determination must be defined and implemented again.

Technically speaking, this is a **determination on modify** with the trigger operation **create** and the field triggers **BookSupplPrice** and **CurrencyCode**.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Determining Values for the Travel Entity](#)

For the managed scenario with draft, define and implement determinations on the travel entity for implicit value calculation.

[Determining Values for the Booking Entity](#)

For the managed scenario with draft, define and implement determinations on the booking entity for implicit value calculation.

[Determining Values for the Booking Supplement Entity](#)

For the managed scenario with draft, define and implement determinations on the booking supplement entity for implicit value calculation.

Determining Values for the Travel Entity

For the managed scenario with draft, define and implement determinations on the travel entity for implicit value calculation.

Determination **setTravelNumber**

Define the determination in the behavior definition and implement it in the behavior implementation class for **Travel**.

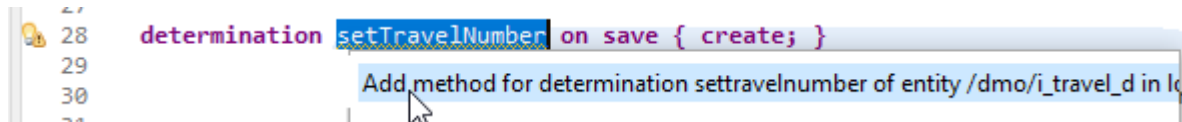
Definition

Define a determination on modify with trigger operation create. Since the travel ID must not be changed externally, define the field `TravelID` as readonly.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) TravelID;
  determination setTravelNumber on save { create; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.



Quick Fix for Generating Method Declaration in the Behavior Pool

Implementation

Determine the number for a new travel instance by selecting the highest available number from the database and add 1. The following steps guide you through the implementation.

1. Read all the travel instances with the imported keys into an internal table and delete the instances that already have a travel number (`TravelID`). Continue to work only with instances that do not have a travel number yet. If the determination is executed several times, the travel numbers must not be calculated again. This is particularly important in order to stick to the rule of idempotence for determinations.

→ Remember

The determination result must not change if the determination is executed several times under the same conditions (idempotence). See [Rules for Determinations](#).

2. Select the maximum travel number from the database table `/DMO/A_TRAVEL_D`.
3. Update the field `TravelID` of all involved instances with a modify request and assign a new `TravelID` by adding 1 to the max travel from before. Write messages in the request's reported table.

i Note

Since the determination is only executed on save, it is secured that adding 1 to the maximum number in the persisted table returns unique results.

i Expand the following code sample to view the source code of Determination `setTravelNumber`.

Sample Code

```
METHOD setTravelNumber.
  "Ensure idempotence
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( TravelID )
      WITH CORRESPONDING #( keys )
  RESULT DATA(travels).

  DELETE travels WHERE TravelID IS NOT INITIAL.
  CHECK travels IS NOT INITIAL.
```

```

"Get max travelID
SELECT SINGLE FROM /DMO/A_TRAVEL_D FIELDS MAX( travel_id ) INTO @DATA(max_travelid).

"update involved instances
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY Travel
  UPDATE FIELDS ( TravelID )
  WITH VALUE #( FOR travel IN travels INDEX INTO i (
                  %tky      = travel-%tky
                  TravelID   = max_travelid + i ) ).

ENDMETHOD.

```

Determination setStatusToOpen

Define the determination in the behavior definition and implement it in the behavior implementation class for **Travel**.

Definition

Define a determination on modify with trigger operation create. The overall status of the travel is only changed by the actions rejectTravel and acceptTravel, see [Accept and Reject Travel](#). Therefore the field is read only for the external consumer.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) TravelID; OverallStatus;
  determination setStatusToOpen on modify { create; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Set the status to **Open** when new instances are created The following steps guide you through the implementation.

1. Read all the travel instances with the imported keys into an internal table and delete the instances that already have an entry in the status field (OverallStatus). Continue to work only with instances that do not have a status yet. If the determination is executed after the status value is set by the consumer, the status must not be set to open.
2. For those instances, that have an initial value in OverallStatus update the field OverallStatus with open by executing a modify request for all instances with the imported keys. The constant open was defined before in the private section of the local types. Write messages in the reported table of the modify request.

i Note

Idempotence is not an issue in this determination. Even if the determination is executed several times and an instance already has the status **Open**, the outcome of the determination does not change with the modify request execution.

i Expand the following code sample to view the source code of Determination setStatusToOpen.

Sample Code

```
METHOD setStatusToOpen.
```

```
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY Travel
  FIELDS ( OverallStatus )
  WITH CORRESPONDING #( keys )
RESULT DATA(travels).
```

```
"If Status is already set, do nothing
DELETE travels WHERE OverallStatus IS NOT INITIAL.
CHECK travels IS NOT INITIAL.
```

```
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
ENTITY Travel
  UPDATE FIELDS ( OverallStatus )
  WITH VALUE #( FOR travel IN travels ( %tky          = travel-%tky
                                           OverallStatus = travel_status-open ) ).
```

```
ENDMETHOD.
```

Determination `calculateTotalPrice`

Define the determination in the behavior definition and implement it in the behavior implementation class for [Travel](#).

i Note

The actual calculation of the total price is done by the action `recalcTotalPrice`. The determination just executes the action. See [Action `recalcTotalPrice`](#).

Definition

Define a determination on modify with operation trigger `create` and field triggers `BookingFee` and `CurrencyCode` and define the field `TotalPrice` as `read only`, as the value must not be defined externally.

≡ Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( readonly ) TravelID; OverallStatus; TotalPrice;
  determination calculateTotalPrice on modify { create; field BookingFee, CurrencyCode; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Execute the action `recalcPrice` when the trigger fields are changed. The following steps guide you through the implementation.

1. Execute a modify request for action execution for all instances with the imported keys. Write messages in the reported table of the modify request.

i Expand the following code sample to view the source code of Determination `calculateTotalPrice`.

≡ Sample Code

```

METHOD calculateTotalPrice.

    MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
    EXECUTE reCalcTotalPrice
    FROM CORRESPONDING #( keys ).

ENDMETHOD.

```

Determining Values for the Booking Entity

For the managed scenario with draft, define and implement determinations on the booking entity for implicit value calculation.

Determination **setBookingNumber**

Define the determination in the behavior definition and implement it in the behavior implementation class for **Travel**.

Definition

Define a determination on save with trigger operation **create**. Since the booking ID must not be changed externally, define the field **BookingID** as **read only**.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
field ( readonly ) BookingID;
determination setBookingNumber on save { create; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Determine the number for a new booking instance by looping at all bookings for one travel and determine the greatest ID. Add 1 for every new booking instance. The following steps guide you through the implementation.

1. Read all the corresponding travel instances for the incoming booking **keys** into an internal table **travels**. If multiple bookings of the travel are requested, the travel is returned only once.
2. Read all bookings for all affected travels into an internal table **bookings**. Return the link table to receive the associated bookings for each travel.
3. Loop at **travels** and define a variable for the maximum booking ID and set the value to '0000' (initial value).
4. Loop at **booking_links** where the travel key matches the affected travel. Compare the booking ID with the maximum booking ID to determine the maximum booking ID available for the travel instance.
5. Provide a booking ID for all bookings of this travel that have none. Only the booking instances that have no booking ID assigned are taken into account. This is particularly important in order to stick to the rule of idempotence for determinations.

→ Remember

The determination result must not change if the determination is executed several times under the same conditions (idempotence). See [Rules for Determinations](#).

Write the new booking IDs into a new internal table typed as table for update.

6. Update the booking entity with the entries of the internal update table.

i Expand the following code sample to view the source code of Determination **setBookingNumber**.

≡ Sample Code

```
METHOD setBookingNumber.
  DATA:
    max_bookingid  TYPE /dmo/booking_id,
    bookings_update TYPE TABLE FOR UPDATE /DMO/R_Travel_D\\Booking,
    booking        TYPE STRUCTURE FOR READ RESULT /DMO/R_Booking_D.

  "Read all travels for the requested bookings
  " If multiple bookings of the same travel are requested, the travel is returned only once.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FIELDS ( TravelUUID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " Read all bookings for all affected travels
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel BY \_Booking
      FIELDS ( BookingID )
      WITH CORRESPONDING #( travels )
      LINK DATA(booking_links)
    RESULT DATA(bookings).

  " Process all affected travels.
  LOOP AT travels INTO DATA(travel).

    " find max used bookingID in all bookings of this travel
    max_bookingid = '0000'.
    LOOP AT booking_links INTO DATA(booking_link) USING KEY id WHERE source-%tky = travel-%tky
      " Short dump occurs if link table does not match read table, which must never happen
      booking = bookings[ KEY id %tky = booking_link-target-%tky ].
      IF booking-BookingID > max_bookingid.
        max_bookingid = booking-BookingID.
      ENDIF.
    ENDLOOP.

    "Provide a booking ID for all bookings of this travel that have none.
    LOOP AT booking_links INTO booking_link USING KEY id WHERE source-%tky = travel-%tky.
      " Short dump occurs if link table does not match read table, which must never happen
      booking = bookings[ KEY id %tky = booking_link-target-%tky ].
      IF booking-BookingID IS INITIAL.
        max_bookingid += 1.
        APPEND VALUE #( %tky          = booking-%tky
                        BookingID = max_bookingid
                        ) TO bookings_update.
      ENDIF.
    ENDLOOP.
  ENDLOOP.

  " Provide a booking ID for all bookings that have none.
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY booking
      UPDATE FIELDS ( BookingID )
      WITH bookings_update.

ENDMETHOD.
```

Determination setBookingDate

Define the determination in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a determination on save with trigger operation `create`. Once the booking date is set, when a booking instance is saved to the database, the date should not be changed afterward. Therefore the field `BookingDate` is read only for the external consumer.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
field ( readonly ) TravelUUID, BookingID, BookingDate;
determination setBookingDate on save { create; }
... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Set the system date for the booking date. The following steps guide you through the implementation.

1. Read all the booking instances with the imported keys into an internal table and delete the instances that already have a booking date (`BookingDate`). Continue to work only with instances that do not have a booking date yet. If the determination is executed several times, the booking date must not be calculated again. This is particularly important in order to stick to the rule of idempotence for determinations.

→ Remember

The determination result must not change if the determination is executed several times under the same conditions (idempotence). See [Rules for Determinations](#).

2. Assign the system date `sy-datum` to `BookingDate` for every incoming booking instance .
3. Update the field `BookingDate` of all involved instances with a modify request.

i *Expand the following code sample to view the source code of Determination `setBookingDate`.*

Sample Code

```
METHOD setBookingDate.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking
      FIELDS ( BookingDate )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  DELETE bookings WHERE BookingDate IS NOT INITIAL.
  CHECK bookings IS NOT INITIAL.

  LOOP AT bookings ASSIGNING FIELD-SYMBOL(<booking>).
    <booking>-BookingDate = cl_abap_context_info=>get_system_date( ).
  ENDLOOP.

  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking
```



```
UPDATE FIELDS ( BookingDate )
WITH CORRESPONDING #( bookings ).
```

```
ENDMETHOD .
```

Determination `calculateTotalPrice`

Define the determination in the behavior definition and implement it in the behavior implementation class for **Booking**.

i Note

The actual calculation of the total price is done by the action `reCalcPrice`. The determination just executes the action. See [Action `reCalcTotalPrice`](#).

Definition

Define a determination on modify with operation trigger `create` and field triggers `FlightPrice` and `CurrencyCode`.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
determination calculateTotalPrice on modify { create; field FlightPrice, CurrencyCode; }

... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Execute the action `reCalcTotalPrice` when the trigger fields are changed. The following steps guide you through the implementation.

1. To get the `TravelUUID`, read the travel instance by a read by association into an internal table. The key of the root instance is needed for the action execution.
2. Execute a modify request for action execution for all affected travel instances.

i Note

The response parameters `failed` and `reported` are not retrieved in this EML call, since the action `reCalcTotalPrice` does not return these parameters. It is assumed that the action cannot fail as it is only called internally and processed by the managed BO provider.

The response parameter `result` is not retrieved as the action doesn't define this parameter.

i Expand the following code sample to view the source code of Determination `calculateTotalPrice`.

Sample Code

```
METHOD calculateTotalPrice.

" Read all parent UUIDs
```

```

READ ENTITIES OF /DM0/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_Travel
  FIELDS ( TravelUUID )
  WITH CORRESPONDING #( keys )
  RESULT DATA(travels).

" Trigger Re-Calculation on Root Node
MODIFY ENTITIES OF /DM0/R_Travel_D IN LOCAL MODE
  ENTITY Travel
  EXECUTE reCalcTotalPrice
  FROM CORRESPONDING #( travels ).

ENDMETHOD.

```

Determining Values for the Booking Supplement Entity

For the managed scenario with draft, define and implement determinations on the booking supplement entity for implicit value calculation.

Determination `setBookSupplNumber`

Define the determination in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a determination on save with trigger operation `create`. Since the booking supplement ID must not be changed externally, define the field `BookingSupplementID` as read only.

Sample Code

```

define behavior for /DM0/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  field ( readonly ) TravelUUID, BookingSupplementID;
  determination setBookSupplNumber on save { create; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

The implementation for determining new booking supplement numbers is the same as the calculation for the booking. For a detailed step-by-step description, see [Determination `setBookingNumber`](#).

i *Expand the following code sample to view the source code of Determination `setBookSupplNumber`.*

Sample Code

```

METHOD setBookSupplNumber.
DATA:
  max_bookingsupplementid TYPE /dmo/booking_supplement_id,
  bookingsupplements_update TYPE TABLE FOR UPDATE /DM0/R_Travel_D\BookingSupplement,
  bookingsupplement TYPE STRUCTURE FOR READ RESULT /DM0/R_BookingSupplement_D.

```

```

"Read all bookings for the requested booking supplements
" If multiple booking supplements of the same booking are requested, the booking is returned
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY BookingSupplement BY \_Booking
  FIELDS ( BookingUUID )
  WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

" Read all booking supplements for the affected bookings
READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_BookingSupplement
  FIELDS ( BookingSupplementID )
  WITH CORRESPONDING #( bookings )
  LINK DATA(bookingsupplement_links)
  RESULT DATA(bookingsupplements).

" Process all affected bookings.
LOOP AT bookings INTO DATA(booking).

  " find max used booking supplement ID in all booking supplements of this booking
  max_bookingsupplementid = '00'.
  LOOP AT bookingsupplement_links INTO DATA(bookingsupplement_link) USING KEY id WHERE source-
    bookingsupplement = bookingsupplements[ KEY id %tky = bookingsupplement_link-target-%tky
    IF bookingsupplement-BookingSupplementID > max_bookingsupplementid.
      max_bookingsupplementid = bookingsupplement-BookingSupplementID.
    ENDIF.
  ENDLOOP.

  "Provide a booking supplement ID for all booking supplements of this booking that have non
  LOOP AT bookingsupplement_links INTO bookingsupplement_link USING KEY id WHERE source-%tky
    bookingsupplement = bookingsupplements[ KEY id %tky = bookingsupplement_link-target-%tky
    IF bookingsupplement-BookingSupplementID IS INITIAL.
      max_bookingsupplementid += 1.
      APPEND VALUE #( %tky = bookingsupplement-%tky
                      bookingsupplementid = max_bookingsupplementid
                      ) TO bookingsupplements_update.
    ENDIF.
  ENDLOOP.
ENDLOOP.

" Provide a booking supplement ID for all booking supplements that have none.
MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY BookingSupplement
  UPDATE FIELDS ( BookingSupplementID ) WITH bookingsupplements_update.

ENDMETHOD.

```

Determination calculateTotalPrice

Define the determination in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

i Note

The actual calculation of the total price is done by the action [recalcPrice](#). The determination just executes the action. See [Action recalcTotalPrice](#).

Definition

Define a determination on modify with operation trigger create and field triggers BookSupplPrice and CurrencyCode.

≡ Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...

```

```

{ ...
  determination calculateTotalPrice on modify { create; field BookSupplPrice, CurrencyCode; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Execute the action `reCalcTotalPrice` when the trigger fields are changed. The steps are exactly the same as in [Determination `calculateTotalPrice`](#).

i *Expand the following code sample to view the source code of Determination `calculateTotalPrice`.*

Sample Code

```

METHOD calculateTotalPrice.
  " Read all parent UUIDs
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Travel
      FIELDS ( TravelUUID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  " Trigger Re-Calculation on Root Node
  MODIFY ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      EXECUTE reCalcTotalPrice
      FROM CORRESPONDING #( travels ).

ENDMETHOD.

```

Developing Feature Control

For the managed scenario with draft, enable and disable functionality with feature control.

Context and Procedure

Feature control is used to define the availability of behavior components, like fields, actions, and operations. You can define if these components are always available for the client or only under certain circumstances (dynamic feature control). For more detailed information, see [Feature Control](#).

For this demo scenario, define and implement field, operation and action control.

- Define **field control**.
 - **field (readonly)**

Fields that are filled by the RAP managed runtime framework (internally) are set to read only as the consumer is not allowed to change these. Marking these fields as read only in the behavior definition makes also has an effect on the UI. The end user cannot provide values for these fields. In the travel scenario, the fields that are set to read only are the administrative fields and fields that are filled internally, for example by determinations. These fields are:

- On the travel entity:

TravelID, OverallStatus, TotalPrice, LocalCreatedAt, LocalCreatedBy, LocalLastChangedAt, LocalLastChangedBy, LastChangedAt

- On the booking entity:

TravelUUID, BookingID, BookingDate, LocalLastChangedAt

- On the booking supplement entity:

TravelUUID, BookingUUID, BookingSupplementID, LocalLastChangedAt

The key fields whose values are set by the RAP managed runtime framework via managed numbering must also be set to read only. They also receive the attribute numbering: `managed`. These fields are:

- On the travel entity: `TravelUUID`
- On the booking entity: `BookingUUID`
- On the booking supplement entity: `BookSupplUUID`

◦ `field (mandatory)`

Fields whose values are checked by validations are set to mandatory. These fields are:

- On the travel entity:
`CustomerID, AgencyID, BeginDate, EndDate`
- On the booking entity:
`CustomerID, AirlineID, ConnectionID, FlightDate`
- On the booking supplement entity:
`SupplementID`

◦ `field (feature: instance)`

Fields whose logic depends on other circumstances are controlled by dynamic feature control. In this scenario, the `BookingFee` (on the travel entity) is set to read only if the instance's status is **Accepted (A)**. The feature control condition must be implemented in the behavior class.

• Define **operation control**.

◦ `create (features : instance)`

Use dynamic operation control for the associations that are enabled for create. In this demo scenario, you can only create new booking instance for a travel if the overall status is not rejected ('X'). The feature control condition must be implemented in the behavior class.

- On the travel entity:
`association _Booking { create (features : instance); }`

• Define **action control**.

◦ `action (features: instance)`

Dynamic operation control is used for actions that are enabled depending on other states of the instance. These actions are

- On the travel entity:

The action `acceptTravel` can only be executed, if the overall status is not already set to **Accepted (A)**:
`action ( features: instance ) acceptTravel result [1] $self;`

The action `rejectTravel` can only be executed, if the overall status is not already set **Rejected (X)**:
`action ( features: instance ) rejectTravel result [1] $self;` on the travel entity.

The action `deductDiscount` can only be executed, if the overall status is not already set **Accepted (A)**:
`action ( features: instance ) deductDiscount result [1] $self;` on the travel entity.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Adding Feature Control for the Travel Entity](#)

For the managed scenario with draft, enable and disable functionality with feature control on the travel entity.

[Adding Feature Control for the Booking Entity](#)

For the managed scenario with draft, enable and disable functionality with feature control on the booking entity.

[Adding Feature Control for the Booking Supplement Entity](#)

For the managed scenario with draft, enable and disable functionality with feature control on the booking supplement entity.

Adding Feature Control for the Travel Entity

For the managed scenario with draft, enable and disable functionality with feature control on the travel entity.

Define the static and dynamic feature control in the behavior definition and implement dynamic feature control in the behavior implementation class for **Travel**.

Definition

Define field, action, and operation control. For dynamic feature control, add `( feature: instance )` to the feature control definition.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  association _Booking { create ( features : instance ); }

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) TravelID, OverallStatus, TotalPrice, LocalCreatedAt, LocalCreatedBy, Local
  field ( mandatory ) CustomerID, AgencyID, BeginDate, EndDate, CurrencyCode;
  field ( features : instance ) BookingFee;

  action ( features : instance ) acceptTravel result [1] $self;
  action ( features : instance ) rejectTravel result [1] $self;
  action ( features : instance ) deductDiscount parameter /DMO/A_TRAVEL_DISCOUNT result [1] $se

... }
```

Via a quick fix, you can generate the method declaration for the feature control implementation in the behavior pool directly from the behavior definition editor.

Dynamic Feature Control Implementation for the Travel Entity

Dynamic feature control must be implemented in the behavior implementation in the method `FOR FEATURES`. You can use the quick fix on one of the dynamic feature control features in the behavior definition for the method declaration. The following steps guide you through the implementation.

1. Read all the instances with the imported `keys` into an internal table `travels`. The fields that are relevant to decide whether features are available must be included in the read result. For the travel entity, only the field `OverallStatus` is relevant.
2. Fill the result table of the method with the respective feature. Use the transaction key `%tky` to identify the travel instance. The importing parameter `keys` contains the `%field`, `%action`, `%assoc` component to determine what kind of feature is controlled. Use a condition statement to define the circumstance and the outcome of each dynamically controlled feature. You can work with the constants, defined in the private section of the behavior class, to refer to the values of the overall status.

i Expand the following code sample to view the source code of `GET_INSTANCE_FEATURES`.

Sample Code

```
METHOD get_instance_features.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( OverallStatus )
      WITH CORRESPONDING #( keys )
      RESULT DATA(travels)
      FAILED failed.

  result = VALUE #( FOR ls_travel IN travels
                    ( %tky                                = ls_travel-%tky
                      %field-BookingFee                    = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-f-read_only
                                                                    ELSE if_abap_behv=>fc-f-unrestrict
                      %action-acceptTravel                  = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled
                                                                    ELSE if_abap_behv=>fc-o-enabled )
                      %action-rejectTravel                  = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled
                                                                    ELSE if_abap_behv=>fc-o-enabled )
                      %action-deductDiscount                = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled
                                                                    ELSE if_abap_behv=>fc-o-enabled )
                      %assoc-_Booking                       = COND #( WHEN ls_travel-OverallStatus = trav
                                                                    THEN if_abap_behv=>fc-o-disabled
                                                                    ELSE if_abap_behv=>fc-o-enabled )
                    ) ).

ENDMETHOD..
```

Adding Feature Control for the Booking Entity

For the managed scenario with draft, enable and disable functionality with feature control on the booking entity.

Define static feature control in the behavior definition. For the **Booking** entity, there is no dynamic feature control.

Definition

Define field control.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{...

    field ( numbering : managed, readonly ) BookingUUID;
    field ( readonly ) TravelUUID, BookingID, BookingDate, LocalLastChangedAt;
    field ( mandatory ) CustomerID, AirlineID, ConnectionID, FlightDate, CurrencyCode;

... }
```

Adding Feature Control for the Booking Supplement Entity

For the managed scenario with draft, enable and disable functionality with feature control on the booking supplement entity.

Define the static feature control in the behavior definition. For the `BookingSupplement`, there is no dynamic feature control.

Definition

Define field control.

Sample Code

```
define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
    field ( numbering : managed, readonly ) BookSupplUUID;
    field ( readonly ) TravelUUID, BookingUUID, BookingSupplementID, LocalLastChangedAt;
    field ( mandatory ) SupplementID, CurrencyCode;

... }
```

Developing Authorization

For the managed scenario with draft, add authorization control to prevent unauthorized modifying access to the business object.

Context and Procedure

With authorization control, you define which users are allowed to make changes on BO instances. The RAP model differentiates between general authorization to use operations (global authorization), and instance-specific authorization, which assigns authorization to use operations depending on values of an instance in question (instance authorization). For more detailed information, see [Authorization Control](#).

In this reference scenario, authorization for each standard operation is considered separately. That means, one user group might be able to create new instances, but is not allowed to update this instance or vice versa. Actions, however, use the same authorization as the operation update. The authorization control can be delegated for these operations.

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

For this scenario, define and implement the following authorization control:

- Define and implement **global authorization**.

Global authorization is checked for all standard operations and actions. The authorization for the operating user is checked with authorization objects for each modifying operation separately, which means there can be a different authorization for the create, update and delete operation. All actions are treated like the update operation. In this example implementation, a simulation is provided to allow all users to execute all operations.

For a detailed description on how to implement global authorization control, see [Implementing Global Authorization](#).

- Define and implement **instance authorization**.

Instance authorization is checked for all instance-related operations. Instance authorization is checked with an authorization object against an authorization field. In this example scenario, users are only authorized to modify travel instances that have an agency with one specific country code. A simulation implementation is provided to allow users to modify instances only, if the country code of the related agency is US. However, you can use any other country code as well.

i Note

In the dedicated authorization methods, it's only possible to check instance authorization against the state of the instance before the current LUW. In other words, authorization is checked against the persistent state of the instance, if there's one. For new instances, the same authorization as for global authorization is applied. To check authorization against incoming values, you need to implement a precheck.

For a detailed description on how to implement instance authorization control, see [Implementing Instance Authorization](#).

- Define and implement **prechecks**.

Prechecks check against incoming values. In this example, the precheck implementation checks that only BOs with AgencyIDs with the `country_code` US can be created or updated.

For a detailed description on how to implement prechecks, see [Implementing Prechecks](#).

i Note

Authorization for authorization-dependent entities is delegated to the authorization master entity. That means, modify requests on authorization-dependent entities are treated like the update operation on the authorization master entity.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Implementing Global Authorization](#)

For the managed scenario with draft, authorize users to generally modify travel instances.

[Implementing Instance Authorization](#)

For the managed scenario with draft, authorize users to modify travel instances based on an authorization field.

[Implementing Prechecks](#)

For the managed scenario with draft, add prechecks to check authorization against incoming values to prevent values from reaching the transactional buffer.

Implementing Global Authorization

For the managed scenario with draft, authorize users to generally modify travel instances.

Define global authorization in the behavior definition and implement it in the behavior implementation class for **Travel**. For conceptual information, see [Authorization Control](#).

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

Definition

If not already done so, define `authorization master (global)` for the travel entity and `authorization dependent by _Travel` for the booking and the booking supplement entity in the behavior definition.

In addition, delegate the authorization control for all actions, except for the internal and draft actions, and the create-by-association to the update operation. The syntax `authorization: update` is used.

≡ Sample Code

```
...
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
...
authorization master ( global )
{...
  association _Booking { create ( features : instance, authorization: update ) ; }
...

  action ( features : instance, authorization: update ) acceptTravel result [1] $self;
  action ( features : instance, authorization: update ) rejectTravel result [1] $self;
  action ( features : instance, authorization: update ) deductDiscount parameter /DMO/A_TRAVEL_
  internal action reCalcTotalPrice;

...}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /DMO/BP_BOOKING_D unique
....
authorization dependent by _Travel
{...
  association _BookingSupplement { create ( authorization: update ); }
...}
```

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_D unique
...
authorization dependent by _Travel
{
...
}

```

Via a quick fix, you can generate the method declaration for the authorization control implementation in the behavior pool directly from the behavior definition editor.

Implementation

Global authorization control must be implemented in the behavior implementation in the method **FOR GLOBAL AUTHORIZATION**. You can use the quick fix in the behavior definition for the method declaration. The method imports the parameter **requested_authorizations**, which indicates the operations for which authorization is requested. In the implementation, you have to fill the parameter **result**, which must be filled with the authorization result for each requested operation. The method also has the changing parameter **reported**, which is to be filled if you want to return messages for unauthorized cases. For more information, about implementation details, see [Authorization Implementation](#).

The authorization concept in this guide assumes that there can be different authorizations for creating, updating, and deleting instances. All actions of the travel BO are considered with the same authorization as the update operation. With this approach, it is possible that one user is allowed to create a new travel instance, but is not allowed to use actions. Actions and create-by-association are defined with the addition **authorization: update** and thus must not be treated separately in the authorization implementation.

As the authorization for create, update, and delete is the same for global and instance authorization (except for the relevant authorization field for instance authorization), it is best practice to outsource the actual authorization check in separate methods that can be called by all authorization checks that are needed in the BO implementation.

The following steps guide you through the implementation.

Methods for Authorization Checks

1. For each operation, create a method to check the authorization for it.
2. We use the authorization object **/DMO/TRVL** to check the authorization for each operation.

For more information, about using and configuring authorization objects, see [Authorization Basics](#).

🏠 For more information, about using and configuring authorization objects, see [User and Role Administration of Application Server ABAP](#)

In each method, call the authorization object and check for the relevant activity.

Authorization Fields in Authorization Object

Operation	Activity Field
create	Field 01 (Add or Create)
update + all actions	Field 02 (Change)
delete	Field 06 (Delete)

The authorization object checks against the authorization field `/DMO/CNTRY`. For global authorization, this is not relevant as the authorization for executing the operations is independent of the state of the respective instance, and thus independent of the value of the authorization field. For that reason, you can use `'DUMMY'` for the authorization field.

The implementation considers the authorization for each modify operation separately. There can be a different authorization for creating, updating, and deleting instances on the travel BO.

3. Evaluate the authorization for the respective operation based on `sy-subrc` and fill the response parameter of the method.
4. As the authorization which is checked by the authorization object is not configured in the backend, we provide a simulation implementation to allow all users for all operations. Do not use this piece of code in productive applications.

Expand the following code sample to view the source code of the methods `is_create_granted`, `is_update_granted`, and `is_delete_granted`.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

    PRIVATE SECTION.

        ...

        METHODS is_create_granted
            RETURNING VALUE(create_granted) TYPE abap_bool.
        METHODS is_update_granted
            RETURNING VALUE(update_granted) TYPE abap_bool.
        METHODS is_delete_granted
            RETURNING VALUE(delete_granted) TYPE abap_bool.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    METHOD is_create_granted.
        AUTHORITY-CHECK OBJECT '/DMO/TRVL'
            ID '/DMO/CNTRY' DUMMY
            ID 'ACTVT'      FIELD '01'.
        create_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

        "Simulation for full authorization
        "(not to be used in productive code)
        create_granted = abap_true.
    ENDIF.

ENDMETHOD.

METHOD is_update_granted.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
        ID '/DMO/CNTRY' DUMMY
        ID 'ACTVT'      FIELD '02'.
    update_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    update_granted = abap_true.

```

```

ENDIF.

ENDMETHOD.

METHOD is_delete_granted.
  AUTHORITY-CHECK OBJECT '/DMO/TRVL'
    ID '/DMO/CNTRY' DUMMY
    ID 'ACTVT'      FIELD '06'.
  delete_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

  "Simulation for full authorization
  "(not to be used in productive code)
  delete_granted = abap_true.
ENDIF.

ENDMETHOD.

ENDCLASS.

```

Global Authorization Implementation

1. For each operation, check if the authorization is requested. In this example, all actions are treated like the update operation. The draft action Edit must be delegated manually in the implementation. Authorizations for all other actions are delegated in the behavior definition.
2. Fill the reported parameter with an adequate message, if the user is unauthorized for the respective operation.

i *Expand the following code sample to view the source code of the method `get_global_authorizations`.*

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

  ...

  METHODS get_global_authorizations FOR GLOBAL AUTHORIZATION
    IMPORTING REQUEST requested_authorizations FOR travel RESULT result.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
  METHOD get_global_authorizations.
    IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
      IF is_create_granted( ) = abap_true.
        result-%create = if_abap_behv=>auth-allowed.
      ELSE.
        result-%create = if_abap_behv=>auth-unauthorized.
        APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                                textid    = /dmo/cm_flight_messages=>not_authorized
                                severity = if_abap_behv_message=>severity-error )
                        %global = if_abap_behv=>mk-on
                      ) TO reported-travel.
      ENDIF.
    ENDIF.
  ENDMETHOD.
ENDCLASS.

```

```

ENDIF.
ENDIF.

"Edit is treated like update
IF requested_authorizations-%update = if_abap_behv=>mk-on OR
   requested_authorizations-%action-Edit = if_abap_behv=>mk-on.

   IF is_update_granted( ) = abap_true.
      result-%update = if_abap_behv=>auth-allowed.
      result-%action-Edit = if_abap_behv=>auth-allowed.

   ELSE.
      result-%update = if_abap_behv=>auth-unauthorized.
      result-%action-Edit = if_abap_behv=>auth-unauthorized.

      APPEND VALUE #( %msg = NEW /dmo/cm_flight_messages(
                           textid = /dmo/cm_flight_messages=>not_authorized
                           severity = if_abap_behv_message=>severity-error )
                     %global = if_abap_behv=>mk-on
                     ) TO reported-travel.

   ENDIF.
ENDIF.

IF requested_authorizations-%delete = if_abap_behv=>mk-on.
   IF is_delete_granted( ) = abap_true.
      result-%delete = if_abap_behv=>auth-allowed.
   ELSE.
      result-%delete = if_abap_behv=>auth-unauthorized.
      APPEND VALUE #( %msg = NEW /dmo/cm_flight_messages(
                           textid = /dmo/cm_flight_messages=>not_authorized
                           severity = if_abap_behv_message=>severity-error )
                     %global = if_abap_behv=>mk-on
                     ) TO reported-travel.
   ENDIF.
ENDIF.

ENDMETHOD.

ENDCLASS.

```

Implementing Instance Authorization

For the managed scenario with draft, authorize users to modify travel instances based on an authorization field.

Define instance authorization in the behavior definition and implement it in the behavior implementation class for **Travel**. For conceptual information, see [Authorization Control](#).

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

Definition

If not already done so, define `authorization master (instance)` for the travel entity and `authorization dependent by _Travel` for the booking and the booking supplement entity in the behavior definition.

If not already done, delegate the authorization control for all actions, except for the internal and draft actions, and the create-by-association to the update operation. The syntax `authorization: update` is used.

Sample Code

```
...
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
...
authorization master ( global, instance )
{...
  association _Booking { create ( features : instance, authorization: update ) ; }
...

  action ( features : instance, authorization: update ) acceptTravel result [1] $self;
  action ( features : instance, authorization: update ) rejectTravel result [1] $self;
  action ( features : instance, authorization: update ) deductDiscount parameter /DMO/A_TRAVEL_
    internal action reCalcTotalPrice;

...}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /DMO/BP_BOOKING_D unique
....
authorization dependent by _Travel
{...
  association _BookingSupplement { create ( authorization: update ); }
...}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_D unique
...
authorization dependent by _Travel
{
  ...
}
```

Via a quick fix, you can generate the method declaration for the authorization control implementation in the behavior pool directly from the behavior definition editor.

Implementation

Instance authorization control must be implemented in the behavior implementation in the method `FOR_INSTANCE_AUTHORIZATION`. You can use the quick fix in the behavior definition for the method declaration. The method imports the parameter `requested_authorizations`, which indicates the operations for which authorization is requested. Like other instance-specific implementations, the method imports the key of the instances, for which authorization is requested. In the implementation, you have to fill the parameter `result`, which must be filled with the authorization result for each requested operation. The method also has the changing parameter `reported`, which is to be filled if you want to return messages for unauthorized cases. For more information, about implementation details, see [Authorization Implementation](#).

The authorization concept in this guide assumes that there can be different authorizations for creating, updating, and deleting instances. All actions of the travel BO are considered with the same authorization as the update operation. With this approach, it is possible that one user is allowed to create a new travel instance, but is not allowed to use actions.

As the authorization for create, update, and delete is the same for global and instance authorization (except for the relevant authorization field for instance authorization), it is best practice to outsource the actual authorization check in separate methods that can be called for all authorization checks that are needed in the BO implementation. The implementation for these methods is explained in detail in [Methods for Authorization Checks](#). Since the instance authorization check is based on a field value, the methods must be adapted to also import a parameter.

The following steps guide you through the implementation.

Methods for Authorization Checks

1. In the methods `is_update_granted` and `is_delete_granted`, add an optional importing parameter to pass the country code of the used agency ID. (The method `is_create_granted` is not used for instance authorization checks as the create operation is not instance specific.)
2. In the method implementation for the corresponding method, differentiate between calls from global and instance authorization by adding an if-block, in which the authorization object is called when the authorization field `country_code` is supplied.
3. Evaluate the authorization for the respective operation based on `sy-subrc` and fill the response parameter of the method.
4. As the authorization which is checked by the authorization object is not configured in the backend, we provide a simulation implementation to allow all users for all operations. Do not use this piece of code in productive applications.
5. To simulate the authorization check against the agency's country code, we provide a simulation implementation to only allow modification on instances that have an agency with country code US. You can change this country code by any other.

Expand the following code sample to view the source code of the methods `is_update_granted`, and `is_delete_granted`.

Sample Code

```
CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

PRIVATE SECTION.

...
METHODS is_update_granted
  IMPORTING country_code          TYPE land1 OPTIONAL
  RETURNING VALUE(update_granted) TYPE abap_bool.
METHODS is_delete_granted
  IMPORTING country_code          TYPE land1 OPTIONAL
  RETURNING VALUE(delete_granted) TYPE abap_bool.

ENDCLASS.
```



```

METHOD is_update_granted.
  "For instance auth
  IF country_code IS SUPPLIED.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
      ID '/DMO/CNTRY' FIELD country_code
      ID 'ACTVT'      FIELD '02'.
    update_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    update_granted = abap_true.

    " simulation of auth check for demo,
    " auth granted for country_code US, else not
    * CASE country_code.
    *   WHEN 'US'.
    *     create_granted = abap_true.
    *   WHEN OTHERS.
    *     create_granted = abap_false.
    *   ENDCASE.

    "For global auth
  ELSE.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
      ID '/DMO/CNTRY' DUMMY
      ID 'ACTVT'      FIELD '02'.
    update_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    update_granted = abap_true.
  ENDIF.

ENDMETHOD.

METHOD is_delete_granted.
  "For instance auth
  IF country_code IS SUPPLIED.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL'
      ID '/DMO/CNTRY' FIELD country_code
      ID 'ACTVT'      FIELD '06'.
    delete_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    delete_granted = abap_true.

    " simulation of auth check for demo,
    " auth granted for country_code US, else not
    * CASE country_code.
    *   WHEN 'US'.
    *     create_granted = abap_true.
    *   WHEN OTHERS.

```

```

*           create_granted = abap_false.
*       ENDCASE.

       "For global auth
ELSE.
    AUTHORITY-CHECK OBJECT '/DMO/TRVL '
        ID '/DMO/CNTRY' DUMMY
        ID 'ACTVT'      FIELD '06'.
    delete_granted = COND #( WHEN sy-subrc = 0 THEN abap_true ELSE abap_false ).

    "Simulation for full authorization
    "(not to be used in productive code)
    delete_granted = abap_true.
ENDIF.

ENDMETHOD.

...

ENDCLASS.

```

Instance Authorization Implementation

In this scenario, the instance authorization is based on the value of the agency's country code that is used in the travel instance. With instance authorization you can only check authorization against the persistent state of the instance. That means, you have to provide the value of the country code as it is on the persistent table (and not the value, which is supplied by the request). In other words, if an update request changes the value of the agency to a value that is not in the authorization scope for this user, the request is not denied during the transaction phase (as the value on the persistent table is one, for which the user is authorized). The request is only denied during the save sequence if a precheck checks the authorization of the incoming value. See [Implementing Prechecks](#).

For instances that don't exist on the persistent database table yet (because they have just been created in the current LUW), authorization is always granted. This also becomes relevant in draft scenarios for edit-drafts.

The following steps guide you through the implementation steps for instance authorization.

1. Read all the instances with the imported keys into an internal table `travels`. As the `AgencyID` is relevant for the authorization, it must be included in the read result.
2. To get the agency's country code of the persistent version of the imported instance keys, select from the persistent table for travels `/DMO/A_TRAVEL_D` with an inner join on the agency table `/DMO/AGENCY`.
3. Loop at the `travels` table to assign the authorization for each instance separately. There are two cases to consider:
 - o For instances that have a persistent state on the database table `/DMO/A_TRAVEL_D`, delegate the authorization check to the relevant method for update or delete and pass the retrieved value for the corresponding country code of the agency. To check if a persistent instance exists, check if an instance with the current travel UUID exists on the select result. If `sy-subrc` returns 0, the read was successful.

If the corresponding authorization check methods deny authorization for the respective operation, fill the `reported` parameter with an adequate message.

 - o For instances that do not have a persistent state on the database table `/DMO/A_TRAVEL_D`, check the authorization for create, as update or delete operations on newly created instances are considered as part of the

create operation as long as the changes are not committed.

4. Fill the result parameter `result` with the determined authorization for each operation.

i Expand the following code sample to view the source code of the method `get_instance_authorizations`.

Sample Code

```

CLASS lhc_travel DEFINITION INHERITING FROM cl_abap_behavior_handler.

    PRIVATE SECTION.

        ...

        METHODS get_instance_authorizations FOR INSTANCE AUTHORIZATION
            IMPORTING keys REQUEST requested_authorizations FOR travel RESULT result.

ENDCLASS.

CLASS lhc_travel IMPLEMENTATION.
    METHOD get_instance_authorizations.

        DATA: update_requested TYPE abap_bool,
              delete_requested TYPE abap_bool,
              update_granted    TYPE abap_bool,
              delete_granted    TYPE abap_bool.

        READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
            ENTITY Travel
                FIELDS ( AgencyID )
                WITH CORRESPONDING #( keys )
        RESULT DATA(travels)
        FAILED failed.

        CHECK travels IS NOT INITIAL.

        "Select country_code and agency of corresponding persistent travel instance
        "authorization only checked against instance that have active persistence
        SELECT FROM /DMO/A_TRAVEL_D AS travel INNER JOIN /DMO/AGENCY AS agency
            ON travel~agency_id = agency~agency_id
            FIELDS travel~travel_uuid , travel~agency_id, agency~country_code
            FOR ALL ENTRIES IN @travels WHERE travel_uuid EQ @travels-TravelUUID
            INTO TABLE @DATA(travel_agency_country).

        "edit is treated like update
        update_requested = COND #( WHEN requested_authorizations-%update           = if_at
                                   requested_authorizations-%action-Edit          = if_at
                                   THEN abap_true ELSE abap_false ).

        delete_requested = COND #( WHEN requested_authorizations-%delete          = if_at
                                   THEN abap_true ELSE abap_false ).

        LOOP AT travels INTO DATA(travel).
            "get country_code of agency in corresponding instance on persistent table

```

```

READ TABLE travel_agency_country WITH KEY travel_uuid = travel-TravelUUID
  ASSIGNING FIELD-SYMBOL(<travel_agency_country_code>).

```

```

"Auth check for active instances that have before image on persistent table
IF sy-subrc = 0.

```

```

  "check auth for update
  IF update_requested = abap_true.
    update_granted = is_update_granted( <travel_agency_country_code>-country_code ).
    IF update_granted = abap_false.
      APPEND VALUE #( %tky = travel-%tky
        %msg = NEW /dmo/cm_flight_messages(
          textid      = /dmo/cm_flight_messages=
          agency_id   = travel-AgencyID
          severity    = if_abap_behv_message=>seve
          %element-AgencyID = if_abap_behv=>mk-on
        ) TO reported-travel.
    ENDIF.
  ENDIF.

```

```

  "check auth for delete
  IF delete_requested = abap_true.
    delete_granted = is_delete_granted( <travel_agency_country_code>-country_code ).
    IF delete_granted = abap_false.
      APPEND VALUE #( %tky = travel-%tky
        %msg = NEW /dmo/cm_flight_messages(
          textid      = /dmo/cm_flight_messages=>not_authorized_f
          agency_id   = travel-AgencyID
          severity    = if_abap_behv_message=>severity-error )
          %element-AgencyID = if_abap_behv=>mk-on
        ) TO reported-travel.
    ENDIF.
  ENDIF.

```

```

  " operations on draft instances and on active instances that have no persistent bel
  " create authorization is checked, for newly created instances
ELSE.

```

```

  update_granted = delete_granted = is_create_granted( ).
  IF update_granted = abap_false.
    APPEND VALUE #( %tky = travel-%tky
      %msg = NEW /dmo/cm_flight_messages(
        textid      = /dmo/cm_flight_messages=>not_authorized
        severity    = if_abap_behv_message=>severity-error )
        %element-AgencyID = if_abap_behv=>mk-on
      ) TO reported-travel.
    ENDIF.
  ENDIF.

```

```

  APPEND VALUE #( LET upd_auth = COND #( WHEN update_granted = abap_true THEN if_abap_t
    ELSE if_abap_behv=>auth-unauthorized )
    del_auth = COND #( WHEN delete_granted = abap_true THEN if_abap_t
    ELSE if_abap_behv=>auth-unauthorized )
  IN
    %tky = travel-%tky

```

```

                                %update                = upd_auth
                                %action-Edit            = upd_auth

                                %delete                = del_auth
                                ) TO result.

        ENDLLOOP.

    ENDMETHOD.

ENDCLASS.

```

Implementing Prechecks

For the managed scenario with draft, add prechecks to check authorization against incoming values to prevent values from reaching the transactional buffer.

Define prechecks in the behavior definition and implement it in the behavior implementation class for Travel. For conceptual information, see [Operation Precheck](#).

Definition

Define the `create` and `update` with the addition (`precheck`). With the addition of `precheck`, the behavior implementation for `precheck` is called, before the main operation is executed. To get an overview of the runtime behavior for `create` and `update` with `precheck` and other standard operations, see [Standard Operations](#). The `precheck` operations receive all entities to be created and returns `failed` and `reported` as changing parameters if the check against the incoming values returns a negative result. For more information about the response parameters, see [Response Parameters](#).

Additionally, add `with additional implementation` to the draft action `Resume`. This is required to trigger a check against the `AgencyID` field in case of an `edit draft`. This additional check is only required for RAP BOs with draft capabilities.

i *Expand the following code sample to view the source code of the behavior definition for `/DMO/R_Travel_D`.*

Sample Code

```

...
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /dmo/bp_travel_d unique
persistent table /dmo/a_travel_d
draft table /dmo/d_travel_d
lock master
total etag LastChangedAt
authorization master ( global, instance )
etag master LocalLastChangedAt
{
    create ( precheck );
    update ( precheck );
    ...
    draft action Resume with additional implementation;
}

```

Via a quick fix, you can generate the method declaration for the prechecks in the behavior pool directly from the behavior definition editor.

Implementation

Prechecks

The authorization to create or save an updated instance with a provided agency ID is checked based on the value of the agency's country/region code. It's only possible to save newly created or updated instance with a certain value of `country_code`. The demo scenario uses US, which can be changed to any other country code. The precheck for the `create` and `update` operation checks whether this condition is fulfilled before a `create` is triggered. Since both operation prechecks check against the same incoming value, a helper method `precheck_auth` was created to streamline the implementation. Both precheck implementations delegate the actual check against the `agencyID` country code to the helper method. Both methods return `failed` and `reported` as changing parameters.

i *Expand the following code sample to view the source code of the methods `precheck_create` **FOR CREATE** and `precheck_update` **FOR UPDATE**.*

Sample Code

```
METHOD precheck_create.
  precheck_auth(
    EXPORTING
      entities_create = entities
    CHANGING
      failed          = failed-travel
      reported        = reported-travel
  ).
ENDMETHOD.

METHOD precheck_update.
  precheck_auth(
    EXPORTING
      entities_update = entities
    CHANGING
      failed          = failed-travel
      reported        = reported-travel
  ).
ENDMETHOD.
```

Method `precheck_auth`

The method `precheck_auth` first differentiates whether it is called from the precheck method `FOR CREATE` or `FOR UPDATE` based on its importing parameters and the `operation` variable. An assert ensures that the method can only be called from one of the precheck implementations and not both at the same time. A `select` retrieves the corresponding `country_code` for the `AgencyID` from the database table. The implementation loops over all entities that were imported and delegates the check against the US value for the `country_code` based on the operation type: If the `operation` variable is flagged as `create`, the method `is_create_granted` is called and received the retrieved `country_code` as importing parameter. In case `operation` is flagged as `update`, the method `is_update_granted` is called and check the `country_code`. If any other value than US is set, the boolean value `is_modify_granted` is set to `abap_false`, meaning that the authorization is denied. In this case, the response parameters `failed` and `reported` are filled.

i Note

In the ABAP Flight Reference Scenario, the authorization objects aren't configured in the back end and thus would prevent any operation for any user. Hence, the code example provides a simulated authorization check without using authorization objects in addition to the check via the authorization object. Don't use the simulation code in productive applications.

i Expand the following code sample to view the source code of the method `precheck_auth`.

≡ Sample Code

```
METHOD precheck_auth.
  DATA:
    entities          TYPE t_entities_update,
    operation          TYPE if_abap_behv=>t_char01,
    agencies           TYPE SORTED TABLE OF /dmo/agency WITH UNIQUE KEY agency_id,
    is_modify_granted TYPE abap_bool.

  " Either entities_create or entities_update is provided. NOT both and at least one.
  ASSERT NOT ( entities_create IS INITIAL EQUIV entities_update IS INITIAL ).

  IF entities_create IS NOT INITIAL.
    entities = CORRESPONDING #( entities_create MAPPING %cid_ref = %cid ).
    operation = if_abap_behv=>op-m-create.
  ELSE.
    entities = entities_update.
    operation = if_abap_behv=>op-m-update.
  ENDIF.

  DELETE entities WHERE %control-AgencyID = if_abap_behv=>mk-off.

  agencies = CORRESPONDING #( entities DISCARDING DUPLICATES MAPPING agency_id = AgencyID EXCE
  CHECK agencies IS NOT INITIAL.
  SELECT FROM /dmo/agency FIELDS agency_id, country_code
          FOR ALL ENTRIES IN @agencies
          WHERE agency_id = @agencies-agency_id
  INTO TABLE @DATA(agency_country_codes).

  LOOP AT entities INTO DATA(entity).
    is_modify_granted = abap_false.

    READ TABLE agency_country_codes WITH KEY agency_id = entity-AgencyID
      ASSIGNING FIELD-SYMBOL(<agency_country_code>).

    "If invalid or initial AgencyID -> validateAgency
    CHECK sy-subrc = 0.
    CASE operation.
      WHEN if_abap_behv=>op-m-create. is_modify_granted = is_create_granted( <agency_country_c
      WHEN if_abap_behv=>op-m-update. is_modify_granted = is_update_granted( <agency_country_c
    ENDCASE.
    IF is_modify_granted = abap_false.
      APPEND VALUE #(
        %cid      = COND #( WHEN operation = if_abap_behv=>op-m-create THEN ent
        %tky      = entity-%tky
      ) TO failed.
```

```

        APPEND VALUE #(
            %cid      = COND #( WHEN operation = if_abap_behv=>op-m-create THEN ent
            %tky      = entity-%tky
            %msg      = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>not_author
                                agency_id = entity-AgencyID
                                severity   = if_abap_behv_message=>severity-erro
            %element-AgencyID = if_abap_behv=>mk-on
        ) TO reported.
    ENDIF.
ENDLOOP.

ENDMETHOD.

```

Resume with Additional Implementation

The draft capabilities of the BO require an additional check implementation for an `edit draft` use case. If a draft has expired and the `AgencyID` was changed and the user comes back to work on the instance, the precheck implementation must be triggered again to ensure the consistency of the authority check against the `country_code` value. The implementation read the `AgencyID` of the travel. Then the `%control-AgencyID` is set to true to trigger the precheck implementation. An `edit draft` scenario is treated as update in case of the precheck, so the helper method `precheck_auth` is called with the exporting parameter `entities_update`. The helper method treats this call as an update in its implementation.

i Expand the following code sample to view the source code of the method **resume**.

Sample Code

```

METHOD resume.
    DATA:
        entities_update TYPE t_entities_update.

    READ ENTITIES OF /dmo/r_travel_d IN LOCAL MODE
    ENTITY Travel
        FIELDS ( AgencyID )
        WITH VALUE #(
            FOR key IN keys
                %is_draft = if_abap_behv=>mk-on
                ( %key = key-%key )
            )
        RESULT DATA(travels).

    " Set %control-AgencyID (if set) to true, so that the precheck_auth checks the permissions.
    entities_update = CORRESPONDING #( travels CHANGING CONTROL ).

    IF entities_update IS NOT INITIAL.
        precheck_auth(
            EXPORTING
                entities_update = entities_update
            CHANGING
                failed          = failed-travel
                reported        = reported-travel
        ).
    
```



```
ENDIF.  
ENDMETHOD.
```

Developing Validations

For the managed scenario with draft, add validations to check the values provided by the client.

Context and Procedure

Validations are used to check whether provided values by a client are consistent. They give direct feedback (messages) before the BO instance is saved to the database. For more detailed information, see [Validations](#).

i Note

You can assign validations to determine actions that are called by RAP side effects. This enables immediate feedback after the user changes UI fields or field groups. For more detailed information, see [Action Definition](#).

As validations verify the state of an instance, state messages are used to return messages.

i Note

State messages reflect the business object state. Since validations check for data inconsistencies, they return state messages. State messages are stored on the database and can be retrieved together with the BO instance they relate to. In draft scenarios, state messages are used to describe data inconsistencies. As state messages are persisted, they must be invalidated in the implementation. For more information, refer to [State Messages](#).

For messages that relate to a change of a BO instance, transition messages are used. Every modify request is a change of state. If a modify requests fails, transition messages are used to describe and explain the failure. Transition messages are not saved on a database. They disappear automatically once the failed operation is rolled back. For more information, refer to [Transition Messages](#).

For the travel demo scenario, define and implement validation for all three BO entities:

- For the **travel entity**, the following validations are described:

- `validateCustomer`

The provided customer ID is checked against the entries in `/DMO/CUSTOMER`. Saving instances with invalid or initial values for the customer ID is rejected. In such a case, the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `CustomerID`.

- `validateAgency`

The provided agency ID is checked against the entries in `/DMO/AGENCY`. Saving instances with invalid or initial values for the agency ID is rejected. In such a case, the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `AgencyID`.

- `validateDates`

The provided starting date is checked against the system date. It cannot be earlier than the system date. In addition, the starting and the end date are compared against it each other. The end date cannot be before the start date. If one of the conditions is not fulfilled, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger fields `BeginDate` and `EndDate`.

- `validateCurrencyCode`

The provided currency code is checked against the entries in `I_Currency`. Saving instances with invalid or initial values for the currency code is rejected. In such a case, the validation returns failed keys and a message

The validation is triggered on `create` and on every update of the trigger field `CurrencyCode`.

- `validateBookingFee`

The validation checks whether the provided booking fee is less than 0. If this is the case, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `BookingFee`.

- For the **booking entity**, the following validations are described:

- `validateCustomer`

See [validateCustomer](#) of travel entity.

- `validateConnection`

The provided values for `AirlineID`, `ConnectionID` and `FlightDate` are checked against the flight data in `/DMO/FLIGHT`. Only existing flights with valid connection and airline IDs can be added to the booking instance. If the condition is not fulfilled, the validation returns failed keys and a message.

The validation is triggered on `create` and every update of the trigger fields `AirlineID`, `ConnectionID`, and `FlightDate`.

- `validateCurrencyCode`

The provided currency code is checked against the entries in `I_Currency`. Saving instances with invalid or initial values for the currency code is rejected. In such a case, the validation returns failed keys and a message

The validation is triggered on `create` and on every update of the trigger field `CurrencyCode`.

- `validateFlightPrice`

The validation checks whether the provided flight price is less than 0. If this is the case, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `FlightPrice`.

- `validateStatus`

The validation checks whether the provided booking status matches one of the predefined values. If this is not the case, the saving is rejected and the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `BookingStatus`.

- For the **booking supplement entity**, the following validation is described:

- `validateSupplement`

The provided supplement ID is checked against the entries in `/DMO/SUPPLEMENT`. Saving instances with invalid or initial values for the supplement ID is rejected. In such a case, the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `SupplementID`.

- `validateCurrencyCode`

The provided currency code is checked against the entries in `I_Currency`. Saving instances with invalid or initial values for the currency code is rejected. In such a case, the validation returns failed keys and a message

The validation is triggered on `create` and on every update of the trigger field `CurrencyCode`.

- **validateSupplementPrice**

The validation checks whether the entered value is less than 0. In such a case, the validation returns failed keys and a message.

The validation is triggered on `create` and on every update of the trigger field `BookSupplPrice`.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Validating Values for the Travel Entity](#)

For the managed scenario with draft, define and implement validations on the travel entity for value consistency checks.

[Validating Values for the Booking Entity](#)

For the managed scenario with draft, define and implement validations on the booking entity for value consistency checks.

[Validating Values for the Booking Supplement Entity](#)

For the managed scenario with draft, define and implement validations on the booking supplement entity for value consistency checks.

Validating Values for the Travel Entity

For the managed scenario with draft, define and implement validations on the travel entity for value consistency checks.

Validation **validateCustomer**

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CustomerID`. Since there must always be a customer assigned to a certain travel, define the field `CustomerID` as mandatory.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CustomerID;
  validation validateCustomer on save { create; field CustomerID; }

... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the customer ID by checking if the provided value is the ID of an entry in the customer database table. Raise adequate messages for the consumer if the value is initial or not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Copy the entries to another internal table and delete all instances with an initial customer ID. If the resulting internal table is not initial, select the entries of `/DMO/CUSTOMER` with the given customer ID. If there is no corresponding entry in this database table, the provided customer ID is not valid.
3. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the customer ID is initial, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CUSTOMER`. The RAP runtime framework provides the method `new_message`, with which you can easily retrieve messages from message classes provide the respective parameters. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.
5. If there is no entry in the customer master data table, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CUSTOMER`.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of *Validation validateCustomer*.

Sample Code

```
METHOD validateCustomer.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( CustomerID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DATA customers TYPE SORTED TABLE OF /DMO/CUSTOMER WITH UNIQUE KEY customer_id.

  " Optimization of DB select: extract distinct non-initial customer IDs
  customers = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING customer_id = CustomerID
  DELETE customers WHERE customer_id IS INITIAL.

  IF customers IS NOT INITIAL.
    " Check if customer ID exists
    SELECT FROM /DMO/CUSTOMER FIELDS customer_id
      FOR ALL ENTRIES IN @customers
      WHERE customer_id = @customers-customer_id
    INTO TABLE @DATA(valid_customers).
  ENDIF.

  " Raise message for non existing customer id
  LOOP AT travels INTO DATA(travel).

    APPEND VALUE #( %tky          = travel-%tky
                   %state_area    = 'VALIDATE_CUSTOMER'
                   ) TO reported-travel.

    IF travel-CustomerID IS INITIAL.
      APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

      APPEND VALUE #( %tky          = travel-%tky
                   %state_area    = 'VALIDATE_CUSTOMER'
                   %msg           = NEW /dmo/cm_flight_messages(
                                     textid   = /dmo/cm_flight_messa
                                     severity = if_abap_behv_message=
                                     %element-CustomerID = if_abap_behv=>mk-on
                                   ) TO reported-travel.
```

```

ELSEIF travel-CustomerID IS NOT INITIAL AND NOT line_exists( valid_customers[ customer_id
  APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

  APPEND VALUE #( %tky          = travel-%tky
                  %state_area   = 'VALIDATE_CUSTOMER'
                  %msg          = NEW /dmo/cm_flight_messages(
                                customer_id = travel-customerid
                                textid      = /dmo/cm_flight_me
                                severity    = if_abap_behv_messa
                                %element-CustomerID = if_abap_behv=>mk-on
                                ) TO reported-travel.
ENDIF.

ENDLOOP.
ENDMETHOD.

```

Validation **validateAgency**

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `AgencyID`. Since there must always be an agency assigned to a certain travel, define the field `AgencyID` as mandatory.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CustomerID, AgencyID;
  validation validateAgency on save { create; field AgencyID; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the agency ID by checking if the provided value is the ID of an entry in the agency database table. Raise adequate messages for the consumer if the value is initial or not valid. The implementation steps are exactly the same as for the validation `ValidateCustomer`.

i *Expand the following code sample to view the source code of Validation **validateAgency**.*

Sample Code

```

METHOD validateAgency.
  DATA: modification_granted TYPE abap_boolean,
        agency_country_code  TYPE land1.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( AgencyID TravelID )
      WITH CORRESPONDING #( keys )
      RESULT DATA(travels).

  DATA agencies TYPE SORTED TABLE OF /dmo/agency WITH UNIQUE KEY agency_id.

```

```

" Optimization of DB select: extract distinct non-initial agency IDs
agencies = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING agency_id = AgencyID EXCEPT
DELETE agencies WHERE agency_id IS INITIAL.

IF agencies IS NOT INITIAL.
" Check if Agency ID exists
SELECT FROM /dmo/agency FIELDS agency_id, country_code
FOR ALL ENTRIES IN @agencies
WHERE agency_id = @agencies-agency_id
INTO TABLE @DATA(valid_agencies).
ENDIF.

" Raise message for non existing Agency id
LOOP AT travels INTO DATA(travel).
  APPEND VALUE #( %tky          = travel-%tky
                  %state_area   = 'VALIDATE_AGENCY'
                  ) TO reported-travel.

  IF travel-AgencyID IS INITIAL.
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

    APPEND VALUE #( %tky          = travel-%tky
                    %state_area   = 'VALIDATE_AGENCY'
                    %msg          = NEW /dmo/cm_flight_messages(
                                textid   = /dmo/cm_flight_messages=>en
                                severity = if_abap_behv_message=>severity
                                %element-AgencyID = if_abap_behv=>mk-on
                                ) TO reported-travel.

  ELSEIF travel-AgencyID IS NOT INITIAL AND NOT line_exists( valid_agencies[ agency_id = travel-agencyid ] )
    APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

    APPEND VALUE #( %tky          = travel-%tky
                    %state_area   = 'VALIDATE_AGENCY'
                    %msg          = NEW /dmo/cm_flight_messages(
                                agency_id = travel-agencyid
                                textid    = /dmo/cm_flight_messages=>en
                                severity  = if_abap_behv_message=>severity
                                %element-AgencyID = if_abap_behv=>mk-on
                                ) TO reported-travel.

  ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation validateDates

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation `create` and trigger fields `BeginDate` and `EndDate`. Since the travel dates are an essential part of the travel data, define the fields `BeginDate` and `EndDate` as mandatory.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  field ( mandatory ) CustomerID, AgencyID, BeginDate, EndDate;
  validation validateDates on save { create; field BeginDate, EndDate; }
}

```

```
... }
```

Implementation

Validate the travel dates by comparing the value of the `BeginDate` with the `EndDate`. The end date must not be before the begin date and the begin date must be after the system date. Raise adequate messages for the consumer if the provided values are not valid. The following steps guide you through the implementation. The messages for the managed scenario with draft are stored in the message class `/DMO/CM_FLIGHT` that is part of the ABAP Flight Reference Scenario.

1. Read all the instances with the imported keys into an internal table `t travels`. This table is the basis to check whether messages must be raised.
2. Loop over the internal table `t travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
3. The travel date must fulfill four conditions:
 - a. Begin date must not be initial.
 - b. End date must not be initial.
 - c. Begin date must not be before system date.
 - d. End date must not be before begin date.

For all of the situations, check with an `if` - loop and append the transactional key to the failed table. In addition, append the corresponding message to the state area `VALIDATE_DATES`. The RAP runtime framework provides the method `new message`, with which you can easily retrieve messages from message classes provide the respective parameters. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.

i Expand the following code sample to view the source code of Validation ***validateDates***.

Sample Code

```
METHOD validateDates.

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( BeginDate EndDate TravelID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

    LOOP AT travels INTO DATA(travel).

      APPEND VALUE #( %tky          = travel-%tky
                     %state_area   = 'VALIDATE_DATES' ) TO reported-travel.

      IF travel-BeginDate IS INITIAL.
        APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

        APPEND VALUE #( %tky          = travel-%tky
                       %state_area   = 'VALIDATE_DATES'
                       %msg          = NEW /dmo/cm_flight_messages(
                                      textid   = /dmo/cm_flight_message
                                      severity = if_abap_behv_message=
                                      %element-BeginDate = if_abap_behv=>mk-on ) TO reported-travel.
      ENDIF.
      IF travel-EndDate IS INITIAL.
        APPEND VALUE #( %tky = travel-%tky ) TO failed-travel.

        APPEND VALUE #( %tky          = travel-%tky
```

```

        %state_area      = 'VALIDATE_DATES'
        %msg              = NEW /dmo/cm_flight_messages(
                                textid    = /dmo/cm_flight_messag
                                severity = if_abap_behv_message=
        %element-EndDate  = if_abap_behv=>mk-on ) T0 reported-travel.
    ENDIF.
    IF travel-EndDate < travel-BeginDate AND travel-BeginDate IS NOT INITIAL
        AND travel-EndDate IS NOT INITIAL.
        APPEND VALUE #( %tky = travel-%tky ) T0 failed-travel.

        APPEND VALUE #( %tky      = travel-%tky
                        %state_area = 'VALIDATE_DATES'
                        %msg        = NEW /dmo/cm_flight_messages(
                                    textid    = /dmo/cm_flight_mess
                                    begin_date = travel-BeginDate
                                    end_date   = travel-EndDate
                                    severity   = if_abap_behv_messag
                        %element-BeginDate = if_abap_behv=>mk-on
                        %element-EndDate  = if_abap_behv=>mk-on ) T0 reported-travel.
    ENDIF.
    IF travel-BeginDate < cl_abap_context_info=>get_system_date( ) AND travel-BeginDate IS NOT
        APPEND VALUE #( %tky      = travel-%tky ) T0 failed-travel.

        APPEND VALUE #( %tky      = travel-%tky
                        %state_area = 'VALIDATE_DATES'
                        %msg        = NEW /dmo/cm_flight_messages(
                                    begin_date = travel-BeginDate
                                    textid     = /dmo/cm_flight_mess
                                    severity   = if_abap_behv_messag
                        %element-BeginDate = if_abap_behv=>mk-on ) T0 reported-travel.
    ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation validateCurrencyCode

Define the validation in the behavior definition and implement it in the behavior implementation class for [Travel](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CurrencyCode`. Since the currency code must always be provided for a booking, define the field `CurrencyCode` as mandatory.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
    field ( mandatory ) CurrencyCode;
    validation validateCurrencyCode on save { create; field CurrencyCode; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the currency code by checking if the provided value is the ID of an entry in the currency database table. Raise adequate messages for the consumer if the value is initial or not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Copy the entries to another internal table and delete all instances with an initial currency code. If the resulting internal table is not initial, select the entries of `I_Currency` with the given currency code. If there is no corresponding entry in this database table, the provided currency code is not valid.
3. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the currency code is initial, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CURRENCYCODE`. The RAP runtime framework provides the method `new_message`, with which you can easily retrieve messages from message classes provide the respective parameters. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.
5. If there is no entry in the currency code master data table, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CURRENCYCODE`.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation `validateCurrencyCode`.

Sample Code

```
METHOD validateCustomer.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( CustomerID )
      WITH CORRESPONDING #( keys )
    RESULT DATA(travels).

  DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

  currencies = CORRESPONDING #( travels DISCARDING DUPLICATES MAPPING currency = currencycode
  DELETE currencies WHERE currency IS INITIAL.

  IF currencies IS NOT INITIAL.
    SELECT FROM I_Currency FIELDS currency
      FOR ALL ENTRIES IN @currencies
      WHERE currency = @currencies-currency
      INTO TABLE @DATA(currency_db).
  ENDIF.

  LOOP AT travels INTO DATA(travel).
    APPEND VALUE #( %tky          = travel-%tky
                   %state_area   = 'VALIDATE_CURRENCYCODE'
                   ) TO reported-travel.
    IF travel-currencycode IS INITIAL.
      " Raise message for empty Currency
      APPEND VALUE #( %tky          = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky          = travel-%tky
                     %state_area   = 'VALIDATE_CURRENCYCODE'
                     %msg          = NEW /dmo/cm_flight_messages(
                                     textid    = /dmo/cm_flight_messages=>cu
                                     severity  = if_abap_behv_message=>severi
                                     %element-currencycode = if_abap_behv=>mk-on
                                   ) TO reported-travel.
    ELSEIF NOT line_exists( currency_db[ currency = travel-currencycode ] ).
      " Raise message for not existing Currency
      APPEND VALUE #( %tky          = travel-%tky ) TO failed-travel.
      APPEND VALUE #( %tky          = travel-%tky
                     %state_area   = 'VALIDATE_CURRENCYCODE'
```

```

%msg                                = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages:
                                severity    = if_abap_behv_message=>se
                                currency_code = travel-currencycode )
%element-currencycode = if_abap_behv=>mk-on
) TO reported-travel.

ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation **validateBookingFee**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Travel**.

Definition

Define a validation on save with trigger operation **create** and trigger field **BookingFee**.

Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
validation validateBookingFee    on save { create; field BookingFee; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the booking fee by checking if the entered value is less than 0. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `travels`. This table is the basis to check whether messages must be raised.
2. Loop over the internal table `travels`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
3. If the booking fee is less than 0, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_BOOKINGFEE` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i *Expand the following code sample to view the source code of Validation **validateBookingFee**.*

Sample Code

```

METHOD validateBookingFee.

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY travel

```

```

        FIELDS ( BookingFee )
        WITH CORRESPONDING #( keys )
RESULT DATA(travels).

LOOP AT travels INTO DATA(travel).
APPEND VALUE #( %tky                = travel-%tky
                %state_area         = 'VALIDATE_BOOKINGFEE'
                ) TO reported-travel.
" Raise message for booking fee < 0
IF travel-BookingFee < 0.
    APPEND VALUE #( %tky                = travel-%tky ) TO failed-travel.
    APPEND VALUE #( %tky                = travel-%tky
                    %state_area         = 'VALIDATE_BOOKINGFEE'
                    %msg                = NEW /dmo/cm_flight_messages(
                                            textid      = /dmo/cm_flight_messages=>boo
                                            severity    = if_abap_behv_message=>severi
                                            %element-BookingFee = if_abap_behv=>mk-on
                                        ) TO reported-travel.
ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validating Values for the Booking Entity

For the managed scenario with draft, define and implement validations on the booking entity for value consistency checks.

Validation `validateCustomer`

Define the validation in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CustomerID`. Since there must always be a customer assigned to a certain travel, define the field `CustomerID` is mandatory.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
    field ( mandatory ) CustomerID;
    validation validateCustomer on save { create; field CustomerID; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

The validation for the customer ID field is done similarly to the validation for the customer ID in the travel entity. See [Validation `validateCustomer`](#).

To retrieve the correct messages of validations in child entities and to display them on the UI, you have to define a path expression to the root entity. To fill the path expression with the correct keys, you have to read the keys of the parent entity first. This is done by a read by association. You do not need the full result of the read by association, but just the link table, which contains the key mappings of source and target entity.

i Expand the following code sample to view the source code of Validation **validateCustomer**.

≡ Sample Code

```

METHOD validateCurrencyCode.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY booking
    FIELDS ( currencycode )
    WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_Travel
    FROM CORRESPONDING #( bookings )
  LINK DATA(travel_booking_links).

DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

currencies = CORRESPONDING #( bookings DISCARDING DUPLICATES MAPPING currency = currencycode
DELETE currencies WHERE currency IS INITIAL.

IF currencies IS NOT INITIAL.
  SELECT FROM I_Currency FIELDS currency
    FOR ALL ENTRIES IN @currencies
    WHERE currency = @currencies-currency
    INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT bookings INTO DATA(booking).
  APPEND VALUE #( %tky = booking-%tky
                  %state_area = 'VALIDATE_CURRENCYCODE'
                  ) TO reported-booking.
  IF booking-currencycode IS INITIAL.
    " Raise message for empty Currency
    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                  %state_area = 'VALIDATE_CURRENCYCODE'
                  %msg = NEW /dmo/cm_flight_messages(
                        textid = /dmo/cm_flight_messages=>cu
                        severity = if_abap_behv_message=>severi
                        %path = VALUE #( travel-%tky = travel_booking_links[ KE
                        %element-currencycode = if_abap_behv=>mk-on
                        ) TO reported-booking.
  ELSEIF NOT line_exists( currency_db[ currency = booking-currencycode ] ).
    " Raise message for not existing Currency
    APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky = booking-%tky
                  %state_area = 'VALIDATE_CURRENCYCODE'
                  %msg = NEW /dmo/cm_flight_messages(
                        textid = /dmo/cm_flight_messages=>se
                        severity = if_abap_behv_message=>se
                        currency_code = booking-currencycode )
                        %path = VALUE #( travel-%tky = travel_booking_links[ KE
                        %element-currencycode = if_abap_behv=>mk-on
                        ) TO reported-booking.
  ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation **validateConnection**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Booking**.

Definition

Define a validation on save with trigger operation `create` and trigger fields `AirlineID`, `ConnectionID`, and `FlightDate`. Since a booking instance must always entail this flight information, define these fields as mandatory.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
  field ( mandatory ) CustomerID, AirlineID, ConnectionID, FlightDate;
  validation validateConnection on save { create; field AirlineID, ConnectionID, FlightDate; }

... }
```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the flight-related fields by checking if the provided values match an entry in the master database table `/DMO/FLIGHT`. Raise adequate messages for the consumer if the values are initial or not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `bookings`. This table is the basis to check whether messages must be raised.
2. Read the corresponding travel instances via a read by association. The parent keys are needed for the path expression `%path` in the reported table. It represents the path the path to the root entity and is important to display the messages correctly on the UI.
3. Loop over the internal table `bookings`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the airline ID is initial, write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CONNECTION`.

Messages are created with the `/dmo/cm_flight_messages` message exception class, with which you can easily retrieve messages from message classes and provide the respective parameters.

The reported table contains the component `%path`, in which you have to maintain the keys of all parent instances, including the root entity. Use the key mappings in the link table to fill the path expression. Only if all parent keys are maintained is the message displayed correctly on the UI.

The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI, evaluates this component and highlights the corresponding input field.

5. Repeat the step for the values in `ConnectionID` and `FlightDate`. Like `AirlineID`, they must not be initial.
6. If all related fields are not empty you can check if the values match an entry in the flight master data table. If the select finds a corresponding row (`=sy-subrc <> 0`), write the transactional key to the failed table and append the corresponding message to the state area `VALIDATE_CONNECTION`.

i *Expand the following code sample to view the source code of Validation **validateConnection**.*

Sample Code

```
METHOD validateConnection.

  READ ENTITIES OF /DMO/R Travel D IN LOCAL MODE
```

```

ENTITY Booking
  FIELDS ( BookingID AirlineID ConnectionID FlightDate )
  WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_Travel
  FROM CORRESPONDING #( bookings )
  LINK DATA(travel_booking_links).

LOOP AT bookings ASSIGNING FIELD-SYMBOL(<booking>).
  "overwrite state area with empty message to avoid duplicate messages
  APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION' ) TO reported-booking.

  " Raise message for non existing airline ID
  IF <booking>-AirlineID IS INITIAL.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                                %path   = VALUE #( travel-%tky = travel_booking_links[ KEY id
                                %element-AirlineID = if_abap_behv=>mk-on
                                ) TO reported-booking.
                  ) TO reported-booking.
  ENDIF.

  " Raise message for non existing connection ID
  IF <booking>-ConnectionID IS INITIAL.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                                %path   = VALUE #( travel-%tky = travel_booking_links[ KEY i
                                %element-ConnectionID = if_abap_behv=>mk-on
                                ) TO reported-booking.
                  ) TO reported-booking.
  ENDIF.

  " Raise message for non existing flight date
  IF <booking>-FlightDate IS INITIAL.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_message
                                severity = if_abap_behv_message=
                                %path   = VALUE #( travel-%tky = travel_booking_links[ KEY i
                                %element-FlightDate = if_abap_behv=>mk-on
                                ) TO reported-booking.
                  ) TO reported-booking.
  ENDIF.

  " check if flight connection exists
  IF <booking>-AirlineID IS NOT INITIAL AND
    <booking>-ConnectionID IS NOT INITIAL AND
    <booking>-FlightDate IS NOT INITIAL.

    SELECT SINGLE Carrier_ID, Connection_ID, Flight_Date FROM /DMO/FLIGHT WHERE carrier_
    AND connection_id = @<booking>-C
    AND flight_date   = @<booking>-F
    INTO @DATA(flight).

  IF sy-subrc <> 0.
    APPEND VALUE #( %tky = <booking>-%tky ) TO failed-booking.

    APPEND VALUE #( %tky          = <booking>-%tky
                  %state_area   = 'VALIDATE_CONNECTION'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid   = /dmo/cm_flight_me
                                carrier_id = <booking>-AirlineI
                                flight_date = <booking>-FlightDa
                                severity   = if_abap_behv_messa

```

```

        %path                = VALUE #( travel-%tky = travel_booking_links[
        %element-FlightDate   = if_abap_behv=>mk-on
        %element-AirlineID    = if_abap_behv=>mk-on
        %element-ConnectionID = if_abap_behv=>mk-on
    ) TO reported-booking.

```

```
ENDIF.
```

```
ENDIF.
```

```
ENDLOOP.
```

```
ENDMETHOD.
```

Validation `validateCurrencyCode`

Define the validation in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CurrencyCode`. Since there must always be a currency code assigned to a booking price, define the field `CurrencyCode` as mandatory.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
    field ( mandatory ) CurrencyCode;
    validation validateCurrencyCode on save { create; field CurrencyCode; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

The validation for the currency code field is done similarly to the validation for the currency code in the travel entity. See [Validation `validateCurrencyCode`](#).

To retrieve the correct messages of validations in child entities and to display them on the UI, you have to define a path expression to the root entity. To fill the path expression with the correct keys, you have to read the keys of the parent entity first. This is done by a read by association. You do not need the full result of the read by association, but just the link table, which contains the key mappings of source and target entity.

i *Expand the following code sample to view the source code of Validation `validateCustomer`.*

Sample Code

```

METHOD validateCurrencyCode.
    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY booking
        FIELDS ( currencycode )
        WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

    DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

```

```

currencies = CORRESPONDING #( bookings DISCARDING DUPLICATES MAPPING currency = currencycode
DELETE currencies WHERE currency IS INITIAL.

IF currencies IS NOT INITIAL.
  SELECT FROM I_Currency FIELDS currency
  FOR ALL ENTRIES IN @currencies
  WHERE currency = @currencies-currency
  INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT bookings INTO DATA(booking).
  APPEND VALUE #( %tky          = booking-%tky
                  %state_area   = 'VALIDATE_CURRENCYCODE'
                  ) TO reported-booking.
  IF booking-currencycode IS INITIAL.
    " Raise message for empty Currency
    APPEND VALUE #( %tky          = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky          = booking-%tky
                  %state_area   = 'VALIDATE_CURRENCYCODE'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>cl
                                severity    = if_abap_behv_message=>severity
                                %element-currencycode = if_abap_behv=>mk-on
                                ) TO reported-booking.
  ELSEIF NOT line_exists( currency_db[ currency = booking-currencycode ] ).
    " Raise message for not existing Currency
    APPEND VALUE #( %tky          = booking-%tky ) TO failed-booking.
    APPEND VALUE #( %tky          = booking-%tky
                  %state_area   = 'VALIDATE_CURRENCYCODE'
                  %msg          = NEW /dmo/cm_flight_messages(
                                textid      = /dmo/cm_flight_messages=>cl
                                severity    = if_abap_behv_message=>severity
                                currency_code = booking-currencycode )
                                %element-currencycode = if_abap_behv=>mk-on
                                ) TO reported-booking.
  ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation `validateFlightPrice`

Define the validation in the behavior definition and implement it in the behavior implementation class for [Booking](#).

Definition

Define a validation on save with trigger operation create and trigger field `FlightPrice`.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
  validation validateFlightPrice on save { create; field FlightPrice; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the flight price by checking if the entered value is less than 0. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `bookings`. This table is the basis to check whether messages must be raised.
2. Read the keys of the corresponding parent instances using the association `_Travel` and store the result in the `link` response. This information will be needed for linking booking instances to travel instances in the reported response.
3. Loop over the internal table `bookings`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the flight price is less than 0, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_FLIGHTPRICE` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field. Use the `travel_booking_links` table populated above to provide the keys of the corresponding travel instances for each booking instance.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation **validateFlightPrice**.

Sample Code

```
METHOD validateFlightPrice.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY booking
      FIELDS ( FlightPrice )
      WITH CORRESPONDING #( keys )
    RESULT DATA(bookings).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Booking BY \_Travel
      FROM CORRESPONDING #( bookings )
    LINK DATA(travel_booking_links).

  LOOP AT bookings INTO DATA(booking).
    APPEND VALUE #( %tky = booking-%tky
                    %state_area = 'VALIDATE_FLIGHTPRICE'
                    ) TO reported-booking.
    " Raise message for flight price < 0
    IF booking-FlightPrice < 0.
      APPEND VALUE #( %tky = booking-%tky ) TO failed-booking.
      APPEND VALUE #( %tky = booking-%tky
                      %state_area = 'VALIDATE_FLIGHTPRICE'
                      %msg = NEW /dmo/cm_flight_messages(
                                textid = /dmo/cm_flight_messages=>fl
                                severity = if_abap_behv_message=>sever
                                %element-FlightPrice = if_abap_behv=>mk-on
                                %path = VALUE #( travel-%tky = travel_booking_links[ KEY
                                ) TO reported-booking.
    ENDIF.
  ENDLOOP.

ENDMETHOD.
```

Validation **validateStatus**

Define the validation in the behavior definition and implement it in the behavior implementation class for **Booking**.

Definition

Define a validation on save with trigger operation `create` and trigger field `BookingStatus`.

Sample Code

```

define behavior for /DMO/R_Booking_D alias Booking
...
{ ...
  validation validateStatus on save { create; field BookingStatus; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the booking status by checking if it matches one of the predefined values. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `bookings`. This table is the basis to check whether messages must be raised.
2. Read the keys of the corresponding parent instances using the association `_Travel` and store the result in the `link` response. This information will be needed for linking booking instances to travel instances in the reported response.
3. Loop over the internal table `bookings`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the booking status differs from the predefined statuses, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_STATUS` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field. Use the `travel_booking_links` table populated above to provide the keys of the corresponding travel instances for each booking instance.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i *Expand the following code sample to view the source code of Validation `validateStatus`.*

Sample Code

```

METHOD validateStatus.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY booking
  FIELDS ( BookingStatus )
  WITH CORRESPONDING #( keys )
  RESULT DATA(bookings).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
  ENTITY Booking BY \_Travel
  FROM CORRESPONDING #( bookings )
  LINK DATA(travel_booking_links).

  LOOP AT bookings INTO DATA(booking).
    APPEND VALUE #( %tky = booking-%tky

```

```

                                %state_area      = 'VALIDATE_STATUS'
                                ) T0 reported-booking.
CASE booking-BookingStatus.
WHEN 'N'.  " New
WHEN 'X'.  " Canceled
WHEN 'B'.  " Booked

WHEN OTHERS.
  APPEND VALUE #( %tky = booking-%tky ) T0 failed-booking.
  APPEND VALUE #( %tky = booking-%tky
                  %state_area      = 'VALIDATE_STATUS'
                  %msg              = NEW /dmo/cm_flight_messages(
                                      textid    = /dmo/cm_flight_messages=>st
                                      status    = booking-BookingStatus
                                      severity = if_abap_behv_message=>sever
                                      %element-BookingStatus = if_abap_behv=>mk-on
                                      %path              = VALUE #( travel-%tky = travel_booking_links[
                                ) T0 reported-booking.

ENDCASE.
ENDLOOP.

ENDMETHOD.

```

Validating Values for the Booking Supplement Entity

For the managed scenario with draft, define and implement validations on the booking supplement entity for value consistency checks.

Validation `validateSupplement`

Define the validation in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `SupplementID`. Since a booking supplement instance always needs a supplement, define the field `SupplementID` as mandatory.

Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  field ( mandatory ) SupplementID
  validation validateSupplement on save { create; field SupplementID; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the supplement ID by checking if the provided value is an entry in the supplement database table. Raise adequate messages for the consumer if the value is initial or not valid. The implementation steps are exactly the same as for the validation `ValidateCustomer`. For a detailed step-by-step description, see [Validation validateCustomer](#).

To fill the %path component in the reported table, you have to provide the keys of all parent instances. Therefore you have to execute a read by association to every entity in the business object structure. In this case, to [Booking](#) and to [BookingSupplement](#).

i Expand the following code sample to view the source code of Validation `validateSupplement`.

Sample Code

```
METHOD validateSupplement.

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
      ENTITY BookingSupplement
      FIELDS ( SupplementID )
      WITH CORRESPONDING #( keys )
      RESULT DATA(bookingsupplements)
      FAILED DATA(read_failed).

    failed = CORRESPONDING #( DEEP read_failed ).

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
      ENTITY BookingSupplement BY \_Booking
      FROM CORRESPONDING #( bookingsupplements )
      LINK DATA(booksuppl_booking_links).

    READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
      ENTITY BookingSupplement BY \_Travel
      FROM CORRESPONDING #( bookingsupplements )
      LINK DATA(booksuppl_travel_links).

    DATA supplements TYPE SORTED TABLE OF /DMO/SUPPLEMENT WITH UNIQUE KEY supplement_id.

    " Optimization of DB select: extract distinct non-initial customer IDs
    supplements = CORRESPONDING #( bookingsupplements DISCARDING DUPLICATES MAPPING supplement_i
    DELETE supplements WHERE supplement_id IS INITIAL.

    IF supplements IS NOT INITIAL.
      " Check if customer ID exists
      SELECT FROM /DMO/SUPPLEMENT FIELDS supplement_id
          FOR ALL ENTRIES IN @supplements
          WHERE supplement_id = @supplements-supplement_id
      INTO TABLE @DATA(valid_supplements).
    ENDIF.

    LOOP AT bookingsupplements ASSIGNING FIELD-SYMBOL(<bookingsupplement>).

      APPEND VALUE #( %tky          = <bookingsupplement>-%tky
                     %state_area = 'VALIDATE_SUPPLEMENT'
                     ) TO reported-bookingsupplement.

      IF <bookingsupplement>-SupplementID IS INITIAL.
        APPEND VALUE #( %tky = <bookingsupplement>-%tky ) TO failed-bookingsupplement.

        APPEND VALUE #( %tky          = <bookingsupplement>-%tky
                       %state_area    = 'VALIDATE_SUPPLEMENT'
                       %msg           = NEW /dmo/cm_flight_messages(
                                       textid = /dmo/cm_flight_message
                                       severity = if_abap_behv_message=
                                       %path      = VALUE #( booking-%tky = booksuppl_booking_links[
                                       travel-%tky = booksuppl_travel_links[
                                       %element-SupplementID = if_abap_behv=>mk-on
                                       ) TO reported-bookingsupplement.

      ELSEIF <bookingsupplement>-SupplementID IS NOT INITIAL AND NOT line_exists( valid_suppleme
        APPEND VALUE #( %tky = <bookingsupplement>-%tky ) TO failed-bookingsupplement.

        APPEND VALUE #( %tky          = <bookingsupplement>-%tky
                       %state_area    = 'VALIDATE_SUPPLEMENT'
                       %msg           = NEW /dmo/cm_flight_messages(
                                       textid = /dmo/cm_flight_message
                                       severity = if_abap_behv_message=
```

```

        %path                = VALUE #( booking-%tky = booksuppl_booking_links[
                                     travel-%tky = booksuppl_travel_links[
        %element-SupplementID = if_abap_behv=>mk-on
        ) TO reported-bookingsupplement.

    ENDIF.

    ENDLLOOP.

    ENDMETHOD.

```

Validation `validateCurrencyCode`

Define the validation in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a validation on save with trigger operation `create` and trigger field `CurrencyCode`. Since there must always be currency code, define the field `CurrencyCode` as mandatory.

Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  field ( mandatory ) SupplementID
  validation validateSupplement on save { create; field SupplementID; }

... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the currency code by checking if the provided value is an entry in `I`. Raise adequate messages for the consumer if the value is initial or not valid. The implementation steps are exactly the same as for the validation `ValidateCustomer`. For a detailed step-by-step description, see [Validation validateCustomer](#).

To fill the `%path` component in the reported table, you have to provide the keys of all parent instances. Therefore you have to execute a read by association to every entity in the business object structure. In this case, to [Booking](#) and to [BookingSupplement](#).

i *Expand the following code sample to view the source code of Validation `validateSupplement`.*

Sample Code

```

METHOD validateCurrencyCode.
  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement
      FIELDS ( currencycode )
      WITH CORRESPONDING #( keys )
    RESULT DATA(booking_supplements).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Booking
    FROM CORRESPONDING #( booking_supplements )
    LINK DATA(booksuppl_booking_links).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Travel

```

```

        FROM CORRESPONDING #( booking_supplements )
        LINK DATA(booksuppl_travel_links).

DATA: currencies TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY currency.

currencies = CORRESPONDING #( booking_supplements DISCARDING DUPLICATES MAPPING currency = c
DELETE currencies WHERE currency IS INITIAL.

IF currencies IS NOT INITIAL.
    SELECT FROM I_Currency FIELDS currency
    FOR ALL ENTRIES IN @currencies
    WHERE currency = @currencies-currency
    INTO TABLE @DATA(currency_db).
ENDIF.

LOOP AT booking_supplements INTO DATA(booking_supplement).
    APPEND VALUE #( %tky                = booking_supplement-%tky
                    %state_area         = 'VALIDATE_CURRENCYCODE'
                    ) TO reported-bookingsupplement.
    IF booking_supplement-currencycode IS INITIAL.
        " Raise message for empty Currency
        APPEND VALUE #( %tky                = booking_supplement-%tky ) TO failed-bookingsupp
        APPEND VALUE #( %tky                = booking_supplement-%tky
                        %state_area         = 'VALIDATE_CURRENCYCODE'
                        %msg                = NEW /dmo/cm_flight_messages(
                                                textid      = /dmo/cm_flight_messages=>cu
                                                severity   = if_abap_behv_message=>severi
                                                %path        = VALUE #( booking-%tky = booksuppl_booking_links[
                                                                travel-%tky = booksuppl_travel_links[
                                                                %element-currencycode = if_abap_behv=>mk-on
                                                                ) TO reported-bookingsupplement.
                        ELSEIF NOT line_exists( currency_db[ currency = booking_supplement-currencycode ] ).
                        " Raise message for not existing Currency
                        APPEND VALUE #( %tky                = booking_supplement-%tky ) TO failed-bookingsupp
                        APPEND VALUE #( %tky                = booking_supplement-%tky
                                        %state_area         = 'VALIDATE_CURRENCYCODE'
                                        %msg                = NEW /dmo/cm_flight_messages(
                                                                textid      = /dmo/cm_flight_messages=>cu
                                                                severity   = if_abap_behv_message=>se
                                                                %path        = VALUE #( booking-%tky = booksuppl_booking_links[
                                                                travel-%tky = booksuppl_travel_links[
                                                                %element-currencycode = if_abap_behv=>mk-on
                                                                ) TO reported-bookingsupplement.
                    ) TO reported-bookingsupplement.
    ENDIF.
ENDLOOP.
ENDMETHOD.

```

Validation validatePrice

Define the validation in the behavior definition and implement it in the behavior implementation class for [BookingSupplement](#).

Definition

Define a validation on save with trigger operation create and trigger field BookSupplPrice.

Sample Code

```

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
    validation validatePrice on save { create; field BookSupplPrice; }
... }

```

Via a quick fix, you can generate the method declaration in the behavior pool directly from the behavior definition editor.

Implementation

Validate the price by checking if the entered value is less than 0. Raise an adequate message for the consumer if the value is not valid. The following steps guide you through the implementation.

1. Read all the instances with the imported keys into an internal table `booking_supplements`. This table is the basis to check whether messages must be raised.
2. Read the keys of the corresponding parent and root instances using the associations `_Booking` and `_Travel` and store the result in the `link` response respectively. This information will be needed for linking supplement instances to booking and travel instances in the reported response.
3. Loop over the internal table `booking_supplements`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area.
4. If the flight price is less than 0, write the transactional key to the failed table and append a corresponding message along with the state area `VALIDATE_PRICE` to the reported table for that instance. The reported table includes the component `%element`. Here you can specify which CDS element is responsible for the state inconsistency. The Fiori Elements UI evaluates this component and highlights the corresponding input field. Use the labels `booksuppl_booking_links` and `booksuppl_travel_links` populated above to provide the keys of the corresponding booking and travel instances for each supplement instance.

The messages for the managed scenario with draft are stored in the message class `/dmo/cm_flight_messages` that is part of the ABAP Flight Reference Scenario.

i Expand the following code sample to view the source code of Validation `validatePrice`.

Sample Code

```
METHOD validatePrice.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement
      FIELDS ( BookSupplPrice )
      WITH CORRESPONDING #( keys )
      RESULT DATA(booking_supplements).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Booking
      FROM CORRESPONDING #( booking_supplements )
      LINK DATA(booksuppl_booking_links).

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY BookingSupplement BY \_Travel
      FROM CORRESPONDING #( booking_supplements )
      LINK DATA(booksuppl_travel_links).

  LOOP AT booking_supplements INTO DATA(booking_supplement).
    APPEND VALUE #( %tky          = booking_supplement-%tky
                    %state_area   = 'VALIDATE_PRICE'
                    ) TO reported-bookingsupplement.
    " Raise message for Supplement Price < 0
    IF booking_supplement-BookSupplPrice < 0.
      APPEND VALUE #( %tky          = booking_supplement-%tky ) TO failed-bookingsuppl
      APPEND VALUE #( %tky          = booking_supplement-%tky
                      %state_area   = 'VALIDATE_PRICE'
                      %msg          = NEW /dmo/cm_flight_messages(
                                      textid    = /dmo/cm_flight_messages=>sup
                                      severity  = if_abap_behv_message=>severi
                                      %path     = VALUE #( booking-%tky = booksuppl_booking_links[
                                                         travel-%tky = booksuppl_travel_links[
                                                         %element-BookSupplPrice = if_abap_behv=>mk-on
```

```
) T0 reported-bookingsupplement.
```

```
ENDIF.  
ENDLOOP.
```

```
ENDMETHOD.
```

Developing RAP Side Effects

For the managed scenario with draft, add RAP side effects to model interdependencies between BO properties for a reload of the affected properties.

Context and Procedure

With RAP side effects, you can trigger data, permission, or message updates based on data changes in UI scenarios with draft-enabled BOs. For more detailed information, see [RAP Side Effects](#).

For this demo scenario, define RAP side effects on all three BO entities. Side effects are defined in the base behavior definition in the entity behavior body.

- For the **travel entity**, the following RAP side effects are relevant:
 - Whenever the field `BookingFee` is changed, the recalculation of the field `TotalPrice` must be triggered to adjust the value with the new amount for the booking fee field.
 - Whenever the field `AgencyID` is changed, the user must receive feedback if the used value is valid for this field. This is validated by the validation `validateAgency`. To be able to define a trigger point for the validation, it must firstly be defined in a determine action.
 - Whenever the field `CustomerID` is changed, the user must receive feedback if the used value is valid for this field. This is validated by the validation `validateCustomer`. To be able to define a trigger point for the validation, it must firstly be defined in a determine action.
 - Whenever the fields `BeginDate` and `EndDate` are changed, the user must receive feedback if the used values are valid for these fields. This is validated by the validation `validateDates`. To be able to define a trigger point for the validation, it must firstly be defined in a determine action.
- For the **booking entity**, the following RAP side effects are described:
 - Whenever the field `FlightPrice` is changed, the recalculation of the field `TotalPrice` of the travel entity must be triggered to adjust the value with the new amount of the flight price field.
- For the **booking supplement entity**, the following RAP side effects are described.
 - Whenever the field `BookSupplPrice` is changed, the recalculation of the field `TotalPrice` of the travel entity must be triggered to adjust the value with the new amount for the booking supplement price field.
 - Whenever the field `BookSupplPrice` is changed, the validation `validatePrice` is executed which checks whether the entered price is less than 0.

For a detailed description on how to implement the described behavior for the relevant entity, check the implementation steps in the following topics.

[Defining RAP Side Effects for the Travel Entity](#)

For the managed scenario with draft, define RAP side effects on the travel entity for immediate feedback on the Fiori Elements UI.

[Defining RAP Side Effects for the Booking Entity](#)

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

[Defining RAP Side Effects for the Booking Supplement Entity](#)

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

Defining RAP Side Effects for the Travel Entity

For the managed scenario with draft, define RAP side effects on the travel entity for immediate feedback on the Fiori Elements UI.

Side effects are defined in the base behavior definition. To be exposed for a UI service, they must be reused in the projection behavior definition like any other BO behavior, see [Projecting the Draft BO Behavior](#).

RAP Side Effects for Total Price Recalculation

The RAP side effect triggers the recalculation of the **Total Price** on every change of the **Booking Fee**.

Define the RAP side effect with field trigger and field target in the behavior body of the behavior definition.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
side effects
{ field BookingFee affects field TotalPrice; }
```

Whenever the **Booking Fee** is changed, the **Total Price** is reloaded and the recalculated total price is displayed on the UI. The recalculation in the backend is triggered by the field change.

RAP Side Effect to Trigger the Validation of the AgencyID

The RAP side effect triggers the validation that checks if the **AgencyID** is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. There fore (authorization: update) is defined.

i Note

The keyword **side effects** can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  determine action ( authorization : update ) validateAgencyID { validation validateAgency; }

  side effects { field BookingFee affects field TotalPrice; }
```

```

        determine action ( authorization : update ) validateAgencyID executed on field .
    ... }

```

Whenever the **AgencyID** is changed the determine action is triggered, which calls the validation and the messages are reloaded. The determine action has the same authorization as update. There fore (authorization: update) is defined.

RAP Side Effect to Trigger the Validation of the CustomerID

The RAP side effect triggers the validation that checks if the **CustomerID** is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. There fore (authorization: update) is defined.

i Note

The keyword **side effects** can only appear once in the behavior definition body. You have to define all side effects for one entity together.

≡, Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
    determine action ( authorization : update ) validateAgencyID { validation validateAgency; }
    determine action ( authorization : update ) validateCustomerID { validation validateCustomer;

    side effects { field BookingFee affects field TotalPrice;
        determine action ( authorization : update ) validateAgencyID   executed on fiel
        determine action ( authorization : update ) validateCustomerID executed on fiel
    }
... }

```

Whenever the **CustomerID** is changed the determine action is triggered, which calls the validation and the messages are reloaded.

RAP Side Effect to Trigger the Validation of the Dates

The RAP side effect triggers the validation that checks if the **BeginDate** and the **EndDate** is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action.

i Note

The keyword **side effects** can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

≡, Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
    determine action ( authorization : update ) validateAgencyID { validation validateAgency; }
    determine action ( authorization : update ) validateCustomerID { validation validateCustomer;

```

```
determine action ( authorization : update ) validateDateRange { validation validateDates; }
```

```

side effects { field BookingFee affects field TotalPrice;
              determine action ( authorization : update ) validateAgencyID executed on field
              determine action ( authorization : update ) validateCustomerID executed on field
              determine action ( authorization : update ) validateDateRange executed on field
            }
... }
```

Whenever the `BeginDate` or the `EndDate` is changed the `determine` action is triggered, which calls the validation and the messages are reloaded.

RAP Side Effect to Trigger the Validation of the Booking Fee

The RAP side effect triggers the validation that checks if the `BookingFee` is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a `determine` action. The `determine` action has the same authorization as `update`. Therefore (`authorization: update`) is defined.

i Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

≡ Sample Code

```

define behavior for /DMO/R_Travel_D alias Travel
...
{ ...
  determine action ( authorization : update ) validateAgencyID
  determine action ( authorization : update ) validateCustomerID
  determine action ( authorization : update ) validateDateRange
  determine action ( authorization : update ) validateBookingCharge {

    side effects { field BookingFee affects field TotalPrice;
                  determine action ( authorization : update ) validateAgencyID
                  determine action ( authorization : update ) validateCustomerID
                  determine action ( authorization : update ) validateDateRange
                  determine action ( authorization : update ) validateBookingCharge
                }
  }
... }
```

Whenever the field `BookingFee` is changed the `determine` action is triggered, which calls the validation and the messages are reloaded.

Defining RAP Side Effects for the Booking Entity

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

RAP side effects are defined in the base behavior definition. To be exposed for a UI service, they must be reused in the projection behavior definition like any other BO behavior, see [Projecting the Draft BO Behavior](#).

RAP Side Effects for Total Price Recalculation

The RAP side effect triggers the recalculation of the `Total Price` on every change of the `Flight Price`.

Define the RAP side effect with field trigger and field target in the behavior body of the behavior definition on the booking entity.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Travel
...
{ ...
  side effects { field FlightPrice affects field _Travel.TotalPrice; }
... }
```

Whenever the `Flight Price` is changed on the booking entity, the `Total Price` on the travel entity is reloaded and the recalculated total price is displayed on the UI. The recalculation in the backend is triggered by the field change

RAP Side Effect to Trigger the Validation of the `flight price`

The RAP side effect triggers the validation that checks if the `flight price` is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. Therefore (authorization: update) is defined.

Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

Sample Code

```
define behavior for /DMO/R_Booking_D alias Travel
...
{ ...
  determine action ( authorization : update ) validateFlightCosts { validation validateFlightPric

  side effects
  { field FlightPrice affects field _Travel.TotalPrice;
    determine action ( authorization : update ) validateFlightCosts executed on field FlightPric
... }
```

Whenever the `FlightPrice` is changed the determine action is triggered, which calls the validation and the messages are reloaded.

Defining RAP Side Effects for the Booking Supplement Entity

For the managed scenario with draft, define RAP side effects on the booking entity for immediate feedback on the Fiori Elements UI.

RAP side effects are defined in the base behavior definition. To be exposed for a UI service, they must be reused in the projection behavior definition like any other BO behavior, see [Projecting the Draft BO Behavior](#).

RAP Side Effect for Total Price Recalculation

The RAP side effect triggers the recalculation of the `Total Price` on every change of the `BookSupplPrice`.

Define the RAP side effect with field trigger and field target in the behavior body of the behavior definition on the booking booking supplements.

Sample Code

```
define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  side effects { field BookSupplPrice affects field _Travel.TotalPrice; }
... }
```

Whenever the `BookSupplPrice` is changed on the booking supplement entity, the `Total Price` on the travel entity is reloaded and the recalculated total price is displayed on the UI. The recalculation in the backend is triggered by the field change

RAP Side Effect to Trigger the Validation of the `BookSupplPrice`

The RAP side effect triggers the validation that checks if the `BookSupplPrice` is valid and reloads the respective messages. To be able to define a trigger point for the RAP side effect, it must firstly be defined in a determine action. The determine action has the same authorization as update. There fore (authorization: update) is defined.

i Note

The keyword `side effects` can only appear once in the behavior definition body. You have to define all RAP side effects for one entity together.

Sample Code

```
define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
...
{ ...
  determine action ( authorization : update ) validateSupplementPrice { validation validatePrice
  side effects
  { field BookSupplPrice affects field _Travel.TotalPrice;
    determine action ( authorization : update ) validateSupplementPrice executed on field BookSupp
... }
```

Whenever the `BookSupplPrice` is changed the determine action is triggered, which calls the validation and the messages are reloaded.

Exposing the Managed Business Object for a UI Business Service

In the previous chapters, you have created a managed business object from scratch. To expose it as a business service the following steps are relevant:

1. [Projecting the Managed BO for Non-Draft Exposure](#)
2. [Defining and Publishing a UI Service](#)

Projecting the Managed BO for Non-Draft Exposure

For the managed scenario with draft, create a projection layer for the managed BO.

Context and Procedure

Whereas the business object that we have created so far is service agnostic, the business object projection is an approach to define a subset of this business object as service-specific for the UI use case. In this scenario, we want to publish a UI service, so UI-specifics are defined in the BO projection. For more information about the BO projection layer, see [Business Object Projection](#).

For the travel demo scenario, define the data model and the behavior projection:

- For the **data model projection**, the following steps are relevant:

- Create **CDS projection views**.

In the CDS projection views define UI-service-specific features, such as search capabilities, text associations, and value helps.

- Create **metadata extensions**.

Metadata extensions are primarily used to define the annotations that are relevant for the UI design. They are outsourced in metadata extension to obtain an easy approach for layering.

For a detailed description on how to define data model projection features, see [Projecting the BO Data Model](#).

- For the **behavior projection**, the following steps are relevant:

- Create a **projection behavior definition**.

For a detailed description on behavior projection features, see [Projecting the BO Behavior](#).

Projecting the BO Data Model

For the managed scenario with draft, create and implement CDS projection views and metadata extensions for the BO entities.

Context and Procedure

Create data definition for projection views for all three BO entities **Travel**, **Booking**, and **Booking Supplement**. In ADT, use the context menu on the underlying CDS view entity to open the wizard for creating CDS data definition.

Select the template **Define Projection View**.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. Since the CDS projection views only represent the active UI service, we use the marker `_A` before the suffix to distinguish the active projection layer from the projection layer that includes draft. In addition, projection views carry the prefix `C_` as they represent the consumption layer. For more information, see [Naming Conventions for Development Objects](#).

For each projection view, create a metadata extension. In ADT, right-click on the data definition to open the creation wizard for metadata extensions.

i Note

According to the virtual data model (VDM) guidelines, the metadata extension use the same name as the related CDS view.

For more information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Projection View `/DM0/C_Travel_A_D`

The projection view is a projection on `/DM0/R_Travel_D`. Include the keyword `root` in the projection view.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension: true`.

Enable search capabilities for the travel entity, with the `@Search` annotations on the entity header and on designated elements. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To indicate that `TravelID` is the semantic identifier of the BO-entity, use the annotation `@ObjectModel.semanticKey: [ 'TravelID' ]` in the header of the projection view.

To retrieve the long texts of the ID elements `CustomerID`, `AgencyID`, and `OverallStatus` denormalize the text elements from the associated text provider views into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements `CustomerID`, `AgencyID`, `CurrencyCode`, and `OverallStatus`. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity.

i Expand the following code sample to view the source code of Projection View `/DM0/C_Travel_A_D`.

≡ Sample Code

```
@EndUserText.label: 'Travel Projection View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true

define root view entity /DM0/C_Travel_A_D
  provider contract transactional_query
  as projection on /DM0/R_Travel_D

{
  key TravelUUID,

  @Search.defaultSearchElement: true
```

```

TravelID,

@Search.defaultSearchElement: true
@ObjectModel.text.element: ['AgencyName']
@Consumption.valueHelpDefinition: [{ entity : {name: '/DMO/I_Agency', element: 'AgencyID'
AgencyID,
_Agency.Name          as AgencyName,

@Search.defaultSearchElement: true
@ObjectModel.text.element: ['CustomerName']
@Consumption.valueHelpDefinition: [{ entity : {name: '/DMO/I_Customer', element: 'Custome
CustomerID,
_Customer.LastName as CustomerName,

BeginDate,

EndDate,

BookingFee,

TotalPrice,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
CurrencyCode,

Description,

@ObjectModel.text.element: ['OverallStatusText']
@Consumption.valueHelpDefinition: [{ entity: {name: '/DMO/I_Overall_Status_VH', element:
OverallStatus,
_OverallStatus._Text.Text as OverallStatusText : localized,

LocalLastChangedAt,
/* Associations */
_Agency,
_Booking : redirected to composition child /DMO/C_Booking_A_D,
_Currency,
_OverallStatus,
_Customer
}

```

Metadata Extension /DMO/C_Travel_A_D

Define the metadata extension as extension of the CDS projection view /DMO/C_Travel_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

To trigger actions via action buttons on the UI, use @UI annotations for data action on both the list report page and the object page. It is not important which element is annotated with this annotation, the action button always appears on the upper right corner on the UI. To enable multi-inline editing on the list report page, annotate the action Deduct_Discount with @UI.lineItem.invocationGrouping: #CHANGE_SET. The action is then processed for all selected instances in once change set.

i Expand the following code sample to view the source code of Metadata Extension /DMO/C_Travel_A_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Travel',
                    typeNamePlural: 'Travels',

```



```

        title: { type: #STANDARD, value: 'TravelID' } },
        presentationVariant: [{ sortOrder: [{ by: 'TravelID', direction: #DESC }] }] }

annotate entity /DMO/C_Travel_A_D
with
{
    @UI.facet: [{ id:          'Travel',
                  purpose:     #STANDARD,
                  type:        #IDENTIFICATION_REFERENCE,
                  label:       'Travel',
                  position:    10 },
                { id:          'Booking',
                  purpose:     #STANDARD,
                  type:        #LINEITEM_REFERENCE,
                  label:       'Booking',
                  position:    20,
                  targetElement: '_Booking' }]
    @UI.hidden: true
    TravelUUID;

    @UI: { lineItem:      [ { position: 10, importance: #HIGH },
                          { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                          { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                          { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
    identification: [ { position: 10, label: 'Travel ID' },
                    { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                    { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                    { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
    selectionField: [ { position: 10 } ] }
    TravelID;

    @UI: { lineItem:      [ { position: 20, importance: #HIGH } ] ,
    identification: [ { position: 20 } ] ,
    selectionField: [ { position: 20 } ] }
    AgencyID;

    @UI: { lineItem:      [ { position: 30, importance: #HIGH } ] ,
    identification: [ { position: 30 } ] ,
    selectionField: [ { position: 30 } ] }
    CustomerID;

    @UI: { lineItem:      [ { position: 40, importance: #MEDIUM } ] ,
    identification: [ { position: 40 } ] }
    BeginDate;

    @UI: { lineItem:      [ { position: 50, importance: #MEDIUM } ] ,
    identification: [ { position: 50 } ] }
    EndDate;

    @UI: { lineItem:      [ { position: 60, importance: #MEDIUM } ] ,
    identification: [ { position: 60 } ] }
    BookingFee;

    @UI: { lineItem:      [ { position: 70, importance: #MEDIUM } ] ,
    identification: [ { position: 70 } ] }
    TotalPrice;

    // CurrencyCode;

    @UI: { lineItem:      [ { position: 90, importance: #MEDIUM } ] ,
    identification: [ { position: 90 } ] }
    Description;

    @UI: { lineItem:      [ { position: 99, importance: #HIGH } ] ,
    identification: [ { position: 99, label: 'Status' } ] ,
    selectionField: [ { position: 40 } ] ,
    textArrangement: #TEXT_ONLY }
    OverallStatus;
    @UI.hidden: true
    OverallStatusText;

    @UI.hidden: true
    LocalLastChangedAt;

```

}

Projection View /DMO/C_Booking_A_D

The projection view is a projection on /DMO/R_Booking_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the booking entity, with the `@Search` annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID elements CustomerID, CarrierID, and BookingStatus denormalize the text elements from the associated text provider views into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements CustomerID, AirlineID, ConnectionID, FlightDate, CurrencyCode, and BookingStatus. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View /DMO/C_Booking_A_D.

Sample Code

```
@EndUserText.label: 'Booking Proj View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
define view entity /DMO/C_Booking_A_D
  as projection on /DMO/R_Booking_D
{
  key BookingUUID,

  TravelUUID,

  @Search.defaultSearchElement: true
  BookingID,

  BookingDate,

  @ObjectModel.text.element: ['CustomerName']
  @Search.defaultSearchElement: true
  @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Customer', element: 'CustomerI
CustomerID,
  _Customer.LastName as CustomerName,

  @ObjectModel.text.element: ['CarrierName']
  @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Carrier', element: 'AirlineID'
AirlineID,
  _Carrier.Name      as CarrierName,

  @Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Flight', element: 'Connection
    additionalBinding: [ { localElement: 'FlightDate',    element: 'FlightDate'}
                        { localElement: 'AirlineID',      element: 'AirlineID'},
                        { localElement: 'FlightPrice',    element: 'Price', usag
                        { localElement: 'CurrencyCode',    element: 'CurrencyCode

  ConnectionID,
```

FlightDate,

```
@Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Flight', element: 'Connection
    additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate'}
                        { localElement: 'AirlineID', element: 'AirlineID'},
                        { localElement: 'FlightPrice', element: 'Price', usag
                        { localElement: 'CurrencyCode', element: 'CurrencyCode
```

FlightPrice,

```
@Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
CurrencyCode,
```

```
@ObjectModel.text.element: ['BookingStatusText']
```

```
@Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Booking_Status_VH', element:
BookingStatus,
_BookingStatus._Text.Text as BookingStatusText: localized,
```

LocalLastChangedAt,

/* Associations */

_BookingSupplement: redirected to composition child /DMO/C_BookingSupplement_A_D,

_BookingStatus,

_Carrier,

_Connection,

_Customer,

_Travel: redirected to parent /DMO/C_Travel_A_D

}

Metadata Extension /DMO/C_Booking_A_D

Define the metadata extension as extension of the CDS projection view /DMO/C_Booking_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension /DMO/C_Booking_A_D.

Sample Code

```
@Metadata.layer: #CORE
```

```
@UI: { headerInfo: { typeName: 'Booking',
                    typeNamePlural: 'Bookings',
                    title: { type: #STANDARD, value: 'BookingID' } } }
```

```
annotate entity /DMO/C_Booking_A_D with
{
```

```
  @UI.facet: [ { id: 'Booking',
                 purpose: #STANDARD,
                 type: #IDENTIFICATION_REFERENCE,
                 label: 'Booking',
                 position: 10 },
               { id: 'BookingSupplement',
                 purpose: #STANDARD,
                 type: #LINEITEM_REFERENCE,
                 label: 'Booking Supplement',
                 position: 20,
                 targetElement: '_BookingSupplement' } ]
```

```
@UI.hidden: true
BookingUUID;
```

```
@UI.hidden: true
TravelUUID;
```

```

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
      identification: [ { position: 20 } ] }
BookingID;

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
      identification: [ { position: 30 } ] }
BookingDate;

@UI: { lineItem:      [ { position: 40, importance: #HIGH } ],
      identification: [ { position: 40 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 50, importance: #HIGH } ],
      identification: [ { position: 50 } ] }
AirlineID;

@UI: { lineItem:      [ { position: 60, importance: #HIGH } ],
      identification: [ { position: 60 } ] }
ConnectionID;

@UI: { lineItem:      [ { position: 70, importance: #HIGH } ],
      identification: [ { position: 70 } ] }
FlightDate;

@UI: { lineItem:      [ { position: 80, importance: #HIGH } ],
      identification: [ { position: 80 } ] }
FlightPrice;

// CurrencyCode;

@UI: { lineItem:      [ { position: 90, importance: #HIGH } ],
      identification: [ { position: 90 } ],
      textArrangement: #TEXT_ONLY }
BookingStatus;

@UI.hidden: true
BookingStatusText;

@UI.hidden: true
LocalLastChangedAt;
}

```

Projection View /DMO/C_BookingSupplement_A_D

The projection view is a projection on /DMO/R_BookingSupplement_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID element `SupplementID`, denormalize the text element from the associated text provider view into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements `SupplementID` and `CurrencyCode`. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the association to the parent to the projection view of the booking entity and the association to the root entity to the respective projection view.

i *Expand the following code sample to view the source code of Projection View /DMO/C_BookingSupplement_A_D.*

≡, Sample Code

```

@EndUserText.label: 'BookingSuppl Proj View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true

define view entity /DMO/C_BookingSupplement_A_D
  as projection on /DMO/R_BookingSupplement_D
{
  key BookSupplUUID,

  TravelUUID,

  BookingUUID,

  @Search.defaultSearchElement: true
  BookingSupplementID,

  @ObjectModel.text.element: ['SupplementDescription']
  @Consumption.valueHelpDefinition: [ {entity: {name: '/DMO/I_Supplement', element: 'Supple
    additionalBinding: [ { localElement: 'BookSupplPrice', element: 'Price', u
                        { localElement: 'CurrencyCode', element: 'CurrencyCode

  SupplementID,
  _SupplementText.Description as SupplementDescription : localized,

  BookSupplPrice,

  @Consumption.valueHelpDefinition: [{entity: {name: 'I_Currency', element: 'Currency' }}]
  CurrencyCode,

  LocalLastChangedAt,

  /* Associations */
  _Booking : redirected to parent /DMO/C_Booking_A_D,
  _Product,
  _SupplementText,
  _Travel : redirected to /DMO/C_Travel_A_D
}

```

Metadata Extension /DMO/C_BookingSupplement_A_D

Define the metadata extension as extension of the CDS projection view /DMO/C_BookingSupplement_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i *Expand the following code sample to view the source code of Metadata Extension /DMO/C_BookingSupplement_A_D.*

≡, Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName:      'Booking Supplement',
                    typeNamePlural: 'Booking Supplements',
                    title:         { type: #STANDARD,
                                    label: 'Booking Supplement',
                                    value: 'BookingSupplementID' } } }

annotate entity /DMO/C_BookingSupplement_A_D
  with
{
  @UI.facet: [ { id:      'BookingSupplement',

```

```

        purpose:      #STANDARD,
        type:         #IDENTIFICATION_REFERENCE,
        label:        'Booking Supplement',
        position:     10 } ]

@UI.hidden:true
BookSupplUUID;
@UI.hidden:true
BookingUUID;
@UI.hidden:true
TravelUUID;

@UI: { lineItem:      [ { position: 10, importance: #HIGH } ],
      identification: [ { position: 10 } ] }
BookingSupplementID;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
      identification: [ { position: 20 } ] }
SupplementID;

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
      identification: [ { position: 30 } ] }
BookSupplPrice;

// CurrencyCode;

@UI.hidden: true
LocalLastChangedAt;
}

```

Projecting the BO Behavior

For the managed scenario with draft, create a projection behavior definition for defining the projection behavior.

Context and Procedure

Create a projection behavior definition for the travel business object. In ADT, use the context menu of the root projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with **Projection**.

Projection Behavior Definition /DM0/C_Travel_A_D

Use the complete behavior that was defined in the underlying behavior definition for the non-draft BO projection.

i *Expand the following code sample to view the source code of Projection Behavior Definition /DM0/C_Travel_A_D.*

≡ Sample Code

```

projection;
strict(2);
use side effects;

define behavior for /DM0/C_Travel_A_D alias Travel
use etag
{
    use create;
    use update;
    use delete;

    use action acceptTravel;
    use action rejectTravel;
    use action deductDiscount;
    use function GetDefaultsFordeductDiscount;
}

```

```

    use association _Booking { create; }
  }

  define behavior for /DMO/C_Booking_A_D alias Booking
  use etag
  {
    use update;
    use delete;

    use association _BookingSupplement { create; }
    use association _Travel;
  }

  define behavior for /DMO/C_BookingSupplement_A_D alias BookingSupplement
  use etag
  {
    use update;
    use delete;

    use association _Travel;
  }

```

Defining and Publishing a UI Service

For the managed scenario with draft, define a UI service and bind it to a protocol.

Context and Procedure

To address the managed BO from the ABAP-external world, it must be exposed as a business service to OData. In the service definition, you define which artifacts are included in the service. The service binding defines the protocol with which the service is addressed. In addition, it provides the means to activate the service locally. The Fiori Elements App Preview can be used as soon as the service is activated.

Create a service definition for the non-draft business service.

Based on the service definition, create a service binding. You can choose between OData V2 and V4 protocol to expose a UI service.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. To distinguish the active-only service from the service with draft, we use the addition `_A` before the suffix. For more information, see [Naming Conventions for Development Objects](#).

Service Definition /DMO/UI_TRAVEL_A_D

Include the projection views of the business object and all related CDS entities that are relevant for UI consumption into the service definition.

i Expand the following code sample to view the source code of Service Definition /DMO/UI_TRAVEL_A_D.

≡ Sample Code

```

@EndUserText.label: 'Travel Draft Scenario(active projection)'
@ObjectModel.leadingEntity.name: '/DMO/C_Travel_A_D'
define service /DMO/UI_TRAVEL_A_D {
  expose /DMO/C_Travel_A_D as Travel;
  expose /DMO/C_Booking_A_D as Booking;
  expose /DMO/C_BookingSupplement_A_D as BookingSupplement;

```

```

    expose /DMO/I_Supplement_StdVH as Supplement;
    expose /DMO/I_SupplementCategory_VH as SupplementCategory;
    expose /DMO/I_Customer_StdVH as Passenger;
    expose /DMO/I_Agency_StdVH as TravelAgency;
    expose /DMO/I_Carrier_StdVH as Airline;
    expose /DMO/I_Connection_StdVH as FlightConnection;
    expose /DMO/I_Flight_StdVH as Flight;
    expose /DMO/I_Airport_StdVH as Airport;
    expose /DMO/I_Overall_Status_VH as OverallStatus;
    expose /DMO/I_Booking_Status_VH as BookingStatus;
    expose I_CurrencyStdVH as Currency;
    expose I_CountryVH as Country;
}

```

Service Binding /DMO/UI_TRAVEL_A_D_02

Expose the service as a UI service with OData protocol V2 and publish it locally. You can then use the preview to test the service.

Draft-Enabling the Managed Business Object

You can now draft-enable the managed business object with exclusive draft capabilities with a few additions in the behavior definition.

In draft business objects, the implementation of the business logic must be able to handle request for draft and for active instances. Above all, the implementation must be able to differentiate those requests.

The business object receives the information if a request is aimed at draft or active instances via the draft indicator `%IS_DRAFT`. This draft indicator is a component of the primary key in all incoming requests in draft scenarios. The transactional key `%tky` includes the draft indicator. So in draft scenarios, it is best practice to use `%tky` for referring to the primary key, as the indicator for drafts or non-drafts is already included there. In non-draft scenarios, the transactional key `%tky` comprises just the primary key information.

If you consequently use the derived type `%tky` to refer to the key of instances in your implementation, draft-enabling is just done in the behavior definition with a few additional syntax elements. The draft indicator `%is_draft` is added as a key component to `%tky` in draft scenarios. This means, the implementation receives the information if the request is aimed at draft or active instances. So, you do not have to adjust your implementation in draft scenarios.

The following steps are necessary to draft-enable a managed business object: [Adding Draft Capabilities to the Managed Business Object](#)

Related Information

[Exclusive Draft Handling](#)

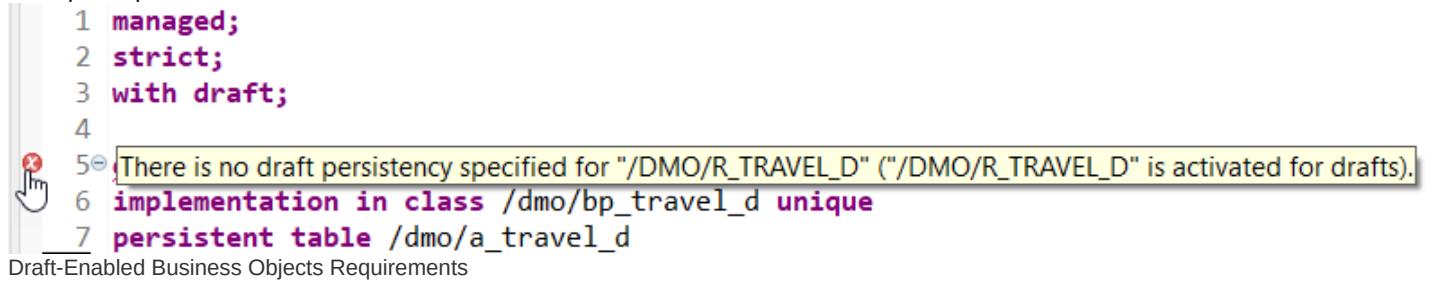
Adding Draft Capabilities to the Managed Business Object

For the managed scenario with draft, add draft behavior to the behavior definition.

Context and Procedure

In the previous sections of this development guide, you have developed a running business service with a managed business object. Draft capabilities are included to the business service by adding the syntax element `with draft` to the behavior definition. As soon as you use this keyword element in the behavior definition, you are guided through the follow-up steps in the

behavior definition by tool support. Syntax warnings and quick fixes are available to adapt the managed business object to the draft prerequisites.



i Note

Business objects with implementation type `unmanaged` are draft-enabled in the same way.

Adaptions in the Behavior Definition

Follow these steps to draft-enable the `travel` business object:

1. Add the syntax element `with draft` to the header of the behavior definition.

i Note

You receive a syntax error that there is no draft persistence available for the draft BO.

2. Create a draft database table for the `Travel` entity by using the quick fix in the behavior definition:

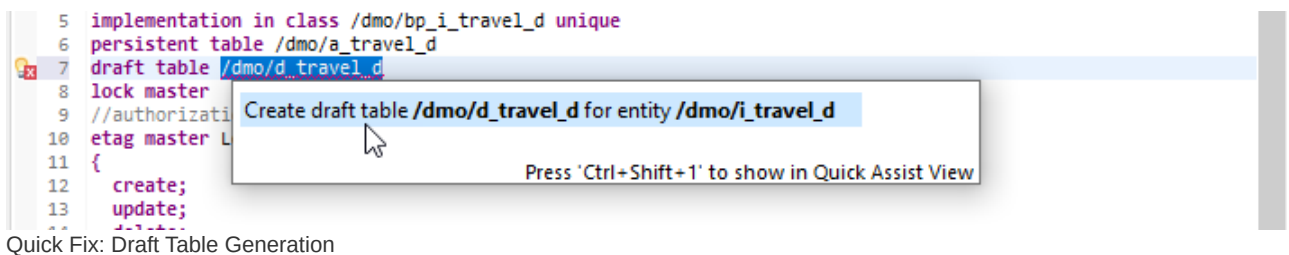
- o Define the draft database table in the behavior definition by using the keyword `draft table` and specify the name for the draft database table.

In the example scenario, we use `/DMO/D_TRAVEL_D`.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. For the draft database tables, we use the prefix `D_` to indicate the draft persistence. For detailed information, see [Naming Conventions for Development Objects](#).

- o Use the quick fix to create a database table with the specified name.



The wizard for creating database tables opens. When you finish the database table generation wizard, the ADT tooling support automatically generates the table elements corresponding to the active persistence. In addition, the draft admin structure include `%admin` is included in the database table.

For more information about the draft database table, see [Draft Database Table](#).

- o Add the annotation `@AbapCatalog.anonymizedWhenDelivered : true` to the elements that store user information. (Only relevant for the travel draft table.)

i Expand the following code sample to view the source code of Database Table `/DMO/D_TRAVEL_D`.

≡, Sample Code

```
@EndUserText.label : 'Draft table for entity /DMO/R_Travel_D'
@AbapCatalog.enhancement.category : #EXTENSIBLE_ANY
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/D_TRAVEL_D {
  key mandt      : mandt not null;
  key traveluuid : sysuuid_x16 not null;
  travelid       : /dmo/travel_id;
  agencyid       : /dmo/agency_id;
  customerid     : /dmo/customer_id;
  begindate      : /dmo/begin_date;
  enddate        : /dmo/end_date;
  @Semantics.amount.currencyCode : '/dmo/d_travel_d.currencycode'
  bookingfee     : /dmo/booking_fee;
  @Semantics.amount.currencyCode : '/dmo/d_travel_d.currencycode'
  totalprice     : /dmo/total_price;
  currencycode   : /dmo/currency_code;
  description    : /dmo/description;
  overallstatus  : /dmo/overall_status;
  @AbapCatalog.anonymizedWhenDelivered : true
  localcreatedby : abp_creation_user;
  localcreatedat : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  locallastchangedby : abp_locinst_lastchange_user;
  locallastchangedat : abp_locinst_lastchange_tstmpl;
  "%admin"          : include sych_bdl_draft_admin_inc;
}
```

3. Repeat the creation of draft database tables for the **Booking** entity and the **BookingSupplement** entity.

In the example scenario, we use the names /DMO/D_BOOKING_D and /DMO/D_BKSUPPL_D.

4. Define a total ETag field for the draft business object.

The total ETag is used to enable optimistic concurrency checks during the transition from draft to active data.

- Add a total ETag field to the persistent database table of the **Travel** entity.

i *Expand the following code sample to view the source code of Database Table /DMO/A_TRAVEL_D.*

≡, Sample Code

```
@EndUserText.label : 'Active Travel Persistence for Draft Reference Scenario'
@AbapCatalog.enhancementCategory : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /DMO/A_TRAVEL_D {
  key client      : abap.clnt not null;
  key travel_uuid : sysuuid_x16 not null;
  travel_id       : /dmo/travel_id not null;
  ...
  @AbapCatalog.anonymizedWhenDelivered : true
  local_created_by      : abp_creation_user;
  local_created_at      : abp_creation_tstmpl;
  @AbapCatalog.anonymizedWhenDelivered : true
  local_last_changed_by : abp_locinst_lastchange_user;
  local_last_changed_at : abp_locinst_lastchange_tstmpl;
  last_changed_at       : abp_lastchange_tstmpl;
}
```

- o Add the element in the CDS view entity `/DMO/R_Travel_D` and annotate it with the annotation `@Semantics.systemDateTime.lastChangedAt: true` to allow the RAP runtime framework to update it automatically.

i Expand the following code sample to view the source code of CDS view entity `/DMO/R_Travel_D`.

Sample Code

```
@EndUserText.label: 'Travel View Entity for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

define root view entity /DMO/R_Travel_D
  as select from /DMO/A_TRAVEL_D

  composition [0..*] of /DMO/R_Booking_D as _Booking

  association [0..1] to /DMO/I_Agency as _Agency on $projection.AgencyID = _Ag
  association [0..1] to /DMO/I_Customer as _Customer on $projection.CustomerID = _
  association [0..1] to I_Currency as _Currency on $projection.CurrencyCode = _

{ ///DMO/A_TRAVEL_D
  key travel_uuid as TravelUUID,

  travel_id as TravelID,
  agency_id as AgencyID,
  customer_id as CustomerID,
  begin_date as BeginDate,
  end_date as EndDate,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  booking_fee as BookingFee,
  @Semantics.amount.currencyCode: 'CurrencyCode'
  total_price as TotalPrice,
  currency_code as CurrencyCode,
  description as Description,
  overall_status as OverallStatus,

  @Semantics.user.createdBy: true
  local_created_by as LocalCreatedBy,
  @Semantics.systemDateTime.createdAt: true
  local_created_at as LocalCreatedAt,
  @Semantics.user.localInstanceLastChangedBy: true
  local_last_changed_by as LocalLastChangedBy,
  //local ETag field --> OData ETag
  @Semantics.systemDateTime.localInstanceLastChangedAt: true
  local_last_changed_at as LocalLastChangedAt,

  //total ETag field
  @Semantics.systemDateTime.lastChangedAt: true
  last_changed_at as LastChangedAt,

  //Associations
  _Booking,

  _Agency,
  _Customer,
  _Currency
}
```

i Note

The total ETag field does not need to be included in the projection view. The ETag is only used for internal processing of draft data and does not need to be exposed for the business service.

- o Add the total ETag field to the mapping prescription in the behavior definition for the [Travel](#) entity.

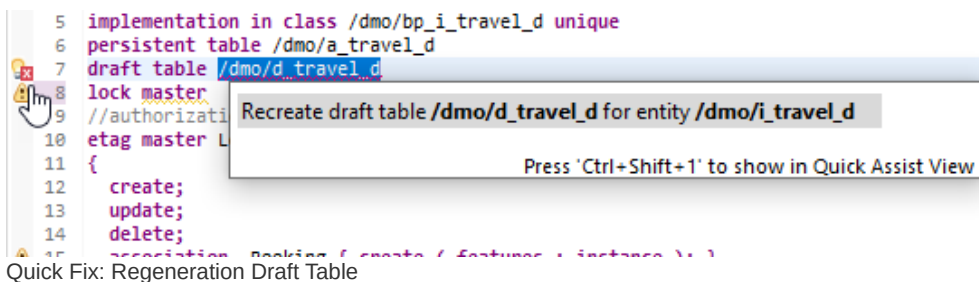
i Expand the following code sample to view the source code of Behavior Definition `/DMO/R_Travel_D`.

≡, Sample Code

```
define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
...
```

```
mapping for /DMO/A_TRAVEL_D
{ AgencyID = agency_id;
  BeginDate = begin_date;
  BookingFee = booking_fee;
  CurrencyCode = currency_code;
  CustomerID = customer_id;
  Description = description;
  EndDate = end_date;
  LastChangedAt = last_changed_at;
  LocalCreatedAt = local_created_at;
  LocalCreatedBy = local_created_by;
  LocalLastChangedAt = local_last_changed_at;
  LocalLastChangedBy = local_last_changed_by;
  OverallStatus = overall_status;
  TotalPrice = total_price;
  TravelID = travel_id;
  TravelUUID = travel_uuid; }
}
```

- o Regenerate the draft database table /DMO/D_TRAVEL_D to include the total ETag field. The behavior definition provides a quick fix for the regeneration.



- o Define the element LastChangedAt as total ETag in the behavior definition by using the keyword total etag.

i Note

The definition of the total ETag is only possible directly after the lock master definition in the behavior definition.

i Note

It is recommended that you specify the ETag field as read-only using the field characteristic readonly in the behavior definition.

≡, Sample Code

```
managed;
strict( 2 );
with draft;

define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_I_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
draft table /DMO/D_TRAVEL_D
lock master
total etag LastChangedAt
...
{
...
field ( readonly ) ...LastChangedAt;
```

```
...
}
```

5. Explicitly draft-enable the compositions within the draft BO.

As soon as you draft-enable a business object by adding `with draft` to the behavior definition, all BO-internal associations are automatically draft-enabled. To make this behavior explicit, the behavior prompts you to specify the compositions within a draft BO with `with draft`.

For more information, see [Draft-Enabled Associations](#).

6. Define the validations and determinations for the draft determine action Prepare.

The determinations and validations that are defined in the `Prepare` action are called before and during the `Activate` action to transfer draft to active instances. For more information, see [Preparing Draft Instances for Activation](#).

Use the `Activate` action in its optimized variant to increase the performance by skipping determination and validation executions that are not required. Note that the best practices for [Determination and Validation Modelling](#) need to be considered to make use of this optimized variant.

- o Define the `Prepare` action in the behavior definition.
- o Include all validations that are defined for the travel BO in the `Prepare`

i Note

It is not generally necessary to include all validations in the `Prepare` action, but only those that you want to check during the transition from draft to active.

≡ Sample Code

```
managed;
strict( 2 );
with draft;

define behavior for /DMO/R_Travel_D alias Travel
...

draft determine action Prepare
{
    validation validateAgency;
    validation validateCustomer;
    validation validateDates;
    validation validateCurrencyCode;
    validation validateBookingFee;
    validation Booking~validateCustomer;
    validation Booking~validateConnection;
    validation Booking~validateCurrencyCode;
    validation Booking~validateFlightPrice;
    validation Booking~validateStatus;
    validation Bookingsupplement~validateSupplement;
    validation Bookingsupplement~validateCurrencyCode;
    validation Bookingsupplement~validatePrice; }
```

7.i Expand the following code sample to view the source code of the entire Behavior Definition `/DMO/R_Travel_D`.

☰ Sample Code

```

managed;
strict( 2 );
with draft;

define behavior for /DMO/R_Travel_D alias Travel
implementation in class /DMO/BP_TRAVEL_D unique
persistent table /DMO/A_TRAVEL_D
draft table /DMO/D_TRAVEL_D
lock master
total etag LastChangedAt
//authorization master ( instance )
etag master LocalLastChangedAt
{
  create;
  update;
  delete;
  association _Booking { create ( features : instance ); with draft; }

  field ( numbering : managed, readonly ) TravelUUID;
  field ( readonly ) TravelID, OverallStatus, TotalPrice, LocalCreatedAt, LocalCreatedBy, Local
  field ( mandatory ) CustomerID, AgencyID, BeginDate, EndDate;
  field ( features : instance ) BookingFee;

  action ( features : instance ) acceptTravel result [1] $self;
  action ( features : instance ) rejectTravel result [1] $self;
  action ( features : instance, authorization : update ) deductDiscount parameter /DMO/A_TRAVEL
  internal action reCalcTotalPrice;

  determination setTravelNumber on save { create; }
  determination setStatusToNew on modify { create; }
  determination calculateTotalPrice on modify { create; field BookingFee, CurrencyCode; }

  validation validateCustomer on save { create; field CustomerID; }
  validation validateAgency on save { create; field AgencyID; }
  validation validateDates on save { create; field BeginDate, EndDate; }

  determine action validationAgency { validation validateAgency; }
  determine action validationCustomer { validation validateCustomer; }
  determine action validationDates { validation validateDates; }

  side effects
  { field BookingFee affects field TotalPrice;
    determine action validationAgency executed on field AgencyID affects messages;
    determine action validationCustomer executed on field CustomerID affects messages;
    determine action validationDates executed on field BeginDate, field EndDate affects messages

  draft action resume;
  draft action Edit;
  draft action Activate optimized;
  draft action Discard;

  draft determine action Prepare
  {
    validation validateAgency;
    validation validateCustomer;
    validation validateDates;
    validation Booking ~ validateCustomer;
    validation Booking ~ validateConnection;
    validation Bookingsupplement ~ validateSupplement; }

  mapping for /DMO/A_TRAVEL_D
  { AgencyID = agency_id;
    BeginDate = begin_date;
    BookingFee = booking_fee;
    CurrencyCode = currency_code;
    CustomerID = customer_id;
    Description = description;
    EndDate = end_date;
    LastChangedAt = last_changed_at;
    LocalCreatedAt = local_created_at;
  }
}

```

```

    LocalCreatedBy = local_created_by;
    LocalLastChangedAt = local_last_changed_at;
    LocalLastChangedBy = local_last_changed_by;
    OverallStatus = overall_status;
    TotalPrice = total_price;
    TravelID = travel_id;
    TravelUUID = travel_uuid; }
}

define behavior for /DMO/R_Booking_D alias Booking
implementation in class /DMO/BP_BOOKING_D unique
persistent table /DMO/A_BOOKING_D
draft table /DMO/D_BOOKING_D
lock dependent by _Travel
//authorization dependent by <association>
etag master LocalLastChangedAt
{
    update;
    delete;
    association _BookingSupplement { create ; with draft; }

    association _Travel { with draft; }

    field ( numbering : managed, readonly ) BookingUUID;
    field ( readonly ) TravelUUID, BookingID, BookingDate, LocalLastChangedAt;
    field ( mandatory ) CustomerID, AirlineID, ConnectionID, FlightDate;

    determination setBookingNumber on save { create; }
    determination setBookingDate on save { create; }
    determination calculateTotalPrice on modify { create; field FlightPrice, CurrencyCode; }

    validation validateCustomer on save { create; field CustomerID; }
    validation validateConnection on save { create; field AirlineID, ConnectionID, FlightDate; }

    side effects
    { field FlightPrice affects field _Travel.TotalPrice; }

mapping for /DMO/A_BOOKING_D
{ AirlineID = carrier_id;
  BookingDate = booking_date;
  BookingID = booking_id;
  BookingStatus = booking_status;
  BookingUUID = booking_uuid;
  ConnectionID = connection_id;
  CurrencyCode = currency_code;
  CustomerID = customer_id;
  FlightDate = flight_date;
  FlightPrice = flight_price;
  LocalLastChangedAt = local_last_changed_at;
  TravelUUID = parent_uuid; }
}

define behavior for /DMO/R_BookingSupplement_D alias BookingSupplement
implementation in class /DMO/BP_BOOKINGSUPPLEMENT_D unique
persistent table /DMO/A_BKSUPPL_D
draft table /DMO/D_BKSUPPL_D
lock dependent by _Travel
//authorization dependent by <association>
etag master LocalLastChangedAt
{
    update;
    delete;

    association _Travel { with draft; }
    association _Booking { with draft; }

    field ( numbering : managed, readonly ) BookSupplUUID;
    field ( readonly ) TravelUUID, BookingUUID, BookingSupplementID, LocalLastChangedAt;
    field ( mandatory ) SupplementID;

    determination setBookSupplNumber on save { create; }
    determination calculateTotalPrice on modify { create; field CurrencyCode, BookSupplPrice; }

```

```

validation validateSupplement on save { create; field SupplementID; }

side effects
{ field BookSupplPrice affects entity _Travel; }

mapping for /DMO/A_BKSUPPL_D
{ BookSupplPrice = price;
  BookSupplUUID = booksuppl_uuid;
  BookingSupplementID = booking_supplement_id;
  BookingUUID = parent_uuid;
  CurrencyCode = currency_code;
  LocalLastChangedAt = local_last_changed_at;
  SupplementID = supplement_id;
  TravelUUID = root_uuid; }

}

```

i Note

Draft-enabled business objects require a feature control method in the behavior pool. If you do not use dynamic feature control in your business logic, include the method declaration and an empty method implementation. In Draft BOs, feature control is called for every request to determine whether draft or active instances are requested.

Adaptions in the Behavior Pool

If you have consequently used %tky to refer to the key fields of instances, you shouldn't have to make changes in the behavior implementation in general.

For authorization, it is different. When draft-enabling a business object, draft actions are implicitly added to the BO behavior. There are two draft actions, which need to be considered in the authorization implementation as they also call the authorization methods during their runtime, see [Runtime Edit Action](#) and [Runtime Activate Action](#).

In this demo example, authorization for actions is handled exactly like the update operation. This can also be applied for the draft actions **Edit** and **Activate**.

Add the relevant draft actions to the action mapping in the behavior implementation for global and instance authorization.

i Expand the following code sample to view the source code of the method `get_global_authorizations`.

≡ Sample Code

```

METHOD get_global_authorizations.
  IF requested_authorizations-%create EQ if_abap_behv=>mk-on.
    IF is_create_granted( ) = abap_true.
      result-%create = if_abap_behv=>auth-allowed.
    ELSE.
      result-%create = if_abap_behv=>auth-unauthorized.
      APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                        textid    = /dmo/cm_flight_messages=>not_authorized
                        severity  = if_abap_behv_message=>severity-error )
                    %global = if_abap_behv=>mk-on ) TO reported-travel.
    ENDIF.
  ENDIF.

  "Actions are treated like update
  IF requested_authorizations-%update
    requested_authorizations-%action-Edit           = if_abap_behv=>mk-on OR
    requested_authorizations-%action-Prepare         = if_abap_behv=>mk-on OR
    requested_authorizations-%action-acceptTravel    = if_abap_behv=>mk-on OR
    requested_authorizations-%action-rejectTravel     = if_abap_behv=>mk-on OR
    requested_authorizations-%action-deductDiscount = if_abap_behv=>mk-on.

```



```

IF is_update_granted( ) = abap_true.
  result-%update           = if_abap_behv=>auth-allowed.
  result-%action-Edit      = if_abap_behv=>auth-allowed.
  result-%action-Prepare   = if_abap_behv=>auth-allowed.
  result-%action-acceptTravel = if_abap_behv=>auth-allowed.
  result-%action-rejectTravel = if_abap_behv=>auth-allowed.
  result-%action-deductDiscount = if_abap_behv=>auth-allowed.

ELSE.
  result-%update           = if_abap_behv=>auth-unauthorized.
  result-%action-Edit      = if_abap_behv=>auth-unauthorized.
  result-%action-Prepare   = if_abap_behv=>auth-unauthorized.
  result-%action-acceptTravel = if_abap_behv=>auth-unauthorized.
  result-%action-rejectTravel = if_abap_behv=>auth-unauthorized.
  result-%action-deductDiscount = if_abap_behv=>auth-unauthorized.

  APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                    textid    = /dmo/cm_flight_messages=>not_authorized
                    severity   = if_abap_behv_message=>severity-error )
                  %global = if_abap_behv=>mk-on ) TO reported-travel.

ENDIF.
ENDIF.

IF requested_authorizations-%delete = if_abap_behv=>mk-on.
  IF is_delete_granted( ) = abap_true.
    result-%delete = if_abap_behv=>auth-allowed.
  ELSE.
    result-%delete = if_abap_behv=>auth-unauthorized.
    APPEND VALUE #( %msg      = NEW /dmo/cm_flight_messages(
                    textid    = /dmo/cm_flight_messages=>not_authorized
                    severity   = if_abap_behv_message=>severity-error )
                  %global = if_abap_behv=>mk-on ) TO reported-travel.

  ENDIF.
ENDIF.

ENDMETHOD.

```

i Expand the following code sample to view the source code of the method `get_instance_authorizations`.

Sample Code

```

METHOD get_instance_authorizations.

  DATA: update_requested TYPE abap_bool,
        delete_requested TYPE abap_bool,
        update_granted   TYPE abap_bool,
        delete_granted   TYPE abap_bool.

  READ ENTITIES OF /DMO/R_Travel_D IN LOCAL MODE
    ENTITY Travel
      FIELDS ( AgencyID )
      WITH CORRESPONDING #( keys )
  RESULT DATA(travels)
  FAILED failed.

  CHECK travels IS NOT INITIAL.

  "Select country_code and agency of corresponding persistent travel instance
  "authorization only checked against instance that have active persistence
  SELECT FROM /DMO/A_TRAVEL_D AS travel INNER JOIN /DMO/AGENCY AS agency
    ON travel~agency_id = agency~agency_id
    FIELDS travel~travel_uuid , travel~agency_id, agency~country_code
  FOR ALL ENTRIES IN @travels WHERE travel_uuid EQ @travels-TravelUUID
  INTO TABLE @DATA(travel_agency_country).

  "all actions are treated like update
  update_requested = COND #( WHEN requested_authorizations-%update           = if_abap_be
                              requested_authorizations-%assoc-Booking      = if_abap_be
                              requested_authorizations-%action-Edit          = if_abap_be

```

```

        requested_authorizations-%action-Prepare      = if_abap_be
        requested_authorizations-%action-acceptTravel = if_abap_be
        requested_authorizations-%action-deductDiscount = if_abap_be
        requested_authorizations-%action-rejectTravel  = if_abap_be
    THEN abap_true ELSE abap_false ).

delete_requested = COND #( WHEN requested_authorizations-%delete      = if_abap_be
    THEN abap_true ELSE abap_false ).

LOOP AT travels INTO DATA(travel).
    "get country_code of agency in corresponding instance on persistent table
    READ TABLE travel_agency_country WITH KEY travel_uuid = travel-TravelUUID
    ASSIGNING FIELD-SYMBOL(<travel_agency_country_code>).

    "Auth check for active instances that have before image on persistent table
    IF sy-subrc = 0.

        "check auth for update
        IF update_requested = abap_true.
            update_granted = is_update_granted( <travel_agency_country_code>-country_code ).
            IF update_granted = abap_false.
                APPEND VALUE #( %tky = travel-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                        textid    = /dmo/cm_flight_messages=>not_authorized_for_age
                        agency_id = travel-AgencyID
                        severity  = if_abap_behv_message=>severity-error )
                    %element-AgencyID = if_abap_behv=>mk-on ) TO reported-travel.
            ENDIF.
        ENDIF.

        "check auth for delete
        IF delete_requested = abap_true.
            delete_granted = is_delete_granted( <travel_agency_country_code>-country_code ).
            IF delete_granted = abap_false.
                APPEND VALUE #( %tky = travel-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                        textid    = /dmo/cm_flight_messages=>not_authorized_for_age
                        agency_id = travel-AgencyID
                        severity  = if_abap_behv_message=>severity-error )
                    %element-AgencyID = if_abap_behv=>mk-on ) TO reported-travel.
            ENDIF.
        ENDIF.

        " operations on draft instances and on active instances that have no persistent before ima
        " create authorization is checked, for newly created instances
        ELSE.
            update_granted = delete_granted = is_create_granted( ).
            IF update_granted = abap_false.
                APPEND VALUE #( %tky = travel-%tky
                    %msg = NEW /dmo/cm_flight_messages(
                        textid    = /dmo/cm_flight_messages=>not_authorized
                        severity  = if_abap_behv_message=>severity-error )
                    %element-AgencyID = if_abap_behv=>mk-on ) TO reported-travel.
            ENDIF.
        ENDIF.

        APPEND VALUE #( LET upd_auth = COND #( WHEN update_granted = abap_true THEN if_abap_behv=>
            ELSE if_abap_behv=>auth-unauthorized )
            del_auth = COND #( WHEN delete_granted = abap_true THEN if_abap_behv=>
            ELSE if_abap_behv=>auth-unauthorized )

            IN
            %tky = travel-%tky
            %update      = upd_auth
            %assoc-Booking = upd_auth
            %action-Edit  = upd_auth
            %action-Prepare = upd_auth
            %action-acceptTravel = upd_auth
            %action-deductDiscount = upd_auth
            %action-rejectTravel  = upd_auth

            %delete      = del_auth
        ) TO result.
    ENDLOOP.

```

ENDMETHOD .

Result

As soon as you activate the behavior definition, the business object is draft-enabled. In general, that means:

- Transactional requests are processed by the draft runtime,
- The importing structures for all methods in the behavior pool receive an additional component, the draft indicator `%is_draft`. In particular, the transactional key `%tky` determines whether the operation is executed on a draft or on an active instance.
- The draft actions are implicitly available and applied during the draft life cycle.
- The existing UI service does not change as long as you do not include the draft capabilities in the projection behavior definition. It still behaves like a non-draft business service. The underlying draft BO processes every request with the draft indicator `is_draft` set to false.

For more information, see [Draft Runtime](#).

The conversion to a draft business object sometimes requires follow-up activities of existing implementations in the business logic. In general, scan the existing code thoroughly to detect cases where you need to differentiate between active and draft processing.

If the transactional key `%tky` is used for referring to the key components, no adaptation is necessary for these implementations. The `%is_draft` is automatically added as key component to the `%tky` structure.

Related Information

[Exclusive Draft Handling](#)

Exposing the Draft Business Object for a UI Business Service

In the previous chapters, you have created a draft business object by draft-enabling a managed business object. Since the managed business object projection without draft capabilities remains untouched, it continues to operate only on active instances.

If you want to have a business service with draft capabilities, you can either add the draft features to the existing business object projection or create a new projection for the draft service. This is done in this demo scenario. In the end, you have two parallel business services, one for active only and one that includes draft features. To expose the draft business as a business service the following steps are relevant:

- [Projecting the Draft BO for Draft Exposure](#)
- [Defining and Publishing a UI Service with Draft Capabilities](#)

Projecting the Draft BO for Draft Exposure

For the managed scenario with draft, create a projection layer for the draft BO.

Context and Procedure

In the first part of this development guide, you have created a non-draft business service based on a managed business object. The business object projection includes all elements in the data model and the behavior features that are relevant for the UI service. This projection remains stable, even when you add draft capabilities to the basic business object. The projection layer filters out the elements and behavior features that are draft-specific as they are not included in the projection.

To expose the draft business object for a UI service, it is necessary that you define a business object projection that also includes the draft specific features. For more information about the BO projection layer, see [Business Object Projection](#).

For the travel demo scenario, define the data model and the behavior projection for the draft projection:

- For the **data model projection**, the following steps are relevant:

- Create **CDS projection views**.

In the CDS projection views define UI-service-specific features, such as search capabilities, text associations, and value helps.

- Create **metadata extensions**.

Metadata extensions are primarily used to define the annotations that are relevant for the UI design. They are outsourced in metadata extension to obtain an easy approach for layering.

For a detailed description on how to define data model projection features, see [Projecting the BO Data Model](#).

- For the **behavior projection**, the following steps are relevant:

- Create a **projection behavior definition**.

For a detailed description on behavior projection features, see [Projecting the BO Behavior](#).

Projecting the Draft BO Data Model

For the managed scenario with draft, create and implement CDS projection views and metadata extensions for the draft BO entities.

Context and Procedure

Create data definition for projection views for all three BO entities **Travel**, **Booking**, and **Booking Supplement**. In ADT, use the context menu on the underlying CDS view entity to open the wizard for creating CDS data definition.

Select the template **Define Projection View**.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix **_D**. Since the new CDS projection views represent the UI service with draft capabilities, we use the marker **_D** before the suffix to distinguish the active projection layer from the projection layer that includes draft. In addition, projection views carry the prefix **C_** as they represent the consumption layer. For more information, see [Naming Conventions for Development Objects](#).

For each projection view, create a metadata extension. In ADT, right-click on the data definition to open the creation wizard for metadata extensions.

i Note

According to the virtual data model (VDM) guidelines, the metadata extension use the same name as the related CDS view.

For more information about the syntax in projection views, see [Define View Entity as Projection \(ABAP Keyword Documentation\)](#).

Projection View /DMO/C_Travel_D_D

The projection view is a projection on /DMO/R_Travel_D. Include the keyword `root` in the projection view.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations on the entity header and on designated elements. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

For the OData V4 consumption of a draft business object via a Fiori Elements UI, the annotation `@ObjectModel.semanticKey: ['SemanticIdentifier']` must be used to ensure that draft indications on the instance are displayed.

To retrieve the long texts of the ID elements `CustomerID`, `AgencyID`, and `OverallStatus` denormalize the text elements from the associated text provider view into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements `CustomerID`, `AgencyID`, `CurrencyCode` and `OverallStatus`. For more information, see [Adding Value Help](#).

In contrast to the ETag master field, which is necessary to ensure optimistic concurrency control in the OData consumption, you do not need to include the total ETag field in the draft business service projection. The total ETag is only necessary for internal draft processing.

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity.

i *Expand the following code sample to view the source code of Projection View /DMO/C_Travel_D_D.*

Sample Code

```
@EndUserText.label: 'Travel Projection View with Draft'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['TravelID']

define root view entity /DMO/C_Travel_D_D
  provider contract transactional_query
  as projection on /DMO/R_Travel_D

{
  key TravelUUID,

  @Search.defaultSearchElement: true
  TravelID,

  @Search.defaultSearchElement: true
  @ObjectModel.text.element: ['AgencyName']
  @Consumption.valueHelpDefinition: [{ entity : {name: '/DMO/I_Agency_StdVH', element: 'Agen
  AgencyID,
  _Agency.Name          as AgencyName,

  @Search.defaultSearchElement: true
```

```

@ObjectModel.text.element: ['CustomerName']
@Consumption.valueHelpDefinition: [{entity: {name: '/DM0/I_Customer_StdVH', element: 'Cust
CustomerID,
_Customer.LastName as CustomerName,

BeginDate,

EndDate,

BookingFee,

TotalPrice,

@Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
CurrencyCode,

Description,

@ObjectModel.text.element: ['OverallStatusText']
@Consumption.valueHelpDefinition: [{ entity : {name: '/DM0/I_Overall_Status_VH', element:
OverallStatus,
_OverallStatus._Text.Text as OverallStatusText: localized,

LocalLastChangedAt,
/* Associations */
_Agency,
_Booking : redirected to composition child /DM0/C_Booking_D_D,
_Currency,
_Customer,
_OverallStatus
}

```

Metadata Extension /DM0/C_Travel_D_D

Define the metadata extension as extension of the CDS projection view /DM0/C_Travel_A_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension /DM0/C_Travel_D_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Travel',
                    typeNamePlural: 'Travels',
                    title: { type: #STANDARD, value: 'TravelID' } },
      presentationVariant: [{ sortOrder: [{ by: 'TravelID', direction: #DESC }], visualizations

annotate entity /DM0/C_Travel_D_D with
{
  @UI.facet: [{ id:
                purpose:
                type:
                label:
                position:
                { id:
                  purpose:
                  type:
                  label:
                  position:
                  targetElement:

```

```

@UI.hidden: true
TravelUUID;

@UI: { lineItem:      [ { position: 10, importance: #HIGH },
                        { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                        { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                        { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
identification: [ { position: 10, label: 'Travel ID' },
                  { type: #FOR_ACTION, dataAction: 'acceptTravel', label: 'Accept Trave
                  { type: #FOR_ACTION, dataAction: 'rejectTravel', label: 'Reject Trave
                  { type: #FOR_ACTION, dataAction: 'deductDiscount', label: 'Deduct Dis
selectionField: [ { position: 10 } ] }
TravelID;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ] ,
      identification: [ { position: 20 } ] ,
      selectionField: [ { position: 20 } ] }
AgencyID;

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ] ,
      identification: [ { position: 30 } ] ,
      selectionField: [ { position: 30 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 40, importance: #MEDIUM } ] ,
      identification: [ { position: 40 } ] }
BeginDate;

@UI: { lineItem:      [ { position: 50, importance: #MEDIUM } ] ,
      identification: [ { position: 50 } ] }
EndDate;

@UI: { lineItem:      [ { position: 60, importance: #MEDIUM } ] ,
      identification: [ { position: 60 } ] }
BookingFee;

@UI: { lineItem:      [ { position: 70, importance: #MEDIUM } ] ,
      identification: [ { position: 70 } ] }
TotalPrice;

// CurrencyCode;

@UI: { lineItem:      [ { position: 90, importance: #MEDIUM } ] ,
      identification: [ { position: 90 } ] }
Description;

@UI: { lineItem:      [ { position: 99, importance: #HIGH } ] ,
      identification: [ { position: 99, label: 'Status' } ] ,
      selectionField: [{ position: 40 }],
      textArrangement: #TEXT_ONLY }
OverallStatus;

@UI.hidden: true
OverallStatusText;

@UI.hidden: true
LocalLastChangedAt;
}

```

Projection View /DM0/C_Booking_D_D

The projection view is a projection on /DM0/R_Booking_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the booking entity, with the @Search annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID elements CustomerID and CarrierID, denormalize the text elements from the associated text provider views into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements CustomerID, AirlineID, ConnectionID, FlightDate, and CurrencyCode. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View /DMO/C_Booking_D_D.

Sample Code

```
@EndUserText.label: 'Booking Proj View with Draft'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['BookingID']

define view entity /DMO/C_Booking_D_D
  as projection on /DMO/R_Booking_D
{
  key BookingUUID,

  TravelUUID,

  @Search.defaultSearchElement: true
  BookingID,

  BookingDate,

  @ObjectModel.text.element: ['CustomerName']
  @Search.defaultSearchElement: true
  @Consumption.valueHelpDefinition: [{entity: {name: '/DMO/I_Customer_StdVH', element: 'Cust
CustomerID,
_Customer.LastName as CustomerName,

  @ObjectModel.text.element: ['CarrierName']
  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Flight_StdVH', element: 'AirlineID'},
      additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                          { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                          { localElement: 'FlightPrice', element: 'Price', usage:
                          { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                          qualifier: 'FlightVH1',
                          label: 'Flights' }
    ]
  AirlineID,
  _Carrier.Name      as CarrierName,

  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Flight_StdVH', element: 'ConnectionID'},
      additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                          { localElement: 'AirlineID', element: 'AirlineID', usage:
                          { localElement: 'FlightPrice', element: 'Price', usage:
                          { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                          qualifier: 'FlightVH2',
                          label: 'Flights' }
    ]
  ConnectionID,

  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Flight_StdVH', element: 'FlightDate'},
      additionalBinding: [ { localElement: 'AirlineID', element: 'AirlineID', usage:
```



```

        { localElement: 'ConnectionID', element: 'ConnectionID', usage:
        { localElement: 'FlightPrice', element: 'Price', usage:
        { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
    label: 'Flights',
    useForValidation: true }
  ]
  FlightDate,
  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DM0/I_Flight_StdVH', element: 'Price'},
      additionalBinding: [ { localElement: 'FlightDate', element: 'FlightDate', usage:
                          { localElement: 'AirlineID', element: 'AirlineID', usage:
                          { localElement: 'ConnectionID', element: 'ConnectionID', usage:
                          { localElement: 'CurrencyCode', element: 'CurrencyCode', usage:
                            label: 'Flights',
                            useForValidation: true }
      ]
    FlightPrice,

    @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
    CurrencyCode,

    @ObjectModel.text.element: ['BookingStatusText']
    @Consumption.valueHelpDefinition: [{entity: {name: '/DM0/I_Booking_Status_VH', element: 'B
    BookingStatus,
    _BookingStatus._Text.Text as BookingStatusText: localized,

    LocalLastChangedAt,

    /* Associations */
    _BookingSupplement: redirected to composition child /DM0/C_BookingSupplement_D_D,
    _BookingStatus,
    _Carrier,
    _Connection,
    _Customer,
    _Travel: redirected to parent /DM0/C_Travel_D_D
  }
}

```

Metadata Extension /DM0/C_Booking_D_D

Define the metadata extension as extension of the CDS projection view /DM0/C_Booking_D_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Metadata Extension View /DM0/C_Booking_D_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Booking',
                    typeNamePlural: 'Bookings',
                    title: { type: #STANDARD, value: 'BookingID' } } }

annotate entity /DM0/C_Booking_D_D with
{
  @UI.facet: [ { id:
                purpose:
                type:
                label:
                position:
                { id:
                purpose:
                  'Booking',
                  #STANDARD,
                  #IDENTIFICATION_REFERENCE,
                  'Booking',
                  10 },
                  'BookingSupplement',
                  #STANDARD,

```

```

        type:          #LINEITEM_REFERENCE,
        label:         'Booking Supplement',
        position:      20,
        targetElement: '_BookingSupplement'} ]
@UI.hidden: true
BookingUUID;

@UI.hidden: true
TravelUUID;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ],
      identification: [ { position: 20 } ] }
BookingID;

@UI: { lineItem:      [ { position: 30, importance: #HIGH } ],
      identification: [ { position: 30 } ] }
BookingDate;

@UI: { lineItem:      [ { position: 40, importance: #HIGH } ],
      identification: [ { position: 40 } ] }
CustomerID;

@UI: { lineItem:      [ { position: 50, importance: #HIGH } ],
      identification: [ { position: 50 } ] }
AirlineID;

@UI: { lineItem:      [ { position: 60, importance: #HIGH } ],
      identification: [ { position: 60 } ] }
ConnectionID;

@UI: { lineItem:      [ { position: 70, importance: #HIGH } ],
      identification: [ { position: 70 } ] }
FlightDate;

@UI: { lineItem:      [ { position: 80, importance: #HIGH } ],
      identification: [ { position: 80 } ] }
FlightPrice;

// CurrencyCode;

@UI: { lineItem:      [ { position: 90, importance: #HIGH } ],
      identification: [ { position: 90 } ],
      textArrangement: #TEXT_ONLY }
BookingStatus;

@UI.hidden: true
BookingStatusText;

@UI.hidden: true
LocalLastChangedAt;
}

```

Projection View /DMO/C_BookingSupplement_D_D

The projection view is a projection on /DMO/R_BookingSupplement_D.

Use all elements of the underlying BO travel entity, in particular the element representing the ETag master.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#).

To retrieve the long texts of the ID element `SupplementID`, denormalize the text element from the associated text provider view into the projection view. For more information, see [Getting Language-Dependent Text in Projection Views](#).

Define value helps for the elements SupplementID and CurrencyCode. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the association to the parent to the projection view of the booking entity and the association to the root entity to the respective projection view.

i Expand the following code sample to view the source code of Projection View /DMO/C_BookingSupplement_D_D.

Sample Code

```
@EndUserText.label: 'BookingSuppl Proj View for Draft RefScen'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@Search.searchable: true
@ObjectModel.semanticKey: ['BookingSupplementID']

define view entity /DMO/C_BookingSupplement_D_D
  as projection on /DMO/R_BookingSupplement_D
{
  key BookSupplUUID,

  TravelUUID,

  BookingUUID,

  @Search.defaultSearchElement: true
  BookingSupplementID,

  @ObjectModel.text.element: ['SupplementDescription']
  @Consumption.valueHelpDefinition: [
    { entity: {name: '/DMO/I_Supplement_StdVH', element: 'SupplementID' },
      additionalBinding: [ {_localElement: 'BookSupplPrice', element: 'Price', us
                           {_localElement: 'CurrencyCode', element: 'CurrencyCode', us
                           useForValidation: true }
    ]
  SupplementID,
  _SupplementText.Description as SupplementDescription : localized,

  BookSupplPrice,

  @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'
  CurrencyCode,

  LocalLastChangedAt,

  /* Associations */
  _Booking : redirected to parent /DMO/C_Booking_D_D,
  _Product,
  _SupplementText,
  _Travel : redirected to /DMO/C_Travel_D_D
}
```

Metadata Extension /DMO/C_BookingSupplement_D_D

Define the metadata extension as extension of the CDS projection view /DMO/C_BookingSupplement_D_D.

Define the layer #CORE.

Use all the @UI annotations to design the user interface for the travel app. For more information, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i Expand the following code sample to view the source code of Projection View /DMO/C_BookingSupplement_D_D.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: { typeName:      'Booking Supplement',
                      typeNamePlural: 'Booking Supplements',
                      title:         { type: #STANDARD,
                                      label: 'Booking Supplement',
                                      value: 'BookingSupplementID' } } }

annotate entity /DMO/C_BookingSupplement_D_D
with
{
  @UI.facet: [ { id:          'BookingSupplement',
                  purpose:    #STANDARD,
                  type:       #IDENTIFICATION_REFERENCE,
                  label:      'Booking Supplement',
                  position:    10 } ]

  @UI.hidden:true
  BookSupplUUID;
  @UI.hidden:true
  BookingUUID;
  @UI.hidden:true
  TravelUUID;

  @UI: { lineItem:    [ { position: 10, importance: #HIGH } ],
        identification: [ { position: 10 } ] }
  BookingSupplementID;

  @UI: { lineItem:    [ { position: 20, importance: #HIGH } ],
        identification: [ { position: 20 } ] }
  SupplementID;

  @UI: { lineItem:    [ { position: 30, importance: #HIGH } ],
        identification: [ { position: 30 } ] }
  BookSupplPrice;

  // CurrencyCode;

  @UI.hidden: true
  LocalLastChangedAt;
}

```

Projecting the Draft BO Behavior

For the managed scenario with draft, create a projection behavior definition for defining the projection behavior.

Projection Behavior Definition /DMO/C_Travel_D_D

Use the complete behavior that was defined in the underlying behavior definition for the draft BO projection. (The template does not include everything, some features must be added manually.) Create a projection behavior definition for the travel business object. In ADT, use the context menu of the root projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with

i *Expand the following code sample to view the source code* **Create a projection behavior definition for the travel business object. In ADT, use of Projection Behavior Definition .**

Sample Code

```

projection;
strict(2);
use draft;
use side effects;

define behavior for /DMO/C_Travel_D_D alias Travel

```

```

use etag
{
    use create;
    use update;
    use delete;

    use action acceptTravel;
    use action rejectTravel;
    use action deductDiscount;
    use function GetDefaultsFordeductDiscount;

    use action validateAgencyID;
    use action validateCustomerID;
    use action validateDateRange;
    use action validateBookingCharge;

    use action Edit;
    use action Activate;
    use action Discard;
    use action Prepare;
    use action Resume;

    use association _Booking { create; with draft; }
}

define behavior for /DMO/C_Booking_D_D alias Booking
use etag
{
    use update;
    use delete;

    use action validateFlightCosts;

    use association _BookingSupplement { create; with draft; }
    use association _Travel { with draft; }
}

define behavior for /DMO/C_BookingSupplement_D_D alias BookingSupplement
use etag
{
    use update;
    use delete;

    use action validateSupplementPrice;

    use association _Travel { with draft; }
    use association _Booking { with draft; }
}

```

Defining and Publishing a UI Service with Draft Capabilities

For the managed scenario with draft, define a UI service and bind it to a protocol.

Context and Procedure

To address the draft BO from the ABAP-external world, it must be exposed as a business service to OData. In the service definition, you define which artifacts are included in the service. The service binding defines the protocol with which the service is addressed. In addition, it provides the means to activate the service locally. The Fiori Elements App Preview can be used as soon as the service is activated.

Create a service definition for the draft business service.

Based on the service definition, create a service binding.

UI services of draft business objects with Fiori Elements are only supported with OData V4. Fiori Elements cannot handle the full scope of draft services that are exposed with OData V2.

i Note

The draft scenario in the ABAP Flight Reference Scenario uses the suffix `_D`. To distinguish the draft service from the service with draft, we use the addition `_D` before the suffix. For more information, see [Naming Conventions for Development Objects](#).

Service Definition /DMO/UI_TRAVEL_D_D

Include the projection views of the business object and all related CDS entities that are relevant for UI consumption into the service definition.

i Expand the following code sample to view the source code of Projection View /DMO/UI_TRAVEL_D_D.

Sample Code

```
@EndUserText.label: 'Travel Draft Scenario (draft projection)'
@ObjectModel.leadingEntity.name: '/DMO/C_Travel_D_D'
define service /DMO/UI_TRAVEL_D_D {
    expose /DMO/C_Travel_D_D as Travel;
    expose /DMO/C_Booking_D_D as Booking;
    expose /DMO/C_BookingSupplement_D_D as BookingSupplement;
    expose /DMO/I_Supplement_StdVH as Supplement;
    expose /DMO/I_SupplementCategory_VH as SupplementCategory;
    expose /DMO/I_Customer_StdVH as Passenger;
    expose /DMO/I_Agency_StdVH as TravelAgency;
    expose /DMO/I_Carrier_StdVH as Airline;
    expose /DMO/I_Connection_StdVH as FlightConnection;
    expose /DMO/I_Flight_StdVH as Flight;
    expose /DMO/I_Airport_StdVH as Airport;
    expose /DMO/I_Overall_Status_VH as OverallStatus;
    expose /DMO/I_Booking_Status_VH as BookingStatus;
    expose I_CurrencyStdVH as Currency;
    expose I_CountryVH as Country;
}
```

Service Binding

Expose the service as a UI service with OData protocol V4 and publish it locally. You can then use the preview to test the service.

Developing Transactional Apps with Collaborative Draft Capabilities

With the collaborative draft capability, you can invite users to edit the same draft instance simultaneously.

Prerequisites

The implementation prerequisites for the collaborative draft are the same as for the exclusive draft. For more information, see [Draft-Enabling the Managed Business Object](#).

You can also generate the necessary artifacts for a draft-enabled business object (BO) from scratch with the [OData UI Service from Scratch](#) wizard. For more information, see [OData UI Service from Scratch](#).

Role Authorizations for SAP S/4HANA Cloud Public Edition and SAP BTP ABAP environment

The new business catalog SAP_CORE_BC_RAP_DRAFT_PC needs to be assigned, which is part of the business role template SAP_BR_EMPLOYEE. Therefore, you initially have to ask your administrator to either generate an additional role based on the role template SAP_BR_EMPLOYEE or let the administrator assign the business catalogue to an existing role which is assigned to developers on your system.

Role Authorizations for SAP S/4HANA Cloud Private Edition

You need to define the necessary developer roles yourself, by adding the authorization object S_DRAFT with the authorization fields DRAFTUSER and ACTVT. Make sure to define the following values for the respective authorization fields:

- DRAFTUSER= *
- ACTVT= W4

Context

The collaborative draft capability for a BO is defined in the behavior definition. You can build collaborative draft BOs from scratch or you can add collaborative capabilities to a BO with already implemented exclusive draft capabilities in both implementation types **managed** or **unmanaged**, so multiple users can edit the same draft instance at the same time. A business object can only either be draft-enabled with an exclusive draft or with a collaborative draft. The draft capability applies to the entire BO. It's not possible to implement draft capabilities for individual entities within a BO. For more information about the draft concept, refer to [Draft](#) and [Draft Handling](#).

This topic explains how you implement the collaborative draft in the context of the ABAP RESTful Application Programming Model (RAP), and how to invite users to a draft instance of your BO on the SAP Fiori UI. The following code examples are based on a managed BO, which you can download and import into your development environment. For more information, refer to [Downloading the ABAP Flight Reference Scenario](#).

Procedure

To enable collaborative draft capabilities to your BO, proceed as follows:

1. In the header of the behavior definition of the base layer, add the syntax element with `collaborative draft`.

i Note

If you use an already draft-enabled BO with exclusive draft, change the syntax element with `draft` to with `collaborative draft`.

Sample Code

```
managed implementation in class /DMO/BP_R_TravelTP_CD unique;
strict ( 2 );
with collaborative draft;
extensible;
{
  ...
}
```

i Note

Initially, a syntax error occurs because you haven't added the draft action `Share` yet.

2. For collaborative draft, a draft query view is mandatory for the `authorization master` entity. Draft query views allow you to define `READ` access limitations for draft data based on the data control language (DCL).

Add a draft query view to the draft table declaration in the base behavior definition with the aid of the keyword `query`:

Sample Code

```
define behavior for /DMO/R_TravelTP_CD alias Travel
persistent table /dmo/a_travel_cd
extensible
draft table /dmo/d_travel_cd query /DMO/R_TravelTPDraft_CD
...
```

You can now create the draft query view with a quick fix. It creates a new root view entity as your draft query view with your draft database table as the referenced object.

i Expand the following code sample to view the source code of Draft Query View `/DMO/R_TRAVELTPDRAFT_CD`.

Sample Code

```
@AccessControl.authorizationCheck: #MANDATORY
@Metadata.allowExtensions: true
@endUserText.label: 'Draft Query View /DMO/D_TRAVEL_CD'
define view entity /DMO/R_TRAVELTPDRAFT_CD
  as select from /dmo/d_travel_cd
{
  key traveluuid as TravelUuid,
  travelid as TravelId,
  begindate as BeginDate,
  enddate as EndDate,
  bookingfee as BookingFee,
  currencycode as CurrencyCode,
  description as Description,
  localcreatedby as LocalCreatedBy,
  localcreatedat as LocalCreatedAt,
  locallastchangedby as LocalLastChangedBy,
  locallastchangedat as LocalLastChangedAt,
  lastchangedat as LastChangedAt,
  draftentitycreationdatetime as draftentitycreationdatetime,
  draftentitylastchangedatetime as draftentitylastchangedatetime,
  draftadministrativeatauid as draftadministrativeatauid,
  draftentityoperationcode as draftentityoperationcode,
  hasactiveentity as hasactiveentity,
  draftfieldchanges as draftfieldchanges
}
```

If you don't use the offered quick fix, make sure that you don't use the annotation

`@Metadata.ignorePropagatedAnnotations` at all or set its value to `false`. If you leave the value set to `true`, be aware that you have to maintain your annotations in the draft query view yourself.

In this scenario, the draft query view selects from the draft database table `/DMO/D_TRAVEL_CD` of the root entity `/DMO/R_TravelTP_CD`. The draft query view must have a DCL that inherits its access conditions from the corresponding DCL of the root entity.

For more information on draft query views, see [Draft Query Views](#).

i Note

A data control language (DCL) is mandatory if your active business object view entity is protected by a DCL.

3. Create a new DCL for your draft query view. Make sure your DCL inherits the conditions from the root view entity of your BO. In this scenario, it inherits the conditions from entity `/DMO/R_TravelTP_CD`.

i Expand the following code sample to view the source code of the Access Control for the Draft Query View `/DMO/R_TRAVELTPDRAFT_CD`.

Sample Code

```
@EndUserText.label: 'Mapping role for /DMO/R_TravelTPDraft_CD'
@MappingRole: true
define role /DMO/R_TravelTPDraft_CD {
  grant
  select
  on
    /DMO/R_TravelTPDraft_CD
  where
    inheriting conditions from entity /DMO/R_TravelTP_CD;
}
```

4. Add draft action Share to the behavior definition of the base layer.

Sample Code

```
define behavior for /DMO/R_TravelTP_CD alias Travel
...
{
...
draft action Activate optimized;
draft action Discard;
draft action Edit;
draft action Resume;
draft determine action Prepare;
draft action Share;
}
```

The resulting base behavior definition looks like this:

i Expand the following code sample to view the source code of Behavior Definition `/DMO/R_TRAVELTP_CD`.

Sample Code

```
managed implementation in class /DMO/BP_R_TravelTP_CD unique;
strict ( 2 );
with collaborative draft;
extensible;

define behavior for /DMO/R_TravelTP_CD alias Travel
persistent table /dmo/a_travel_cd
extensible
draft table /dmo/d_travel_cd query /DMO/R_TravelTPDraft_CD
etag master LocalLastChangedAt
lock master total etag LastChangedAt
authorization master ( global, instance )
{
  field ( readonly )
    TravelUuid,
```

```

    LocalCreatedBy,
    LocalCreatedAt,
    LocalLastChangedBy,
    LocalLastChangedAt,
    LastChangedAt;

    field ( numbering : managed )
    TravelUuid;

    field ( readonly : update, mandatory : create )
    TravelId;

    create ( authorization : global );
    update;
    delete;

    draft action Activate optimized;
    draft action Discard;
    draft action Edit;
    draft action Resume;
    draft determine action Prepare;
    draft action Share;

    association _Booking { create; with draft; }

    mapping for /dmo/a_travel_cd corresponding extensible
    {
        BeginDate          = begin_date;
        BookingFee          = booking_fee;
        CurrencyCode        = currency_code;
        Description          = description;
        EndDate              = end_date;
        LastChangedAt        = last_changed_at;
        LocalCreatedAt       = local_created_at;
        LocalCreatedBy       = local_created_by;
        LocalLastChangedAt   = local_last_changed_at;
        LocalLastChangedBy   = local_last_changed_by;
        TravelId             = travel_id;
        TravelUUID           = travel_uuid;
    }
}

define behavior for /DMO/R_BookingTP_CD alias Booking
persistent table /dmo/a_book_cd
extensible
draft table /dmo/d_book_cd
etag dependent by _Travel
lock dependent by _Travel
authorization dependent by _Travel
{
    field ( readonly )
    BookingUuid,
    TravelUuid;

```

```

    field ( numbering : managed )
    BookingUuid;

    field ( readonly : update, mandatory : create )
    BookingId;

    update;
    delete;

    association _Travel { with draft; }

    mapping for /dmo/a_book_cd corresponding extensible
    {
        BookingDate = booking_date;
        BookingId   = booking_id;
        BookingUUID = booking_uuid;
        CurrencyCode = currency_code;
        FlightDate  = flight_date;
        FlightPrice = flight_price;
        TravelUUID  = parent_uuid;
    }
}

```

5. In the header of the behavior definition of the projection layer, add the syntax element `use collaborative draft`. In case you have used an interface layer, you must also use the statement there.

i Note

If you use an already draft-enabled BO with exclusive draft, change the syntax element `use draft` to `use collaborative draft`.

Sample Code

```

projection implementation in class /DMO/BP_C_TravelTP_CD unique;
strict ( 2 );
extensible;
use collaborative draft;
use side effects;
{
...
}

```

i Note

Initially, a warning occurs because you haven't declared the usage of the draft action `Share` yet.

6. Add the statement `use action Share` to the projection layer of the behavior definition. In case you have used an interface layer, you must also use the statement there.

Sample Code

```

define behavior for /DMO/C_TravelTP_CD alias Travel
...
{
...
use action Activate;

```

```

use action Discard;
use action Edit;
use action Resume;
use action Prepare;
use action Share;
}

```

The resulting projection behavior definition looks like this:

i *Expand the following code sample to view the source code of Behavior Definition /DMO/C_TRAVELTP_CD.*

Sample Code

```

projection implementation in class /DMO/BP_C_TravelTP_CD unique;
strict ( 2 );
extensible;
use collaborative draft;
use side effects;

define behavior for /DMO/C_TravelTP_CD alias Travel
extensible
use etag
{
    use create;
    use update;
    use delete;

    use action Activate;
    use action Discard;
    use action Edit;
    use action Resume;
    use action Prepare;
    use action Share;

    use association _Booking { create; with draft; }
}

define behavior for /DMO/C_BookingTP_CD alias Booking
extensible
use etag
{
    use update;
    use delete;

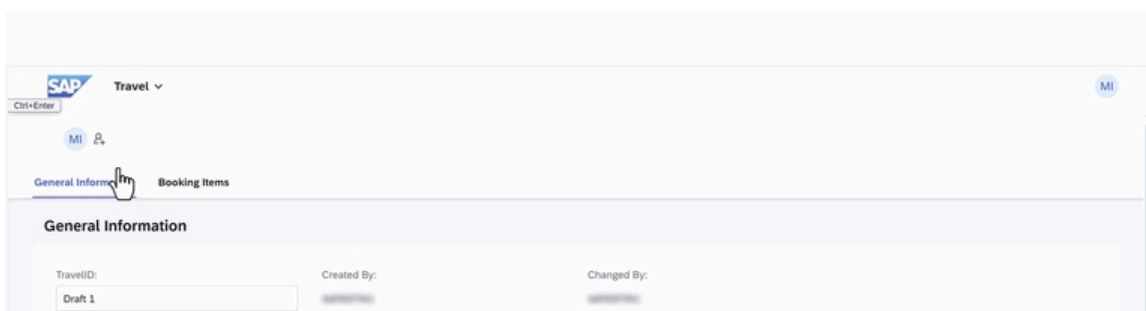
    use association _Travel { with draft; }
}

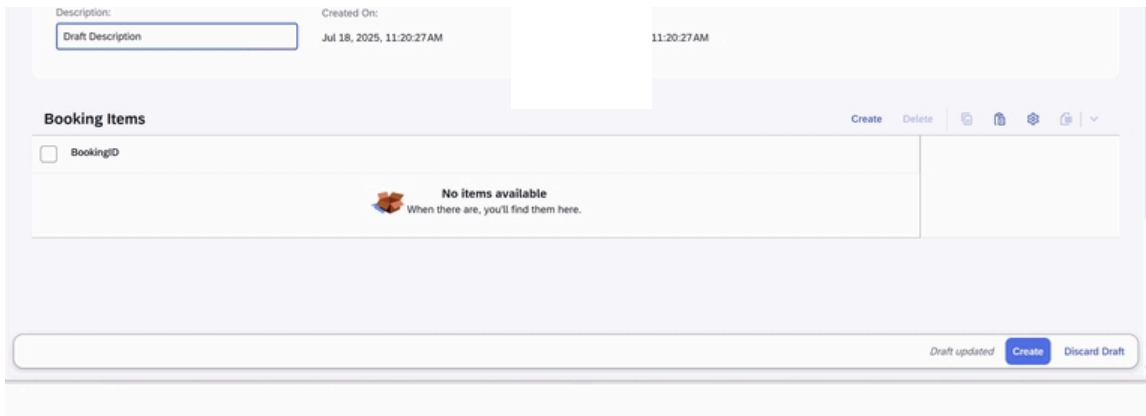
```

Result

As soon as you activate all your artifacts, the collaborative draft capability is enabled for your BO. A business user who has created a draft instance can now invite other business users to the draft instance with the aid of a user list. The business user can access the user list by navigating from the list report into the draft instance and clicking the add user icon.

Video of a User Invitation with the Aid of the User List





[Open this video in a new window](#)

Related Information

[Collaborative Draft Handling](#)

Developing Cross-BO Scenarios with Draft Scope

Using cross-BO draft enablement, you can edit multiple related business objects as a single logical unit of work (LUW), ensuring consistency across related draft instances. In this topic, you will learn how to add a draft scope to two separate, draft enabled BOs.

Prerequisites

- You need two independent, draft enabled BOs. The two BOs should be separate from each other and fully functional. For more information on how to develop a draft enabled BO, see: [Developing Transactional Apps with Exclusive Draft Capabilities](#).
- In a cross-BO scenario, one of the BOs must hold the keys of the other BO as the foreign key. For more information, see: [Foreign Key Relationship](#).

Procedure

In the ABAP flight reference scenario, you can find the package /DM0/FLIGHT_XB0. It includes a cross BO scenario with the two independent, draft enabled BOs Agency and Travel.

For the Travel BO, we show two different use cases in this cross-BO scenario, using two separate consumption layers:

- /DM0/C_TRAVELTP_XB0 is the conventional consumption layer. In this case, the Travel BO is independent from the Agency BO.
- /DM0/C_TRAVELTP_XB0_DRAFTDP is defined as an additional draft scope enabled consumption layer. In this case, Travel is the **dependent draft** BO, with Agency as the **principal draft** BO

→ Tip

The database table /DM0/A_AGENCY_XB0 uses agency_uuid as the key field, which is automatically filled with a UUID when an instance is created. The field agency_id serves as a semantic (human readable) id field.

Similarly, the database table /DM0/A_TRVL_XB0 uses the field travel_uuid as key field, with the field travel_id as the semantic id field. Moreover, it holds the keys of the Agency BO instances in the field agency_uuid

1. In the Agency data definition on the R-layer (/DMO/R_AGENCYTP_XB0), create and expose a to-many association (_Travel) from Agency to Travel, using the AgencyUUID as key.

≡ Sample Code

```
define root view entity /DMO/R_AGENCYTP_XB0
as select from /dmo/a_agncy_xbo
association to many /DMO/R_TRAVELTP_XB0 as _Travel on $projection.AgencyUUID = _Travel.Ager
{
  key agency_uuid as AgencyUUID,
  ...
  /* Associations */
  _Travel
}
```

2. Likewise, in the Travel data definition (/DMO/R_TRAVELTP_XB0), create and expose the association (_Agency) of many-to-one from Travel to Agency.

→ Tip

If you use the RAP BO generator, you might have to add the foreign key field agencyUUID to the Travel BO manually.

≡ Sample Code

```
define root view entity /DMO/R_TRAVELTP_XB0
as select from /dmo/a_trvl_xbo
association to one /DMO/R_AGENCYTP_XB0 as _Agency on $projection.AgencyUUID = _Agency.Agency
{
  key travel_uuid as TravelUUID,
  travel_id as TravelId,
  agency_uuid as AgencyUuid,
  ...
  /* Associations */
  _Agency
}
```

3. In the behavior definition of the dependent draft BO (/DMO/R_TRAVELTP_XB0), do the following:

- Add the statement with `cross associations;` to the header section of the BDEF.
- Declare the association `_Agency`, including the additions with `draft`, `link action`, `unlink action` and `inverse function`.
- If necessary, add a draft query view. For more information, see: [Draft Query Views](#)

i Note

For more information on cross-BO associations, refer to [Cross-BO Associations](#).

≡ Sample Code

```
managed implementation in class /dmo/bp_r_traveltp_xbo unique;
strict ( 2 );
extensible;
with draft;

with cross associations;

define behavior for /DMO/R_TRAVELTP_XB0 alias Travel
persistent table /dmo/a_trvl_xbo
```

```

extensible
draft table /dmo/d_trvl_xbo query /DMO/R_TravelDraft_XBO

...

association _Agency {
  with draft;
  link action linkAction;
  unlink action unlinkAction;
  inverse function inverseFunction;
}

```

4. In the behavior definition of the principal draft BO (/DMO/R_AGENCYTP_XBO), declare the association `_Travel` as a reverse association of `_Agency`, including the additions `create` and `with draft`.

→ Tip

It is recommended to add the statement “with cross associations” in the behavior definition header for every managed BO in a cross-BO scenario.

≡ Sample Code

```

...
with cross associations;
...
association _Travel using /DMO/R_TRAVELTP_XBO \_Agency{
  create;
  with draft;
}

```

5. Create a projection view for the principal draft BO on the C-layer (/DMO/C_AGENCYTP_XBO). Expose the association to the dependent draft BO (`_Travel`) and redirect it to the projection view of the dependent draft BO, which you are going to create in the next step. For this reason, don't activate the principal draft BO projection view yet.

≡ Sample Code

```

define root view entity /DMO/C_AGENCYTP_XBO
  provider contract transactional_query
  as projection on /DMO/R_AGENCYTP_XBO
{
  key AgencyUUID,

  ...

  /* Associations */
  _Travel: redirected to /DMO/C_TRAVELTP_XBO_DRAFTDP
}

```

6. Create a projection view for the dependent draft BO on the C-layer /DMO/C_TRAVELTP_XBO_DRAFTDP. Expose the association to the principal draft BO (`_Agency`) and redirect it to the projection view of the principal draft BO, which you have created in the previous step.

≡ Sample Code

```

define root view entity /DMO/C_TRAVELTP_XBO
  provider contract transactional_query
  as projection on /DMO/R_TRAVELTP_XBO
{
  key TravelUUID,

  ...
}

```

```

    /* Associations */
    _Agency: redirected to /DM0/C_AGENCYTP_XB0
}

```

7. Activate both projection views.

8. In the C-layer BDEF of the dependent draft BO (/DM0/C_TRAVELTP_XB0_DRAFTDP), add `use draft as dependent;` to the header section.

In the behavior body, use the association to the principal draft BO, and give it draft capability (`use association _Agency { with draft; }`).

→ Tip

If existing, remove the statement `use draft;` from the header section, as well as existing draft actions.

≡ Sample Code

```

projection;
strict ( 2 );
extensible;
use side effects;
use draft as dependent;
//use draft;

define behavior for /DM0/C_TRAVELTP_XB0_DRAFTDP alias Travel
extensible
use etag
{
    use create;
    use update;
    use delete;

    // use action Activate;
    // use action Discard;
    // use action Edit;
    // use action Resume;
    // use action Prepare;

    use association _Agency { with draft; }
}

```

9. In the C-layer BDEF of the principal draft BO (/DM0/C_AGENCYTP_XB0), use the association to the dependent draft BO (`_Travel`), and give it the additions `create;` and `with dependent draft;`.

→ Tip

Make sure that the C-layer BDEF of the principal draft BO has `use draft;` in the header and all necessary draft actions.

≡ Sample Code

```

projection;
strict ( 2 );
extensible;
use side effects;
use draft;

define behavior for /DM0/C_AGENCYTP_XB0 alias Agency
extensible
use etag
{
    use create;

```



```

    use update;
    use delete;

    use action Activate;
    use action Discard;
    use action Edit;
    use action Resume;
    use action Prepare;

    use association _Travel {
        create;
        with dependent draft;
    }
}

```

10. In the metadata extension of the projection view of the principal draft BO `/DM0/C_AGENCYTP_XB0`, add the `@UI . facet` annotation for the association directing to the dependent draft BO (`_Travel`). This ensures that the items of the dependent draft BO are displayed on the object page of the principal draft BO.

Sample Code

```

annotate view /DM0/C_AGENCYTP_XB0 with

{
    @UI.facet: [ ...
        { id: 'Travel',
          purpose: #STANDARD,
          type: #LINEITEM_REFERENCE,
          label: 'Travel',
          position: 40,
          targetElement: '_Travel' } ]
}

```

11. In the service definition of the principal draft BO (`/DM0/UI_AGENCYTP_04_XB0`), expose the dependent draft BO as well.

Sample Code

```

define service /DM0/UI_AGENCYTP_04_XB0
    provider contracts odata_v4_ui {
        expose /DM0/C_AGENCYTP_XB0 as Agency;
        expose /DM0/C_TRAVELTP_XB0_DRAFTDP as Travel;
    }
}

```

Result

When an instance of the principal draft BO (Agency) is set to edit mode and thus set to draft, all related instances of the dependent draft BO (Travel) are set to draft as well. Moreover, when an instance of the dependent draft BO is set to draft, all of its siblings are set to draft as well.

Related Information

[Draft Scope Definition in Cross-BO Scenarios](#)

Developing Apps with Hierarchical Data Structures

The following topics explain how to use CDS hierarchies in the context of the ABAP RESTful Application Programming Model (RAP), and expose the hierarchical structures as treeviews on the SAP Fiori UI.

ABAP CDS provides means to query hierarchical data stored in database tables. A typical example of a hierarchical data set connected by parent-child relationships is the organizational structure of a company. The root node of this hierarchy is the CEO, because the CEO doesn't have a parent node, that is, a superior employee. Each employee is represented by a hierarchy node, and the hierarchy edges represent the relationships between the employees, that is, manager – subordinate relationships or peer relationships of employees at the same level.

The SAP HANA database provides built-in functions for handling hierarchical data. ABAP SQL and ABAP CDS wrap these built-in functions so that you can use them directly in ABAP without any additional implementation. The SAP HANA hierarchy functions are described in [Hierarchy Functions](#).

You can implement treeviews with the following two capabilities:

Treeviews with Read-Only Capabilities

You can display the hierarchy representation on the UI. For conceptual information on treeviews, see [Treeviews](#).

To implement read-only treeviews, see [Treeviews with Read-Only Capabilities](#).

Editable Treeviews with Draft Capabilities

You can display and edit the hierarchy representation on the UI directly by using, for example, drag and drop or cut and paste. For conceptual information on treeviews, see [Treeviews](#).

To implement editable treeviews in your managed with draft business object, see [Editable Treeviews: Development Process in Overview](#).

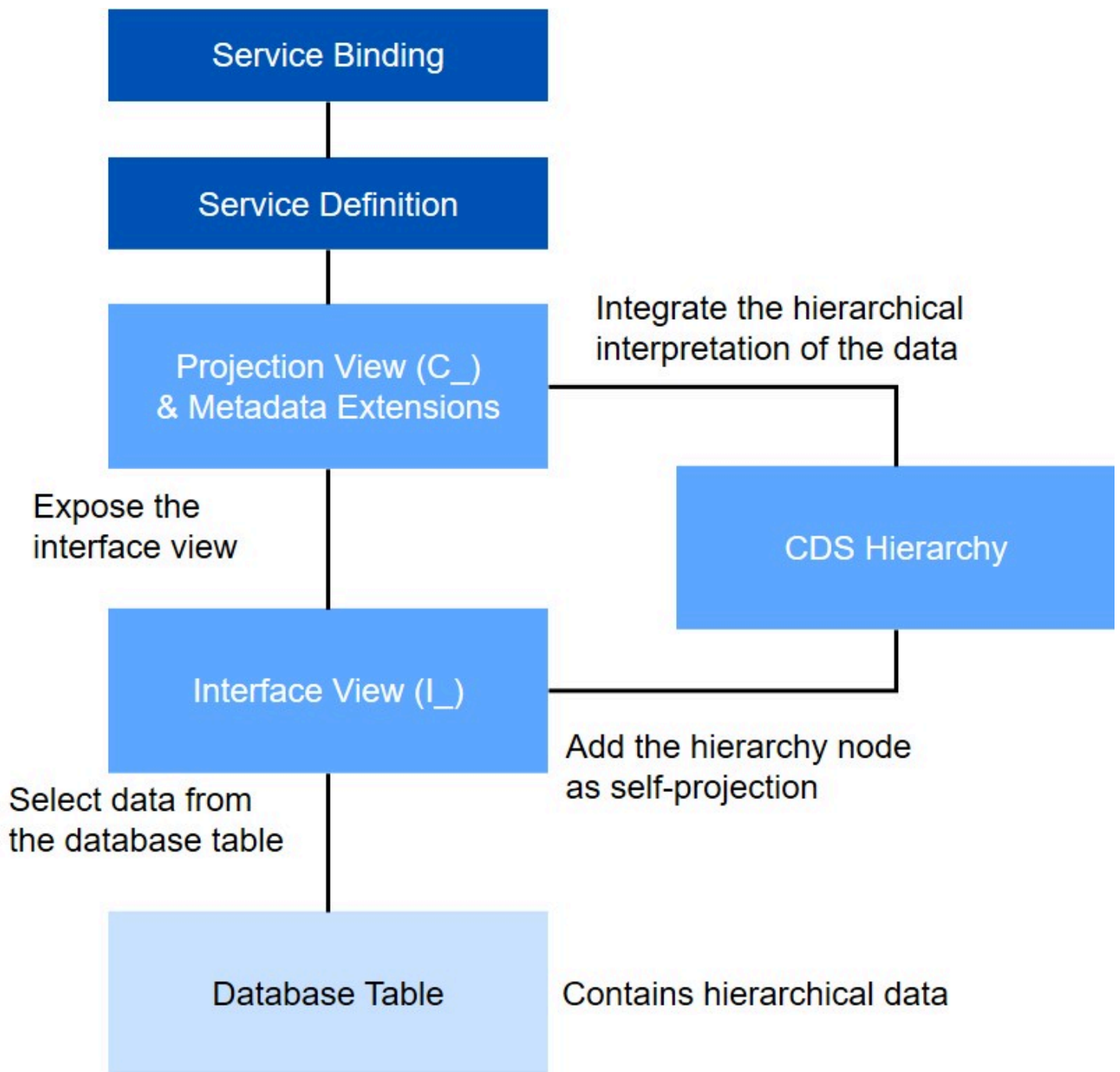
CDS Hierarchy Generation Wizard

The CDS Hierarchy Generation Wizard simplifies the creation of hierarchically-structured data in ABAP CDS by generating all necessary objects and allowing customization of the hierarchy's structure and properties. For more information, see [CDS Hierarchy Generation Wizard](#).

Treeviews with Read-Only Capabilities

In read-only mode, the following artifacts work together to define the hierarchical structure of the data and expose it to the UI.

1. Database table (/DMO/employee_hr): This is the underlying database table where the data is physically stored.
2. Interface view (/DMO/I_Employee_HR): This is the first layer, referring to the underlying database table. It contains a self-association to that same CDS view.
3. CDS hierarchy (/DMO/I_EmployeeHN_HR): This artifact adds the hierarchical interpretation to the data exposed by the interface view.
4. Projection view (/DMO/C_Employee_HR): The last entity is essentially what the front-end or application layer interacts with. It puts together the interface view with the hierarchical interpretation of the data using the annotation `@odata.hierarchy.recursiveHierarchy` that refers back to the hierarchy definition.
5. To complete the business object, you also need to create a service definition and a service binding to expose the selected data and its hierarchy to the UI.



Creating the Database Table

In this example, provided with the ABAP Flight Reference Scenario, we use the table `/dmo/employee_hr`. In this example table, each employee can also be a manager. Therefore, the employee ID (node ID) can also be a manager ID (parent node ID).

EMPLOYEE	FIRST_NAME	LAST_NAME	MANAGER
00000001	Friedrich	Meier	00000000
00000002	Johannes	Rahn	00000001
00000003	Lee	Neubasler	00000001
00000004	Holm	Illner	00000003
00000005	Max	Mustermann	00000004
00000006	Christoph	Mustermann	00000004
00000007	Max	Matthaeus	00000006
00000008	Thomas	Barth	00000006
00000009	Sophie	Rahn	00000006
00000010	Thomas	Mustermann	00000006

The column EMPLOYEE indicates the employee id for each employee. The column MANAGER indicates which employee is the manager for the respective person. As an example, employees 007-010 report to employee 006, Christoph Mustermann. Christoph Mustermann reports to employee 004, which is Holm Illner.

Employee 001, Friedrich Meier, is the head of the agency and thus has no manager above him. Therefore, his manager is indicated as zero.

Database Table /DMO/EMPLOYEE_HR

i Expand the following code sample to view the source code of database table /dmo/employee_hr.

Sample Code

```
@EndUserText.label : 'Hierarchy: Read Only: Employee'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/employee_hr {

    key client      : abap.clnt not null;
    key employee    : /dmo/employee_id not null;
    first_name      : /dmo/first_name;
    last_name       : /dmo/last_name;
    @EndUserText.label : 'Salary'
    @Semantics.amount.currencyCode : '/dmo/employee_hr.salary_currency'
    salary          : abap.curr(21,2);
    salary_currency : /dmo/currency_code;
    manager         : /dmo/employee_id;

}
```

Creating the Interface View

The data definition /DMO/I_Employee_HR is the interface view for table /DMO/employee_hr.

In this view, you need to add a self-association to the parent node of that same CDS view. This adds the hierarchical logic to the existing data, indicating that an employee ID can also serve as the manager ID for another employee. Since one manager can have several employees, the cardinality is defined as **many to one**.

Sample Code

```
define view entity /DMO/I_Employee_HR
as select from /dmo/employee_hr
association of many to one /DMO/I_Employee_HR
as _Manager on $projection.Manager = _Manager.Employee
```

Interface View /DMO/I_Employee_HR

 *Expand the following code sample to view the source code of data definition /DMO/I_Employee_HR.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Read Only: Employee'

define view entity /DMO/I_Employee_HR
as select from /dmo/employee_hr
association of many to one /DMO/I_Employee_HR as _Manager on $projection.Manager = _Manager.Employee
{
  key employee      as Employee,
    first_name      as FirstName,
    last_name       as LastName,

    @Semantics.amount.currencyCode: 'SalaryCurrency'
    salary          as Salary,
    salary_currency as SalaryCurrency,

    @EndUserText.label: 'Manager'
    manager         as Manager,

    _Manager
}
```

Defining the Hierarchy Interpretation

Define how the hierarchical data is interpreted with a parent-child hierarchy in the artifact /DMO/I_EmployeeHN_HR:

Sample Code

```
define hierarchy /DMO/I_EmployeeHN_HR as parent child hierarchy
```

Refer to the hierarchical data /DMO/I_Employee_HR as a source:

Sample Code

```
source /DMO/I_Employee_HR child to parent association _Manager
```

The hierarchy typically starts where there is no manager above, which is the CEO. Therefore, the root node of the hierarchy is where the manager ID is zero (initial).

Sample Code

```
start where Manager is initial
```

In this example, siblings are employees who report to the same manager. You can sort siblings by their last name, for example:

Sample Code

```
siblings order by LastName ascending
```

Hierarchy /DMO/I_EmployeeHN_HR

i *Expand the following code sample to view the source code of data definition /DMO/I_EmployeeHN_HR.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Read Only: Employee Hierarchy'
```

```
define hierarchy /DMO/I_EmployeeHN_HR
as parent child hierarchy(
  source /DMO/I_Employee_HR
  child to parent association _Manager
  start where
    Manager is initial
  siblings order by
    LastName ascending
)
{
  key Employee,
    Manager
}
```

Creating the Projection View

In the projection view, the exposed data from the interface view gets connected with the hierarchical interpretation of the data from the hierarchy entity.

To connect the hierarchy, add the following annotation, pointing to the hierarchy entity:

```
@odata.hierarchy.recursiveHierarchy:[{ entity.name: '/DMO/I_EmployeeHN_HR' }]
```

Define the projection view as a projection on the interface view:

Sample Code

```
define view entity /DMO/C_Employee_HR
  as select from /DMO/I_Employee_HR

  association of many to one /DMO/C_Employee_HR as _Manager
  on $projection.Manager = _Manager.Employee
```

Consumption View /DMO/C_Employee_HR

i Expand the following code sample to view the source code of data definition /DMO/C_Employee_HR.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Read Only: Employee'

@Metadata.allowExtensions: true
@Search.searchable: true

@OData.hierarchy.recursiveHierarchy:[{ entity.name: '/DMO/I_EmployeeHN_HR' }]

define view entity /DMO/C_Employee_HR
  as select from /DMO/I_Employee_HR

  association of many to one /DMO/C_Employee_HR as _Manager on $projection.Manager = _Manager.Em
{
  key Employee,

    @Search: {
      defaultSearchElement: true,
      fuzzinessThreshold: 0.87
    }
  FirstName,

    @Search: {
      defaultSearchElement: true,
      fuzzinessThreshold: 0.87
    }
  LastName,

  Salary,
  SalaryCurrency,
  Manager,
  /* Associations */
  _Manager
}
```

Creating the Business Services

To complete your RAP business object, add a service definition. In our example, this is `/DMO/UI_Employee_HR`.

i *Expand the following code sample to view the source code of data definition `/DMO/UI_Employee_HR`.*

Sample Code

```
@EndUserText.label: 'Hierarchy: Read Only: Employee'
@ObjectModel.leadingEntity.name: '/DMO/C_Employee_HR'
define service /DMO/UI_Employee_HR {
  expose /DMO/C_Employee_HR as Employee;
}
```

Moreover, you need to set up a Service Binding. In our example, this is `/DMO/UI_Employee_HR_04`.

For more information on how to create business services, see [Business Service](#).

Displaying the Treeview on the SAP Fiori UI

Create a metadata extension for the CDS projection view `/DMO/C_Employee_HR` for the display on the SAP Fiori UI.

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: {
  typeName: 'Employee',
  typeNamePlural: 'Employees',
  title: { type: #STANDARD, value: 'Employee' }
},
presentationVariant: [{
  sortOrder: [{ by: 'Employee', direction: #ASC }],
  visualizations: [{type: #AS_LINEITEM}]
}]
}
annotate view /DMO/C_Employee_HR
with
{
  @UI: {
    facet: [
      { id: 'Employee',
        purpose: #STANDARD,
        type: #IDENTIFICATION_REFERENCE,
        label: 'Employee',
        position: 10
      }
    ]
  }

  @UI: {
    lineItem: [{ position: 20 }],
    identification: [{ position: 20 }]
  }
}
```


Employee;

```
@UI: {
  selectionField: [{ position: 30 }],
  lineItem:       [{ position: 30 }],
  identification: [{ position: 30 }]
}
```

FirstName;

```
@UI: {
  selectionField: [{ position: 30 }],
  lineItem:       [{ position: 40 }],
  identification: [{ position: 40 }]
}
```

LastName;

```
@UI: {
  lineItem:       [{ position: 50 }],
  identification: [{ position: 50 }]
}
```

Salary;

```
@UI: {
  lineItem:       [{ position: 70 }],
  identification: [{ position: 70 }]
}
```

Manager;

}

When exposed to the SAP Fiori UI, the hierarchical structure is displayed as a collapsible treeview.

Employees (39,595)						
☐	Employee	≡	First Name	Last Name	Manager	
☐	▼ 1		Friedrich	Meier		>
☐	2		Johannes	Rahn	1	>
☐	▼ 3		Lee	Neubasler	1	>
☐	▼ 4		Holm	Illner	3	>
☐	5		Max	Mustermann	4	>
☐	▼ 6		Christoph	Mustermann	4	>
☐	7		Max	Matthaeus	6	>
☐	8		Thomas	Barth	6	>
☐	9		Sophie	Rahn	6	>
☐	10		Thomas	Mustermann	6	>

Editable Treeviews with Draft Capabilities

In this chapter, you will be guided through all steps necessary to implement editable treeviews based on the implementation type managed with draft of the ABAP RESTful Application Programming Model (RAP).

Why and When to Use Editable Treeviews

In the context of the RAP, a treeview visualizes data modeled by a CDS hierarchy, for example, by using a SAP Fiori Elements treetable. If you have the business need to display or edit hierarchical relationships, such as those between employees and their respective managers, you can use RAP to efficiently create treeviews and to display them on your SAP Fiori User Interface (UI).

Example

A Use Case of Editable Treeviews

You can use editable treeviews to maintain the hierarchical relationships between employees and their respective managers in a company. You can simply make changes to your editable treeviews on the UI by:

- reassigning a hierarchy node to another parent node by using drag and drop or cut and paste.
- moving a hierarchy node to a dedicated position among its siblings.
- deleting a hierarchy node including its children.
- editing inline data on a tree node directly on the user interface. For example, in an HR hierarchy rendered as a treeview, you can change the names and other employee data directly on the UI.

For conceptual information of treeviews, see [Treeviews](#).

For more information on treeview features, see [Treeview Features](#).

Architecture Overview

For an architecture overview, see [Architecture Overview: Editable Treeview with Draft](#).

Essential to the implementation of editable treeviews are hierarchy directories, see [Hierarchy Directories](#) for details.

Development Guide

To implement editable treeviews in your managed with draft business object, see [Editable Treeviews: Development Process in Overview](#).

Related Information

[Treeviews](#)

[Treeviews: Architecture Overview](#)

[Editable Treeviews: Development Process in Overview](#)

[Treeview Features](#)

[Hierarchy Directories](#)

Editable Treeviews: Development Process in Overview

In this chapter, we are using the ABAP Flight Reference Scenario to demonstrate how you can implement editable treeviews in your managed with draft business object. The ABAP Flight Reference Scenario contains demo content that you can refer to and use to build your own sample implementations.

Development Process in Overview

To implement editable treeviews in your managed with draft business object, you need to perform the following activities:

1. [Creating the Database Tables](#)
2. [Creating the Interface View](#)
3. [Creating the Root View Entity for the Directory](#)
4. [Creating the View Entity for the Directory as Composition Child](#)
5. [Defining the Structure of the Hierarchy](#)
6. [Creating the Behavior Definition for the Directory](#)
7. [Defining the Behavior for the Hierarchy](#)
8. [Creating the Projection Layer and the Metadata Extension](#)
9. [Finalizing the Business Service](#)

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).

→ Remember

The namespace `/DM0/` is reserved for the demo content. Apart from the downloaded ABAP Flight Scenario, do not use the namespace `/DM0/` and do not create any development objects in the downloaded packages.

You can access the development objects in `/DM0/` from your own namespace.

However, if you want to recreate all development objects of this demo content, make sure that you use different names from this documentation.

Related Information

[Managed Implementation Type](#)

Creating the Database Tables

As a first step, you have to create your database tables. In this demonstration, we are using the two active database tables /dmo/agency_a_hd and /dmo/employ_a_hd from the package /DMO/FLIGHT_HIERARCHY_DRAFT, which can be found after [Downloading the ABAP Flight Reference Scenario](#).

Below is a brief description of the two database tables.

Database Table /dmo/agency_a_hd

The /dmo/agency_a_hd database table lists out all agencies with its own unique agency id.

AGENCY	NAME	STREET	POSTAL_CODE	CITY	COUNTRY_CODE
004005	Your Choice	Gustav-Jung-Str. 425	90455	Nuernberg	DE
004006	Bella Italia	Via Marconi 123	00139	Roma	IT
004007	Hot Socks Travel	224 Balnagask Rd	8053	Sydney	AU
004008	Burns Nuclear	14 Science Park Drive	118228	Singapore	SG
004009	Honauer Reisen GmbH	Baumgarten 8	4212	Neumarkt	AT
004011	Voyager Enterprises	Gustavslundsvaegen 151	70563	Stockholm	SE
004012	Ben McCloskey Ltd.	74 Court Oak Rd	B17 9TN	Birmingham	GB
004013	Pillepalle Trips	Gorki Park 4	8008	Zuerich	CH
004014	Kangeroos	Lancaster drive 435	20001	London	GB
004015	Bavarian Castle	Pilnizerstr. 241	01069	Dresden	DE

Table 1. List View of Database Table /dmo/agency_a_hd.

Expand the following code sample to view the source code of database table /DMO/AGENCY_A_HD.

Sample Code

```
@EndUserText.label : 'Hierarchy: Draft: Agency'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/agency_a_hd {

    key client          : abap.clnt not null;
    key agency          : /dmo/agency_id not null;
    name                : /dmo/agency_name;
    street              : /dmo/street;
    postal_code         : /dmo/postal_code;
    city                : /dmo/city;
    country_code        : land1;
    phone_number        : /dmo/phone_number;
    email_address       : /dmo/email_address;
    web_address         : /dmo/web_address;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_created_by    : abp_creation_user;
    local_created_at    : abp_creation_tstmpl;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_last_changed_by : abp_locinst_lastchange_user;
    local_last_changed_at : abp_locinst_lastchange_tstmpl;
    last_changed_at     : abp_lastchange_tstmpl;
```

}

Database Table /dmo/employ_a_hd

The column AGENCY in the below /dmo/employ_a_hd database table shows the agency id for each of its respective employee. Under one agency, there are many employees. Some of the employees could also have the role of a manager. In the below screenshot, all employees shown are working in the same agency (004001) and they all have the same manager (00000096), so they are on the same hierarchy level. If you want to, you can assign each of them with a sibling order number, so that they would appear in a specific order on a list. For example, Kurt Leisert is assigned with the sibling order number 007, whereas Peter Illner with 008, which is the subsequent number. As a result, Peter will always appear after Kurt when you sort the SIBLING_ORDER_NUMBER in an ascending order.

AGENCY	EMPLOYEE	FIRST_NAME	LAST_NAME	MANAGER	SIBLING_ORDER_NUMBER
004001	00000098	Sophie	Heller	00000096	002
004001	00000099	Pierre	Zimmermann	00000096	003
004001	00000100	Theresia	Prinz	00000096	004
004001	00000101	Allen	Jacqmain	00000096	005
004001	00000102	Ruth	Kramer	00000096	006
004001	00000103	Kurt	Leisert	00000096	007
004001	00000104	Peter	Illner	00000096	008
004001	00000105	Roland	Benjamin	00000096	009
004001	00000106	Mathilde	Buehler	00000096	010
004001	00000107	Chantal	Hoffen	00000096	011
004001	00000108	Friedrich	Wohl	00000096	012
004001	00000109	August	Hansmann	00000096	013
004001	00000110	August	Kreiss	00000096	014

Table 2. List View of Database Table /dmo/employ_a_hd.

Expand the following code sample to view the source code of database table /DMO/EMPLOY_A_HD.

Sample Code

```
@EndUserText.label : 'Hierarchy: Draft: Employee'
@AbapCatalog.enhancement.category : #NOT_EXTENSIBLE
@AbapCatalog.tableCategory : #TRANSPARENT
@AbapCatalog.deliveryClass : #A
@AbapCatalog.dataMaintenance : #RESTRICTED
define table /dmo/employ_a_hd {

    key client          : abap.clnt not null;
    key agency          : /dmo/agency_id not null;
    key employee        : /dmo/employee_id not null;
    first_name          : /dmo/first_name;
    last_name           : /dmo/last_name;
    @Semantics.amount.currencyCode : '/dmo/employ_a_hd.salary_currency'
    salary              : /dmo/salary;
    salary_currency     : /dmo/currency_code;
    manager             : /dmo/employee_id;
    sibling_order_number : /dmo/sibling_order_number;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_created_by    : abp_creation_user;
```

```

    local_created_at      : abp_creation_tstmpl;
    @AbapCatalog.anonymizedWhenDelivered : true
    local_last_changed_by : abp_locinst_lastchange_user;
    local_last_changed_at : abp_locinst_lastchange_tstmpl;

}

```

Creating the Interface View

In our demonstration, we are using these two interface views `/DMO/I_Employee_HD` and `/DMO/I_Agency_HD`.

Interface view `/DMO/I_Agency_HD`

The data definition `/DMO/I_Agency_HD` is the interface view for table `/dmo/agency_a_hd`.

The association shown below has the cardinality of exact one to many. It implies that in each single agency, there can be many employees.

Sample Code

```
association of exact one to many /DMO/I_Employee_HD as _Employee on $projection.Agency = _Employ
```

Expand the following code sample to view the source code of data definition `/DMO/I_Agency_HD`.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@endUserText.label: 'Hierarchy: Draft: Agency'

@Search.searchable: true

define root view entity /DMO/I_Agency_HD
  as select from /dmo/agency_a_hd as Agency

  association of exact one to many /DMO/I_Employee_HD      as _Employee on $projection.Agency = _E

  association of one      to one I_Country                  as _Country  on $projection.CountryCode

{
  @ObjectModel.text.element: ['Name']
  key Agency.agency          as Agency,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.8
  @Semantics.text: true
  @Semantics.organization.name: true
  Agency.name              as Name,

  @Semantics.address.street: true

```

```

Agency.street                as Street,

@Semantics.address.zipCode: true
Agency.postal_code          as PostalCode,

@Search.defaultSearchElement: true
@Semantics.address.city: true
Agency.city                  as City,

@Consumption.valueHelpDefinition: [{entity: { name: 'I_CountryVH', element: 'Country' } }]
@Semantics.address.country: true
Agency.country_code          as CountryCode,


@Semantics.telephone.type: [#WORK]
Agency.phone_number          as PhoneNumber,

@Semantics.eMail.address: true
Agency.email_address          as EMailAddress,

Agency.web_address           as WebAddress,


@Semantics.user.createdBy: true
Agency.local_created_by      as LocalCreatedBy,

@Semantics.systemDateTime.createdAt: true
Agency.local_created_at      as LocalCreatedAt,

@Semantics.user.lastChangedBy: true
Agency.local_last_changed_by as LocalLastChangedBy,

@Semantics.systemDateTime.localInstanceLastChangedAt: true
Agency.local_last_changed_at as LocalLastChangedAt,

@Semantics.systemDateTime.lastChangedAt: true
Agency.last_changed_at       as LastChangedAt,

/* Associations */
_Employee,
_Country
}

```

Interface view /DMO/I_Employee_HD

The data definition /DMO/I_Employee_HD is the interface view for table /dmo/employ_a_hd. It selects data from the existing database table /dmo/employ_a_hd.

In this view, there are several associations. The first association specifies a many-to-one relationship between employees and agencies. It indicates that many employees can be associated with one agency.

Sample Code

```
association of many to one /DMO/I_Agency_HD as _Agency on $projection.Agency = _Agency.Agen
```

The second association specifies also a many-to-one relationship but with the same entity view (/DMO/I_Employee_HD) itself, indicating that an employee can have a manager, who is also an employee. This adds the hierarchical logic to the existing data, indicating that an employee ID can also serve as the manager ID for another employee. Since many employees are associated with one manager, the cardinality of the below association is defined as many to one.

Sample Code

```
association of many to one /DMO/I_Employee_HD as _Manager on $projection.Agency = _Manager.Age
and $projection.Manager = _Manager.E
```

The _Employee association enables you to identify all employees reporting to the current manager.

Sample Code

```
association of many to many /DMO/I_Employee_HD as _Employee on $projection.Agency = _Employee
and $projection.Employee = _Employ
```

Expand the following code sample to view the source code of data definiton /DMO/I_Employee_HD.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee'
```

```
define view entity /DMO/I_Employee_HD
as select from /dmo/employ_a_hd
```

```
association of many to one /DMO/I_Agency_HD as _Agency on $projection.Agency = _Agency.A
association of many to one /DMO/I_Employee_HD as _Manager on $projection.Agency = _Manager
and $projection.Manager = _Manager
association of many to many /DMO/I_Employee_HD as _Employee on $projection.Agency = _Employ
and $projection.Employee = _Employ
association to one I_Currency as _Currency on $projection.SalaryCurrency = _
{
  key agency as Agency,
  key employee as Employee,

  @Semantics.name.givenName: true
  first_name as FirstName,
  @Semantics.name.familyName: true
  last_name as LastName,

  @Semantics.amount.currencyCode: 'SalaryCurrency'
  salary as Salary,
  salary_currency as SalaryCurrency,

  @EndUserText.label: 'Manager'
  manager as Manager,

  sibling_order_number as SiblingOrderNumber,

  local_created_by as LocalCreatedBy,
  local_created_at as LocalCreatedAt,
  local_last_changed_by as LocalLastChangedBy,
  local_last_changed_at as LocalLastChangedAt,

  _Agency,
  _Manager,
```



```

    _Employee,
    _Currency
}

```

Creating the Root View Entity for the Directory

Create the transactional root view entity `/DMO/R_AgencyTP_HD` based on the interface view `/DMO/I_AGENCY_HD`.

To express the parent-child relationship between Agency and Employees, you have to define the composition. For more information on composition, see [Compositions](#). In our scenario, since under one agency, there are at least one or more hierarchical relationships between the employees, the cardinality of the composition is defined as exact one to many. This **exact one to many** specifies that one entry in the root view entity (`/DMO/R_AgencyTP_HD`) is associated with one or more entries in the child entity (`/DMO/R_EmployeeTP_HD`).

Sample Code

```
composition of exact one to many /DMO/R_EmployeeTP_HD as _Employee
```

Expand the following code sample to view the source code of data definition `/DMO/R_AgencyTP_HD`.

Sample Code

```

@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Agency'

define root view entity /DMO/R_AgencyTP_HD
  as select from /DMO/I_Agency_HD

  composition of exact one to many /DMO/R_EmployeeTP_HD as _Employee
{
  key Agency,
    Name,
    Street,
    PostalCode,
    City,
    CountryCode,
    PhoneNumber,
    EMailAddress,
    WebAddress,
    LocalCreatedBy,
    LocalCreatedAt,
    LocalLastChangedBy,
    LocalLastChangedAt,
    LastChangedAt,
    /* Associations */
    _Country,
    _Employee
}

```

Creating the View Entity for the Directory as Composition Child

Create the view entity `/DMO/R_EmployeeTP_HD` with the interface view `/DMO/I_Employee_HD` for the directory as compositional child. Here, you must define the to-parent association.

In our example, the association `_Agency` is used as the composition parent of the transactional hierarchy directory. The association `_Agency` connects each employee to their parent agency.

Sample Code

```
association          to parent /DM0/R_AgencyTP_HD as _Agency    on  $projection.Agency    = _Agency
```

Since several employees can be assigned to the same manager, the cardinality of the below association is defined as many to one.

Sample Code

```
association of many to one /DM0/R_EmployeeTP_HD as _Manager on  $projection.Agency    = _Manager
                                     and $projection.Manager    = _Manager
```

The third association has a many-to-many relationship. The `_Employee` association is the reverse association of the `_Manager` association. It carries precisely the mirrored association condition.

Sample Code

```
association of one to many /DM0/R_EmployeeTP_HD as _Employee on  $projection.Agency    = _Employee
                                     and $projection.Employee    = _Employee
```

i Expand the following code sample to view the source code of data definition `/DM0/R_EmployeeTP_HD`.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee'
```

```
define view entity /DM0/R_EmployeeTP_HD
  as select from /DM0/I_Employee_HD
```

```
association          to parent /DM0/R_AgencyTP_HD as _Agency    on  $projection.Agency = _Agency
association of many to one /DM0/R_EmployeeTP_HD as _Manager on  $projection.Agency = _Manager
                                     and $projection.Manager = _Manager
association of one to many /DM0/R_EmployeeTP_HD as _Employee on  $projection.Agency = _Employee
                                     and $projection.Employee = _Employee
association of one to one I_Currency          as _Currency on  $projection.SalaryCurrency =
```

```
{
  key Agency,
  key Employee,
    FirstName,
    LastName,
    @Semantics.amount.currencyCode: 'SalaryCurrency'
    Salary,
    SalaryCurrency,
    Manager,
    SiblingOrderNumber,
    LocalCreatedBy,
    LocalCreatedAt,
    LocalLastChangedBy,
    LocalLastChangedAt,
    /* Associations */
    _Agency,
    _Manager,
    _Employee,
    _Currency
}
```

Defining the Structure of the Hierarchy

Define the structure of the hierarchy in the data definition /DMO/I_EmployeeHN_HD.

In order for editable treeviews to work, hierarchies with directories are the only technically possible implementation. If you create an editable treeview without a directory, you will only get a read-only treeview whose tree nodes can be edited on their object pages, but none of the features to change the tree itself, such as drag and drop, and so on.

Whenever the hierarchy has a directory, the CDS syntax requires a filter condition which either uses a literal or a parameter. In our example, since the selection of the directory entry is variable, we are using a parameter.

Use the below parameter to allow the hierarchy to be filtered based on the agency ID.

Sample Code

```
define hierarchy /DMO/I_EmployeeHN_HD
  with parameters
    P_Agency : /dmo/agency_id
```

The parent-child hierarchy makes reference to the associations defined in the data definition DMO/I_Employee_HD.

Sample Code

```
as parent child hierarchy(
  source /DMO/I_Employee_HD
```

The child to parent association shown below indicates that the relationship is defined by the _Manager association. This association connects employees to their managers. For more information on child to parent association, see [Hierarchy Definition](#).

Sample Code

```
child to parent association _Manager
```

! Restriction

The hierarchy-to parent association must be a pure foreign key relation, that is, this association must precisely bind the hierarchy-parent primary key. For example, binding a non-primary semantic key is not supported.

The hierarchy directory of this example is defined by the association _Agency. Here, the directory filter works on the level of agency instances. As a result, the hierarchies shown in the treeview always contain only the employees from one selected single agency instance. Employees from multiple agency instances would never be mixed in one treeview. For more information, see [Hierarchy Directories](#).

Sample Code

```
directory _Agency filter by
  Agency = $parameters.P_Agency
```

In the HR hierarchy used here as an example, the treeview starts where there is no manager above. Therefore, the root node of the treeview is where the manager id is initial.

Sample Code

```
start where
    Manager is initial
```

You have the option to define the siblings order, for example, as ascending or descending. In our example below, it specifies that within the same hierarchical level, employees (siblings) should be ordered by the SiblingOrderNumber field in ascending order.

Sample Code

```
siblings order by
    SiblingOrderNumber ascending
```

You must include the key elements, and we also recommend you to include the foreign key elements of the hierarchy child-to-parent associations, but no further elements.

Sample Code

```
key Agency,
key Employee,
    Manager,
    SiblingOrderNumber
```

Expand the following code sample to view the source code of data definition /DMO/I_EmployeeHN_HD.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee HierarchyNode'

define hierarchy /DMO/I_EmployeeHN_HD
with parameters
    P_Agency : /dmo/agency_id

as parent child hierarchy(
    source /DMO/I_Employee_HD

    child to parent association _Manager

    directory _Agency filter by
        Agency = $parameters.P_Agency

    start where
        Manager is initial

    siblings order by
        SiblingOrderNumber ascending
)
{
    key Agency,
    key Employee,
        Manager,
        SiblingOrderNumber
}
```

Related Information

Creating the Behavior Definition for the Directory

Next, create the behavior definition `/DMO/R_AgencyTP_HD` based on the root view entity `/DMO/R_AgencyTP_HD`.

This demonstration is based on the implementation type managed with draft. Therefore, here you have to use managed with draft.

≡ Sample Code

```
managed;
strict ( 2 );
with draft;
```

Expose the composition to the hierarchy entity with `create` if you want to allow insertion of hierarchy instances.

≡ Sample Code

```
association _Employee { create; with draft; }
```

Generating the Draft Database Tables

If not already done so, in the source code of behavior definition `/DMO/R_AgencyTP_HD`, you can use quick fix to generate the draft database tables `/dmo/agency_d_hd` and `/dmo/employ_d_hd`.

For further information on how to draft-enable your managed business object, see [Draft-Enabling the Managed Business Object](#).

→ Remember

Whenever you change the transactional model, you must regenerate the draft table to align them. The behavior definition offers a quick fix for regeneration, but it is the responsibility of the application developer to keep the draft table in sync with the transactional model. Bear in mind that everytime when a draft database table is regenerated, it might be necessary to adjust the database table for preserving the existing data.

Expand the following code sample to view the source code of behavior definition `/DMO/R_AGENCYTP_HD`.

≡ Sample Code

```
managed;
strict ( 2 );
with draft;

define behavior for /DMO/R_AgencyTP_HD alias Agency
implementation in class /dmo/bp_r_agencytp_hd unique
early numbering
persistent table /dmo/agency_a_hd
draft table /dmo/agency_d_hd
lock master
total etag LastChangedAt
authorization master ( none )
etag master LocalLastChangedAt
{
  create;
  update;
```

```

delete;

field ( readonly ) Agency, LocalCreatedBy, LocalCreatedAt, LocalLastChangedBy, LocalLastChange
field ( mandatory ) CountryCode, EMailAddress, Name;

association _Employee { create; with draft; }

draft action Resume;
draft action Edit;
draft action Activate optimized;
draft action Discard;

validation validateEMailAddress on save { create; field EMailAddress; }
validation validateCountryCode on save { create; field CountryCode; }
validation validateName on save { create; field Name; }

draft determine action Prepare
{
  validation validateEMailAddress;
  validation validateCountryCode;
  validation validateName;
  validation Employee~validationSalaryPositive;
  validation Employee~validationCurrencyExists;
  validation Employee~validationNameGiven;
}

mapping for /dmo/agency_a_hd
{
  Agency          = agency;
  Name             = name;
  Street           = street;
  PostalCode       = postal_code;
  City             = city;
  CountryCode      = country_code;
  PhoneNumber      = phone_number;
  EMailAddress     = email_address;
  WebAddress       = web_address;
  LocalCreatedBy   = local_created_by;
  LocalCreatedAt   = local_created_at;
  LocalLastChangedBy = local_last_changed_by;
  LocalLastChangedAt = local_last_changed_at;
  LastChangedAt    = last_changed_at;
}

[...]
```

Related Information

[Draft-Enabling the Managed Business Object](#)

Defining the Behavior for the Hierarchy

In the source code of behavior definition /DMO/R_AgencyTP_HD, declare the following actions:

The Link and Unlink Action

Declare a link and unlink action to enable the reassignments of child nodes to parent nodes. It allows you to make the reassignment, for example, by dragging an employee entry and dropping it off to a new manager on the UI.

Sample Code

```

association _Manager
{
    with draft;
    link action linkManager;
    unlink action unlinkManager;
}

```

The Reorder Action

To enable moving a hierarchy node to a dedicated position among its siblings on the UI, you must declare a **reorder** action (either a managed or unmanaged one) in the behavior definition for the instance hierarchy.

For the managed reorder action, you must declare a **SiblingOrderNumber** field in the behavior definition **/DMO/R_AgencyTP_HD**. You must ensure that the value of this field matches that same single **SiblingOrderNumber** field in the CDS hierarchy **/DMO/I_EmployeeHN_HD**. Also pay attention that it must be a single field in the value of either **INT** or **NUMC**.

i Note

A newly created child node does not have a sibling order number by default. Sibling order numbers are only calculated by the RAP framework when the reorder action is performed.

When a node is assigned to another parent node, there can be duplicate sibling numbers until the reorder action is performed. To ensure that the sibling numbers are always unique and non-empty, you must implement a determination.

Moreover, for the managed reorder action, you must also declare an ascending association, which is the hierarchy-to-parent association and a corresponding reverse association as a descending association. Here, you have the option to add the declaration `{ with cascading delete; }`. With this declaration in place, whenever you delete a hierarchy node, the framework will also delete all its children.

Even though the cascading delete behavior is optional, the UI expects that the server cascades the deletion for subtrees and it doesn't behave consistently without the cascade as it doesn't send requests for multi-selected subtree nodes.

For more information, see [Treeview Features](#).

The Copy Action

You have the option to declare a copy action. The copy action determines how a hierarchy node can be duplicated in an editable treeview scenario. This action is useful when you want to create new hierarchy nodes based on an existing one in an editable treeview scenario. To implement a copy action, see [Implementing the Copy Action for the Hierarchy](#).

For conceptual information, see [Treeview Features](#).

≡ Sample Code

```

instance hierarchy /DMO/I_EmployeeHN_HD
{
    managed reorder action changeNextSibling;
    field ( hierarchy-index ) SiblingOrderNumber;
    ascending association _Manager;
    descending association _Employee { with cascading delete; }
    copy action copy;
}

```

Expand the following code sample to view the source code of behavior definition /DMO/R_AgencyTP_HD.

≡ Sample Code

```
managed;
strict ( 2 );
with draft;

define behavior for /DMO/R_AgencyTP_HD alias Agency
[...]
{
  [...]

define behavior for /DMO/R_EmployeeTP_HD alias Employee
implementation in class /dmo/bp_r_employeetp_hd unique
early numbering
persistent table /dmo/employ_a_hd
draft table /dmo/employ_d_hd
lock dependent by _Agency
authorization dependent by _Agency
etag master LocalLastChangedAt
{
  update;
  delete;

  field ( readonly ) Agency, Employee, SiblingOrderNumber;
  field ( readonly ) Manager;

  association _Agency { with draft; }
  association _Manager
  {
    with draft;
    link action linkManager;
    unlink action unlinkManager;
  }

  association _Employee { with draft; }

instance hierarchy /DMO/I_EmployeeHN_HD
{
  managed reorder action changeNextSibling;
  field ( hierarchy-index ) SiblingOrderNumber;
  ascending association _Manager;
  descending association _Employee { with cascading delete; }
  copy action copy;
}

validation validationSalaryPositive on save { create; field Salary; }
validation validationCurrencyExists on save { create; field SalaryCurrency; }
validation validationNameGiven on save { create; field FirstName, LastName; }

determine action validateSalaryPositive { validation validationSalaryPositive; }
determine action validateCurrencyExists { validation validationCurrencyExists; }
determine action validateNameGiven { validation validationNameGiven; }

side effects
{
  determine action validateSalaryPositive executed on field Salary affects messages;
  determine action validateCurrencyExists executed on field SalaryCurrency affects messages;
  determine action validateNameGiven executed on field FirstName, field LastName affects messa
}

mapping for /dmo/employ_a_hd
{
  Agency          = agency;
  Employee         = employee;
  FirstName        = first_name;
  LastName         = last_name;
  Salary           = salary;
  SalaryCurrency   = salary_currency;
  Manager          = manager;
  SiblingOrderNumber = sibling_order_number;
  LocalCreatedBy   = local_created_by;
```



```

        LocalCreatedAt      = local_created_at;
        LocalLastChangedBy = local_last_changed_by;
        LocalLastChangedAt = local_last_changed_at;
    }

}

```

Related Information

[Treeview Features](#)

Implementing the Copy Action for the Hierarchy

The copy action determines how a hierarchy node can be duplicated in an editable treeview scenario. It is especially useful when you want to create new hierarchy nodes based on an existing one in an editable treeview scenario.

For conceptual information, see [Treeview Features](#).

For details on syntax, see [RAP - Copy Action \(ABAP Keyword Documentation\)](#).

Definition

In the source code of behavior definition /DMO/R_AgencyTP_HD, under instance hierarchy for employee, declare the copy action. Use the syntax `copy action copy;`.

Sample Code

```

instance hierarchy /DMO/I_EmployeeHN_HD
{
    managed reorder action changeNextSibling;
    field ( hierarchy-index ) SiblingOrderNumber;
    ascending association _Manager;
    descending association _Employee { with cascading delete; }
    copy action copy;
}

```

Via a quick fix, you can generate the method declaration for the copy action implementation in the behavior pool directly from the behavior definition editor.

Implementation

The following is an implementation example of a copy action. In this example, the implementation of the methods in the `lhc_hierarchy_operation` class handle hierarchical copy operations for employee data. Within this `lhc_hierarchy_operation` class, there are two methods, namely, `copy` and `_copy_recursive`.

Be aware that the `_copy_recursive` method makes use of a recursive copying logic to copy employee data, so that all the child nodes of an employee are also copied. However, a recursive copying logic is not a prerequisite for a copy action to take place. For instance, if you only want to copy the data of one single employee but not all the subordinates reporting to this person, you do not need to apply a recursive copying logic. You should always consider your own use case when deciding on how your implementation should look like.

The following steps guide you through the implementation.

1. Declare the `_copy_recursion` method to handle recursive operations. Import essential parameters for processing copy actions. Export relevant fields to categorize employees according to the operation's status.
2. Invoke the `_copy_recursion` method within the `copy` method.
3. Perform a read operation to get the information of employee instance to be copied.
4. Introduce a `LOOP AT` iteration over the keys of the instance to be copied and make sure to handle the case of invalid keys, such as, duplicates or other failures, by filling the parameters `failed` and `reported` accordingly.
5. Perform a read-by-association operation to gather data about employees and their links to agencies. Make sure to fill the `failed` parameter accordingly.
6. Loop across the table holding the directory links and specify the fields you want to copy.
7. If an employee has child nodes, make sure that you copy them as well.
8. Make sure to remove any children links in case `failed` is filled. Then, fill the `parents_copy_map` with values provided from the read-by-association.
9. Iterate this also for every child. Use recursive calls to copy child entities within hierarchy using `result_rba_children` and insert them into `parents_copy_map`.

Expand the following code sample to see the source code of the `copy` and the `_copy_recursive` methods.

Sample Code

```
CLASS lhc_hierarchy_operation DEFINITION FINAL INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
```

```
  TYPES:
```

```
    copy_keys      TYPE TABLE FOR ACTION IMPORT /DMO/R_EMPLOYEEETP_HD~copy,
    mapped         TYPE TABLE FOR MAPPED /DMO/R_EMPLOYEEETP_HD,
    failed         TYPE TABLE FOR FAILED /DMO/R_EMPLOYEEETP_HD,
    reported       TYPE TABLE FOR REPORTED EARLY /DMO/R_EMPLOYEEETP_HD,
    failed_agencies TYPE TABLE FOR FAILED /DMO/R_AGENCYTP_HD.
```

```
  TYPES BEGIN OF parent_copy_map.
```

```
  TYPES:
```

```
    agency      TYPE /DMO/R_AGENCYTP_HD-agency,
    employee    TYPE /DMO/R_EMPLOYEEETP_HD-employee,
    is_draft    TYPE abp_behv_flag,
    copy_employee TYPE /DMO/R_EMPLOYEEETP_HD-employee.
```

```
  TYPES END OF parent_copy_map.
```

```
  TYPES parents_copy_map TYPE SORTED TABLE OF parent_copy_map WITH UNIQUE KEY agency employee
```

```
  METHODS copy FOR MODIFY IMPORTING keys FOR ACTION employee~copy.
```

```
  "Declare the _copy_recursion method to handle recursive operations.
```

```
  "Import essential parameters for processing copy actions.
```

```
  "Export relevant fields to categorize employees according to the operation's status.
```

```
  METHODS _copy_recursion IMPORTING keys              TYPE copy_keys
                           parents_copy_map_param    TYPE parents_copy_map OPTIONAL
                           EXPORTING mapped_employees TYPE mapped
                           failed_employees          TYPE failed
                           reported_employees        TYPE reported.
```

ENDCLASS.

CLASS lhc_hierarchy_operation IMPLEMENTATION.

METHOD copy.

"Invoke the _copy_recursion method.

```
_copy_recursion( EXPORTING keys = keys
                  IMPORTING mapped_employees = mapped-employee
                          failed_employees = failed-employee
                          reported_employees = reported-employee ).
```

ENDMETHOD.

METHOD _copy_recursion.

CONSTANTS: my_cid_kid TYPE abp_behv_cid VALUE 'My%CID_KID' ##NO_TEXT.

DATA:

```
employees_cba    TYPE TABLE FOR CREATE /DMO/R_AGENCYTP_HD\_employee,
employees_rba    TYPE TABLE FOR READ IMPORT /DMO/R_EMPLOYEETP_HD\_employee,
employee_rba     LIKE LINE OF employees_rba,
parents_copy_map TYPE parents_copy_map.
```

FIELD-SYMBOLS <control> TYPE x.

CLEAR: mapped_employees, failed_employees.

"Read to get the information of employee instance to be copied.

READ ENTITY IN LOCAL MODE /DMO/R_EMPLOYEETP_HD

ALL FIELDS WITH CORRESPONDING #(keys)

RESULT DATA(result_sources)

REPORTED DATA(reported_source)

FAILED DATA(failed_source).

"Iterate over the keys of the instance to be copied and make sure to handle the case of invalid LOOP AT keys INTO DATA(key).

READ TABLE keys ASSIGNING FIELD-SYMBOL(<dupl_key>) WITH KEY id COMPONENTS %tky = key-%tky

IF <dupl_key>-%cid <> key-%cid.

```
failed_source-employee = VALUE #( ( %fail    = VALUE #( cause = if_abap_behv=>cause
                                %cid    = key-%cid
                                Agency   = key-Agency
                                Employee = key-Employee ) ).
```

INSERT LINES OF failed_source-employee INTO TABLE failed_employees.

CLEAR failed_source.

APPEND VALUE #(%cid = key-%cid

%tky = key-%tky

%msg = new_message_with_text(severity = if_abap_behv_message=>severity

```

INSERT LINES OF reported_source-employee INTO TABLE reported_employees.
CLEAR reported_source.

```

```
ELSE.
```

```
READ TABLE result_sources ASSIGNING FIELD-SYMBOL(<source>) WITH KEY id COMPONENTS %tky =
```

```
IF sy-subrc <> 0.
```

```

failed_source-employee = VALUE #( ( %fail      = VALUE #( cause = if_abap_behv=>cause
                                   %cid       = key-%cid
                                   Agency     = key-Agency
                                   Employee   = key-Employee ) ).

```

```

INSERT LINES OF failed_source-employee INTO TABLE failed_employees.
CLEAR failed_source.

```

```

APPEND VALUE #( %cid      = key-%cid
                %tky      = key-%tky
                %msg      = new_message_with_text(
                severity = if_abap_behv_message=>severity-error ) ) TO reported_source

```

```

INSERT LINES OF reported_source-employee INTO TABLE reported_employees.
CLEAR reported_source.

```

```
ENDIF.
```

```
ENDIF.
```

```
ENDLOOP.
```

"Perform a read-by-association operation to gather data about employees and their links to a
 "Make sure to handle the filling of failed parameter accordingly.

```

READ ENTITY IN LOCAL MODE /DMO/R_EMPLOYEE_TP_HD
  BY \_agency
  FIELDS ( agency ) WITH CORRESPONDING #( result_sources )
LINK DATA(directory_links)
RESULT DATA(result_rba)
REPORTED DATA(reported_rba)
FAILED failed_source.

```

```
INSERT LINES OF failed_source-employee INTO TABLE failed_employees.
```

"Loop into the directory links and specify the fields you want to copy.
 LOOP AT directory_links ASSIGNING FIELD-SYMBOL(<directory_link>).

```
ASSIGN employees_cba[ KEY id %tky = <directory_link>-target-%tky ] TO FIELD-SYMBOL(<cba>)
```

```
IF sy-subrc <> 0.
```

```

  INSERT VALUE #( %tky = <directory_link>-target-%tky ) INTO TABLE employees_cba ASSIGNING
ENDIF.

```

```

ASSIGN result_sources[ KEY id %tky = <directory_link>-source-%tky ] TO FIELD-SYMBOL(<source>)
ASSERT sy-subrc = 0.

```

```

INSERT VALUE #( %data = CORRESPONDING #( <source_directory> ) ) INTO TABLE <cba>-%target .

IF parents_copy_map_param IS SUPPLIED.
    ASSIGN parents_copy_map_param[   agency   = <source_directory>-agency
                                   employee = <source_directory>-manager
                                   is_draft  = <source_directory>-%is_draft ] TO FIELD-SYMB

    ASSERT sy-subrc = 0.
    <cba_target>-manager = <parent_copy_map>-copy_employee.

ELSE.
    CLEAR <cba_target>-manager.
ENDIF.

employee_rba-%tky = <source_directory>-%tky.
<cba_target>-%cid = keys[ KEY id %tky = <source_directory>-%tky ]-%cid.
<cba_target>-%is_draft = <source_directory>-%is_draft.

" business logic: fields to be copied
<cba_target>-firstname      = <source_directory>-firstname.
<cba_target>-lastname       = <source_directory>-lastname.
<cba_target>-salary         = <source_directory>-salary.
<cba_target>-salarycurrency = <source_directory>-salarycurrency.
<cba_target>-siblingordernumber = <source_directory>-siblingordernumber.
" business logic end

ASSIGN <cba_target>-%control TO <control> CASTING.
CLEAR <control> WITH if_abap_behv=>mk-on IN BYTE MODE. "as long as there is no selective i
CLEAR: <cba_target>-%control-agency, <cba_target>-%control-employee.
INSERT employee_rba INTO TABLE employees_rba.

ENDLOOP.

READ ENTITY IN LOCAL MODE /DMO/R_EMPLOYEE_TP_HD
  BY \_employee FROM employees_rba
LINK DATA(children_links)
RESULT DATA(result_rba_children)
FAILED DATA(failed_rba_children)
REPORTED DATA(reported_rba_children).

"If an employee has child nodes, make sure to copy them as well.
MODIFY ENTITY IN LOCAL MODE /DMO/R_AGENCY_TP_HD
  CREATE BY \_employee FROM employees_cba
MAPPED DATA(mapped_create)
FAILED DATA(failed_create)
REPORTED DATA(reported_create).

"Make sure to remove any children links in case failed is filled.
"Then, fill the parents_copy_map with values provided from the read by association.
LOOP AT failed_create-employee ASSIGNING FIELD-SYMBOL(<failed>).
  DELETE children_links USING KEY draft WHERE source-%tky = <failed>-%tky.
ENDLOOP.

```

```

IF NOT parents_copy_map_param IS SUPPLIED.
    mapped_employees = mapped_create-employee.
ENDIF.

IF result_rba_children IS INITIAL.
    RETURN.
ENDIF.

LOOP AT employees_rba ASSIGNING FIELD-SYMBOL(<rba>).
    ASSIGN mapped_create-employee[ KEY cid COMPONENTS %cid = keys[ KEY id %tky = <rba>-%tky ] ]-
    '
    CHECK sy-subrc = 0.
    ASSERT <mapped>-agency      = <rba>-agency AND <mapped>-%is_draft = <rba>-%is_draft.

    INSERT VALUE #( agency      = <rba>-agency
                    employee    = <rba>-employee
                    is_draft    = <rba>-%is_draft
                    copy_employee = <mapped>-employee ) INTO TABLE parents_copy_map.
ENDLOOP.

"Iterate this also for every child. Use recursive calls to copy child entities within hierar
"using result_rba_children and insert them into parents_copy_map.
_copy_recursion( keys = VALUE #( FOR <child> IN result_rba_children INDEX INTO lv_index
                                ( %cid = my_cid_kid && lv_index %tky = <child>-%tky ) )
                parents_copy_map_param = parents_copy_map ).

ENDMETHOD.
ENDCLASS.

```

Related Information

[Treeview Features](#)

[Defining the Behavior for the Hierarchy](#)

Creating the Projection Layer and the Metadata Extension

Creating the Projection Layer

In the projection view, the exposed data from the base CDS view entity gets connected with the hierarchical interpretation of the data from the hierarchy entity.

To connect the hierarchy, add the following annotation in the data definition /DMO/C_EmployeeTP_HD :

```
@OData.hierarchy.recursiveHierarchy: [{ entity.name: '/DMO/I_EmployeeHN_HD' }]
```

Define the projection view as a projection on the root view entity (base CDS view entity) /DMO/R_EmployeeTP_HD:

Sample Code

```
define view entity /DMO/C_EmployeeTP_HD
  as projection on /DMO/R_EmployeeTP_HD
```

Define the projection view for view entity /DMO/R_AgencyTP_HD:

Sample Code

```
define root view entity /DMO/C_AgencyTP_HD
  provider contract transactional_query
  as projection on /DMO/R_AgencyTP_HD
```

i *Expand the following code sample to view the source code of data definition /DMO/C_EmployeeTP_HD.*

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Hierarchy: Draft: Employee'

@Metadata.allowExtensions: true

@OData.hierarchy.recursiveHierarchy: [{ entity.name: '/DMO/I_EmployeeHN_HD' }]

@Search.searchable: true

define view entity /DMO/C_EmployeeTP_HD
  as projection on /DMO/R_EmployeeTP_HD

{
  @ObjectModel.text.element: ['AgencyName']
  key Agency,
  key Employee,

  _Agency.Name      as AgencyName,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.87
  FirstName,

  @Search.defaultSearchElement: true
  @Search.fuzzinessThreshold: 0.87
  LastName,
  Salary,
  @Consumption.valueHelpDefinition: [
    { entity: {
      name: 'I_CurrencyStdVH',
      element: 'Currency'
    },
    useForValidation: true
  }
]
  SalaryCurrency,
```

```

    @Consumption.valueHelpDefinition: [
      {
        entity: { name: '/DMO/I_Employee_ManagerVH_HD', element: 'Employee' },
        additionalBinding: [{ element: 'Agency', localElement: 'Agency', usage: #FILTER }]
      }
    ]
    Manager,
    _Manager.FirstName as ManagerFirstName,
    _Manager.LastName as ManagerLastName,
    SiblingOrderNumber,
    LocalLastChangedAt,

    /* Associations */
    _Agency : redirected to parent /DMO/C_AgencyTP_HD,
    _Manager : redirected to /DMO/C_EmployeeTP_HD,
    _Employee : redirected to /DMO/C_EmployeeTP_HD,
    _Currency

  }

```

Add projection in the behavior definition /DMO/C_AgencyTP_HD

Sample Code

```

projection;
strict ( 2 );
use draft;
use side effects;

```

Don't forget to reuse the reorder action with the syntax `use action changeNextSibling`. You must also specify the `link` and `unlink` action when you reuse the association `_Manager` with the syntax `use association _Manager`.

Sample Code

```

use action changeNextSibling;
use action copy;

use association _Agency { with draft; }
use association _Manager
{
  with draft;
  link action linkManager;
  unlink action unlinkManager;
}

```

Expand the following code sample to view the source code of behavior definition /DMO/C_AgencyTP_HD.

Sample Code


```

projection;
strict ( 2 );
use draft;
use side effects;

define behavior for /DMO/C_AgencyTP_HD alias Agency
use etag
{
    use create;
    use update;
    use delete;

    use action Resume;
    use action Edit;
    use action Activate;
    use action Discard;
    use action Prepare;

    use association _Employee { create; with draft; }
}

define behavior for /DMO/C_EmployeeTP_HD alias Employee
use etag
{
    use update;
    use delete;

    use action changeNextSibling;
    use action copy;

    use association _Agency { with draft; }
    use association _Manager
    {
        with draft;
        link action linkManager;
        unlink action unlinkManager;
    }
    use association _Employee { with draft; }
}

```

Creating the Metadata Extension

For the projection view, we are creating two metadata extensions /DMO/C_EmployeeTP_HD and /DMO/C_AgencyTP_HD for the display on the SAP Fiori UI.

Expand the following code sample to view the source code of metadata extension /DMO/C_EmployeeTP_HD.

Sample Code

```

@Metadata.layer: #CORE

@UI:
{
    headerInfo:
    {

```

```

        typeName: 'Employee',
        typeNamePlural: 'Employees',
        description: { type: #STANDARD, value: 'Employee' },
        title: { type: #AS_CONNECTED_FIELDS, valueQualifier: 'FullName' }
    },
    presentationVariant:
    [
        {
            sortOrder: [{ by: 'Employee', direction: #ASC }],
            visualizations: [{type: #AS_LINEITEM}]
        }
    ]
}

```

```

annotate view /DMO/C_EmployeeTP_HD with
{

```

```

@UI:
{
    facet:
    [
        {
            id: 'Employee',
            purpose: #STANDARD,
            type: #COLLECTION,
            label: 'Employee',
            position: 10
        },
        {
            id: 'General',
            targetQualifier: 'General_FG',
            parentId: 'Employee',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'General',
            position: 10
        },
        {
            id: 'Name',
            targetQualifier: 'Name_FG',
            parentId: 'Employee',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Name',
            position: 20
        },
        {
            id: 'Salary',
            targetQualifier: 'Salary_FG',
            parentId: 'Employee',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Salary Data',
            position: 30
        }
    ],
    hidden: true
}
Agency;

@UI: {
    lineItem: [{ position: 20 }],
    fieldGroup: [{ position: 20, qualifier: 'General_FG' }]
}
Employee;

@UI: {
    selectionField: [{ position: 30 }],
    lineItem: [{ position: 30 }],
    identification: [{ position: 30 }],
    fieldGroup: [{ position: 30, qualifier: 'Name_FG' }],
    connectedFields: [{
        qualifier: 'FullName',

```

```

        groupLabel: 'Full Name',
        name: 'First',
        template: '{First} {Last}'
    ]]
}
FirstName;

@UI: {
    lineItem: [{ position: 40 }],
    fieldGroup: [{ position: 40, qualifier: 'Name_FG' }],
    connectedFields: [{
        qualifier: 'FullName',
        name: 'Last'
    }]
}
LastName;

@UI: {
    lineItem: [{ position: 50 }],
    fieldGroup: [{ position: 50, qualifier: 'Salary_FG' }]
}
Salary;

@UI: {
    lineItem: [{ position: 70 }],
    fieldGroup: [{ position: 70, qualifier: 'General_FG', type: #AS_CONNECTED_FIELDS, valueQua
    connectedFields: [{
        qualifier: 'ManagerFullName',
        groupLabel: 'Manager',
        name: 'ID',
        template: '{ID} {First} {Last}'
    }]
}
Manager;

@UI: {
    connectedFields: [{
        qualifier: 'ManagerFullName',
        name: 'First'
    }]
}
ManagerFirstName;

@UI: {
    connectedFields: [{
        qualifier: 'ManagerFullName',
        name: 'Last'
    }]
}
ManagerLastName;

@UI: {
    lineItem: [{ position: 80 }],
    fieldGroup: [{ position: 80, qualifier: 'General_FG' }]
}
SiblingOrderNumber;

@UI.hidden: true
LocalLastChangedAt;
}

```

Expand the following code sample to view the source code of metadata extension /DMO/C_AgencyTP_HD.

Sample Code

```

@Metadata.layer: #CORE

@UI: { headerInfo: {
    typeName: 'Agency',

```

```

        typeNamePlural: 'Agencies',
        title: { type: #STANDARD, value: 'Name' },
        description: { type: #STANDARD, value: 'Agency' }
    },
    presentationVariant: [{
        sortOrder: [{ by: 'Agency', direction: #ASC }],
        visualizations: [{type: #AS_LINEITEM}]
    }]
}

```

annotate view /DM0/C_AgencyTP_HD with

```

{
    @UI.facet: [
        {
            id: 'Agency',
            purpose: #STANDARD,
            type: #COLLECTION,
            label: 'Agency',
            position: 10
        },
        {
            id: 'General',
            targetQualifier: 'General_FG',
            parentId: 'Agency',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'General',
            position: 10
        },
        {
            id: 'Address',
            targetQualifier: 'Address_FG',
            parentId: 'Agency',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Address',
            position: 20
        },
        {
            id: 'Contact',
            targetQualifier: 'Contact_FG',
            parentId: 'Agency',
            purpose: #STANDARD,
            type: #FIELDGROUP_REFERENCE,
            label: 'Contact Data',
            position: 30
        },
        {
            id: 'Employee',
            purpose: #STANDARD,
            type: #LINEITEM_REFERENCE,
            label: 'Employee',

```

```

        position:      20,
        targetElement: '_Employee'
    }
}
]
@UI: { lineItem:      [ { position: 10, importance: #HIGH } ] ,
      selectionField: [ { position: 10 } ] }
@UI.textArrangement: #TEXT_SEPARATE
Agency;

@UI: { lineItem:      [ { position: 20, importance: #HIGH } ] ,
      fieldGroup:     [ { position: 20, qualifier: 'General_FG' } ],
      selectionField: [ { position: 20 } ] }
Name;

@UI: { fieldGroup:     [ { position: 30, qualifier: 'Address_FG' } ],
      identification: [ { position: 30 } ] }
Street;

@UI: { fieldGroup:     [ { position: 40, qualifier: 'Address_FG' } ],
      identification: [ { position: 40 } ] }
PostalCode;

@UI: { lineItem:      [ { position: 50, importance: #MEDIUM, type: #AS_FIELDGROUP, valueQuali
      identification: [ { position: 50 } ],
      fieldGroup:     [ { position: 50, qualifier: 'Address_FG' } ],
      selectionField: [ { position: 50 } ] }
City;

@UI: { identification: [ { position: 60 } ],
      fieldGroup:     [ { position: 60, qualifier: 'Address_FG' } ],
      selectionField: [ { position: 60 } ] }
@UI.textArrangement: #TEXT_FIRST
CountryCode;

@UI: { lineItem:      [ { position: 70, importance: #LOW, type: #AS_FIELDGROUP, valueQualifie
      fieldGroup:     [ { position: 70, qualifier: 'Contact_FG' } ],
      identification: [ { position: 70 } ] }
PhoneNumber;

@UI: { fieldGroup:     [ { position: 80, qualifier: 'Contact_FG' } ],
      identification: [ { position: 80 } ] }
EmailAddress;

@UI: { fieldGroup:     [ { position: 90, qualifier: 'Contact_FG' } ],
      identification: [ { position: 90 } ] }
WebAddress;

@UI.hidden: true
LocalLastChangedAt;
}

```

Finalizing the Business Service

To complete the service exposure, add a service definition. In our example, this is the service definition `/DMO/UI_Agency_HD`.

i Expand the following code sample to view the source code of service definition `/DMO/UI_Agency_HD`.

Sample Code

```
@EndUserText.label: 'Agency'
@ObjectModel.leadingEntity.name: '/DMO/C_AgencyTP_HD'
define service /DMO/UI_Agency_HD {
  expose /DMO/C_AgencyTP_HD as Agency;
  expose /DMO/C_EmployeeTP_HD as Employee;
  expose I_CurrencyStdVH as Currency;
}
```

Moreover, you need to set up a service binding. In our example, this is the service binding `/DMO/UI_Agency_HD_04`.

For more information on how to create business services, see [Business Service](#).

Finally, when exposed to the SAP Fiori UI, the hierarchical structure is displayed as a collapsible treeview.

Employees (39,593)						
Employee		First Name	Last Name		Salary	Manager
1	⌵	Friedrich	Meier	128,147.24	EUR	
2		Johannes	Rahn	59,133.76	EUR	1
3	⌵	Lee	Neubasler	90,975.40	EUR	1
4	⌵	Holm	Illner	79,575.07	EUR	3
168	⌵	Andreas	Martin	84,177.44	EUR	3
725	⌵	Felix	Benz	72,435.73	EUR	3
787	⌵	Georg	Benz	61,311.15	EUR	3
798	⌵	Anna	Babilon	76,002.62	EUR	3
799		Guenther	Neubasler	52,552.62	EUR	798
800	⌵	Johann	Wohl	69,110.67	EUR	798
815	⌵	Martin	Koslowski	66,392.76	EUR	798

Related Information

[Business Service](#)

Developing Transactional Apps with Multi-Inline-Edit Capabilities

The following section guides you through the implementation steps to create an application with multi-inline-edit capabilities.

Introduction

Multi-inline-edit capabilities are always necessary if you want the end user of a UI app to be able to edit, add, and delete all instances directly on a Fiori UI list report without navigating to the object page of one instance. It is always relevant for customizing and maintaining complete table data. Multi-inline-capabilities on the UI represent the same feature scope as transaction SM30 in SAP GUI.

The following preview displays a multi-inline-edit app for maintaining and administrating flight carrier information. All instances can be edited on the same screen in inline mode. New instances can be added or deleted via the create and delete button on the UI.

i *Expand the following item to view an animated GIF of Multi-Inline-Editing on a Fiori UI.*

Multi-Inline-Editing on a Fiori Elements UI

Multi-Inline-Editing on a Fiori UI

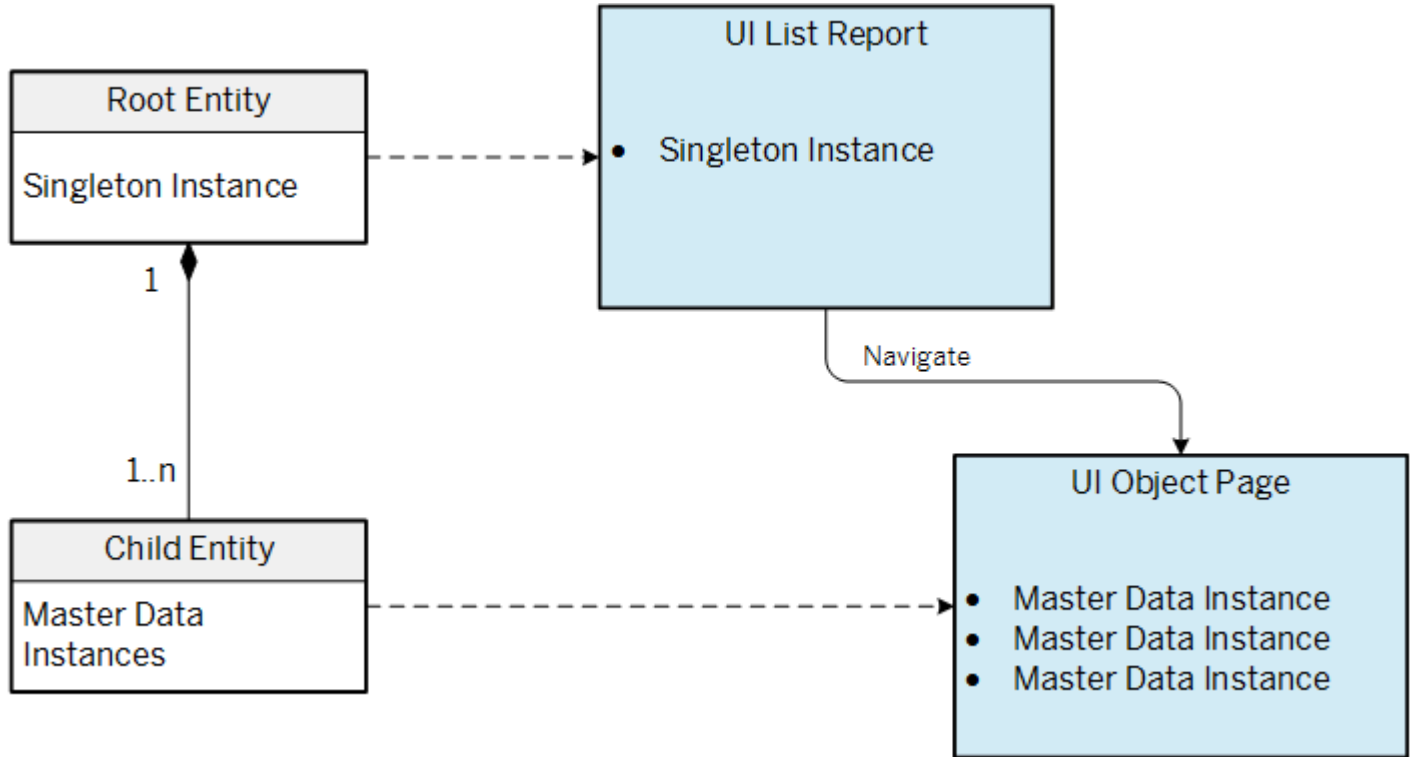
Technical Architecture of a Multi-Inline-Edit App

The technical requirements of a RAP BO that enables multi-inline-editing on a Fiori Elements UI app are defined by the structure of the Fiori Elements UI. A simple list reporting app consists of a list report page, on which all instances of the root view entity are displayed. From any instance, you can navigate to its object page, which displays all the instances of the child entity in a list, if the UI facet of the object page is modeled correctly. The list of child instances can always be edited in multi-inline-edit mode, which means by choosing **Edit** on the instance's object page, all fields of every child instance are editable at the same time. In this multi-inline-edit mode, you can change fields of different instances and then save the complete composition structure (root entity instance and all its child instances) at the same time. Since RAP BOs use a lock master/dependent mechanism, it is ensured that by choosing the **Edit** button, all instance of the child entity are locked.

The existing multi-inline-edit capabilities of the list of child instances are used to enable multi-inline-editing also for standalone master data entities. In this case, standalone refers to entities that are by default root entities in a RAP BO.

To enable multi-inline-editing also for default root view entities, RAP offers the solution to model a technical root entity with just one instance, from which you can navigate to its object page. This singleton pattern provides a single-entrance point to all master data instances. The technical root entity also provides the technical requirement of a lock master entity, which is essential for multi-inline-editing on the child entity. The master data entity is then modeled as a child entity in order to enable the standard multi-inline-edit capabilities of the object page of a UI. Like this, you can edit the master data of all instances at the same time.

In other words, for this scenario, you need to model a root entity with a single instance which has a composition to the master data entity that is editable as a list on the object page of the single root instance on the UI.



Multi-Inline-Scenario with Singleton Pattern in RAP BO

i Note

You cannot expose the same business object in a regular list report app and in a multi-inline-edit-enabled app at the same time. To expose a BO as a multi-inline-edit-scenario, a dummy root entity needs to be introduced and changes the common BO structure drastically.

Example Scenario

This example scenario develops an app to configure airline master data of the flight reference scenario. The master data contains the airline ID, the full name and the currency code that the airline uses. The data of all airlines shall be editable at the same time on one list.

The scenario is based on the database table `/dmo/carrier` and the interface view `/DMO/I_Carrier`. The transactional view `/DMO/I_Carrier_S` of the new business object selects from the interface view and thus, the new BO is built on it.

This development guide illustrates how a BO for a multi-inline-edit scenario is developed from scratch. It does not show development steps to migrate a regular business object into a business object for a multi-inline-edit scenario.

The example scenario can be downloaded into your system from Git Hub. You find the carrier multi-inline-edit scenario in the package `/DMO/FLIGHT_REAUSE_CARRIER`.

Prerequisites

Developing the scenario that is described in the subsequent chapters requires the following:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

- You fulfill the requirements described in [Prerequisites](#).
- You have installed ABAP Development Tools (ADT).

SAP recommends to use the latest version of the client installation. The ADT download is available on the update site <https://tools.hana.ondemand.com/>.

- To recreate the demo scenario, the **ABAP Flight Reference Scenario** must be available in your ABAP system. You can download the complete reference scenario from **GitHub**: [Downloading the ABAP Flight Reference Scenario](#).

Development Steps

[Defining the Data Model and the Business Object Structure](#)

The multi-inline-edit scenario uses flight carrier master data of the ABAP Flight Reference Scenario. The final app processes carrier data in multi-inline-edit mode.

[Developing Transactional Behavior](#)

The multi-inline-edit scenario implements some standard behavior for draft BOs and additional specific behavior that is required to enable multi-inline-editing on the UI.

[Projecting the BO for UI Exposure](#)

For the multi-inline-edit scenario, create a business object projection for UI exposure.

[Defining and Publishing a UI Service with Multi-Inline-Edit Capabilities](#)

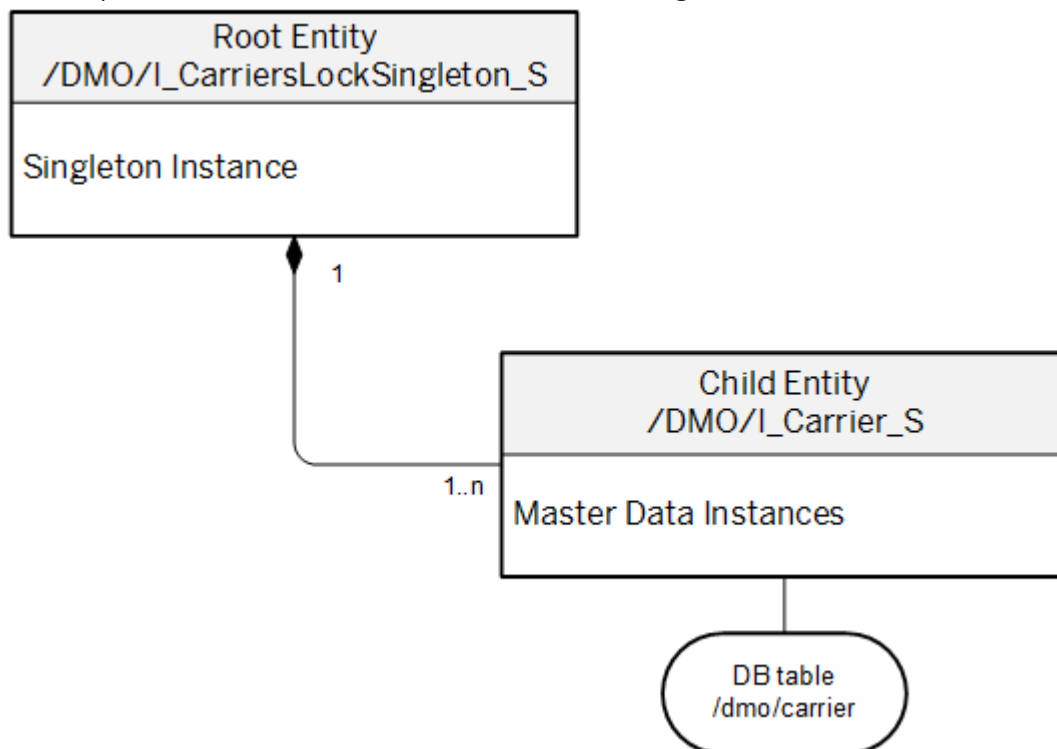
For the multi-inline-edit, define a UI service and bind it to a protocol.

Defining the Data Model and the Business Object Structure

The multi-inline-edit scenario uses flight carrier master data of the ABAP Flight Reference Scenario. The final app processes carrier data in multi-inline-edit mode.

The RAP business object in this scenario consists of two BO entities. The root entity represents the singleton entry point, whereas the first child entity comprises the general carrier data, like an ID, the full name and the currency code that is used by the airline.

The composition tree of the Carrier BO looks like the following.



Carrier Business Object with Singleton Root Entity

The root entity `/DM0/I_CarriersLockSingleton_S` is a technical artifact to ensure the single entrance point to the list of carrier data and the lock master for all carrier instances. It always retrieves only one dummy instance that has no relevance for any data of the BO. The real data substance of the BO lies in the child entity `/DM0/I_Carrier_S`. This CDS view selects from the database table `/dmo/carrier`, where the data is stored.

Tasks for the Developer

- [Creating the Singleton Root Entity](#)
- [Creating the Master Data Entity](#)

Creating the Singleton Root Entity

For the multi-inline-edit scenario, create a data definition for a CDS view entity that defines the singleton pattern for the BO.

Context

The multi-inline-edit scenario requires a singleton root entity with exactly one instance that represents the single entrance point for the carrier data list on the UI.

The singleton pattern is implemented in a RAP BO by one simple trick: You model a root entity that selects from an always existing CDS view and restrict the selection result with a where clause to ensure that there is always only one single result. Such a CDS view can be the released CDS view `I_Language`, which is always available in your SAP system. By restricting the selection result to the session language, you receive the required result. There is always only one session language, which is by default always available in the select list of the CDS view `I_Language`. The actual instance that is modeled in the root CDS view entity does not contain elements of the underlying CDS view `I_Language`, but defines a key element with a dummy value and other elements that are relevant for the draft processing. It is important to point out that the underlying CDS view `I_Language` does not have any relevance for the scenario. It just ensures the singleton mechanism for a RAP business object.

Procedure

1. Create a data definition for a CDS view entity to model the singleton pattern for the BO.

In ADT, use the wizard for creating CDS data definitions.

Select the template **Define Root View Entity** for the singleton entity and enter the name and a description for the view entity.

The CDS view entity `/DM0/I_CarriersLockSingleton_S` uses the CDS view `I_Language` as a data source. This CDS view is released for every system and can be used for any RAP BO. The language data is not used in the singleton entity.

2. Model the root view entity as a singleton entity to receive a single entrance point for the UI.

Restrict the selection list by the system language with a where clause. This ensures that there is always only one instance that is retrieved for requests on this entity.

Define a dummy field as key for the instance. You can use any hard-coded value.

3. Define a total ETag field to enable draft capabilities for the BO.

In this scenario, the total ETag in the root view is determined by the maximum value of all its child instances. The child carrier instances use the field `last_changed_at` as ETag fields. Whenever an instance is updated the field receives a new value and the total ETag is updated.

To get access to the data of the child instance, model a left outer join with the database table of the child instances `/dmo/carrier`. The join must use a dummy join condition as there is no field on the left side that matches any field on the right side of the join. By using a dummy join condition such as `on 0 = 0`, every instance (there is just one) receives the fields that are selected from the joined data source.

Add the maximum of the field `last_changed_at` of the carrier database table to the select list of the root view entity.

4. Define the composition structure of the carrier BO.

Once the CDS view entity of the child entity `/DMO/I_Carrier_S` exists, define the composition to is and use the association in the select list.

i Expand the following code sample to view the source code of CDS root view entity `/DMO/I_CarriersLockSingleton_S`.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Carrier Singleton Root View'
@Semantics.valueRange.maximum: '1'

define root view entity /DMO/I_CarriersLockSingleton_S
as select from I_Language
    left outer join /dmo/carrier as carr on 0 = 0
    composition [0..*] of /DMO/I_Carrier_S as _Airline
{
    key 1
        as CarrierSingletonID,
        max (carr.last_changed_at) as LastChangedAtMax,
        _Airline
}

where
    I_Language.Language = $session.system_language
```

Creating the Master Data Entity

For the multi-inline-edit scenario, create a data definition for a CDS view entity that selects the carrier master data.

Context

The multi-inline-edit scenario requires that the master data entity is modeled as a child entity of a root entity that enables a singleton mechanism.

Procedure

1. Create a data definition for a CDS view entity to model the carrier master data.

In ADT, use the wizard for creating CDS data definitions.

Select the template **Define View Entity with To-Parent Association** for the child entity and enter the name and a description for the view entity.

The CDS view entity `/DMO/I_Carrier_S` uses the database table `/dmo/carrier` as data source. All fields are selected.

2. Define the association to the parent view entity.

Include a dummy field with the same hard-coded value as the key field in the root view entity in the select list. This field is used as the on-condition of the association.

3. Use the `@Semantics` annotations that automatically update the administrative fields in the managed scenario.
4. Suppress search and text annotations that are inherits from the underlying CDS view by using the header annotation `@Metadata.ignorePropagatedAnnotations: true`.

i Expand the following code sample to view the source code of CDS view entity `/DMO/I_Carrier_S`.

Sample Code

```
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'Transactional Carrier View'
@Metadata.ignorePropagatedAnnotations: true

define view entity /DMO/I_Carrier_S
as select from /DMO/I_Carrier as Airline
association
to parent /DMO/I_CarriersLockSingleton_S as _CarrierSingleton on $projectio

{
  key AirlineID,
  1 as CarrierSingletonID,

  Name,

  CurrencyCode,

  @Semantics.user.createdBy: true
  LocalCreatedBy,
  @Semantics.systemDateTime.createdAt: true
  LocalCreatedAt,
  @Semantics.user.lastChangedBy: true
  LocalLastChangedBy,
  //local etag field --> odata etag
  @Semantics.systemDateTime.localInstanceLastChangedAt: true
  LocalLastChangedAt,

  //total etag field
  @Semantics.systemDateTime.lastChangedAt: true
  LastChangedAt,

  /* Associations */
  _CarrierSingleton
}
```

Developing Transactional Behavior

The multi-inline-edit scenario implements some standard behavior for draft BOs and additional specific behavior that is required to enable multi-inline-editing on the UI.

Context

The behavior for the singleton business object is defined in a behavior definition. Since the carrier business object is draft-enabled, there is some standard draft behavior that must be available for the draft BO. Other behavior results as a consequence of the special singleton pattern that is used in this BO.

Procedure

Create the behavior definition by using the creation wizard in ADT. By using the context menu on the root view (`/DMO/I_CarriersLockSingleton_S`), you can directly create a behavior definition for the carrier singleton data model. In the wizard, choose the implementation type **Managed**.

Standard Behavior in the Template

The template for the `managed` implementation types provides a suggestion for the behavior pool and the strict addition as best practice for RAP BOs. For every entity in the composition tree a behavior definition is added and the persistent table is specified. As the root entity uses a join on its data source, a persistent table is not found.

The standard operations `create`, `update` and `delete` are included for each entity. The create operation on child entities is not supported in RAP scenarios. Instances of child entities can only be created by the create-by-association operation, which is also included for parent entities.

The template also proposes to include the behavior characteristics `lock`, `authorization` and `etag`. The master for lock and authorization is defined on the root entity with dependents on the child entity. The strict mode requires that authorization is defined for the business object. With the addition `none`, no authorization checks are executed. This is a demo application, which is why the addition `none` is used. This approach is not recommended in real-world scenarios and violates best practices. For more information, see [Authorization Control](#). Since this carrier BO only has little behavior that must implemented, you can use one implementation class for both entities. The template proposes the name of the behavior pool that is used for both entities.

The root entity instance can never be modified. Therefore it does not need an ETag. The ETag master is defined on the child entity with the ETag field `LocalLastChangedAt`. It is recommended that you specify the respective ETag field as read-only in the behavior definition, using the field characteristic `readonly`.

The child entity uses different field names in the CDS view than on the database table. Add a mapping definition for the carrier child entity for all fields.

Singleton-Specific Behavior

Since the root view entity is only designed as an entrance point to the app, it is expected that there is always only just one single instance. It is not intended that you can create, update, or delete instances on the root entity. Delete this modification behavior from the template on the root entity.

The implementation type `managed` requires the specification of a persistent table for each BO entity. For the root view entity, there is no persistent table, which leads to a syntax error in the behavior definition. Modification on the root view entity is not allowed in this scenario. Therefore, no persistent table is required. To solve the syntax issue in the behavior definition, define an `unmanaged save` for the root view entity with the syntax `with unmanaged save`. Since this scenario does not intend to save any modification of the root entity instances at all, the saver implementation method in the behavior pool remains empty. It is then called during the save sequence of the BO, but does not change anything for the single root view entity instance.

Set the fields `CarrierSingletonID` in the root entity to read-only. As no modification is allowed on the root entity, the field should not be editable on the UI.

Draft-Specific Behavior

To draft-enable the carrier business object, define `with draft` in the behavior definition header.

Draft-enablement requires that a draft database table is available for each entity. By adding `draft table` and a name for the draft table on each entity, you can make use of the behavior definition quick fix that triggers the creation of a draft table. The draft table for the root entity `/dmo/d_carr_lock` is just a dummy table that is not used, as no modification on the root entity instance is possible in this scenario. The draft table for the child entity `/dmo/d_carrier` is used for the draft life cycle management of carrier draft instances.

The draft-enablement also requires the definition of a total ETag field. The field `LastChangedAtMax` is calculated in the root view entity to be used as total ETag field.

Add the addition with `draft` to the associations `association _Airline {create;}` and `association _CarrierSingleton;` to explicitly define that the association is always followed with regards to the draft indicator. This is automatically implicitly enabled, but for transparency reasons, it is best practice to explicitly add it to the behavior definition. The field `CarrierSingletonID` is used in the on-condition for the to-parent association `_CarrierSingleton` in the child entity. It must be set to read-only to ensure that the connecting element between parent and child cannot be changed.

All master fields of the child entity are mandatory as no instance should exist without the `AirlineID`, `Name` or `CurrencyCode`. In addition, the key field `AirlineID` must be read-only on update.

Define all draft actions for the root entity.

i Expand the following code sample to view the source code of behavior definition `/DMO/I_CarriersLockSingleton_S`.

Sample Code

```
managed implementation in class /dmo/bp_i_carrierslocksingl_s unique;

strict(2);
with draft;

define behavior for /DMO/I_CarriersLockSingleton_S alias CarriersLockSingleton
with unmanaged save
draft table /dmo/d_carr_lock
lock master
total etag LastChangedAtMax
authorization master ( none )

{
  association _Airline { create; with draft; }

  field ( readonly ) CarrierSingletonID, LastChangedAtMax;

  draft action Edit;
  draft action Activate optimized;
  draft action Discard;
  draft action Resume;
  draft determine action Prepare
}

define behavior for /DMO/I_Carrier_S alias Carrier
persistent table /dmo/carrier
draft table /dmo/d_carrier
lock dependent by _CarrierSingleton
authorization dependent by _CarrierSingleton
etag master LocalLastChangedAt
{
  update;
  delete;

  field ( readonly ) CarrierSingletonID, LocalCreatedBy, LocalCreatedAt, LocalLastChangedBy, LocalLastChangedAt;
  field ( mandatory : create, readonly : update ) AirlineID;
  field ( mandatory ) Name, CurrencyCode;

  association _CarrierSingleton { with draft; }

  mapping for /dmo/carrier
  { AirlineID = carrier_id;
    CurrencyCode = currency_code;
    Name = name;
    LocalCreatedBy = local_created_by;
    LocalCreatedAt = local_created_at;
    LocalLastChangedBy = local_last_changed_by;
    LocalLastChangedAt = local_last_changed_at;
    LastChangedAt = last_changed_at;
  }
}
```

Developing Feature Control for the Carrier Entity

For the multi-inline-edit scenario, enable and disable functionality with feature control on the carrier entity.

Context

The carrier data that is used in this scenario to demonstrate multi-inline-edit capabilities on a Fiori UI is also used by other business objects as master data for travel instances. The booking entity in a travel business object has an association to the carrier interface view and validates if the carrier that is used in a booking instance exists on the database table for carrier data. This ensures that only existing airlines are used. Up to now, the carrier data remained stable as no other business object modified data on the database table. With the current scenario, this changes. When maintaining the carrier data, it must be ensured that no carrier instances can be deleted that is already used by another business object. This is enforced by defining feature control for the delete operation. The instance feature control checks if the airline is used in a flight connection that is maintained on the database table `/dmo/connection`. The delete operation is only enabled if the airline ID is not used in a connection.

Procedure

Define feature control for the delete operation on the carrier entity.

Sample Code

```
define behavior for /DMO/I_Carrier_S alias Carrier
...
{
  update;
  delete ( features : instance );
... }
```

Use the quick fix in the behavior definition to generate the method definition for the instance feature control handler in the behavior pool.

The implementation is done in the method `FOR INSTANCE FEATURES`.

Read all the instances with the imported keys into an internal table and use the `AirlineID` to select the corresponding `AirlineIDs` from the connection interface view `/DMO/I_Connection` to check if the airline IDs in the carrier BO are used in flight connections.

Disable the delete operation in the result table of the method if the respective `AirlineID` exists in a flight connection. If it doesn't exist, set the permission value to `enabled`.

Fill the reported table with an adequate information message that is returned in the response structure of the feature control method.

i *Expand the following code sample to view the source code of `GET_INSTANCE_FEATURES`.*

Sample Code

```
CLASS lhc_carrier DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

  ...
  METHODS get_instance_features FOR INSTANCE FEATURES
    IMPORTING keys REQUEST requested_features FOR Carrier RESULT result.
```

ENDCLASS.

CLASS lhc_carrier IMPLEMENTATION.
METHOD get_instance_features.

DATA: airline_ids TYPE SORTED TABLE OF /dmo/i_carrier WITH UNIQUE KEY AirlineID.

READ ENTITIES OF /DMO/I_CarriersLockSingleton_S IN LOCAL MODE
ENTITY Carrier
FIELDS (AirlineID)
WITH CORRESPONDING #(keys)
RESULT DATA(Carrier)
FAILED failed.

airline_ids = CORRESPONDING #(Carrier DISCARDING DUPLICATES MAPPING AirlineID = AirlineID E

IF airline_ids IS NOT INITIAL.
SELECT DISTINCT AirlineID
FROM /DMO/I_Connection
FOR ALL ENTRIES IN @airline_ids
WHERE AirlineID = @airline_ids-AirlineID
INTO TABLE @DATA(connections_db).
ENDIF.

LOOP AT Carrier ASSIGNING FIELD-SYMBOL(<carrier>).
" Delete is not allowed if any Connection exists
IF line_exists(connections_db[AirlineID = <carrier>-AirlineID]).
APPEND VALUE #(%tky = <carrier>-%tky
%delete = if_abap_behv=>fc-o-disabled) TO result.
APPEND VALUE #(%tky = <carrier>-%tky
%msg = NEW /dmo/cx_carriers_s(
textid = /dmo/cx_carriers_s=>airli
severity = if_abap_behv_message=>sev
airline_id = <carrier>-AirlineID)
%delete = if_abap_behv=>mk-on
%path-carrierslocksingleton = CORRESPONDING #(<carrier>)
) TO reported-carrier.
ELSE.
APPEND VALUE #(%tky = <carrier>-%tky
%delete = if_abap_behv=>fc-o-enabled) TO result.
ENDIF.
ENDLOOP.

ENDMETHOD.

ENDCLASS.

Developing Validations

For the multi-inline-edit scenario, define and implement validations for value consistency checks.

Context

Carrier instances are only completely usable if the values for each field are supplied. An airline without a name or currency code doesn't make any sense. It is therefore indispensable that the values are checked on creating new instances and on editing. The fields of carrier instances must not be empty. By defining the key field `AirlineID` to be mandatory on create and read-only on update, the RAP runtime engine ensures that the value is supplied when creating a new instance and that it does not get changed during the modification process. The other fields `Name` and `CurrencyCode` are the developer's responsibility. These fields must be checked by validations. Before saving a draft instance to the persistent table, the validations are executed and instances without all values are rejected.

The validations must have the trigger condition `create` and the corresponding field.

Procedure

Define validations to check the **Name** and the **CurrencyCode** when creating new carrier instances and editing these fields on the carrier entity.

Sample Code

```
define behavior for /DM0/I_Carrier_S alias Carrier
...
{
  validation validateName on save { create; field Name; }
  validation validateCurrencyCode on save { create; field CurrencyCode; }
... }
```

The messages of validations are only correctly presented on the UI, if the validations are defined for the draft determine action **Prepare** on the root entity.

Sample Code

```
define behavior for /DM0/I_CarrierLockSingleton_S alias Carrier
...
{
  draft determine action Prepare
  {
    validation Carrier ~ validateName;
    validation Carrier ~ validateCurrencyCode;
  }

... }
```

Use the quick fix in the behavior definition to generate the method definition for the validation handler in the behavior pool.

The implementation is done in the methods **FOR VALIDATE ON SAVE**.

Validation **ValidateName**

Read all the instances with the imported keys into an internal table. Pass the failed entries to the **failed** table of the validation.

Loop over the internal table **Carriers**. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area. If the respective field is initial, write the instance identifier to the failed table and append a corresponding message to the respective state area.

 *Expand the following code sample to view the source code of **validateName**.*

Sample Code

```
CLASS lhc_carrier DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS validateName FOR VALIDATE ON SAVE
      IMPORTING keys FOR Carrier~validateName.

ENDCLASS.
```

```

CLASS lhc_carrier IMPLEMENTATION.
  METHOD validateName.

```

```

  READ ENTITIES OF /DMO/I_CarriersLockSingleton_S IN LOCAL MODE
  ENTITY Carrier
    FIELDS ( CarrierSingletonID Name )
    WITH CORRESPONDING #( keys )
  RESULT DATA(Carrier).

```

```

  " Raise message for empty Airline Name
  LOOP AT Carrier ASSIGNING FIELD-SYMBOL(<carrier>).

```

```

    APPEND VALUE #( %tky      = <carrier>-%tky
                    %state_area = 'VALIDATE_NAME'
                    ) TO reported-carrier.

```

```

    IF <carrier>-name IS INITIAL.

```

```

      APPEND VALUE #( %tky      = <carrier>-%tky ) TO failed-carrier.
      APPEND VALUE #( %tky      = <carrier>-%tky
                      %msg      = NEW /dmo/cx_carriers_s(
                                  textid      = /dmo/cx_carriers_s=>name_r
                                  severity    = if_abap_behv_message=>seve
                                  airline_id = <carrier>-AirlineID )
                                  %element-name = if_abap_behv=>mk-on
                                  %state_area  = 'VALIDATE_NAME'
                                  %path-carrierslocksingleton = CORRESPONDING #( <carrier> )
                                ) TO reported-carrier.
    ENDIF.
  ENDLLOOP.

```

```

ENDMETHOD.
ENDCLASS.

```

Validation **ValidateCurrencyCode**

In addition to checking if the provided value for the carrier instance is not initial, it is also relevant here to check whether the provided currency code actually exists.

Read all the instances with the imported keys into an internal table. Pass the failed entries to the `failed` table of the validation.

Copy the entries to another internal table and delete duplicates. If the resulting table is not initial, select the entries of `/DMO/I_Currency` with the given currency code. If there is no corresponding result entry, the provided currency code is not valid.

Loop over the internal table `Carriers`. To avoid duplicate state messages for the consumer, append an empty message to the reported table to clear the state area. If the respective field is initial, write the instance identifier to the failed table and append a corresponding message to the respective state area. If the select from `/DMO/I_Currency` does not return values for the provided currency code, write the instance identifier to the failed table and append a corresponding message to the respective state area.

i *Expand the following code sample to view the source code of **validateCurrencyCode**.*

Sample Code

```

CLASS lhc_carrier DEFINITION INHERITING FROM cl_abap_behavior_handler.

  PRIVATE SECTION.

    METHODS validatecurrencycode FOR VALIDATE ON SAVE
      IMPORTING keys FOR carrier~validatecurrencycode.

ENDCLASS.

CLASS lhc_carrier IMPLEMENTATION.
  METHOD validateCurrencyCode.

```

DATA: currency_codes TYPE SORTED TABLE OF I_Currency WITH UNIQUE KEY Currency.

READ ENTITIES OF /DMO/I_CarriersLockSingleton_S IN LOCAL MODE

```
ENTITY Carrier
  FIELDS ( CarrierSingletonID CurrencyCode )
  WITH CORRESPONDING #( keys )
  RESULT DATA(Carrier).
```

currency_codes = CORRESPONDING #(Carrier DISCARDING DUPLICATES MAPPING Currency = CurrencyC

```
IF currency_codes IS NOT INITIAL.
  SELECT FROM I_Currency FIELDS Currency
  FOR ALL ENTRIES IN @currency_codes
  WHERE Currency = @currency_codes-Currency
  INTO TABLE @DATA(currency_codes_db).
ENDIF.
```

" Raise message for not existing Currency Codes
LOOP AT Carrier ASSIGNING FIELD-SYMBOL(<carrier>).

```
  APPEND VALUE #( %tky          = <carrier>-%tky
                  %state_area = 'VALIDATE_CURRENCY_CODE'
                  ) TO reported-carrier.
```

IF <carrier>-CurrencyCode IS INITIAL.

```
  APPEND VALUE #( %tky          = <carrier>-%tky ) TO failed-carrier.
  APPEND VALUE #( %tky          = <carrier>-%tky
                  %msg          = NEW /dmo/cx_carriers_s(
                                textid      = /dmo/cx_carriers_s=>
                                severity    = if_abap_behv_message
                                airline_id  = <carrier>-AirlineID
                                = if_abap_behv=>mk-on
                                = 'VALIDATE_CURRENCY_CODE'
                                = CORRESPONDING #( <carrier> )
                                )
                  %element-currencycode = if_abap_behv=>mk-on
                  %state_area          = 'VALIDATE_CURRENCY_CODE'
                  %path-carrierslocksingleton
                  ) TO reported-carrier.
```

ELSEIF NOT line_exists(currency_codes_db[Currency = <carrier>-CurrencyCode]).

```
  APPEND VALUE #( %tky          = <carrier>-%tky ) TO failed-carrier.
  APPEND VALUE #( %tky          = <carrier>-%tky
                  %msg          = NEW /dmo/cx_carriers_s(
                                textid      = /dmo/cx_carriers_
                                severity    = if_abap_behv_mess
                                currency_code = <carrier>-Currenc
                                = if_abap_behv=>mk-on
                                = 'VALIDATE_CURRENCY_CODE'
                                = CORRESPONDING #( <carrier> )
                                )
                  %element-currencycode = if_abap_behv=>mk-on
                  %state_area          = 'VALIDATE_CURRENCY_CODE'
                  %path-carrierslocksingleton
                  ) TO reported-carrier.
```

```
  ENDIF.
ENDLOOP.
```

ENDMETHOD.

ENDCLASS.

Projecting the BO for UI Exposure

For the multi-inline-edit scenario, create a business object projection for UI exposure.

The business object with the singleton mechanism is aimed to be used in a UI scenario. It defines basic features that are essential for the UI use case of multi-inline-editing. To define the data model and its functionality that is needed for the use case of a carrier UI service, the business object must be projected. In this BO projection, you define value helps, search and UI layout specifics.

Tasks for the Developer

Create CDS projection views for the root and the child entity, see [Projecting the BO Data Model](#).

Create a CDS projection behavior definition to define the behavior that is to be used in the UI service, see [Projecting the BO Behavior](#).

Projecting the BO Data Model

For the multi-inline-edit scenario, create and define the data model for the BO projection.

Context

The multi-inline-edit scenario provides one projection use case that reuses all elements that are defined in the business object view entities to be exposed for a UI service. In the projection layer the data model is enriched with UI functionality that is implemented via CDS annotations or keywords in the projection view and manifested in the service metadata.

Procedure

Create a data definition for projection views for the two business object entities of the carrier BO. In ADT, use the context menu on the underlying CDS view entity to open the wizard for creating a CDS data definition and choose the template **Define Projection View**.

Projection View /DMO/C_CarrierLockSingleton_S

The projection view is a projection on /DMO/I_CarrierLockSingleton_S. Include the keyword root in the projection view.

Use all elements of the underlying BO travel entity.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

To indicate that `CarrierSingleton` is the semantic identifier of the BO entity, use the annotation `@ObjectModel.semanticKey: ['CarrierSingleton']` in the header of the projection view.

The `CarrierSingleton` is just used for the on-condition of the composition. It is not necessary on the UI representation. Use the annotation `@Consumption.hidden:true`.

As soon as all projection views of the composition structure of the business object are available, redirect the composition to child to the projection view of the carrier entity.

i *Expand the following code sample to view the source code of the projection view /DMO/C_CarrierLockSingleton_S.*

Sample Code

```
@EndUserText.label: 'Carrier Singleton Projection View'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@Semantics.valueRange.maximum: '1'
@Metadata.allowExtensions: true
@ObjectModel.semanticKey: ['CarrierSingletonID']

define root view entity /DMO/C_CarriersLockSingleton_S
  as projection on /DMO/I_CarriersLockSingleton_S
  {
    key CarrierSingletonID,

    @Consumption.hidden: true
    LastChangedAtMax,
    /* Associations */
```

```

    _Airline : redirected to composition child /DM0/C_Carrier_S
}

```

Projection View /DM0/C_Carrier_S

The projection view is a projection on /DM0/I_Carrier_S.

Use all elements of the underlying BO travel entity. You do not need to use the administrative data in the projection view, except for the element representing the ETag master LocalLastChangedAt.

Enable the usage of a metadata extension with the annotation `@Metadata.allowExtension:true`.

Enable search capabilities for the travel entity, with the `@Search` annotations on the entity header and on designated elements. For more information, see [Enabling Text and Fuzzy Searches in SAP Fiori Apps](#)

To indicate that `CarrierID` is the semantic identifier of the BO entity, use the annotation `@ObjectModel.semanticKey: ['AirlineID']` in the header of the projection view.

The `CarrierSingletonID` is just used for the on-condition of the composition. It is not necessary on the UI representation. Use the annotation `@Consumption.hidden:true`.

Define a value help for the element `CurrencyCode`. For more information, see [Adding Value Help](#).

As soon as all projection views of the composition structure of the business object are available, redirect the association to the root entity to the respective projection view.

i *Expand the following code sample to view the source code of the projection view /DM0/C_Carrier_S.*

Sample Code

```

@EndUserText.label: 'Projection View Carrier'
@AccessControl.authorizationCheck: #NOT_REQUIRED

@Metadata.allowExtensions: true
@ObjectModel.semanticKey: ['AirlineID']

define view entity /DM0/C_Carrier_S
as projection on /DM0/I_Carrier_S
{
    key AirlineID,

    @Consumption.hidden: true
    CarrierSingletonID,

    Name,

    @Consumption.valueHelpDefinition: [{entity: {name: 'I_CurrencyStdVH', element: 'Currency'}
    CurrencyCode,

    LocalLastChangedAt,

    /* Associations */
    _CarrierSingleton : redirected to parent /DM0/C_CarriersLockSingleton_S
}

```

Adding UI Metadata for UI Consumption

For the multi-inline-edit scenario, define UI annotations in metadata extensions for the business object entities.

Context

The layout of a Fiori Elements UI can be mostly defined in the RAP BO with @UI annotations. These UI annotations are then exposed with the metadata of the UI service and are applied for defining a UI. It is best practice to maintain UI-relevant annotations in metadata extensions.

Procedure

Create a metadata extension for each projection view in the carrier BO. In ADT, right-click on the data definition to open the creation wizard for metadata extensions.

Metadata Extension /DMO/C_CARRIERSLOCKSSINGLETON_S

Define the metadata extension as extension of the CDS projection view /DMO/C_CARRIERSLOCKSSINGLETON_S.

Define the layer #CORE.

Since the root view of the carrier business object has just one instance, the UI layout for the list report page of a Fiori Elements has only limited layout requirements.

To specify the header texts for Fiori UIs, the annotation @UI.headerInfo is used at entity level. It specifies the list title as well as the title for the object page.

The main building blocks of the UI are specified as UI facets. Define a facet for the root view using the @UI.facet annotation and a navigation to the object page by defining the target elements.

For more information about UI layout configuration, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i *Expand the following code sample to view the source code of the metadata extension /DMO/C_CarriersLockSingleton_S.*

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName: 'Manage Carriers',
                    typeNamePlural: 'Carrier Singleton',
                    title: { type: #STANDARD, value: 'CarrierSingletonID' } } }

annotate entity /DMO/C_CarriersLockSingleton_S
with
{
  @UI.facet: [
    {
      purpose: #STANDARD,
      type: #LINEITEM_REFERENCE,
      label: 'Carrier Multi Inline Edit',
      position: 10,
      targetElement: '_Airline'
    }
  ]

  @UI.lineItem: [{ position: 10 }]
  CarrierSingletonID;
}
```

Metadata Extension /DMO/C_CARRIER_S

Define the metadata extension as extension of the CDS projection view /DMO/C_CARRIER_S.

Define the layer #CORE.

Like in the metadata extension for the root view, specify the header information, a facet as `identification_reference`, which indicates the list on the object page, and the position for the elements as line item and identification.

For more information about UI layout configuration, see [Defining CDS Annotations for Metadata-Driven UIs](#).

i *Expand the following code sample to view the source code of the metadata extension /DMO/C_Carrier_S.*

Sample Code

```
@Metadata.layer: #CORE

@UI: { headerInfo: { typeName:      'Carrier',
                      typeNamePlural: 'Carriers',
                      title:        { type: #STANDARD,
                                      label: 'Airline',
                                      value: 'AirlineID' } } }

annotate entity /DMO/C_Carrier_S
with
{
  @UI.facet: [{ type: #IDENTIFICATION_REFERENCE }]

  @UI.lineItem: [{ position: 10 }]
  @UI.identification: [{ position: 10 }]
  AirlineID;

  @UI.lineItem: [{ position: 20 }]
  @UI.identification: [{ position: 20 }]
  Name;

  @UI.lineItem: [{ position: 30 }]
  @UI.identification: [{ position: 30 }]
  CurrencyCode;
}
```

Projecting the BO Behavior

For the multi-inline-edit scenario, create a projection behavior definition for defining the projection behavior.

Context

In a behavior definition, you define the behavior characteristics and operations of the underlying behavior definition that are to be used in the service the BO is exposed to. For the carrier business object, all behavior that is defined in the base BO is used in the projection behavior definition.

Procedure

Create a projection behavior definition for the carrier business object. In ADT, use the context menu of the root projection view to open the wizard for creating behavior definitions. The implementation type is prefilled with Projection.

Use the complete behavior that was defined in the underlying behavior definition for the carrier BO.

i Expand the following code sample to view the source code of the projection behavior definition `/DMO/C_CarriersLockSingleton_S`.

Sample Code

```
projection;
strict(2);
use draft;

define behavior for /DMO/C_CarriersLockSingleton_S alias CarriersLockSingleton
{
    use action Activate;
    use action Discard;
    use action Edit;
    use action Prepare;
    use action Resume;

    use association _Airline { create; with draft; }
}

define behavior for /DMO/C_Carrier_S alias Carrier
use etag
{
    use update;
    use delete;

    use association _CarrierSingleton { with draft; }
}
```

Defining and Publishing a UI Service with Multi-Inline-Edit Capabilities

For the multi-inline-edit, define a UI service and bind it to a protocol.

Context

The RAP BO with the singleton mechanism is designed to be exposed as UI service for consumption via Fiori Elements UI with multi-inline-edit capabilities. For this to work, the carrier BO must be exposed as a business service to OData. In the service definition, you define which artifacts are included in the service. The service binding defines the protocol with which the service is addressed. In addition, it provides the means to activate the service locally. The Fiori Elements App Preview can be used as soon as the service is activated.

Procedure

Create a service definition for the business service.

Include the projection views of the business object and all related CDS entities that are relevant for UI consumption into the service definition.

i Expand the following code sample to view the source code of the service definition `/DMO/UI_Carriers_S`.

Sample Code

```
@EndUserText.label: 'Carrier Multi-Inline-Edit Scenario'
@ObjectModel.leadingEntity.name: '/DMO/C_CarriersLockSingleton_S'
define service /DMO/UI_Carriers_S {
    expose /DMO/C_CarriersLockSingleton_S as CarriersLockSingleton;
```


7/6/26, 6:28 PM

```
    expose /DM0/C_Carrier_S as Carrier;  
}
```

Based on the service definition, create a service binding.

Expose the service as a UI service with OData protocol V4 and publish it locally. You can then use the preview to test the service.